Key concepts, loading and inspecting the data

Chapter resources

Slides (PDF) Open in Colab Practice quiz

In this lecture, we will begin to see the main definition and foundational concepts in data analysis.

What is Data? An informal Definition

We will start by giving an informal definition of data. It is not by any means a complete definition, but it will suffice as a starting point to get an intuitive understanding of what data is:

> Data is a **set of values collected** with respect to some **variables** which describe a given **phenomenon**.

In the definition above, we mentioned some key concepts, which are discussed in the following sections.

Observations, Populations, Samples

We will see three fundamental concepts: observations, populations, samples.

Observations

When we deal with data, we actually deal with *multiple instances* of values associated to the same event or phenomenon. Examples:

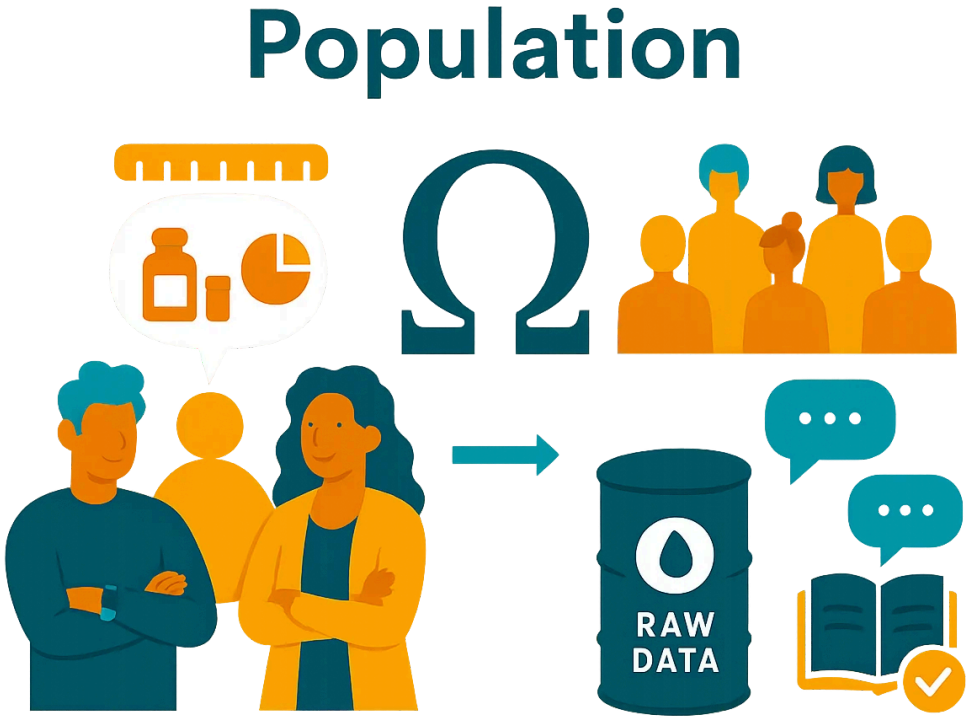
Example	Comment
We want to study how the heights of students change with the years	We need sets of recorded heights, not just one height
We want to study how a given drug affects the recovering from a disease	We need to record sets of values about drug assumption and recover, just one case will not be enough
We need to create a system which can generate text from a prompt	We need to record several examples of prompt-text generation, just one would not be enough to study how such generation should be made

We will call **observations, the units by which we measure data**. These could be persons, cars, animals, plants, etc. We often indicate an observation as

$$\omega$$

We should consider observation as **"abstract" entities**, not necessarily numerical observations. E.g., "let's consider a person ω".

Population



When we study a given phenomenon, we will be interested in a set of observations, which is called a "population". For instance:

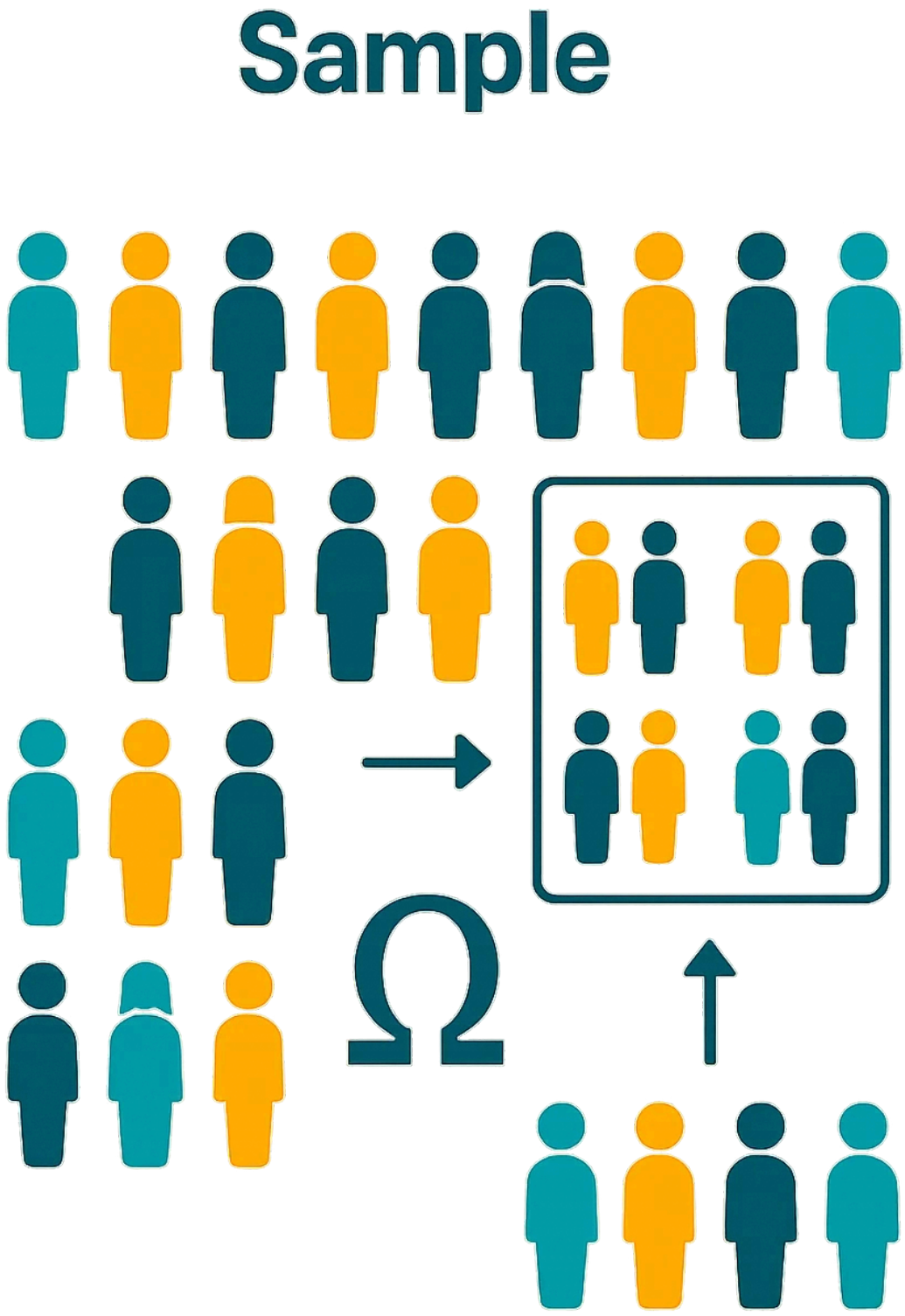
- if we want to study the distribution of heights of people in the world, we will need to look at the population of all people in the world.
- if we want to study the age of people attending a computer science course in Italy, then we need to look at the population of all students of computer science courses in Italy.

Note that **a population can sometimes be a theoretical concept** and identify sets of elements which are not even finite. E.g., *"all movies which will ever be filmed"*.

We can denote a population with the symbol Ω. Al our observations will be

$$\omega \in \Omega$$

Sample



In practice, working with population can be very hard, as it is not always possible to obtain

observations from those large sets. Intuitively, in practice, working on a large enough set of observations from a population could be good enough. We refer to a subset of a population as "a sample":

$$\{\omega^{(1)}, \omega^{(2)}, \dots, \omega^{(n)}\} \subseteq \Omega$$

Example

We want to study how the height of people in the world changed in the years. Here:

- the **population** Ω is the set of people ever existed on earth,
- a **sample** $\{\omega^{(1)}, \dots, \omega^{(n)}\} \subseteq \Omega$ is a subset of people for which we have some data (e.g., say we measured the heights of many people in most countries since the 16th century),
- an observation is a person $\omega^{(i)}$.

Variables (or Features)

We have identified our problem, defined a suitable population and identified a sample of observations. While observations are abstract concepts such as "a person" or "a student", we **usually want to capture specific features of such observations**, such as "the person's age" or "the student's height". We collect these features by means of **statistical variables**.

Statistical variables are concept similar to that of mathematical of programming variables, in that they can be seen as sorts of **"containers for the data"**.

We may also be interested in different features of an observation. For instance, for each person in a population, **we may want to record their age, gender, and height**. We can introduce a variable to capture each of these features. For instance, given observation ω, we may obtain "height = 180cm", "weight=80Kg", "gender=male".

Formally, we'll define a variable X as:

$$X : \Omega \rightarrow S$$

$$\omega \mapsto x$$

Where S is the set of possible values for variable X. The definition above specifies that a variable maps an abstract observation ω to some (possibly more concrete) value $x \in S$.

> Note: we use a capital letter X to denote the variable, while a lowercase letter x to represent the value assumed by the variable. For instance, we could write $X = x \in S$.

The concept of variable is often used interchangeably with the term **feature**, which is more used in the Machine Learning community and jargon. This is because a variable capture a specific "feature" or "characteristic" of each observation. For instance, the variable "height" capture the height feature of a person.

Example

Given the population of all people currently living in the world Ω, we define a variable H to collect the heights of the observed people ω:

$$H : \Omega \rightarrow \mathbb{R}$$

$$\omega \mapsto h$$

Given an observation $\omega^{(1)}$, we may obtain $H(\omega) = 180$. We often say that H assumes the value 180 and write: $H = 180$.

Quantitative and Qualitative Variables

Variables can be of two main kinds:

- **Qualitative**: represent properties that can be observed and cannot generally be measured with a numerical value (e.g., 'color', 'sex');
- **Quantitative**: represent properties that can be quantified with a number (e.g., 'height', 'weight', 'age').

Discrete and Continuous Variables

Variables can also be discrete or continues:

- **Discrete variables** can assume a finite number (or a countable infinite number) of possible values. Binary variables are a special case of discrete variables which can take only two possible values (0 and 1).
- **Continuous variable** assume a continuous, infinite number of values, which can be generally denoted with real numbers

Variables which take numerical values are generally called also "numeric" variables.

Scalar and Multi-Dimensional Variables

Variables can be:

- **scalar**, **uni-dimensional** or **uni-variate**: they assume real numbers (e.g., $X = 1$)
- **multi-dimensional** or **multi-variate**: they assume vector or matrix values, (e.g., $X = \binom{1}{3}$, or $X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$).

Examples

Let's see some examples:

Discrete Scalar Variables

We want to assess if a coin is fair or not.

- We consider as population all possible tosses of that coin.
- An observation will be a specific tossing.
- A discrete scalar variable X may record the outcome of a given tossing. The set of possible values will be $S = \{head, \ tail\}$ (discrete values). The variable is scalar as it will contain a single value.
- If we toss a coin, we may get $X = tail$.

Continuous Scalar Variables

We want to study the heights in centimeters of students in this class.

- Our population is the set of all students in this class.
- We can use a continuous scalar variable X to record the heights of the students. In this case, we can choose $S = \mathbb{R}_+$.
- If we pick a student, we may get $X = 175$.

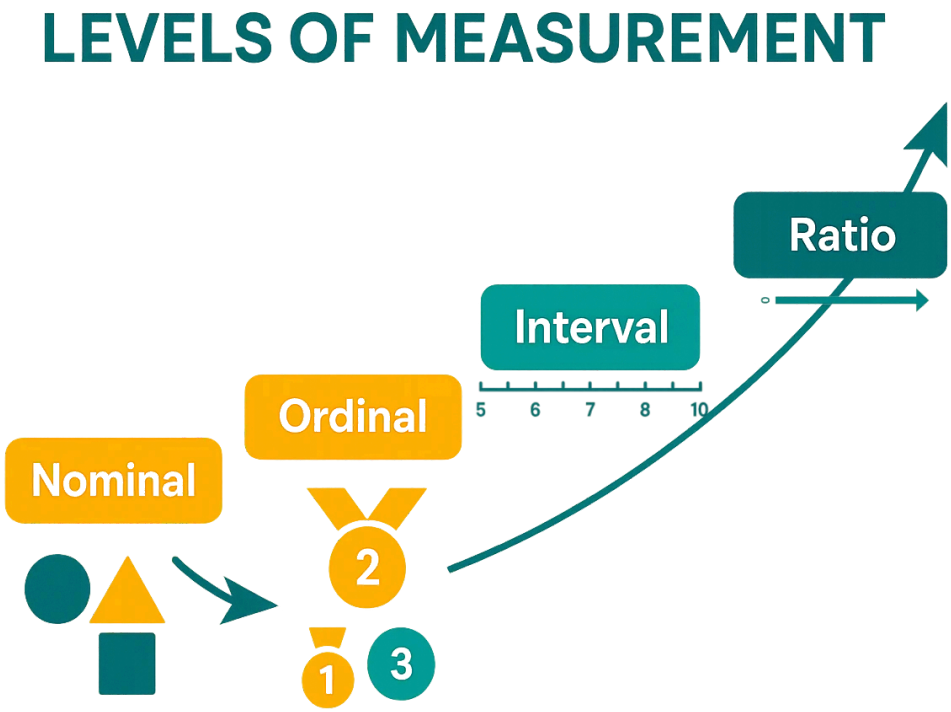
Continuous Multi-Dimensional Variables

We want to study the positions of all cars in the world.

- Our population is the set of all cars in the world.

- We could use the variable X to denote the latitude and longitude coordinates of a car in the world. The set of possible values may be $S = \mathbb{R}^2$.
- Once we pick a car, we may have $X = \binom{37}{15}$.

Scales



When working with variables, it's important to understand whether their values can be meaningfully ordered. This is captured by the concept of **measurement scales**:

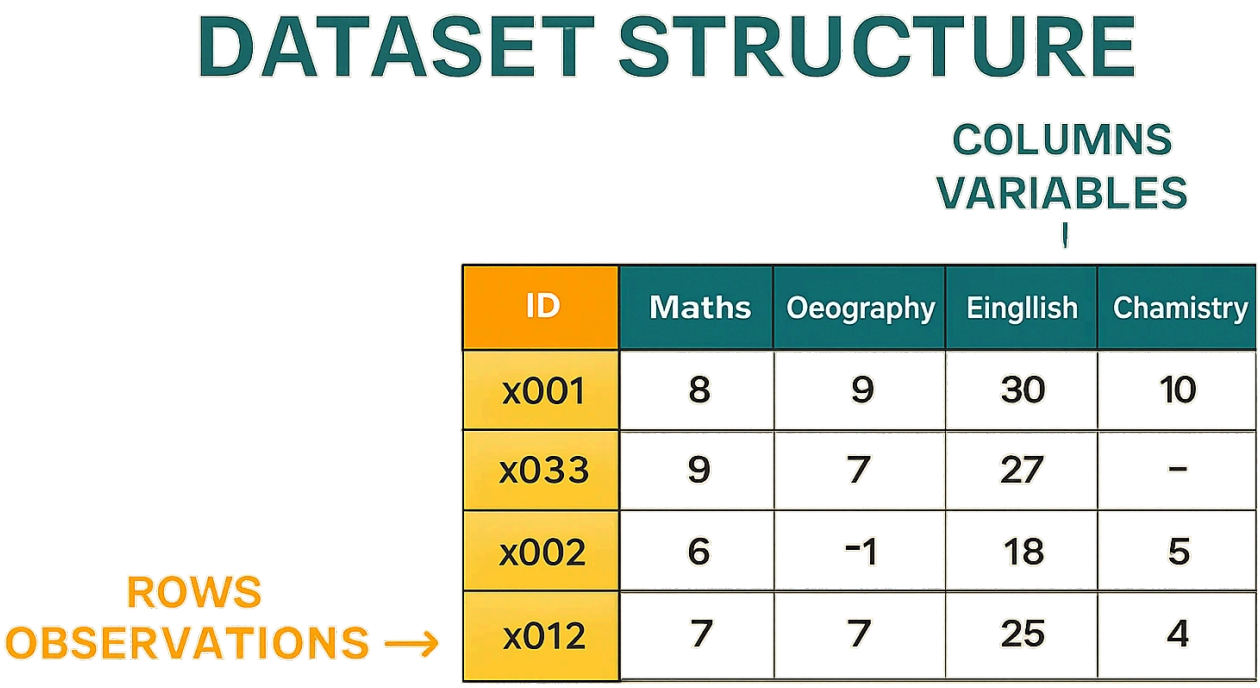
- **Nominal scale**: values **cannot be ordered**.
  Examples: gender, eye color, car manufacturer. We cannot say that "green > brown".

- **Ordinal scale**: values **can be ordered**, but **differences are not meaningful**.
  Example: levels of expertise (novice, amateur, intermediate, expert). We can say "novice < amateur", but "amateur - novice" is usually different than "intermediate - amateur".

- **Interval scale**: values **can be ordered**, and **differences are meaningful**,
  but there is **no true zero denoting the absence of the quantity**.
  Example: temperature in Celsius or Fahrenheit. We can say 25° - 15° = 32°-22°, but 0° does not denote that "there is no temperature".

- **Ratio scale**: values **can be ordered**, **differences are meaningful**, and there **is a true zero**.
  Examples: height, weight, age, income. 0cm or 0Kg denotes the absence of the quantity, plus differences are meaningful and ordering is possible.

Data Sets, Design Matrix, and Missing Values



A set of data related to a specific phenomenon is called a "dataset". Datasets are usually stored in tables in which **columns represent the different variables** and **rows represent the different observations**. If you have ever had a look at a spreadsheet, you probably already saw an example of a dataset!

Let's consider the following example of a dataset of marks of 5 subjects obtained by three students:

ID	Maths	Geography	English	Physics	Chemistry
x001	8	9	30	8	10
x038	9	7	27	6	
x002	6	-1	18	5	6
x012	7	7	25	4	10
x042	10	10	30	10	10

We can find the different elements that we discussed about in this table:

- **Observations**: each row of the table is an observation;
- **Variables**: each column of a table is a different table, mapping an observation to a numerical value. In the example, above the variable ID is qualitative, while all others are quantitative, discrete, and ratio scale (they have a true zero);
- **Features**: same as variables, each column is a "feature" or a characteristic of each observation;

- **Samples and Populations**: the dataset can be seen as a sample (a subset of students) of a wider population (e.g., all students in the school);

This kind of matrix is also called **design matrix**.

Note that, due to the way the data was collected, one value is not there. We call these values **missing data**. These may be due to errors in the way we collected and stored our data or due to specific phenomena. For instance, in this case, the student may have not taken yet the test for that specific subject. We will see later how to deal with **missing data**.

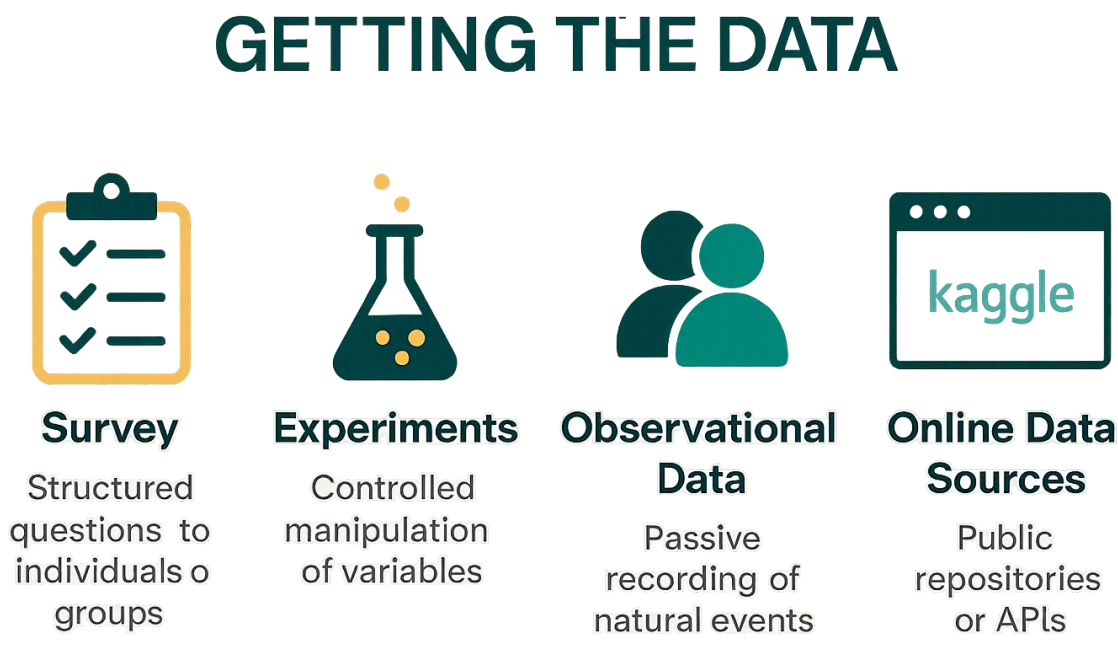
Before moving on, think for a moment what you could do with a dataset like this (maybe imagine a larger one):

- You could take the average of all marks obtained by a student (average by rows) to get a ranking of the students. This could be useful to understand which students may need help.
- You could compute the average of the votes obtained in each course (average by column) to identify the subjects which are “more difficult” for the students than others.
- You could group the courses into humanity-based and science-based to identify which students excel in each field.

The examples above are all (very simple) examples of data analysis. As you can see, even with a simple dataset like this and no knowledge of complex notions of data analysis, we can already do a lot of analysis.

The “table” structure containing a dataset is often called a **data matrix** or a **design matrix**. In this format, each column of the matrix represents a variable, which is often also referred to as a **feature**, while each row is a different observation.

Getting the Data



Data collection is the essential first step in any data analysis workflow. It involves gathering observations that will serve as the foundation for modeling, visualization, and interpretation. The method you choose depends on your goals, the type of data needed, and the resources available.

Surveys

Surveys are structured tools used to collect information from individuals or groups. They typically involve asking predefined questions and are ideal for capturing opinions, preferences, demographics, or behavioral patterns.

Common formats:

- Online forms
- Paper questionnaires
- Phone or in-person interviews

Example:
A university wants to understand students' satisfaction with online courses. It distributes a digital survey asking students to rate their experience across different dimensions (e.g., content clarity, platform usability, instructor engagement).

Key features:

- Standardized questions
- Scalable across large populations

- Quantifiable and analyzable responses

Experiments

Experiments allow researchers to collect data in a **controlled** environment, often to test causal relationships between variables. By manipulating one or more independent variables and observing the outcome, experiments can provide strong evidence for cause-and-effect.

Most common type:

- **Randomized Controlled Trials (RCTs)**

Example:
To evaluate the effectiveness of a new drug, researchers randomly assign participants to two groups: one receives the drug, the other a placebo. Recovery rates are compared to assess the drug's impact.

Key features:

- Controlled manipulation of variables
- Random assignment to reduce bias
- Replicable design for hypothesis testing

Observational Data

Observational methods involve recording data as it naturally occurs, without intervention. This approach is useful when experiments are impractical, unethical, or too costly.

Example:
A public health researcher studies the impact of air pollution on respiratory health by analyzing hospital admission records and pollution levels across different cities.

Key features:

- No manipulation of variables
- Captures real-world behavior
- Suitable for exploratory and correlational analysis

Online Data Sources

In modern data science, many projects begin by retrieving data from online platforms and public repositories. These sources offer ready-to-use datasets for exploration, benchmarking, and model development.

Popular platforms:

- Kaggle: competitions, datasets, notebooks
- UCI Machine Learning Repository
- Google Dataset Search

- APIs (e.g., Twitter, OpenWeather, Spotify)
- Web scraping tools

Example:
A student downloads a Kaggle dataset of movie reviews to train a sentiment analysis model using natural language processing.

Key features:

- Fast access to diverse datasets
- Often pre-cleaned and well-documented
- Ideal for prototyping and learning

Data Loading in Python

We will now put our hands on a real dataset. We will consider the `Titanic` dataset, which is a popular dataset used in data science education and competitions. It contains information about passengers aboard the Titanic and whether they survived the disaster.

This dataset is particularly useful for practicing data cleaning, exploratory data analysis, and predictive modeling. It includes both **qualitative** and **quantitative** variables, and offers a rich context for discussing missing data, bias, and causal inference.

In the `Titanic` dataset, each row represents a passenger, and each column represents a variable describing that passenger. Common variables include:

- `PassengerId`: Unique identifier for each passenger
- `Survived`: Whether the passenger survived (0 = No, 1 = Yes)
- `Pclass`: Ticket class (1st, 2nd, or 3rd)
- `Name`: Full name of the passenger
- `Sex`: Gender
- `Age`: Age in years
- `SibSp`: Number of siblings/spouses aboard
- `Parch`: Number of parents/children aboard
- `Ticket`: Ticket number
- `Fare`: Price paid for the ticket
- `Cabin`: Cabin number (often missing)
- `Embarked`: Port of embarkation (C = Cherbourg, Q = Queenstown, S = Southampton)

The dataset is a good didactic example. Indeed:

- It mixes **categorical** and **numerical** data
- It contains **missing values**, which are common in real-world datasets
- It allows us to explore **correlations** and build **classification models**
- It's historically grounded, making it easier to interpret and explain

In the next steps, we will:

1. **Load the dataset** and inspect its structure
2. **Clean the data** by handling missing values and inconsistencies
3. **Explore the variables** using summary statistics and visualizations
4. **Build a simple model** to predict survival based on selected features

We will start by loading the dataset using the Pandas library in Python. We will use a csv version of the dataset, originally published at `https://raw.githubusercontent.com/agconti/kaggle-titanic/master/data/train.csv` and served by this site at `https://antoninofurnari.github.io/fad-2627/data/titanic.csv` — which is the copy the code below actually reads, so the same code runs here, on a local clone and on Colab.

Before loading the data with Python, let's briefly point our browser to https://antoninofurnari.github.io/fad-2627/data/titanic.csv to see the content.

The data is in CSV format, meaning **Comma-Separated Values**. This is a plain text format where each line represents a row in the dataset, and each value (or cell) is separated by a comma.

CSV files are widely used because they are:

- Simple to read and write
- Compatible with spreadsheet software like Excel or Google Sheets
- Easily imported into programming environments like Python (e.g., using `pandas.read_csv()`)

Here's a small example of how a CSV file might look (first line of the dataset):

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
```

Note that Pandas can also read other formats, such as tab separated values file and Excel files.

Let's load and visualize the data:

```
import pandas as pd
titanic = pd.read_csv(DATA + "titanic.csv",
                      index_col='PassengerId')
titanic
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...
887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 11 columns

As you can see, the visualization has been truncated, removing some rows, as the dataset is big. Another way to visualize such big dataset is to use the `.head()` method, optionally specifying a number of rows to be shown (5 by default):

```
titanic.head()
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

The dataset shows different observations related to the passengers of the Titanic. We can see each row of this matrix as a multi-dimensional variable (a vector) summarizing the main properties of the observations. Indeed, in the example above, each passenger is a different observation, while the different column are the different **features**.

The `NaN` values denote missing data.

With this data in front of us, let's try to answer these questions:

- How many observations are there?
- How many variables are there?
- Which variables are qualitative?
- Which variables are quantitative?
- Which variables are discrete?
- Which variables are continuous?
- What are the scales of each variable?

To make our life easier answering those questions, we can use the `.info()` method of the Pandas dataframe:

```
titanic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 891 entries, 1 to 891
Data columns (total 11 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Survived  891 non-null    int64
 1   Pclass    891 non-null    int64
 2   Name      891 non-null    object
 3   Sex       891 non-null    object
 4   Age       714 non-null    float64
 5   SibSp     891 non-null    int64
 6   Parch     891 non-null    int64
 7   Ticket    891 non-null    object
 8   Fare      891 non-null    float64
 9   Cabin     204 non-null    object
 10  Embarked  889 non-null    object
dtypes: float64(2), int64(4), object(5)
memory usage: 83.5+ KB
```

This table shows some features of each column, including the name, the number of non-null (non missing) data and the type of the column.

In this dataset, each observation is identified by the `PassengerId` variable, which is a qualitative discrete variable. This act as an "index" of the dataset, which is also called a "DataFrame" in Pandas. We can verify this as follows:

```
titanic.index
```

```
Index([  1,   2,   3,   4,   5,   6,   7,   8,   9,  10,
       ...
       882, 883, 884, 885, 886, 887, 888, 889, 890, 891],
      dtype='int64', name='PassengerId', length=891)
```

We can select a single observation using this notation:

```
titanic.loc[101]
```

```
Survived                          0
Pclass                            3
Name        Petranec, Miss. Matilda
Sex                          female
Age                            28.0
SibSp                             0
Parch                             0
Ticket                       349245
Fare                         7.8958
Cabin                           NaN
Embarked                          S
Name: 101, dtype: object
```

A row of a DataFrame is a Pandas Series, which is a one-dimensional array-like object. Each element of the Series corresponds to a column in the DataFrame, and the index of the Series corresponds to the column names.

As we can see, the observation contains different "features" or variable values, such as the Name, Sex, Age etc. of the passenger. The property "Name" of the Series denotes the value of the index. We can access a given feature as follows:

```
titanic.loc[101]['Name']
```

```
'Petranec, Miss. Matilda'
```

Note that we can discard the index and just take the i-th element in order with the following notation:

```
titanic.iloc[101]
```

```
Survived                                    0
Pclass                                      3
Name        Petroff, Mr. Pastcho ("Pentcho")
Sex                                     male
Age                                      NaN
SibSp                                      0
Parch                                      0
Ticket                                349215
Fare                                  7.8958
Cabin                                    NaN
Embarked                                   S
Name: 102, dtype: object
```

As we can see, this returned a different observation (the one with id/Name equal to 102).

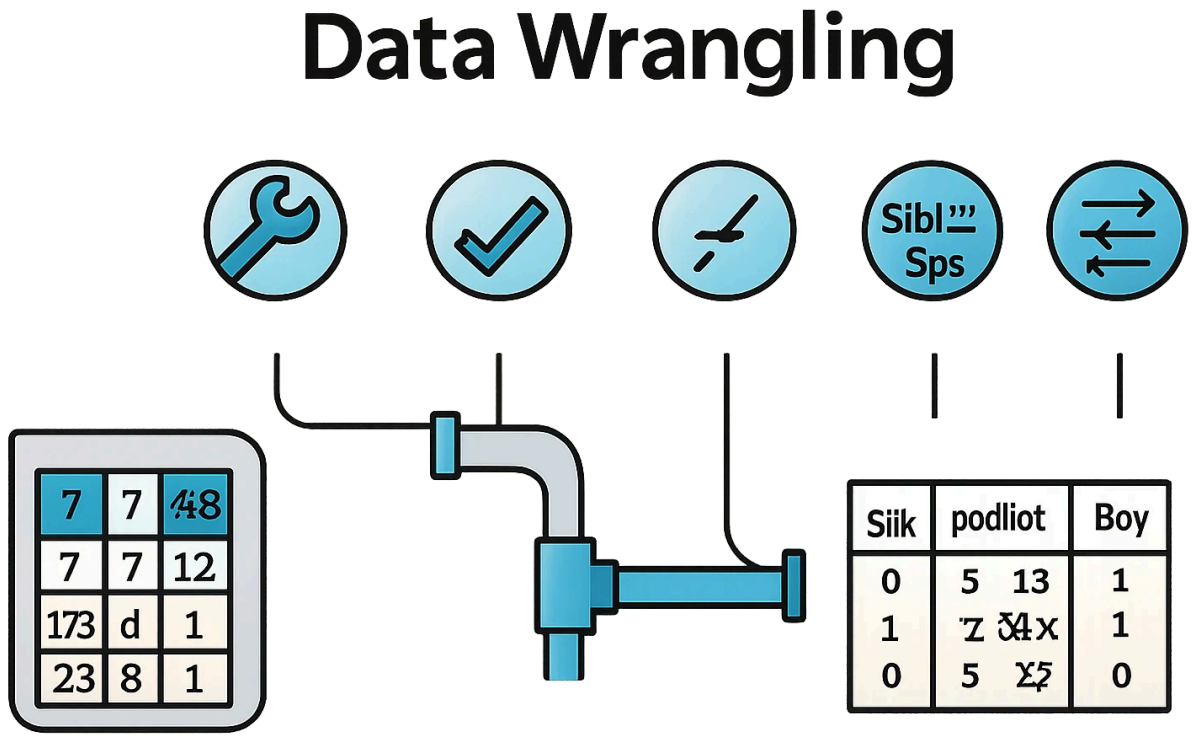
We can extract a given column from the dataset using this notation:

```
titanic['Age']
```

```
PassengerId
1      22.0
2      38.0
3      26.0
4      35.0
5      35.0
       ...
887    27.0
888    19.0
889     NaN
890    26.0
891    32.0
Name: Age, Length: 891, dtype: float64
```

This is still a Pandas series, but this time it represents an **uni-variate** sample.

Data Wrangling (or Data Munging)



Data never comes perfectly clean or ready to use. Real-world datasets often contain missing values, inconsistent formats, outliers, or irrelevant information. **Data wrangling** is the process of transforming raw data into a structured and usable format—preparing it for analysis or modeling. This may involve tasks such as renaming columns, converting data types, handling null values, filtering rows, or creating new features. Good wrangling is essential: it ensures that the insights we extract are based on reliable, well-organized information.

Data wrangling is an unstructured and varied process. We will get back on it multiple times, but here are the main concepts which is good to know:

Handling Missing Values

Real-world datasets often contain missing entries. These can appear as empty cells, `NaN`, or placeholders like `?`.

Titanic example:
The `Age` and `Cabin` columns contain missing values.
We can choose to:

- Remove rows with missing data (`dropna`)
- Fill them with a default or statistical value (`fillna`)
- Flag them as missing and treat them separately

For instance, we can obtain a dataset free of `NaN` values as follows:

```
titanic_clean = titanic.dropna()
titanic_clean.info()
titanic_clean.head()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 183 entries, 2 to 890
Data columns (total 11 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Survived  183 non-null    int64
 1   Pclass    183 non-null    int64
 2   Name      183 non-null    object
 3   Sex       183 non-null    object
 4   Age       183 non-null    float64
 5   SibSp     183 non-null    int64
 6   Parch     183 non-null    int64
 7   Ticket    183 non-null    object
 8   Fare      183 non-null    float64
 9   Cabin     183 non-null    object
 10  Embarked  183 non-null    object
dtypes: float64(2), int64(4), object(5)
memory usage: 17.2+ KB
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.0	1	1	PP 9549	16.7000	G6	S
12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.5500	C103	S

The dataset is now super-clean, but also very smaller (183 rows vs 891!). This is due to the fact that we removed all rows containing at least one missing value. This is often not a good idea, as we may loose a lot of data.

If we are working with a specific variable, we may want to **first select it, then remove missing values**. For example:

```
age_clean = titanic['Age'].dropna()
age_clean.info()
age_clean.head()
```

```
<class 'pandas.core.series.Series'>
Index: 714 entries, 1 to 891
Series name: Age
Non-Null Count  Dtype
--------------  -----
714 non-null    float64
dtypes: float64(1)
memory usage: 11.2 KB
```

```
PassengerId
1    22.0
2    38.0
3    26.0
4    35.0
5    35.0
Name: Age, dtype: float64
```

We have a much larger sample with 714 elements.

A third way to handle missing values is to **fill them in**. For example, we can replace missing ages with the average age:

```
age_clean2 = titanic['Age'].fillna(titanic['Age'].mean())
age_clean2.info()
age_clean2.head()
```

```
<class 'pandas.core.series.Series'>
Index: 891 entries, 1 to 891
Series name: Age
Non-Null Count  Dtype
--------------  -----
891 non-null    float64
dtypes: float64(1)
memory usage: 46.2 KB
```

```
PassengerId
1    22.0
2    38.0
3    26.0
4    35.0
5    35.0
Name: Age, dtype: float64
```

We now have 891 entries and no missing data. The process of filling in missing data is also called **data imputation**.

We can also mix the different strategies depending on the variables. For instance, replacing ages with the mean age may make sense, but we cannot do the same with `Cabin`. Let's see an example to get rid of missing value with a mix of techniques:

```
titanic2 = titanic.copy() # to avoid modifying the original dataset
titanic2['Age'] = titanic2['Age'].fillna(titanic2['Age'].mean())
titanic2['Cabin'] = titanic2['Cabin'].fillna('Unknown')
titanic2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 891 entries, 1 to 891
Data columns (total 11 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Survived  891 non-null    int64
 1   Pclass    891 non-null    int64
 2   Name      891 non-null    object
 3   Sex       891 non-null    object
 4   Age       891 non-null    float64
 5   SibSp     891 non-null    int64
 6   Parch     891 non-null    int64
 7   Ticket    891 non-null    object
 8   Fare      891 non-null    float64
 9   Cabin     891 non-null    object
 10  Embarked  889 non-null    object
dtypes: float64(2), int64(4), object(5)
memory usage: 115.8+ KB
```

In practice, we filled age with the mean value, but marked all missing Cabin with “Unknown”.

Converting Data Types

Sometimes we need to convert columns to the correct type—especially when reading from CSV files.

Titanic example:
Convert `Pclass` from integer to categorical using `astype('category')`.

Let’s first inspect the type of the `Pclass` variable. This can be done with info, but also with dtype as follows:

```
titanic2['Pclass'].dtype
```

```
dtype('int64')
```

While we have an integer value, this represents the class of passengers, so it probably contains only few unique values. Let’s check this by seeing the unique values of the column (without duplicates):

```
titanic2['Pclass'].unique()
```

```
array([3, 1, 2])
```

It does make sense to convert this to a categorical variable (a nominal variable). Let’s do it as follows:

```
titanic2['Pclass'] = titanic2['Pclass'].astype('category')
titanic2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 891 entries, 1 to 891
Data columns (total 11 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Survived  891 non-null    int64
 1   Pclass    891 non-null    category
 2   Name      891 non-null    object
 3   Sex       891 non-null    object
 4   Age       891 non-null    float64
 5   SibSp     891 non-null    int64
 6   Parch     891 non-null    int64
 7   Ticket    891 non-null    object
 8   Fare      891 non-null    float64
 9   Cabin     891 non-null    object
 10  Embarked  889 non-null    object
dtypes: category(1), float64(2), int64(3), object(5)
memory usage: 109.9+ KB
```

Renaming and Reformatting

Clean, consistent column names make analysis easier.

You might want to rename columns, standardize formats, or strip whitespace.

Titanic example:

Rename `SibSp` to `SiblingsSpouses` for clarity.

```
titanic2.rename(columns={'SibSp': 'SiblingsSpouses'}, inplace=True)
titanic2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 891 entries, 1 to 891
Data columns (total 11 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Survived         891 non-null    int64
 1   Pclass           891 non-null    category
 2   Name             891 non-null    object
 3   Sex              891 non-null    object
 4   Age              891 non-null    float64
 5   SiblingsSpouses  891 non-null    int64
 6   Parch            891 non-null    int64
 7   Ticket           891 non-null    object
 8   Fare             891 non-null    float64
 9   Cabin            891 non-null    object
 10  Embarked         889 non-null    object
dtypes: category(1), float64(2), int64(3), object(5)
memory usage: 109.9+ KB
```

Creating New Features

Sometimes, we need to make some characteristics of the data more explicit. We can derive new columns from existing ones to enrich your analysis.

Titanic example:
Create a `FamilySize` column by summing `SibSp` and `Parch`.

```
titanic2['FamilySize'] = titanic2['SiblingsSpouses'] + titanic2['Parch']
titanic2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 891 entries, 1 to 891
Data columns (total 12 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Survived         891 non-null    int64
 1   Pclass           891 non-null    category
 2   Name             891 non-null    object
 3   Sex              891 non-null    object
 4   Age              891 non-null    float64
 5   SiblingsSpouses  891 non-null    int64
 6   Parch            891 non-null    int64
 7   Ticket           891 non-null    object
 8   Fare             891 non-null    float64
 9   Cabin            891 non-null    object
 10  Embarked         889 non-null    object
 11  FamilySize       891 non-null    int64
dtypes: category(1), float64(2), int64(4), object(5)
memory usage: 116.8+ KB
```

Filtering and Subsetting

You may want to focus on a subset of the data—e.g., only adult passengers, or only those who embarked from Southampton.

In the case of Titanic, we may select only adult passengers as follows:

```
titanic2 = titanic2[titanic2['Age'] ⩾ 18]
titanic2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 778 entries, 1 to 891
Data columns (total 12 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Survived         778 non-null    int64
 1   Pclass           778 non-null    category
 2   Name             778 non-null    object
 3   Sex              778 non-null    object
 4   Age              778 non-null    float64
 5   SiblingsSpouses  778 non-null    int64
 6   Parch            778 non-null    int64
 7   Ticket           778 non-null    object
 8   Fare             778 non-null    float64
 9   Cabin            778 non-null    object
 10  Embarked         776 non-null    object
 11  FamilySize       778 non-null    int64
dtypes: category(1), float64(2), int64(4), object(5)
memory usage: 73.8+ KB
```

We can check that our subsetting worked as follows:

```
titanic2['Age'].min()
```

```
np.float64(18.0)
```

We can also drop a column as follows:

```
titanic2.drop(columns=['Ticket'], inplace=True)
titanic2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 778 entries, 1 to 891
Data columns (total 11 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Survived         778 non-null    int64
 1   Pclass           778 non-null    category
 2   Name             778 non-null    object
 3   Sex              778 non-null    object
 4   Age              778 non-null    float64
 5   SiblingsSpouses  778 non-null    int64
 6   Parch            778 non-null    int64
 7   Fare             778 non-null    float64
 8   Cabin            778 non-null    object
 9   Embarked         776 non-null    object
 10  FamilySize       778 non-null    int64
dtypes: category(1), float64(2), int64(4), object(4)
memory usage: 67.7+ KB
```

Besides this, we can also select a subset of variables as follows:

```
titanic2[['Age', 'Cabin', 'Fare']].head()
```

	Age	Cabin	Fare
PassengerId			
1	22.0	Unknown	7.2500
2	38.0	C85	71.2833
3	26.0	Unknown	7.9250
4	35.0	C123	53.1000
5	35.0	Unknown	8.0500

Long vs Wide Formats

Data can be organized in different formats depending on the goals of the analysis. Understanding these formats helps us reshape and prepare data effectively for modeling, visualization, or statistical testing.

Wide Format

- Each **variable** has its own column
- Each **observation** is a single row
- **Advantages**: Easy to read, intuitive structure, compatible with most analysis tools
- **Use cases**: Statistical modeling, machine learning, spreadsheet-style inspection

Example:
A dataset where each row is a student and each column is a subject score.

```
Wide Format:
   Student  Math  Science  English
0    Alice    85       88       92
1      Bob    92       85       88
2  Charlie    78       92       85
```

Long Format

- Variables are **stacked** into fewer columns
- Each observation may span **multiple rows**
- **Advantages**: More compact, flexible for grouping and plotting
- **Use cases**: Time series, repeated measures, tidy data workflows, faceted plots

Example:
Instead of one row per student, we have one row per student–subject pair, with columns like `StudentID`, `Subject`, and `Score`.

Most of the time, especially in introductory analysis and machine learning, we'll encounter datasets in **wide format**. However, certain tasks—like time-based analysis or grouped visualizations—may require us to **reshape the data into long format**. We'll revisit these concepts and apply them later when working with real datasets.

```
Long Format:
   Student  Subject  Score
0    Alice     Math     85
1      Bob     Math     92
2  Charlie     Math     78
3    Alice  Science     88
4      Bob  Science     85
5  Charlie  Science     92
6    Alice  English     92
7      Bob  English     88
8  Charlie  English     85
```

Conversion across formats can be done as follows:

```python
# Example: Wide vs Long format

# Create sample data in wide format
wide_data = pd.DataFrame({
    'Student': ['Alice', 'Bob', 'Charlie'],
    'Math': [85, 92, 78],
    'Science': [88, 85, 92],
    'English': [92, 88, 85]
})

print("Wide Format:")
print(wide_data)

# Convert to long format
long_data = pd.melt(wide_data,
                    id_vars=['Student'],
                    var_name='Subject',
                    value_name='Score')

print("\nLong Format:")
print(long_data)

# Convert back to wide format
wide_again = long_data.pivot(index='Student', columns='Subject', values='Score')
print("\nBack to Wide Format:")
print(wide_again)
```

```
Wide Format:
   Student  Math  Science  English
0    Alice    85       88       92
1      Bob    92       85       88
2  Charlie    78       92       85

Long Format:
   Student  Subject  Score
0    Alice     Math     85
1      Bob     Math     92
2  Charlie     Math     78
3    Alice  Science     88
4      Bob  Science     85
5  Charlie  Science     92
6    Alice  English     92
7      Bob  English     88
8  Charlie  English     85

Back to Wide Format:
Subject  English  Math  Science
Student
Alice         92    85       88
Bob           88    92       85
Charlie       85    78       92
```

These are some of the techniques used in data wrangling. In practice, depending on the kind of analysis, we may perform different operations, such as:

- **Converting units of measurement** (e.g., from inches to centimeters, or Fahrenheit to Celsius)
- **Merging different tables** (e.g., using a join operation like in SQL to combine passenger data with ticket prices)
- **Parsing and extracting information** from strings (e.g., splitting full names into first and last names)
- **Normalizing or scaling values** (e.g., rescaling age or fare to a 0–1 range for modeling)
- **Binning continuous variables** into categories (e.g., grouping ages into child, adult, senior)
- **Removing duplicates or irrelevant columns**
- **Filtering outliers** or suspicious values (e.g., negative ages or fares)
- **Reshaping data** between wide and long formats depending on the analysis or visualization needs

We'll revisit many of these techniques and apply them to real datasets as we move forward. For now, it's important to understand that data wrangling is not a fixed recipe—it's a flexible toolbox that adapts to the structure and goals of each project.

The Data Analysis Workflow

Now that we've seen the main concepts, terminology, and tools involved in working with data, we can introduce a broader perspective: the **data analysis workflow**. This is the structured process that guides how we move from raw data to insights, decisions, and predictions.

Let's begin with a working definition, adapted from Wikipedia:

> Data analysis is the **process** of inspecting, **cleaning**, **transforming**, and **modeling** data with the goal of **discovering useful information**, **informing conclusions**, and **supporting decision-making**.

While this definition is informal, it provides a useful starting point. In the sections below, we'll unpack its key components and introduce the main types and goals of data analysis.

Data Analysis as a Process

The definition above emphasizes that data analysis is a **process**—not a single algorithm or technique, but a sequence of steps that combine statistical reasoning, domain knowledge, and computational tools. These steps often include:

- **Inspecting**: Assessing the structure and content of the dataset—number of rows and columns, types of variables, typical values, and potential anomalies. For example, in a dataset of student marks, we might expect scores to range from 0 to 10.
- **Cleaning**: Correcting or removing problematic data entries. This could involve handling missing values, removing duplicates, or filtering out invalid entries (e.g., negative marks or scores above the maximum).
- **Transforming**: Modifying the data to make it more suitable for analysis. This might include creating new columns (e.g., a mean score per student), rescaling variables, or converting formats (e.g., dividing English scores out of 30 by 3 to match other subjects scored out of 10).
- **Modeling**: Applying statistical or machine learning models to summarize, explain, or predict aspects of the data. A model can help us understand relationships between variables—for instance, predicting English scores based on performance in other subjects.

The Typical Workflow

A standard data analysis workflow includes the following stages:

1. **Define** your data analysis question
2. **Collect** the data needed to answer that question
3. **Clean and format** the data
4. **Explore and describe** the data

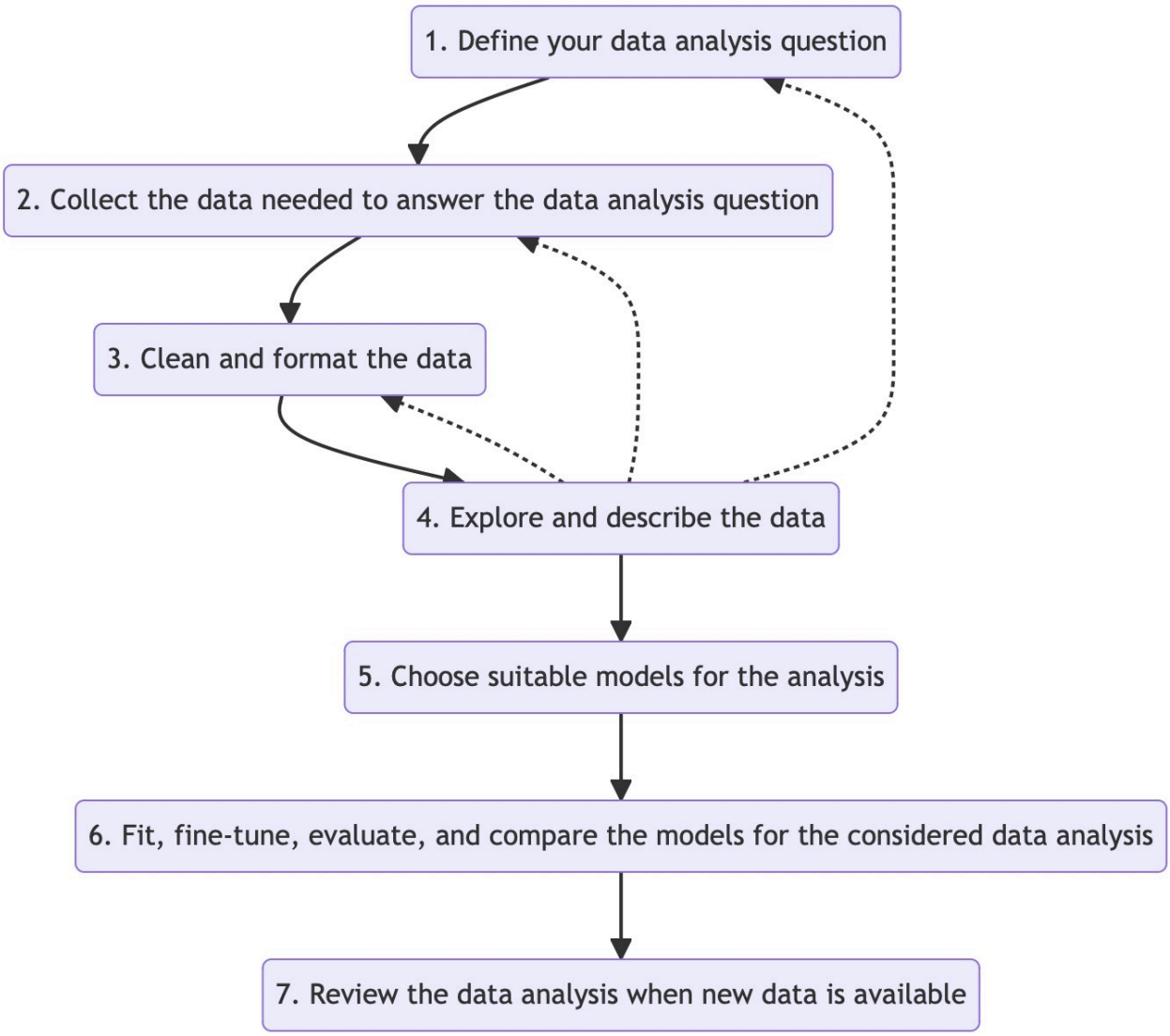
5. **Choose suitable models** for the analysis
6. **Fit, fine-tune, evaluate, and compare** the models
7. **Review and update** the analysis as new data becomes available

A Non-Linear Process

Although the workflow is presented as a sequence, real-world data analysis is rarely linear. Analysts often revisit earlier steps based on what they learn along the way. For example:

- After exploring the data (step 4), you might discover outliers and return to cleaning (step 3)
- You might realize that your initial question (step 1) needs refinement
- You might need additional data (step 2) to improve your model

The diagram below illustrates this flexible structure. Solid arrows show the main flow, while dashed arrows indicate possible backward steps—especially after exploration.



Example: Analyzing Customer Reviews

Let's walk through a practical example. A data analyst at an e-commerce company is tasked with improving product quality and customer satisfaction by analyzing customer reviews.

- **Step 1**: Define the question: “What are the common themes and issues in customer reviews?”
- **Step 2**: Collect reviews from the company website, social media, and third-party platforms
- **Step 3**: Clean the data—remove duplicates, correct spelling, standardize formatting
- **Step 4**: Explore the data—use word clouds, sentiment analysis, and frequency counts
- **Step 5**: Choose a model—apply topic modeling (e.g., Latent Dirichlet Allocation)
- **Step 6**: Evaluate the model—realize that topics are overlapping and unclear
- **Step 4 (again)**: Re-explore the data—identify that product and service issues are mixed
- **Step 5 (again)**: Separate reviews into categories and reapply topic modeling
- **Step 6**: Obtain clearer topics like “defective products” and “responsive support”
- **Step 7**: Commit to ongoing review as new reviews come in

This iterative approach allows the analyst to refine insights and adapt strategies over time.

This workflow will guide our journey through the course. As we move forward, we’ll apply each step to real datasets, starting with the Titanic dataset and expanding to more complex scenarios.

Exercises

1. **Population and sample.** For each, name the population, the sample and the unit of observation: (a) an exit poll on election day; (b) the server logs of a web shop for one month; (c) a clinical trial of a new drug. Which of the three lets you generalise, and to what?
2. **Variable types.** Classify every column of the Titanic dataset as nominal, ordinal, discrete or continuous. Two of them are genuinely debatable — say which, and why the choice matters for the analysis.
3. **Loading and inspecting.** Load the Titanic data and answer, with code: how many rows and columns? Which columns have missing values, and how many? What is the data type of each column, and is it the type you would expect?
4. **Missing is information.** `Cabin` is missing for most passengers. Is it missing at random? Compare the survival rate of passengers with and without a recorded cabin, and say what the comparison suggests about *why* the value is missing.

References

- Chapter 1 of *Heumann, Christian, and Michael Schomaker Shalabh. Introduction to statistics and data analysis. Springer International Publishing Switzerland, 2016.*

© 2026 Antonino Furnari · Department of Mathematics and Computer Science, University of Catania Built with Quarto

Credits and data sources

Describing and visualizing the data

Chapter resources

Slides (PDF) Open in Colab Practice quiz

We have seen how to access the data and perform some basic manipulations. In practice, datasets are often large enough to make it difficult to grasp their structure or patterns by simply printing rows or scanning tables. This is where **descriptive statistics** and **visualization techniques** become essential tools.

Describing and visualizing the data helps us:

- Gather insights on the dataset
- Understand the main characteristics of each variable
- Detect missing or anomalous values
- Identify relationships between variables
- Spot trends or clusters of data which are similar
- Guide the choice of pre-processing and further modeling

To make some example, we will consider a simple dataset where weights (in pounds), heights (in inches), and sex of various subjects. The dataset will look like this:

```
import pandas as pd
hw=pd.read_csv(DATA + "height_weight_pounds.csv")
hw.info()
hw.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4231 entries, 0 to 4230
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   sex     4231 non-null   object
 1   height  4231 non-null   int64
 2   weight  4231 non-null   float64
dtypes: float64(1), int64(1), object(1)
memory usage: 99.3+ KB
```

	sex	height	weight
0	M	74	53.484771
1	M	70	38.056472
2	F	61	34.970812
3	M	68	35.999365
4	F	66	34.559390

Since we are inspecting the dataset, it is worth to answer these questions:

- How many observations do we have?
- How many variables?
- What are the measuring scales of each variable?
- Which variables are discrete? Which continuous? Which numerical?
- Are there missing values?

We will begin by focusing on the univariate sample of heights:

```
heights = hw['height']
heights
```

```
0       74
1       70
2       61
3       68
4       66
        ..
4226    69
4227    69
4228    64
4229    64
4230    61
Name: height, Length: 4231, dtype: int64
```

Absolute and Relative Frequencies

Absolute Frequencies

A first way to describe data is to calculate the number of times each value appears. These are called “absolute frequencies”. Absolute frequencies are generally calculated for discrete variables where observations take a finite number of values.

Let

$$a_1, a_2, \ldots, a_n$$

be the values that the variable under consideration can take.

The absolute frequencies n_i are defined as the number of times that a_i appears in the sample. Note that:

$$\sum_i n_i = N$$

Where N is the total number of elements in the sample.

We can compute unique values a_j as follows:

```
heights.unique()
```

```
array([74, 70, 61, 68, 66, 65, 64, 67, 72, 71, 76, 69, 63, 75, 60, 59, 73])
```

The absolute frequencies can be computed as follows:

```
heights.value_counts()
```

```
height
64    435
61    411
66    391
67    377
68    355
65    351
69    302
63    291
70    272
72    260
71    235
73    146
60    130
74    104
75     72
59     52
76     47
Name: count, dtype: int64
```

Note that this will sort the values by frequency. We can sort them by value as follows:

```
heights.value_counts().sort_index()
```

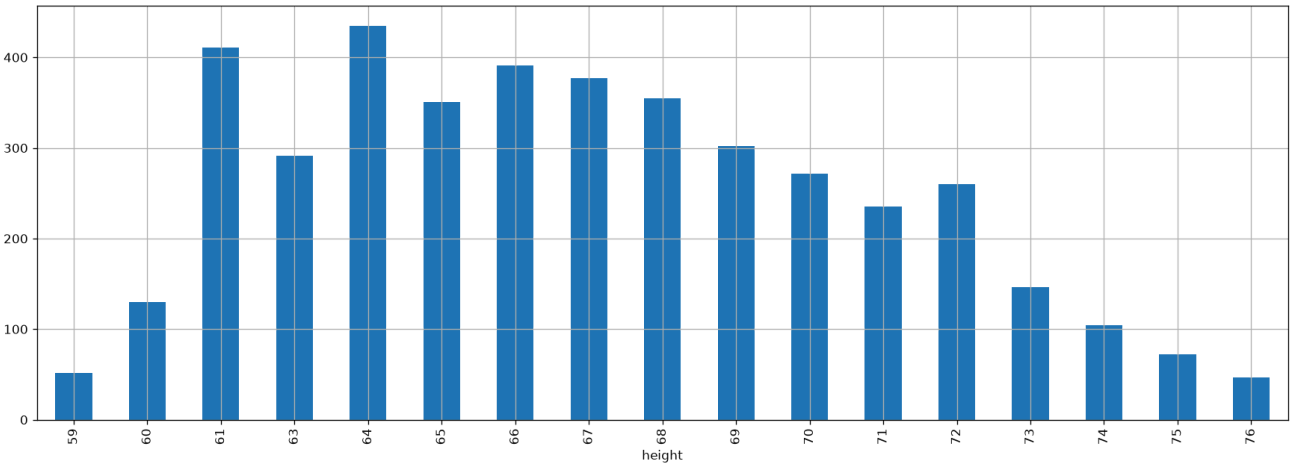
```
height
59     52
60    130
61    411
63    291
64    435
65    351
66    391
67    377
68    355
69    302
70    272
71    235
72    260
73    146
74    104
75     72
76     47
Name: count, dtype: int64
```

Bar Chart of Absolute Frequencies

The data frequencies can be graphically represented using a bar chart that places the unique value (a_i) on the x-axis and represents the absolute frequency n_i as the height of the bar.

In our example:

```
from matplotlib import pyplot as plt
hw['height'].value_counts().sort_index().plot.bar(figsize=(18,6))
plt.grid()
plt.show()
```



The graph above tells us something about the number of occurrences of a given value and also gives us an idea of which values are more or less frequent. We see that the data follows a “bell” shape. We

will often find this type of shape and will discuss it further later.

Relative Frequencies

Absolute frequencies allow us to get a more precise idea of how data are distributed, regardless of the size of our sample. For example, we know that the sample contains more individuals with a height of $167.64\ cm$ than individuals with a height of $193.04\ cm$. However, this representation is linked to the total number of elements contained in the sample. For example, a similarly distributed sample, but with more observations, will result in larger absolute frequencies. We can obtain a representation independent of the sample size by analyzing relative frequencies, defined as follows:

$$f_j = f(a_j) = \frac{n_j}{N}, j = 1, 2, \ldots, k$$

Note that, given the definition, we will have:

$$n_j \leq n \Rightarrow f_j \leq 1 \; \forall j$$

$$\sum_j f_j = \sum_j \frac{n_j}{N} = \frac{1}{N} \sum_j n_j = \frac{N}{N} = 1$$

We can obtain relative frequencies still with value_counts, but this time we will pass the argument `normalize=True`:

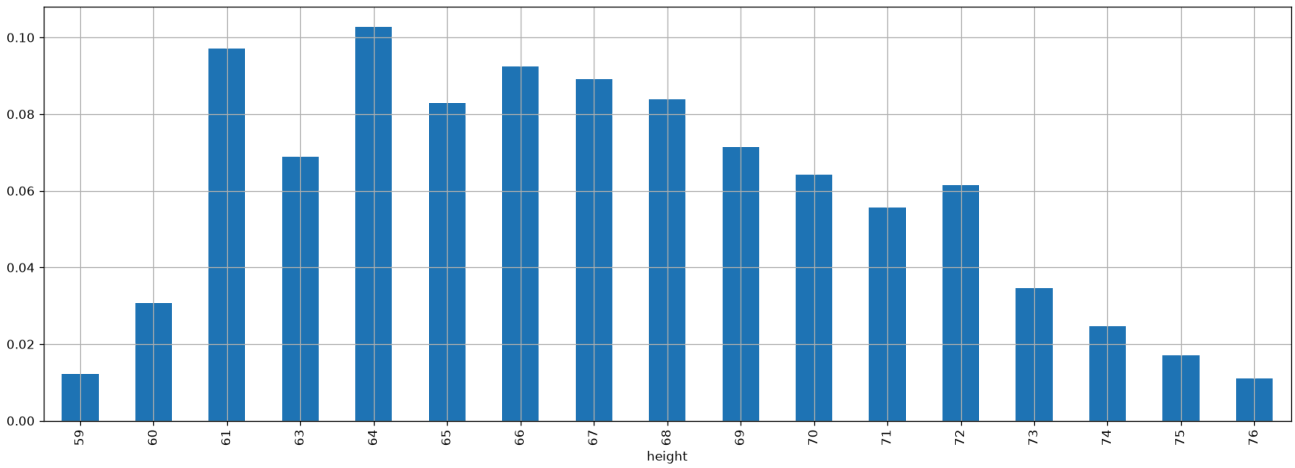
```
heights.value_counts(normalize=True).sort_index()
```

```
height
59    0.012290
60    0.030726
61    0.097140
63    0.068778
64    0.102813
65    0.082959
66    0.092413
67    0.089104
68    0.083905
69    0.071378
70    0.064287
71    0.055542
72    0.061451
73    0.034507
74    0.024580
75    0.017017
76    0.011108
Name: proportion, dtype: float64
```

Bar Chart of Relative Frequencies

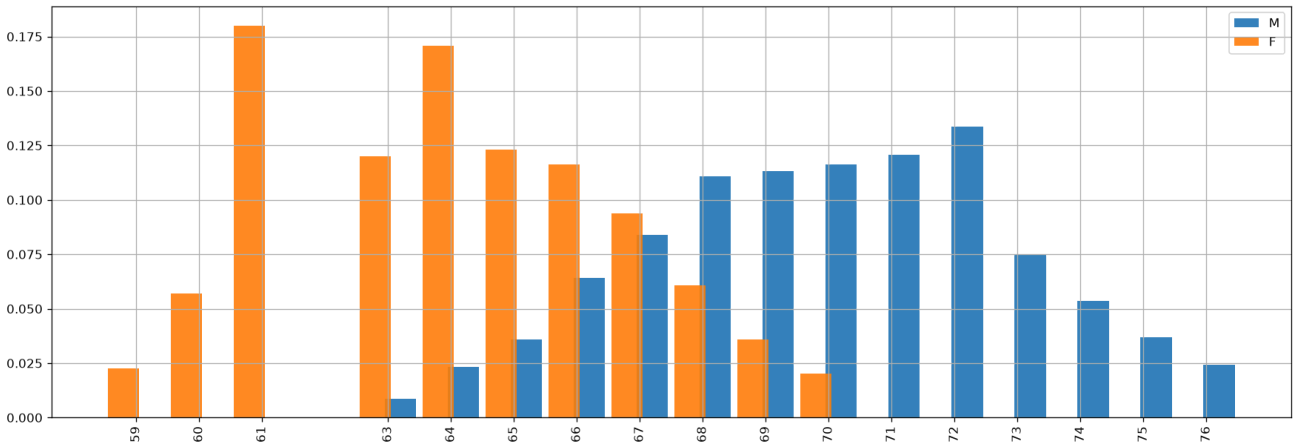
By plotting the relative frequencies using a bar chart, we obtain a graph very similar to the one showing absolute frequencies, but with a different scale on the y-axis.

```
(hw['height'].value_counts(normalize=True).sort_index()).plot.bar(figsize=(18,6))
plt.grid()
plt.show()
```



Relative frequency bar charts are useful for comparing different samples. For example, we can consider the heights of men and women in the weights-and-heights dataset:

```
pmf_height_m = hw[hw['sex']=='M']['height'].value_counts(normalize=True).sort_index()
pmf_height_f = hw[hw['sex']=='F']['height'].value_counts(normalize=True).sort_index()

plt.figure(figsize=(18,6))
# We add and subtract 0.2 from the indices to "shift" the bars and make them
# visible when overlapping. We also set alpha=0.9 to make the bars
# partially transparent.
plt.bar(pmf_height_m.index+0.2, pmf_height_m.values, width=0.5, alpha=0.9)
plt.bar(pmf_height_f.index-0.2, pmf_height_f.values, width=0.5, alpha=0.9)
plt.xticks(hw['height'].unique(), rotation='vertical')
plt.legend(['M','F']) # Display a legend
plt.grid()
plt.show()
```

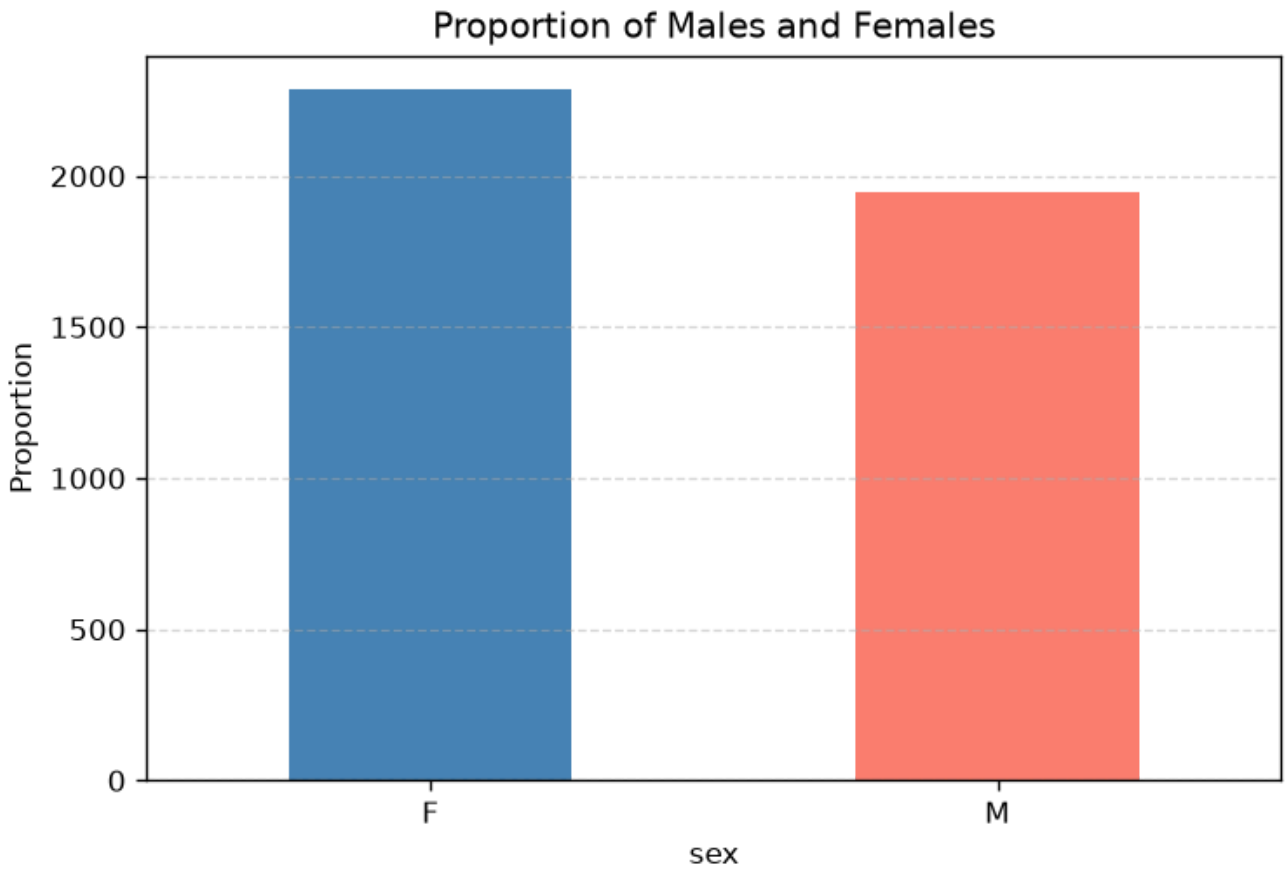


From the comparison above, we can already make some qualitative observations about the two samples. In particular, we notice (unsurprisingly) that men are generally taller than women. This doesn't mean that there aren't men who are shorter than some women, or vice versa, but such cases are reasonably less frequent.

Barplots and Stacked Barplots

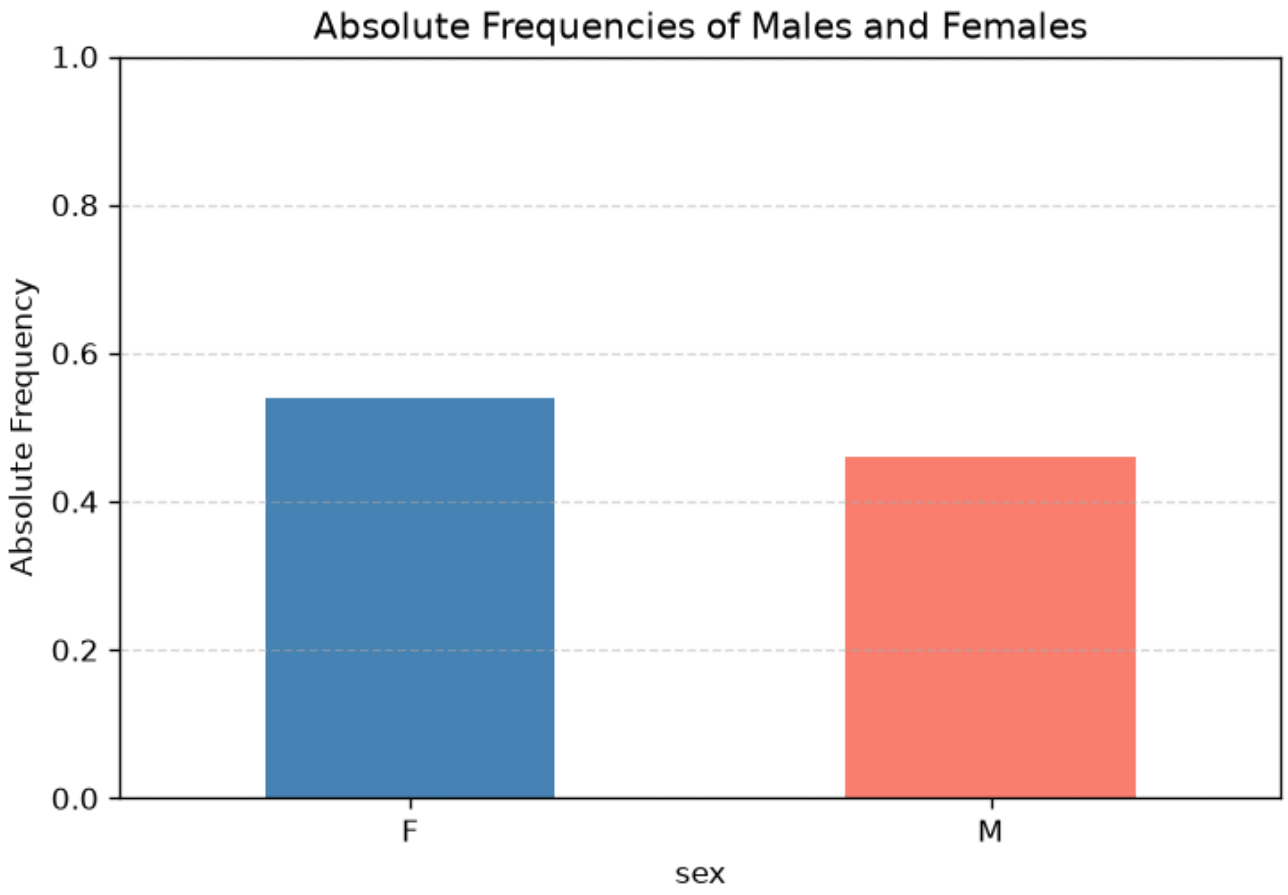
We can also used barplots and stacked barplots to count the number of elements in a given class. For instance, a barplot counting the number of subjects in the "male" or "female" class looks like this:

```
import matplotlib.pyplot as plt

# Calculate proportions
sex_counts = hw['sex'].value_counts()

# Create bar plot
sex_counts.plot.bar(color=['steelblue', 'salmon'])

# Customize plot
plt.title('Proportion of Males and Females')
plt.ylabel('Proportion')
plt.xticks(rotation=0)
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.show()
```



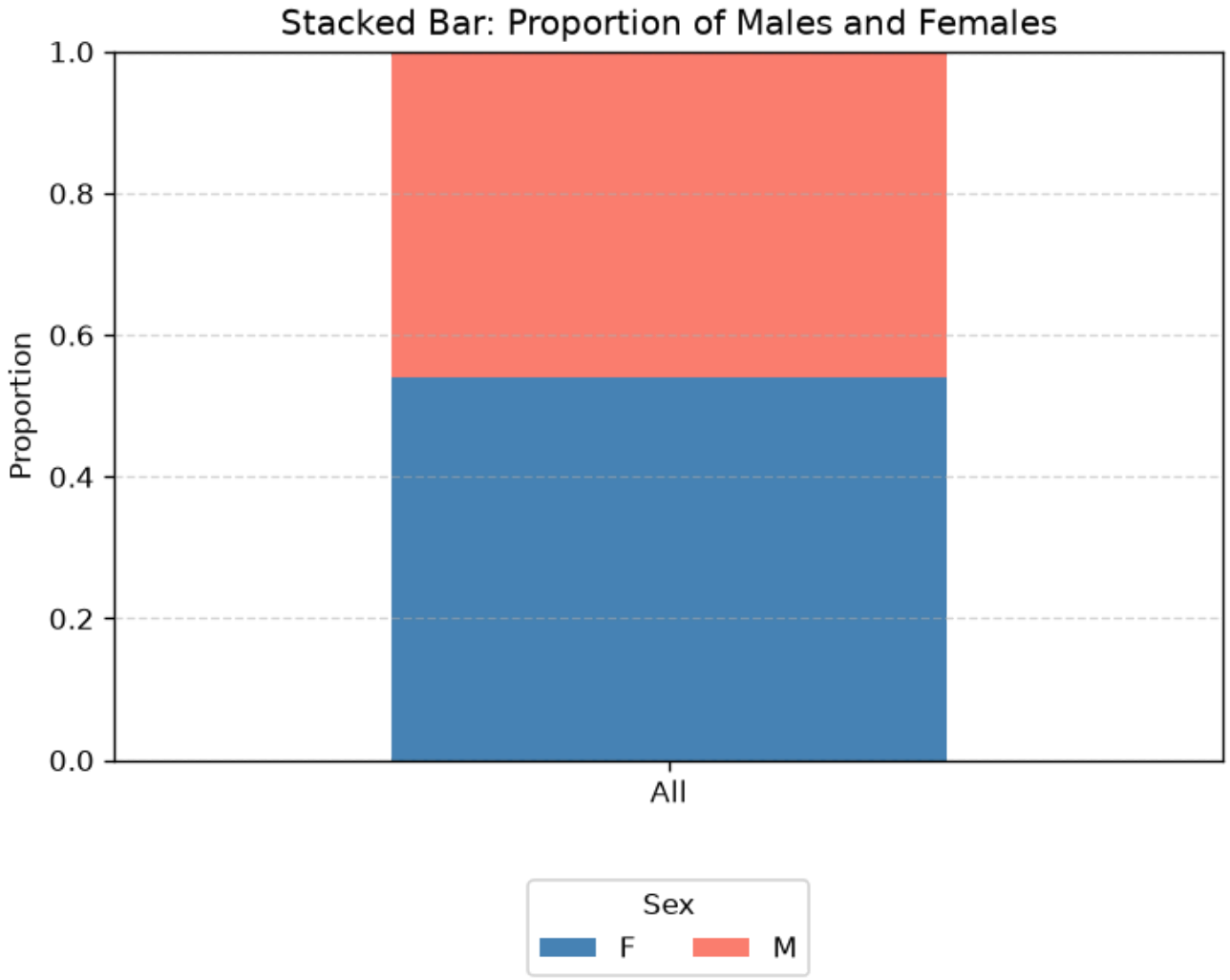
We can obtain the normalized version by passing normalize=True to value_counts:

```
import matplotlib.pyplot as plt

# Calculate proportions
sex_counts = hw['sex'].value_counts(normalize=True)

# Create bar plot
sex_counts.plot.bar(color=['steelblue', 'salmon'])

# Customize plot
plt.title('Absolute Frequencies of Males and Females')
plt.ylabel('Absolute Frequency')
plt.xticks(rotation=0)
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.show()
```



When numbers sum to one, another popular visualization is that of the stacked plot:

```
import matplotlib.pyplot as plt

# Compute proportions grouped by a single dummy category
sex_props = hw.groupby(lambda _: 'All')['sex'].value_counts(normalize=True).unstack()

# Plot as stacked bar
sex_props.plot(kind='bar', stacked=True, color=['steelblue', 'salmon'])

# Customize plot
plt.title('Stacked Bar: Proportion of Males and Females')
plt.ylabel('Proportion')
plt.xticks(rotation=0)
plt.ylim(0, 1)
plt.legend(title='Sex', loc='upper center', bbox_to_anchor=(0.5, -0.15), ncol=2)
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.show()
```

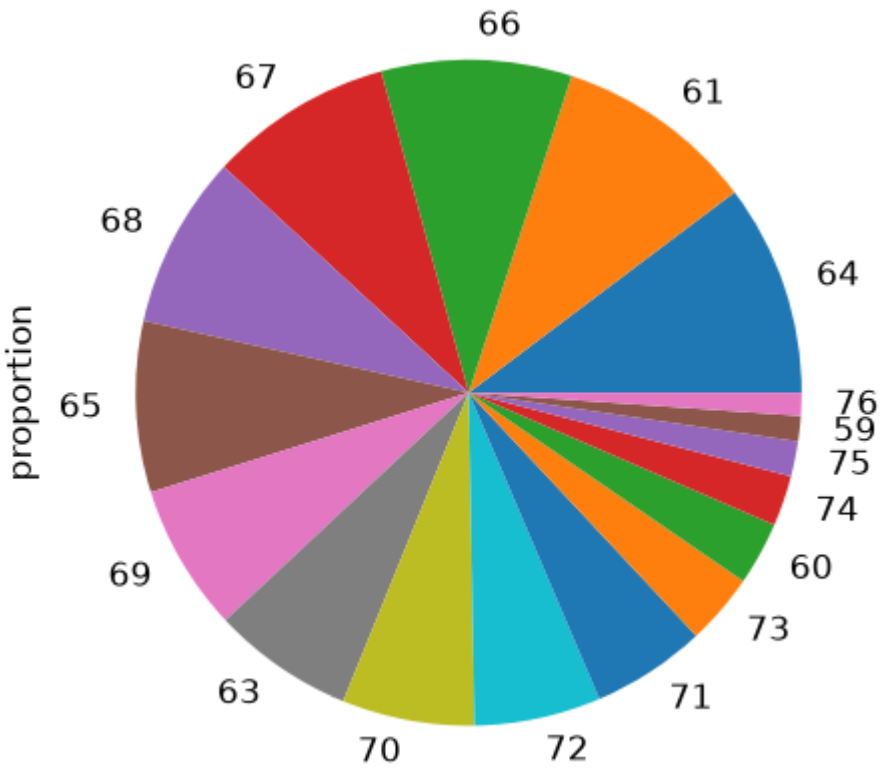


Pie Charts

As an alternative to bar charts, relative frequencies can also be displayed using pie charts.

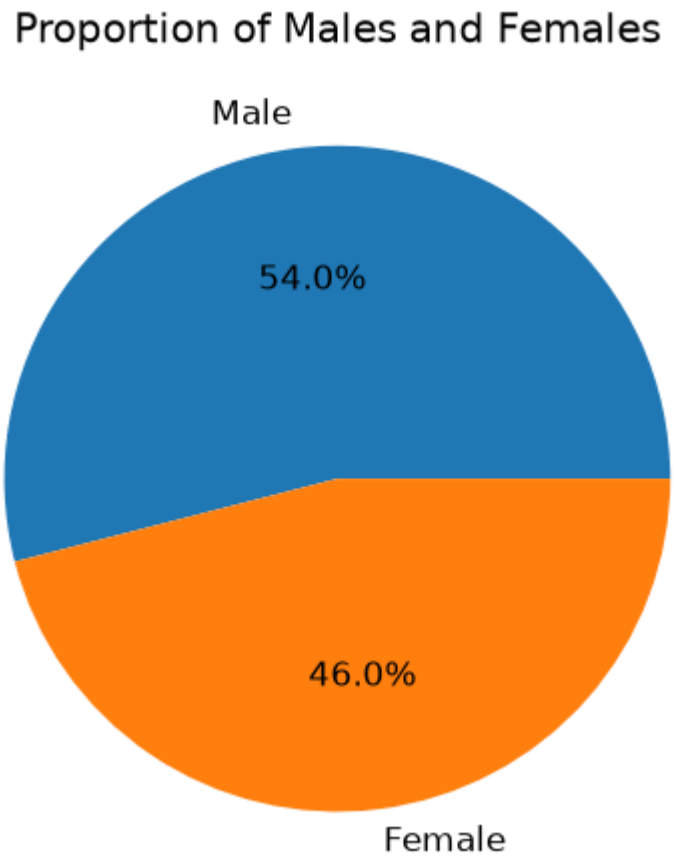
In the case of heights:

```
heights.value_counts(normalize=True).plot.pie()
plt.show()
```



Pie charts works better when we have fewer elements. For instance, let's say we want to show the percentage of males and females in our dataset:

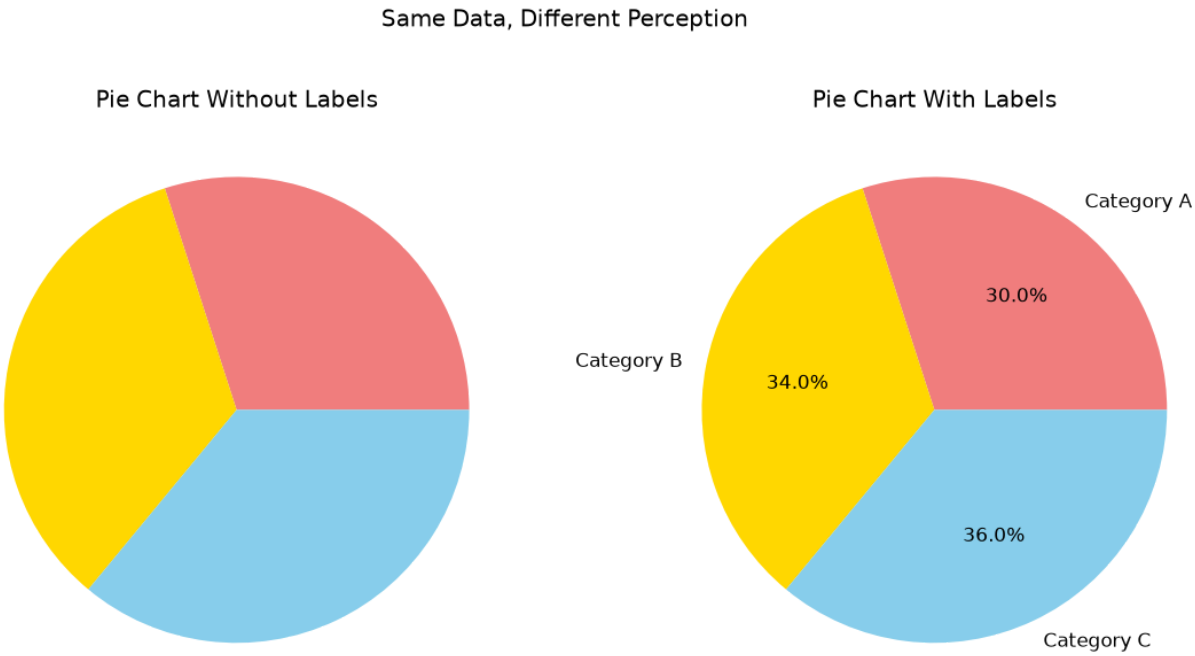
```
hw['sex'].value_counts(normalize=True).plot.pie(autopct='%1.1f%%', labels=['Male',
        'Female'])
plt.title('Proportion of Males and Females')
plt.ylabel('')
plt.show()
```



While pie charts can be intuitive when there is no specific ordering of the elements we want to show, they often introduce more problems than other chart types. In particular, it's difficult for the human eye to accurately compare angles or assess relative sizes—especially when slices are similar in proportion or not arranged consistently. This can lead to misinterpretation or visual clutter, making bar charts or other alternatives generally more effective for comparing categories.

An illustrative example is below:

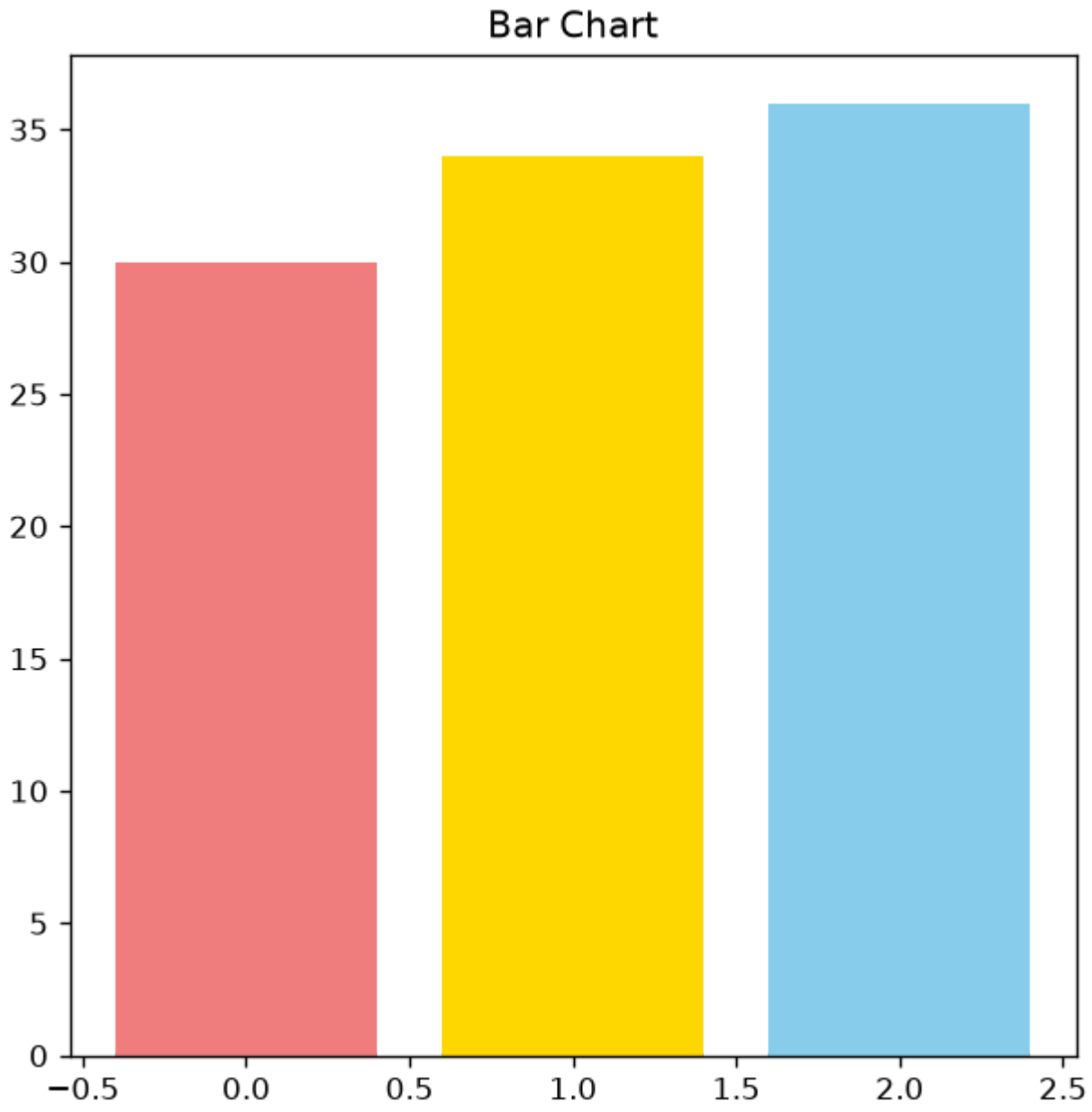
```python
import matplotlib.pyplot as plt

# Sample data: three categories with similar proportions
sizes = [30, 34, 36]
labels = ['Category A', 'Category B', 'Category C']
colors = ['lightcoral', 'gold', 'skyblue']

fig, axs = plt.subplots(1, 2, figsize=(12,6))

# Pie chart without labels
axs[0].pie(sizes, colors=colors)
axs[0].set_title('Pie Chart Without Labels')

# Pie chart with labels
axs[1].pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%')
axs[1].set_title('Pie Chart With Labels')

plt.suptitle('Same Data, Different Perception')
plt.show()
```



The figure shows the same plot with and without labels. Without labels, even significant differences as 4 are not easy to spot.

Compare the pie chart above with the bar chart below. Where is it easier to spot differences, even without labels?

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6,6))
ax.bar(range(len(sizes)), sizes, color=colors)
ax.set_title('Bar Chart')
plt.show()
```



In short: pie charts should be used with extra care!

Empirical Cumulative Distribution Function (ECDF)

Relative frequencies work particularly well when there are few unique values. However, when the number of unique values grows, the discrete frequencies calculated for the values become very small and thus subject to noise (e.g., due to measurement errors).

Let's now consider the sample of weights and its relative frequencies:

```
weights = hw['weight']
weights.value_counts(normalize=True).sort_index()
```

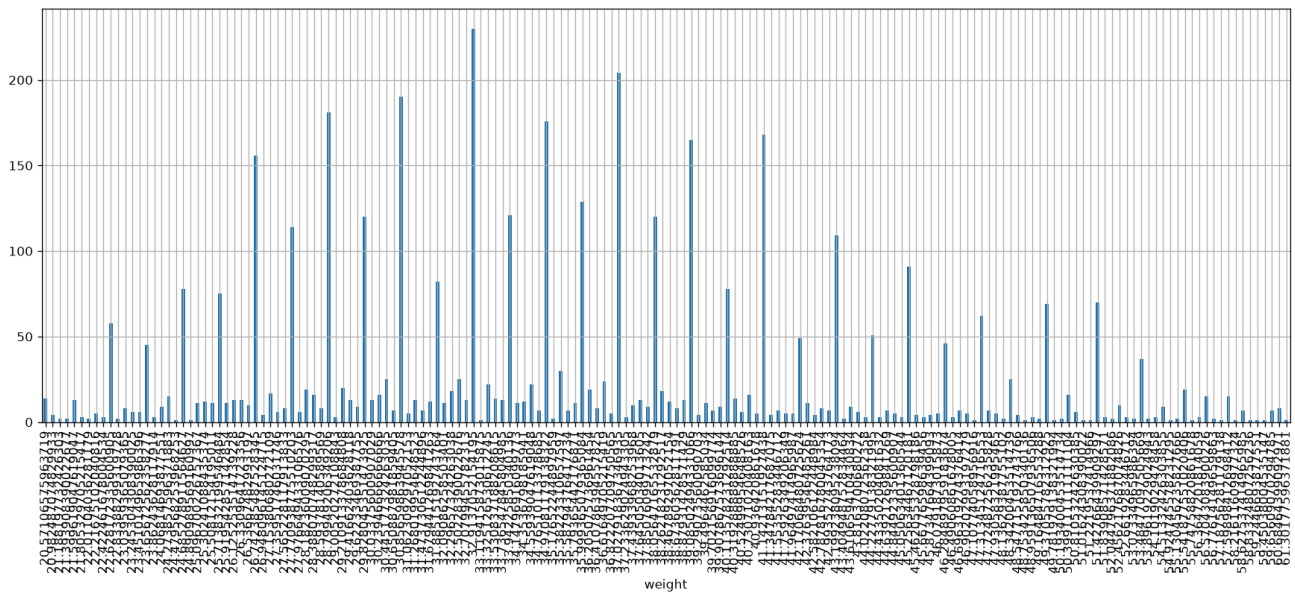
```
weight
20.571066    0.003309
20.982487    0.000945
21.188198    0.000473
21.393908    0.000473
21.599619    0.003073
               ...
59.244669    0.000236
59.450380    0.000236
59.656091    0.001654
60.684644    0.001891
61.301776    0.000236
Name: proportion, Length: 172, dtype: float64
```

We note that we now have many more unique values (172) and that weights now have decimal values as well.

This suggests that, while heights were discretized—probably rounded when collected—weights were recorded with greater precision. As a result, the weight variable behaves more like a continuous numerical feature, whereas height appears more discretized or categorical in nature. This distinction influences how we visualize and model these variables.

Let's see the plot of absolute frequencies:

```
weights.value_counts().sort_index().plot.bar(figsize=(18,6))
plt.grid()
```



Not very easy to read, right?

The representation is affected by noise, whereby two very close values are treated as two distinct cases in probability calculations. We will see some ways to overcome this problem. One of them consists of calculating an **Empirical Cumulative Distribution Function (ECDF)**. An **ECDF** calculates for a value x the sum of the relative frequencies of all values y less than or equal to x:

$$ECDF(x) = \sum_{a_j : a_j \leq x} f(a_j)$$

Where a_j are the unique values within the sample, while in general $x \in \mathbb{R}$.

Consider a simple dataset of numerical values:

```
a = pd.Series([1, 5, 2, 6, 5, 4, 3, 5, 4, 2, 4, 5, 6, 4, 4, 3])
a
```

```
0     1
1     5
2     2
3     6
4     5
5     4
6     3
7     5
8     4
9     2
10    4
11    5
12    6
13    4
14    4
15    3
dtype: int64
```

The relative frequencies will be as follows:

```
a.value_counts(normalize=True).sort_index()
```

```
1    0.0625
2    0.1250
3    0.1250
4    0.3125
5    0.2500
6    0.1250
Name: proportion, dtype: float64
```

The ECDF calculated on the unique values will be the following:

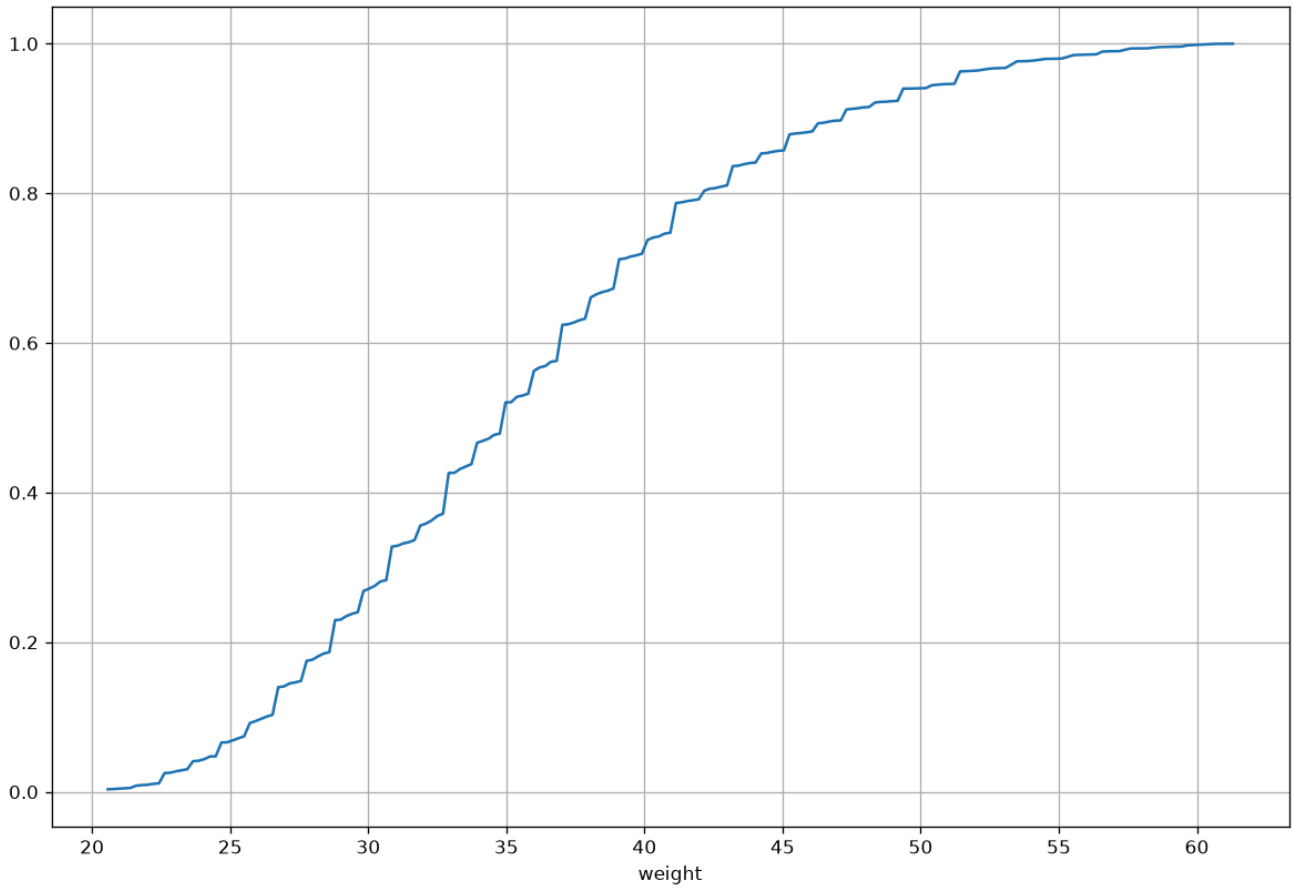
```
a.value_counts(normalize=True).sort_index().cumsum()
```

```
1    0.0625
2    0.1875
3    0.3125
4    0.6250
5    0.8750
6    1.0000
Name: proportion, dtype: float64
```

It should be noted that the values of the ECDF are always increasing and the last value is equal to 1 (the sum of all relative frequencies).

An ECDF can be represented graphically by placing the x values on the x-axis and the $ECDF(x)$ values on the y-axis. For example, the ECDF of the weights in our weights-heights dataset will be the following:

```
hw['weight'].value_counts(normalize=True).sort_index().cumsum().plot(figsize=(12,8))
plt.grid()
```



The definition of the ECDF tells us that, given a point with coordinates (x, y), the sum of the frequencies of elements less than or equal to x is equal to y. Alternatively, we can say that $y\%$ of the

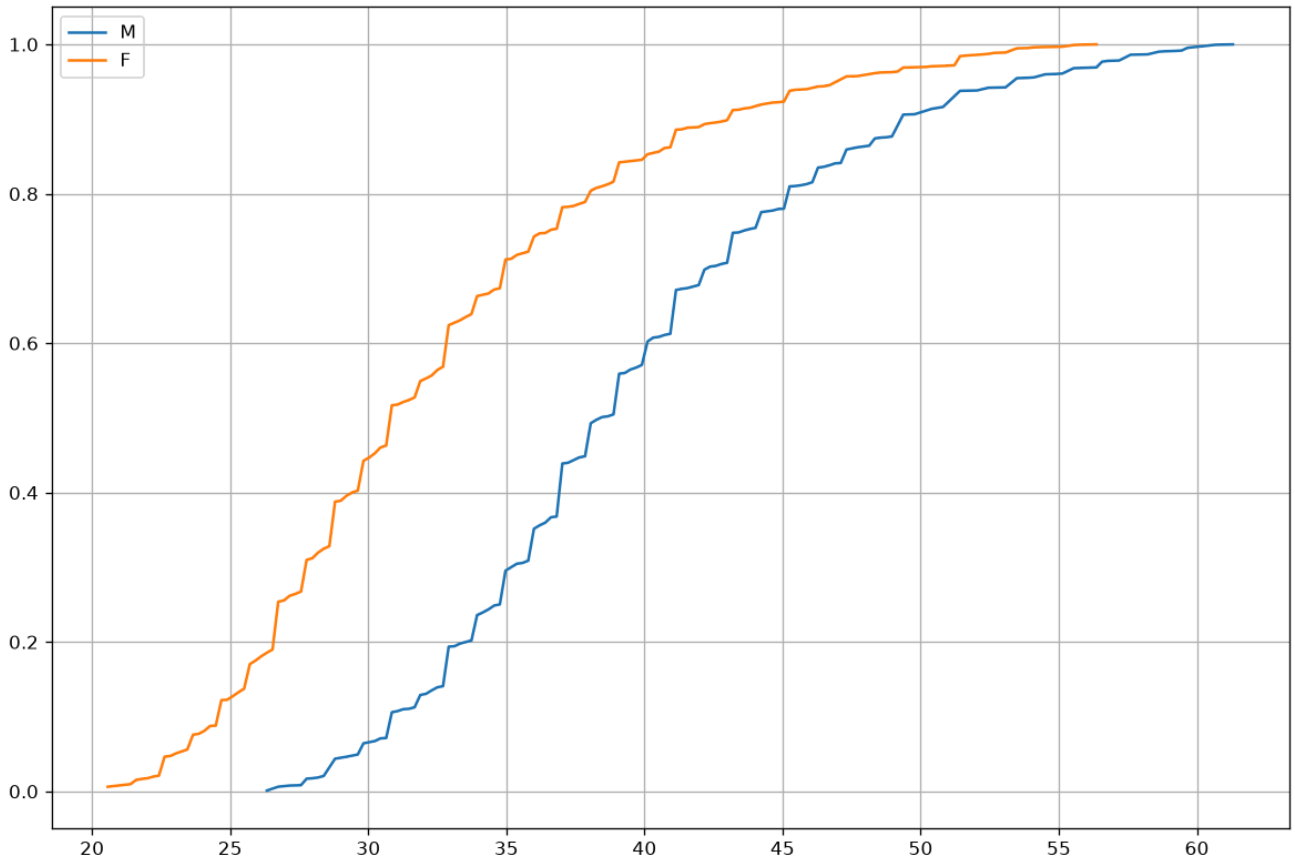
elements have a value less than x.

Observing the graph above, we can say that:

- Approximately 40% of subjects have a weight less than ≈ 33 pounds (about $66kg$);
- Approximately 80% of subjects have a weight less than ≈ 42 pounds (about $84Kg$);

ECDFs are useful for graphically checking if two phenomena have similar distributions. We can, for example, use them to compare the weight distributions of men and women:

```
ecdf_weight_m = hw[hw['sex']=='M']
        ['weight'].value_counts(normalize=True).sort_index().cumsum()
ecdf_weight_f = hw[hw['sex']=='F']
        ['weight'].value_counts(normalize=True).sort_index().cumsum()

plt.figure(figsize=(12,8))
plt.plot(ecdf_weight_m.index, ecdf_weight_m.values)
plt.plot(ecdf_weight_f.index, ecdf_weight_f.values)
plt.legend(['M','F'])
plt.grid()
plt.show()
```



Observing the graph above, we can say that:

- Approximately 40% of men weigh less than $80Kg$;
- Approximately 40% of women weigh less than $60Kg$;
- 100% of men weigh less than $140Kg$;

- 100% of women weigh less than $130Kg$.

In general, the graph above tells us that men tend to be heavier than women.

Histograms

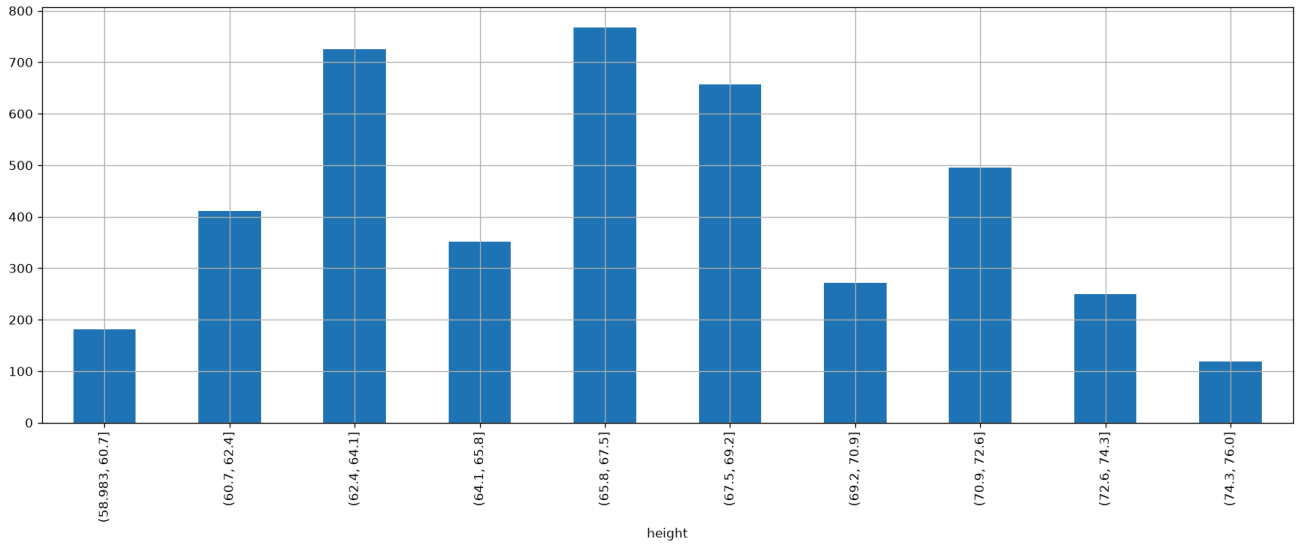
Another way to fix the noise problem is to group similar observations in the same "class". In practice, we will define intervals, which we call "bins" and assign each element to its corresponding bin. While this can be done in a variety of ways, bins are often chosen to be equals. This can be done using the cut function of Pandas:

```
heights_quantized = pd.cut(heights, bins=10)
heights_quantized
```

```
0       (72.6, 74.3]
1       (69.2, 70.9]
2       (60.7, 62.4]
3       (67.5, 69.2]
4       (65.8, 67.5]
           ...
4226    (67.5, 69.2]
4227    (67.5, 69.2]
4228    (62.4, 64.1]
4229    (62.4, 64.1]
4230    (60.7, 62.4]
Name: height, Length: 4231, dtype: category
Categories (10, interval[float64, right]): [(58.983, 60.7] < (60.7, 62.4] < (62.4,
64.1] < (64.1, 65.8] ... (69.2, 70.9] < (70.9, 72.6] < (72.6, 74.3] < (74.3, 76.0]]
```

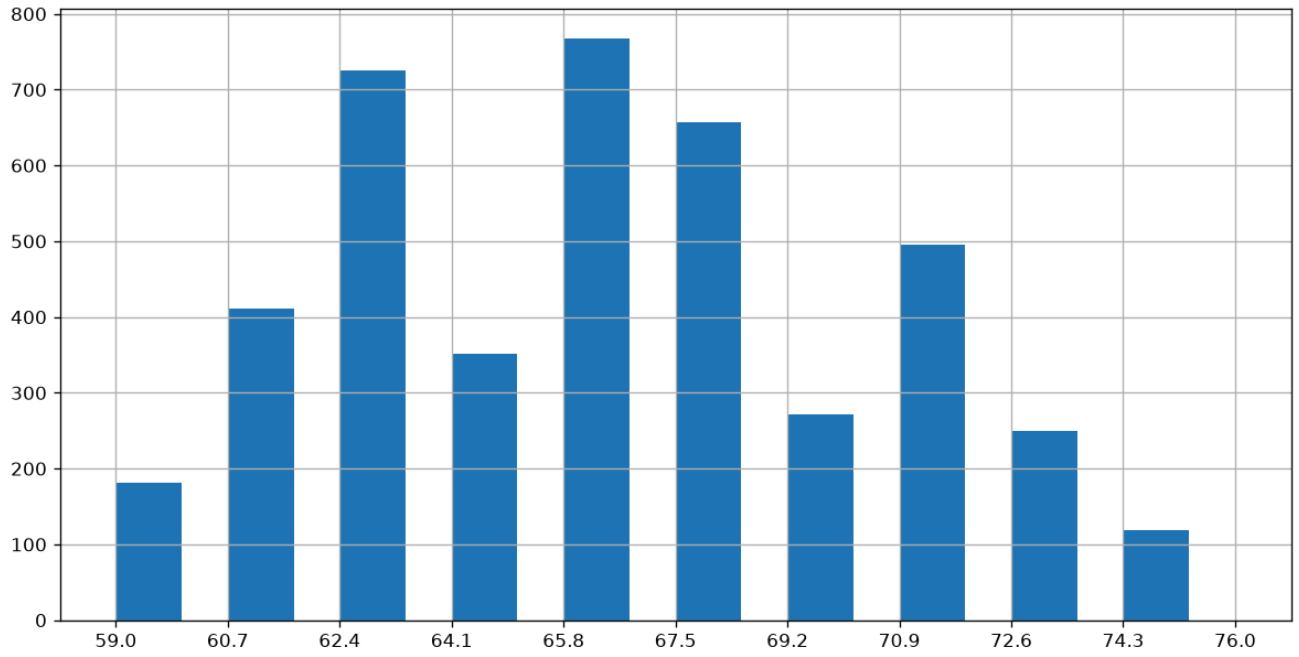
Let's visualize absolute frequencies of this binned version of the data:

```
heights_quantized.value_counts().sort_index().plot.bar(figsize=(18,6))
plt.grid()
plt.show()
```



The visualization we obtain is an histogram, and can be obtained also as follows:

```
plt.figure(figsize=(12,6))
_,edges,_=plt.hist(heights, width=1, bins=10) # We capture the "edges" of the bins
plt.xticks(edges) # Set x-ticks to the edges of the bins for better readability
plt.grid()
plt.show()
```



Each histogram “bin” covers a specific range. Common practice is to divide the data range, from minimum to maximum, into a certain number of bins of equal width.

Choosing the Number of Bins: Sturges and Rice

The number of bins can be specified arbitrarily or determined based on the graphical result you want to achieve. There are two heuristic criteria for finding starting values:

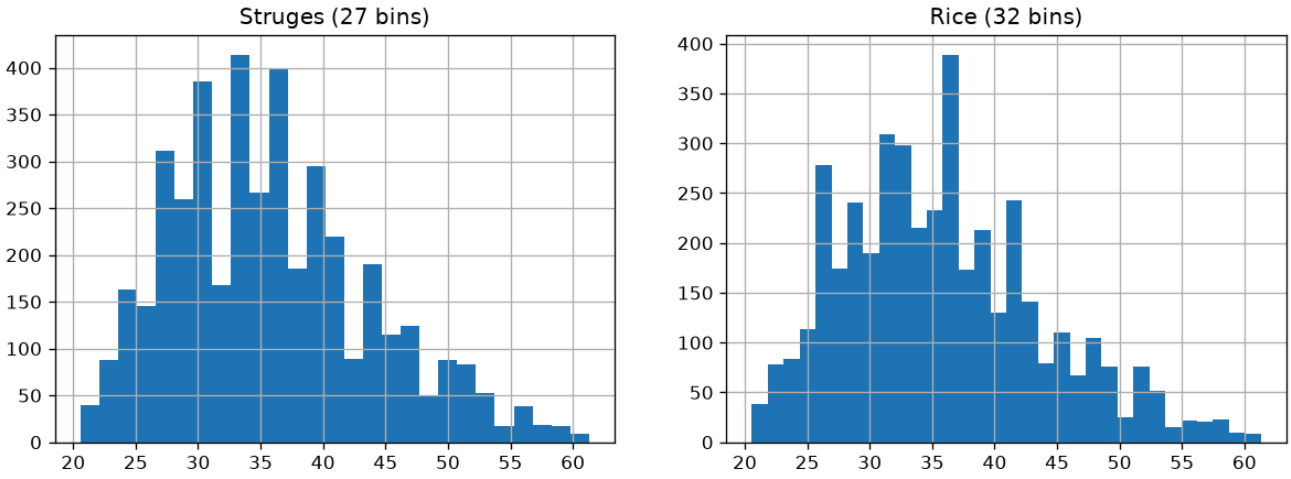
- Sturges: $\#bins = 3.3 \log(n)$

- Rice: $\#bins = 2 \cdot n^{1/3}$

Where n is the number of elements.

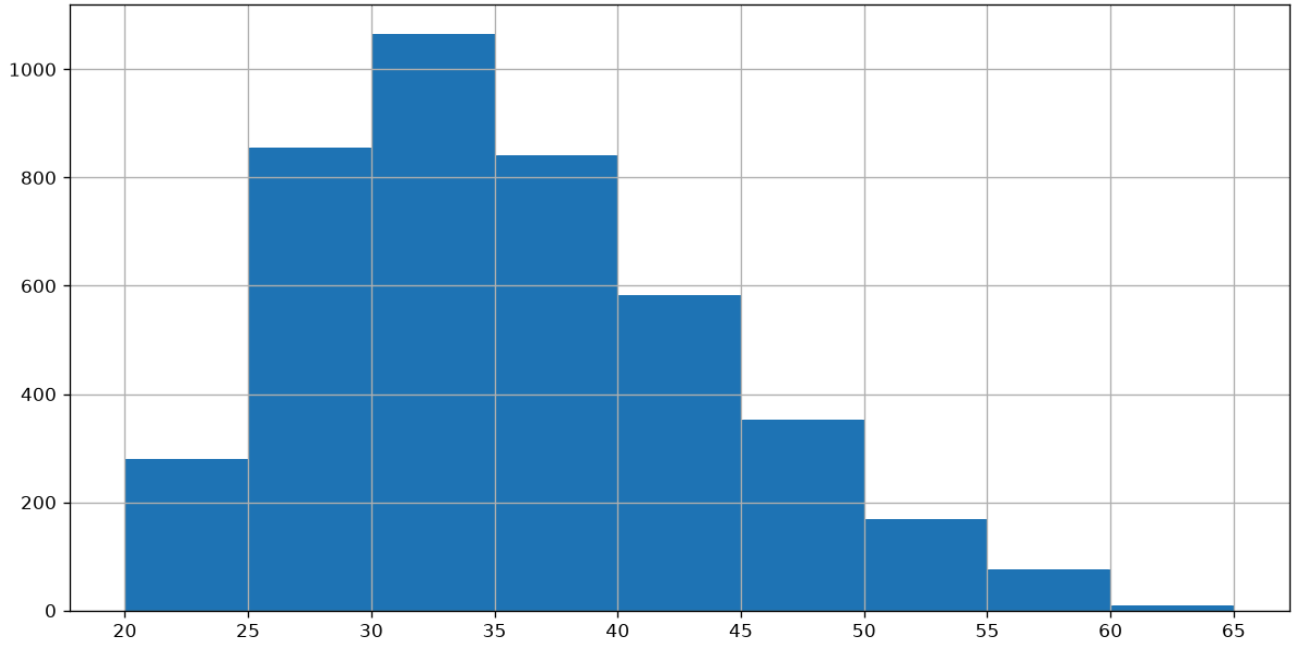
It should be noted that the number of bins can change the graphical result. Below are two examples obtained by calculating the number of bins with the two criteria considered:

```
import numpy as np
bins_struges=int(3.3*np.log(len(weights)))
bins_rice=int(2*len(weights)**(1/3))

plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.title('Struges ({} bins)'.format(bins_struges))
plt.hist(weights, bins=bins_struges)
plt.grid()
plt.subplot(1,2,2)
plt.title('Rice ({} bins)'.format(bins_rice))
plt.grid()
plt.hist(weights, bins=bins_rice)
plt.show()
```



Alternatively, we can define arbitrary ranges for our bins. For example, considering the following limits for the bins `[20,25,30,35,40,45,50,55,60,65]`, we would obtain the following histogram:

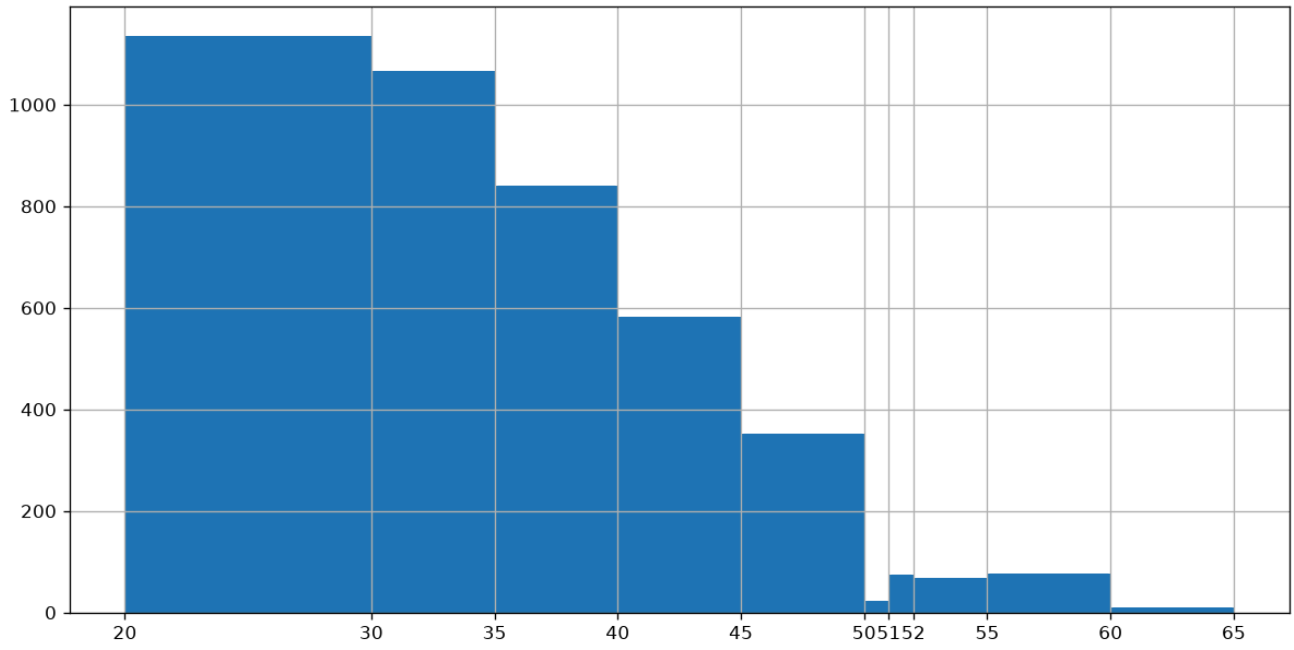
```
plt.figure(figsize=(12,6))
_,edges,_=plt.hist(weights, bins=[20,25,30,35,40,45,50,55,60,65]) # construct a
        histogram with defined bins
plt.xticks(edges)
plt.grid()
plt.show()
```



The histogram shown above reports the absolute frequencies for each bin and allows us to answer questions such as "how many subjects have a weight between 30 and 35 pounds"?

In particular cases, it is also possible to define histograms with bins of variable size. For example:

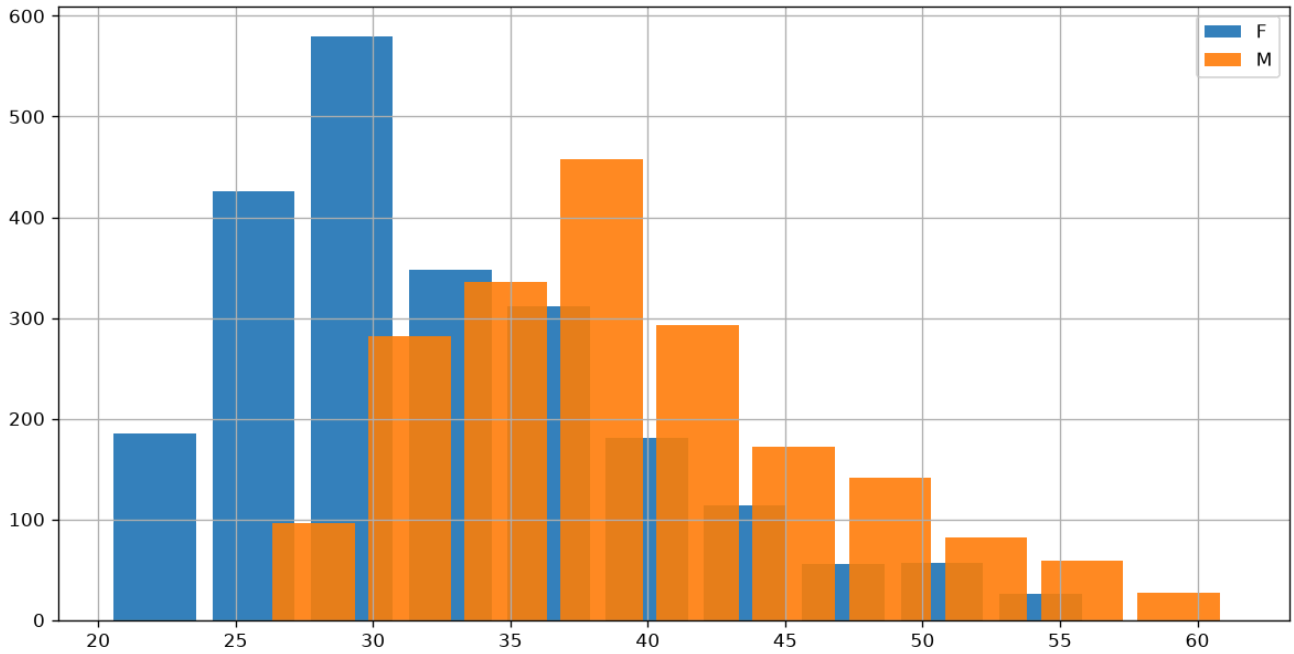
```
plt.figure(figsize=(12,6))
_,edges,_=plt.hist(weights, bins=[20,30,35,40,45,50,51,52,55,60,65]) # construct a
        histogram with the defined bins
plt.xticks(edges)
plt.grid()
plt.show()
```



Comparing Samples Using Histograms

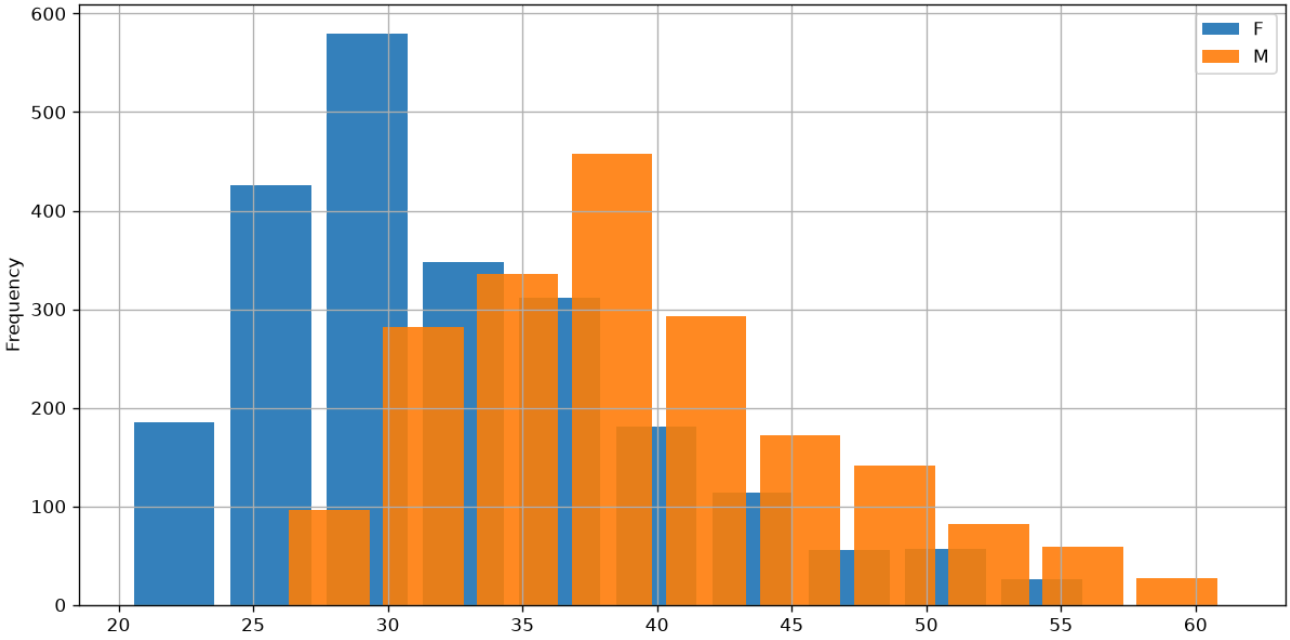
Histograms can be useful for comparing samples. The following graph compares the weights of men and women in our weights-heights sample:

```
plt.figure(figsize=(12,6))
plt.hist(hw[hw['sex']=='F']['weight'], alpha=0.9, width=3)
plt.hist(hw[hw['sex']=='M']['weight'], alpha=0.9, width=3)
plt.legend(['F','M'])
plt.grid()
plt.show()
```



Note that we can obtain a similar results directly from Pandas using the “groupby” function:

```
hw.groupby('sex')['weight'].plot.hist(width=3, alpha=0.9, figsize=(12,6))
plt.legend()
plt.grid()
plt.show()
```



Density Estimation

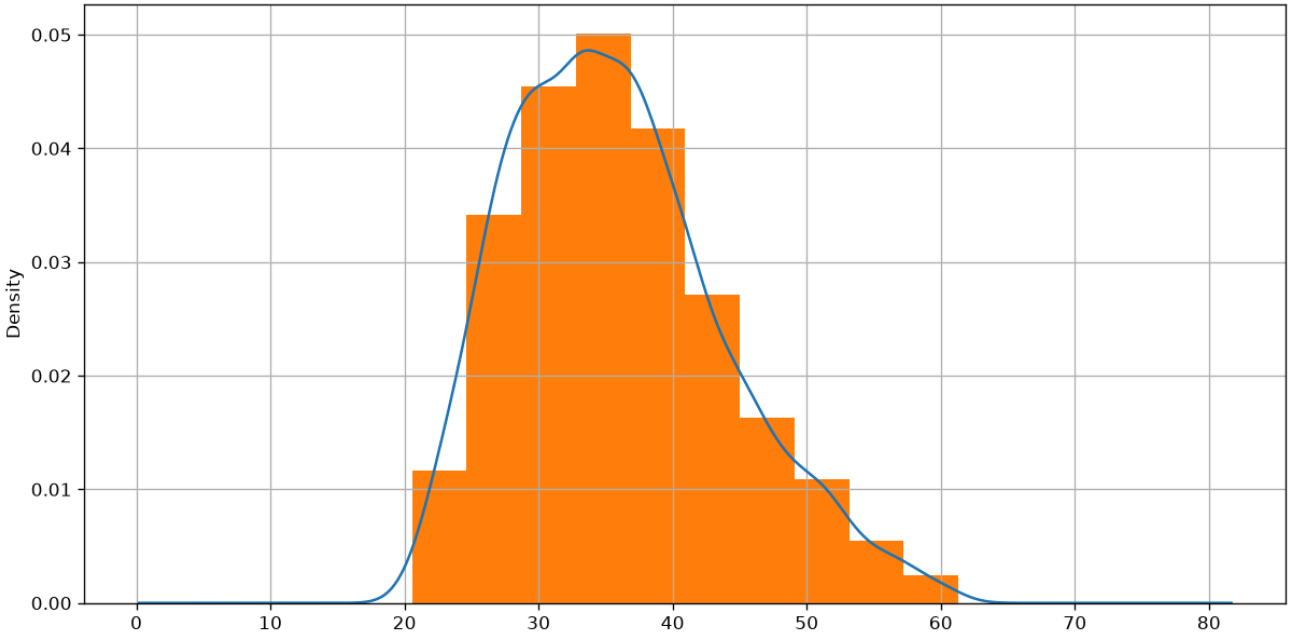
One disadvantage of histograms is that they arbitrarily categorize continuous data using bins. The choice of bin intervals changes the final appearance of the histogram.

Density estimation attempts to solve this problem by obtaining a "continuous" version of the histogram. Instead of dividing the x-axis into bins, density estimation calculates a value for each point on the x-axis, thus obtaining a continuous representation.

We will look more closely at how density estimation is performed later in the course. In the one-dimensional case, it can be calculated as shown in the optional deep dive below. For now, we know that density estimation can be calculated with a dedicated function "density" that depends on a single **bandwidth** parameter (with a good default value being computed by the libraries), determining the "sensitivity to detail" of the density estimation.

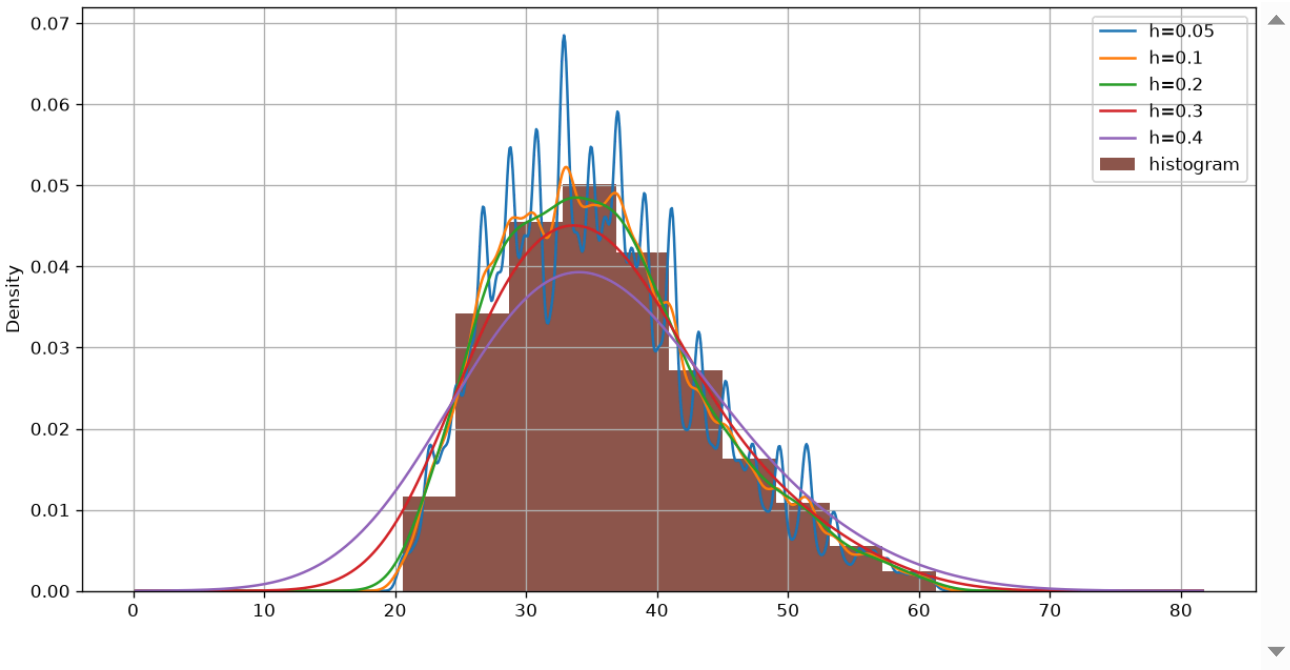
Let's compare the weight density estimation in our weight-height dataset with its corresponding histogram:

```
hw['weight'].plot.density(figsize=(12,6))
plt.hist(hw['weight'], density=True)
plt.grid()
plt.show()
```



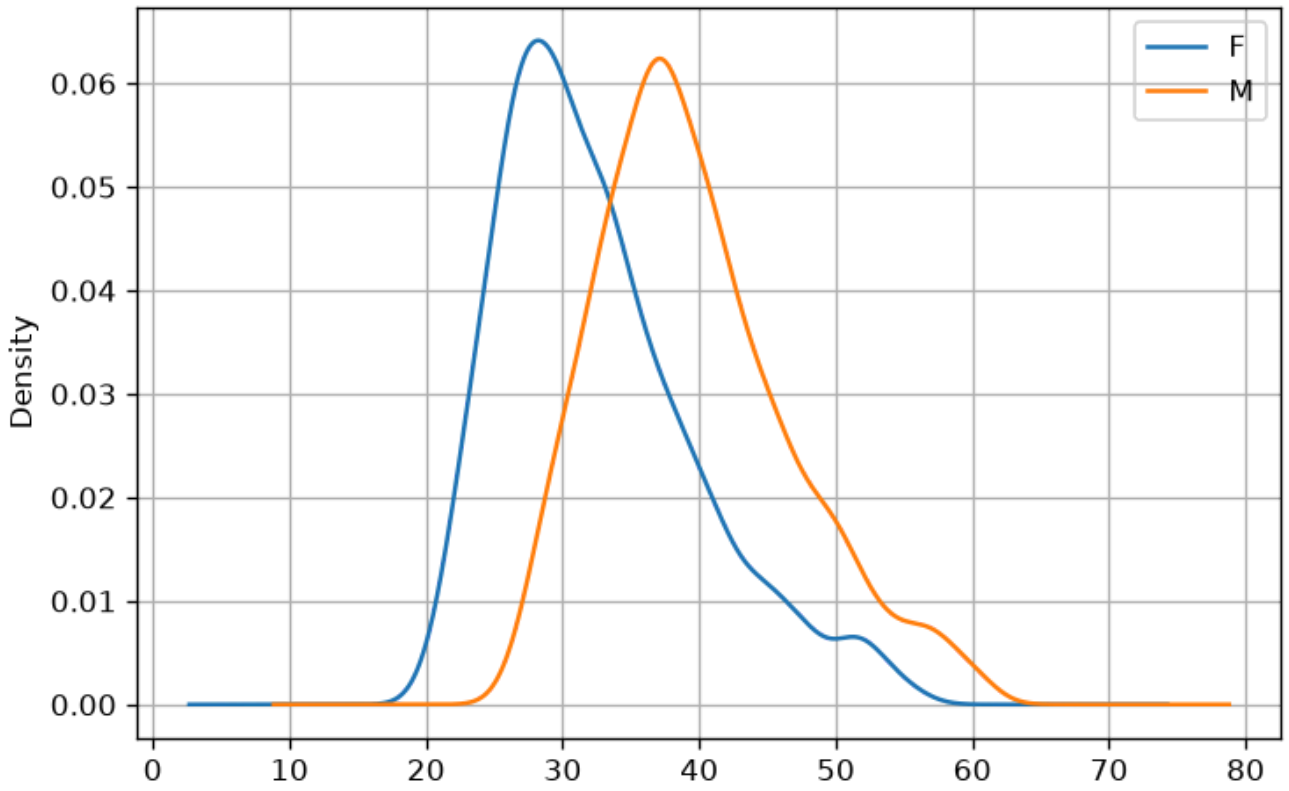
If in the case of histograms changing the number of bins changed the graphical result, here it is changing the bandwidth that changes the graphical result. The following graph shows several examples of density estimation with different bandwidth values:

```
fig, ax = plt.subplots()
hw['weight'].plot.density(figsize=(12,6), bw_method=0.05)
hw['weight'].plot.density(figsize=(12,6), bw_method=0.1)
hw['weight'].plot.density(figsize=(12,6), bw_method=0.2)
hw['weight'].plot.density(figsize=(12,6), bw_method=0.5)
hw['weight'].plot.density(figsize=(12,6), bw_method=0.8)
plt.hist(hw['weight'], density=True)
ax.legend(['h=0.05','h=0.1','h=0.2','h=0.3','h=0.4','histogram'])
plt.grid()
plt.show()
```



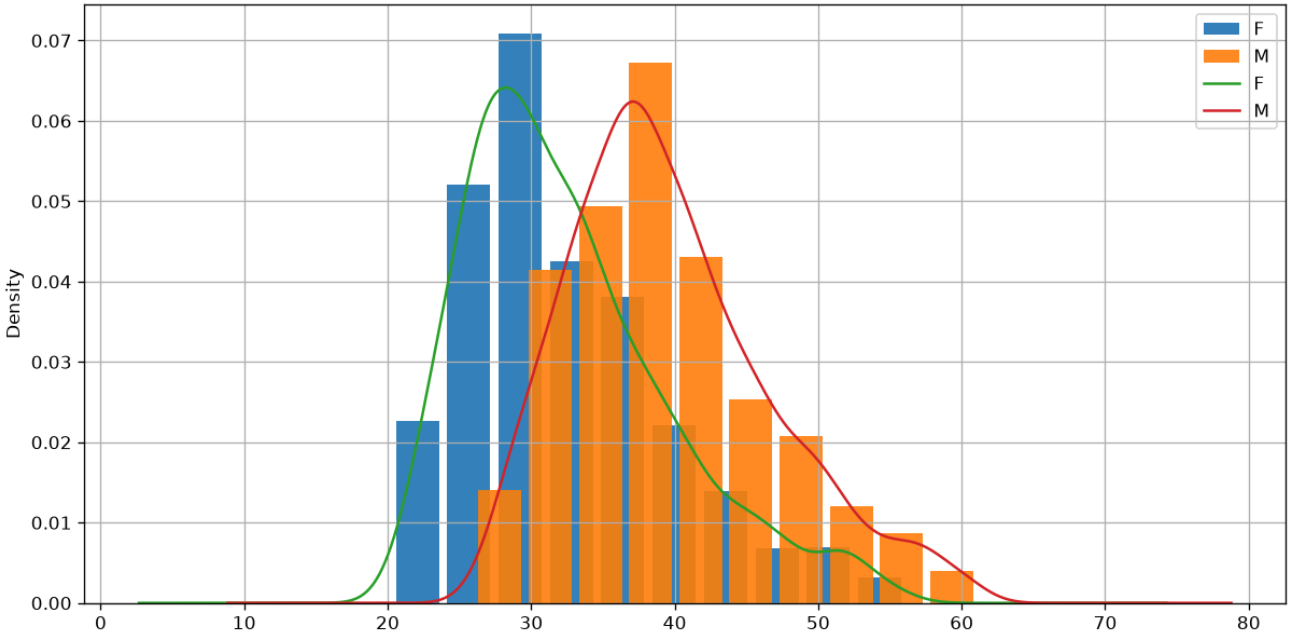
We can compare two samples using density estimations in Pandas very easily as follows:

```
hw.groupby('sex')['weight'].plot.density()
plt.legend()
plt.grid()
plt.show()
```



We can also mix histograms and density plots as follows:

```
hw.groupby('sex')['weight'].plot.hist(width=3, alpha=0.9, figsize=(12,6),
        density=True) # density=True to normalize the histograms
hw.groupby('sex')['weight'].plot.density()
plt.legend()
plt.grid()
plt.show()
```



In this case, we had to specify “density=True” when computing the histogram to make sure that the ranges on the y axes of density estimation and histogram were the same. This leads us to a **density histogram**. We will see the theoretical details of this in a later lecture.

Summary Statistics

For now, we have seen graphical ways to represent the data. In practice, it can be useful to attach some numerical values to the data in order to summarize it.

Descriptive statistics deals with describing, representing, and summarizing a data sample pertaining to a population. The tools of descriptive statistics can be both numerical and graphical. The analyzed data can be described according to various aspects. Therefore, there are various objective “indicators”:

- **sample size**;
- **central indicators**: mean, median, mode;
- **Dispersion indicators**: extremes, range, quantiles, percentiles, quartiles, interquartile range, variance;

We will investigate these tools on the `Titanic` dataset. Let’s load it:

```
import pandas as pd
titanic = pd.read_csv(DATA + "titanic.csv",
                      index_col='PassengerId')
titanic.head()
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

We will start by focusing on the **univariate** sample of ages:

```
age = titanic['Age']
age
```

```
PassengerId
1      22.0
2      38.0
3      26.0
4      35.0
5      35.0
       ...
887    27.0
888    19.0
889     NaN
890    26.0
891    32.0
Name: Age, Length: 891, dtype: float64
```

The sample contains `NaN` values, which will have to be handled appropriately.

Size

The size of a univariate sample $\{x^{(i)}\}_i^N$ is given by the number of values it contains: $|\{x^{(i)}\}_i^N| = N$. The size of each column can also be different as there may be missing values,

generally indicated by `NA` or `NaN`. In our case:

```
age.info()
```

```
<class 'pandas.core.series.Series'>
Index: 891 entries, 1 to 891
Series name: Age
Non-Null Count  Dtype
--------------  -----
714 non-null    float64
dtypes: float64(1)
memory usage: 13.9 KB
```

The summary above indicates that, although we have 891 values, only 714 of these are non-null. These are the "real" size of the sample.

Measures of central tendency

Central measures give an approximate idea of the order of magnitude of the sample values.

Mean

The mean of a sample is defined as the sum of its values divided by its size:

$$\overline{X} = \frac{1}{N} \sum_{i}^{N} x^{(i)}$$

The mean of our sample will be:

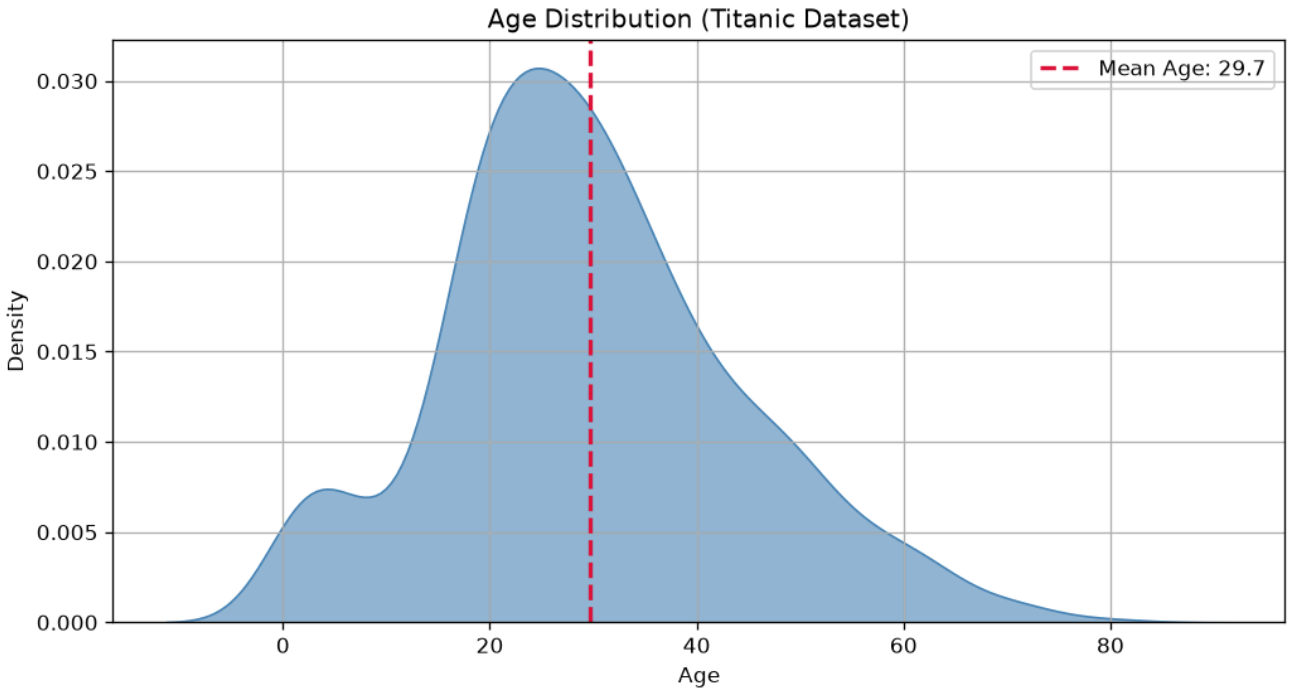
```
age.mean()
```

```
np.float64(29.69911764705882)
```

Note that the mean function will automatically ignore the missing data.

This statistic is very easy to interpret: the average age is about 30 years.

Let's see where this descriptive statistic stands in a graphical representation of the density of the sample:



Median

When the elements of a sample can be ordered (for example, if they are numerical values), the median of a sample (or the median element) is the element that divides the ordered set of sample values into two equal parts.

The median element can be obtained by ordering the sample values and proceeding as follows:

- If the number of elements is odd, take the central element. For example $[1, 2, \mathbf{2}, 3, 5] \rightarrow 2$.
- If the number of elements is even, take the average of the two central ones. For example $[1, 2, \mathbf{2}, \mathbf{3}, 3, 5] \rightarrow \frac{2+3}{2} = 2.5$.

In the case of our sample:

```
print("Median value:",age.median())
```

```
Median value: 28.0
```

From a formal point of view, if we have n observations $x^{(1)}, \ldots, x^{(n)}$, which can be ordered as $x^{(i_i)}, \ldots, x^{(i_n)}$, the calculation of the median can be expressed as follows:

$$\tilde{x}_{0.5} = \begin{cases} x^{(n+1)/2} & \text{if } n \textit{ is odd} \\ \frac{1}{2}(x^{(n/2)} + x^{(n/2+1)}) & \text{otherwise.} \end{cases}$$

To be convinced of the above formulation, we can see two toy examples:

```
odd_sample = pd.Series([1,2,3,4,100])
print(odd_sample.sort_values())
odd_sample.median()
```

```
0      1
1      2
2      3
3      4
4    100
dtype: int64
```

```
np.float64(3.0)
```

The median is the central element 3.

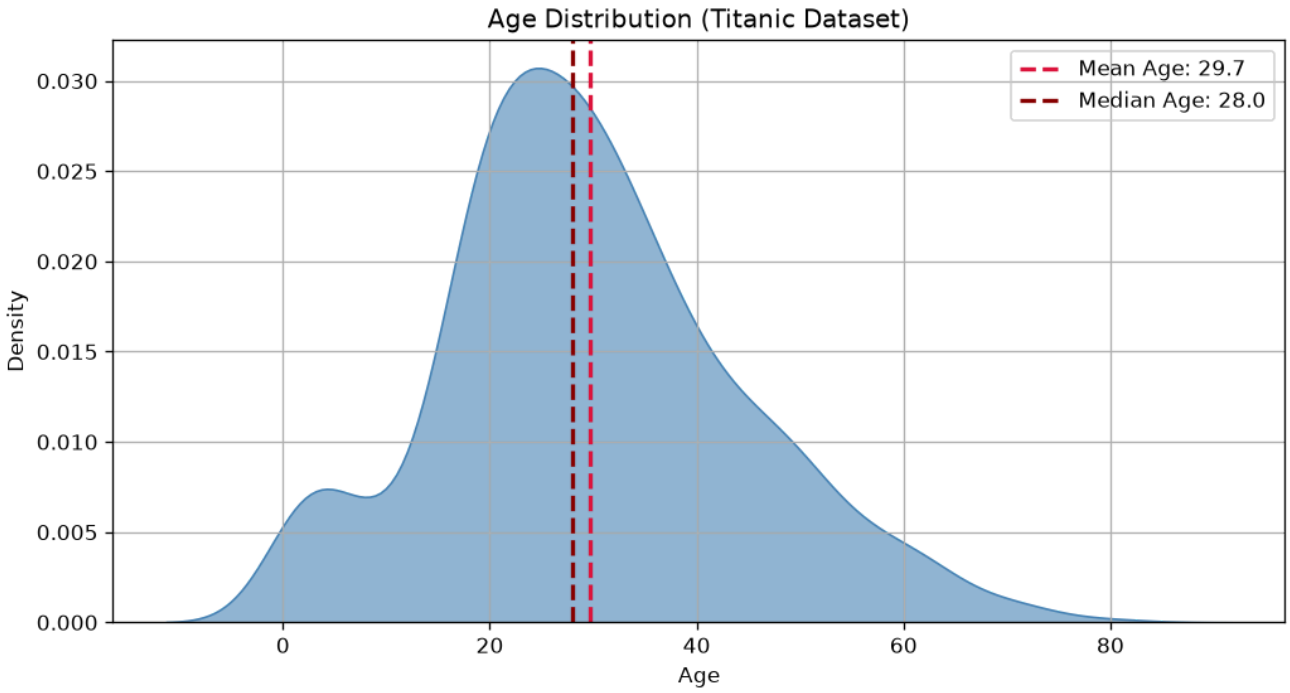
```
even_sample = pd.Series([1,2,3,4,5,100])
print(even_sample.sort_values())
even_sample.median()
```

```
0      1
1      2
2      3
3      4
4      5
5    100
dtype: int64
```

```
np.float64(3.5)
```

The median is the average between the two central elements 3 and 4.

Let's see where this descriptive statistic stands in a graphical representation of the density of the sample:



As we can see, depending on the density of the data, the two values are not always identical.

Quantiles, Percentiles, and Quartiles

Quantiles, percentiles, and quartiles generalize the concept of the median.

Quantiles

A quantile of order α is a value q_α that divides a sample into two parts with sizes proportional to α and $1-\alpha$. Values less than or equal to q_α belong to the first part of the division, while values greater than q_α belong to the second part.

For example, given the already ordered sample `[1,2,3,3,4,5,6,6,7,8,8,9]`, a quantile $q_{0.25}$ will divide the sample into two parts with sizes proportional to 0.25 and $1-0.25=0.75$. In this case $q_{0.25}=3$ and the two parts will be `[1,2,3,3]` and `[4,5,6,6,7,8,8,9]`.

In this case too, as with the median, averages of adjacent values are taken where appropriate.

Quantiles should be interpreted as follows:

> If a quantile of order α is equal to the number x, then it means that $\alpha \times n$ elements have a value less than or equal to x, where n is the number of elements in the sample.

It should be noted that:

- The minimum is a quantile of order 0;

- The maximum is a quantile of order 1;
- The median is a quantile of order 0.5.

Let's see some examples on our sample:

```
print("Order 0 Quantile (minimum):", age.quantile(0))
print("Order 0.5 Quantile (median):", age.quantile(0.5))
print("Order 1 Quantile (maximum):", age.quantile(1))
print("Order 0.15 Quantile:", age.quantile(0.15))
```

```
Order 0 Quantile (minimum): 0.42
Order 0.5 Quantile (median): 28.0
Order 1 Quantile (maximum): 80.0
Order 0.15 Quantile: 17.0
```

From the above data we deduce that:

- 50% of the passengers are younger than 28.
- 15% of the passengers are younger than 17.

Percentiles

Percentiles are simply quantiles expressed as a percentage. A quantile of order 0.25 is a percentile of order 25%. In practice, we still use the quantile function to compute them:

```
age.quantile(25/100)
```

```
np.float64(20.125)
```

Quartiles

Quartiles are specific quantiles that divide the sample into four parts. In particular:

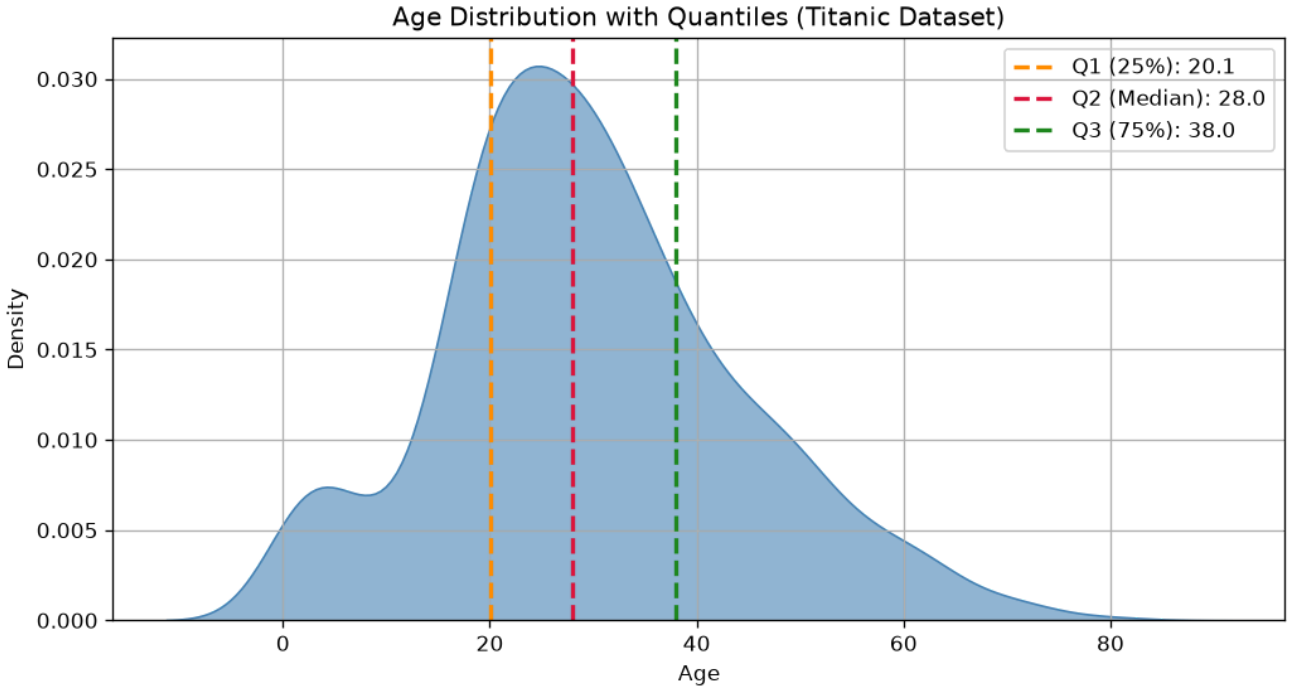
- The quartile of order 0 is a quantile of order 0;
- The quartile of order 1 is a quantile of order $1/4 = 0.25$;
- The quartile of order 2 is a quantile of order $2/4 = 0.5$;
- The quartile of order 3 is a quantile of order $3/4 = 0.75$;
- The quartile of order 4 is a quantile of order $4/4 = 1$.

Also in this case, we use the quantile function:

```
print("Quartile of order 0 (minimum):", age.quantile(0/4))
print("Quartile of order 1:", age.quantile(1/4))
print("Quartile of order 2 (median):", age.quantile(2/4))
print("Quartile of order 3:", age.quantile(3/4))
print("Quartile of order 4 (maximum):", age.quantile(4/4))
```

```
Quartile of order 0 (minimum): 0.42
Quartile of order 1: 20.125
Quartile of order 2 (median): 28.0
Quartile of order 3: 38.0
Quartile of order 4 (maximum): 80.0
```

Let's see this in graphical terms:



Mode

The **mode** of a sample is the value that appears **most frequently** within the data. It represents the most common observation. It can se computed as follows:

```
age.mode()
```

```
0    24.0
Name: Age, dtype: float64
```

In formal terms, the mode $\overline{x}_M$ of a sample is given by:

$$\overline{x}_M = a_j \Leftrightarrow n_j = \max\{n_1, \ldots, n_k\}$$

Where:

- a_j are the unique values of the sample (values without repetition)
- n_j are their respective absolute frequencies (how many times a value repeats)

Let's see this with a toy example:

Let's see how to compute it manually, with a toy example:

```
data=pd.Series([1,2,3,4,2,5,4,2,6,5,8,4,3,2,3])
data
```

```
0     1
1     2
2     3
3     4
4     2
5     5
6     4
7     2
8     6
9     5
10    8
11    4
12    3
13    2
14    3
dtype: int64
```

The unique values a_j with the respective absolute frequencies n_j (counts) are computed as:

```
print(data.value_counts())
```

```
2    4
3    3
4    3
5    2
1    1
6    1
8    1
Name: count, dtype: int64
```

In the output above, the first column represents the unique values a_j, while the second column represents the absolute frequencies n_j. Since the list is already sorted by counts, we can extract the maximum as follows:

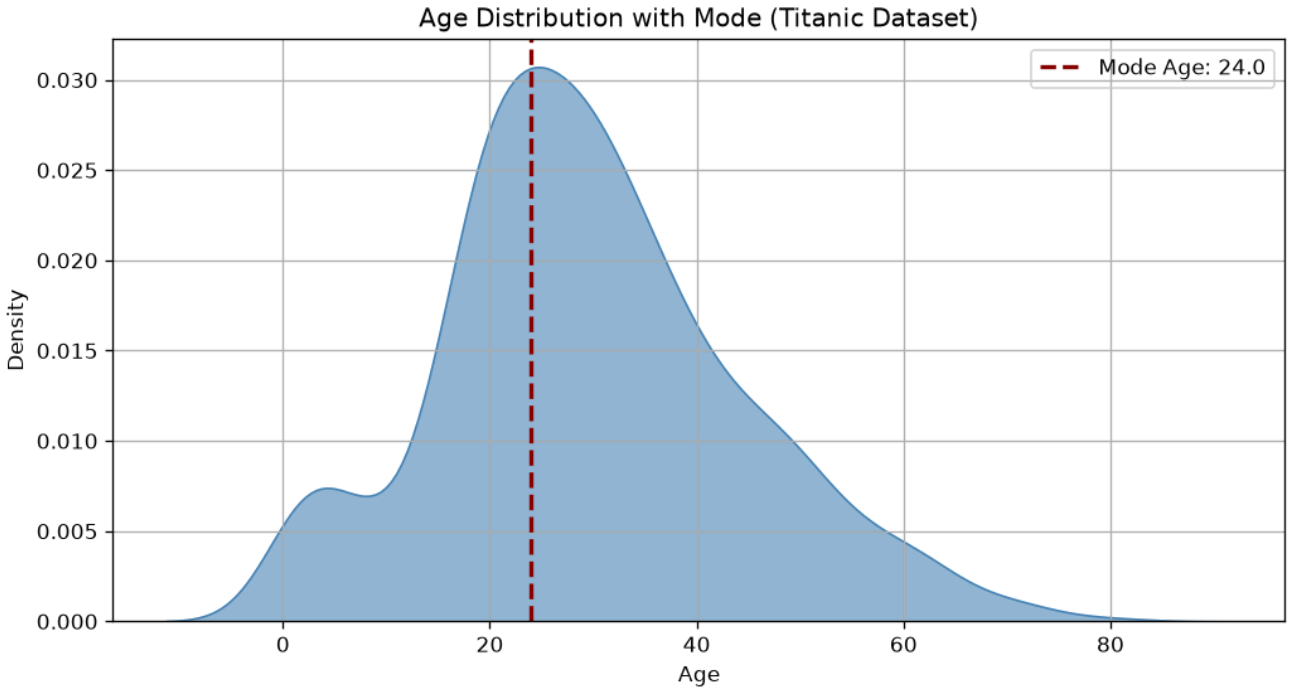
```
print("Mode:\n",data.value_counts().index[0])
```

```
Mode:
 2
```

The mode is 2 as this number repeats most times (it repeats 4 times).

In graphical terms:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Drop missing ages
ages = titanic['Age'].dropna()

# Compute mode (may return multiple values, so we take the first)
mode_age = ages.mode().iloc[0]

# Plot density
plt.figure(figsize=(10,5))
sns.kdeplot(ages, fill=True, color='steelblue', alpha=0.6)
plt.axvline(mode_age, color='darkred', linestyle='--', linewidth=2, label=f'Mode Age:
        {mode_age:.1f}')

# Customize plot
plt.title('Age Distribution with Mode (Titanic Dataset)')
plt.xlabel('Age')
plt.ylabel('Density')
plt.legend()
plt.grid()
plt.show()
```



The mode is the “peak” of the density of the data as it represents the most common value.

Measures of Dispersion

Measures of dispersion serve to quantify to what extent the values of a distribution are “dispersed”, or “far from each other”.

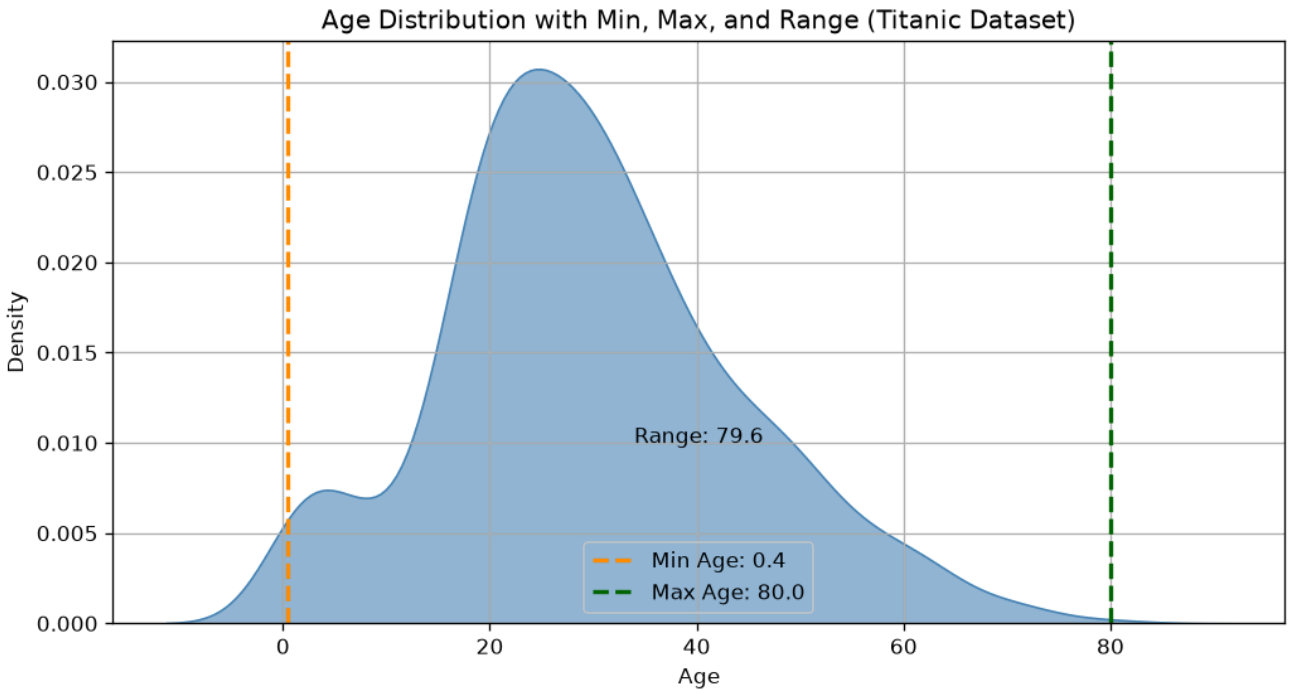
Minimum, Maximum, and Range

Simple indices of dispersion are the minimum ($min\{x^{(i)}\}_i^N$), the maximum ($max\{x^{(i)}\}_i^N$), and the range ($max\{x^{(i)}\}_i^N$ - $min\{x^{(i)}\}_i^N$). These are easily computed as follows:

```
print("Min value:",age.min())
print("Max value:",age.max())
print("Range:",age.max()-age.min())
```

```
Min value: 0.42
Max value: 80.0
Range: 79.58
```

Let’s see this graphically:



We can see that there is some density before 0 because the density estimation function “interpolates” what may happen before the zero if we had data there. This is an artifacts of the continuous nature of this visualization.

Interquartile Range (IQR)

The range is not a very robust measure of dispersion, as it does not take into account the presence of any outliers. Consider, for example, the following “artificial” samples:

```
from matplotlib import pyplot as plt
import numpy as np
import pandas as pd
sample1 = np.linspace(3,7,10)
sample2 = sample1.copy()
sample2[0]=-5 # we replace two values with the outliers -5
sample2[-1]=15 # and 15

df = pd.DataFrame({'s1':sample1,'s2':sample2})
print(df)

# df.plot.box()
# plt.grid()
# plt.show()

print("Range sample 1:",df['s1'].max()-df['s1'].min())
print("Range sample 2:",df['s2'].max()-df['s2'].min())
```

```
         s1         s2
0  3.000000  -5.000000
1  3.444444   3.444444
2  3.888889   3.888889
3  4.333333   4.333333
4  4.777778   4.777778
5  5.222222   5.222222
6  5.666667   5.666667
7  6.111111   6.111111
8  6.555556   6.555556
9  7.000000  15.000000
Range sample 1: 4.0
Range sample 2: 20.0
```

The samples are similar, but the presence of two outliers (-5 and 15) in the second sample makes the ranges very different (4 and 20).

A **more expressive measure of dispersion** is therefore the **interquartile range** (or **interquartile distance**), which is measured as the difference between the third and first quartiles:

```
q11,q13 = df['s1'].quantile([1/4,3/4])
q21,q23 = df['s2'].quantile([1/4,3/4])
print("The interquartile range of sample 1 is:",q13-q11)
print("The interquartile range of sample 2 is:",q23-q21)
```

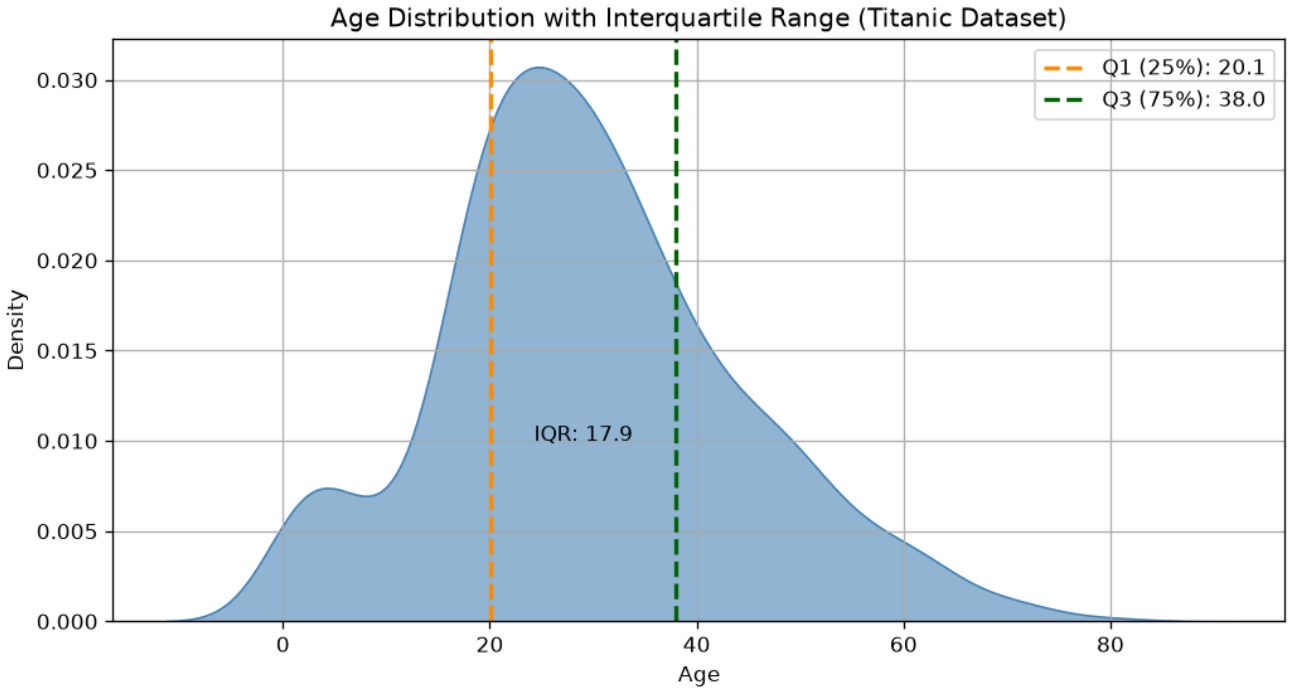
```
The interquartile range of sample 1 is: 2.0
The interquartile range of sample 2 is: 2.0
```

Let's see how to compute the IQR in the case of the Age sample:

```
q1,q3 = age.quantile([1/4,3/4])
print("The interquartile range of age is:",q3-q1)
```

```
The interquartile range of age is: 17.875
```

Graphically:



Variance and Standard Deviation

Variance (also known as **mean squared deviation**) provides an estimate of how much the observed data deviates from the mean. Variance calculates the mean of the squares of the deviations of the values from the mean, penalizing large deviations from the mean value (due to outliers) more heavily than small deviations:

$$s_n^2 = \frac{\sum_{i=1}^{n}(x_i - \overline{x})^2}{n}$$

It can be shown that, when computing variance from smaller samples, it can lead to **biased estimates**. Hence, it is very common to use the **Sample variance** with $n-1$ in the denominator:

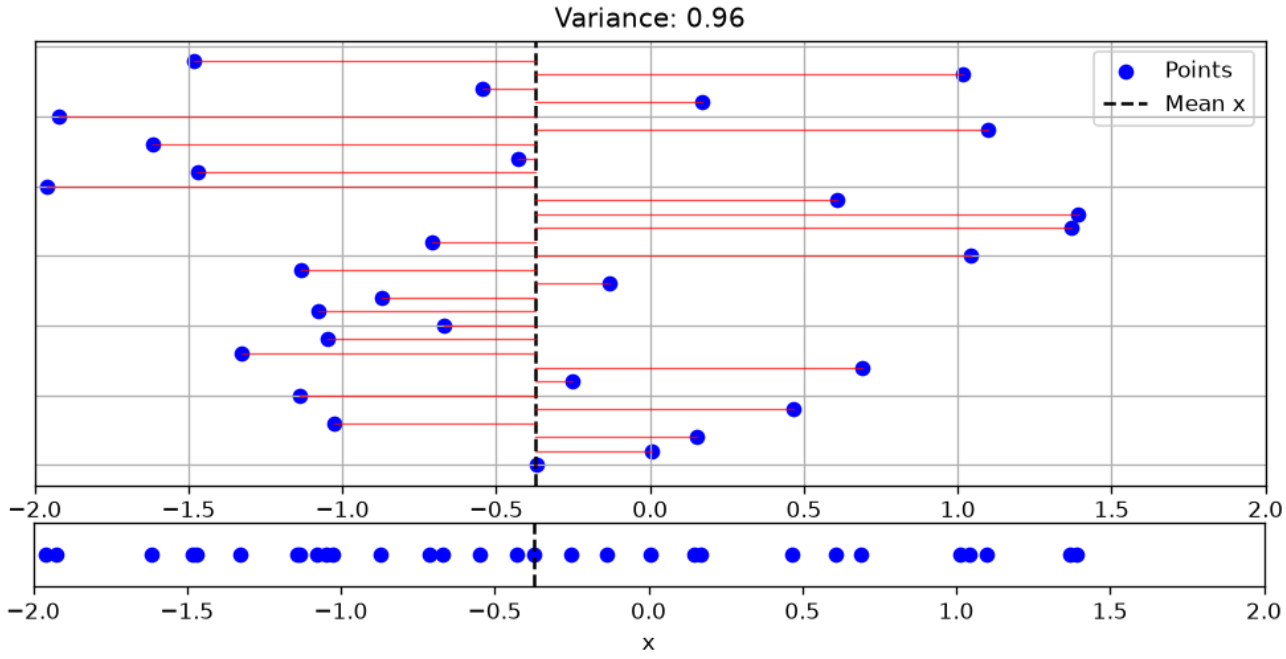
$$s_{n-1}^2 = \frac{\sum_{i=1}^{n}(x_i - \overline{x})^2}{n-1}$$

We will see it better later in the course, but for now consider that:

- The variance formula can be used when we have access to the whole population;
- The sample variance formula should be used in all other cases;
- When sample size is large enough, the two give very similar result (for large values of n: $n-1 \approx n$).

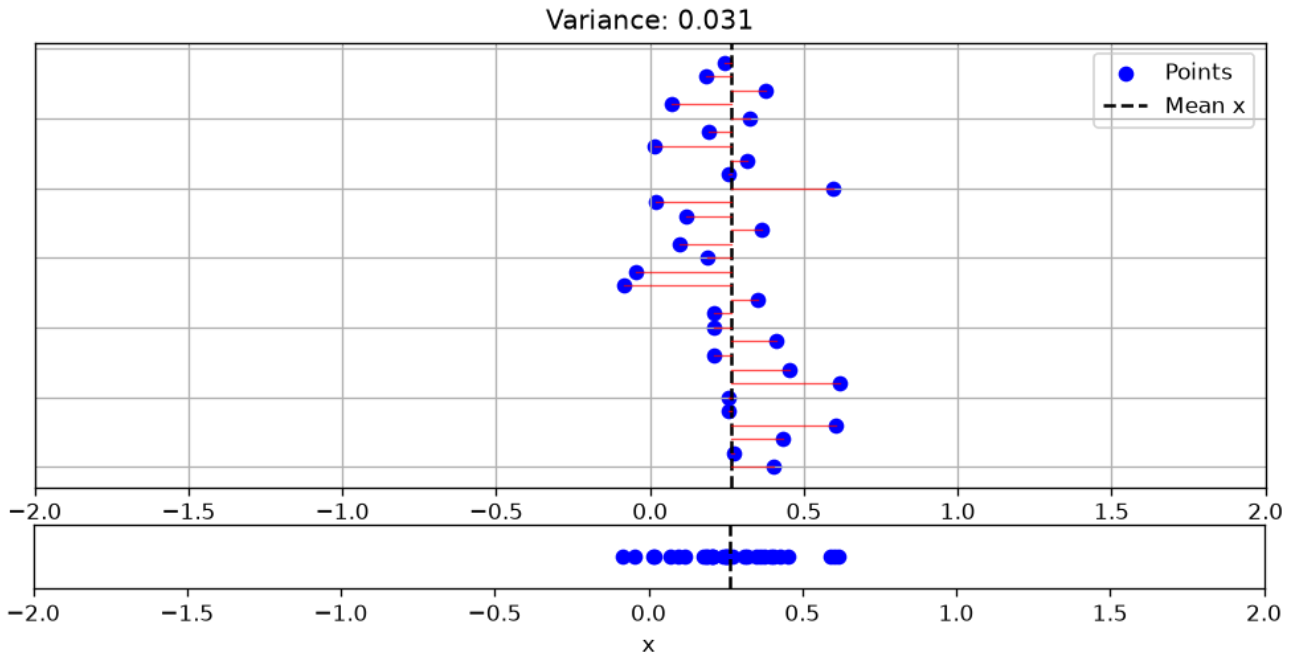
Here we use the symbol s instead of the symbol σ seen in the case of probability distributions, because s is a value "estimated" from the data and not "theoretical" as in the case of σ. It should be noted, however, that in practice, there is no universal convention, so in many cases σ is also used for estimated values.

We can attribute a geometric meaning to variance, as shown in the graph below:



In the graph, the bottom plot shows a univariate sample $\{x_i\}_i^N$. The top plot shows the same sample "exploded" on the y-axis for visualization purposes. In the top plot, the black dashed line indicates the sample mean, while the red lines are the terms $(x_i - \overline{x})$ that appear in the variance formula. The variance calculates the average of the lengths of these segments.

The following graph shows a less "dispersed" sample:



The variance of age will be:

```
age.var()
```

```
np.float64(211.01912474630802)
```

The dispersion indices seen so far (excluding variance) have the same **unit of measurement** as the input data. In the case of ages, the data are measured in years. It is therefore correct to say that **minimum**, **maximum**, **range**, **interquartile range**, and **mean absolute deviation** calculated on weights are measured in years.

The same does not apply to the variance, which will be measured in **years squared**. If we want to obtain a **commensurable** measure of dispersion, we can calculate the square root of the variance, thus obtaining the **standard deviation** (or **root mean square deviation**), which is defined as follows:

$$s_n = \sqrt{s_n^2} = \sqrt{\frac{\sum_{i=1}(x_i - \overline{x})^2}{n}}$$

Similarly, we often use the **sample standard deviation**:

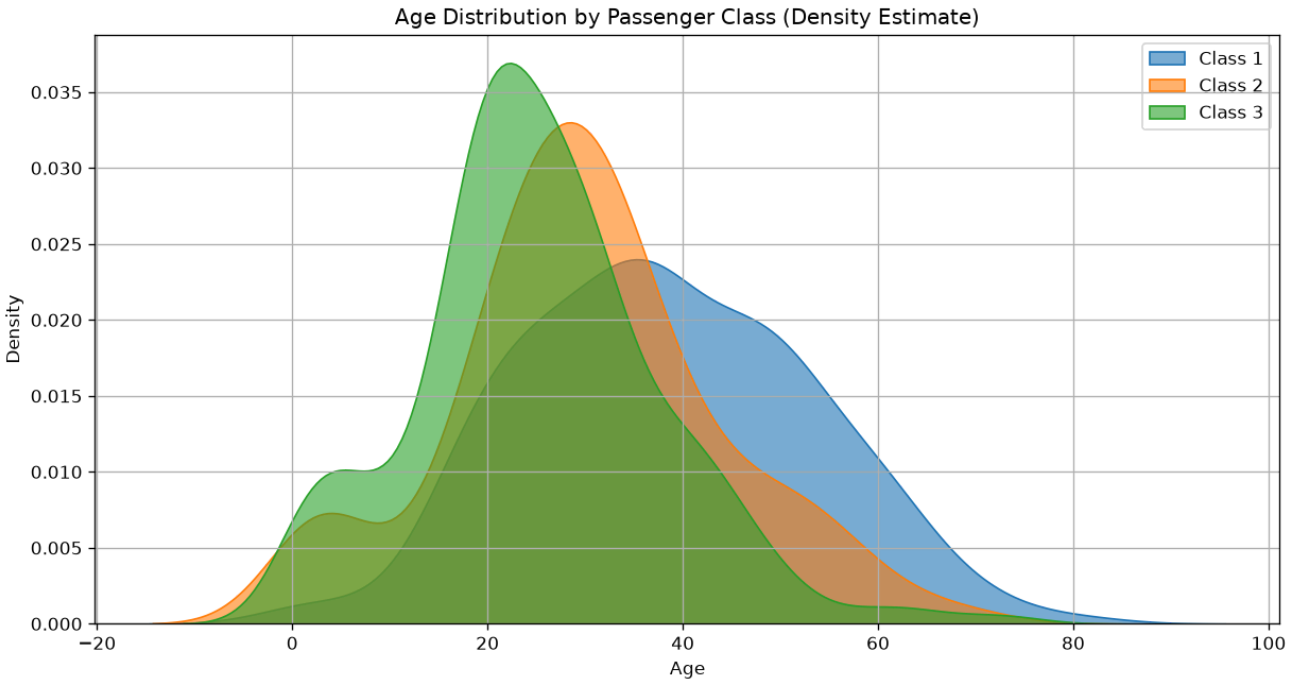
$$s_{n-1} = \sqrt{s_{n-1}^2} = \sqrt{\frac{\sum_{i=1}(x_i - \overline{x})^2}{n-1}}$$

We can compute the standard deviation as follows:

```
age.std()
```

```
np.float64(14.526497332334042)
```

Let's see how these measures of dispersion relate to distributions with different shapes. We will consider the ages of passengers in the three classes of the Titanic:




```
Class 1 ages:
Mean: 38.233440860215055
Min: 0.92
Max: 80.0
Range: 79.08
IQR: 22.0
Variance: 219.12454269107815
Std: 14.80285589645046

Class 2 ages:
Mean: 29.87763005780347
Min: 0.67
Max: 70.0
Range: 69.33
IQR: 13.0
Variance: 196.0301519088587
Std: 14.0010768124762

Class 3 ages:
Mean: 25.14061971830986
Min: 0.42
Max: 74.0
Range: 73.58
IQR: 14.0
Variance: 156.1349764510225
Std: 12.495398210982414
```

We can see a correspondence between the graphical forms of the distributions and the dispersion measures. Indeed, passengers in class 3 are “less dispersed” and “younger” than in the other classes.

Data Normalizations

The observed data dispersion indicators strongly depend on the nature of the data and their unit of measurement. For example, ages are measured in years, while weights are in Kg or pounds. Therefore, there are data normalization techniques that make data based on different units of measurement comparable to each other.

Normalization between 0 and 1

This normalization scales the data so that the minimum and maximum values are exactly equal to 0 and 1, using the following formula:

$$x_i^{norm} = (x_i - x_{min})/(x_{max} - x_{min})$$

In the case of weights:

```
weights_norm = (weights-weights.min())/(weights.max()-weights.min())
weights_norm
```

```
0       0.808081
1       0.429293
2       0.353535
3       0.378788
4       0.343434
          ...
4226    0.080808
4227    0.434343
4228    0.353535
4229    0.191919
4230    0.050505
Name: weight, Length: 4231, dtype: float64
```

Normalization between -1 and 1

In this case, the data are rescaled so that the new minimum and maximum are -1 and 1, using the following formula:

$$x_i^{norm} = (x_{max} + x_{min} - 2 \cdot x_i)/(x_{max} - x_{min})$$

We can perform this transformation in Pandas as follows:

```
weights_norm2 = (weights.max()+weights.min()-2*weights)/(weights.max()-weights.min())
weights_norm2
```

```
0      -0.616162
1       0.141414
2       0.292929
3       0.242424
4       0.313131
          ...
4226    0.838384
4227    0.131313
4228    0.292929
4229    0.616162
4230    0.898990
Name: weight, Length: 4231, dtype: float64
```

Standardization (z-scoring)

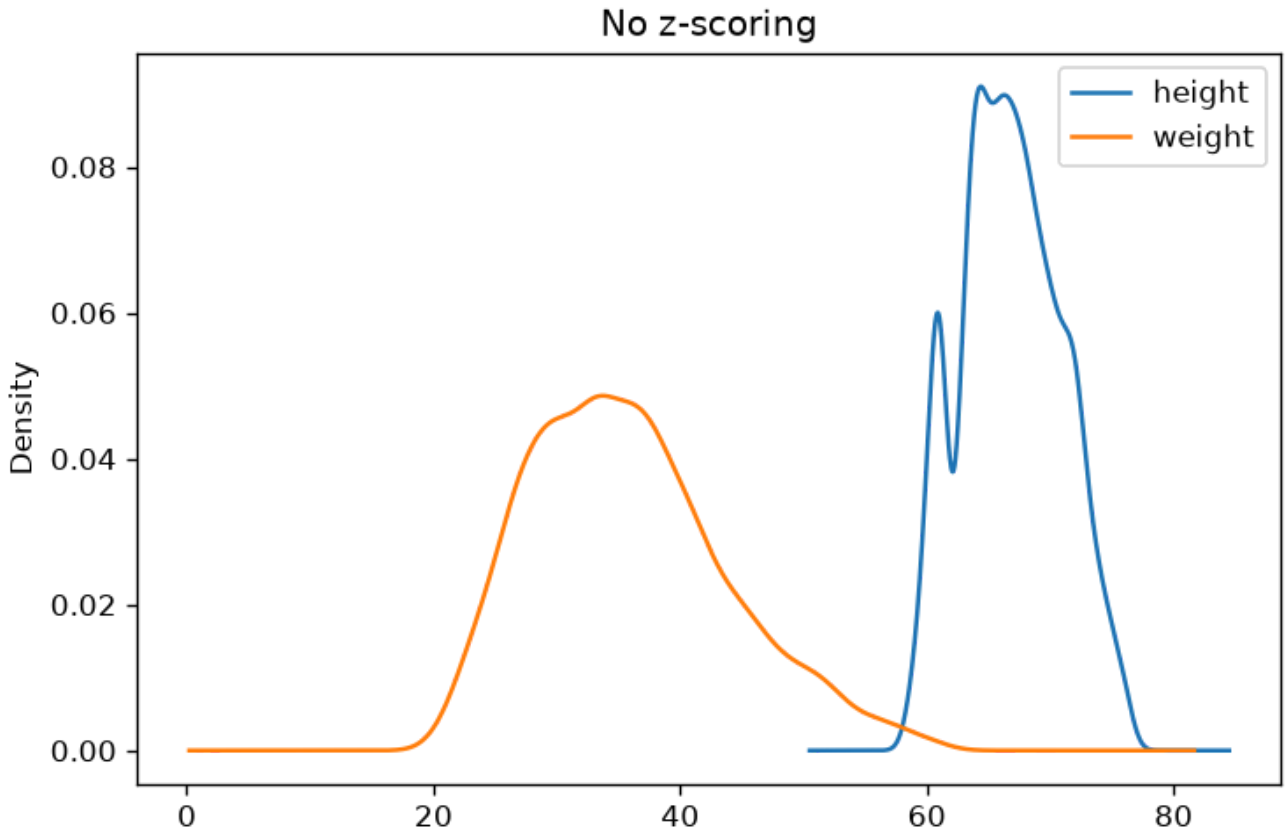
In many cases, it is useful to normalize data so that they have zero mean and unit standard deviation. This type of normalization is called "z-scoring" and is performed by subtracting the mean from the data and dividing by the standard deviation.

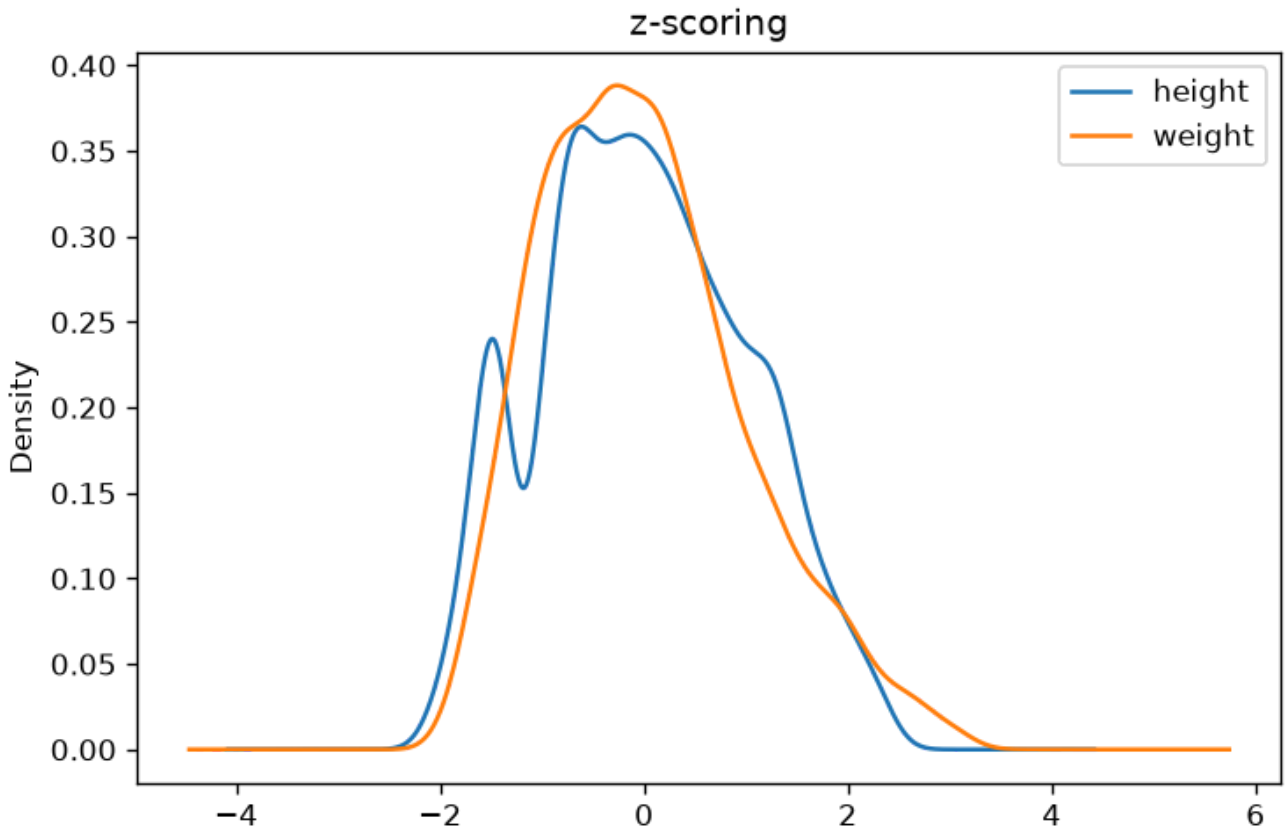
$$z_i = \frac{x_i - \overline{x}}{s_x}$$

where s_x is the standard deviation of the population to which X belongs. Note that zeta scores are **dimensionless** (i.e., they have no unit of measurement).

To understand the effect of this normalization, let's observe the density estimates of the samples before and after normalization:

```
from scipy.stats import zscore
hwz = hw.copy()
hwz['height'] = zscore(hwz['height'])
hwz['weight'] = zscore(hwz['weight'])

hw.plot.kde()
plt.title('No z-scoring')
plt.show()

hwz.plot.kde()
plt.title('z-scoring')
plt.show()
```





Shape Indicators

Let's now look at some indicators that allow us to get an idea of certain aspects of the "shape" of data distribution.

Asymmetry (skewness)

Skewness is an indicator of the "imbalance" to the left (negative value) or to the right (positive value) of a data sample with respect to the central value. The formula for skewness is as follows:

$$\sum_{i}^{n} \frac{(x_i - \overline{x})^3}{n \cdot s_x^3}$$

The skewness values will be:

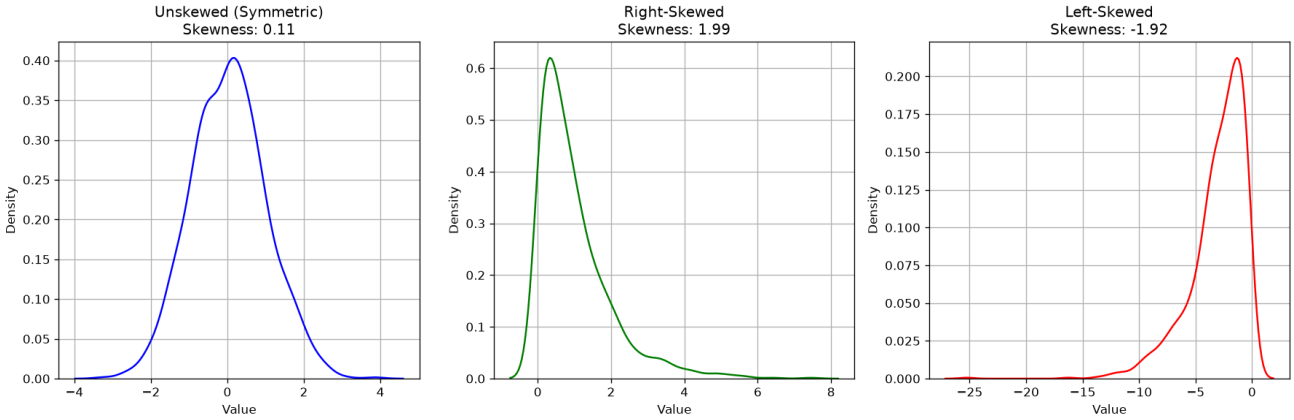
- **Negative** if the distribution is skewed to the left;
- **Positive** if the distribution is skewed to the right;
- **Close to zero** in the case of unskewed distributions.

We can compute the skewness as follows

```
age.skew()
```

```
np.float64(0.38910778230082704)
```

Let's look at some examples of distributions and skewness:



Kurtosis (kurtosis)

The kurtosis index measures the "thickness" of the tails of a density distribution. It is defined as follows:

$$K = \frac{1}{n} \sum_{i=1}^{n} \left(\frac{X_i - \bar{X}}{s} \right)^4 - 3$$

The index is interpreted as follows:

- If it is greater than zero, the distribution is *leptokurtic*, i.e., more "peaked" than a Normal distribution;
- If it is less than zero, the distribution is *platykurtic*, i.e., more "flat" than a Normal distribution;

- If it is equal to zero, the distribution is *mesokurtic*, i.e., the tails are similar to those of a normal distribution.

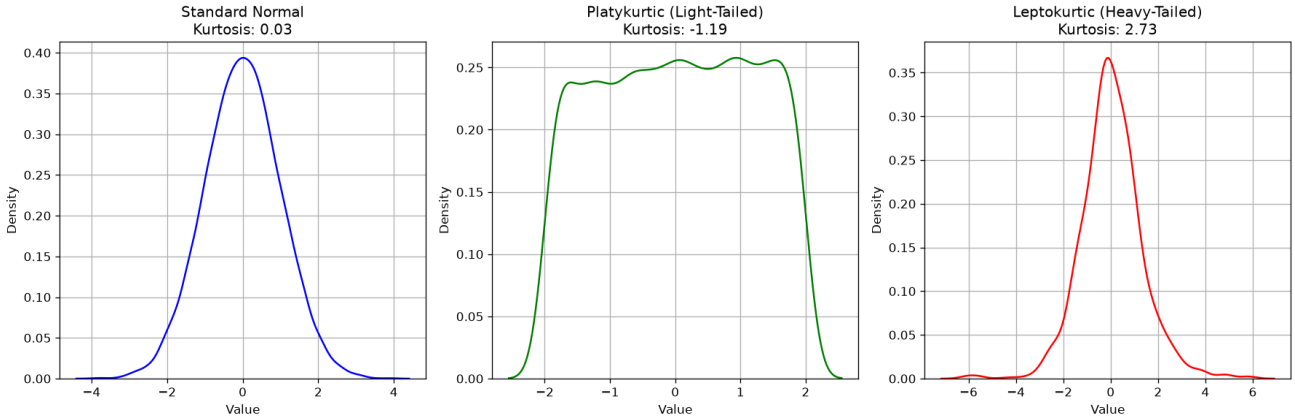
We will see what a normal distribution is later in the course.

We can compute the kurtosis as follows:

```
age.kurtosis()
```

```
np.float64(0.17827415364210353)
```

Let's look at some examples of Kurtosis values:



Statistical Summary

When we compute values like **mean**, **median**, **mode**, **minimum**, **maximum**, **range**, **standard deviation**, **variance**, and **interquartile range**, we're building what is known as a **statistical summary** of a dataset. These measures help us understand the **central tendency**, **spread**, and **shape** of a distribution—whether it's symmetric, skewed, concentrated, or dispersed.

In practice, we can obtain many of these summary statistics quickly using the `.describe()` method in **pandas**:

```
age.describe()
```

```
count    714.000000
mean      29.699118
std       14.526497
min        0.420000
25%       20.125000
50%       28.000000
75%       38.000000
max       80.000000
Name: Age, dtype: float64
```

This returns:

- **count**: number of non-missing values
- **mean**: average value
- **std**: standard deviation
- **min** and **max**: smallest and largest values
- **25%**, **50%**, **75%**: the first quartile (Q1), median (Q2), and third quartile (Q3)

If you want to include other measures like mode or variance, you can compute them separately:

```
print(f"Mode (most frequent age): {age.mode()[0]:.1f}")
print(f"Variance: {age.var():.2f}")
print(f"IQR (Interquartile Range): {age.quantile(0.75) - age.quantile(0.25):.2f}")
```

```
Mode (most frequent age): 24.0
Variance: 211.02
IQR (Interquartile Range): 17.88
```

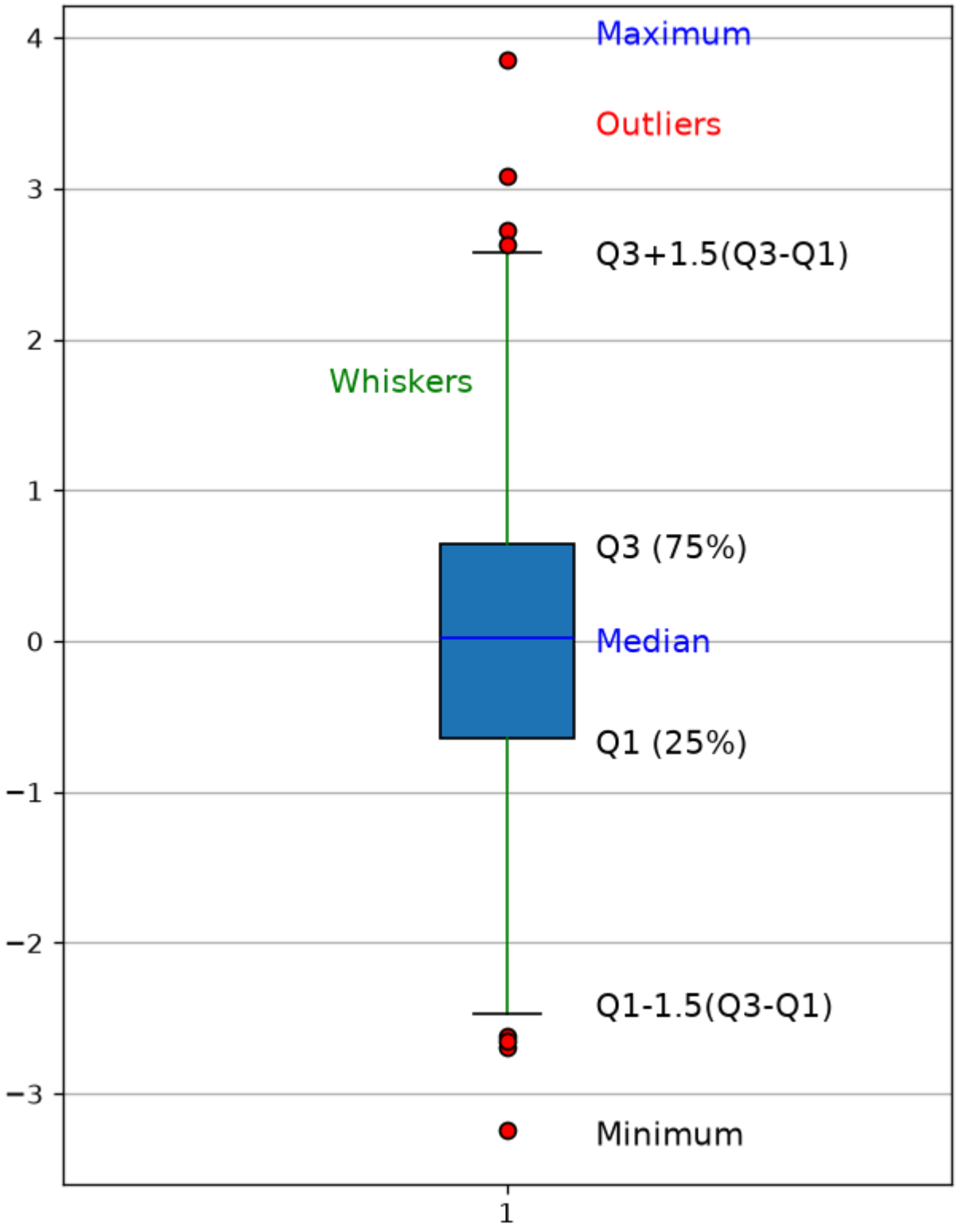
These tools are essential for both exploratory data analysis and communicating insights clearly—whether you're comparing groups, detecting outliers, or preparing data for modeling.

Boxplots

Boxplots are a compact visualization method for representing certain descriptive characteristics of the data under analysis. In particular, given a sample, a boxplot can effectively represent the following quantities:

- Median value;
- First and third quartile;
- Minimum and maximum (depending on the boxplot's "version," as discussed below).

A boxplot appears as follows:



A **boxplot** is a graphical summary of a dataset that displays its distribution using a rectangular “box” and two “whiskers.” Specifically:

- **Bottom edge of the box**: marks the first quartile (Q1), the 25th percentile of the data.
- **Top edge of the box**: marks the third quartile (Q3), the 75th percentile.
- **Horizontal line inside the box**: represents the median (Q2), the 50th percentile.
- **Lower whisker**: extends to the smallest data point that is greater than or equal to (Q1 - 1.5 (Q3 - Q1)).

- **Upper whisker**: extends to the largest data point that is less than or equal to (Q3 + 1.5 (Q3 - Q1)).
- **Individual points beyond the whiskers**: represent outliers—values that fall outside the range defined by the whiskers.

To illustrate the usefulness of boxplots, we will consider the dataset of weights and heights seen previously:

```
hw
```

	sex	height	weight
0	M	74	53.484771
1	M	70	38.056472
2	F	61	34.970812
3	M	68	35.999365
4	F	66	34.559390
...	...	...	...
4226	F	69	23.862436
4227	M	69	38.262182
4228	F	64	34.970812
4229	F	64	28.388071
4230	F	61	22.628172

4231 rows × 3 columns

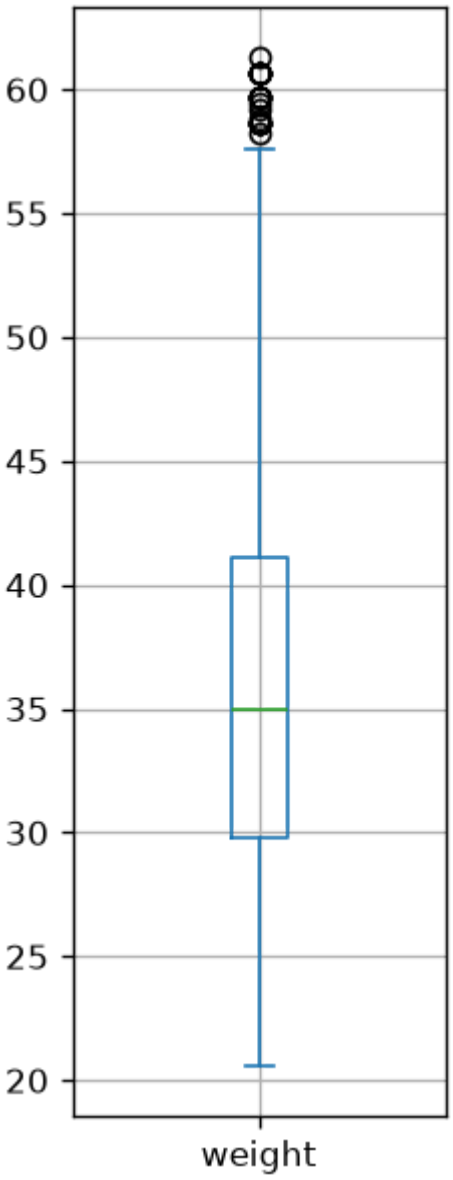
Let's choose the "weight" variable and show the **statistical summary**, that is, the list of all the descriptive statistics indicators discussed so far:

```
hw['weight'].describe()
```

```
count    4231.000000
mean       35.818062
std         7.987908
min        20.571066
25%        29.828045
50%        34.970812
75%        41.142132
max        61.301776
Name: weight, dtype: float64
```

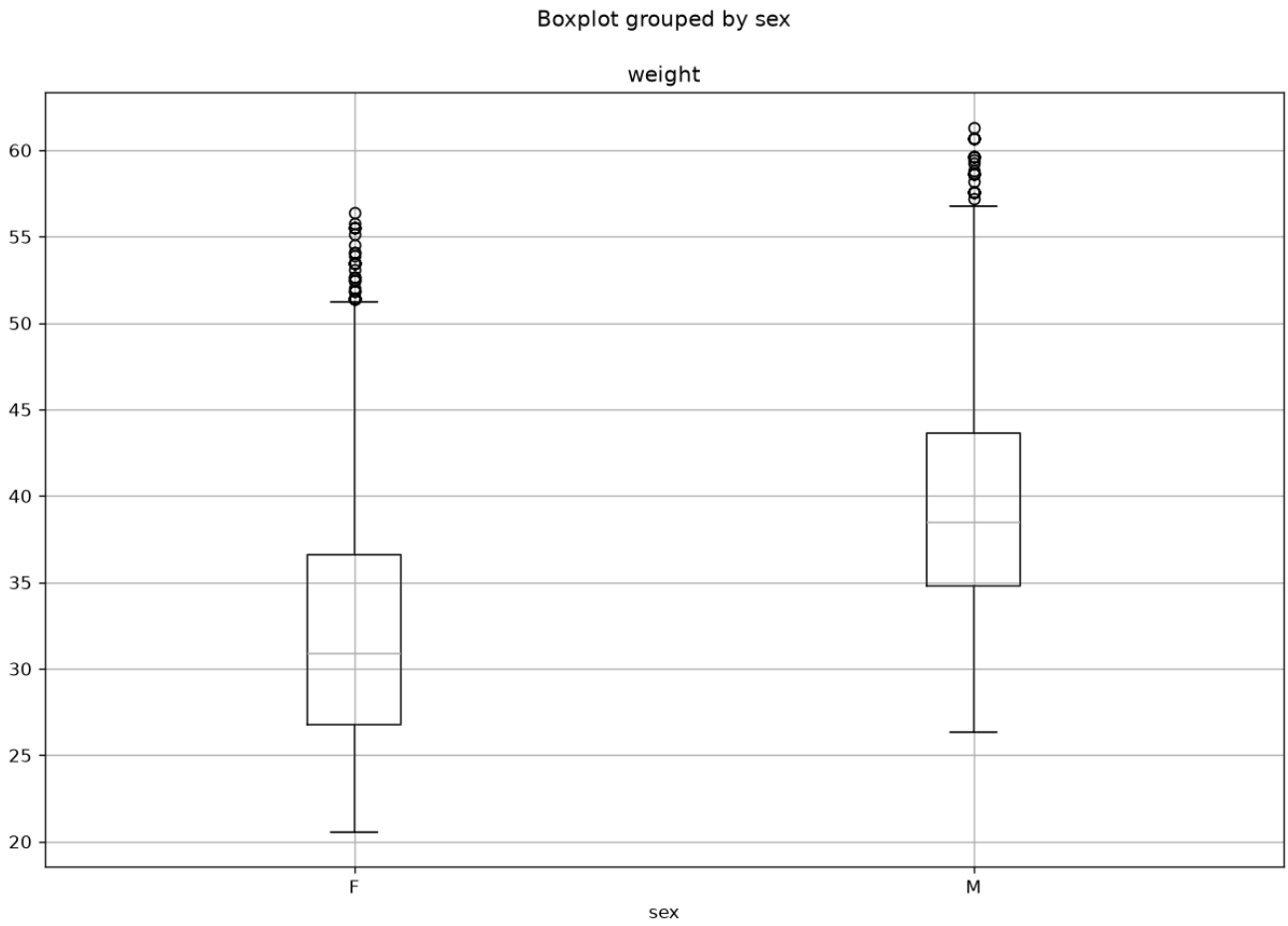
The boxplot of the weights is computed as follows:

```
from matplotlib import pyplot as plt
hw['weight'].plot.box(figsize=(2,6))
plt.grid()
plt.show()
```



Boxplots can be useful to compare samples. For example, the following boxplots compare the distributions of weights between men and women:

```
hw.boxplot(column='weight', by='sex', figsize=(12,8))
plt.show()
```



From the graph above, we can note that weight has different distributions in males and females, with males overall weighing more, as one would expect.

Exercises

Exercise 1

Considering the Titanic dataset, show the bar charts of absolute and relative frequencies of the values in the `Pclass` column. Randomly choosing among the passengers, what is the probability that they embarked in second class?

Exercise 2

Considering the Titanic dataset, show the histogram of the ages of passengers embarked in first class. Use an appropriate criterion to choose the number of bins.

Exercise 3

Modify the plot from the previous exercise to construct a histogram that answers the question "if I randomly choose a passenger boarded in first class, what is the probability that their age is between 20 and 30 years?"

Exercise 4

Considering the Titanic dataset, compare the cumulative age distributions of passengers embarked in the various classes. Are the distributions similar? In which class are the youngest subjects embarked?

Exercise 5

Considering the Titanic dataset, show with a stacked bar plot the percentages of passengers belonging to the three classes separately for the two sexes. Are there any differences in the distribution?

Exercise 6

Considering the Titanic dataset, calculate for each variable the count, mean, standard deviation, minimum, maximum, median, first quartile, and third quartile. After calculating the requested values individually, use the `describe` method to obtain these values. Which variable is the most dispersed?

Exercise 7

Considering the Titanic dataset, for each of the three classes, calculate the mean and variance of passenger ages. In which class are the ages least dispersed? Which class contains the youngest individuals? Complete the analysis with bar charts.

References

- Chapter 2 of: Heumann, Christian, and Michael Schomaker Shalabh. Introduction to statistics and data analysis. Springer International Publishing Switzerland, 2016.
- Chapter 3 of: Heumann, Christian, and Michael Schomaker Shalabh. Introduction to statistics and data analysis. Springer International Publishing Switzerland, 2016.

© 2026 Antonino Furnari · Department of Mathematics and Computer Science, University of Catania Built with Quarto

Credits and data sources

Probability for data analysis

Chapter resources

Slides (PDF) Open in Colab Practice quiz

When working with data, we often ask: **what values will variables assume?**

In some cases, we can predict these values with perfect accuracy—like solving a system of equations that describes the speed of an object using Newtonian physics. These are **deterministic systems**, where outcomes follow directly from initial conditions.

But in many real-world scenarios, we face **uncertainty**. Modeling the relationship between variables in a deterministic way is not always possible. This uncertainty can arise from several sources:

- **Inherent randomness**: Some events are truly stochastic, like rolling a die or drawing a card. Even with perfect knowledge, the outcome is unpredictable.
- **Incomplete observability**: Sometimes we can't see everything. In the Monty Hall problem, the game is deterministic, but the player lacks full information—so the outcome feels uncertain.
- **Incomplete modeling**: Even in deterministic systems, we may lack the tools to model them fully. For example, a robot with only an RGB camera may estimate object positions in 2D, but cannot reconstruct full 3D coordinates. The missing dimensions introduce uncertainty.

Real-World Examples

- **Tossing a coin or rolling a die**: These are often treated as random events, even if they could be modeled deterministically in theory. The complexity of initial conditions makes prediction intractable.
- **Diagnosing a medical condition**: Different pathologies may share symptoms. Even with detailed observations, we may not be able to determine the exact diagnosis with certainty.

Why Probability Theory Matters

Probability theory provides a **consistent framework** for reasoning about uncertainty. It allows us to:

- Quantify uncertain events
- Combine known and unknown information
- Derive new probabilistic statements using formal rules

In data analysis, probability is essential for:

- Modeling randomness and variability
- Making predictions under uncertainty

- Interpreting patterns and relationships in data

In this lecture, we'll revisit the core concepts of probability theory—starting with random variables—and apply them to real datasets like the Titanic. We'll use tools like the **sum rule**, **product rule**, and **Bayes' theorem** to answer questions such as:

- What is the probability that a passenger survived?
- What is the probability that a passenger was in 1st class, given that they survived?
- What is the probability that a passenger was in 3rd class, given that they died?

These examples will help us connect abstract probability rules to concrete data-driven reasoning.

Random Experiments

In practice, when data acquisition is affected by uncertainty, we refer to the process as a **random experiment**. Informally, we define a random experiment as:

> An experiment that can be repeated any number of times, potentially leading to different outcomes.

We introduce the following terminology:

- **Sample space** $\Omega = \{\omega_1, \dots, \omega_k\}$ The set of all possible outcomes of the experiment.
- **Simple event** ω_i: a single possible outcome.
- **Event** $A \subseteq \Omega$: a subset of the sample space representing a condition or outcome of interest. We usually denote $\overline{A} = \Omega \setminus A$ as the complementary event to A, i.e., the event that A does not occur.

By definition:

- The sample space Ω is called the **sure event** or **certain event**, since it includes all possible outcomes.
- The empty set $\emptyset$ is called the **impossible event**, as it contains no outcomes.

Example: Rolling a Die

Consider the random experiment of rolling a fair six-sided die. The outcome is the number shown on the top face when the die lands.

- **Sample space**: $\Omega = \{1, 2, 3, 4, 5, 6\}$
- **Simple event**: $\omega_1 = 1$ represents the outcome "we roll a 1".
- **Event examples**:
  - "We obtain a 1": $A = \{1\}$
  - "We obtain an even number": $A = \{2, 4, 6\}$
  - Complement of the above: $\overline{A} = \{1, 3, 5\}$ Represents "we obtain an odd number".

This framework allows us to describe uncertainty in a structured way, preparing us to assign probabilities to events and reason about their likelihood.

Random Variables

We have so far talked about "statistical variables". When dealing with uncertain events, we need to use the concept of 'random variables'. Informally (from wikipedia):

> A random variable is a variable whose values depend on outcomes of a random phenomenon.

A random variable is characterized by a set of possible values often called *sample space*, *probability space*, or *alphabet* (this last term comes from information theory, where we often deal with sources emitting symbols from an alphabet, in which case the values of X will be the symbols).

The definition of a random variable is very similar to that of a statistical variable. Formally, if Ω is the *sample space*, we will define a random variable as a function:

$$X : \Omega \rightarrow E$$

Where **E is a measurable space** and often $E = \mathbb{R}$. This definition is similar to the one of statistical variable we have given before.

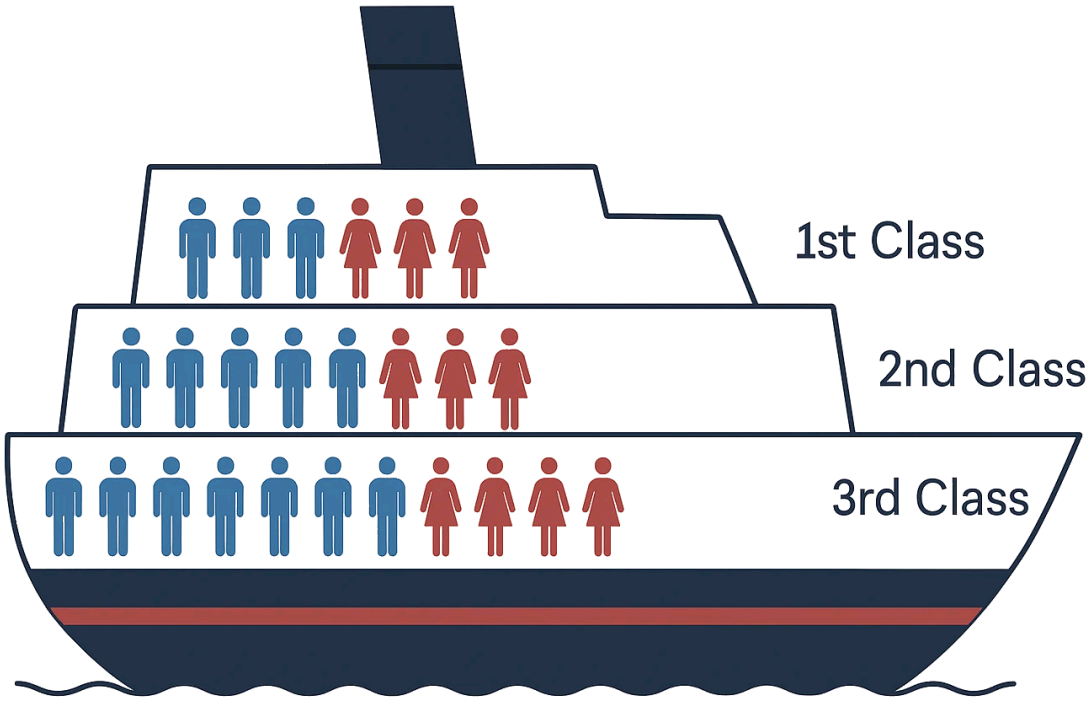
A random variable is generally denoted by a *capital letter*, such as X.

Similar to statistical variables, random variables can be *discrete* or *continuous*, *scalar* or *multi-dimensional*.

The following table lists some examples.

	Discrete	**Continuous**
Scalar	Tossing a coin	Height of a person
Multidimensional	Pair of dice	Coordinates of a car

Toy Titanic Example



We will now consider a toy example derived inspired to the Titanic dataset.

In this example, we will assume to have a very small ship with passengers in three classes and belonging to two possible genders (male or female).

To reason in terms of probability, we have to think of a random experiment first. To this aim, let's imagine the process of randomly picking a passenger from the list. This passenger will belong to one of the three classes and one of the two sexes. We will see how to use probability theory to answer different questions such as:

- Is it more likely to pick a woman or a man?
- If I pick a person from first class, is it more likely to pick a woman or a man?
- If I know I picked a woman, is it more likely that I am picking from first, second or third class?

As you can see, answering these questions allows us to better understand and characterize the data, but can also be useful for practical applications. For instance, we may deliver different services to the different classes depending on the people who populate the classes.

More technically, we will consider the experiment of randomly drawing a passenger from one of the three classes. This happens in two stages:

- We first randomly pick one of the three classes;
- Then we randomly pick one of the passengers in the class;

- After observing the sex of the passenger, we replace them in the same class.

The outcome of the experiment can be characterized by *two random variables*:

- C represents the class, which can take values $1, 2, 3$
- S represents the sex of the passenger, which can take values $male$ and $female$

If we pick a male passenger marble from the third class, then the outcome of the experiment can be characterized by the values $C = 3$, $S = male$;

Working Definition of Data

We will define "data" as follows:

> The values assumed by a random variable

Example

- For instance, if the sex of a passenger is $male$, then $S = male$ *is data*;
- It should be clear that the 'data' is *the pair* <random variable, value> and not just the value. Indeed $male$ alone would not be very useful (we don't know which phenomenon it is related to), whereas $S = male$ can be useful, as we know that S is the random variable describing the sex of the passenger;
- In this example, the data $S = male$ is representing a fact: 'I randomly picked a passenger and their sex was $male$'. This is also called 'an event';

Probability

Since random variables are related to stochastic phenomena, we cannot say much about the outcome of a single phenomenon.

However, we expect to be able to characterize the class of experiments related to a random variable, to infer rules on what values the random variable is likely to take.

For instance, in the case of coin tossing, we can observe that, if I toss a coin for a large number of times, the number of heads will be roughly similar to the number of tails.

This kind of observations is useful, as it can give us a prior on what values we are likely to encounter and what are not.

To formally express such rules, we can define the concept of probability on a random variable.

Specifically, it is possible to assign a probability value to a given outcome. This is generally represented with a capital P:

- For instance, $P(C = 1)$ represents the probability of picking a from the first class in the previous example;
- A probability $P(C = 1)$ is a number comprised between 0 and 1 which quantifies how likely we believe the event to be;
  - 0 means impossible;
  - 1 means certain;

When it is clear from the context which variable we are referring to, the probability can also be expressed simply as:

$$P(male) = P(S = male)$$

Probability Axioms

Kolmogorov in 1933 provided three axioms which define the "main rules" that probability should follow:

Axiom 1 Any event has a probability in the range $[0, 1]$:

$$0 \leq P(A) \leq 1, \forall A \subseteq \Omega$$

Axiom 2 The certain event has probability 1:

$$P(\Omega) = 1$$

Axiom 3 If A and B are disjoint event (i.e., $A \cap B = \emptyset$), then:

$$P(A \cup B) = P(A) + P(B)$$

From these axioms, the following corollaries follow:

Complement rule

$$P(\overline{A}) = 1 - P(A)$$

Probability of the impossible event

$$P(\emptyset) = P(\overline{\Omega}) = 1 - P(\Omega) = 0$$

General addition rule (for any events A and B)

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Monotonicity (if $A \subseteq B$)

$$A \subseteq B \Rightarrow P(A) \leq P(B)$$

Laplace Probability

When all outcomes in a random experiment are considered equally probable, this is called a **Laplace experiment**. In this case, we can calculate the probability of a given event A as the ratio between the favorable outcomes and the possible outcomes:

$$P(A) = \frac{|A|}{|\Omega|} = \frac{\text{number of favorable outcomes}}{\text{number of possible outcomes}}$$

For instance, if we are tossing a die:

- The probability of obtaining any of the faces will be: $\frac{1}{6}$;
- The probability of obtaining an even number will be: $\frac{|\{2,4,6\}|}{|\{1,2,3,4,5,6\}|} = \frac{3}{6} = \frac{1}{2}$.

Estimating probabilities from observations

What about the cases in which we cannot make any assumption on the equal probability of events? In those cases we would like to estimate probabilities from observations. There are two main approaches to do so: frequentist and Bayesian.

Frequentist approach

Probability theory was initially developed to analyze the frequency of events. For instance, it can be used to study events like drawing a certain hand of cards in a poker game. These events are repeatable and can be dealt with using frequencies. In this sense, when we say that an event has probability p of occurring, it means that if we repeat the experiment infinitely many times, then a proportion p of the repetitions would result in that outcome.

According to the frequentist approach, we can estimate probabilities by repeating an experiment for a large number of times and then computing:

- The number of trials: how many times we performed the experiment;
- The number of favorable outcomes: how many times the outcome of the experiment was favorable.

The probability is hence obtained by dividing the number of favorable outcomes by the number of trials.

For instance, let's suppose we want to estimate the probability of obtaining a 'head' by tossing a coin. Let's suppose we toss the coin 1000 times and obtain 499 heads and 501 tails. We can compute the probability of obtaining head as follows:

- Number of trials: 1000;
- Number of favorable outcomes: 499.

The probability of obtaining head will be 499/1000=0.49

This is the approach we have seen so far in the course when dealing with relative frequencies.

Examples

- The probability of obtaining ‘head’ when tossing a coin is 0.5. We know that because, if we toss a coin for a large number of times, about half of the times, we will obtain ‘head’;
- The probability of picking a red ball from a box with 40 red balls and 60 blue balls is 0.4. We know this because, if we repeat the experiment for a large number of times, we will observe that proportion.

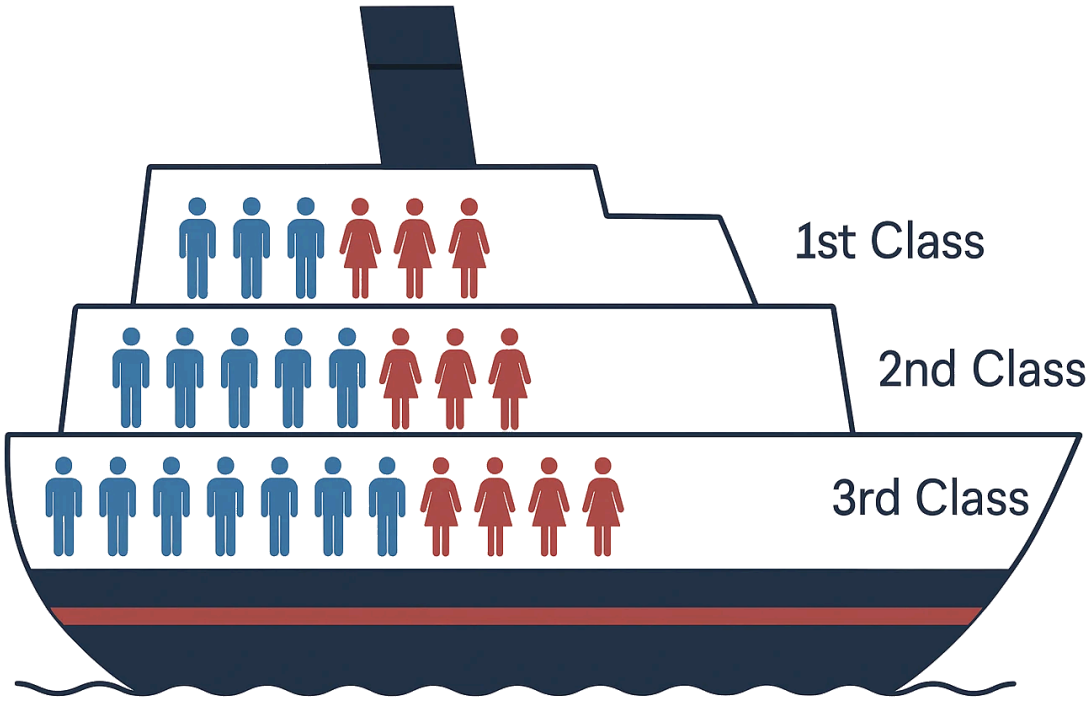
Bayesian Approach

The Bayesian approach to probability offers a different perspective on probability theory. Unlike the frequentist approach, which focuses on analyzing the frequency of events based on observations, the **Bayesian approach allows us to incorporate prior knowledge and update our beliefs as new information becomes available**.

In Bayesian probability, we view **probability as a measure of uncertainty or belief**. When we assign a probability to an event, it reflects our **subjective degree of belief in the event’s likelihood**. This approach is particularly useful when dealing with unique or one-time events where frequency-based analysis may not be applicable (e.g., “**what is the probability that the sun will extinguish in 5 billion years?**”).

We will discuss better the Bayesian approach when we’ll discuss Bayes theorem.

Example of Probability



Let's consider the previous example of randomly picking passengers.

Suppose we repeat this experiment for 100 times and observe that:

- We pick the first class 24 times;
- We pick the second class 32 times;
- We pick the third class 44 times;
- Once we selected a urn, we are equally likely to select any of the passengers contained in it, but we know that sexes are not distributed evenly in the classes (see figure).

Using a frequentist approach, we can define the probabilities:

- $P(C=1) = \frac{24}{100}$
- $P(C=2) = \frac{32}{100}$
- $P(C=3) = \frac{44}{100}$

This is done by using the formula:

- $P(X=x) = \frac{\#\ of\ times\ X=x}{\#\ trials}$

Computation of frequentist probability values

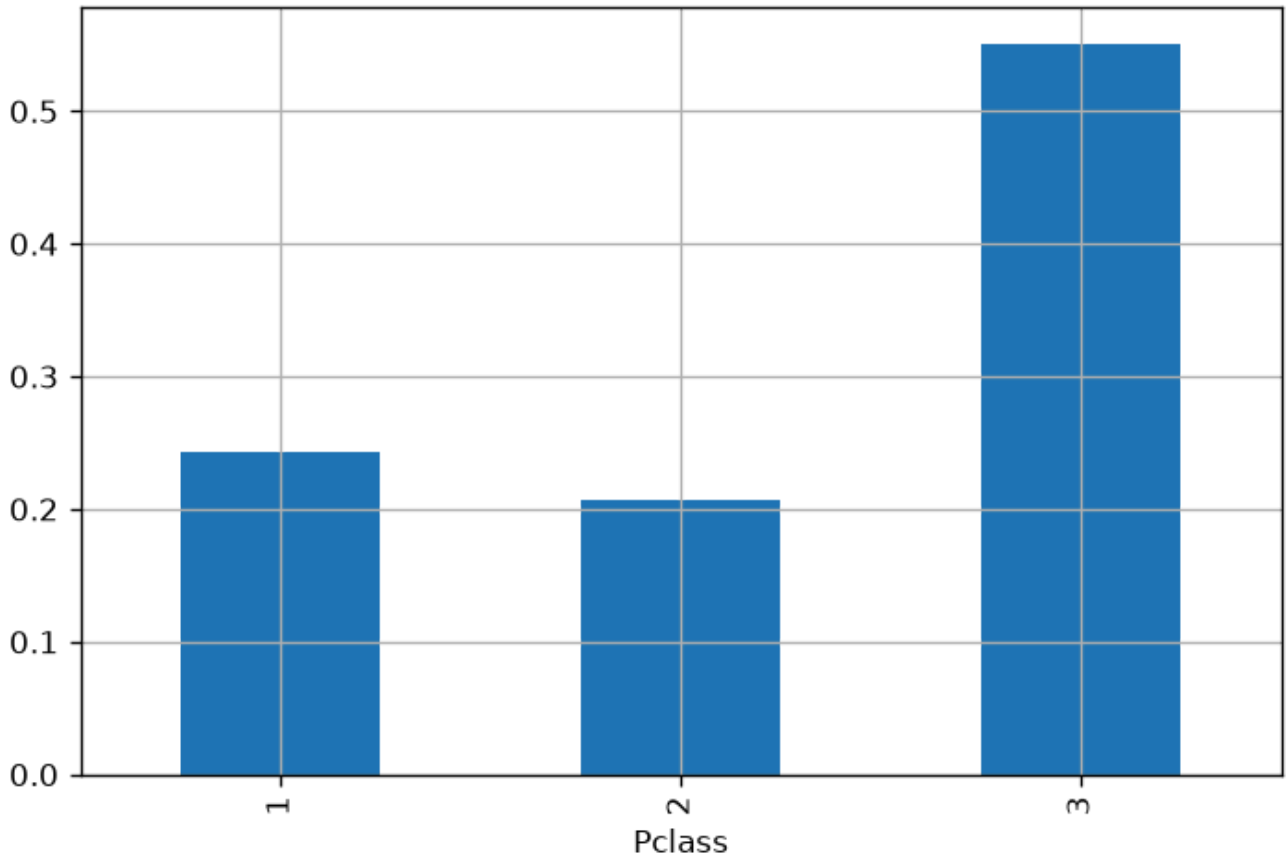
In frequentist terms, probabilities are the same as the relative frequencies of chapter 2. Let's see how to compute such probabilities on the real Titanic dataset:

```
import pandas as pd
titanic = pd.read_csv(DATA + "titanic.csv",
                      index_col='PassengerId')
titanic['Pclass'].value_counts(normalize=True).sort_index()  # relative frequencies
```

```
Pclass
1    0.242424
2    0.206510
3    0.551066
Name: proportion, dtype: float64
```

We can also visualize them for more clarity:

```
from matplotlib import pyplot as plt
titanic['Pclass'].value_counts(normalize=True).sort_index().plot.bar()  # relative
        frequencies
plt.grid()
plt.show()
```



Joint probability

We can define **univariate** (= with respect to only one variable) probabilities $P(C)$ as we have seen in the previous examples.

However, in many cases, it is useful to define probabilities on more than one variable at the time. For instance, we could be interested in studying the probability of picking a given fruit from a given box. In this case, we would be interested in the **joint probability** $P(C, S)$.

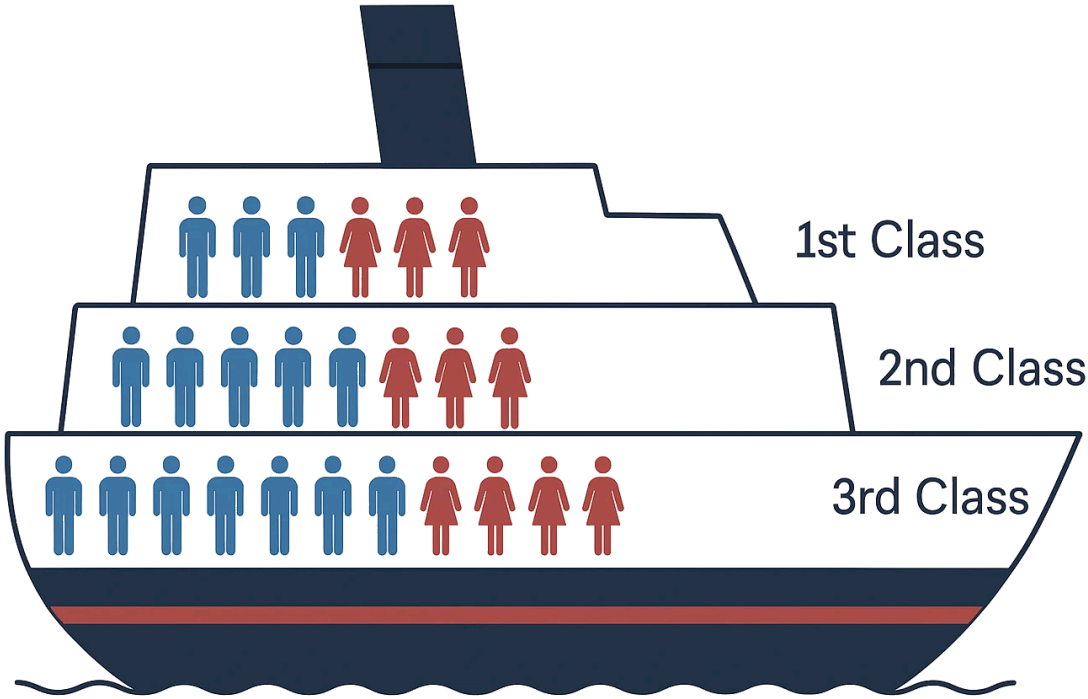
In general, we can have joint probabilities with arbitrary numbers of variable. For instance, $P\left(X_1, X_2, X_3, \ldots, X_n\right)$.

Joint probabilities are symmetric, i.e., $P(X, Y) = P(Y, X)$.

We should note that, when dealing with multiple unidimensional variables, we can always define a new multi-variate variable comprising all of them:

- $X = [X_1, X_2]$;
- $P(X) = P\left(X_1, X_2\right)$.

Example Joint Probability



We can see the concept of joint probability in the context of the Titanic example.

We have seen how to define the univariate probability $P(C)$ over the whole probability space of C.

However, we could be interested in the probability of both variables jointly: $P(C, S)$, i.e., the joint probability of C and S.

To 'measure' the joint probability, we could repeat the experiment many times and observe the outcomes.

We can then build a table which keeps track of how many times we observed a given combination:

	First Class	Second Class	Third Class	All
male	3	5	7	15
female	3	3	4	10
All	6	8	11	25

A table like this is called a "**contincency table**".

From the table above, we can easily derive the joint probability of a given pair of values using the **frequentist approach**. For instance:

$$P(C=1, S=male) = \frac{\#\ times\ (1, male)\ occurs}{\#\ trials} = \frac{3}{25}$$

Similarly, we can derive other values:

- $P(C=1, S=female) = \frac{3}{25}$
- $P(C=2, S=male) = \frac{5}{25}$
- $P(C=2, S=female) = \frac{3}{25}$
- $P(C=3, S=male) = \frac{7}{25}$
- $P(C=3, S=female) = \frac{4}{25}$

Note that we can also obtain the univariate probabilities by using the values in the "sum" row and column. For instance:

$$P(C=1) = \frac{\#\ times\ C=1\ occurred}{\#\ trials} = \frac{3+3}{25} = \frac{6}{25}$$

Similarly:

- $P(S=male) = \frac{15}{25}$
- $P(S=female) = \frac{10}{25}$
- $P(C=2) = \frac{8}{25}$
- $P(C=3) = \frac{11}{25}$

These univariate probabilities computed starting from joint probabilities are usually called **marginal probabilities**, because we are using the sums in the margin of the table.

We can obtain a joint probability table by dividing the table by the total number of trials (25):

	First Class	Second Class	Third Class	All
male	3/25	5/25	7/25	15/25
female	3/25	3/25	4/25	10/25
All	6/25	8/25	11/25	25/25

Alternatively, with decimal values:

	First Class	Second Class	Third Class	All
male	0.12	0.2	0.28	0.6
female	0.12	0.12	0.16	0.8
All	0.24	0.32	0.44	1

Computation of a Joint Probability Table

To compute a joint probability table from data, we should first compute a contingency table, which is also called a crosstab in Pandas:

```
pd.crosstab(titanic['Sex'], titanic['Pclass'])
```

Pclass	1	2	3
Sex			
female	94	76	144
male	122	108	347

The table above indicates, for example, that 94 passengers in class 1 were female. By specifying `margins=True`, we can obtain row and column sums, which will indicate the absolute frequencies of each variable:

```
pd.crosstab(titanic['Sex'], titanic['Pclass'],margins=True)
```

Pclass	1	2	3	All
Sex				
female	94	76	144	314
male	122	108	347	577
All	216	184	491	891

We can obtain the joint relative frequency table by passing `normalize=True` to the crosstab function:

```
pd.crosstab(titanic['Sex'], titanic['Pclass'], normalize=True, margins=True)
```

Pclass	1	2	3	All
Sex				
female	0.105499	0.085297	0.161616	0.352413
male	0.136925	0.121212	0.389450	0.647587
All	0.242424	0.206510	0.551066	1.000000

It should be noted that the marginal relative frequencies obtained are identical to the relative frequencies of the univariate samples:

```
titanic['Sex'].value_counts(normalize=True)
```

```
Sex
male      0.647587
female    0.352413
Name: proportion, dtype: float64
```

That are exactly the values in the last column of the previous table. Similarly, the univariate probabilities of Pclass are the same as the last row of the previous table:

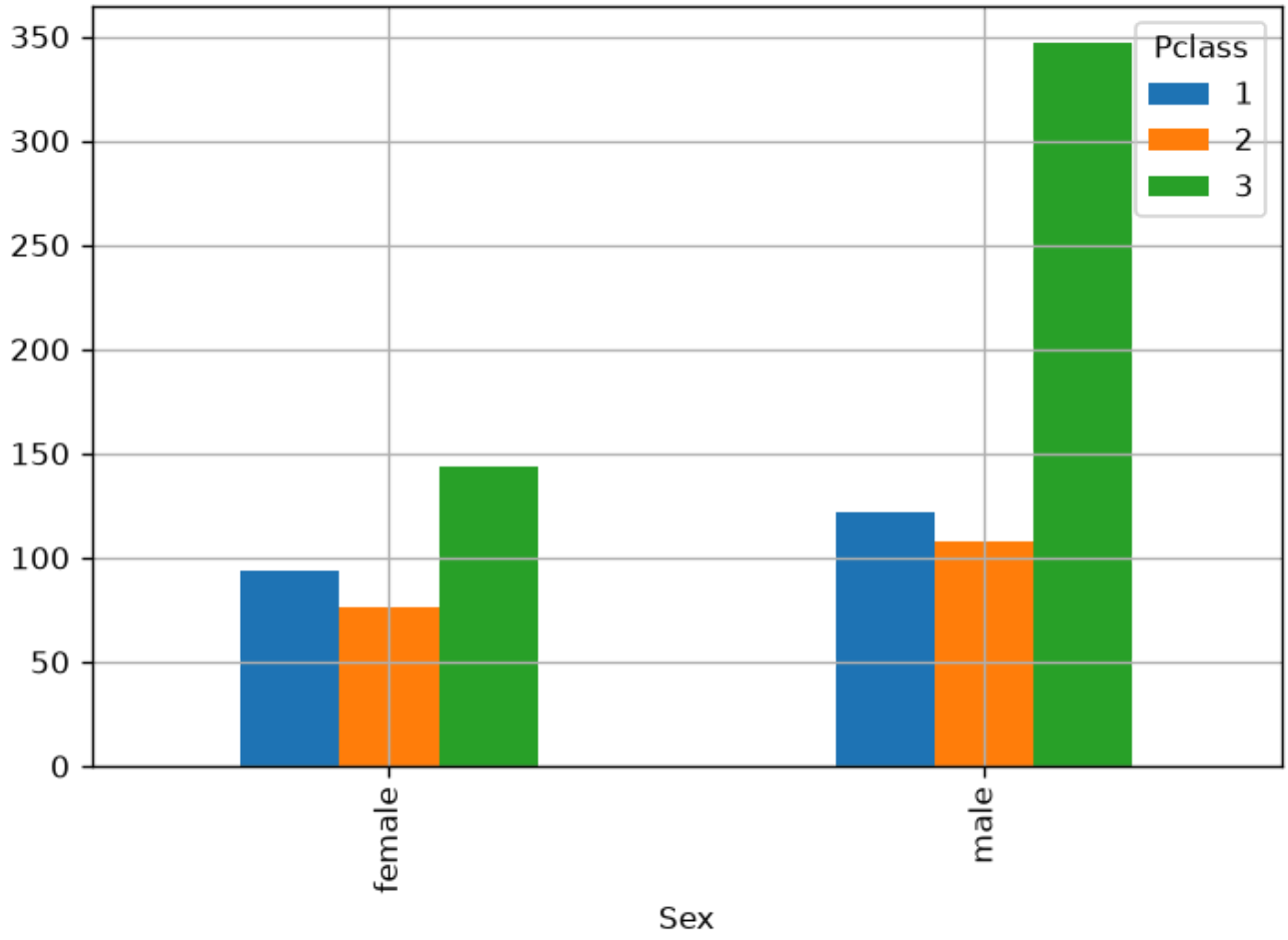
```
titanic['Pclass'].value_counts(normalize=True)
```

```
Pclass
3    0.551066
1    0.242424
2    0.206510
Name: proportion, dtype: float64
```

Graphical representations

We can use contingency tables and joint probability tables to obtain graphical representations. For example:

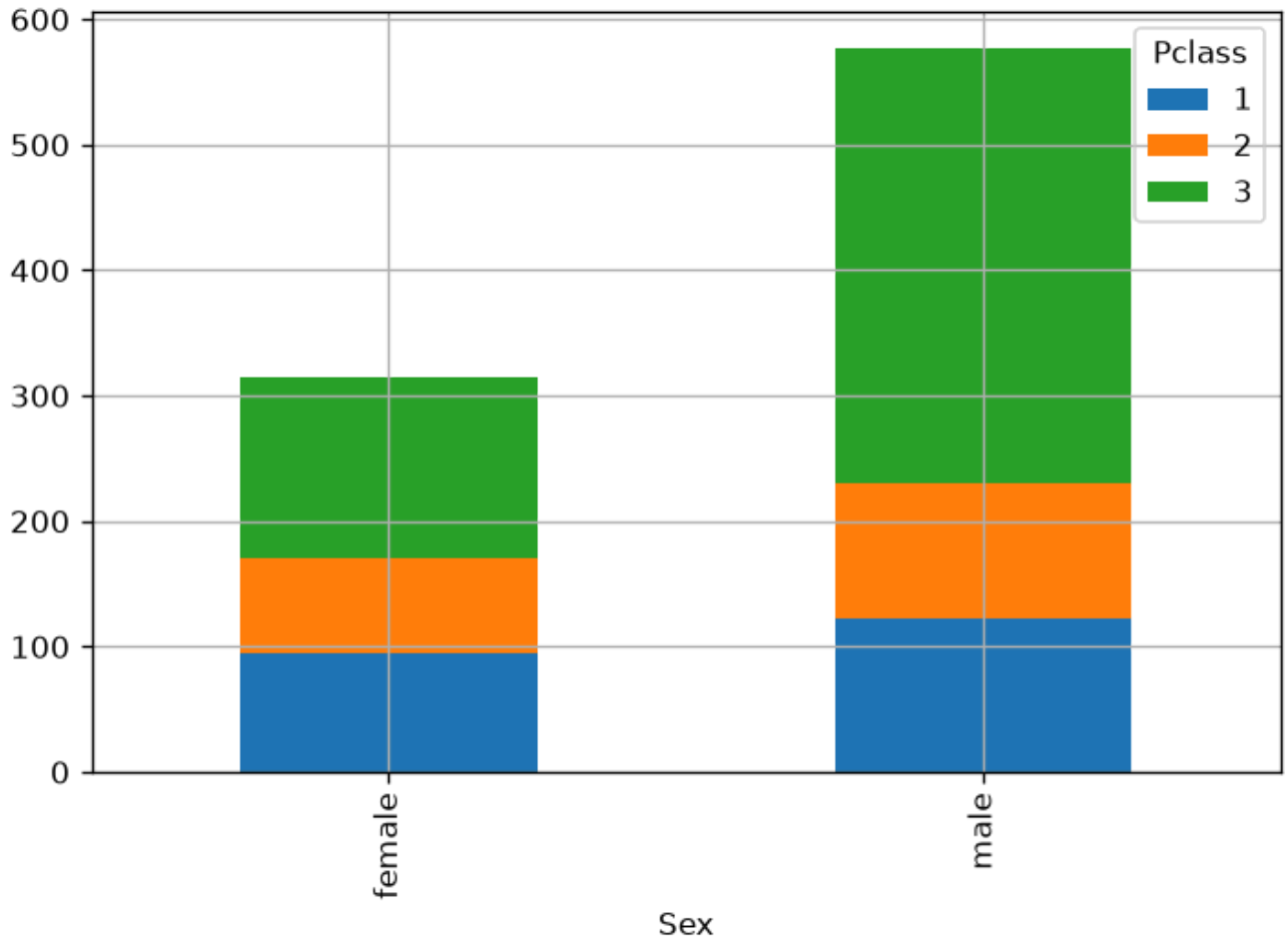
```
pd.crosstab(titanic['Sex'], titanic['Pclass']).plot.bar()
plt.grid()
plt.show()
```



The diagram above shows the distributions of people in the two classes, grouped by the two different sexes.

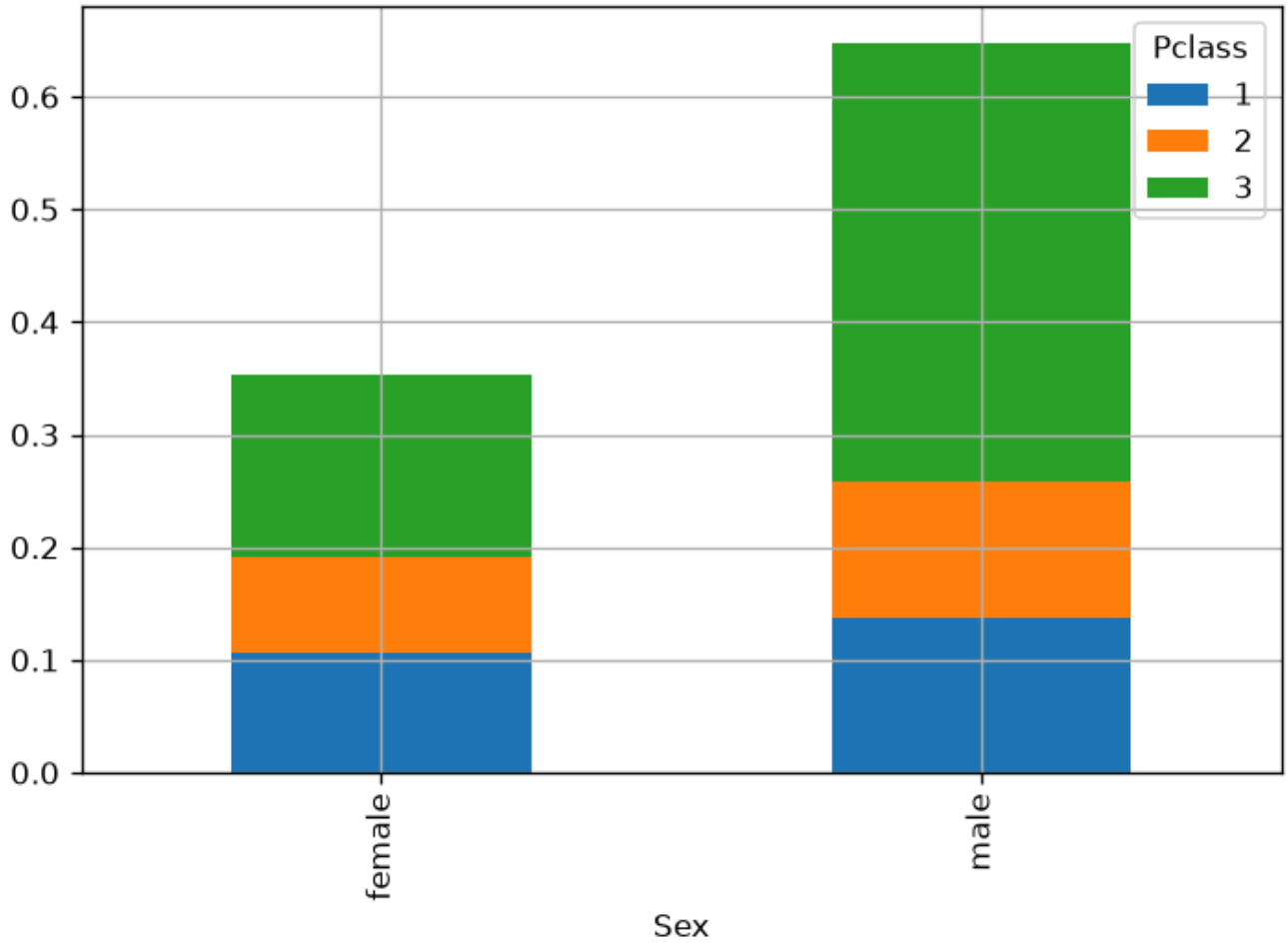
From this, we note that many passengers are men and in third class. In these cases, a stacked plot can sometimes be useful:

```
pd.crosstab(titanic['Sex'], titanic['Pclass']).plot.bar(stacked=True)
plt.grid()
plt.show()
```



We can reason in terms of joint probabilities by normalizing the counts:

```
pd.crosstab(titanic['Sex'], titanic['Pclass'], normalize=True).plot.bar(stacked=True)
plt.grid()
plt.show()
```



Sum Rule (Marginal Probability)

In the previous example, we have seen how we can compute **marginal (univariate) probabilities** from the contingency table. This is possible because the contingency table contains information on how the different possible outcomes distribute over the sample space.

In general, we can compute marginal probabilities form joint probabilities (i.e., we don't need to have the non-normalized frequency counts of the contingency table). Let us consider the general contingency table:

	Y= y_1	Y= y_2	...	Y= y_l	Total
X= x_1	n_{11}	n_{12}	...	n_{1l}	$n_{1\cdot}$
X= x_2	n_{21}	n_{22}	...	n_{2l}	$n_{2\cdot}$
...	...	...	...	...	...
X= x_k	n_{k1}	n_{k2}	...	n_{kl}	$n_{k\cdot}$
Total	$n_{\cdot 1}$	$n_{\cdot 2}$	...	$n_{\cdot l}$	n

where:

- n_{ij} is the number of times that we observe $X = x_i$ and $Y = y_j$ simultaneously (e.g., $C = 1$ and $S = male$);
- n_{i+} is the sum of all occurrences in which $X = x_i$ (i.e., we are summing all values in row i): $n_{i+} = \sum_j n_{ij}$;
- n_{+j} is the sum of all occurrences in which $Y = y_j$ (i.e., we are summing all values in column j): $n_{+j} = \sum_i n_{ij}$;

We can compute the joint probability $P(X = x_i, Y = y_j)$ with a frequentist approach using the formula:

$$P\left(X = x_i,\ Y = y_j\right) = \frac{n_{ij}}{n}$$

We note that we can define the marginal probabilities of X and Y as follows:

- $P\left(X = x_i\right) = \frac{n_{i+}}{n}$.
- $P\left(Y = y_j\right) = \frac{n_{+j}}{n}$.

We can hence write the marginal probability of X as follows:

$$P\left(X=x_i\right)=\frac{n_{i+}}{n}=\frac{\sum_j n_{ij}}{n}=\sum_j \frac{n_{ij}}{n}=\sum_j P(X=x_i,Y=y_j)$$

This result is known as the **sum rule of probability**, which allows to estimate marginal probabilities from joint probabilities. This can be seen in more general terms as:

$$P(X=x)=\sum_y P(X=x,Y=y)$$

The act of computing $P(X)$ from $P(X,Y)$ is also known as marginalization.

Conditional Probability and Product Rule

In many cases, we are interested in the probability of some event, given that some other event happened.

This is called **conditional probability** and is denoted as

$$P(X=x|Y=y)$$

and read as

> P of X=x given that Y=y

In this context, $Y=y$ is the condition, and we are interested in studying the probability of X only in the cases in which the condition is verified.

For instance, in our toy Titanic, we could be interested in $P(S=male|C=1)$, i.e., what is the probability of picking a male, given that we know that we are drawing from the first class?

Let's consider our example contingency table of two variables again:

	Y= y_1	Y= y_2	...	Y= y_l	Total
X= x_1	$n_{1:}$	$n_{1:}$	...	n_1	$n_{1\cdot}$
X= x_2	$n_{2:}$	$n_{2:}$	...	n_2	$n_{2\cdot}$
...	...	...	...	...	...
X= x_k	n_k	$n_{k:}$	...	n_k	$n_{k\cdot}$
Total	n_+	n_+	...	n_+	n

We can compute the conditional probability $P(X = x_i|Y = y_j)$ using the frequentist approach:

$$P\left(X = x_i|Y = y_j\right) = \frac{\#\ cases\ in\ which\ X = x_i\ and\ Y = y_i}{\#\ cases\ in\ which\ Y = y_j} = \frac{n_{ij}}{n_{+j}}$$

If we multiply the expression above by $1 = \frac{n}{n}$, we obtain:

$$P\left(X = x_i|Y = y_j\right) = \frac{n_{ij}}{n}\frac{n}{n_{+j}} = \frac{\frac{n_{ij}}{n}}{\frac{n_{+j}}{n}} = \frac{P\left(X = x_i, Y = y_j\right)}{P(Y = y_j)}$$

This leads us to the general definition of conditional probability:

$$P\left(X = x|Y = y\right) = \frac{P(X = x,\ Y = y)}{P(Y = y)}$$

The conditional probability is defined only when $P(Y = y) > 0$, that is, we cannot define a probability conditioned on an event that never happens. It should be noted that, in general $P\left(X|Y\right) \neq P(X)$.

Product Rule (Factorization)

We can see the definition of conditional probability:

$$P\left(X = x|Y = y\right) = \frac{P(X = x,\ Y = y)}{P(Y = y)}$$

As follows:

$$P(X = x, Y = y) = P\left(X = x | Y = y\right) P(Y = y)$$

which is often referred to as **the product rule**.

The product rule allows to compute joint probabilities starting from conditional probabilities and marginal probabilities. This is useful because measuring joint probabilities generally involves creating large tables, whereas conditional and marginal probabilities might be easier to derive.

This operation of expressing a joint probability in terms of two factors is known as *factorization*.

How to compute a conditional probability?

We just said that factorization can be useful for computing joint probabilities starting from conditional probabilities. However, two questions arise:

- How can we compute a conditional probability?
- Is it easier than computing a joint probability?

Since conditional probabilities are obtained by *restricting* the probability space to *a subset* of the events, we can compute conditional probabilities by considering the observations which satisfy the condition.

For example, let's say we want to compute the conditional probability:

$$P(S|C = 1)$$

That is to say, the probability of picking a person of a given sex, given that we know that we are considering the first class. Let's consider again our contingency table:

	First Class	Second Class	Third Class	All
male	3	5	7	15
female	3	3	4	10
All	6	8	11	25

To compute this probability, we can just *consider all the observations that satisfy the condition* $C = 1$, which is equivalent to taking the first column of the full contingency table and compute the probabilities in a frequentist way:

	First Class
male	3
female	3
All	6

$$P(S = male|C = 1) = \frac{3}{6}$$

$$P(S = female|C = 1) = \frac{3}{6}$$

Note that, in general, when the number of variables is large, this approach allows to save a lot of space and time as it is not necessary to even build the first contingency table, but only the second, restricted one is required (for instance, one may choose not to record all observations in which we where picking form the other classes).

Of course, the table above can be normalized to obtain a conditional probability table:

	First Class
male	0.5
female	0.5
All	1

In Python, it is possible to obtain the conditional relative frequencies by normalizing by the appropriate row. Rather than conditioning on class, let's condition on sex in this case. We can do so by normalizing by rows:

```
# normalize=0 indicates to condition on the first variable
pd.crosstab(titanic['Sex'], titanic['Pclass'], normalize=0, margins=True)
```

Pclass	1	2	3
Sex			
female	0.299363	0.242038	0.458599
male	0.211438	0.187175	0.601386
All	0.242424	0.206510	0.551066

From the table above, for example, we can infer:

- $f(Pclass = 1|Sex = female) = 0.299363$
- $f(Pclass = 2|Sex = female) = 0.242038$
- $f(Pclass = 3|Sex = female) = 0.458599$

These are the frequencies of women in the three classes. Similarly, the second row represents the conditional frequencies with respect to `Sex=male`. From the table, we notice that the distribution of passengers changes across the three classes. In particular, among men, the third class is more frequent than among women.

We can obtain the complementary perspective by conditioning on class instead:

```
# normalize=1 indicates conditioning on the first variable
pd.crosstab(titanic['Sex'], titanic['Pclass'], normalize=1, margins=True)
```

Pclass	1	2	3	All
Sex				
female	0.435185	0.413043	0.293279	0.352413
male	0.564815	0.586957	0.706721	0.647587

In this case, each column will be a probability distribution. For example:

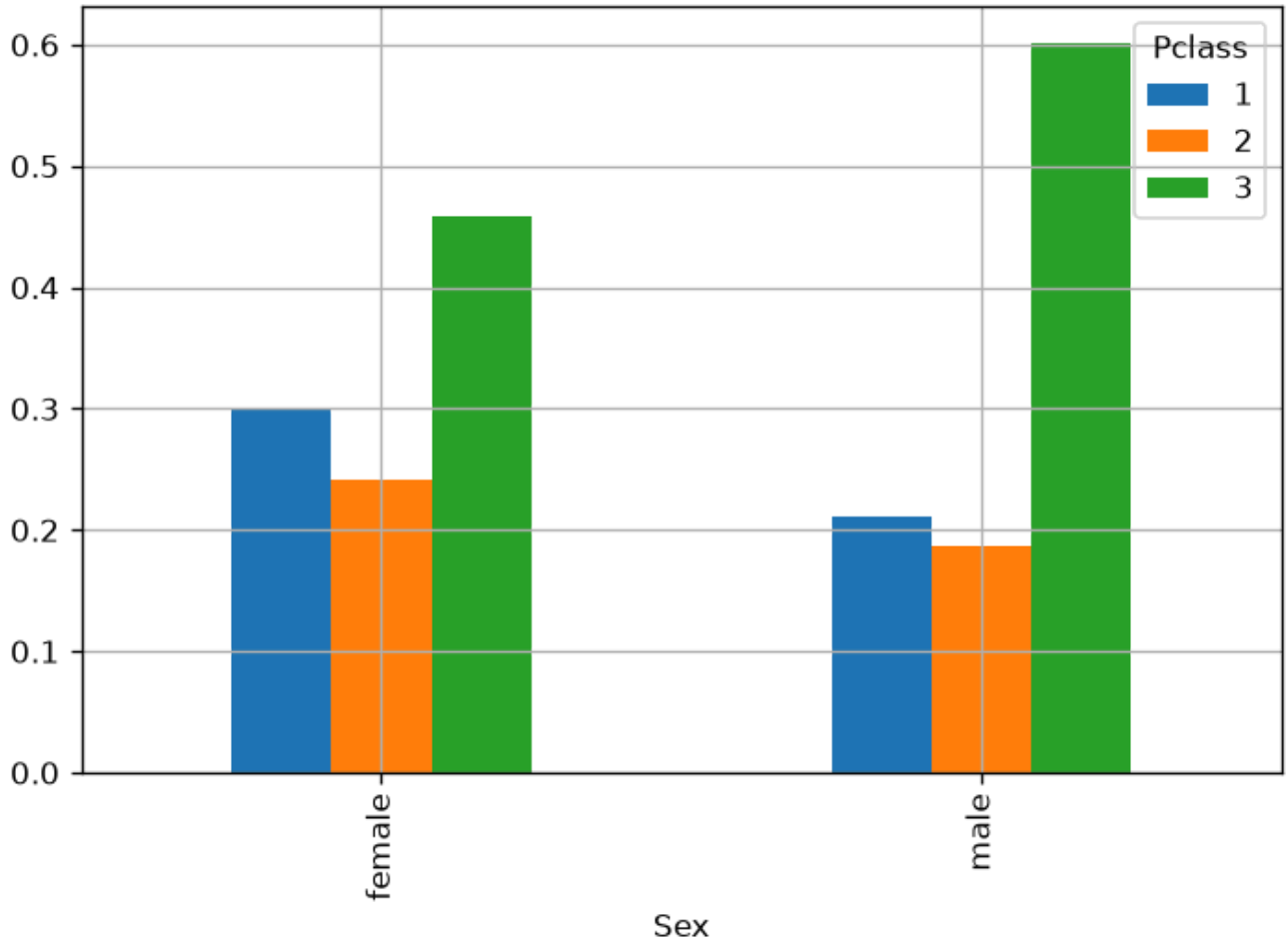
- $P(Sex = female|Pclass = 1) = 0.435185$
- $P(Sex = male|Pclass = 1) = 0.564815$

We note that the proportion between men and women changes in the three classes, and in particular in the third class there are many more men than women.

Graphical Representations

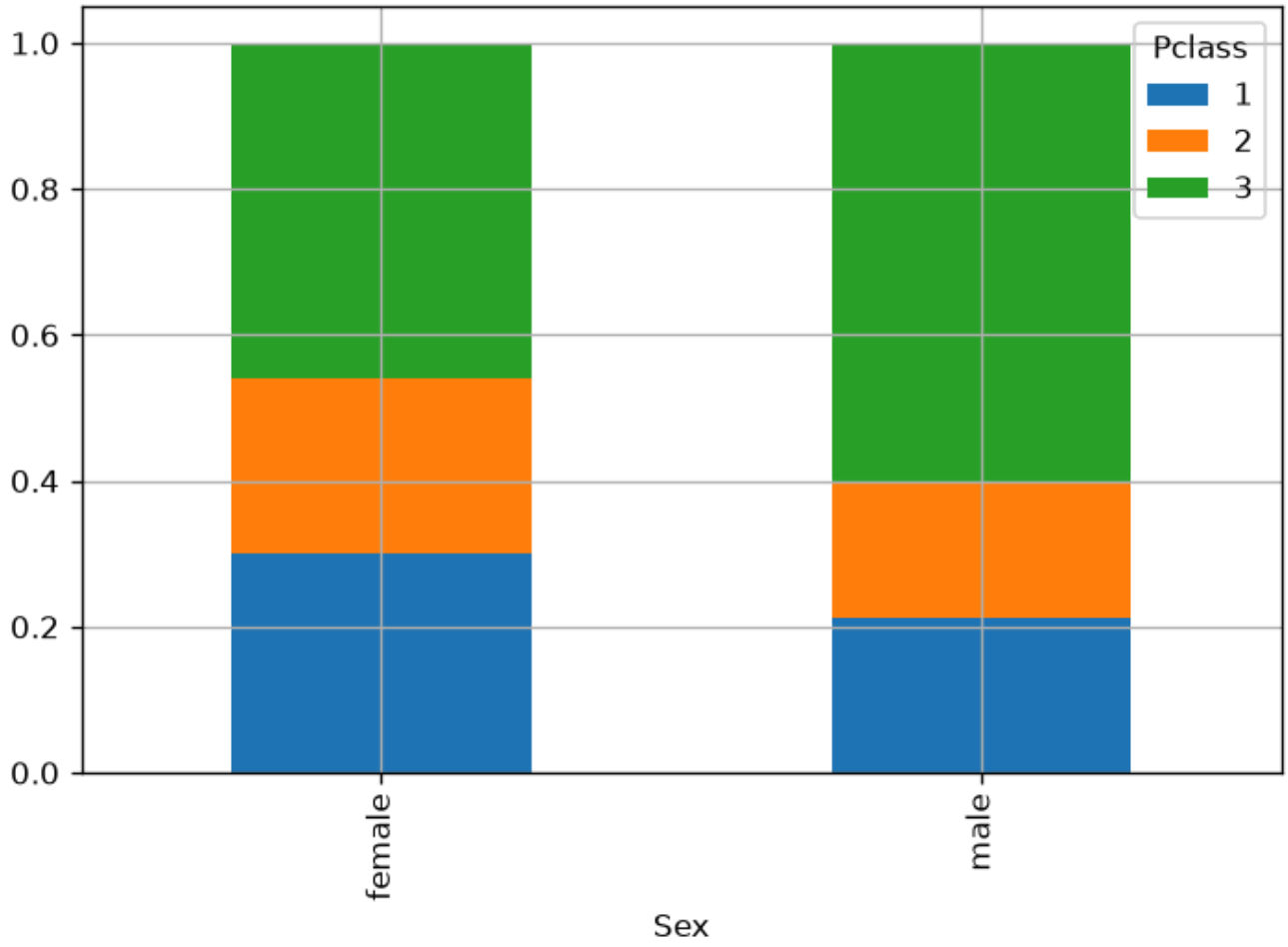
Similarly to joint probabilities, we can also obtain visualizations from conditional probabilities. For instance:

```
pd.crosstab(titanic['Sex'], titanic['Pclass'], normalize=0).plot.bar()
plt.grid()
plt.show()
```



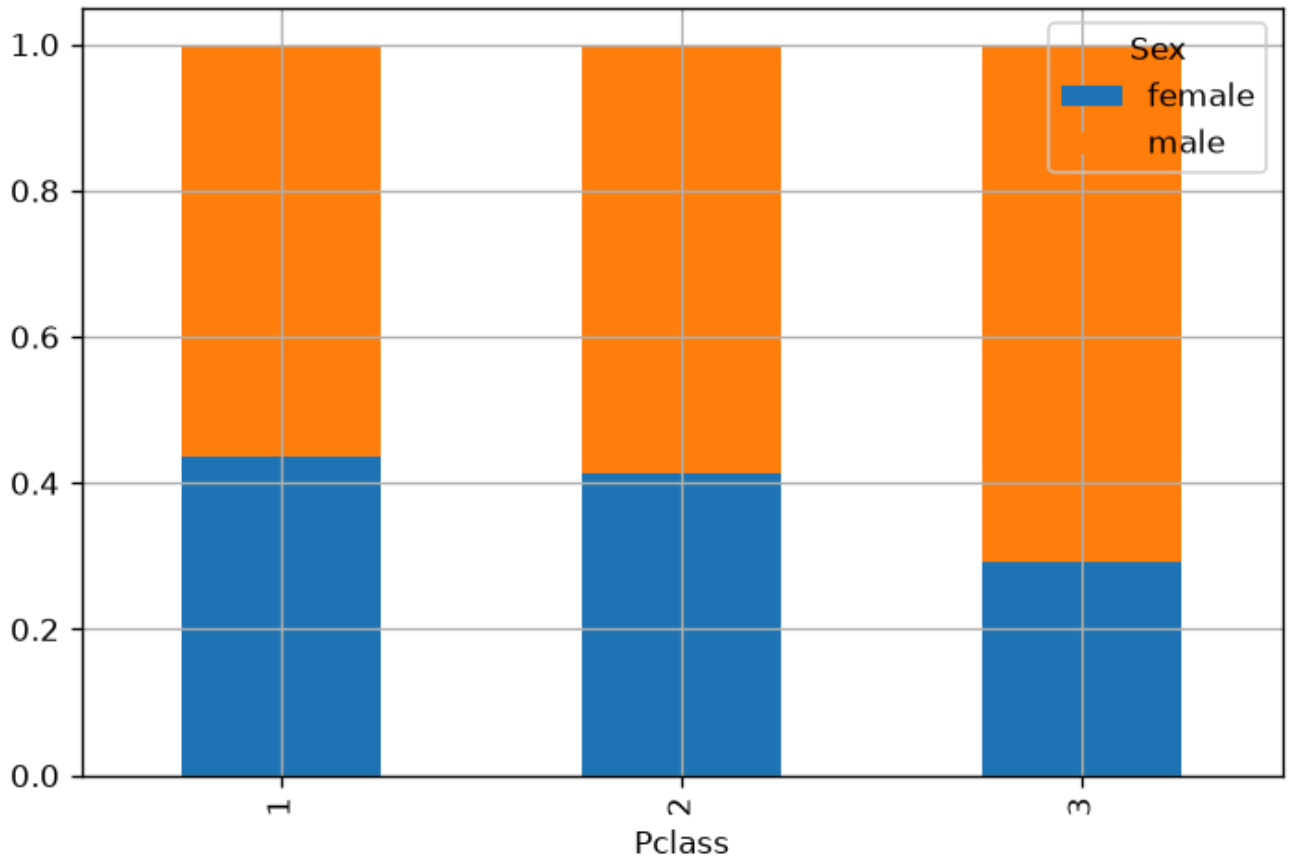
The diagram above shows the distributions of passengers in the three classes for the two sexes. Now, these are two valid probability distributions summing to one for all the three values (three classes). This is clearer with a stacked plot:

```
pd.crosstab(titanic['Sex'], titanic['Pclass'], normalize=0).plot.bar(stacked=True)
plt.grid()
plt.show()
```



We can obtain the graph of conditional frequencies with respect to the classes for a complementary view. In this case, to have a meaningful graph, we need to transpose the contingency table with a `.T`:

```
# normalize=1 indicates conditioning on the first variable
pd.crosstab(titanic['Sex'], titanic['Pclass'], normalize=1).T.plot.bar(stacked=True)
plt.grid()
plt.show()
```



This graph now shows the distribution of sexes in the three classes, clearly showing that class 3 contains mostly males.

The Chain Rule of Conditional Probabilities

When dealing with multiple variables, the product rule can be applied in an iterative fashion, thus obtaining the 'chain rule' of conditional probabilities.

For instance:

$$P(X, Y, Z) = P(X|Y, Z)\, P(Y, Z)$$

Since:

$$P(Y, Z) = P(Y|Z)\, P(Z)$$

We obtain:

$$P(X, Y, Z) = P(X|Y, Z)\, P(Y|Z)\, P(Z)$$

Since joint probabilities are symmetric, we could equally obtain:

$$P(X, Y, Z) = P(Z|Y, X)\, P(Y|X)\, P(X)$$

This rule can be formalized as follows:

$$P\left(X_1, \ldots, X_n\right) = P\left(X_1\right) \prod_{i=2}^{n} P(X_i | X_1, \ldots, X_{i-1})$$

Bayes' Theorem

Given two variables A and B, from the product rule, we obtain:

- $P(A, B) = P\left(A|B\right) P(B)$
- $P(B, A) = P\left(B|A\right) P(A)$

Since joint probabilities are symmetric, we have:

$$P(A, B) = P(B, A) \rightarrow P\left(A|B\right) P(B) = P\left(B|A\right) P(A)$$

Which implies:

$$P\left(A|B\right) = \frac{P\left(B|A\right) P(A)}{P(B)}$$

This last expression is known as Bayes' Theorem (or Bayes' rule).

Technically speaking, the Bayes' rule can be used to "turn" probabilities of the kind $P(A|B)$ into probabilities of the kind $P(B|A)$.

More formally, the Bayes' rule can be used to **update our expectation that some event will happen (event A) when we observe some evidence (event B)**. This links to the Bayesian interpretation of probability, according to which probability can be seen as **a *reasonable expectation* representing *the quantification of a degree of belief*** (i.e., how much we believe some event will happen).

The different terms in the Bayes' rule have specific names:

- $P(A)$ is called 'the prior' – this is our expectation that A happens when we do not have any other data to rely on
- $P(B|A)$ is called 'the likelihood' – this quantifies how likely it is to observe event B happening if we assume that event A has happened
- $P(B)$ is called 'the evidence' – this models the probability of observing event B
- $P(A|B)$ is called 'the posterior' – this is our updated probability, i.e., how likely it is A to happen once we have observed B happening

We'll see an example in the section below.

Bayesian Probability Example

Let's imagine we are trying to understand if a friend has COVID or not. If we do not know anything about our friend's symptoms (i.e., we don't know if they have any symptoms or not), then we would expect our friend to have COVID with a "prior" probability $P(C)$. If we know that currently one people over two has COVID, we expect:

$$P(C) = \frac{1}{2}$$

Now, if our friend tells us he has fever, things change a bit. We are now interested in modeling the probability: $P\left(C|F\right)$. We can try to model it using Bayes' rule:

$$P\left(C|F\right) = \frac{P\left(F|C\right)P(C)}{P(F)}$$

Note that it is not straightforward to estimate $P(C|F)$ in a frequentist way. Ideally, we should take all people with a fever on earth and check how many of them has COVID. This is not feasible as people with just fever may never do a COVID test. On the contrary, measuring $P(F|C)$ is easier: we take all people which we know have COVID (these may not be all people with COVID, but probably a large enough sample) and see how many of them have a fever. Let's suppose that one people with COVID out of three has a fever. Then we can expect:

$$P\left(F|C\right) = \frac{1}{3}$$

Now we need to estimate the evidence $P(F)$. This can be done by considering how frequent it is for people to have a fever. Let's say we use historical data and find out that one person out of five has fever. We finally have:

$$P\left(C|F\right) = \frac{\frac{1}{3}\frac{1}{2}}{\frac{1}{5}} = \frac{5}{6}$$

We can interpret this use of the Bayes' theorem as follows:

- Before knowing anything about symptoms, we could only guess that our friend had COVID with a prior probability of $P(C) = \frac{1}{2}$
- When we get to know that our friend has a fever, our expectation changes. We know that people with COVID often get a fever, but we also know that not all people with COVID get a fever, so we are not going to say that we are 100% certain that our friend has fever. Instead, we use Bayes' rule to obtain an updated estimate $P\left(C|F\right) = \frac{5}{6}$ based on our knowledge of the likelihood $P(F|C)$ (how likely it is for people with symptom to have COVID) and the evidence $P(F)$ (how common is this symptom – if it is too common, then it is not so informative)

Independence and Conditional Independence

Two variables X and Y are independent if the outcome of one of the two does not influence the outcome of the other. Formally speaking, two variables are **said to be independent** if and only if:

$$P(X,Y) = P(X)P(Y)$$

It should be noted that the expression above **is generally not true** as we cannot always assume that two variables are independent.

Independence can be denoted as:

$$X \perp Y$$

Moreover, if two variables X and Y are independent, then:

$$P\left(X|Y\right) = \frac{P(X,Y)}{P(Y)} = \frac{P(X)P(Y)}{P(Y)} = P(X)$$

This makes sense because it means that "the fact that Y happens does not influence the fact that X happens", which is what we would expect of two independent variables X and Y.

Examples

Intuitively, two variables are independent if the values of one of them do not affect the values of the other one:

- Weight and height of a person are **not independent**. Indeed, taller people are usually heavier.
- Height and richness are **independent**, as the richness does not depend on the height of a person.

Conditional independence

Two random variables X and Y are said to be **conditionally independent** given a random variable Z if:

$$P\left(X,Y|Z\right) = P\left(X|Z\right)P\left(Y|Z\right)$$

Conditional independence can be denoted as:

$$X \perp Y \mid Z$$

Example

Height and vocabulary are not independent: taller people are usually older, and hence they have a more sophisticated vocabulary (they know more words). However, if we condition on age, they become independent. Indeed, among people of the same age, height should not influence vocabulary. Hence, height and vocabulary are **conditionally independent** with respect to age.

Example of Probability Manipulation

Let's now consider a real probability manipulation example. This time we will consider the following variables: `Pclass` and `Survived`. We will use observed proportions to answer questions such as:

1. What is the overall probability that a passenger survived?
2. What is the probability that a passenger was in 1st class and they survived?
3. What is the probability that a passenger survived, given that they were in 3rd class?

We will start by defining two random variables:

- $Pclass \in \{1, 2, 3\}$: Passenger class
- $Survived \in \{0, 1\}$: Survival outcome (0 = died, 1 = survived)

What is the overall probability that a passenger survived?

To answer this question, we need to compute the overall probabilities of surviving ($P(Survived)$):

```
titanic['Survived'].value_counts(normalize=True)
```

```
Survived
0    0.616162
1    0.383838
Name: proportion, dtype: float64
```

From the table above, we can say:

- $P(Survived = 1) = 0.39$
- $P(Survived = 0) = 0.61$

What is the probability that a passenger was in 1st class and they survived?

To answer this question, we need to compute the joint probability:

```
pd.crosstab(titanic['Survived'], titanic['Pclass'], normalize=True, margins=True)
```

Pclass	1	2	3	All
Survived				
0	0.089787	0.108866	0.417508	0.616162
1	0.152637	0.097643	0.133558	0.383838
All	0.242424	0.206510	0.551066	1.000000

From the table above, we can answer our question as follows:

$$P(Pclass = 1, Survived = 1) = 0.15$$

Read the cell where the row $Survived = 1$ meets the column $Pclass = 1$. The cell just above it, 0.09, is the probability of being in first class **and dying** — a different question, and an easy one to answer by mistake.

What is the probability that a passenger survived, given that they were in 3rd class?

To answer this question, we need to compute a conditional distribution:

```
pd.crosstab(titanic['Survived'], titanic['Pclass'], normalize=1, margins=True)
```

Pclass	1	2	3	All
Survived				
0	0.37037	0.527174	0.757637	0.616162
1	0.62963	0.472826	0.242363	0.383838

To answer the question, we can say:

$$P(Survived = 1|Pclass = 3) = 0.24$$

Exercises

Suppose that we have three colored boxes r (red), b (blue), and g (green). Box r contains 3 apples, 4 oranges, and 3 limes, box b contains 1 apple, 1 orange, and 0 limes, and box g contains 3 apples, 3 oranges, and 4 limes. If a box is chosen at random with probabilities P(r)=0.2, P(b)=0.2, P(g)=0.6, and a piece of fruit is extracted from the box (with equal probability of selecting any of the items in the box), then what is the probability of selecting an apple? If we observe that the selected fruit is in fact an orange, what is the probability that it came from the green box?

References

- Parts of chapter 1 of [1];
- Most of chapter 3 of [2];
- Parts of chapters 5-7 of [3].

[1] Bishop, Christopher M. *Pattern recognition and machine learning*. springer, 2006. https://www.microsoft.com/en-us/research/uploads/prod/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf

[2] Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016. https://www.deeplearningbook.org/

[3] Heumann, Christian, and Michael Schomaker Shalabh. Introduction to statistics and data analysis. Springer International Publishing Switzerland, 2016.

© 2026 Antonino Furnari · Department of Mathematics and Computer Science, University of Catania Built with Quarto

Credits and data sources

Association between variables

Chapter resources

Slides (PDF) Open in Colab Practice quiz

We have seen how to describe and visualize "univariate" samples, i.e., samples composed of a single variable. More precisely, even when several variables were available, we analyzed the variables one by one. In practice, it often happens that the observations under analysis are composed of multiple variables. In these cases, it is useful to consider methods to describe and visualize the data in a "multivariate" manner, in order to study the interactions between the various factors that describe the phenomenon under analysis.

> In simple terms, we will say that two variables are associated if observing the value of one of the two variables tell us something about the value of the other variable.

As we will see, this is closely **related to the concept of dependence in probability**. In fact, **if two variables are independent, they typically show no statistical association**—knowing the value of one provides no information about the other.

Examples of associated variables are:

- Weight and height of a person: tall people tend to weigh more than short people;
- Education level and income: individuals with higher education often earn more;
- Age and blood pressure: older individuals tend to have higher blood pressure;
- Hours studied and exam score: more study time is often linked to better performance;

Note that correlation does not imply causation: two variables may be statistically associated without one directly causing the other. Confounding factors or reverse causality may be at play.

We will use again the Titanic dataset for our examples. Let's load it:

```
import pandas as pd
from matplotlib import pyplot as plt
titanic = pd.read_csv(DATA + "titanic.csv",
                      index_col='PassengerId')
titanic.head()
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Given the dataset above, we might ask ourselves if some variables influence the values of others. For example:

- does being in first, second, or third class (variable `Pclass`) somehow influence the probability of survival (variable `Survived`)?
- does age (variable `Age`) or the price paid (`Fare`) somehow influence the probability of being saved (`Survived`)?

In this lesson, we will see different ways to summarize the distributions of two variables and verify any associations (or correlations) between the variables.

Measures of Association between two Discrete Variables

Now let's look at measures that allow us to estimate if two variables are "associated" or not, meaning to what extent the values of one variable influence those of the other. We have seen that the variables `Sex` and `Pclass` seem to have some degree of association. Let's now look at a case of stronger association, by exploring the relationship between `Pclass` and `Survived`.

Let's consider three relative frequency distributions of `Survived` conditioning on the `Pclass` values:

- f(Survived|Pclass=1)
- f(Survived|Pclass=2)
- f(Survived|Pclass=3)

The table of conditional frequencies that follows summarizes these values

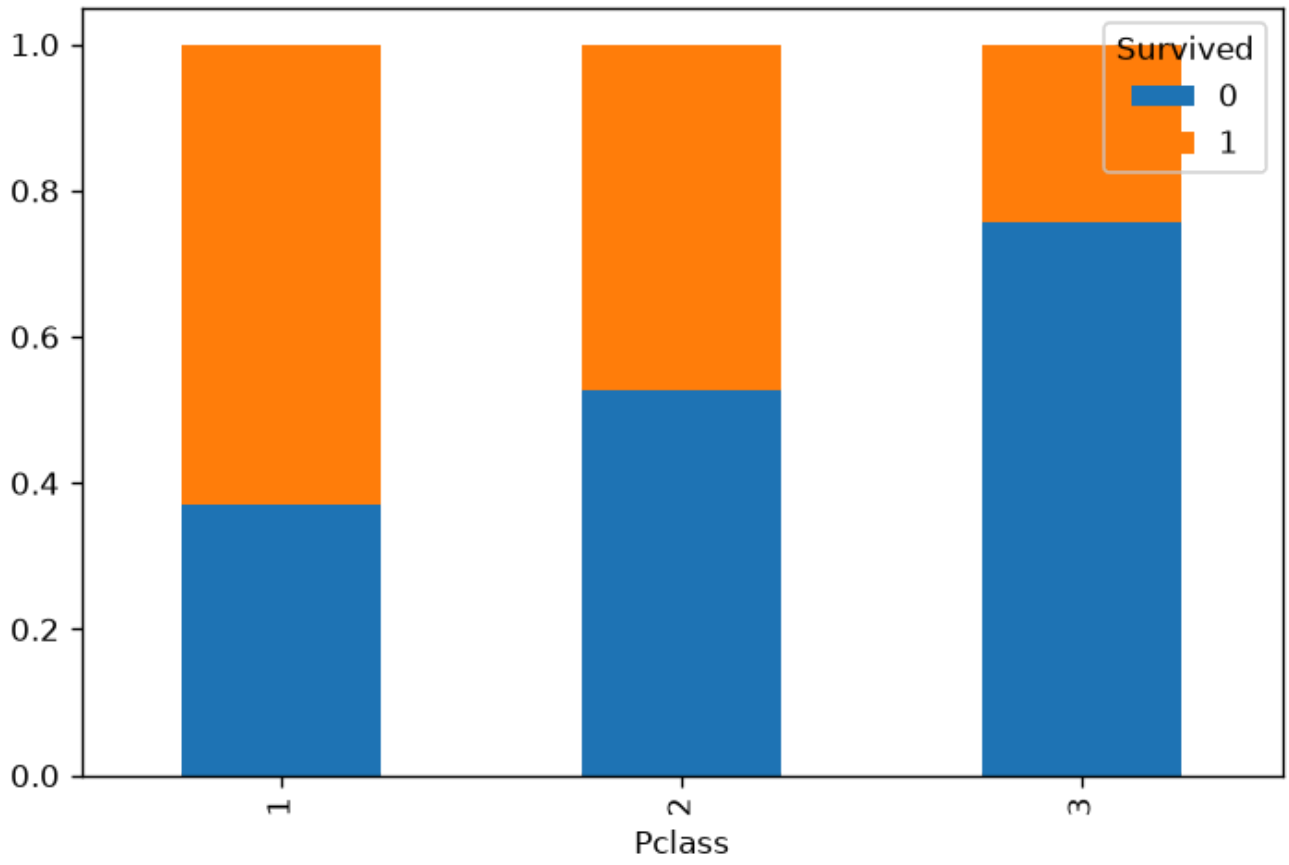
```
pd.crosstab(titanic['Pclass'], titanic['Survived'], normalize=0)
```

Survived	0	1
Pclass		
1	0.370370	0.629630
2	0.527174	0.472826
3	0.757637	0.242363

Looking at the numbers, we see that the distributions are very different, depending on the conditioning applied. This is not surprising, because we imagine that passengers in first and second class had different treatment compared to those in third class.

Let's visualize the distributions graphically:

```
pd.crosstab(titanic['Pclass'], titanic['Survived'],
        normalize=0).plot.bar(stacked=True)
plt.show()
```



The graph clearly shows the imbalance, from which we deduce that `Pclass` and `Survived` are in fact **associated** (or **correlated**). Indeed, knowing the class, tell us something about the **likely values of survived**.

Independence and Expected Frequencies

Before proceeding to illustrate some measures of association between variables, we must discuss the concept of **independence**, which is the opposite of association.

Two variables are said to be independent if the observation of one variable does not allow us to make any kind of prediction about the possible values of the other variable.

Imagine two randomly chosen people living in different, randomly chosen parts of the world. If we observe the way one of these two people dresses, this observation will tell us nothing about how the other might dress. The variables “clothes of the first person” and “clothes of the second person” are independent. The same does not apply if the people live in the same city. In that case, if it rains, for example, we expect the two people to wear similar clothes, so observing how one of the two people dresses can tell us something about how the other will dress.

Let’s consider the generic contingency:

	Y= y_1	**Y=** y_2	...	**Y=** y_l	**Total**
X= x_1	n_{11}	n_{12}	...	n_{1l}	$n_{1\cdot}$
X= x_2	n_{21}	n_{22}	...	n_{2l}	$n_{2\cdot}$
...	...	...	...	...	...
X= x_k	n_{k1}	n_{k2}	...	n_{kl}	$n_{k\cdot}$
Total	n_{+1}	n_{+2}	...	n_{+l}	n

Now suppose we cannot observe the co-occurrences, but can only observe the marginal values:

	Y= y_1	Y= y_2	...	Y= y_l	Total
X= x_1			...		$n_{1\cdot}$
X= x_2			...		$n_{2\cdot}$
...	...	...	...	...	...
X= x_k			...		$n_{k\cdot}$
Total	n_{+}	n_{+}	...	n_{+}	n

Now suppose we need to "reconstruct" the missing values. If the two variables are independent, then:

$$P(X = x_i, Y = y_j) = P(X = x_i)P(Y = y_j)$$

Recall that:

$$P(X = x_i) = \frac{n_{i+}}{n} \qquad P(Y = y_j) = \frac{n_{+j}}{n} \qquad P(X = x_i, Y = y_j) = \frac{n_{ij}}{n}$$

From here we obtain that the reconstructed frequencies $\tilde{n}_{ij}$ can be computed as follows:

$$\tilde{n}_{ij} = n \cdot P(X = x_i, Y = y_j) = n \cdot P(X = x_i) \cdot P(Y = y_j) = n \cdot \frac{n_{i+}}{n}\frac{n_{+j}}{n} = \frac{n_{i+}n_{+j}}{n}$$

From which:

$$\tilde{n}_{ij} = P(X = x_i)n_{+j} = P(Y = y_j)n_{+i}$$

Where the tilde on $\tilde{n}_{ij}$ indicates that this is an estimated value, not the actually observed value.

What has been seen above suggests that the frequencies n_{ij} follow the same distribution as the marginal frequencies in case of independence between the variables.

Example

Let's see an example on our contingency table putting `Pclass` and `Survived` in relation:

Survived	0	1	All
Pclass			
1	80	136	216
2	97	87	184
3	372	119	491
All	549	342	891

The marginal probability $P(Survived)$ is given by:

```
Survived
0    0.616162
1    0.383838
Name: proportion, dtype: float64
```

These are the probabilities $P(Survived = 0)$ and $P(Survived = 1)$.

If the two variables were not correlated, we would expect to have absolute frequencies proportional to the marginal probabilities:

$$\tilde{n}_{ij} = P(Y = y_j) \cdot n_{i+}$$

These values will be given by the following table:

Survived	0	1	All
Pclass			
1	133.090909	82.909091	216.0
2	113.373737	70.626263	184.0
3	302.535354	188.464646	491.0
All	549.000000	342.000000	891.0

Comparing these frequencies with the actual frequencies, we realize that there are discrepancies, reasonably due to the fact that the variables are not in fact independent.

Pearson's χ^2 Statistic

Pearson's χ^2 statistic measures these discrepancies with the following formula:

$$\chi^2 = \sum_{i=1}^{k} \sum_{j=1}^{l} \frac{(n_{ij} - \tilde{n}_{ij})^2}{\tilde{n}_{ij}}$$

The above formula calculates the squared differences between the observed and expected absolute frequencies (to eliminate negative signs) and scales the result by the expected frequencies. **Scaling the**

result ensures that small discrepancies obtained for small expected frequencies weigh more than small discrepancies obtained for large expected frequencies.

Note that if the expected frequencies are identical to the observed ones, the statistic is zero.

We can calculate Pearson's χ^2 statistic as follows:

```
from scipy.stats import chi2_contingency
contingency = pd.crosstab(titanic['Pclass'], titanic['Survived'])
chi2_contingency(contingency)
```

```
Chi2ContingencyResult(statistic=np.float64(102.88898875696056),
pvalue=np.float64(4.549251711298793e-23), dof=2, expected_freq=array([[133.09090909,
82.90909091],
       [113.37373737,  70.62626263],
       [302.53535354, 188.46464646]]))
```

The function returns:

- The statistic value (102.89);
- The p-value of an associated statistical test (we will see this in a future lecture);
- The number of degrees of freedom, a technical details of the distribution underlying the statistical test;
- The "expected frequencies", i.e., the frequencies we would expect in case of independence.

Please note that when computing the chi-squared statistics, **certain conditions must be met to ensure validity**. Most importantly, expected frequencies in each cell of the contingency table should be sufficiently large—typically at least 5. This helps approximate the sampling distribution to a chi-squared distribution. Additionally, observations should be independent, and the data should be categorical, organized into a contingency table with mutually exclusive categories.

For the moment, we will stop here. We will get back to this statistic when we'll talk about statistical tests.

Cramer's V Statistic

Pearson's χ^2 statistic is not normalized, in the sense that its maximum value depends on the sample size. To overcome this problem, Cramer's V statistic can be used, defined as follows:

$$V = \sqrt{\frac{\chi^2}{n(min(k,l)-1)}}$$

The result will be a value between 0 and 1.

Cramer's V statistic can be computed as follows:

```
from scipy.stats.contingency import association

print(f"{association(pd.crosstab(titanic['Pclass'], titanic['Survived'])):0.2f}")
```

```
0.34
```

This value indicates a small association between the two variables, because the value is greater than 0 but still not close to 1.

Associations Between Continuous Variables

When variables are continuous, it is no longer possible to construct contingency tables. In those cases, other tools are used, which we will see in this section.

We will consider the "diabetes" dataset for this part. The dataset was introduced in 1979 by Reaven and Miller and examines the relationship between blood measurements and insulin in 145 adults:

```
from statsmodels.datasets import get_rdataset
data = get_rdataset('Diabetes','heplots').data
data.head()
```

	relwt	glufast	glutest	instest	sspg	group
0	0.81	80	356	124	55	Normal
1	0.95	97	289	117	76	Normal
2	0.94	105	319	143	105	Normal
3	1.04	90	356	199	108	Normal
4	1.00	90	323	240	143	Normal

The 6 variables have the following meanings:

- `relwt` relative weight, expressed as the ratio of actual weight to expected weight, given the person's height, a numeric vector
- `glufast` fasting plasma glucose level, a numeric vector
- `glutest` test plasma glucose level, a measure of glucose intolerance, a numeric vector
- `instest` plasma insulin during test, a measure of insulin response to oral glucose, a numeric vector
- `sspg` steady state plasma glucose, a measure of insulin resistance, a numeric vector
- `group` diagnostic group, a factor with levels `Normal` `Chemical_Diabetic` `Overt_Diabetic`

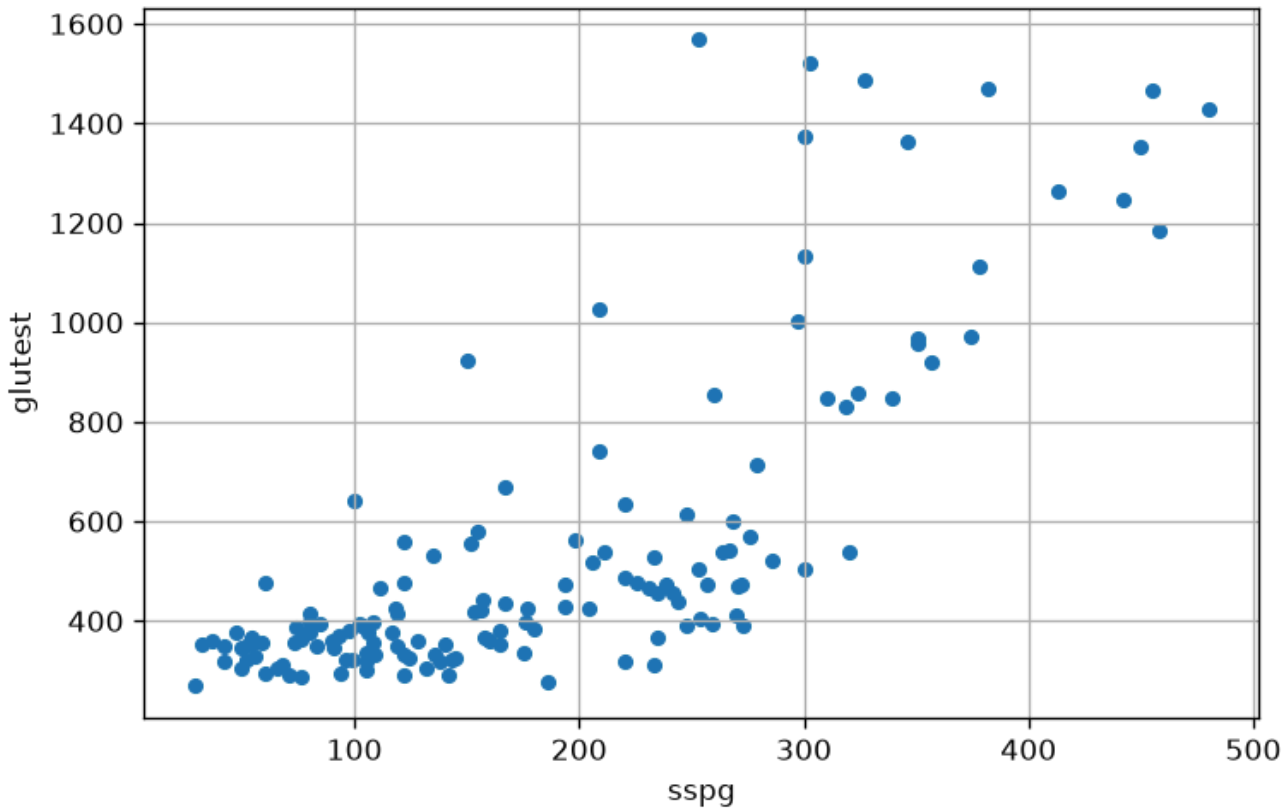
Graphical Representation of Two Continuous Variables

Let's start by considering the variables `sspg` and `glutest`.

Scatter plots

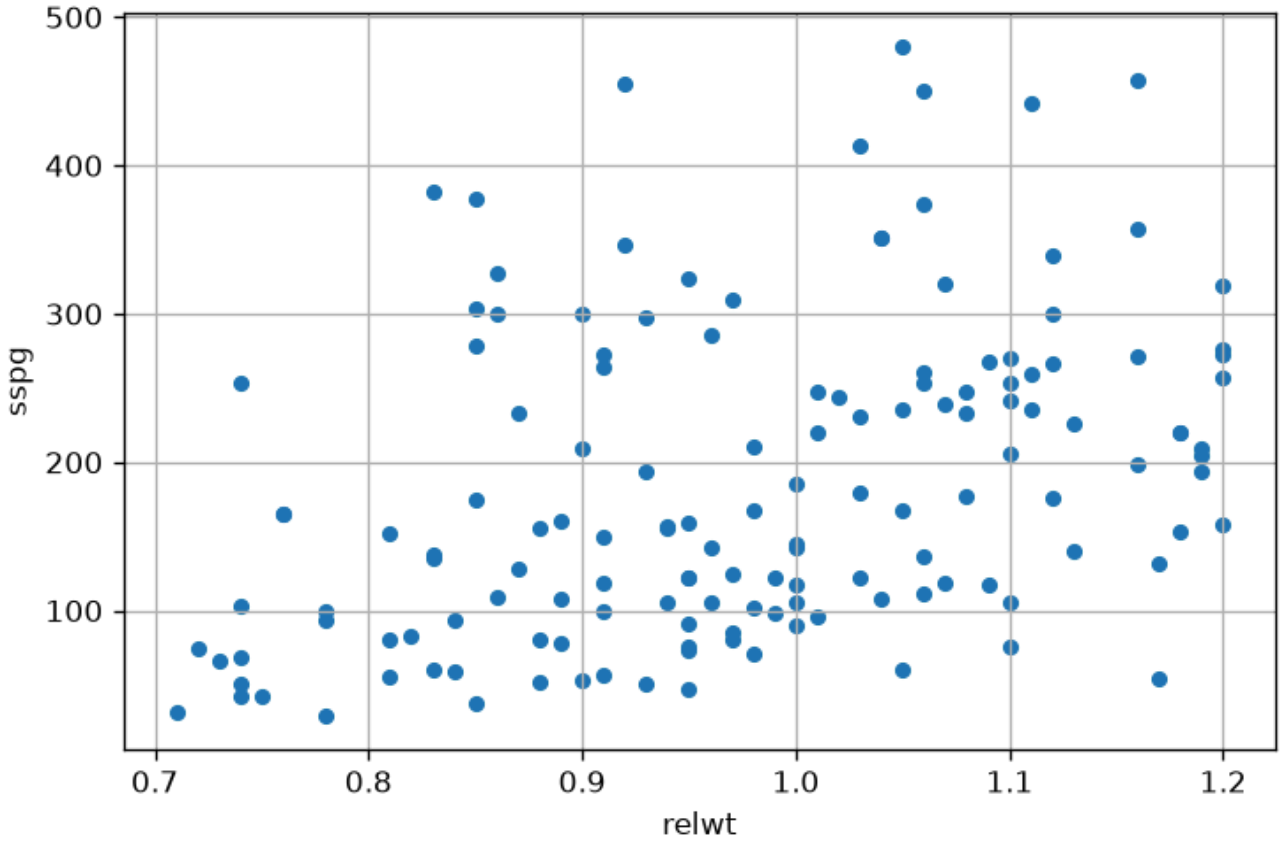
The first useful form of representation is the **scatter plot**, which visualizes observations as two-dimensional data:

```
from matplotlib import pyplot as plt
data.plot.scatter(x='sspg',y='glutest')
plt.grid()
```



We note that there seems to be a relationship between the two variables. Indeed, as `sspg` increases, the values of `glutest` tend to increase. There appears to be a positive **correlation** (or **association**). Let's now consider the pair `relwt` and `sspg`:

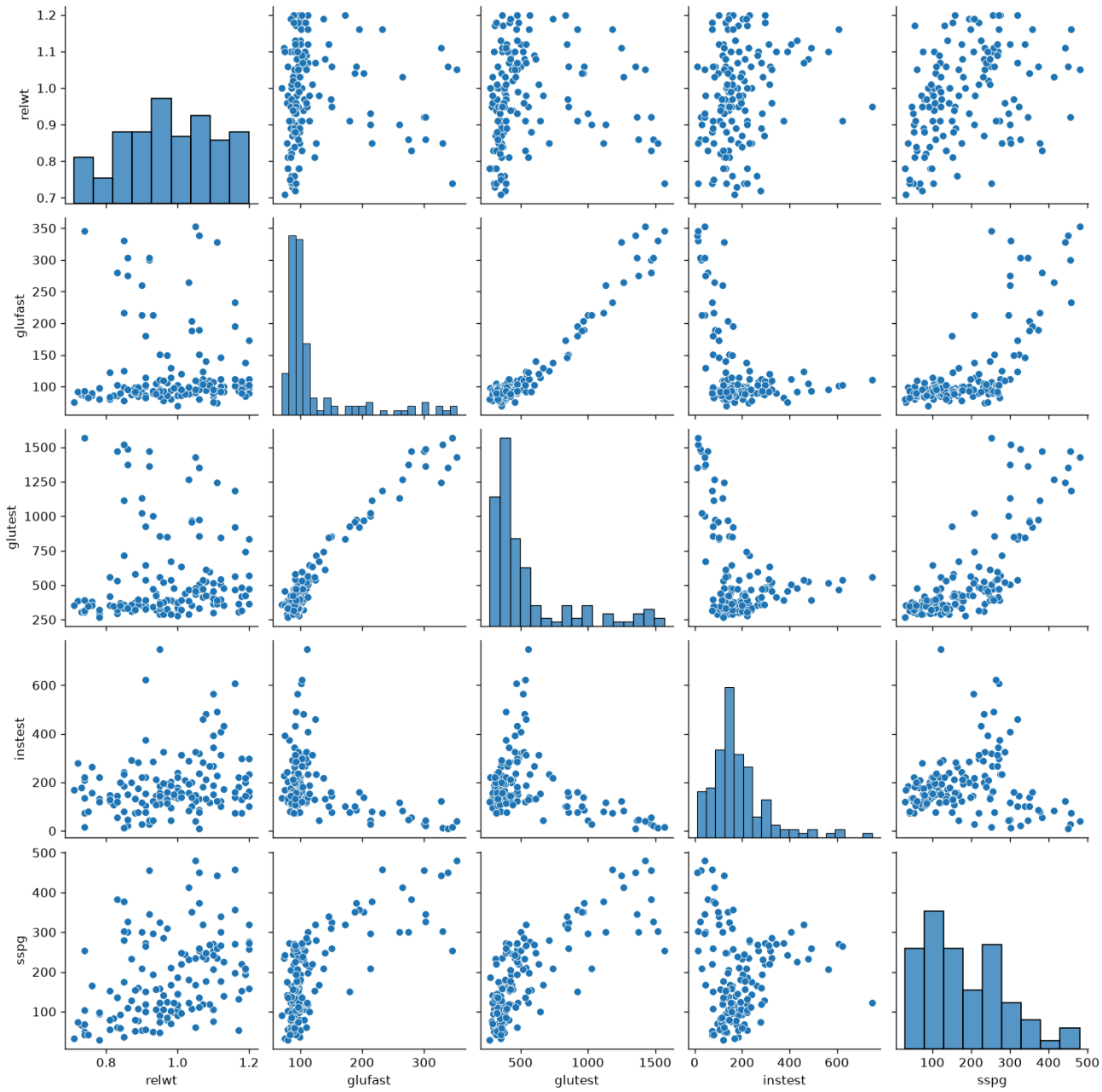
```
data.plot.scatter(x='relwt',y='sspg')
plt.grid()
```



Scatter Matrix

We note that in this case the relationship appears less marked, but we still note an ascending trend of `sspg` as `relwt` grows. It is often useful to try to study correlations with a **scattermatrix** that shows all possible scatterplots. In the case of our example:

```
from matplotlib import pyplot as plt
import seaborn as sns
sns.pairplot(data)
plt.show()
```

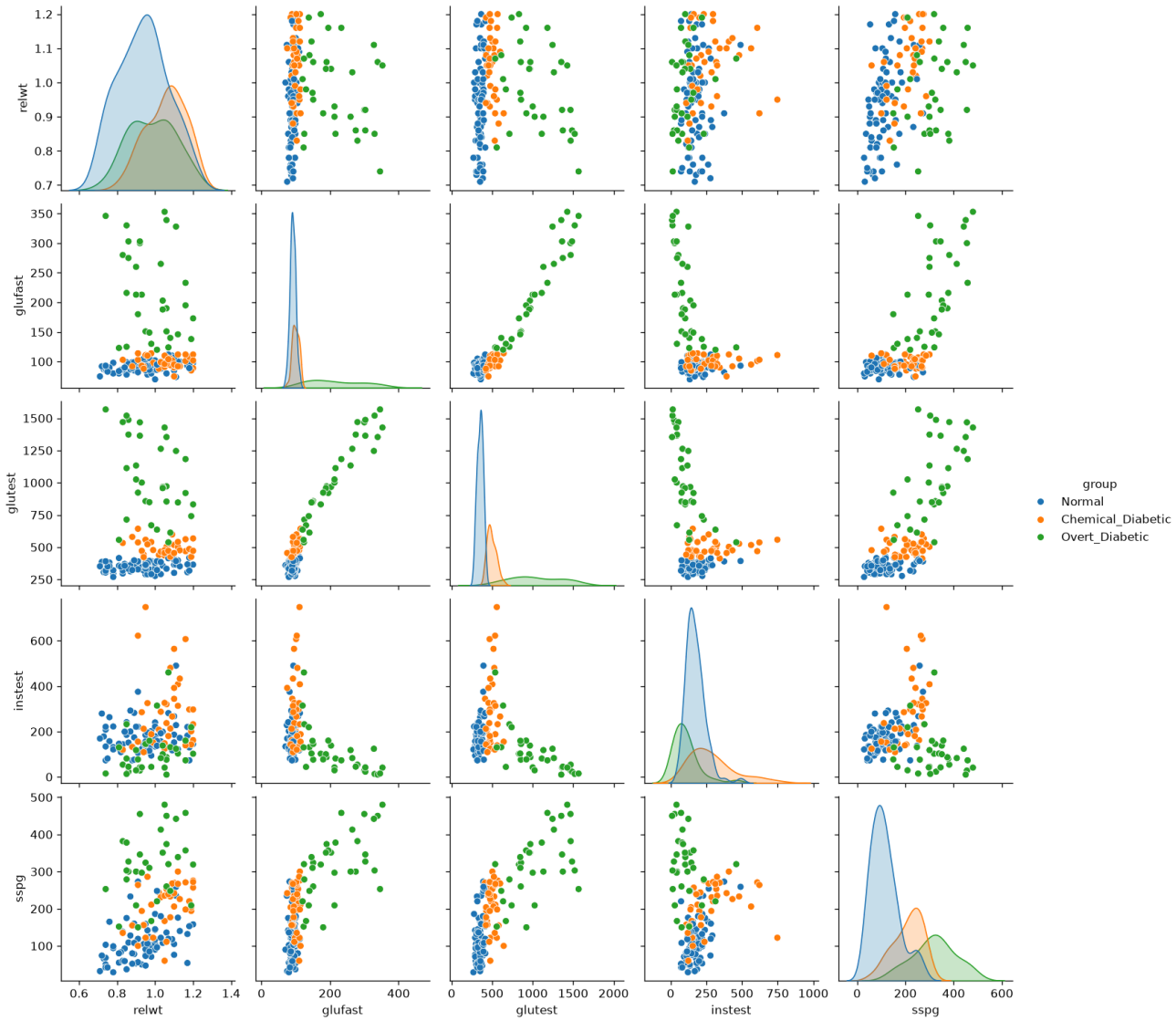


From the visualization above, we can deduce a few things:

- some pairs of variables are positively correlated (e.g., `relwt` - `sspg`), while others are negatively so (`glutest` - `instest`);
- some relationships are "linear" (e.g., `glutest` and `glufast`), while others tend not to be (e.g., `glufast` and `sspg`):
- some pairs of variables (e.g., `sspg` and `relwt`) do not appear to be correlated.

In our case, the data also contain a discrete variable `group` dividing the data into groups. We can reflect this in our visualization by specifying `hue='group'`:

```
from matplotlib import pyplot as plt
import seaborn as sns
sns.pairplot(data, hue='group')
plt.show()
```



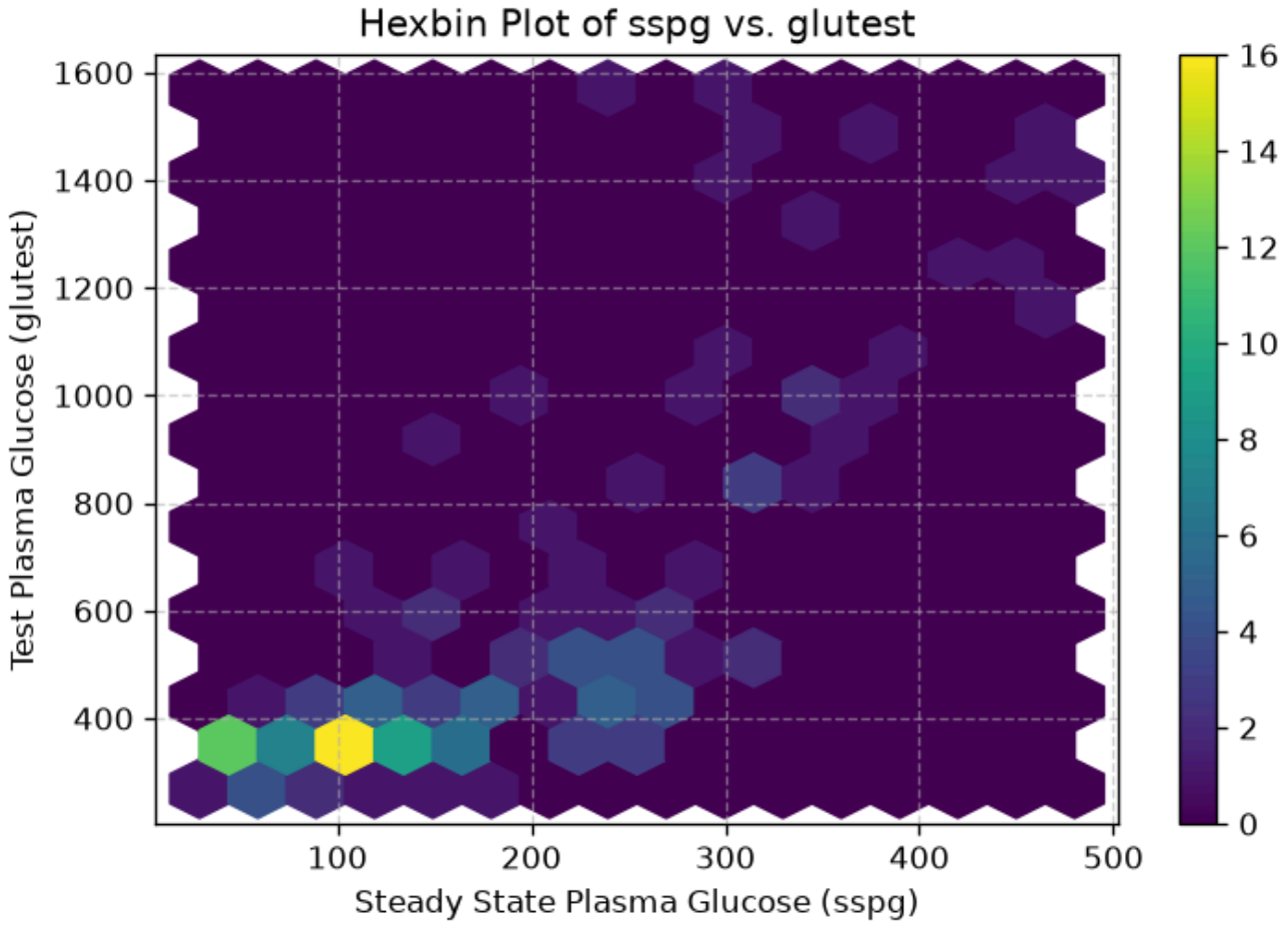
Hexbin Plots: A 2D Histogram

When a scatterplot contains a large number of data points, we can encounter the problem of **overplotting**, where points are plotted on top of each other. This makes it difficult to discern the density of the data, as heavily populated areas can look the same as moderately populated ones.

> A **hexbin plot** is an excellent alternative that addresses this issue[cite: 48]. It divides the two-dimensional space into a grid of hexagonal bins and then colors each hexagon based on the number of data points it contains. Think of it as a two-dimensional histogram, which effectively visualizes data density.

Let's generate a hexbin plot for the `sspg` and `glutest` variables. We can adjust the `gridsize` parameter to control the number of bins (and thus their size).

```
from matplotlib import pyplot as plt
# let's use the hexbin function
# cmap is the color map
# gridsize regulates the number of hexagons
data.plot.hexbin(x='sspg', y='glutest', gridsize=15, cmap='viridis')
plt.xlabel('Steady State Plasma Glucose (sspg)')
plt.ylabel('Test Plasma Glucose (glutest)')
plt.title('Hexbin Plot of sspg vs. glutest')
plt.grid(linestyle='--', alpha=0.6)
plt.show()
```



This plot clearly shows that the highest concentration of data points (the darkest hexagons) is where both `sspg` and `glutest` values are low. This is a level of detail about the data's distribution that might be obscured in a dense, standard scatterplot.

Density and Contour Plots: A Smoother View of Distribution

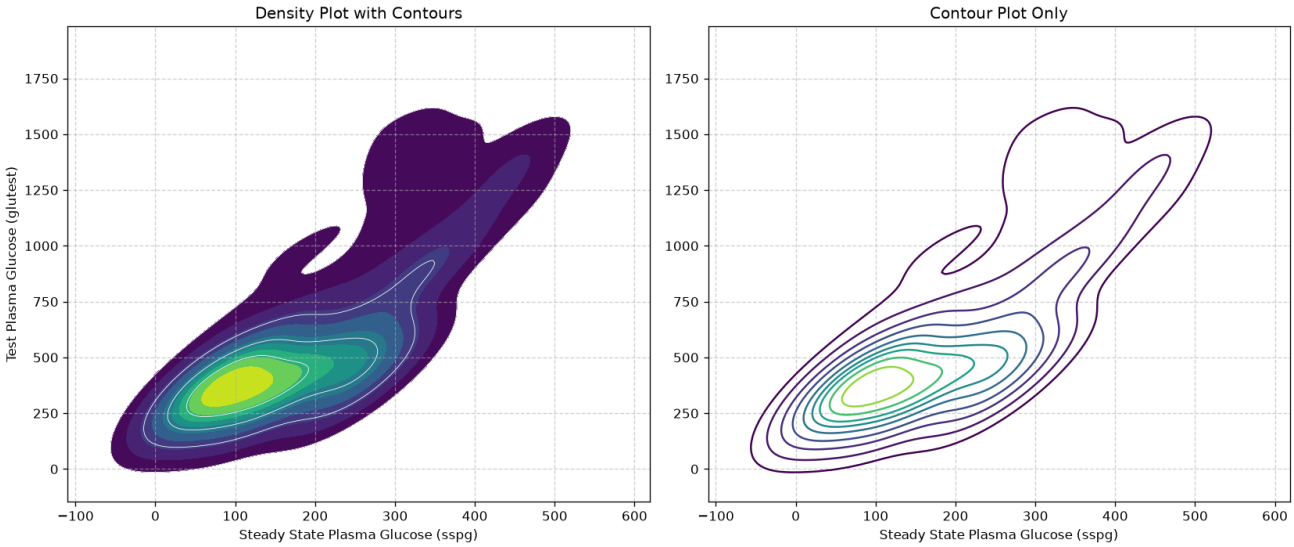
Just as we used Kernel Density Estimation (KDE) to create a smooth curve representing the distribution of a single variable, we can apply the same principle to two variables. This gives us a smoother alternative to the binned approach of the hexbin plot.

The result is a **2D density plot**, which visualizes the estimated probability density of data points using a continuous color gradient. We can also overlay **contour lines**, which connect points of equal density, much like a topographical map shows lines of equal elevation. 🗺️

This method is excellent for identifying the shape and peaks of a bivariate distribution without being constrained by the grid of a histogram or hexbin plot. We'll revisit the theory behind **Kernel Density Estimation** in a later lecture, but for now, let's focus on its application.

Let's create a density plot for the `sspg` and `glutest` variables using the `seaborn` library.

```
import seaborn as sns
from matplotlib import pyplot as plt

# Create a figure and a set of subplots (1 row, 2 columns)
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# --- Plot 1: Left Subplot (Density + Contour) ---
# we'll use kdeplot from seaborn
# we use it twice: first to fill the density, then to draw the contours
sns.kdeplot(data=data, x='sspg', y='glutest', fill=True, cmap='viridis', ax=axes[0])
# levels regulates the number of contour lines
# color and linewidths regulate the appearance of the contour lines
sns.kdeplot(data=data, x='sspg', y='glutest', levels=5, color='white',
        linewidths=0.5, ax=axes[0])
axes[0].set_title('Density Plot with Contours')
axes[0].set_xlabel('Steady State Plasma Glucose (sspg)')
axes[0].set_ylabel('Test Plasma Glucose (glutest)')
axes[0].grid(linestyle='--', alpha=0.6)

# --- Plot 2: Right Subplot (Contour Only) ---
# only the contours
sns.kdeplot(data=data, x='sspg', y='glutest', cmap='viridis', ax=axes[1])
axes[1].set_title('Contour Plot Only')
axes[1].set_xlabel('Steady State Plasma Glucose (sspg)')
axes[1].set_ylabel('') # Hide y-axis label for a cleaner look
axes[1].grid(linestyle='--', alpha=0.6)

# Adjust the layout to prevent titles/labels from overlapping
plt.tight_layout()

# Display the plots
plt.show()
```



In the plot on the left, the color-filled areas provide an intuitive map of the data's density; the darkest regions represent the highest concentration of points, while the contour lines enclose areas of progressively lower density. This visualization clearly highlights a single, dense cluster in the lower-left corner.

The **contour-only plot** on the right is particularly useful when you need to **compare multiple distributions on the same axes**. While overlaying several filled density plots would result in colors obscuring each other, using only the contour lines for each group allows you to clearly see how the centers and shapes of different distributions overlap or diverge without the visual clutter.

Covariance

We have seen that **variance** indicates how dispersed a univariate sample is around its mean. We can use a similar idea to quantify to what extent **two variables are distributed similarly around their mean**. We expect that, if the variables are correlated, then they will be distributed similarly.

The concept of **variance** is generalized by the concept of **covariance** in the case of bivariate data. Covariance measures how much the two variables vary together and is measured as follows:

$$Cov(X,Y) = \frac{1}{n}\sum_{i}^{n}(x^{(i)} - \overline{x})(y^{(i)} - \overline{y})$$

where X and Y are the two variables of interest, $x^{(i)}$ is the value of X for the i-th observation, $y^{(i)}$ is the value of Y for the i-th observation, and $\overline{x}$, $\overline{y}$ are the means of the observations for the two variables X and Y.

Note that we can obtain an unbiased estimation of the covariance similar to the variance as follows:

$$Cov(X,Y) = \frac{1}{n-1}\sum_{i}^{n}(x^{(i)} - \overline{x})(y^{(i)} - \overline{y})$$

This is important especially with small samples.

The main "actors" in the formula above are the terms:

- $(x^{(i)} - \overline{x})$: measures the distance between a point $x^{(i)}$ and its respective mean $\overline{x}$. If the value of the point in question is above the mean, then this difference will be positive, otherwise negative.
- $(y^{(i)} - \overline{y})$: measures the distance between a point $y^{(i)}$ and its respective mean $\overline{y}$. If the value of the point in question is above the mean, then this difference will be positive, otherwise negative.

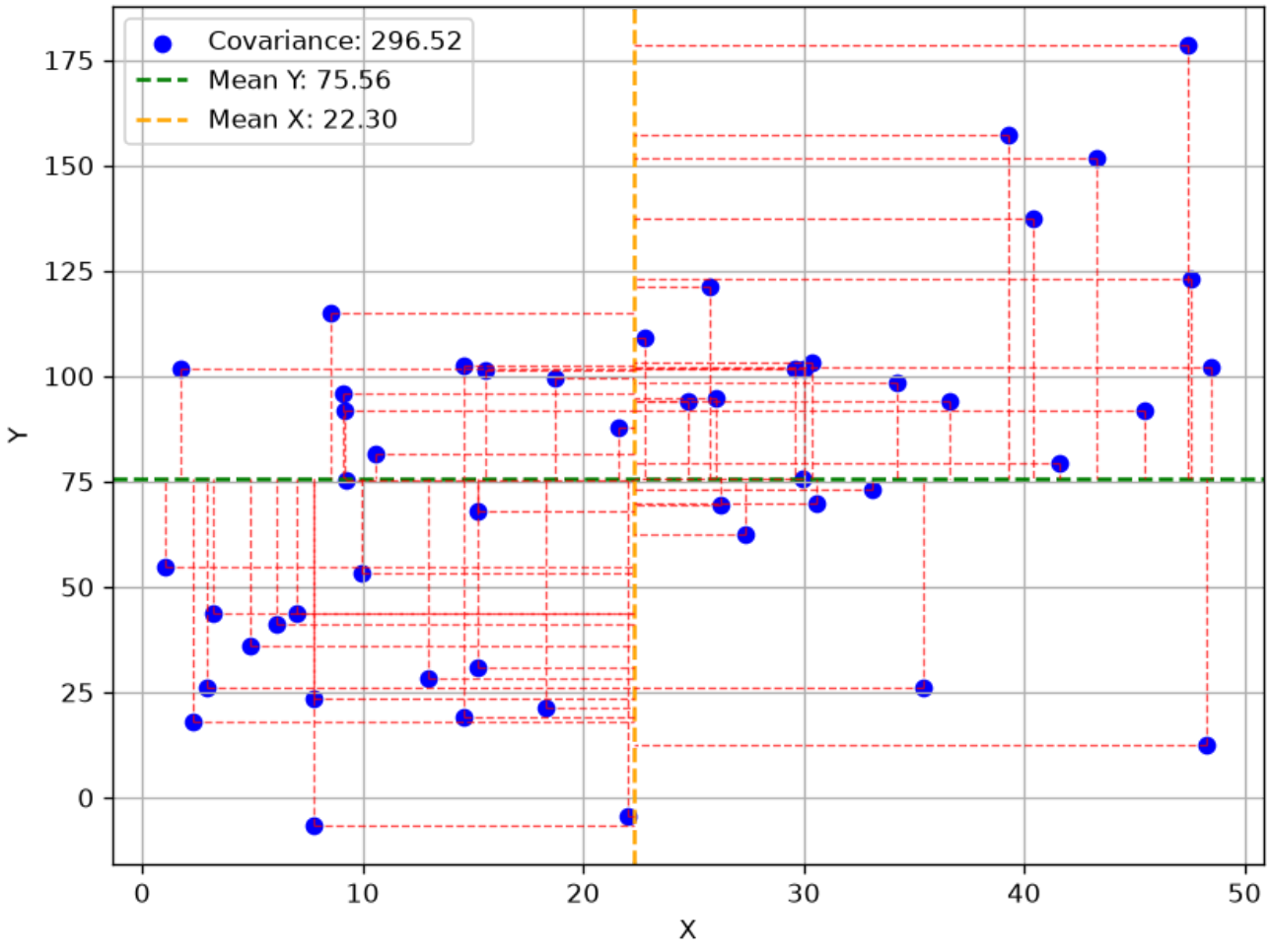
Each term of the sum in the covariance formula performs the products between these two terms:

$$(x^{(i)} - \overline{x})(y^{(i)} - \overline{y})$$

Which will be:

- **positive** if the elements $x^{(i)}$ and $y^{(i)}$ are "concordant", i.e., if they are both above their respective means or both below their respective means;
- **negative** if the elements $x^{(i)}$ and $y^{(i)}$ are "discordant", i.e., if one of the two is above its respective mean, while the other is below its respective mean;
- **zero or close to zero** if the elements $x^{(i)}$ and $y^{(i)}$ are very close to their respective means.

These quantities have a geometric interpretation. Let's consider the following plot:



The graph shows a sample of points and their respective means with a horizontal line (for Y) and a vertical line (for X). The two means divide the graph into four quadrants. For each point, the red dashed lines indicate the values of the magnitudes $(x^{(i)} - \overline{x})$ and $(y^{(i)} - \overline{y})$. Specifically:

- The points in the upper right quadrant will have both these magnitudes positive. The products $(x^{(i)} - \overline{x})(y^{(i)} - \overline{y})$ will be positive.
- The points in the lower right quadrant will have both these magnitudes negative. The products $(x^{(i)} - \overline{x})(y^{(i)} - \overline{y})$ will be positive.
- The points in the other two quadrants will have one of the magnitudes positive and the other negative. The products $(x^{(i)} - \overline{x})(y^{(i)} - \overline{y})$ will be negative.

In practice:

- points in the first and third quadrants contribute **positively** to the correlation coefficient;
- points in the second and fourth quadrants contribute **negatively** to the correlation coefficient.
- Points close to the mean (the intersection between the two means) will correspond to very small magnitudes and **will not contribute** to changing the covariance value.

Intuitively, the covariance will take on very positive values when X and Y vary in agreement (i.e., if X takes high values, Y takes high values, and if X takes low values, Y takes low values). Indeed, in such a case, the signs of the two factors within the summation will agree, and their product will have a positive sign. If, conversely, X and Y vary in disagreement, the product will have a negative sign, and the covariance will take on negative values.

We note that the covariance of a variable X with itself is equal to the variance:

$$Cov(X, X) = \frac{1}{n}\sum_{i}^{n}(x_i - \overline{x})(x_i - \overline{x}) = s_X$$

It should be noted that:

- Negative values indicate a negative correlation (when one of the two variables increases, the other decreases);
- Positive values indicate a positive correlation (the two variables increase or decrease together);
- Zero values (or close to zero) indicate that the two variables are not correlated (or are weakly correlated).

It should also be noted that covariance values are not normalized and depend on the ranges of the individual variables, therefore it is not possible to compare covariances. For example, the covariance between `relwt` and itself (thus the variance of `relwt`) is close to zero; however, this does not mean that `relwt` is not correlated with itself!

In practice, in the presence of multiple variables, the covariances of all possible pairs of variables are calculated, somewhat as seen in the case of the scatter matrix. We can obtain all these covariances with the cov function, as follows:

```
data.drop('group',axis=1).cov() #drop the group column to compute covariance only on
        numerical columns
```

	relwt	glufast	glutest	instest	sspg
relwt	0.016702	-0.072815	0.982426	3.473373	5.266255
glufast	-0.072815	4087.097031	19546.064080	-3063.463649	4849.905651
glutest	0.982426	19546.064080	100457.849808	-12918.162739	25908.490182
instest	3.473373	-3063.463649	-12918.162739	14625.312548	101.482519
sspg	5.266255	4849.905651	25908.490182	101.482519	11242.331897

The matrix seen above is also called "covariance matrix" and denoted by the letter Σ. Given the variables $X = (X_i, \dots, X_k)$, Σ is a $[k \times k]$ matrix whose general term is:

$$\Sigma_{ij} = Cov(X_i, X_j)$$

Pearson Correlation Coefficient

The Pearson correlation coefficient attempts to solve the problems of covariance by providing a normalized quantitative score. Given a bivariate sample $\{(x^{(i)}, y^{(i)})\}$, the Pearson correlation coefficient is defined as the covariance between the two variables, divided by the product of their standard deviations:

$$\rho(x, y) = \frac{Cov(X, Y)}{s_X s_Y}$$

We note that:

$$\rho(x, y) = \frac{Cov(X, Y)}{s_X s_Y} = \frac{1}{n} \sum \frac{(x_i - \overline{x})}{s_X} \frac{(y_i - \overline{y})}{s_X} = Cov(z(X), z(Y))$$

Where

$$z(X) = \frac{X - \overline{X}}{s_x}$$

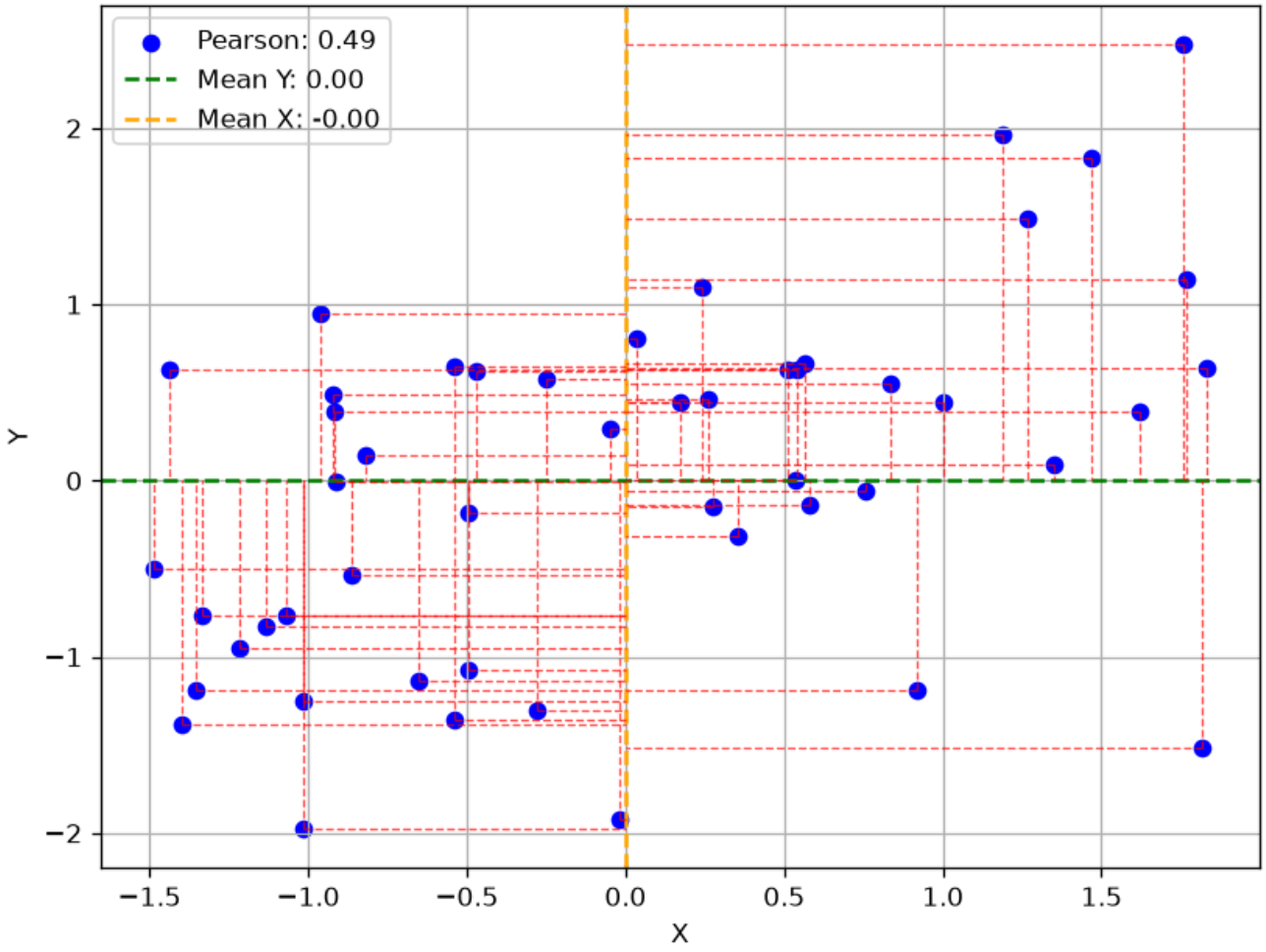
is the z-scoring function.

We can therefore view the Pearson correlation coefficient as the covariance of variables normalized with z-scoring. It can be easily seen that the Pearson correlation coefficient can also be written as follows:

$$\rho(x, y) = \frac{\sum_{i=1}^{n}(x^{(i)} - \overline{x})(y^{(i)} - \overline{y})}{\sqrt{\sum_{i=1}^{n}(x^{(i)} - \overline{x})^2}\sqrt{\sum_{i=1}^{n}(y^{(i)} - \overline{y})^2}}$$

Where $\overline{x}$ and $\overline{y}$ are respectively the means of the values of the two variables.

The following graph shows a plot analogous to the previous one after performing z-scoring. Note how the ranges of x and y are now normalized and the mean is at the origin.



Pearson's correlation coefficient is a number between -1 (maximum anticorrelation) and 1 (maximum correlation).

In practice, you will have:

- $\rho(x, -x) = -1$
- $\rho(x, x) = 1$

The correlation coefficient between the variables `relwt` and `sspg` can be computed with the pearsonr function:

```
from scipy.stats import pearsonr
pearsonr(data['relwt'], data['sspg'])
```

```
PearsonRResult(statistic=np.float64(0.38431980355592876),
pvalue=np.float64(1.820843846790297e-06))
```

This function returns both the statistic (0.38) and a p-value, associated to a statistical test, as we will see in a future lecture.

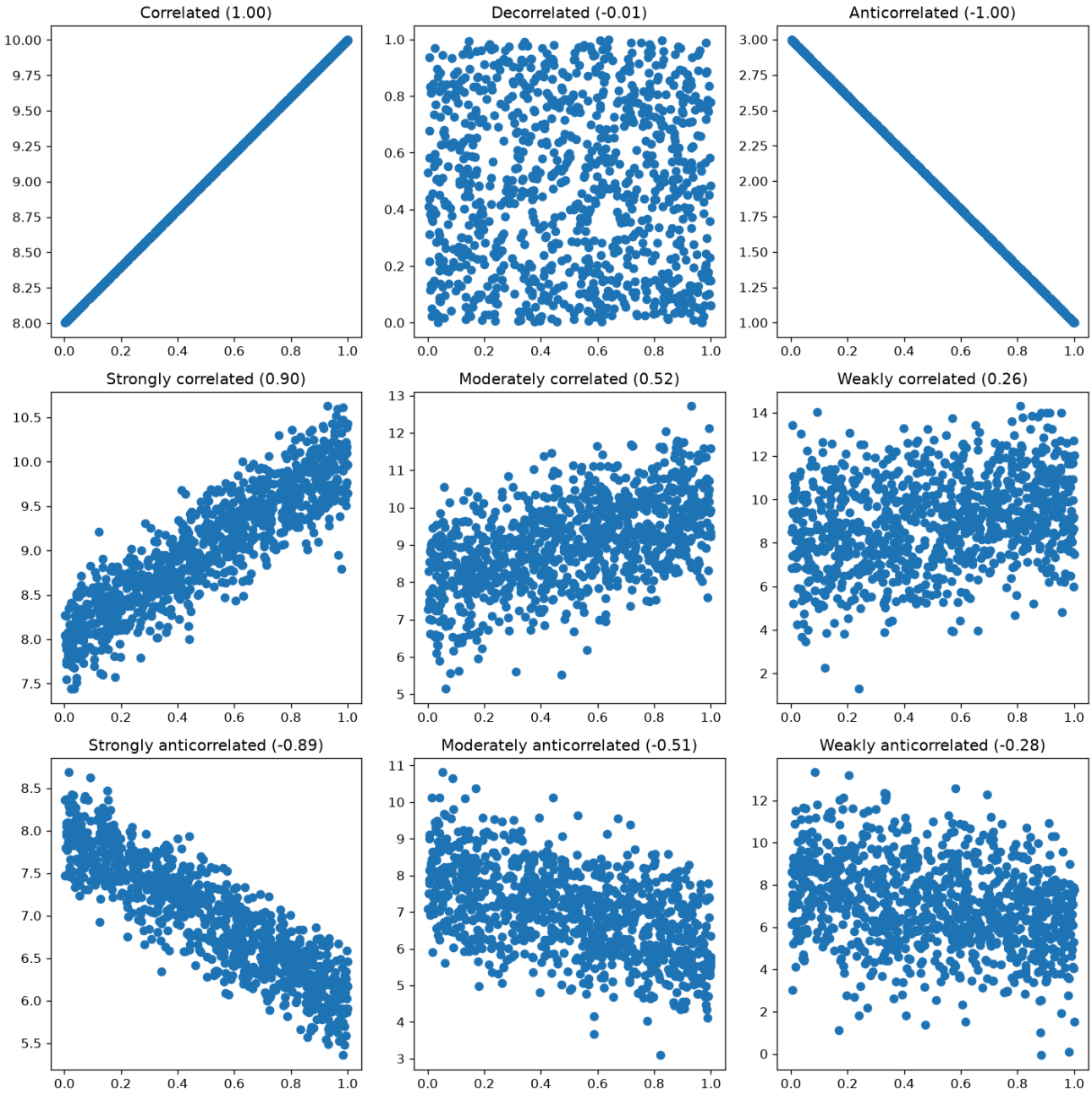
Interpretation of Pearson's Correlation Coefficient Values

Pearson's correlation coefficient is a number between -1 and 1 which indicates whether there is a correlation (i.e., when the values of the first variable increase, the values of the second variable also increase) or an anticorrelation (i.e., when the values of the first variable increase, the values of the second variable decrease and vice versa). Values close to -1 indicate that the variables are anticorrelated; values close to 1 indicate that the variables are correlated; values close to 0 indicate that the variables are decorrelated. In practice:

- The sign of the coefficient indicates the "direction" of the correlation:
  - Positive: the variables are correlated;
  - Negative: the variables are anticorrelated;
- The absolute value of the coefficient indicates how strong the correlation (or anticorrelation) is:
  - for values between 0 and 0.3 it indicates a **weak** correlation (or anticorrelation);
  - for values between 0.3 and 0.7 it indicates a **moderate** correlation (or anticorrelation);
  - for values between 0.7 and 1 it indicates a **strong** correlation (or anticorrelation).

Note that Pearson's correlation coefficient only measures **linear correlations**. Therefore, a non-linear correlation might exist between two variables even if the obtained correlation coefficient is low.

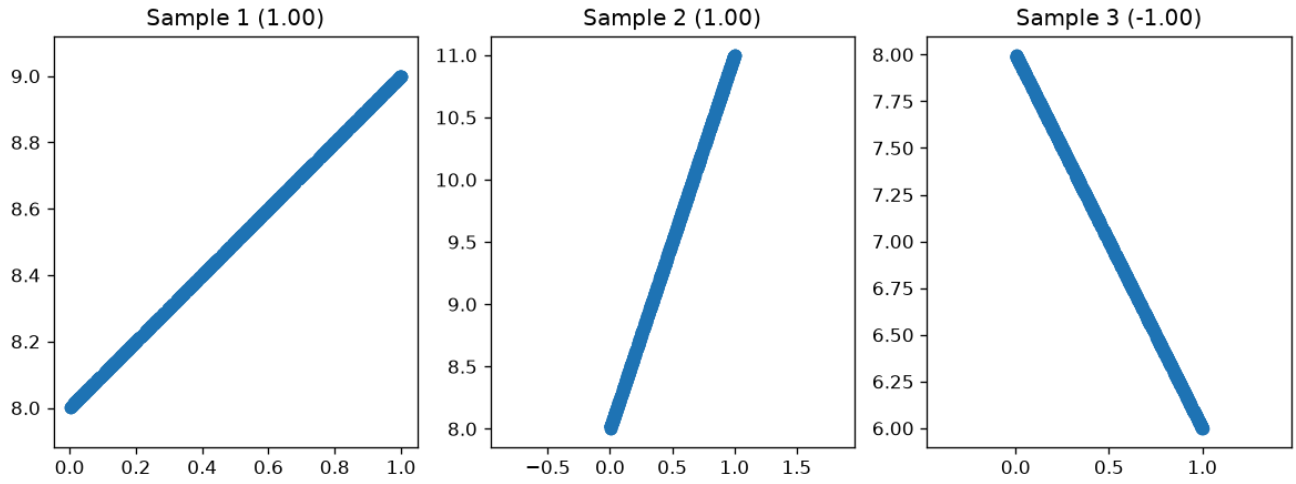
The following graph shows the scatterplots of some example bivariate samples with different degrees of correlation, together with their respective Pearson correlation coefficients:



Observing the graphs shown above, we can note that the following relationships exist between index and scatterplot:

- A positive index identifies an upward trend (a line with a positive slope);
- A negative index identifies a downward trend (a line with a negative slope);
- The absolute value of the index is correlated with the “width” of the “corridor” formed by the data.

Note that there is no relationship between Pearson’s correlation coefficient and the slope of the line. We can see this in the following graph:



Pearson Correlation Coefficient and Discrete Variables

It's important to remember that the **Pearson correlation coefficient is designed to measure the linear relationship between two continuous variables**. It is not suitable for nominal categorical variables (e.g., "City" or "Favorite Color"), as the categories have no inherent numerical order.

However, a notable exception exists for the specific case where you have **one continuous variable and one dichotomous** variable (a discrete variable with only two categories, like "Yes/No" or "Survived/Died"). In this scenario, you can convert the two categories into the numbers `0` and `1`. By applying the standard Pearson formula to this data, you calculate what is known as the **point-biserial correlation coefficient (r_{pb})**.

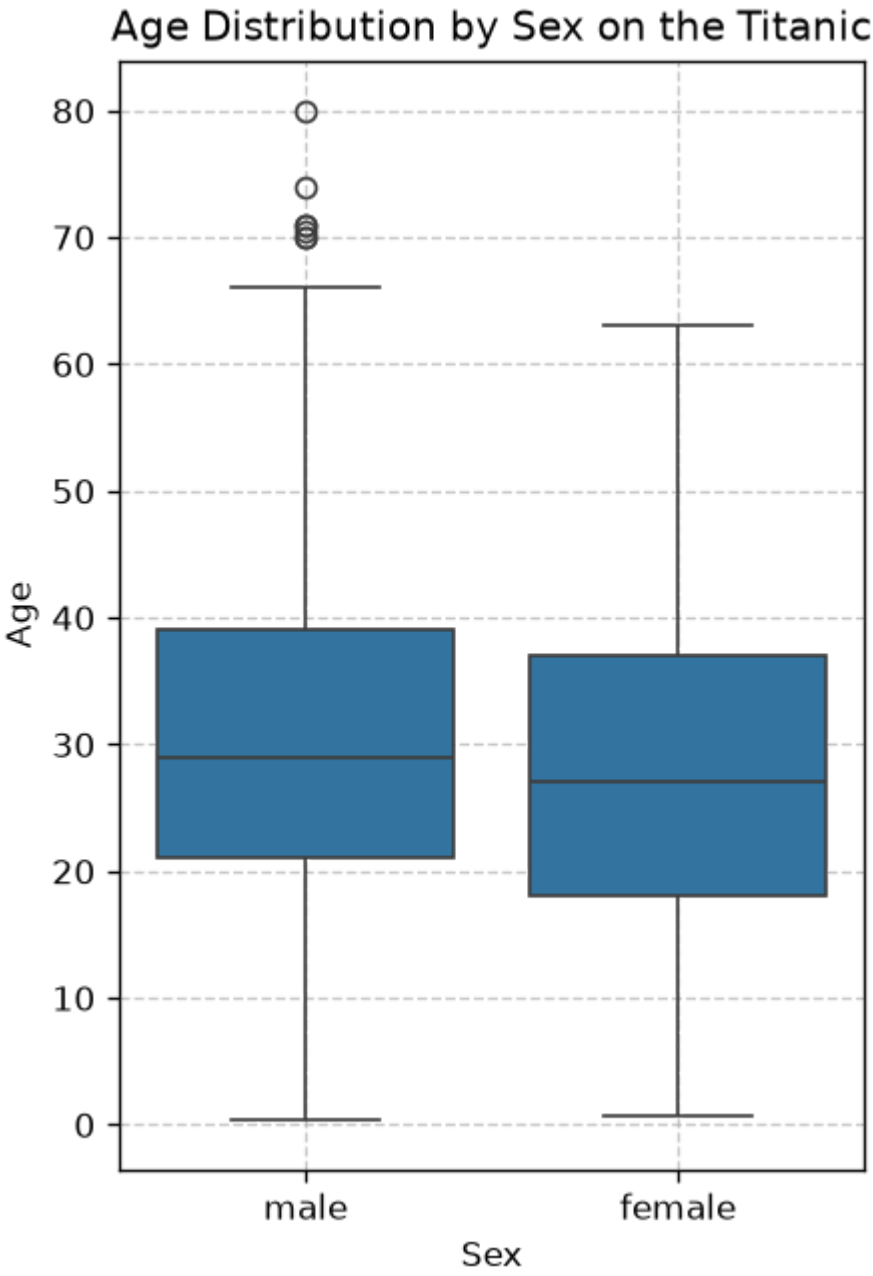
The example below shows how to compute the point-biserial correlation coefficient between `Sex` and `Age`:

```python
import pandas as pd
from scipy.stats import pointbiserialr, pearsonr

# Load the dataset
titanic = pd.read_csv(DATA + "titanic.csv")

# Drop rows where 'Age' is missing to perform the correlation
titanic_cleaned = titanic.dropna(subset=['Age'])

# Extract the two variables of interest
age = titanic_cleaned['Age']
survived = titanic_cleaned['Sex'].replace({'male':'1','female':'0'}).astype(int)

# --- Method 1: Using the specific pointbiserialr function ---
pbr_result = pointbiserialr(age, survived)
print(f"Point-Biserial Correlation: {pbr_result.correlation:.4f}")

# --- Method 2: Verifying with the standard Pearson correlation function ---
# The result should be identical
pearson_result = pearsonr(age, survived)
print(f"Pearson Correlation (on the same data): {pearson_result[0]:.4f}")
```

```
Point-Biserial Correlation: 0.0933
Pearson Correlation (on the same data): 0.0933
```

We see a small positive association: males tend to be older. This is also confirmed by the boxplot below:

```python
import pandas as pd
import seaborn as sns
from matplotlib import pyplot as plt

# Set up the plot
plt.figure(figsize=(4, 6))

# Create boxplots to compare the age distribution by sex
sns.boxplot(data=titanic_cleaned, x='Sex', y='Age')

# Add titles and labels for clarity
plt.title('Age Distribution by Sex on the Titanic')
plt.xlabel('Sex')
plt.ylabel('Age')
plt.grid(linestyle='--', alpha=0.7)

# Display the plot
plt.show()
```



Spearman's Rank Correlation Coefficient

Pearson's coefficient captures linear relationships between variables. However, there are cases where the relationship between two variables may not be linear, but it still makes sense to try and understand if a correlation exists.

Imagine we have a series of users who assign scores to products they review. Each user will use a different and probably non-linear subjective scale. Pearson's correlation coefficient would not be the most appropriate to check if the scores assigned by two users are correlated.

Spearman's correlation coefficient tries to solve this problem by moving from scores to ranks. Instead of reasoning about individual scores, Spearman's correlation coefficient orders the products by score and checks if the resulting orderings are similar.

Consider the sample $\{(x^{(i)}, y^{(i)})\}_i$. Let $R(x^{(i)})$ and $R(y^{(i)})$ be the ranks associated with the values of the two variables, and let $d_i = R(x^{(i)}) - R(y^{(i)})$. Spearman's rank correlation coefficient is defined as follows:

$$R = 1 - \frac{6 \sum_{i=1}^{n} d_i^2}{n(n^2 - 1)}$$

The values of Spearman's coefficient are normalized between -1 and $+1$ and have a similar interpretation to Pearson's coefficient.

Consider the following example. We ask Alice, Bob, Charlie, and David to assign a score from 1 to 5 to four ice cream flavors, obtaining these results:

```
import pandas as pd
from scipy.stats import spearmanr

# Create a DataFrame with ice cream flavor scores
data = {
    'Participant': ['Alice', 'Bob', 'Charlie', 'David'],
    'Chocolate': [5, 3, 4, 2],   # Scores out of 5
    'Vanilla': [4, 5, 3, 3],      # Scores out of 5
    'Strawberry': [3, 4, 5, 2],   # Scores out of 5
    'MintChip': [2, 2, 2, 5]      # Scores out of 5
}

df = pd.DataFrame(data)
df
```

	Participant	Chocolate	Vanilla	Strawberry	MintChip
0	Alice	5	4	3	2
1	Bob	3	5	4	2
2	Charlie	4	3	5	2
3	David	2	3	2	5

Given that each child can use a different scale, we use Spearman's coefficient, obtaining the following values:

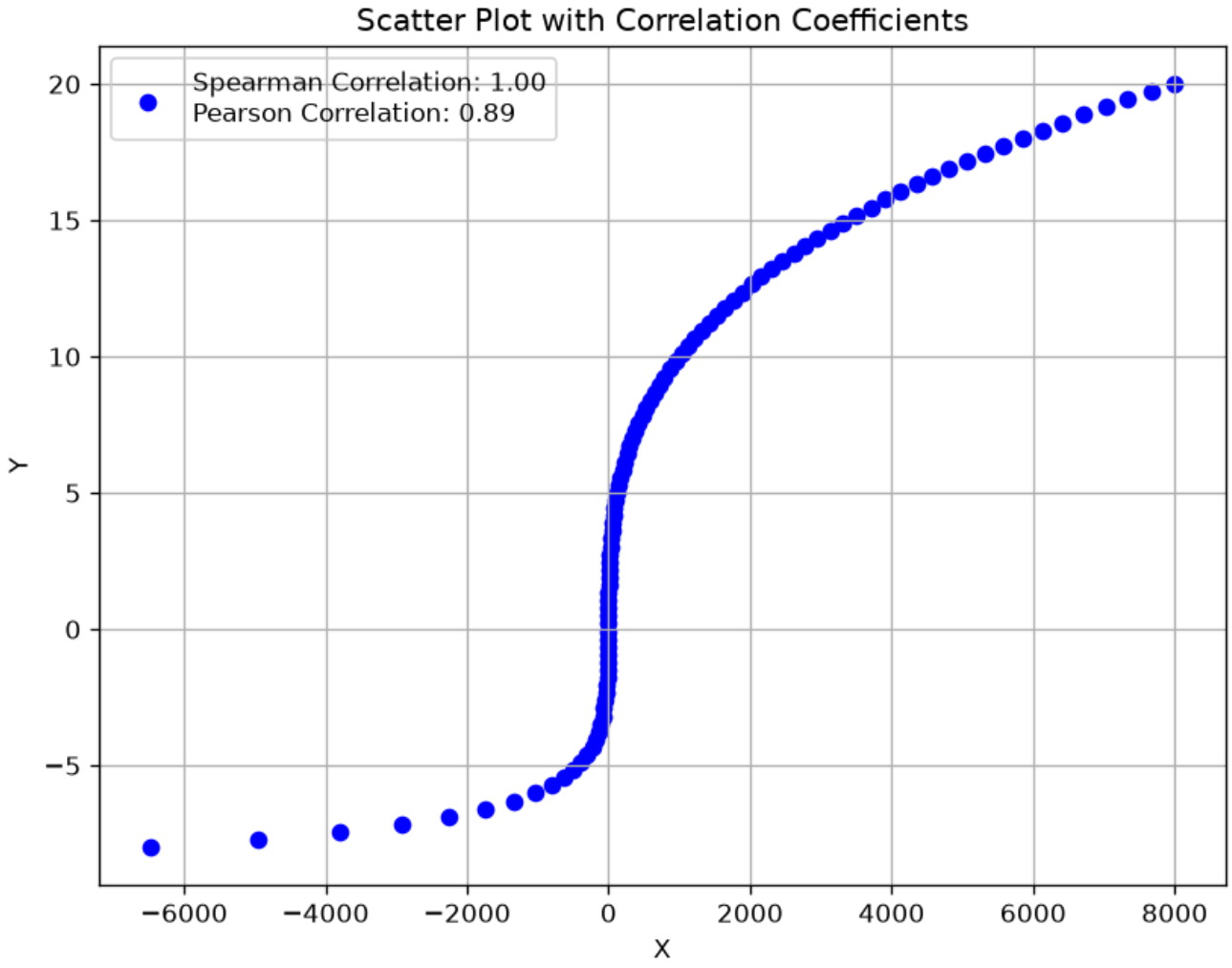
```
# Calculate the Spearman correlation coefficients
spearman_corr_chocolate_vs_vanilla, _ = spearmanr(df['Chocolate'], df['Vanilla'])
spearman_corr_chocolate_vs_strawberry, _ = spearmanr(df['Chocolate'],
        df['Strawberry'])
spearman_corr_chocolate_vs_mintchip, _ = spearmanr(df['Chocolate'], df['MintChip'])

# Calculate and display the Spearman correlation coefficients
print(f"Spearman Correlation (Chocolate vs. Vanilla):
        {spearman_corr_chocolate_vs_vanilla:.2f}")
print(f"Spearman Correlation (Chocolate vs. Strawberry):
        {spearman_corr_chocolate_vs_strawberry:.2f}")
print(f"Spearman Correlation (Chocolate vs. MintChip):
        {spearman_corr_chocolate_vs_mintchip:.2f}")
```

```
Spearman Correlation (Chocolate vs. Vanilla): 0.21
Spearman Correlation (Chocolate vs. Strawberry): 0.40
Spearman Correlation (Chocolate vs. MintChip): -0.77
```

Spearman correlation values are computed using the `spearmanr` function.

These values tell us that the ice cream flavor that received scores “most similar” to chocolate is strawberry.

Unlike the Pearson coefficient, Spearman’s index can capture non-linear relationships, as shown in the following example:



Kendall's Correlation Coefficient

Kendall's rank correlation coefficient, also known as **Kendall's τ**, is also used to measure the level of association between two variables, considering their ranks.

Let's consider the sample $\{(x^{(i)}, y^{(i)})\}_i$. Each pair of observations (x_i, y_i) and (x_j, y_j), with $i < j$ is called:

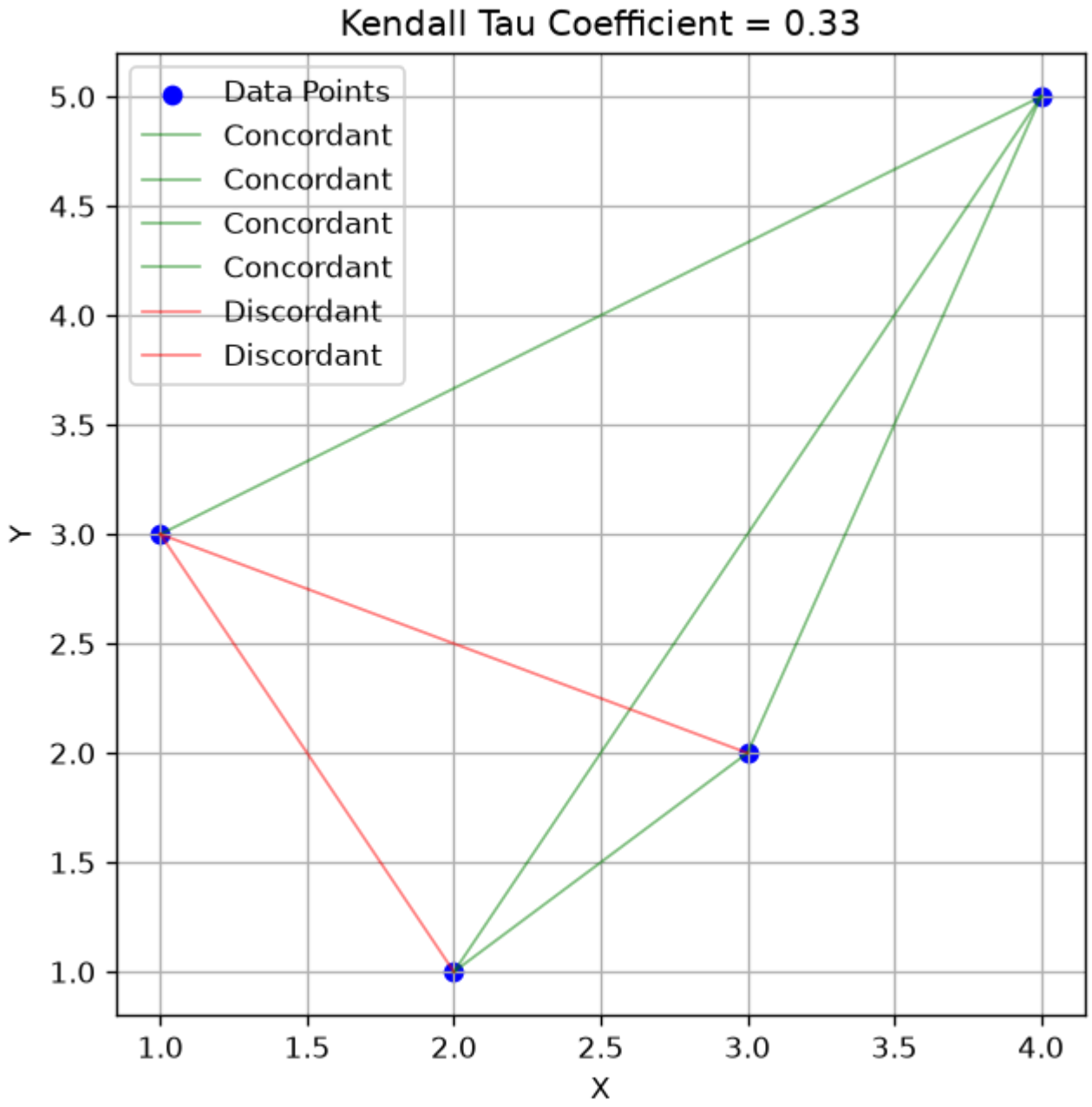
- **concordant** if: $x_i > x_j$ and $y_i > y_j$ or $x_i < x_j$ and $y_i < y_j$;
- **discordant** if: $x_i < x_j$ and $y_i > y_j$ or $x_i > x_j$ and $y_i < y_j$;

Kendall's τ coefficient is defined as follows:

$$\tau = \frac{\text{concordant pairs} - \text{discordant pairs}}{\text{pairs}}$$

The following graph shows a sample with six points. Each line shows a possible pair of points. The green lines show the concordant pairs, while the red ones show the discordant pairs. Kendall's coefficient for this example is given by:

$$\tau = \frac{4-2}{6} = \frac{1}{3}$$



The kendall correlation coefficients for our earlier examples can be computed as follows:

```
from scipy.stats import kendalltau

# Calculate the Kendall correlation coefficients
kendall_corr_chocolate_vs_vanilla, _ = kendalltau(df['Chocolate'], df['Vanilla'])
kendall_corr_chocolate_vs_strawberry, _ = kendalltau(df['Chocolate'],
        df['Strawberry'])
kendall_corr_chocolate_vs_mintchip, _ = kendalltau(df['Chocolate'], df['MintChip'])

# Display the Kendall correlation coefficients
print(f"Kendall Correlation (Chocolate vs. Vanilla):
        {kendall_corr_chocolate_vs_vanilla:.2f}")
print(f"Kendall Correlation (Chocolate vs. Strawberry):
        {kendall_corr_chocolate_vs_strawberry:.2f}")
print(f"Kendall Correlation (Chocolate vs. MintChip):
        {kendall_corr_chocolate_vs_mintchip:.2f}")
```

```
Kendall Correlation (Chocolate vs. Vanilla): 0.18
Kendall Correlation (Chocolate vs. Strawberry): 0.33
Kendall Correlation (Chocolate vs. MintChip): -0.71
```

Correlation Matrix

Similarly to what was seen in the case of scatterplots, when we have multiple variables, we can calculate the correlation indices between all variables in the dataset. In this way, we obtain a "correlation matrix". We can compute a correlation matrix with the `corr` function. The correlation matrix calculated using Pearson's coefficient, in the case of our `diabetes` dataset, will be:

```
from statsmodels.datasets import get_rdataset
data = get_rdataset('Diabetes','heplots').data
data.drop('group',axis=1).corr()
```

	relwt	glufast	glutest	instest	sspg
relwt	1.000000	-0.008813	0.023984	0.222238	0.384320
glufast	-0.008813	1.000000	0.964628	-0.396235	0.715480
glutest	0.023984	0.964628	1.000000	-0.337020	0.770942
instest	0.222238	-0.396235	-0.337020	1.000000	0.007914
sspg	0.384320	0.715480	0.770942	0.007914	1.000000

We can specify spearman or kendall coefficient as follows:

```
data.drop('group',axis=1).corr(method='spearman')
```

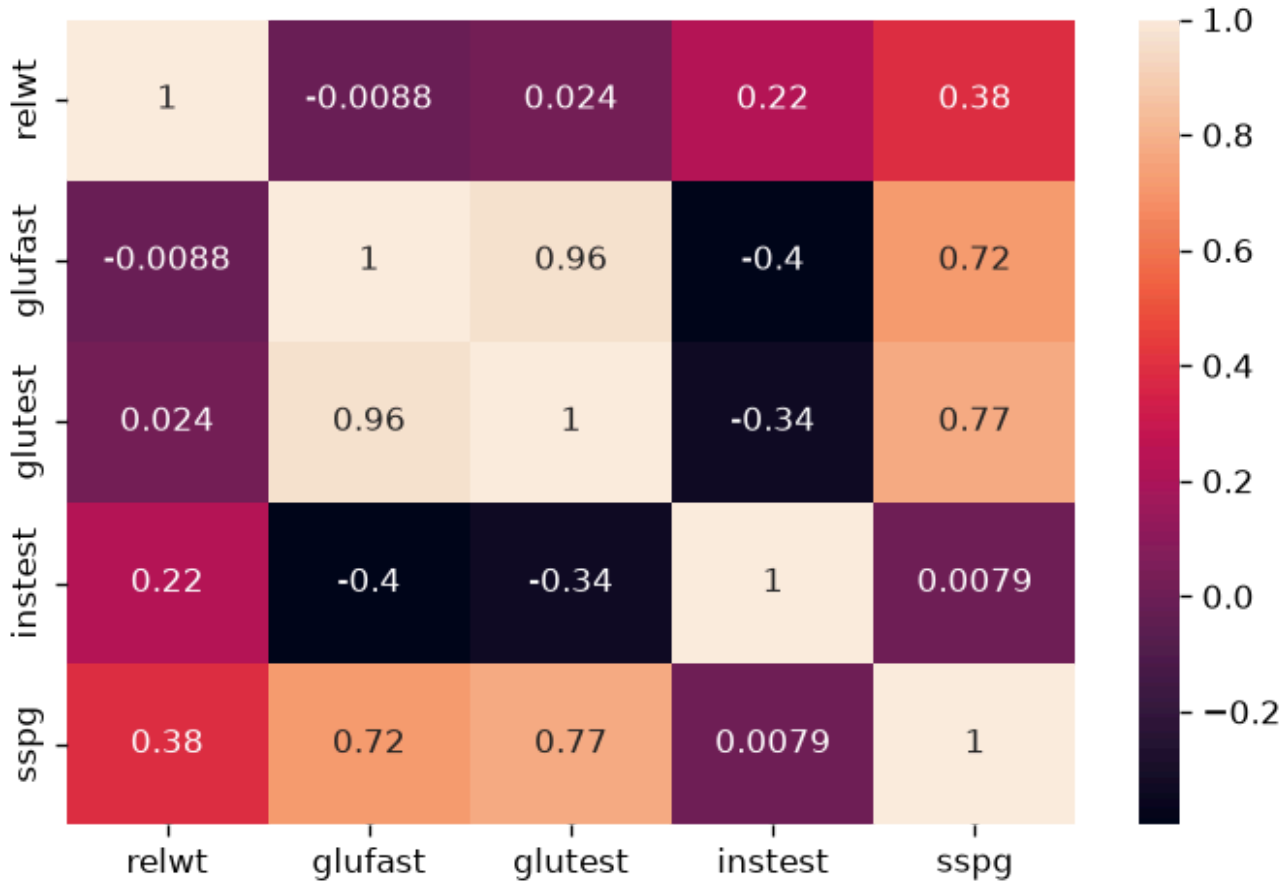
	relwt	glufast	glutest	instest	sspg
relwt	1.000000	0.204332	0.244952	0.231419	0.421998
glufast	0.204332	1.000000	0.767299	-0.265128	0.615134
glutest	0.244952	0.767299	1.000000	-0.164513	0.747564
instest	0.231419	-0.265128	-0.164513	1.000000	0.019898
sspg	0.421998	0.615134	0.747564	0.019898	1.000000

```
data.drop('group',axis=1).corr(method='kendall')
```

	relwt	glufast	glutest	instest	sspg
relwt	1.000000	0.137786	0.162601	0.158357	0.295005
glufast	0.137786	1.000000	0.608882	-0.175293	0.452355
glutest	0.162601	0.608882	1.000000	-0.091613	0.552304
instest	0.158357	-0.175293	-0.091613	1.000000	0.051423
sspg	0.295005	0.452355	0.552304	0.051423	1.000000

There is also a graphical representation of the correlation matrix, often called “correlation plot”:

```
sns.heatmap(data.drop('group',axis=1).corr(), annot=True)
plt.show()
```



Additional Measures of Correlation Between Discrete Variables (Optional)

This section reports optional material on two measures of correlation between discrete variables which are often used in epidemiology and biostatistics.

Relative Risk

Relative risk is one of the most common methods for measuring the degree of correlation between two discrete variables in epidemiology.

Let's look at an example (from here): suppose that in a village of 1000 people, the incidence of a rare disease has increased. Investigating, we discover that recently a part of the population has been exposed to radiation.

The data relating to diseased and healthy subjects exposed or not exposed to the risk (the radiation) is summarized by the following contingency table that relates the "Diseased/Healthy" variable to the "Exposed/Non Exposed" one:

```
contingency = pd.DataFrame({
    "Exposed": [20, 380],
    "Non Exposed": [6,594]
}, index=["Diseased", "Healthy"])
contingency
```

	Exposed	Non Exposed
Diseased	20	6
Healthy	380	594

The table indicates the number of people who have contracted a given disease or not, in relation to whether they have been exposed to a certain risk or not (e.g., to a pollutant).

The risk of developing the disease if exposed can be calculated with the following probability:

$$P(Diseased|Exposed) = \frac{\#\left(Diseased, Exposed\right)}{\#\ Exposed} = \frac{20}{20+380} = 0.05$$

We see that we have a 5% risk of developing the disease if exposed to radiation. For example, among 100 people exposed to the risk, we expect to find 5 sick.

The risk calculated above, by itself, does not tell us much about the association between the variables "Exposed" and "Non Exposed". Let's now consider the probability of contracting the disease if not exposed to the risk:

$$P(Diseased|Non\ Exposed) = \frac{\#\left(Diseased, Non\ Exposed\right)}{\#\ Non\ Exposed} = \frac{6}{6+594} = 0.01$$

This value tells us that we have a risk of about 1% of contracting the disease if not exposed. In practice, out of 100 subjects not exposed to the risk, 1 gets sick.

It is useful to compare these two risk values. Indeed, although the 5% risk of getting sick among exposed subjects is not a very high number in absolute terms, it must also be said that the proportion of sick subjects not exposed to the risk is much lower (1%). We therefore define relative risk as the ratio between these two risks (or probabilities):

$$RR = \frac{P(Diseased|Exposed)}{P(Diseased|Non\ Exposed)} = \frac{\frac{\#\left(Diseased, Exposed\right)}{\#\ Exposed}}{\frac{\#\left(Diseased, Non\ Exposed\right)}{\#\ Non\ Exposed}} = \frac{0.05}{0.01} = 5$$

We interpret this value as follows:

> The proportion of subjects who contract the disease is 5 times greater among those who have been exposed to the risk, compared to the group of those who have not.

For example, if among 100 people not exposed to the risk, generally 10 get sick (a risk of 10%), then we expect that among 100 people exposed to the risk, as many as 50% will get sick!

Relative risk tells us nothing about the absolute risk (the proportion of people who get sick), but only about how observing subjects exposed to the risk, compared to observing subjects not exposed to the risk, can influence the proportion of sick subjects.

Generally:

- $RR = 1$ indicates that exposure to the risk is irrelevant in the development of the disease (the variables are independent);
- $RR > 1$ indicates that exposure to the risk is positively associated with the development of the disease (there are more sick people among subjects exposed to the risk);
- $RR < 1$ indicates that exposure to the risk is negatively associated with the development of the disease (there are fewer sick people among subjects exposed to the risk). In this case, it is said that the risk "protects" against the onset of the disease (useful if the "risk" is actually taking a drug).

Note that relative risk measures a correlation between variables, not a cause-and-effect relationship, so it is not always correct to say that in the presence of a relative risk greater than 1, exposure to the risk "causes" the disease. However, there are tools (causal analysis) that allow us to verify in which cases we can give this interpretation. These tools are fundamental for ascertaining whether a particular drug causes (or is co-responsible for) a recovery or whether a particular habit (e.g., smoking) causes (or is co-responsible for) the onset of a disease. We will see more examples later in the course.

Odds Ratio

Relative risk cannot always be calculated. In fact, in the previous case, we assumed we had access to all population data: for all 1000 inhabitants, we knew who had been exposed to the risk and who had not, and who had developed the disease and who had not. In many cases, we might not have all this data. For example, imagine that the incident occurred 50 years ago and that at the time only a random sample of 50% of the 1000 inhabitants of the village was interviewed. Suppose we obtain the following contingency table:

```
contingency2 = pd.DataFrame({
    "Exposed": [20, 10],
    "Non Exposed": [6,16]
}, index=["Diseased", "Healthy"])
contingency2
```

	Exposed	Non Exposed
Diseased	20	6
Healthy	10	16

Note that this case (that of sampling) will be the most frequent, as we often do not have access to the entire population, but only to a sample.

In this case, we cannot calculate the relative risk as the real numbers of people exposed and not exposed to risk, which were in the denominator of the risk calculation expressions, are not available. If we still apply the formula, we will get an estimate biased by our sampling:

$$RR = \frac{P(Diseased|Exposed)}{P(Diseased|Non\ Exposed)} = \frac{\frac{\#\ (Diseased, Exposed)}{\#\ Exposed}}{\frac{\#\ (Diseased, Non\ Exposed)}{\#\ Non\ Exposed}} = \frac{20/(20+10)}{6/(6+16)} = 2.45$$

Although we cannot calculate the relative risk as we do not know the absolute number of sick and exposed people, **we expect our sample to be a good representation of the population**, therefore we can reason in terms of "odds".

Odds are a probability concept widely used in betting, because they allow the bettor to easily calculate the payout to collect. For example, if team A's victory over team B in a soccer match is given as $5:1$, the bookmaker expects team A to be 5 times more likely to win than team B. If I bet 2 euros on team B and team B wins, then I will win $5 \times 2 = 10$ euros (I bet on a rare event). The odd in this case would be defined as:

$$odd = \frac{5}{1} = 5$$

Alternatively, I can reason in terms of probability. Given the scheme above, I expect that, if we played the match 100 times, then the probability of victories for team A would be equal to $P(A\ wins) = \frac{5}{5+1}$, while the probability of loss would be equal to $P(A\ loses) = \frac{1}{5+1}$. The odd can be defined in terms of these probabilities:

$$odd = \frac{P(A\ wins)}{P(A\ loses)} = \frac{\frac{5}{5+1}}{\frac{1}{5+1}} = 5$$

In general, the odd for an event E is the probability that the event occurs, divided by the probability that the event does not occur:

$$odd = \frac{P(E)}{1 - P(E)}$$

The odd is a concept similar to risk because it tells us how much greater the frequency of an event occurring (A wins) is compared to the frequency of the event not occurring. Let's apply this concept to our example. We define two odds (just as we defined two risks):

- The odds of getting sick if exposed: $\frac{P(Diseased|Exposed)}{P(Healthy|Exposed)} = \frac{\frac{20}{20+10}}{\frac{10}{20+10}} = \frac{20}{10} = 2$
- The odds of getting sick if not exposed: $\frac{P(Diseased|Non\ Exposed)}{P(Healthy|Non\ Exposed)} = \frac{\frac{6}{6+16}}{\frac{16}{6+16}} = \frac{6}{16} = 0.375$

It should be noted that in the formulas above, the absolute counts, i.e., those we do not trust given the sampling, “cancel out”.

The odds ratio will be defined as follows:

$$odd = \frac{\frac{P(Diseased|Exposed)}{P(Healthy|Exposed)}}{\frac{P(Diseased|Non\ Exposed)}{P(Healthy|Non\ Exposed)}} = \frac{2}{0.375} \approx 5.3$$

We note that the obtained odds ratio is very close to the previously calculated relative risk of 5. **In practice, it can be demonstrated that, if it is assumed that the disease is rare (rare-disease assumption), the odds ratio and the relative risk converge to similar values.** It is therefore a fairly common practice to make this assumption and calculate the odds ratio instead of the relative risk.

Relative Risk and Odds Ratio in Python

Let’s consider this contingency table:

```
import pandas as pd
contingency = pd.DataFrame({
    "Exposed": [20, 380],
    "Non Exposed": [6,594]
}, index=["Diseased", "Healthy"])
contingency
```

	Exposed	Non Exposed
Diseased	20	6
Healthy	380	594

We can calculate the relative risk as follows:

```
from scipy.stats.contingency import relative_risk
relative_risk(contingency['Exposed']
        ['Diseased'],contingency['Exposed'].sum(),contingency['Non Exposed']
        ['Diseased'], contingency['Non Exposed'].sum())
```

```
RelativeRiskResult(relative_risk=5.0, exposed_cases=20, exposed_total=400,
control_cases=6, control_total=600)
```

The function requires that the various necessary values be passed rather than the contingency matrix to avoid ambiguity.

The odds ratio is calculated as follows:

```
from scipy.stats.contingency import odds_ratio
odds_ratio(contingency)
```

```
OddsRatioResult(statistic=5.202179487843029)
```

Exercises

Exercise 1

Considering the Titanic dataset, calculate for each variable the count, mean, standard deviation, minimum, maximum, median value, first and third quartile. After calculating the requested values individually, use the `describe` method to obtain these values. Which variable is the most dispersed?

Exercise 2

Considering the Titanic dataset, for each of the three classes, calculate the means and variances of passenger ages. In which class are the ages least dispersed? Which class contains the youngest individuals? Complete the analysis with bar charts.

Exercise 3

Add a new column “Old” to the Titanic dataset, which takes a value of 1 for observations with an age greater than the average value and 0 otherwise. Then, calculate a crosstab that computes the number of “elderly” subjects (subjects for whom old is equal to 1) with respect to the Pclass and Sex variables. Display a barplot based on the table. In which class do men tend to be much older than women?

Exercise 4

Load the mpg dataset using the seaborn library (use the instruction `t=sns.load_dataset('mpg')`). Visualize the scatter matrix by differentiating the colors of the points according to the values of the `origin` variable. Who produces the heaviest cars? Are there pairs of variables that are a decisive factor in distinguishing one origin from the others?

Exercise 5

Considering the `mpg` dataset, choose a group of variables for which to obtain a parallel coordinates plot. What can we infer from the plot?

References

- Chapter 3 of: Heumann, Christian, and Michael Schomaker Shalabh. Introduction to statistics and data analysis. Springer International Publishing Switzerland, 2016.
- https://en.wikipedia.org/wiki/Odds_ratio

© 2026 Antonino Furnari · Department of Mathematics and Computer Science, University of Catania Built with Quarto

Credits and data sources

Data distributions

Chapter resources

Slides (PDF) Open in Colab Practice quiz

We have seen how it is possible to assign a probability value to a given outcome of a random variable.

In practice, it is often useful to assign probability values to **all the values** that the random variable can assume.

To do so, we can define a **function**, which we will call **probability distribution** which assigns a probability value to each of the possible values of a random variable.

In the case of discrete variables, we will talk about "**probability mass functions**", whereas in the case of continuous variables, we will refer to "**probability density functions**".

A probability distribution characterizes the random variable and defines which outcomes it is more likely to observe.

Once we find that a given random variable X is characterized by a probability distribution $P(X)$, we can say that **"X follows P"** and write:

$$X \sim P$$

Discrete Variables

We will start by focusing on discrete variables.

Probability Mass Functions (PMF)

If X is discrete, $P(X)$ is called a "probability mass function" (PMF).

P can be seen as a function mapping the values of X to real numbers indicating whether a given value is more or less likely.

A PMF on a random variable X is a function:

$$P : \Omega \rightarrow [0, 1]$$

which satisfies the following property:

$$\sum_{\mathbf{x} \in \mathbf{\Omega}} \mathbf{P}(\mathbf{x}) = \mathbf{1}$$

where Ω is the sample space X,

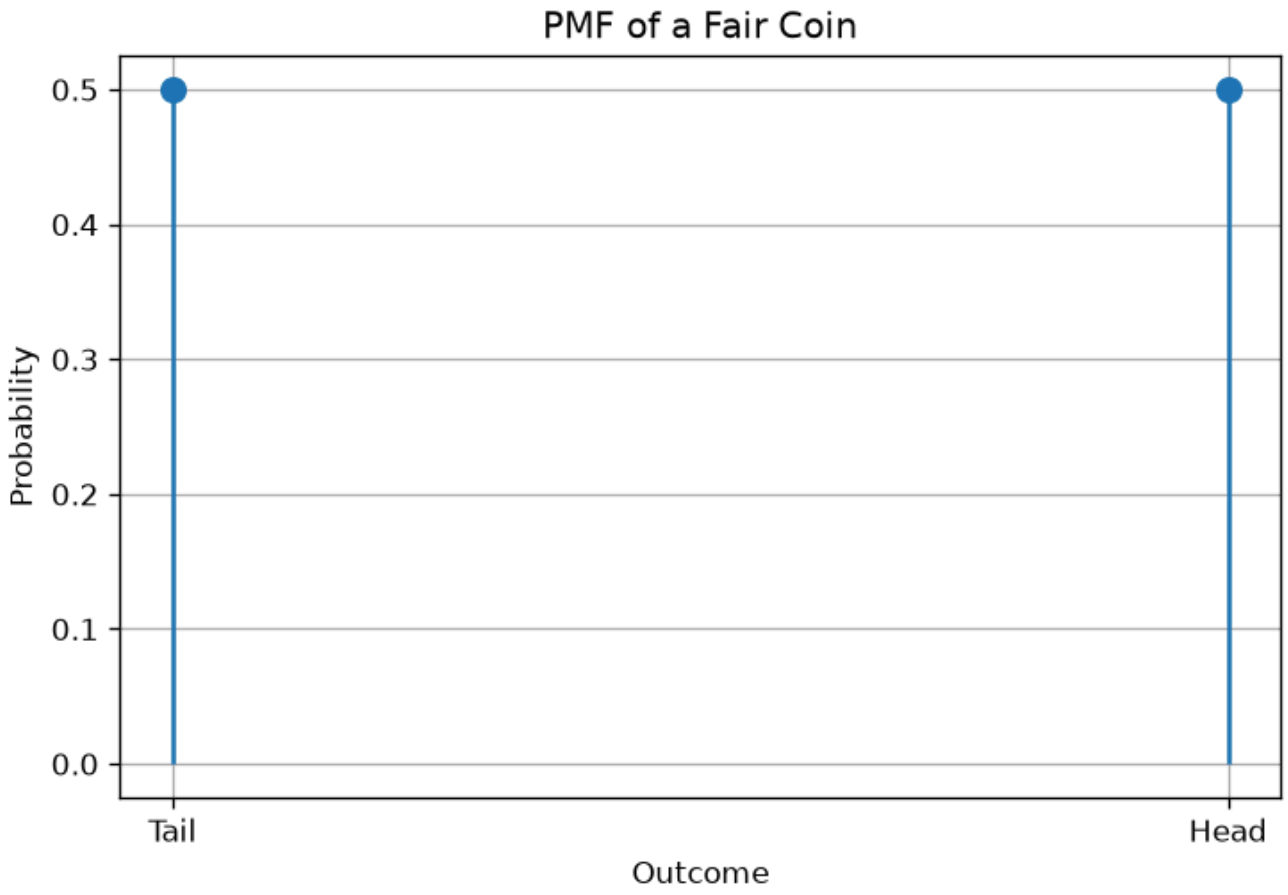
This condition implies that the probability distribution is normalized (and probability axioms are satisfied).

Example: Probability Mass Function for a Fair Coin

Let X be the random variable indicating the outcome of a coin toss.

- The space of all possible outcomes (the domain of $P(X)$) is $\{head,\ tail\}$.
- The probabilities $P(head)$ and $P(tail)$ must be l**arger than or equal to zero and smaller than or equal to 1**.
- Also, $P(head) + P(tail) = 1$. This is obvious, as one of the two outcomes will always happen. Indeed, if we had $P(tail) = 0.3$, this would mean that, 30 times out of 100 times we toss a coin, the outcome will be tail. What will happen in all other cases? The outcome will be head, hence, $P(head) = 0.7$, so $P(head) + P(tail) = 1$.
- In the case of a fair coin, we can characterize $P(X)$ as a **"discrete uniform distribution"**, i.e., a distribution which maps any value $x \in X$ to a constant, such that the properties of the probability mass functions are satisfied.
- If we have N possible outcomes, the discrete uniform probability will be $P(X = x) = \frac{1}{N}$, which means that all outcomes have the same probability.
- This definition satisfies the constraints. Indeed:
  - $\frac{1}{N} \geq 0,\ \forall N$
  - $\sum_i P\left(X = x_i\right) = 1$

A probability mass function can be visualized as a 2D diagram where the values of the function ($P(x)$) is plotted against the values of the independent variable x. This is the diagram associated to the PMF of the previous example, where $P(head) = P(tail) = 0.5$.



Example: Probability Mass Function for a Biased Coin

Now suppose we tossed our coin for 10000 times and discovered that 6000 times the outcome was "head", whereas 4000 times it was "tail". We deduce the coin is not fair.

Using a **frequentist** approach, we can manually assign values to our PMF using the general formula:

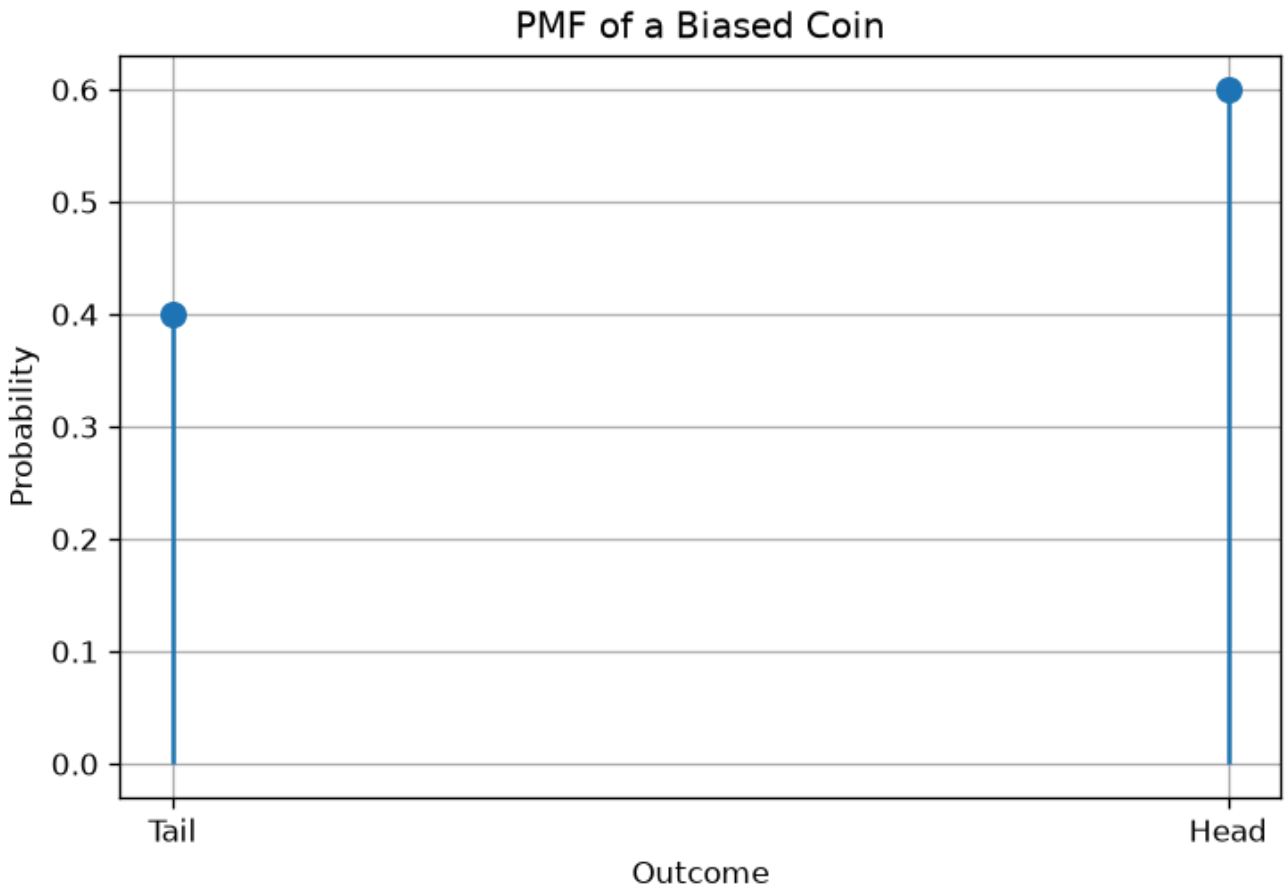
$$P(x) = \frac{\#trials\ in\ which\ X = x}{\#\ trials}$$

That is, in our case:

$$P(head) = \frac{6000}{10000} = 0.6; P(tail) = \frac{4000}{10000} = 0.4$$

We shall note that the probability we just defined satisfies all properties of probabilities, i.e.:

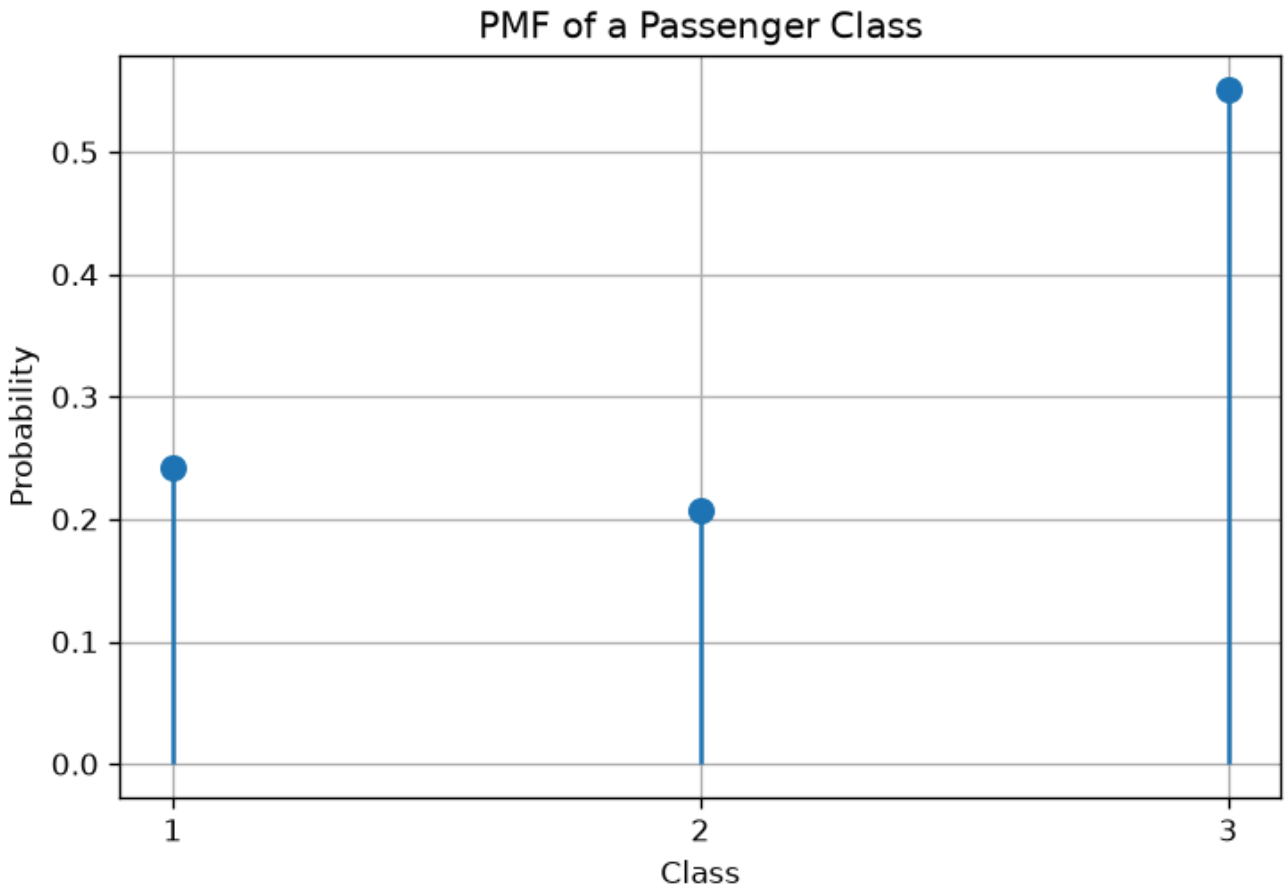
- $0 \leq P(x) \leq 1\ \forall x$
- $\sum_x P(x) = 1.$



Computing Probability Mass Functions from Data

In practice, Probability Mass Functions are computed exactly as relative frequencies. Let's see a quick example:

```
import pandas as pd
titanic = pd.read_csv(DATA + "titanic.csv",
                      index_col='PassengerId')
pclass = titanic['Pclass'].value_counts(normalize=True).sort_index()

plt.vlines(pclass.index.values, ymin=0, ymax=pclass.values,  linestyles='solid')
plt.plot(pclass.index.values, pclass.values, 'o', markersize=8)
plt.xlabel('Class')
plt.ylabel('Probability')
plt.title('PMF of a Passenger Class')
plt.xticks(pclass.index.values)
plt.grid(True)

# Show the plot
plt.show()
```



Exercise: Probability Mass Function

Let X be a random variable representing the outcome of rolling a fair dice with 6 faces:

- What is the space of possible values of X?
- What is its cardinality?
- What is the associated probability mass function $P(X)$?
- Suppose the dice is not fair and $P(X = 1) = 0.2$, whereas all other outcomes are equally probable. What is the probability mass function of $P(X)$?
- Draw the PMF obtained for the dice.

Cumulative Distribution Functions of Discrete Variables

If the discrete variable X can be ordered (i.e., it is at least ordinal), we can define the **Cumulative Distribution Function (CDF)** as:

$$F(x) = P(X \leq x) = \sum_{y \leq x} P(y)$$

The CDF records the cumulative probability of all values less than or equal to a given value. For example, if H measures the height of people in centimeters, $F(180)$ indicates the **probability of**

selecting a person with height at most 180 cm.

Example – PMF and CDF of a fair die

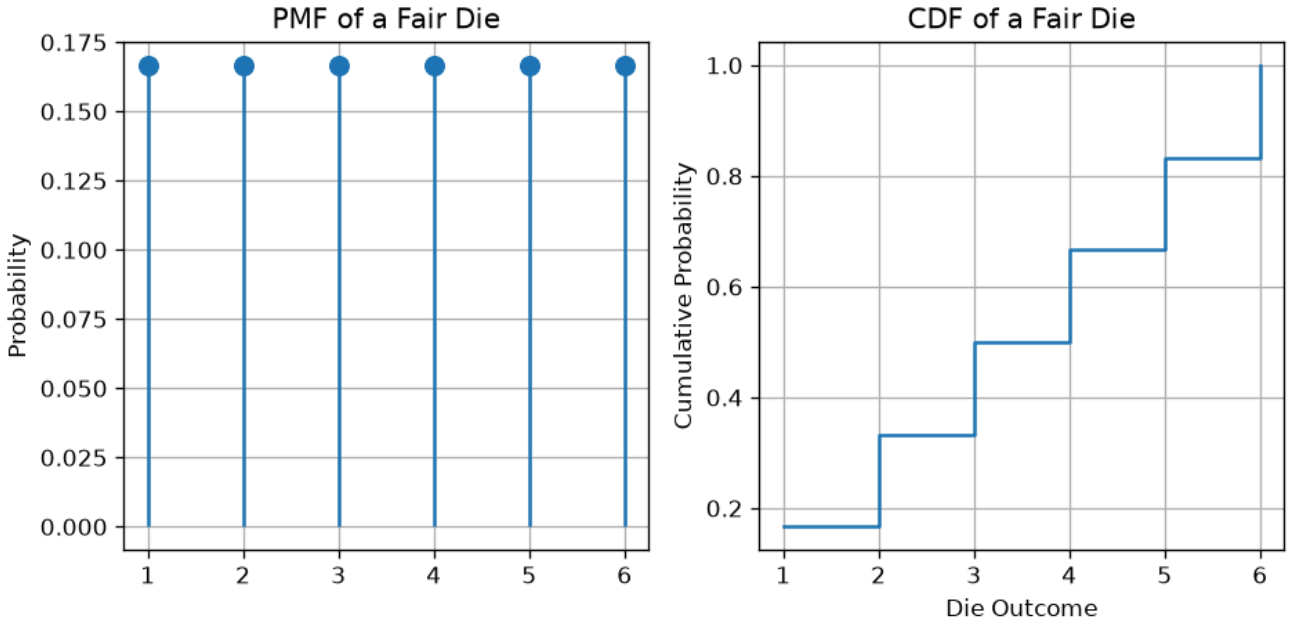
For a fair die, the PMF is:

$$f(x) = \begin{cases} \frac{1}{6} & \text{if } x \in \{1,2,3,4,5,6\} \\ 0 & \text{otherwise} \end{cases}$$

The CDF is:

$$F(x) = \begin{cases} 0 & \text{if } x < 1 \\ \frac{1}{6} & \text{if } 1 \leq x < 2 \\ \frac{2}{6} & \text{if } 2 \leq x < 3 \\ \frac{3}{6} & \text{if } 3 \leq x < 4 \\ \frac{4}{6} & \text{if } 4 \leq x < 5 \\ \frac{5}{6} & \text{if } 5 \leq x < 6 \\ 1 & \text{if } x \geq 6 \end{cases}$$

The diagram below shows an example:

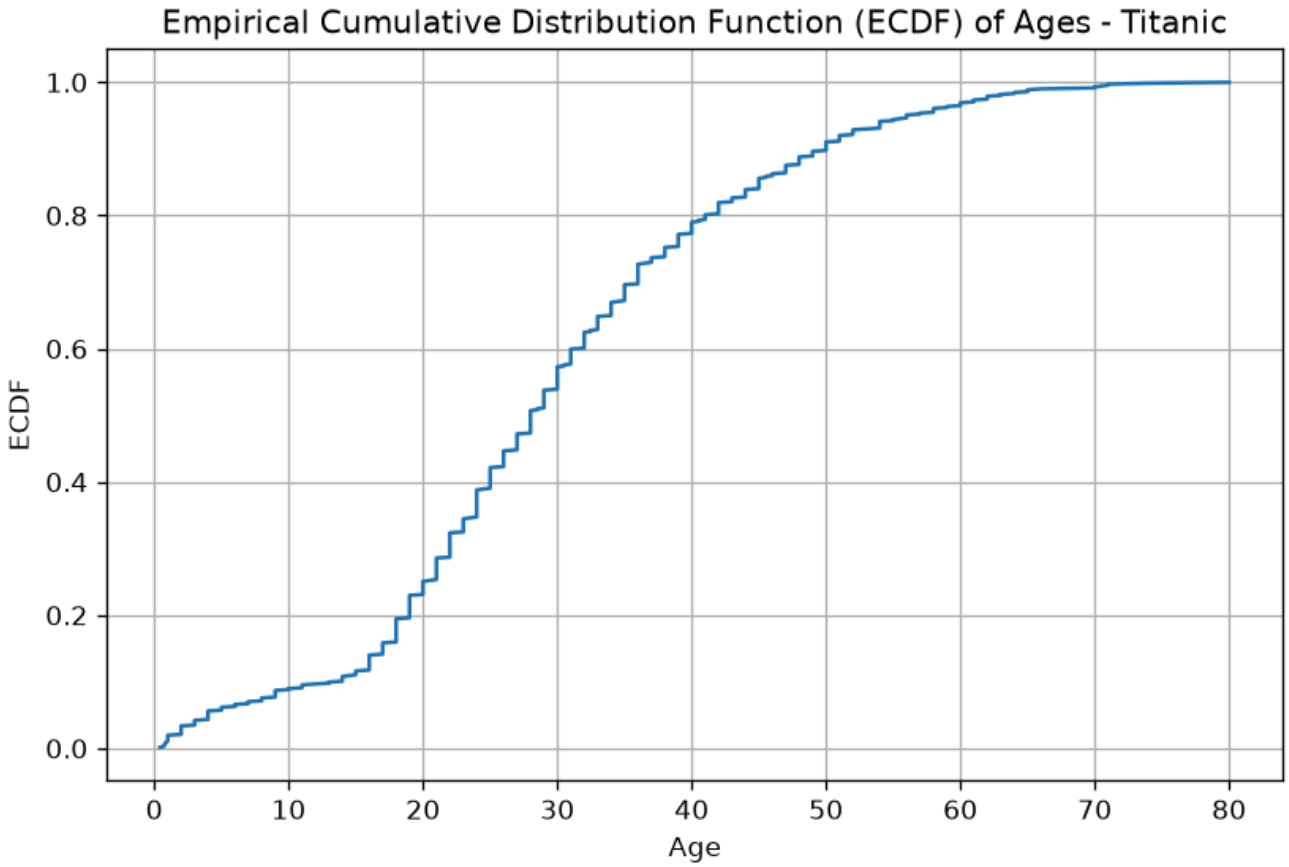


Computing CDFs for Discrete Variables from Data

Empirical Cumulative Distribution Functions (ECDFs), introduced in the lecture on data description, allow us to estimate the CDF directly from observed data. For a discrete variable, the ECDF at a value x is simply the proportion of data points less than or equal to x.

Here is a quick example:

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the Titanic dataset
titanic = pd.read_csv(DATA + "titanic.csv", index_col='PassengerId')

# Extract the age column, removing missing values
ages = titanic['Age'].dropna()

# Sort the ages
ages_sorted = np.sort(ages)

# Compute the ECDF
ecdf = np.arange(1, len(ages_sorted)+1) / len(ages_sorted)

# Plot the ECDF
plt.figure(figsize=(8, 5))
plt.plot(ages_sorted, ecdf)
plt.xlabel('Age')
plt.ylabel('ECDF')
plt.title('Empirical Cumulative Distribution Function (ECDF) of Ages - Titanic')
plt.grid(True)
plt.show()
```



Continuous Variables

We will now focus on the case in which X is a continuous variable. In this case, we cannot use the same definitions as for discrete variables and we have to resort to new definition.

Probability Density Functions (PDF)

In order to define a probability distribution over a continuous variable, we first have to introduce the concept of "probability density function".

A probability density function over a variable X is defined as follows:

$$f : \Omega \rightarrow [0, 1]$$

and must satisfy the following property:

$$\int f(x)\, dx = 1$$

This condition is equivalent to $\sum P(x) = 1$ in the case of a discrete variable. The sum becomes an integral for continuous variables.

We can then define the concept of probability based on the density as follows:

$$P(a \leq x \leq b) = \int_a^b f(x)\, dx$$

Intuitively, the density function $f(x)$ describes how likely it is to observe values near x. The higher the density at a point, the more likely it is to find the variable in a small interval around that point. In other words, the density can be seen as a "concentration" of probability: regions where $f(x)$ is high correspond to intervals where the variable is more frequently observed. To obtain an actual probability, we must integrate the density over an interval.

> For a continuous random variable, the probability of observing exactly x is zero: $P(X = x) = 0$. However, the density function $f(x)$ at x is generally not zero; it represents the probability per unit interval around x. Probability is obtained by integrating $f(x)$ over an interval, not at a single point.

Example: Uniform PDF

Let us consider a random number generator which outputs numbers comprised between a and b.

Let X be a random variable assuming the values generated by the random number generator.

The PDF of X will be a uniform distribution such that:

- $P(x) = 0 \; \forall x < a \;\; or \; x > b$;

- $P(x) = \frac{1}{b-a} \; \forall a \leq x \leq b$;

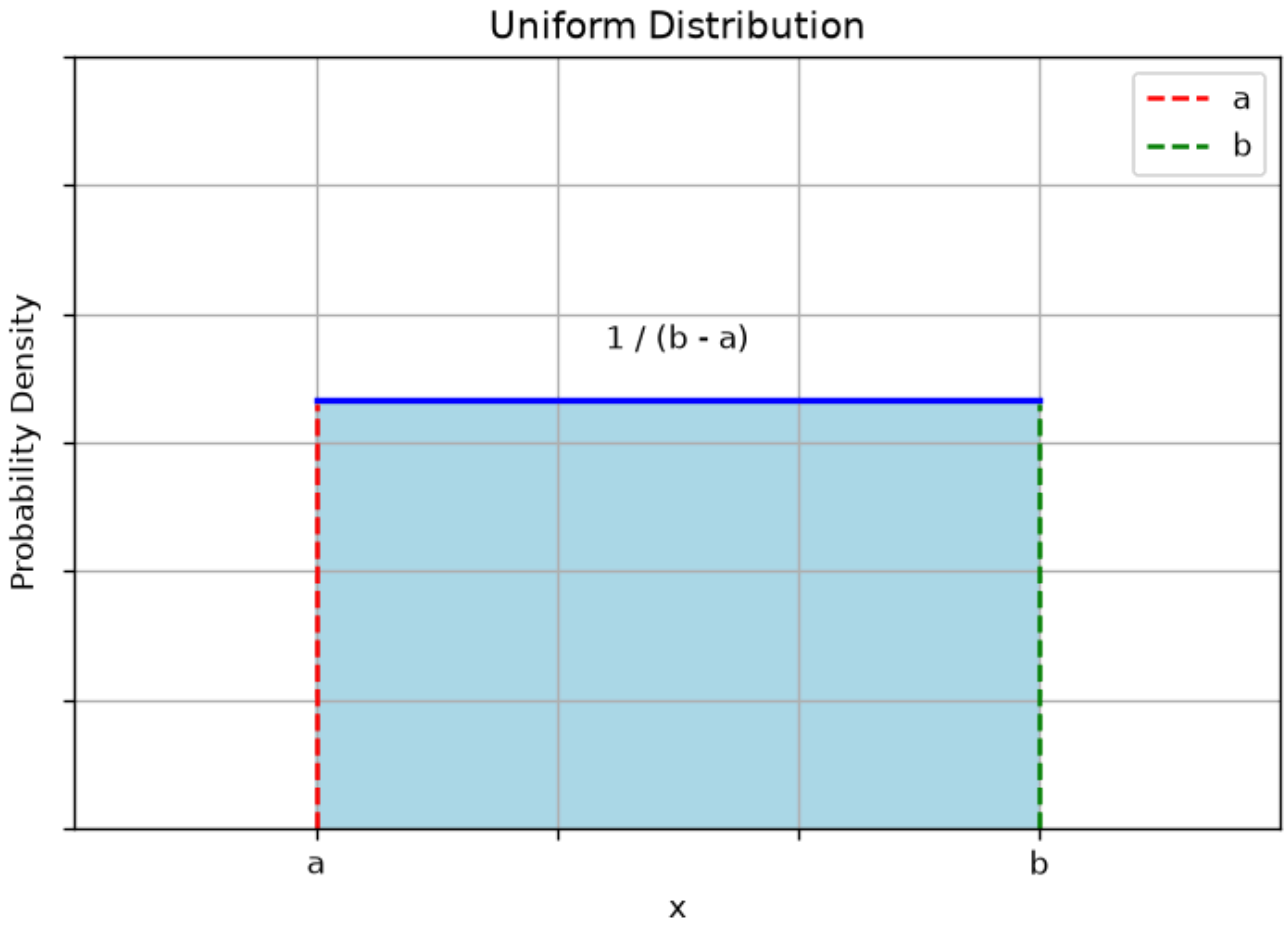
We can see that this PDF satisfies all constraints:

- $P(x) \geq 0 \; \forall x.$
- $\int P(x)dx = 1$ (prove that this is true as an exercise).

The diagram below shows an illustration of a uniform PDF with bounds a and b:

$$U(a, b)$$

Of course, continuous distributions can be (and generally are) much more complicated than that.



Approximating a PDF with Density Histograms and Density Estimates

When working with real data, we can try to approximate the PDF of a continuous variable with histograms.

Let the height and width of bin b_j be h_j and w_j respectively. In a **normalized histogram**, we define the height of bin b_j as follows:

$$h_j = \frac{c_j}{n}$$

where c_j is the number of observations that fall in bin b_j.

Note that, while **normalized histograms** are such that each bar height is comprised between 0 and 1, they do not satisfy the properties of PDFs.

Let the area of bin b_j be:

$$A_j = w_j \cdot h_j$$

The integral over histogram H (i.e., the area under the histogram) will be:

$$\int_{-\infty}^{\infty} H(x)dx = \sum_j A_j = \sum_j w_j \frac{c_j}{n} = \frac{1}{n}\sum_j w_j \cdot c_j = \sum w_j \neq 1$$

To ensure that this property is satisfied, we define a new kind of histogram, the **density histogram**, where bin heights are defined as:

$$h_j = \frac{c_j}{w_j \cdot n}$$

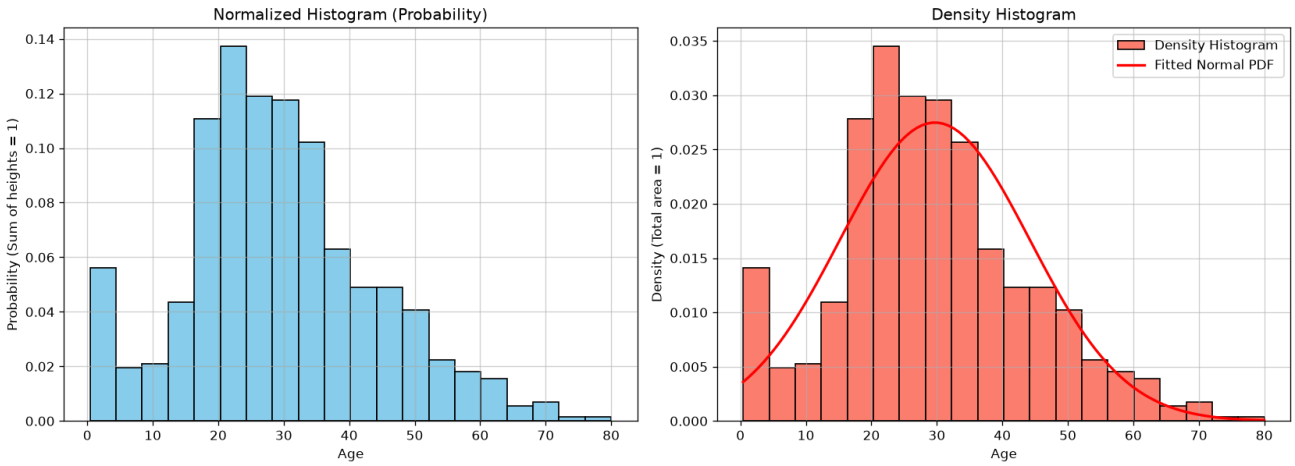
In this case:

$$\sum_j A_j = \sum_j w_j \frac{c_j}{w_j \cdot n} = \frac{1}{n}\sum_j \frac{w_j}{w_j} \cdot c_j = 1$$

To create a density histogram in Python, simply pass `density=True` to the `hist` function in matplotlib or pandas.

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Load the Titanic dataset and extract the 'Age' column, dropping missing values
titanic = pd.read_csv(DATA + "titanic.csv")
ages = titanic['Age'].dropna()

# Create a figure with two subplots for side-by-side comparison
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# --- Plot 1: Normalized Histogram (Probability) ---
# The sum of bar heights will equal 1
weights = np.ones_like(ages) / len(ages)
ax1.hist(ages, bins=20, weights=weights, color='skyblue', edgecolor='black')
ax1.set_title('Normalized Histogram (Probability)')
ax1.set_xlabel('Age')
ax1.set_ylabel('Probability (Sum of heights = 1)')
ax1.grid(True, alpha=0.6)

# --- Plot 2: Density Histogram ---
# The total area of the bars will equal 1
ax2.hist(ages, bins=20, density=True, color='salmon', edgecolor='black',
         label='Density Histogram')

# Fit a normal distribution and overlay its PDF
mu_hat, sigma_hat = norm.fit(ages)
x = np.linspace(ages.min(), ages.max(), 100)
pdf_fitted = norm.pdf(x, loc=mu_hat, scale=sigma_hat)
ax2.plot(x, pdf_fitted, 'r-', lw=2, label='Fitted Normal PDF')

ax2.set_title('Density Histogram')
ax2.set_xlabel('Age')
ax2.set_ylabel('Density (Total area = 1)')
ax2.grid(True, alpha=0.6)
ax2.legend()

# Add an overall title and show the plot
plt.suptitle('Normalized vs. Density Histograms for Passenger Ages', fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])  # Adjust layout to make space for suptitle
plt.show()
```

Normalized vs. Density Histograms for Passenger Ages



The figure above contrasts two important ways to normalize a histogram using Titanic passenger ages, each with a distinct but related interpretation:

- **Normalized Histogram (Left)**:
  The y-axis represents **probability**. Each bar's height shows the proportion of passengers in that age bin, so the **sum of all bar heights equals 1**. This treats the binned data like a discrete probability distribution and allows direct reading of probabilities (e.g., the chance of being 20–24 years old is roughly 0.12 or 12%).

- **Density Histogram (Right)**:
  The y-axis represents **density**—probability per unit on the x-axis (e.g., per year of age). Here, it's the **area** of each bar (height × width) that reflects probability. The **total area of all bars equals 1**, mirroring the behavior of a continuous Probability Density Function (PDF).

Key takeaway:
The density histogram is the correct empirical approximation of a continuous PDF. That's why the smooth red PDF curve aligns with the right plot's y-axis but would be mis-scaled on the left. While both histograms are normalized, only the density version properly reflects continuous probability.

Cumulative Distribution Functions (CDF)

Just like in the case of discrete variables, we can define Cumulative Distribution Functions (CDFs) for continuous random variables using their density functions. The CDF of a random variable is generally defined as:

$$F(x) = \int_{-\infty}^{x} f(x)\, dx$$

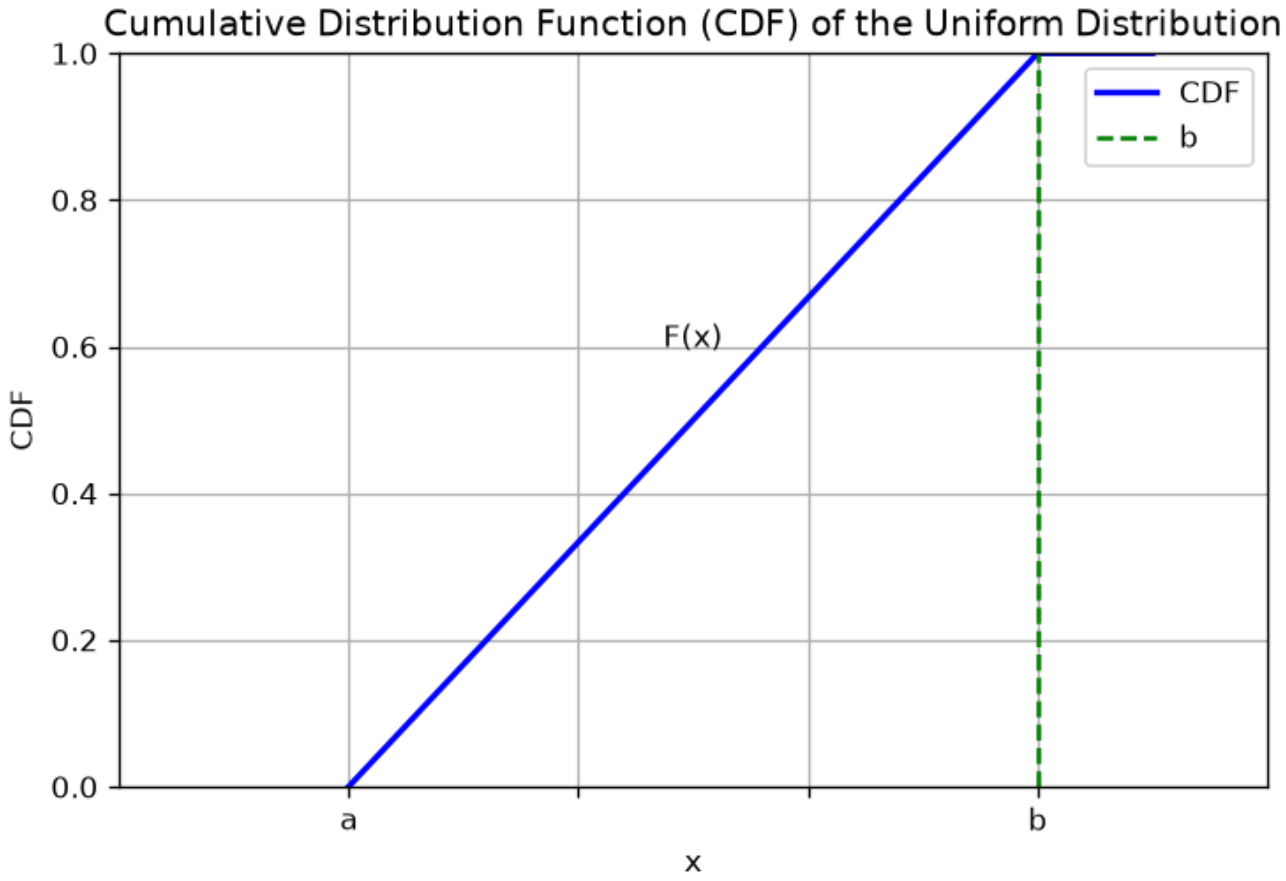
Example

The CDF of the uniform distribution will be given by:

- $F(x) = 0$ for $x < a$

- $F(x) = \frac{x-a}{b-a}$ for $a \leq x \leq b$
- $F(x) = 1$ for $x > b$

The plot below shows a diagram:



Common Probability Distributions

There are several common probability distributions which can be used to describe random events. **These distributions have an analytical formulation which depends generally on one or more parameters.**

When we have **enough evidence that a given random variable is well described by one of these distributions**, we can simply "fit" the distribution to the data (i.e., choose the correct parameters for the distribution) and use the analytical formulation to deal with the random variable.

It is hence useful to know the **most common probability distributions** so that we can recognize the cases in which they can be used.

Discrete Uniform Distribution

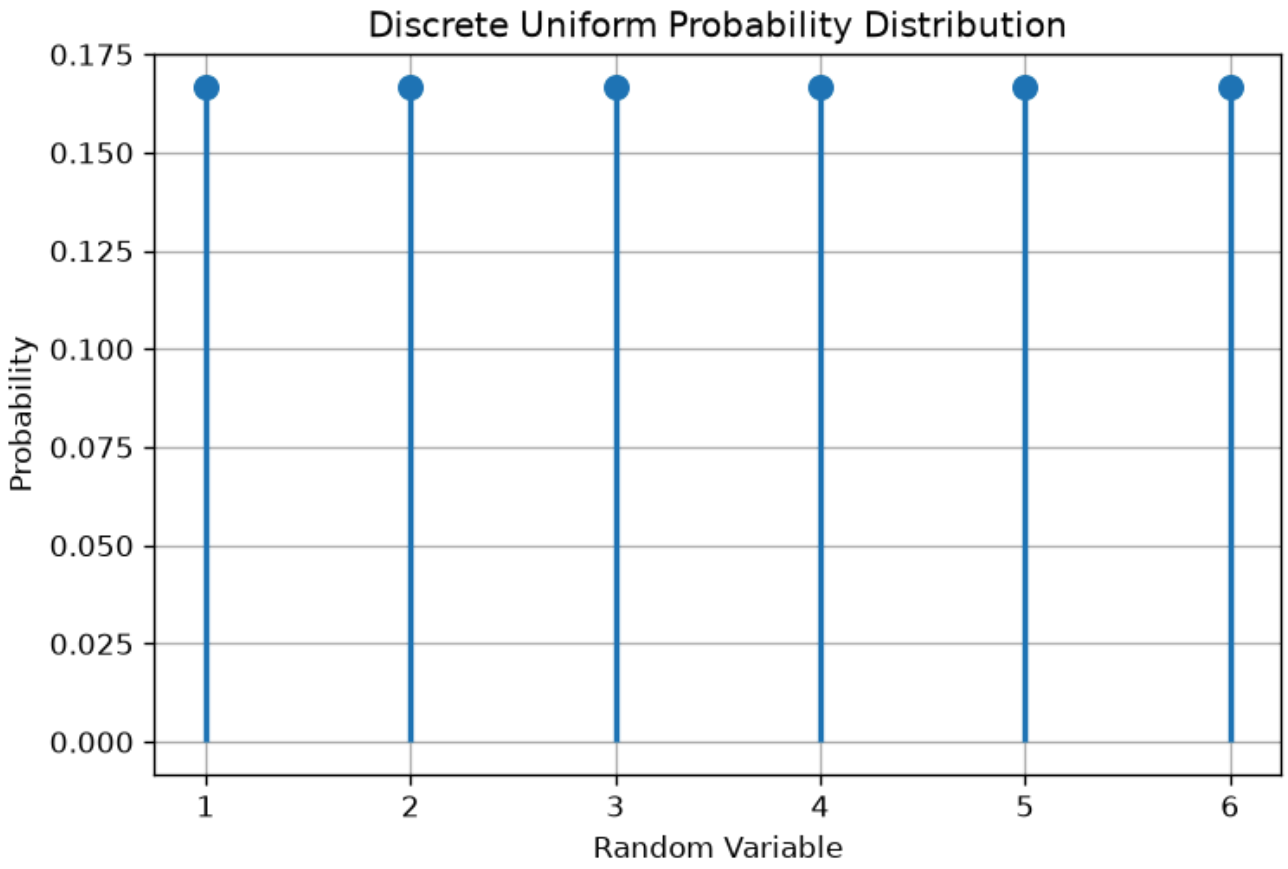
The discrete uniform distribution is controlled by a parameter $k \in \mathbb{N}$ and assumes that all outcomes have the same probability of occurring:

$$P(X = a_i) = \frac{1}{k}$$

Where $\Omega = \{a_1, \ldots, a_k\}$.

Example

The outcomes of rolling a fair die follow a uniform distribution with $k = 6$, as shown in the diagram below:



Bernoulli Distribution

The Bernoulli distribution is a distribution over a single binary random variable, i.e., the variable X can take only two values: $\{0, 1\}$.

The distribution is controlled by a single parameter $\phi \in [0, 1]$, which gives the probability of the variable to be equal to 1.

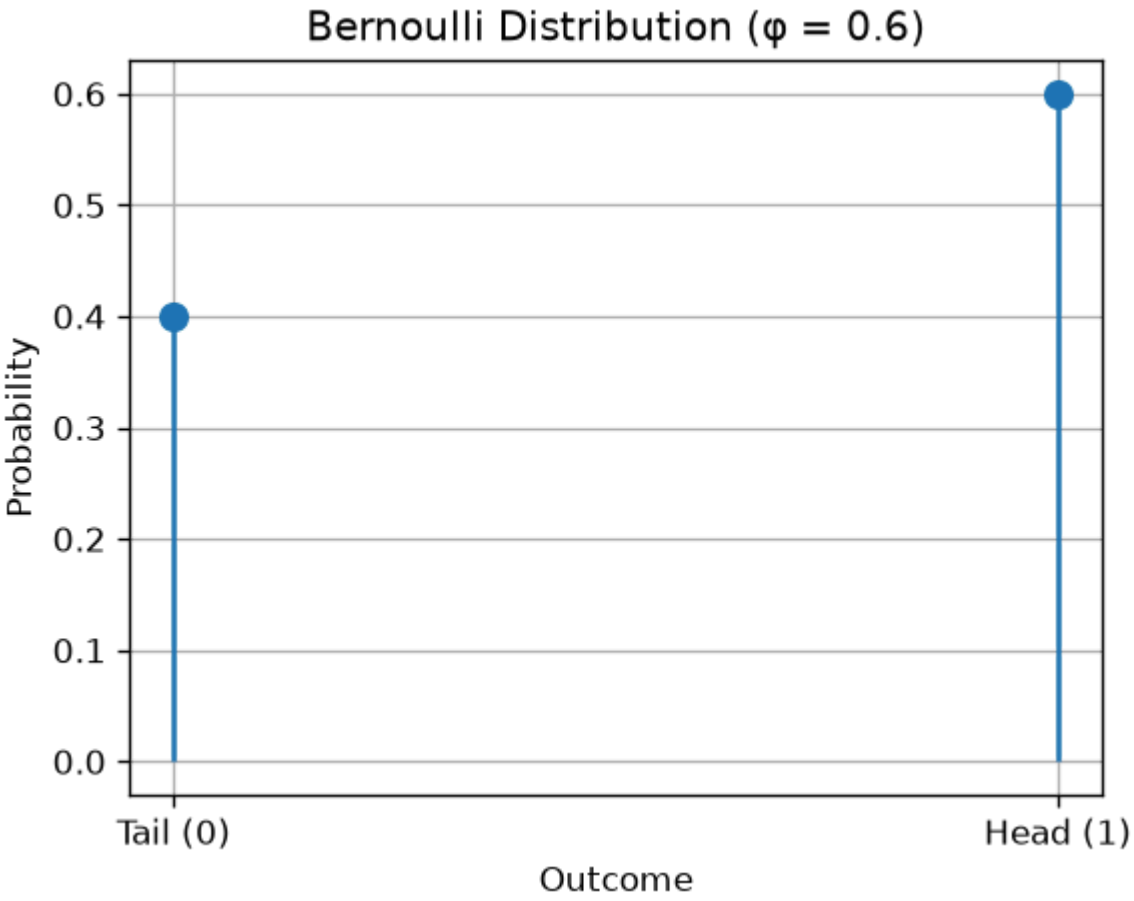
The analytical formulation of the Bernoulli distribution is very simple:

- $P(X = 1) = \phi$;
- $P(X = 0) = 1 - \phi$

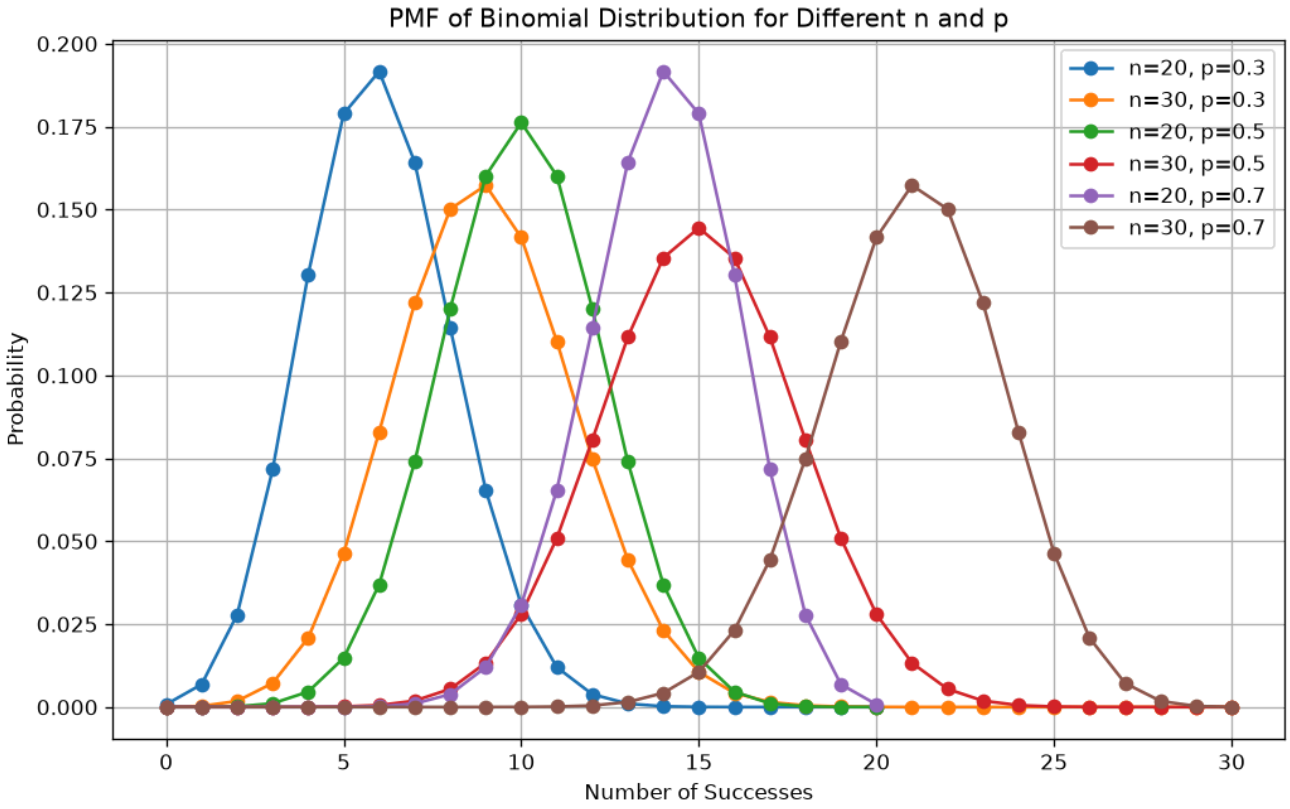
Example

A skewed coin lands on "head" 60% of the times. If we define $X = 1$ when the outcome is head and $X = 0$ when the outcome is tail, then the variable follows a Bernoulli distribution with $\phi = 0.6$.

The diagram below gives an example:



Binomial Distribution



The binomial distribution is a discrete probability distribution (PMF) over natural numbers with parameters $\mathbf{n}$ and $\mathbf{p}$

It models **the probability of obtaining $\mathbf{k}$ successes in a sequence of $\mathbf{n}$ independent experiments which follow a Bernoulli distribution with parameter $\mathbf{p}$ ($\phi = \mathbf{p}$);**

The probability mass function of the distribution is given by:

$$P(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

Where:

- k is the number of successes
- n is the number of independent trials
- p is the probability of a success in a single trial

Example

What is the probability of tossing a coin three times and obtaining three heads? We have:

- $k = 3$: number of successes (three times head)
- $n = 3$: number of trials

- $p = 0.5$: the probability of getting a head when tossing a coin

The required probability will be given by:

$$P(3) = \binom{3}{3} 0.5^3 (1 - 0.5)^{3-3} = 0.5^3 = 0.125$$

Exercise

What is the probability of tossing an unfair coin ($P\left('head'\right) = 0.6$) 7 times and obtaining 2 tails?

Categorical Distribution

The **multinoulli** or **categorical** distribution is a distribution of a *single discrete variable with k different states*, where k is finite.

- The distribution is parametrized by a vector $\mathbf{p} \in [0, 1]^k$, where p_i gives the probability of the i^{th} state.
- $\mathbf{p}$ must be such that $\sum_{i=1}^{k} p_i = 1$ to obtain a valid probability distribution.
- The analytical form of the distribution is given by: $p(x = i) = p_i$;

This distribution is the **generalization of the Bernoulli distribution to the case of multiple states**.

Example: Rolling a Biased Die

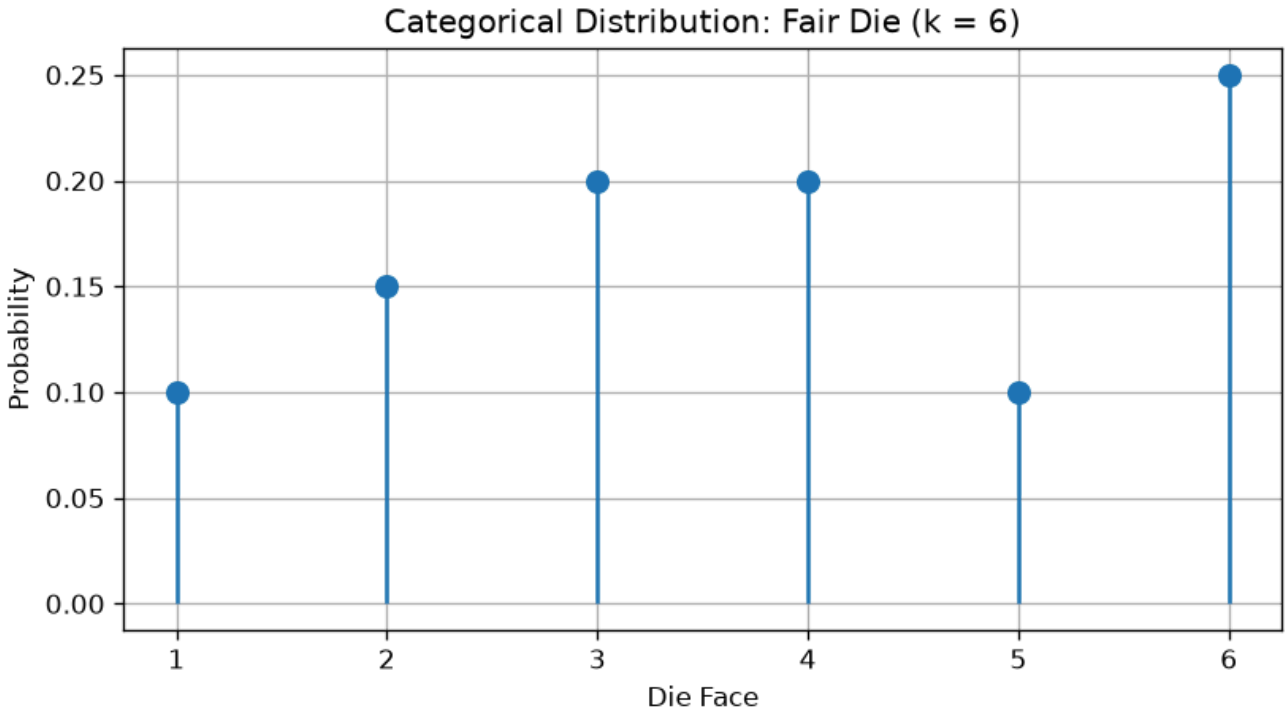
Consider a single roll of a biased six-sided die. The outcomes are discrete and finite:
$X \in \{1, 2, 3, 4, 5, 6\}$.

If the die is biased such that the probabilities are:

- $P(X = 1) = 0.10$
- $P(X = 2) = 0.15$
- $P(X = 3) = 0.20$
- $P(X = 4) = 0.20$
- $P(X = 5) = 0.10$
- $P(X = 6) = 0.25$

Then the outcome follows a **Categorical distribution** with $k = 6$ and the specified probabilities $\{p_1, p_2, \ldots, p_6\} = \{0.1, 0.15, 0.2, 0.2, 0.1, 0.25\}$.

The figure below shows an example:



Multinomial Distribution

The multinomial distribution **generalizes the binomial distribution to the case in which the experiments are not binary**, but they can have multiple outcomes (e.g., *a dice vs a coin*).

In particular, the multinomial distribution models the probability of obtaining exactly $(n_1, \ldots, n_k)$ occurrences (with $n = \sum_i n_i$) for each of the k possible outcomes in a sequence of n independent experiments which follow a Categorial distribution with probabilities $p_1, \ldots, p_k$.

The parameters of the distribution are:

- n: the number of trials
- k: the number of possible outcomes
- $p_1, \ldots, p_k$ the probabilities of obtaining a given class in each trial (with $\sum_{i=1}^{k} p_i = 1$)

The PMF of the distribution is:

$$P(n_1, \ldots, n_k) = \frac{n!}{n_1! \ldots n_k!} p_1^{n_1} \cdot \ldots \cdot p_k^{n_k}$$

Example

Given a fair die with 6 possible outcomes, what is the probability of getting 3 times 1, 2 times 2, 4 time 3, 5 times 4, 0 times 5, and 1 time 6, rolling the dice for 15 times?

We have:

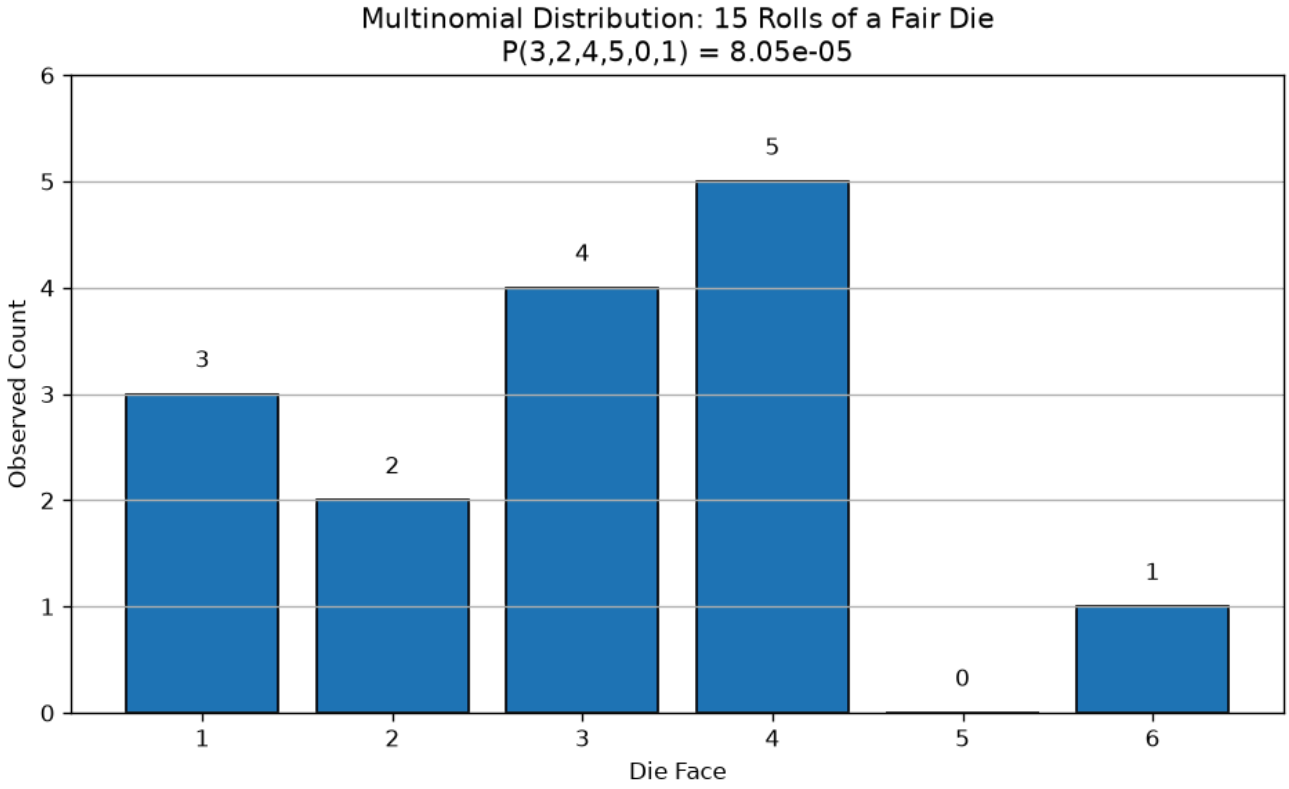
- $n = 15$
- $k = 6$
- $p_1 = p_2 = \ldots p_6 = \frac{1}{6}$

The required probability is given by:

$$P(3,2,4,5,0,1) = \frac{15!}{3!2!4!5!0!1!} \cdot \frac{1}{6^3} \cdot \frac{1}{6^2} \cdot \frac{1}{6^4} \cdot \frac{1}{6^5} \cdot \frac{1}{6^0} \cdot \frac{1}{6^1} = 8.04 \cdot 10^{-5}$$

The bar chart below illustrate the example, showing the observed counts and the estimated probability.

This low probability reflects the combinatorial nature of the multinomial distribution, which accounts for both the number of ways the outcomes can be arranged and the likelihood of each individual outcome.



Gaussian Distribution

The Bernoulli and Categorical distributions are PMF, i.e., distributions over discrete random variables.

A common PDF when dealing with real values is the **Gaussian distribution,** also known as **Normal Distribution**.

The distribution is characterized by two parameters:

- The mean $\mu \in \Re$

- The standard deviation $\sigma \in (0, +\infty)$

In practice, the distribution is often *seen in terms of* μ *and* σ^2 rather than σ, where σ^2 is called **the variance**.

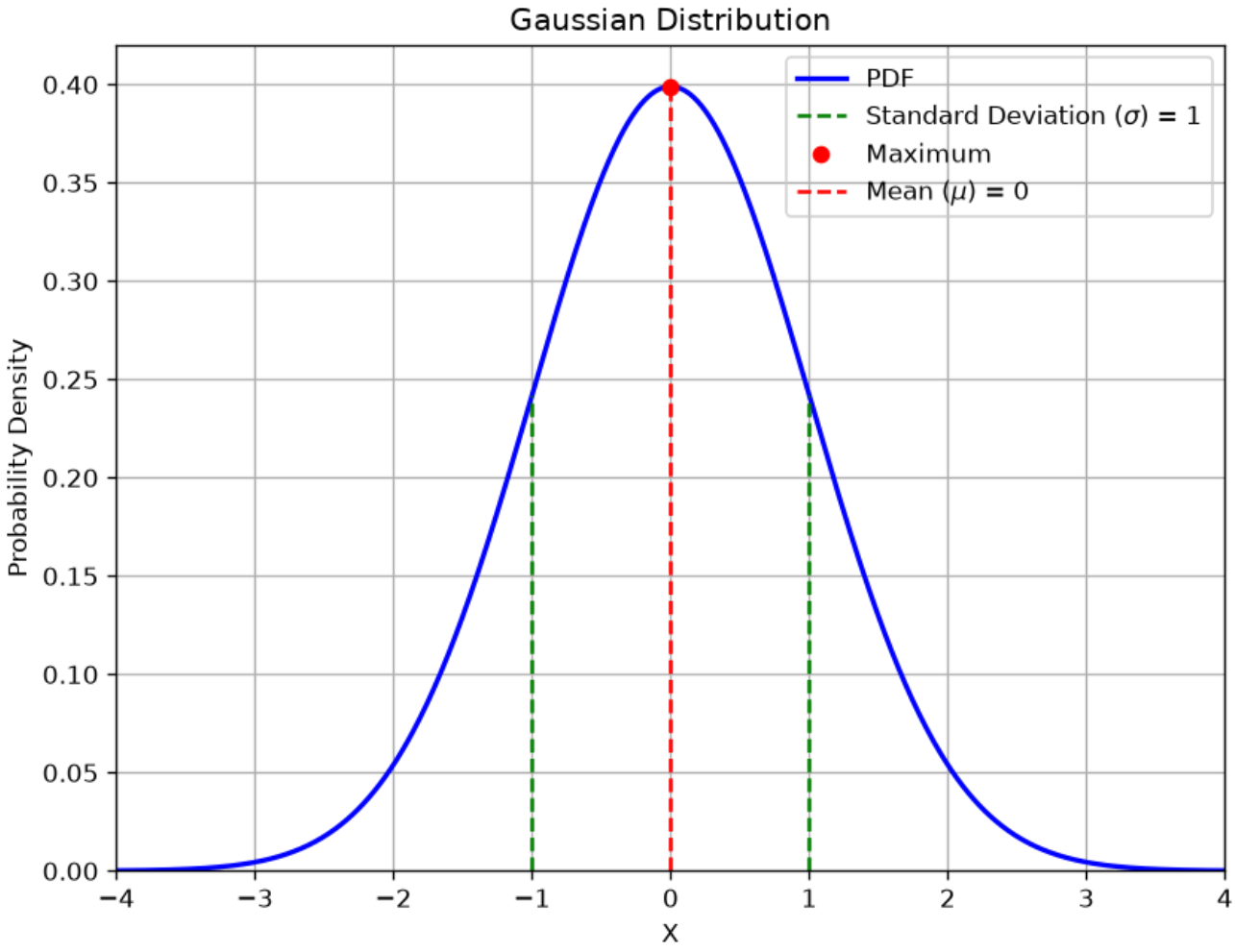
The analytical formulation of the Normal distribution is as follows:

$$N\left(x; \mu, \sigma^2\right) = \sqrt{\frac{1}{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}$$

The term under the square root is a normalization term which ensures that the distribution integrates to 1.

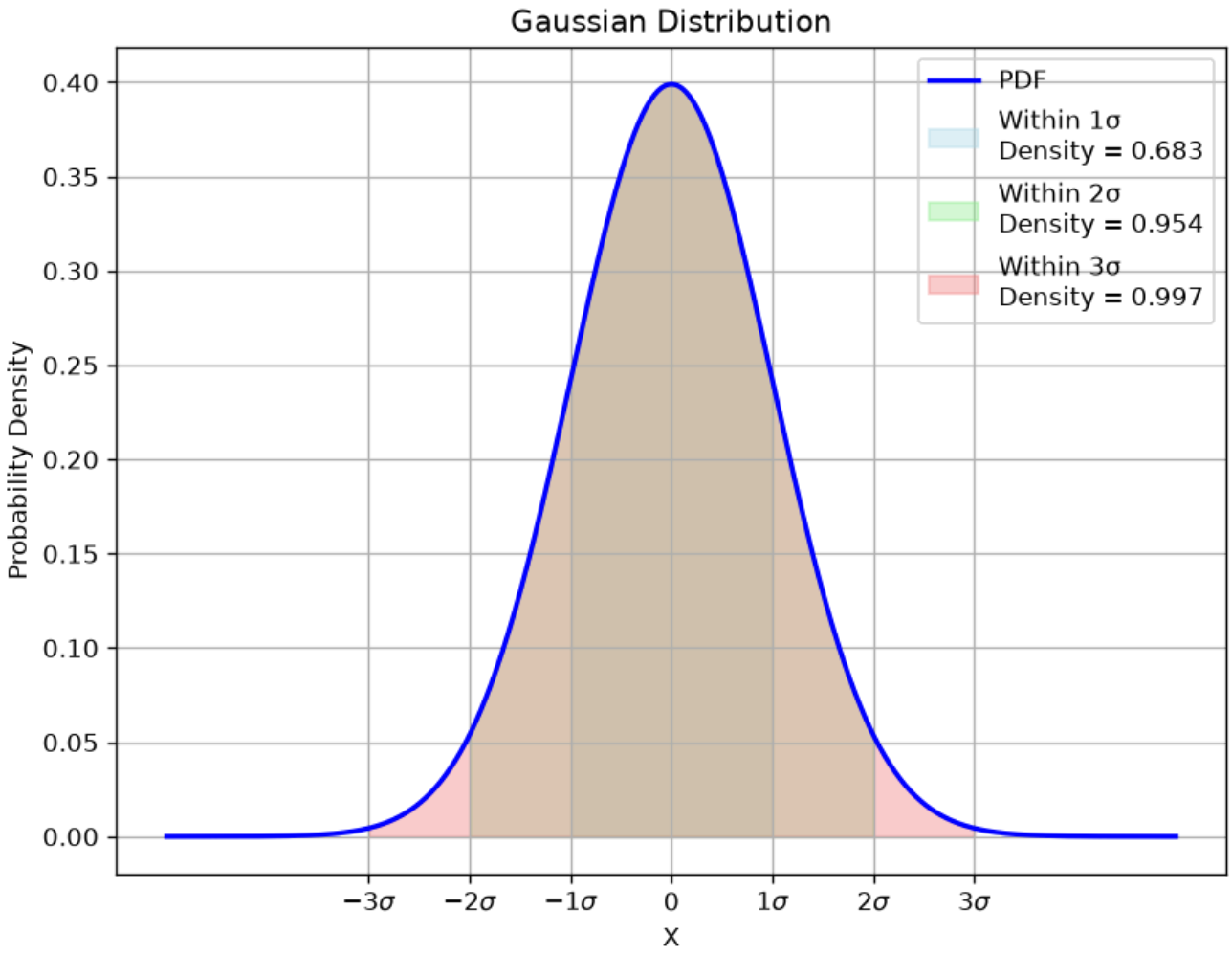
The Gaussian distribution is very used when we do not have much prior knowledge on the real distribution we wish to model. This in mainly due to the **central limit theorem**, which states that the sum of many independent random variables with the same distribution is approximately normally distributed.

Interpretation



If we plot the PDF of a Normal distribution, we can find that it is easy to interpret the meaning of its parameters:

- The resulting curve has a maximum (highest probability) when $x = \mu$
- The curve is symmetric, with the inflection points at $x = \mu \pm \sigma$
- The example shows a normal distribution for $\mu = 0$ and $\sigma = 1$



Another notable property of the Normal distribution is that:

- About 68% of the density is comprised in the interval $[-\sigma, \sigma]$;
- About 95% of the density is comprised in the interval $[-2\sigma, 2\sigma]$;
- Almost 100% of the density is comprised in the interval $[-3\sigma, 3\sigma]$.

Central Limit Theorem

The **Central Limit Theorem (CLT)** is a fundamental statistical principle stating that the distribution of the sum (or average) of a large number of independent and identically distributed (i.i.d.) random variables $\{X_i\}_{i=1}^{n}$ tends toward a normal (Gaussian) distribution as $n \to \infty$, regardless of the original distribution of each X_i.

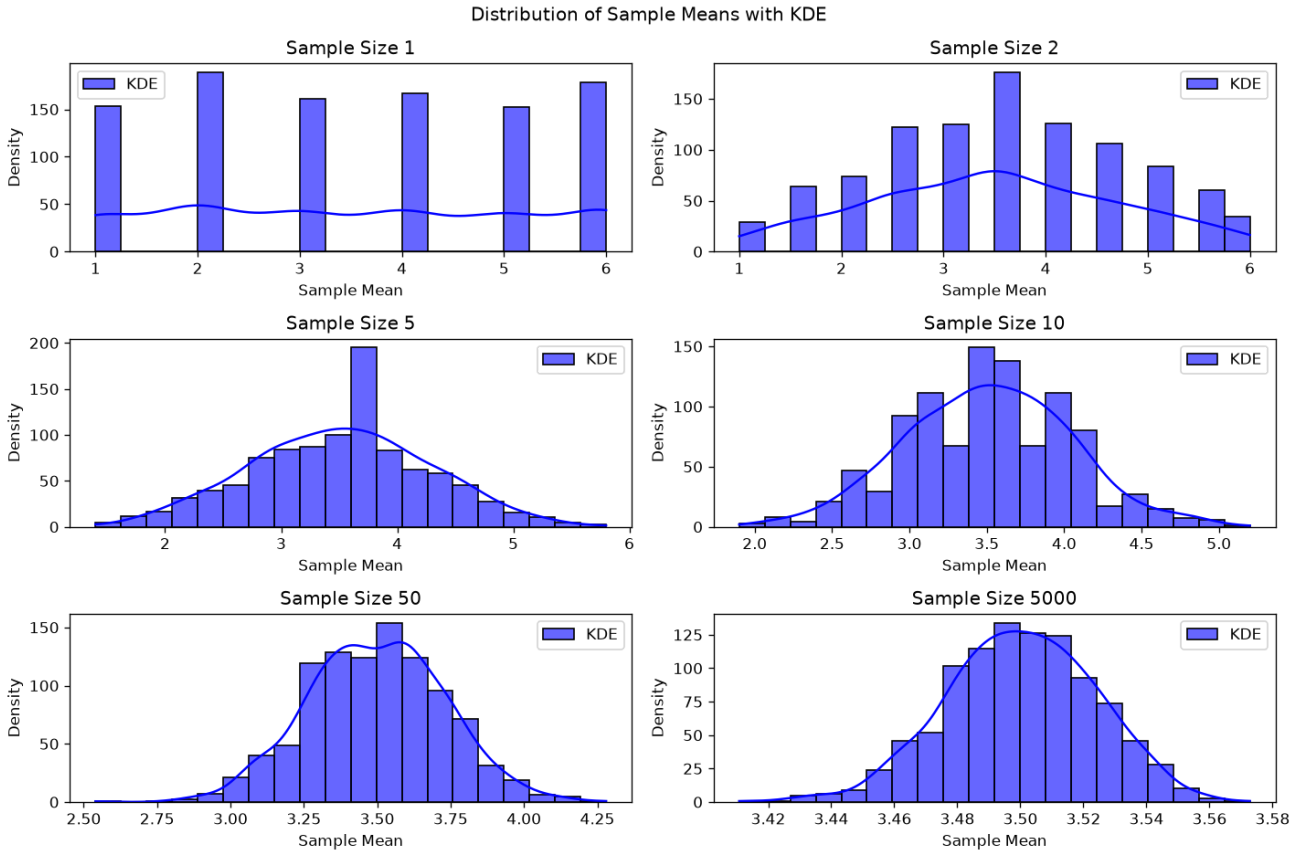
This result is crucial because it explains why the Gaussian distribution is so commonly observed in data analysis and natural phenomena, even when the underlying data are not normally distributed.

Key points:

- The CLT applies to sums or averages of i.i.d. random variables.
- The convergence to the normal distribution becomes more accurate as the number of variables n increases.
- The original distribution of the variables can be arbitrary (e.g., uniform, binomial, etc.).

Illustration:
The plot below shows the distributions of the average outcome when rolling different numbers of dice. Each die represents an independent random variable. As the number of dice increases, the distribution of the average outcome becomes increasingly similar to a Gaussian curve.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns  # Import seaborn for KDE

# Simulate rolling a fair six-sided die
num_rolls = 1000
sample_sizes = [1, 2, 5, 10, 50, 5000]

# List to store the means of each sample
sample_means = []

for sample_size in sample_sizes:
    means = [np.mean(np.random.randint(1, 7, sample_size)) for _ in range(num_rolls)]
    sample_means.append(means)

# Plot the sample means for different sample sizes with KDE
plt.figure(figsize=(12, 8))
for i, sample_size in enumerate(sample_sizes):
    plt.subplot(3, 2, i + 1)
    sns.histplot(sample_means[i], bins=20, kde=True, color='blue', label='KDE',
        alpha=0.6)
    plt.title(f'Sample Size {sample_size}')
    plt.xlabel("Sample Mean")
    plt.ylabel("Density")
    plt.legend()

plt.suptitle('Distribution of Sample Means with KDE')
plt.tight_layout()
plt.show()
```



Multivariate Gaussian

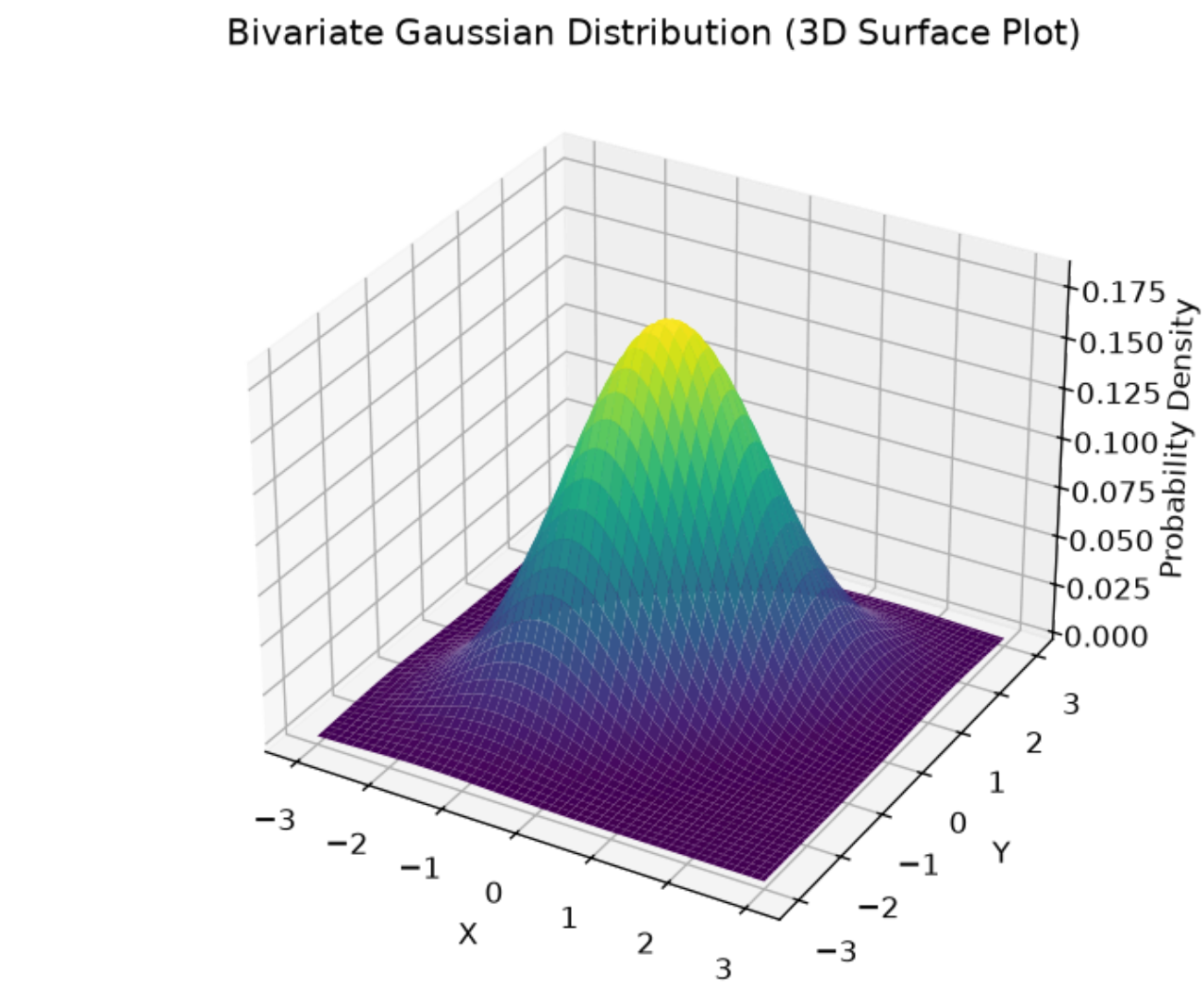
The formulation of the Gaussian distribution generalizes to the multivariate case, i.e., the case in which X is d-dimensional.

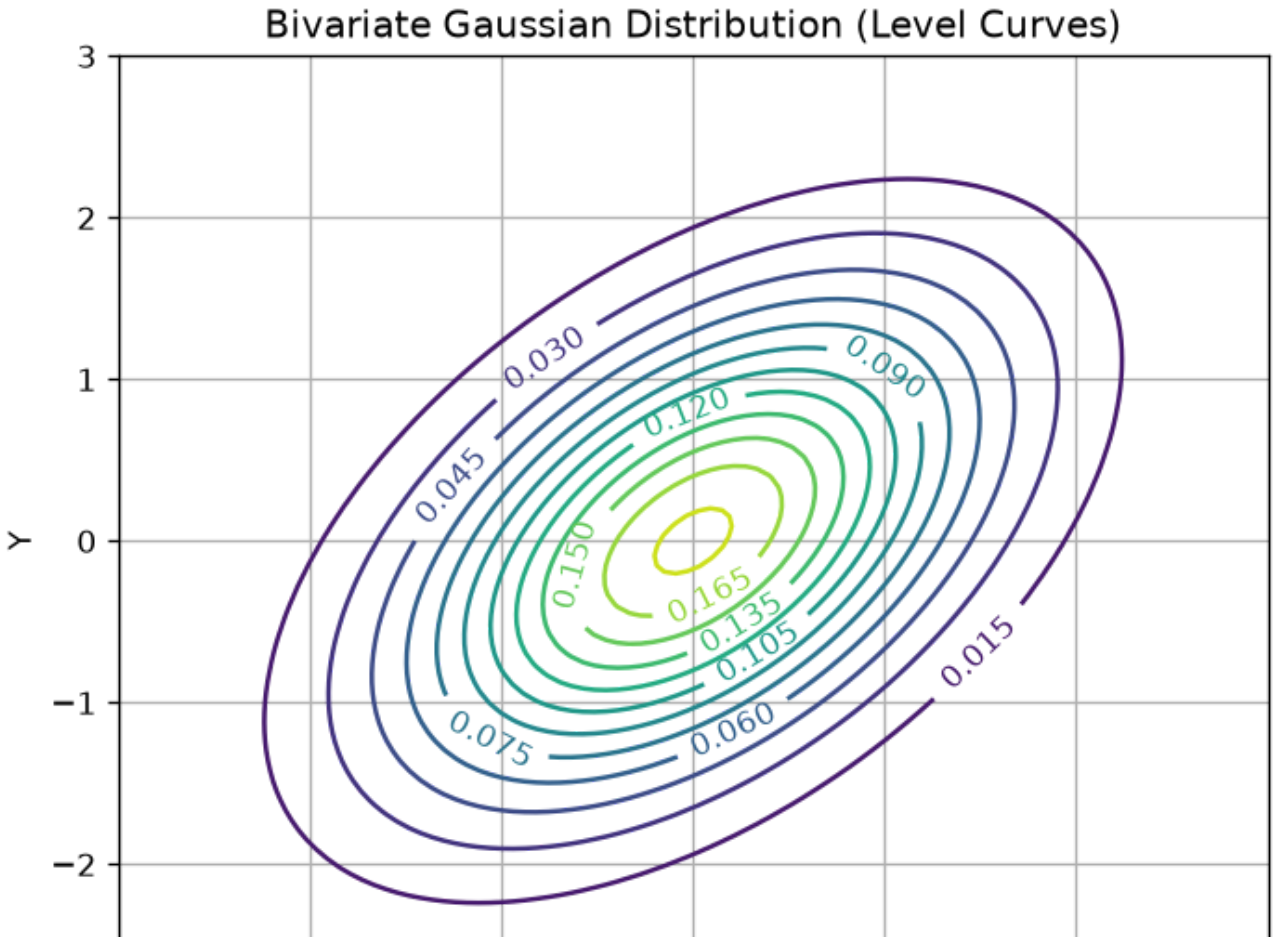
In that case, the distribution is parametrized by a **d-dimensional vector μ** and a **$\mathbf{d} \times \mathbf{d}$ positive definite symmetric matrix $\mathbf{\Sigma}$**. The formulation of the multi-variate Gaussian is:

$$N(\mathbf{x}; \mu, \mathbf{\Sigma}) = \sqrt{\frac{1}{(2\pi)^d \det(\Sigma)}} e^{(-\frac{1}{2}(\mathbf{x}-\mu)^T \Sigma^{-1}(\mathbf{x}-\mu))}$$

In the 2D case, μ is a 2D point representing the center of the Gaussian (the position of the mode), whereas the matrix Σ influences the "shape" of the Gaussian.

Examples of bivariate Gaussian distributions are shown below.


Bivariate Gaussian Distribution (3D Surface Plot)
Probability Density
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
X
Y
−3
−2
−1
0
1
2
3


Bivariate Gaussian Distribution (Level Curves)
X
Y
0.030
0.045
0.090
0.120
0.150
0.165
0.135
0.105
0.075
0.060
0.015

The two plots above are common representations for bivariate continuous distributions:

- The plot on the top shows a 3D representation of the PDF in which the X and Y axes are the values of the variables, while the third axis reports the probability density.
- Since it's often hard to draw 3D graphs, we often use a contour plot to represent the 3D curves. In the 3D plot, curves of the same color represent points which have the same density in the 3D plot.

Effect of Σ

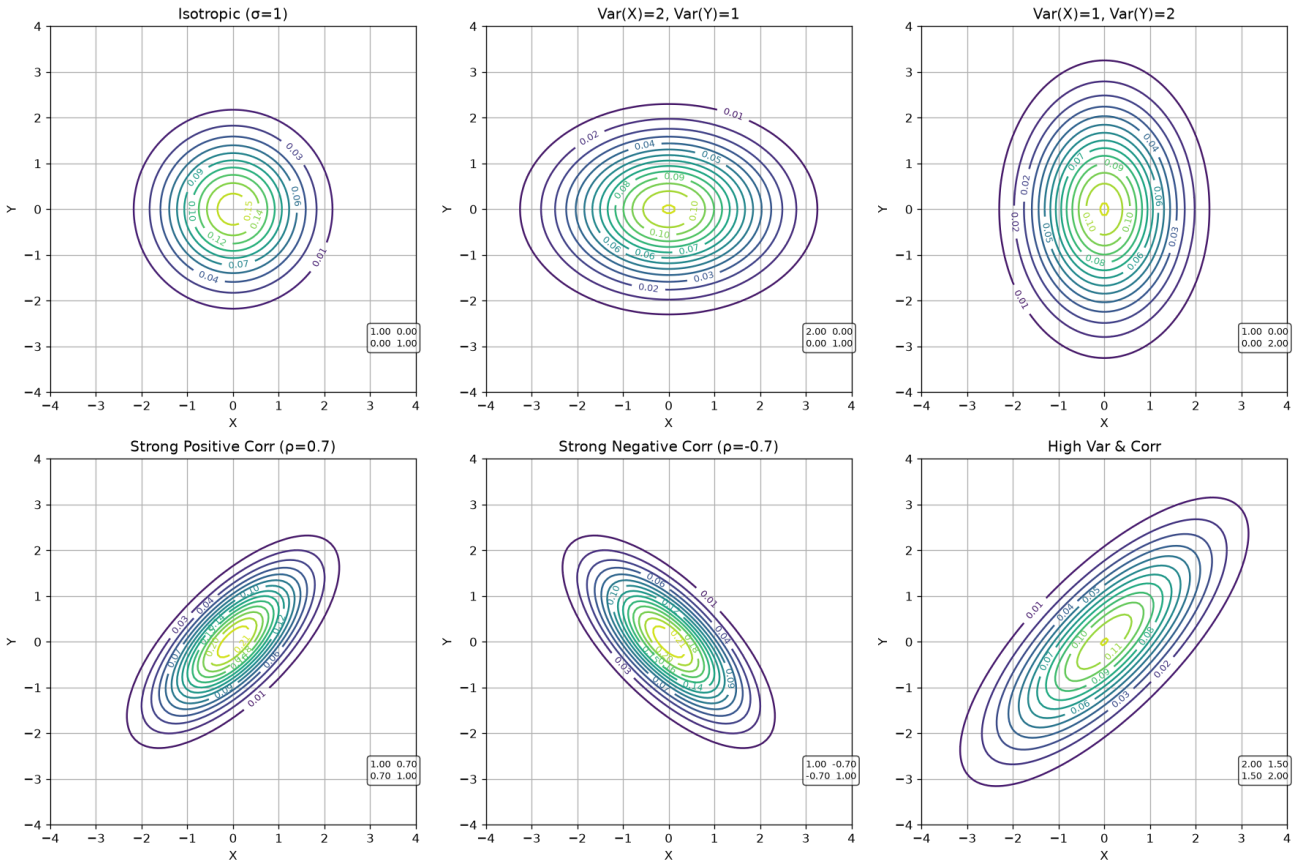
Similar to how variance affects the dispersion of a 1D Gaussian, the covariance matrix Σ affects the dispersion in both axes. As a result, changing the values of the matrix will affect the shape of the distribution. Let's consider the general covariance matrix:

$$\Sigma = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{yx} & \sigma_y^2 \end{bmatrix}$$

The covariance matrix Σ determines the shape, orientation, and spread of a bivariate Gaussian distribution:

- When Σ is diagonal ($\sigma_{xy} = \sigma_{yx} = 0$), the distribution is symmetric and spreads equally along both axes, resulting in circular contours.
- If the variances along the axes differ, the distribution becomes elongated along the axis with higher variance.
- Non-zero off-diagonal elements introduce correlation: a positive covariance causes the distribution to tilt so that higher values of one variable are associated with higher values of the other, while a negative covariance produces an opposite tilt.
- Larger absolute values of covariance lead to more pronounced stretching along a diagonal direction. When both variances and covariances are large, the distribution is highly dispersed and strongly correlated. These effects are clearly visible when comparing different covariance matrices in contour plots.

Some examples are shown below:



Estimation of the Parameters of a Gaussian Distribution

We have noted that in many cases we can assume a random variable follows a Gaussian distribution. However, it is not yet clear how to choose the parameters of the Gaussian distribution.

Given some data (remember, data is values assumed by random variables!), we can obtain the parameters of the Gaussian distribution related to the data with a **maximum likelihood** estimation.

This consists in computing the mean and variance parameters using the following formula (in the univariate case):

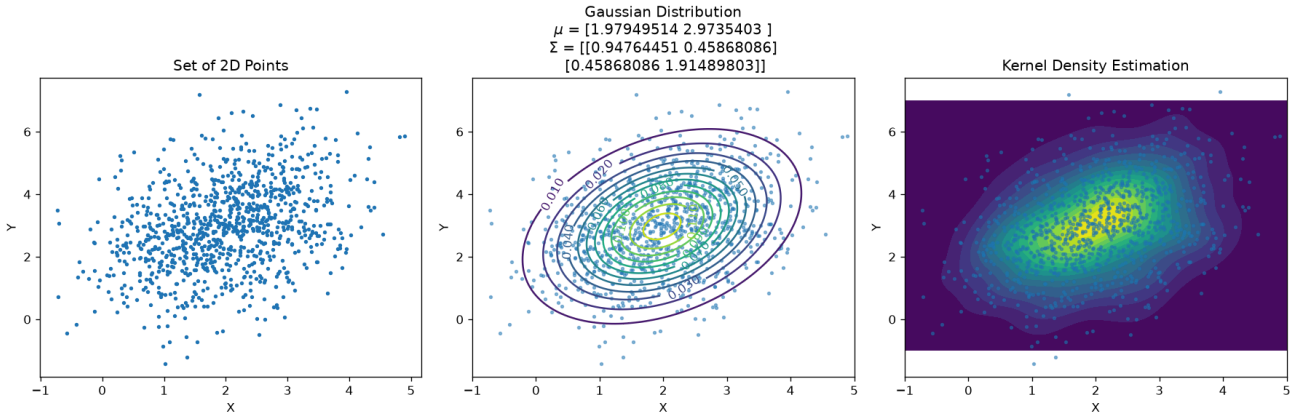
- $\mu = \frac{1}{n} \sum_j x_j$
- $\sigma^2 = \frac{1}{n-1} \sum_j (x_j - \mu)^2$

Where x_j represent the different data points.

In the multi-variate case, the computation of the multi-dimensional μ vector is similar:

- $\mu = \frac{1}{n} \sum_j \mathbf{x}_j$
- Σ is instead computed as the covariance matrix related to X: $\Sigma = Cov(\mathbf{X})$, i.e., $\Sigma_{ij} = Cov(\mathbf{X}_i, \mathbf{X}_j)$

The diagram below shows an example in which we fit a Gaussian to a set of data and compare it with a 2D KDE of the data.



Describing Data Distributions

To effectively summarize and interpret the properties of a probability distribution, it is useful to rely on a set of descriptive statistics. Measures such as **expectation** (mean), **variance**, and **covariance** allow us to capture the central tendency, variability, and relationships between variables, while **standardization** allows us to compare variables on a common scale. These concepts provide essential tools for understanding the behavior of random variables and for analyzing data distributions in practice.

Expectation

When it is known that a random variable follows a probability distribution, **it is possible to characterize that variable** (and hence the related probability distribution) **with some statistics**.

The most straightforward of them is the expectation. **The concept of expectation is very related to the concept of mean.** When we compute the mean of a given set of numbers, we usually sum all the numbers together and then divide by the total.

Since a probability distribution tell us which values will be more frequent than others, we compute this mean with a **weighted average**, where the weights are given by the probability distribution.

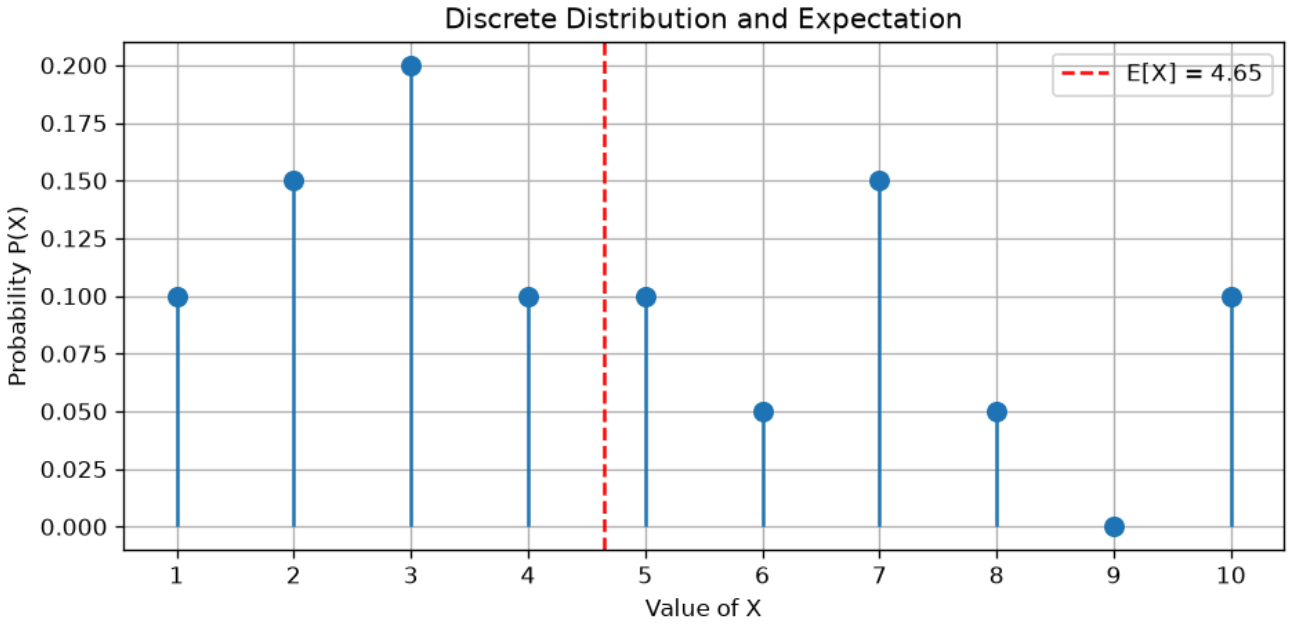
Specifically, we can define the expectation of a **discrete random variable** X as follows:

$$E_{X\sim P}[X] = \sum_{x\in\Omega} xP(x)$$

In the case of **continuous variables**, the expectation takes the form of an integral:

$$E_{X\sim P}[X] = \int_{x\in\Omega} xf(x)dx$$

The diagram below shows an example of expectation:



Variance and Standard Deviation

The variance gives a measure of how much variability there is in a variable X around its mean $E[X]$.

Similarly to the concept of variance as a descriptive measure, we can define the variance in this context as follows:

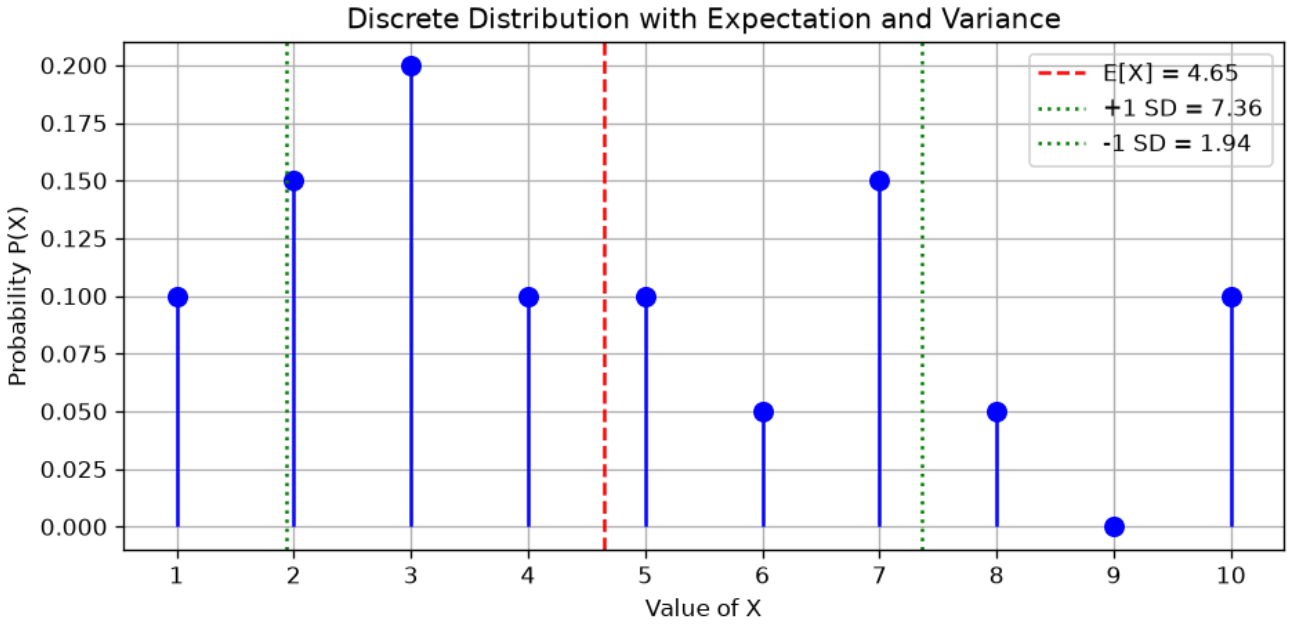
$$Var_{X\sim P}[X] = E_{X\sim P}[(X - E_{X\sim P}[X])^2]$$

As seen with data description, to make the measure of spread more interpretable, we often use the **standard deviation**, which is simply the square root of the variance:

$$Std_{X\sim P}[X] = \sqrt{Var_{X\sim P}[X]}$$

Standard deviation tells us, on average, how far values tend to deviate from the mean, using the same units as the original variable.

The diagram below shows an example:



Covariance

The covariance gives a measure of how two variables are linearly related to each other. It allows to **measure to what extent the increase of one of the variables corresponds to an increase of the value of the other one**.

Given two random variables $X \sim P_X$ and $Y \sim P_Y$, the covariance is defined as follows:

$$Cov_{X \sim P_X, Y \sim P_Y}(X, Y) = E_{X \sim P_X, Y \sim P_Y}[(X - E_{X \sim P_X}[X])(Y - E_{Y \sim P_Y}[Y])]$$

We can distinguish the following terms:

- $E[X]$ and $E[Y]$ are the expectations of X and Y.
- $(X - E[X])$ and $(Y - E[Y])$ are the differences between the samples and the expected values.
- $(X - E[X])(Y - E[Y])$ computes the product between the differences.

We have:

- If the signs of the terms agree, the product is positive.
- If the signs of the terms disagree, the product is negative.

In practice, if when X is larger than the mean, then Y is larger than the mean and vice versa, when X is lower than the mean then Y is lower than the mean, then the two variables are *correlated*, and the covariance is high.

If X is a multi-dimensional variable $X = [X_1, X_2, \ldots, X_n]$, we can compute all the possible covariances between variable pairs: $Cov[X_i, X_j]$. This allows to create a matrix, which is generally referred to as **the covariance matrix**. The general term of the covariance matrix $Cov(X)$ is given by:

$$Cov(X)_{i,j} = \Sigma_{ij} = Cov(X_i, X_j)$$

Entropy (Optional)

Another fundamental way to characterize a probability distribution is through **entropy**, which measures the average uncertainty or unpredictability associated with the possible values of a random variable.

Self-Information

To understand entropy, we first introduce the concept of **self-information**. The self-information of an event x with probability $P(x)$ is defined as:

$$I(x) = -\log_2 P(x)$$

(we will use $\log$ in the future in this context to denote $\log_2$ for brevity)

This definition is motivated by several properties of the logarithm:

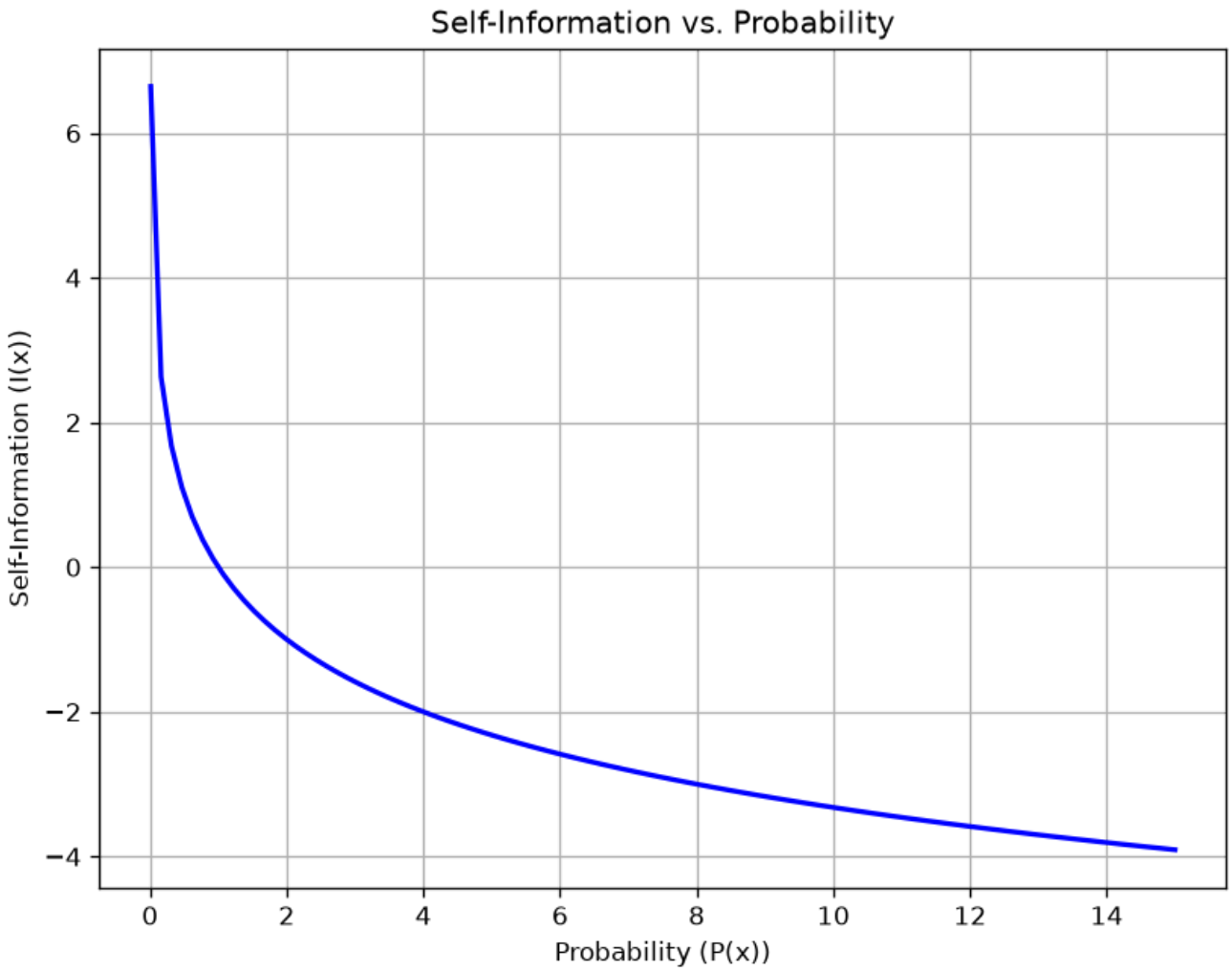
- The logarithm makes self-information **additive** for independent events: the information gained from observing two independent events is the sum of their individual self-informations.

$$I(X = x, Y = y) = -\log\left[P(X = x, Y = y)\right] =$$

$$= \log\left[P(X = x)P(Y = y)\right] = -logP(X = x) - \log P(Y = y) = I(x) + I(y)$$

- The negative logarithm reflects the intuition that **rare events are more informative**: as $P(x)$ decreases, $I(x)$ increases (see plot below)
- The base 2 logarithm measures information in **bits**, which is standard in information theory

For example, if an event is certain ($P(x) = 1$), $I(x) = 0$ (no new information). If an event is very unlikely ($P(x) \to 0$), $I(x) \to \infty$ (maximum surprise).



Entropy of a Distribution

The **entropy** of a random variable X is the average self-information over all possible values of X, weighted by their probabilities (i.e., the expected self-information):

- For discrete variables:

$$H(X) = -\sum_{x \in \Omega} P(x) \log_2 P(x)$$

- For continuous variables (differential entropy):

$$h(X) = -\int_{x \in \Omega} f(x) \log_2 f(x) dx$$

Entropy is highest when the distribution is uniform (maximum uncertainty) and lowest when the distribution is concentrated on a few values (minimum uncertainty).

Note: Entropy is measured in bits if the logarithm base 2 is used and nats if we use a natural logarithm (base e).

Example

- A fair coin ($P(head) = 0.5, P(tail) = 0.5$): $H(X) = 1$ bit.
- A highly biased coin ($P(head) = 0.99, P(tail) = 0.01$): $H(X) \approx 0.08$ bits.

Entropy thus provides a synthetic measure of the uncertainty of a distribution, useful for comparing different random variables and for applications in information theory and machine learning.

Entropy of a Bernoulli variable

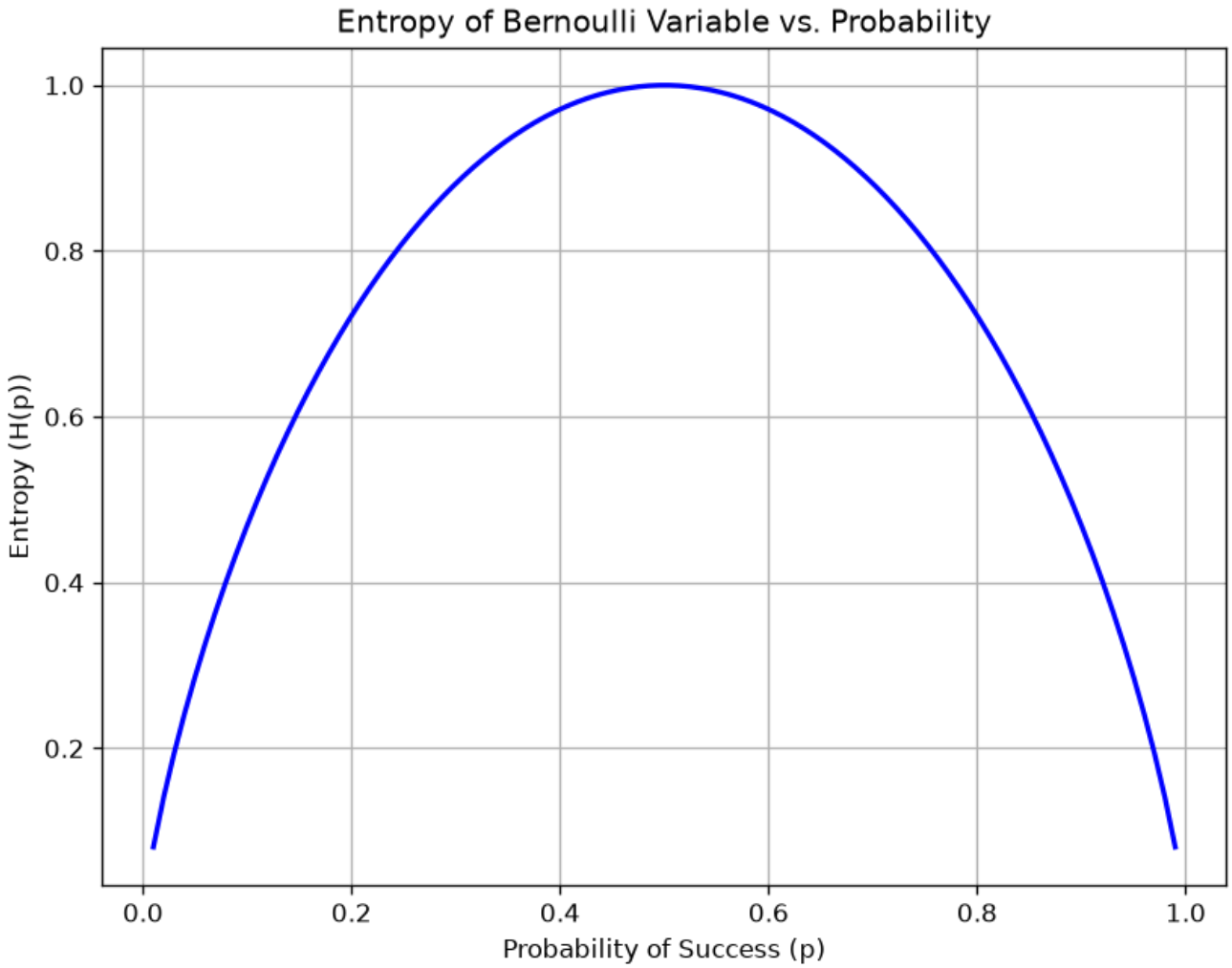
Let's consider a variable X following a Bernoulli distribution with probability p, i.e.:

$$X_p = \begin{cases} 1 & \text{with probability } p \\ 0 & \text{with probability } 1-p \end{cases}$$

Then the entropy of this variable will be:

$$H(X_p) = -p \log p - (-1-p) \log(1-p)$$

If we plot $H(X_p)$ with respect to p, we obtain the following graph:



Standardization

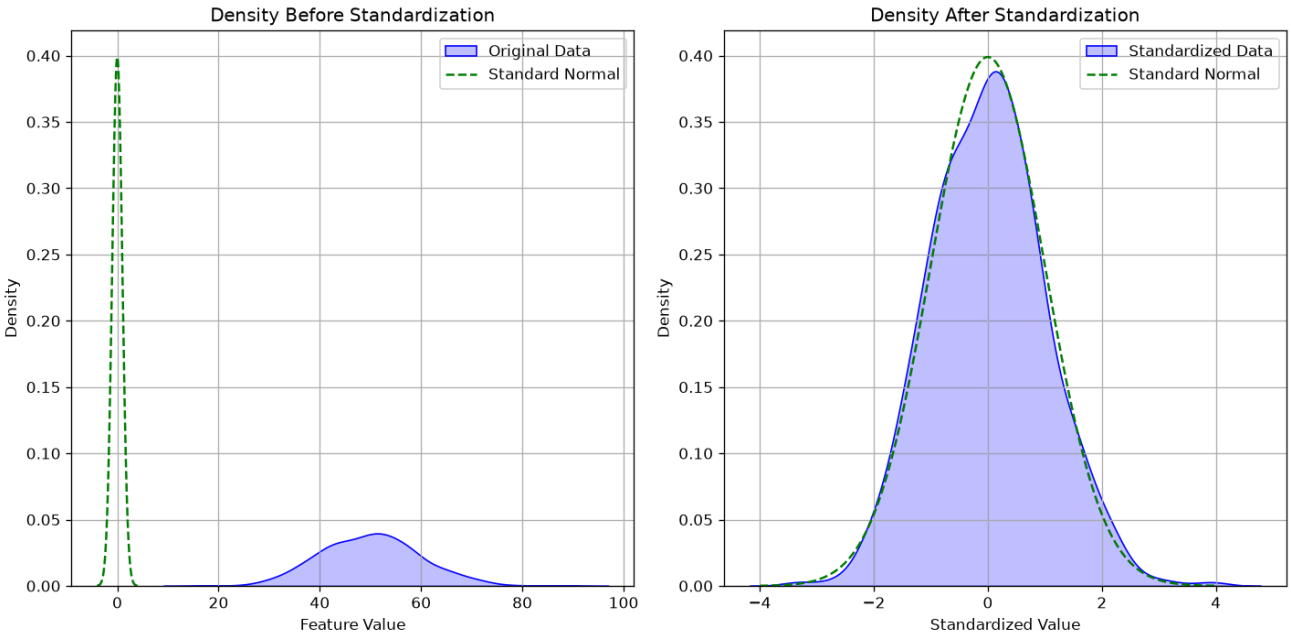
Standardization transforms a random variable X into a variable Z so that it has:

- Expectation equal to zero: $E(Z) = 0$.
- Variance equal to one: $Var(Z) = 1$.

The standardized variable will be:

$$Z = \frac{X - \mu_X}{\sigma_X} = \frac{X - E[X]}{\sqrt{Var[X]}}$$

The plot below shows a data distribution before and after standardization.

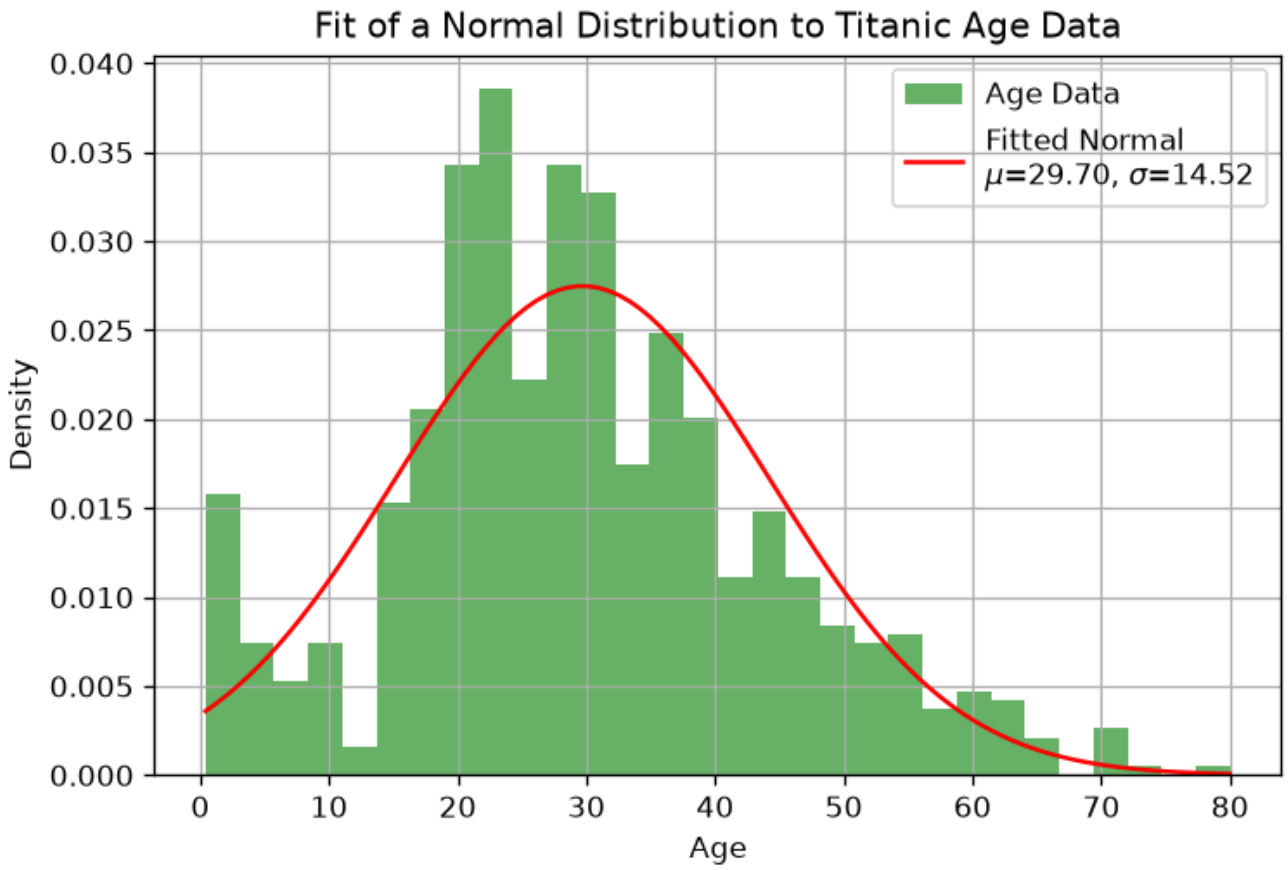


Fitting Distributions to Data

Often, we want to fit a theoretical probability distribution to observed data. This means estimating the parameters of the distribution so that it best describes the data. In Python, this can be done easily using `scipy.stats`.

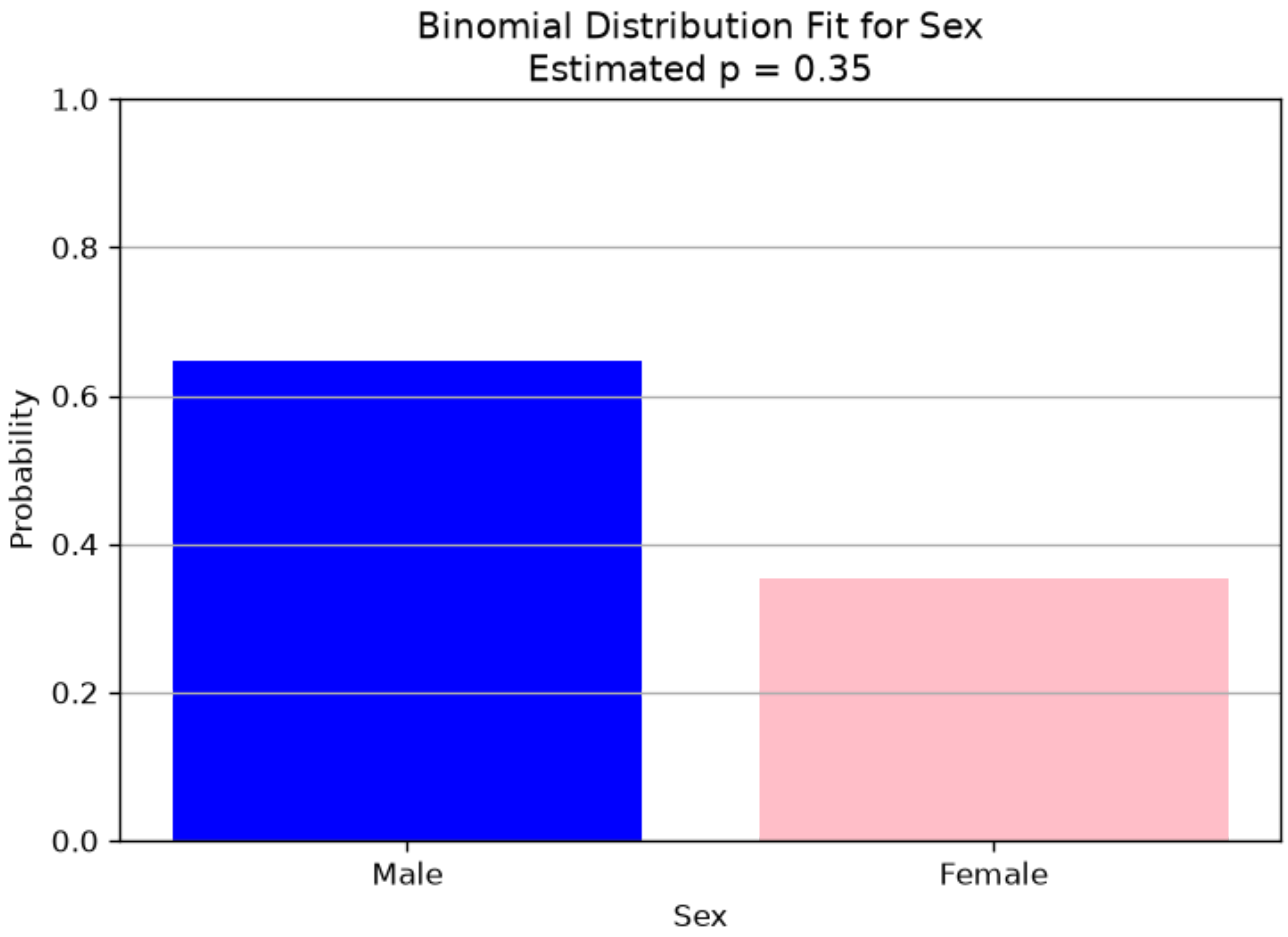
Below is an example of fitting a normal (Gaussian) distribution to a dataset:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Load the Titanic dataset
titanic = pd.read_csv(DATA + "titanic.csv", index_col='PassengerId')

# Extract the Age column and drop missing values
ages = titanic['Age'].dropna()

# Fit a normal distribution to the Age data
mu_hat, sigma_hat = norm.fit(ages)

# Plot the histogram of Age
plt.hist(ages, bins=30, density=True, alpha=0.6, color='g', label='Age Data')

# Plot the fitted normal distribution
x = np.linspace(ages.min(), ages.max(), 100)
pdf_fitted = norm.pdf(x, loc=mu_hat, scale=sigma_hat)
plt.plot(x, pdf_fitted, 'r-', label=f'Fitted Normal\n$\mu$={mu_hat:.2f}, $\sigma$=
        {sigma_hat:.2f}')

plt.xlabel('Age')
plt.ylabel('Density')
plt.title('Fit of a Normal Distribution to Titanic Age Data')
plt.legend()
plt.grid(True)
plt.show()
```



We can do the same with other distribution. For instance, with the binomial distribution:

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import binom

# Load the Titanic dataset
titanic = pd.read_csv(DATA + "titanic.csv", index_col='PassengerId')

# Encode 'Sex': female=1, male=0
sex_numeric = (titanic['Sex'] == 'female').astype(int)

# Estimate p (probability of being female)
p_hat = sex_numeric.mean()
n = len(sex_numeric)

# Binomial PMF for n trials (here, n=1 for each passenger)
x = [0, 1]
pmf = binom.pmf(x, n=1, p=p_hat)

# Plot the PMF
plt.bar(['Male', 'Female'], pmf, color=['blue', 'pink'])
plt.title(f'Binomial Distribution Fit for Sex\nEstimated p = {p_hat:.2f}')
plt.ylabel('Probability')
plt.xlabel('Sex')
plt.ylim(0, 1)
plt.grid(True, axis='y')
plt.show()
```



Exercises

1. **Recognise the shape.** Plot the distribution of `Fare` and of `Age` in the Titanic data. Describe each in words: symmetric or skewed, unimodal or not, heavy-tailed or not. Which summary statistic would you report for each, and why?
2. **Fit and check.** Fit a normal distribution to `Age` and overlay it on the histogram. Then produce a Q-Q plot. Where does the fit fail, and does it matter for a statement about the mean age?
3. **The log-normal.** Plot `Fare` on a logarithmic axis. Does the transformed variable look normal? What does it mean, in the language of the problem, if fares are log-normally distributed?
4. **Simulate the central limit theorem.** Draw 1000 samples of size n from a strongly skewed distribution (an exponential, say) for $n \in \{2, 5, 30, 100\}$, and plot the distribution of the sample means. At which n does it start to look normal?
5. **Discrete distributions.** `SibSp` counts siblings and spouses aboard. Fit a Poisson distribution to it and compare the expected and observed counts. Where does the Poisson model break down, and what feature of the data explains it?

References

- Parts of chapter 1 of [1];

- Most of chapter 3 of [2];
- Parts of chapter 8 of [3].

[1] Bishop, Christopher M. *Pattern recognition and machine learning*. springer, 2006. https://www.microsoft.com/en-us/research/uploads/prod/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf

[2] Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016. https://www.deeplearningbook.org/

[3] Heumann, Christian, and Michael Schomaker Shalabh. Introduction to statistics and data analysis. Springer International Publishing Switzerland, 2016.

© 2026 Antonino Furnari · Department of Mathematics and Computer Science, University of Catania Built with Quarto

Credits and data sources

Statistical inference

Chapter resources

Slides (PDF) Open in Colab Practice quiz

So far, we have seen methods for describing a sample of data (descriptive statistics) and we have reasoned on abstract concepts using basic probability theory concepts. In practice, we are often interested in the properties of a **population**, rather than a sample or some abstract quantities. Examples are:

- What is the percentage of votes each candidate will get at an election?
- What is the proportion of defective goods in a manufacturing process?
- Is there a relationship between smoking and developing a given disease in the world population?

One approach to answer these questions would be to collect the whole population, but this is often unfeasible (e.g., interviewing **all voters**) and sometimes impossible.

The statistician's approach is instead to **sample** a subset of the whole population and try to **infer some of the properties of the population from the sample**. This part of statistics is called **statistical inference**. Analyzing data using such techniques is often called an **inferential analysis**. In this part of the course, we will review different statistical tools for inferential analysis and show some concrete examples, without giving a formal definition of such tools, which is left to other courses.

Sampling

The first step towards an inferential analysis is the **sampling process**. When we acquire a pre-made dataset, sampling is already done, while when we collect data, we are actually sampling from the population. In both cases, it is important to reason on the properties of the sample we will work on.

Simple random sample

The easiest way to sample from a population is **randomly**. A simple random sample makes two assumptions:

- **Unbiasedness**: each element of the population has the same probability of being selected;
- **Independence**: selecting one of the elements of the population does not affect the selection of the other elements in any way.

This approach guarantees that, if we collect a large number of elements, the obtained sample will be a good representative of the population. For instance, if in the population of interest we have 10% of people over 70, we expect this proportion to be roughly represented in the sample as well.

An example of bad sampling:

> We want to ask the inhabitants of a city whether they are satisfied with the quality of life in that city. To sample a large quantity of subjects, we go to the main square and ask passengers to reply to a few questions. If a ground of friends stops we interview all of them to maximize the number of examples we can obtain.

The sampling design outlined above has two important issues:

- **Unbiasedness**: the selection process is biased (**selection bias**). We selected a single location in the city (the main square) and hence we are **oversampling** people who tend to spend time there (e.g., because they work in the city center), versus people who do not spend much time there (e.g., because they work in the periphery).
- **Independence**: when we interview groups of people, in fact, we are breaking the independence assumption, Indeed, selecting one of the people is not independent of selecting others (the members of the same group).

Another example of flawed sampling:

> We want to check how many people believe a given conspiracy theory. To do so, I send a message to all my contacts (500). About 200 of them reply to my message and 180 of them say they do believe that theory. 80% of people actually believe it!

Also here there are important issues:

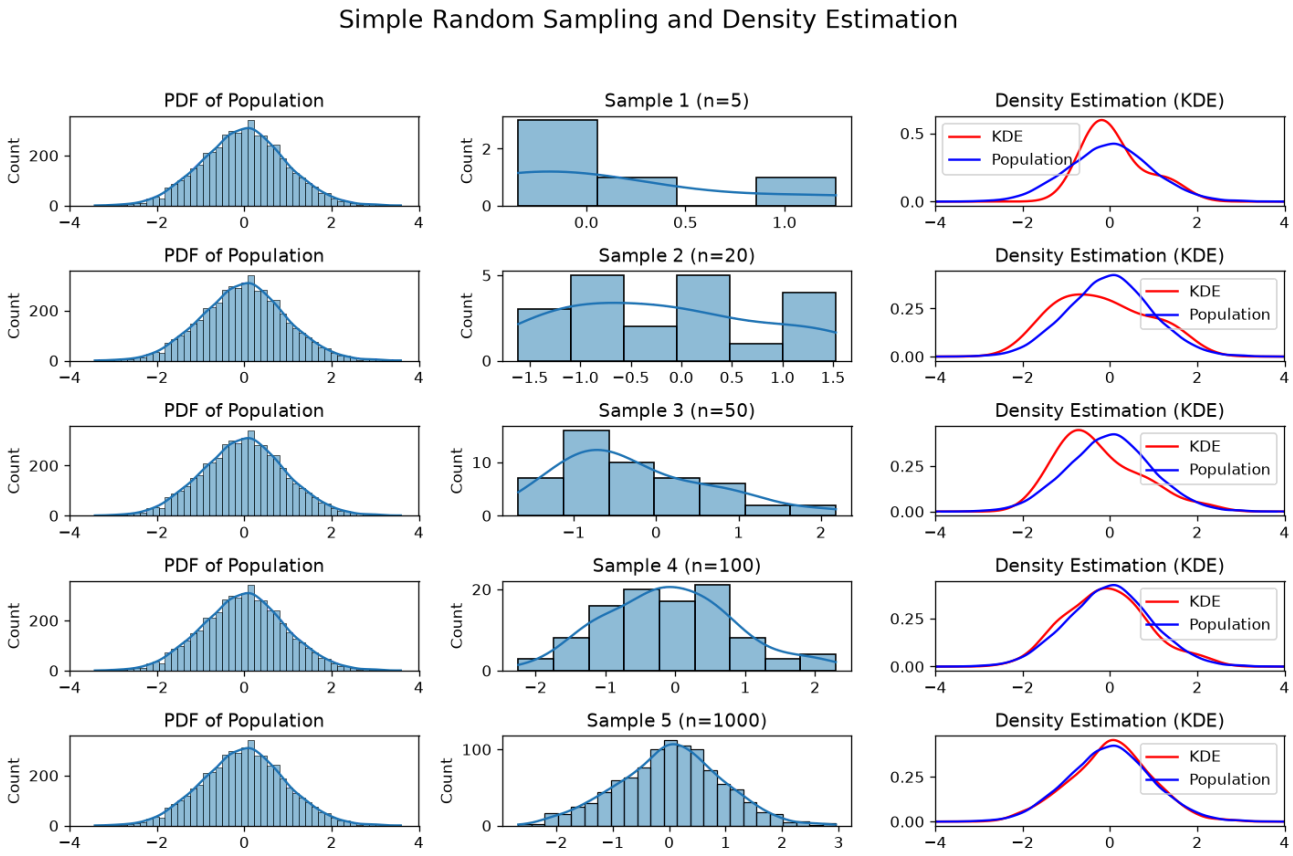
- **Selection bias**: I am not randomly sampling. Instead, I am choosing among my contacts.
- **Response bias**: Only 200 people replied. Chances are that only people who are very motivated will reply. Maybe most of the believers did, while the others just ignored my message.

Another example:

> We interview people on their voting preferences by dialing random phone numbers.

While this may seem sound, we will not end up with a simple random sample because we will not select people without a phone number and we will oversample people with more than one numbers (e.g., work and home).

The plot below shows the importance of sample size:



Stratified Sampling

Sometimes, the population under study is **heterogeneous**, meaning it contains distinct subgroups with different characteristics. In such cases, **simple random sampling** may fail to capture the diversity of the population, leading to **under-representation** of certain groups.

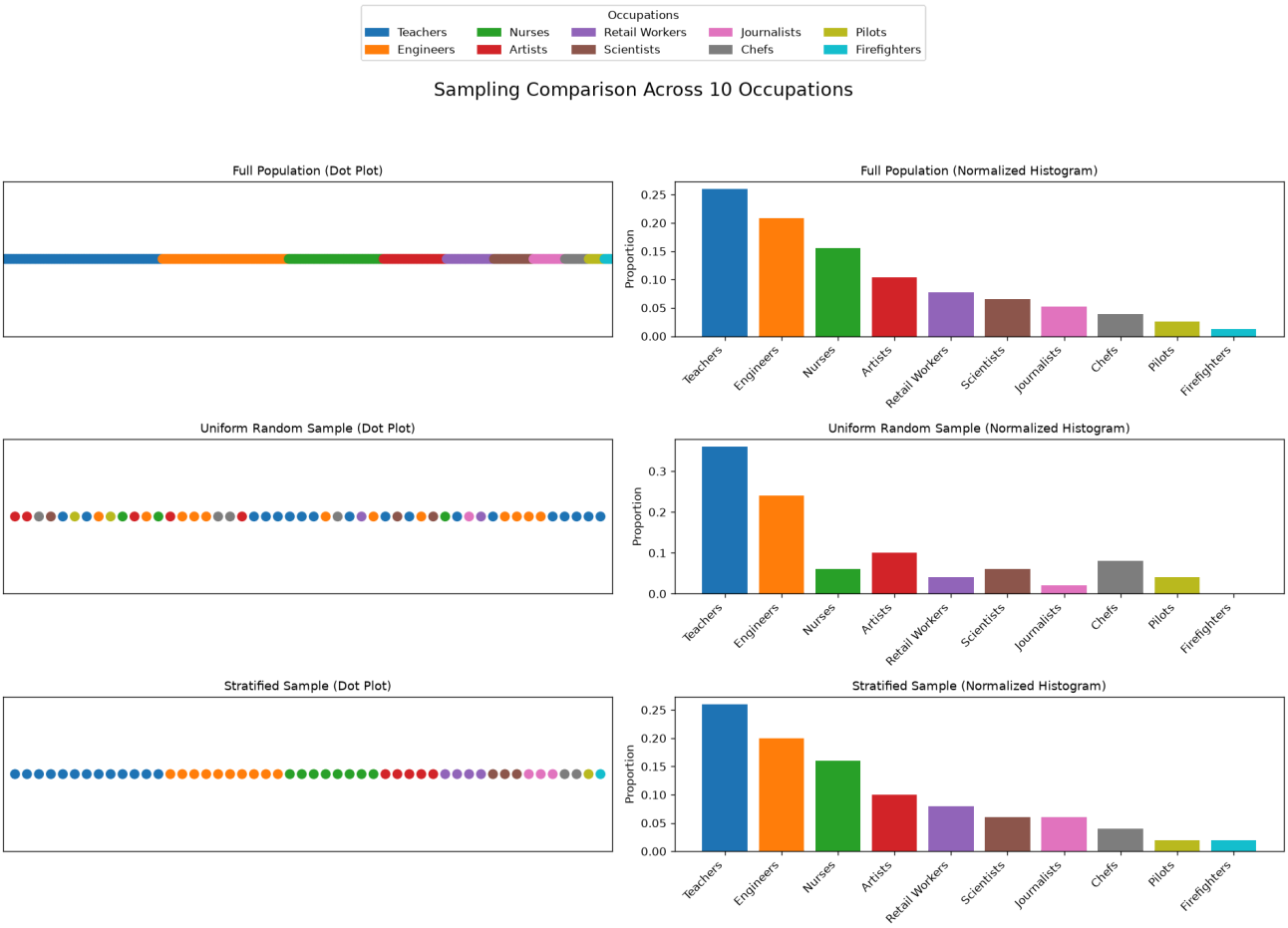
For example, suppose we hypothesize that the variable `Occupation` influences voting preferences. If we randomly sample from the entire population, we might unintentionally select too few individuals from certain occupations, skewing our results.

To address this, we can use **stratified sampling**. This involves:

- Dividing the population into **homogeneous subgroups** (*strata*) based on a relevant variable, such as `Occupation`
- Performing **random sampling within each stratum**, ensuring proportional representation

This method improves representativeness but requires **prior knowledge or assumptions** about the population structure.

The figure below shows a heterogeneous population, a sample obtained via uniform random sampling, and a sample obtained via stratified sampling.

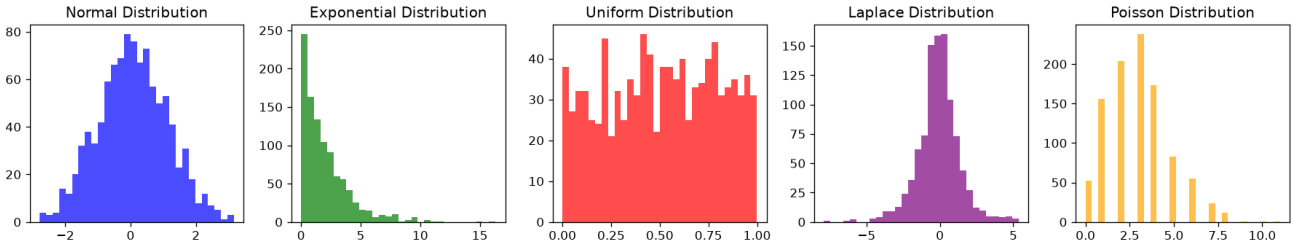


Sampling in Python

In Python we can easily sample from known distributions using numpy `random` module, as shown in the example below:

```python
import numpy as np
import matplotlib.pyplot as plt

# Set the sample size
sample_size = 1000

# Sample from the Normal (Gaussian) Distribution
mean_normal = 0
std_dev_normal = 1
samples_normal = np.random.normal(mean_normal, std_dev_normal, sample_size)

# Sample from the Exponential Distribution
rate_exponential = 0.5
samples_exponential = np.random.exponential(1/rate_exponential, sample_size)

# Sample from the Uniform Distribution
low_uniform = 0
high_uniform = 1
samples_uniform = np.random.uniform(low_uniform, high_uniform, sample_size)

# Sample from the Laplace Distribution
loc_laplace = 0
scale_laplace = 1
samples_laplace = np.random.laplace(loc_laplace, scale_laplace, sample_size)

# Sample from the Poisson Distribution
lam_poisson = 3
samples_poisson = np.random.poisson(lam_poisson, sample_size)

# Create histograms to visualize the distributions
plt.figure(figsize=(15, 3))

plt.subplot(1, 5, 1)
plt.hist(samples_normal, bins=30, color='blue', alpha=0.7)
plt.title('Normal Distribution')

plt.subplot(1, 5, 2)
plt.hist(samples_exponential, bins=30, color='green', alpha=0.7)
plt.title('Exponential Distribution')

plt.subplot(1, 5, 3)
plt.hist(samples_uniform, bins=30, color='red', alpha=0.7)
plt.title('Uniform Distribution')

plt.subplot(1, 5, 4)
plt.hist(samples_laplace, bins=30, color='purple', alpha=0.7)
plt.title('Laplace Distribution')
```

```
plt.subplot(1, 5, 5)
plt.hist(samples_poisson, bins=30, color='orange', alpha=0.7)
plt.title('Poisson Distribution')

plt.tight_layout()
plt.show()
```



We can also sample from an existing sample to obtain a **subsample**. This can be done starting from a dataset. For instance, let us consider the *weight-height* dataset:

```
import pandas as pd
data=pd.read_csv(DATA + "height_weight.csv")
data.info()
data.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4231 entries, 0 to 4230
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   sex     4231 non-null   object
 1   BMI     4231 non-null   float64
 2   height  4231 non-null   float64
 3   weight  4231 non-null   float64
dtypes: float64(3), object(1)
memory usage: 132.3+ KB
```

	sex	BMI	height	weight
0	M	33.36	187.96	117.933920
1	M	26.54	177.80	83.914520
2	F	32.13	154.94	77.110640
3	M	26.62	172.72	79.378600
4	F	27.13	167.64	76.203456

We can obtain a subsample of 1000 observations as follows:

```
sample1=data.sample(1000)
sample1.info()
sample1.head()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1000 entries, 129 to 3443
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   sex     1000 non-null   object
 1   BMI     1000 non-null   float64
 2   height  1000 non-null   float64
 3   weight  1000 non-null   float64
dtypes: float64(3), object(1)
memory usage: 39.1+ KB
```

	sex	BMI	height	weight
129	M	33.90	182.88	113.398000
3065	F	22.32	162.56	58.966960
2755	M	31.68	185.42	108.862080
3505	F	21.65	165.10	58.966960
1785	F	20.92	162.56	55.338224

Sampling will be random and uniform. As can be seen the indexes of the rows are shuffled. By default, Pandas will sample without replacement. We can sample with replacement specifying `replace=True`. For instance, we can obtain a larger sample (with repetitions) as follows:

```
sample2=data.sample(5000, replace=True)
sample2.info()
sample2.head()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 5000 entries, 1637 to 1037
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   sex     5000 non-null   object
 1   BMI     5000 non-null   float64
 2   height  5000 non-null   float64
 3   weight  5000 non-null   float64
dtypes: float64(3), object(1)
memory usage: 195.3+ KB
```

	sex	BMI	height	weight
1637	F	26.53	157.48	65.77084
4082	M	26.54	177.80	83.91452
1890	F	28.33	152.40	65.77084
3205	M	24.39	177.80	77.11064
477	F	24.02	162.56	63.50288

We can perform stratified sampling as follows:

```
# Define the stratification variable
strata_variable = 'sex'

# Number of samples per stratum
samples_per_stratum = 500

# Draw the same number of rows from every stratum. `.sample()` on a groupby does
# this directly, and unlike `.apply()` it keeps the grouping column.
stratified_sample = (
    data.groupby(strata_variable, group_keys=False)
        .sample(samples_per_stratum)
        .reset_index(drop=True)
)

# Inspect the result
stratified_sample.info()
stratified_sample.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   sex     1000 non-null   object
 1   BMI     1000 non-null   float64
 2   height  1000 non-null   float64
 3   weight  1000 non-null   float64
dtypes: float64(3), object(1)
memory usage: 31.4+ KB
```

	sex	BMI	height	weight
0	F	26.31	170.18	76.203456
1	F	27.45	167.64	77.110640
2	F	28.29	165.10	77.110640
3	F	30.03	162.56	79.378600
4	F	29.16	162.56	77.110640

Note that, with this method, sampled observations will be sorted by `sex`. To get rid of this bias, we can shuffle the dataframe as follows:

```
stratified_sample = stratified_sample.iloc[np.random.choice(len(stratified_sample),
        len(stratified_sample), replace=False)]
stratified_sample.head()
```

	sex	BMI	height	weight
898	M	26.55	175.26	81.646560
452	F	29.01	162.56	76.657048
665	M	28.89	187.96	102.058200
200	F	31.26	152.40	72.574720
378	F	21.92	170.18	63.502880

Males and females will be perfectly balanced by design in this stratified sample:

```
stratified_sample['sex'].value_counts()
```

```
sex
M    500
F    500
Name: count, dtype: int64
```

Sampling Distribution of the Mean

Let's consider this problem:

> A bakery sells packages of cookies labeled as weighing **1kg**. Due to small variations in the production process, the actual weight of each package may differ slightly. To ensure quality, the bakery wants to verify that the average weight is close to the target.

We take a **random sample of** $n = 1000$ packages and measure their weights $x_1, x_2, \ldots, x_n$. We compute the sample mean:

$$\overline{x} = \frac{x_1 + x_2 + \ldots + x_n}{n} = 1000.2\text{g}$$

This value is close to the target of 1000g, so we might conclude that the production process is accurate. But what if we took another sample? Would we get the same result?

$$\overline{x} = \frac{x_1 + x_2 + \ldots + x_n}{n} = 999.7\text{g}$$

This is still close to the target, but slightly lower than before. If we repeated this process many times, we would get a **distribution of sample means**.

We treat each package's weight as a **random variable** X_i, with:

$$E[X_i] = \mu$$
$$Var[X_i] = \sigma^2$$

where μ and σ are the **true mean and standard deviation** of the population.

The sample mean $\overline{X}$ is also a random variable:

$$\overline{X} = \frac{X_1 + X_2 + \ldots + X_n}{n}$$

Note that, while we cannot make assumptions for the individual X_i, for the **central limit theorem**, we know that:

> For large n, the distribution of $\overline{X}$ will be approximately **normal**, regardless of the shape of the original data.

We can derive the expected value, variance and standard deviation of X as follows:

$$E[\overline{X}] = E\left[\frac{1}{n}\sum_{i=1}^{n} X_i\right] = \frac{1}{n}\sum_{i=1}^{n} E[X_i] = \mu$$

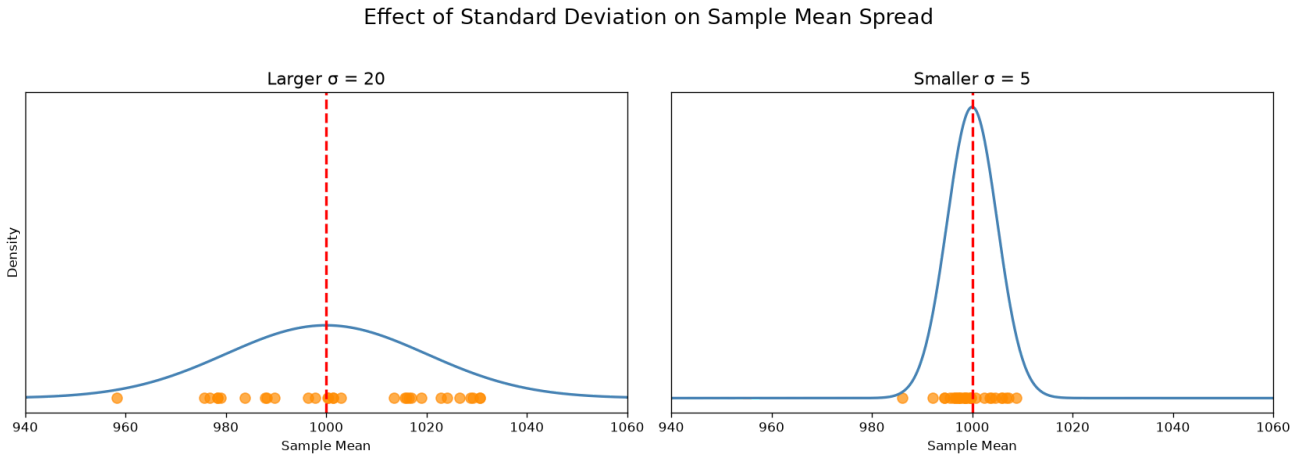
$$Var[\overline{X}] = Var\left[\frac{1}{n}\sum_{i=1}^{n} X_i\right] = \frac{1}{n^2}\sum_{i=1}^{n} Var[X_i] = \frac{\sigma^2}{n}$$

$$Std[\overline{X}] = \frac{\sigma}{\sqrt{n}}$$

We note that:

- The **expected value of the sample means** (the mean of the means) converges to the **mean of the population**. This is the value we want to estimate: the actual average weight of packages.
- The **standard deviation of the distribution** quantifies the precision according to which we can measure the mean. A small standard deviation indicates that sample means have small variability, so a single estimate will likely be closer to the true mean and hence more reliable.

This is exemplified by the following figure:



Standard Error

While useful to quantify our precision in the estimation of the true mean, the standard deviation depends on the variance of the population, which is unknown.

In practice, we estimate it using the sample standard deviation:

$$s_{n-1} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2}$$

and define the Standard Error (SE) as follows:

$$SE(\overline{x}) = \frac{s_{n-1}}{\sqrt{n}}$$

Similar to the standard deviation, **the standard error quantifies our uncertainty in the estimation of the true mean**.

Note that the standard error is **inversely proportional to the square root of the sample size**:

$$SE(\overline{x}) \propto \frac{1}{\sqrt{n}}$$

This relationship is common in many statistical estimators. It means that:

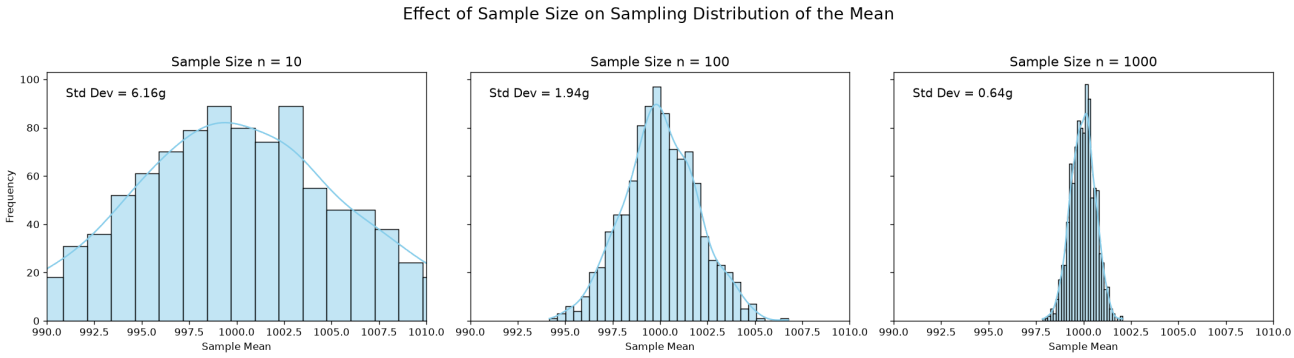
- If we **double the sample size**, the error **reduces by about 30%**.

- To **halve the error**, we need to **quadruple the sample size**.

This is a key insight: reducing uncertainty requires much more data than we might expect.

The standard error tells us how close our sample mean $\overline{x}$ is likely to be to the true mean μ. A small standard error means that repeated samples will yield similar results, and our estimate is precise. A large standard error means more variability and less confidence in the estimate.

This is shown in the graph below:



This figure illustrates how **sample size affects the precision of the sample mean**. Each panel shows the distribution of sample means obtained by repeatedly sampling from the same population (mean = 1000g, standard deviation = 20g), but with different sample sizes:

- **Left (n = 10)**: The distribution is wide, indicating high variability. Small samples lead to more fluctuation in the estimated mean.
- **Center (n = 100)**: The distribution narrows, showing improved stability. The sample means are more tightly clustered around the true mean.
- **Right (n = 1000)**: The distribution is very narrow. Large samples yield highly consistent estimates, with minimal deviation from the true mean.

This visual reinforces the key idea that **larger samples reduce the standard error**, making our estimates more reliable. The spread of the sampling distribution shrinks proportionally to $1/\sqrt{n}$.

t-Student Distribution

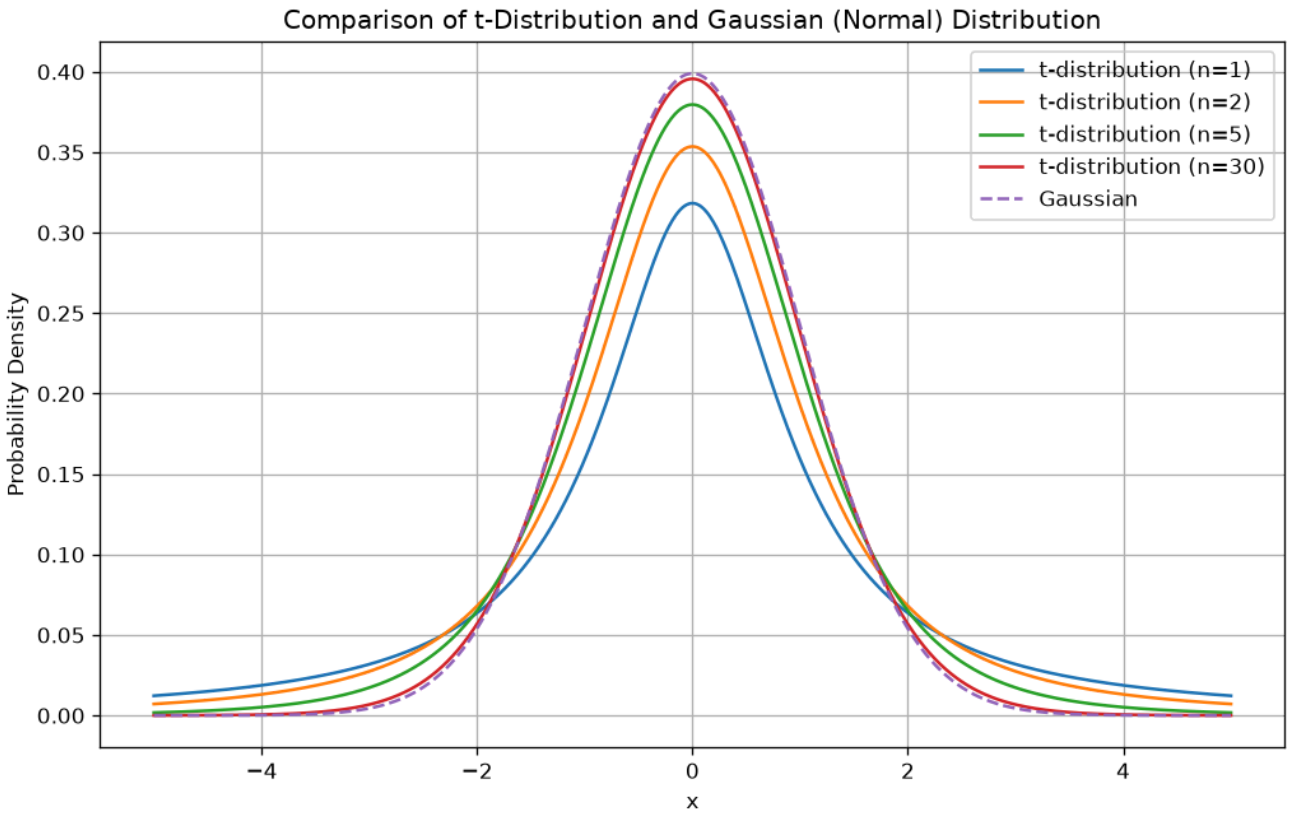
When n is small, the distribution of $\overline{X}$ is **not exactly normal**. Instead, we use the **t-Student distribution**, which accounts for extra uncertainty:

$$t_{n-1} = \frac{\overline{X} - \mu}{s_{n-1}/\sqrt{n}}$$

This follows a t-distribution with $n - 1$ degrees of freedom. The t-distribution looks like a Gaussian but has **heavier tails**, meaning more room for extreme values. As n increases, the t-distribution **converges to the normal distribution**.

Note that the t-Student distribution is obtained by applying standardization using the standard error rather than the standard deviation (which is unknown).

The relationship between a t-Student and Gaussian distribution is shown in the figure below:



As n gets larger, the t-Distribution approximates a Gaussian distribution.

We can now characterize how the means will distribute. In particular, **for large values of n, sample means will distribute according to a Gaussian distribution with standard deviation equal to the sample standard deviation**.

Confidence Intervals

Let's return to our bakery example. We want to estimate the **true average weight** μ of cookie packages, which are labeled as 1kg. We take a sample of $n = 1000$ packages and compute:

$$\overline{x} = 1000.2\text{g}$$

This is our estimate of the true mean. However, we know that this value may vary depending on the sample we draw. If we took another sample, we might get a slightly different result. It is interesting to define the **bounds of such variation**, to understand where the true mean μ is likely to be found relative to our estimate $\overline{x}$.

Remember that the sample mean $\overline{X}$ follows a **Gaussian distribution** centered at μ due to the **Central Limit Theorem**.

We also know that about 68.3% of the density of the Gaussian distribution will be within $\mu - \sigma$ and $\mu + \sigma$, so we can write:

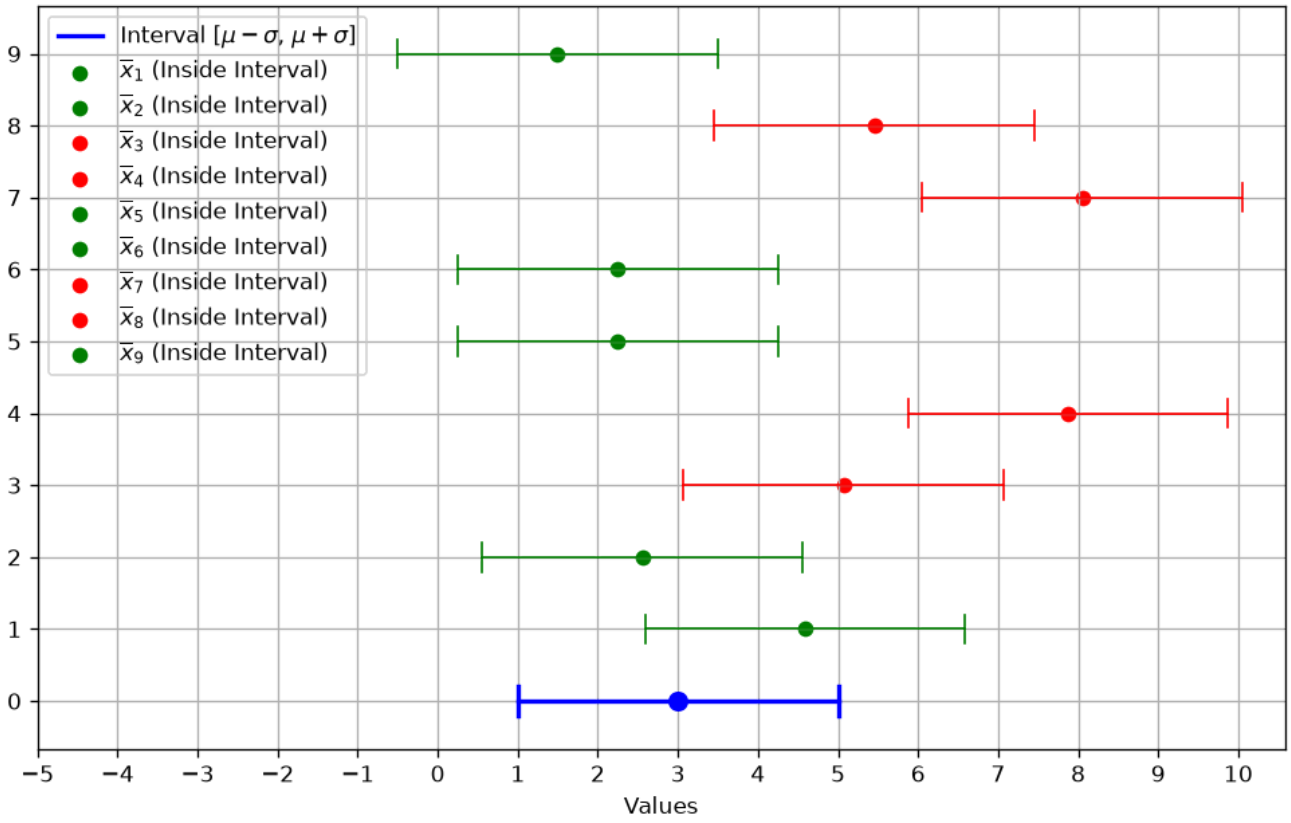
$$P\left(\mu - \sigma \leq \overline{x} \leq \mu + \sigma\right) = 0.683$$

Which means that, if we perform several independent samplings, with sample size n, the probability of obtaining a sample mean $\overline{x}$ in the range $[\mu - \sigma, \mu + \sigma]$ is 68.3%.

It is easy to show that:

$$\overline{x} \in [\mu - \sigma, \mu + \sigma] \Leftrightarrow \mu \in [\overline{x} - \sigma, \overline{x} + \sigma]$$

This is graphically shown in the plot below. The blue segment is the one of bounds $[\mu - \sigma, \mu + \sigma]$. Note that, all times a point $\overline{x}$ happens to be in the blue segment centered around μ, then μ is in the segment centered around $\overline{x}$.



This allows us to write:

$$P\left(\overline{x} - \sigma \leq \mu \leq \overline{x} + \sigma\right) = 0.683$$

which has a powerful interpretation:

If we draw many independent samples of size n and compute $\overline{x}$ from the samples, the true mean μ will lie in the interval $[\overline{x} - \sigma, \overline{x} + \sigma]$ 68.3% of the times

Alternatively

We can say with a confidence of 68.3% that the true mean will be in the $[\overline{x} - \sigma, \overline{x} + \sigma]$ interval.

In this context, $[\overline{x} - \sigma, \overline{x} + \sigma]$ is called a **confidence interval**.

We still have to compute actual numbers for our confidence interval, but we don't have the standard deviation σ. In practice, we replace it with the standard error and obtain the confidence interval:

$$[\overline{x} - SE(\overline{x}), \overline{x} + SE(\overline{x})]$$

We note that 68.3 is a very peculiar number and also a relatively low probability. In general, given a chosen percentage p, that we call a **confidence level**, we can obtain a number β such that:

$$P\left(\overline{x} - \beta\sigma \leq \mu \leq \overline{x} + \beta\sigma\right) = p$$

leading to the following confidence interval:

$$[\overline{x} - \beta SE(\overline{x}), \overline{x} + \beta SE(\overline{x})]$$

Alternatively, we will say that we chose a **significance level** $\alpha = 0.05$, which quantifies the number of times we admin the true mean will be out of the confidence interval.

It is very common to set $\alpha = 0.05$, which leads to $\beta = 1.96$ and the following confidence interval:

$$[\overline{x} - 1.96 \cdot SE(\overline{x}), \overline{x} + 1.96 \cdot SE(\overline{x})]$$

Example: Confidence Interval for Cookie Weights

Let's suppose we want to estimate the **true average weight** μ of cookie packages sold by a bakery. We take a sample of $n = 1000$ packages and compute the sample mean:

$$\overline{x} = 1000.2\text{g}$$

We also compute the sample standard deviation:

$$s_{n-1} = 0.1\text{g}$$

Our estimate $\overline{x}$ is close to the labeled weight of 1000g, but we know it may vary depending on the sample. It is useful to define the **bounds of such variation**, to understand where the true mean μ is

likely to be found relative to our estimate.

To do this, we build a **confidence interval**. We first choose a **confidence level**. For example, if we want to be **95% confident**, we define:

$$\alpha = 0.05$$

This means we accept a 5% chance that the true mean lies outside our interval. Since the sample mean $\overline{X}$ follows a **Gaussian distribution**, we know that **95% of the density** lies within 1.96 standard deviations from the mean. So we write:

$$P\left(\overline{X} - 1.96 \cdot SE(\overline{X}) \leq \mu \leq \overline{X} + 1.96 \cdot SE(\overline{X})\right) = 0.95$$

We compute the standard error:

$$SE(\overline{x}) = \frac{s_{n-1}}{\sqrt{n}} = \frac{0.1}{\sqrt{1000}} = 0.0032\text{g}$$

Then the confidence interval becomes:

$$[1000.2 - 1.96 \cdot 0.0032,\ 1000.2 + 1.96 \cdot 0.0032] = [1000.194,\ 1000.206]$$

This means we are **95% confident** that the true average weight of cookie packages lies between **1000.194g and 1000.206g**.

Since the interval is narrow, we can say that our estimate is precise. If the sample size were smaller, the interval would be wider, reflecting greater uncertainty in the estimate.

Computing Confidence Intervals in Practice

We have seen how to compute confidence interval “by hand” in the case of the estimation of proportions (defect rate).In practice, depending on the quantities for which we want to estimate confidence bounds, we will need to use different distributions. For instance, when estimating means, we will have to use the t-Student distribution with $n - 1$ degrees of freedom. We will not see all methods in detail, but the main libraries implement all confidence bounds estimation procedure for us.

The main estimation procedures are related to:

- Estimation of confidence bounds for means;
- Estimation of confidence bounds for variances;
- Estimation of confidence bounds for proportions.

We will see how to compute these practically in the laboratory sessions.

Confidence Intervals for Means

We can compute confidence intervals for the estimation of means with `scipy`. Let us see how to compute confidence intervals for the mean of `height`:

```
from scipy import stats
mean = data['height'].mean()
std = np.std(data['height'])
standard_error = std/np.sqrt(len(data))

confidence_level = 0.95

interval = stats.norm.interval(confidence_level, loc=mean, scale=standard_error)

print(f"Estimated mean: {mean:0.2f}")
print(f"Standard deviation of the sample: {std:0.2f}")
print(f"Standard error of the sample: {standard_error:0.2f}")
print(f"Confidence interval: [{interval[0]:0.2f}, {interval[1]:0.2f}]")
```

```
Estimated mean: 169.89
Standard deviation of the sample: 9.96
Standard error of the sample: 0.15
Confidence interval: [169.59, 170.19]
```

Let us set the confidence level to 0.99:

```
from scipy import stats
mean = data['height'].mean()
std = np.std(data['height'])
standard_error = std/np.sqrt(len(data))

confidence_level = 0.99

interval = stats.norm.interval(confidence_level, loc=mean, scale=standard_error)

print(f"Estimated mean: {mean:0.2f}")
print(f"Standard deviation of the sample: {std:0.2f}")
print(f"Standard error of the sample: {standard_error:0.2f}")
print(f"Confidence interval: [{interval[0]:0.2f}, {interval[1]:0.2f}]")
```

```
Estimated mean: 169.89
Standard deviation of the sample: 9.96
Standard error of the sample: 0.15
Confidence interval: [169.50, 170.29]
```

Confidence Intervals for Variances

To compute confidence intervals for the estimation of variances, we have to use the χ^2 distribution:

```
import numpy as np
from scipy import stats

# Set the desired confidence level
confidence_level = 0.95  # Change this to the desired confidence level (e.g., 0.95
        for 95% confidence)

# Calculate the confidence interval for the population variance
confidence_interval = stats.chi2.interval(confidence_level, df=len(data) - 1)

# Calculate the sample variance
sample_variance = np.var(data['height'], ddof=1)  # ddof=1 for sample variance

# Calculate the lower and upper bounds of the confidence interval
variance_lower = (len(data) - 1) * sample_variance / confidence_interval[1]
variance_upper = (len(data) - 1) * sample_variance / confidence_interval[0]

print(f"Sample Variance: {sample_variance:.2f}")
print(f"Confidence Interval for Variance: ({variance_lower:.2f},
        {variance_upper:.2f})")
```

```
Sample Variance: 99.13
Confidence Interval for Variance: (95.04, 103.49)
```

Confidence Intervals for Proportion

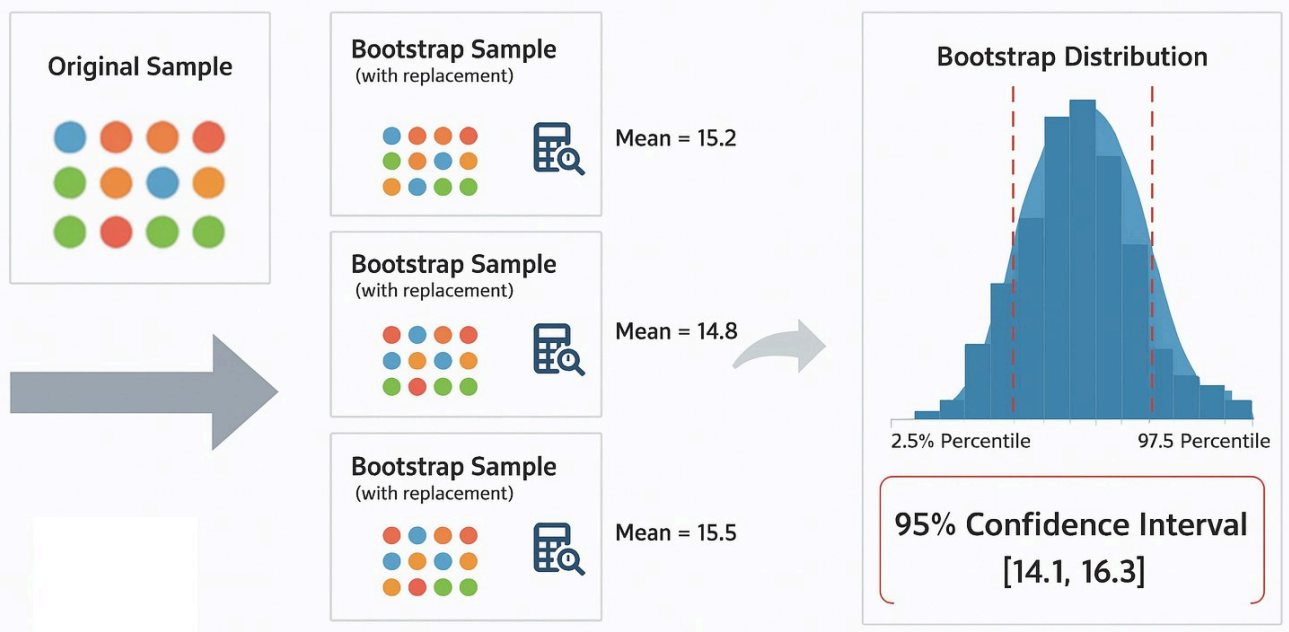
Let us see how to compute confidence intervals for the proportion of females over males:

```
import statsmodels.api as sm
import numpy as np

females = data['sex'].value_counts()['F']
total = len(data)

# Set the desired confidence level
confidence_level = 0.95  # Change this to the desired confidence level (e.g., 0.95
        for 95% confidence)

# Calculate the proportion (sample proportion)
proportion = females / total

# Compute the confidence interval for the proportion
conf_interval = sm.stats.proportion_confint(females, total, alpha=1 -
        confidence_level, method='normal')

print(f"Sample Proportion: {proportion:.2f}")
print(f"Confidence Interval for Proportion: ({conf_interval[0]:.2f},
        {conf_interval[1]:.2f})")
```

```
Sample Proportion: 0.54
Confidence Interval for Proportion: (0.53, 0.56)
```

Bootstrapping: Inference Through Resampling

So far, we've built confidence intervals using formulas that rely on the Central Limit Theorem and assumptions about the underlying distribution (e.g., normality). But **what if our sample size is small**, the distribution is clearly not normal, or **we want to find a confidence interval for a statistic where no simple formula exists (like the median)**?



This is where **bootstrapping** comes in. It's a powerful computational method that allows us to estimate the sampling distribution of any statistic using only our original sample. The name comes from the phrase "to pull oneself up by one's own bootstraps", reflecting the idea that we use the sample itself to learn about its own uncertainty.

The core idea is to treat the **sample as a stand-in for the population**. We then simulate the process of drawing new samples by resampling *from our own sample*.

The process is straightforward and relies on computation rather than complex formulas:

1. **Start with your original sample** of size n.
2. **Create a "bootstrap sample"** by drawing n observations from your original sample **with replacement**. This new sample will be the same size as the original, but some data points may appear multiple times, while others may not appear at all.
3. **Calculate the statistic of interest** (e.g., mean, median, standard deviation) for this bootstrap sample.
4. **Repeat steps 2 and 3 thousands of times** (e.g., 10,000 times), recording the statistic each time.
5. **Build the bootstrap distribution**. The collection of all the statistics you calculated forms an empirical sampling distribution. We can now use this distribution to estimate properties like the standard error or confidence intervals.

For a 95% confidence interval, we can simply take the 2.5th and 97.5th percentiles of our bootstrap distribution.

Example: Bootstrap Confidence Interval for Mean Height

Let's use bootstrapping to find the 95% confidence interval for the mean height in our `height_weight` dataset.

```
import numpy as np
import pandas as pd
from scipy import stats

# Load the dataset
data = pd.read_csv(DATA + "height_weight.csv")
original_sample = data['height']

# The data must be passed as a sequence (e.g., a tuple)
data_tuple = (original_sample,)

# Use scipy.stats.bootstrap to perform the resampling and calculate the interval
# We pass the data, the statistic to compute (np.mean), and the number of resamples.
bootstrap_result = stats.bootstrap(data_tuple, np.mean, n_resamples=10000,
        confidence_level=0.95)

# Extract the confidence interval from the result object
confidence_interval = bootstrap_result.confidence_interval

print(f"Original sample mean: {original_sample.mean():.2f}")
print(f"SciPy Bootstrap 95% Confidence Interval for the Mean:
        [{confidence_interval.low:.2f}, {confidence_interval.high:.2f}]")
```

```
Original sample mean: 169.89
SciPy Bootstrap 95% Confidence Interval for the Mean: [169.60, 170.19]
```

Bias and Variance of Estimators

Let's return to our bakery example. We want to estimate the **true average weight** μ of cookie packages. To do this, we take a sample of n packages and compute the sample mean:

$$\overline{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

This formula is called an **estimator** of the population mean. The value we obtain from a specific sample is called an **estimate**. Since the sample changes each time we repeat the experiment, the estimate will vary.

It is useful to study how this estimator behaves across different samples—specifically, we want to understand its **bias** and **variance**.

Bias of an Estimator

Let X be a random variable and let $x = (x_1, x_2, \ldots, x_n)$ be a sample from the population. Let $T(X)$ be an estimator of a population quantity ϕ (for instance, the mean weight). Then:

$$T(X) = \frac{1}{n}\sum_{i=1}^{n} x_i$$

is our estimator for ϕ. Since the sample changes, $T(X)$ is a **random variable**.

The **bias** of the estimator is defined as:

$$Bias_\phi(T(X)) = E[T(X)] - \phi$$

This measures whether the estimator systematically overestimates or underestimates the true value. If:

$$E[T(X)] = \phi$$

then the estimator is **unbiased**, meaning that, on average, it gives the correct result.

In our case, the sample mean $\overline{x}$ is an **unbiased estimator** of the population mean. If we repeated the sampling many times and averaged all the sample means, we would get a value close to μ.

Unbiased Estimator for the Variance

To estimate the variance of cookie weights, we may compute:

$$s_n^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})^2$$

This formula is **biased**—it tends to underestimate the true variance σ^2. Specifically:

$$E[s_n^2] = \frac{n-1}{n}\sigma^2$$

To correct this, we use the **unbiased estimator**:

$$s_{n-1}^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2$$

This is the version we use when estimating the standard error and building confidence intervals. For large n, the difference between s_n^2 and s_{n-1}^2 becomes negligible.

Variance of an Estimator

The **variance** of an estimator tells us how much the estimate fluctuates across different samples. It is defined as:

$$Var(T(X)) = E[(T(X) - E[T(X)])^2]$$

A low variance means that repeated samples give similar results. A high variance means that estimates are unstable and vary widely.

Bias-Variance Tradeoff

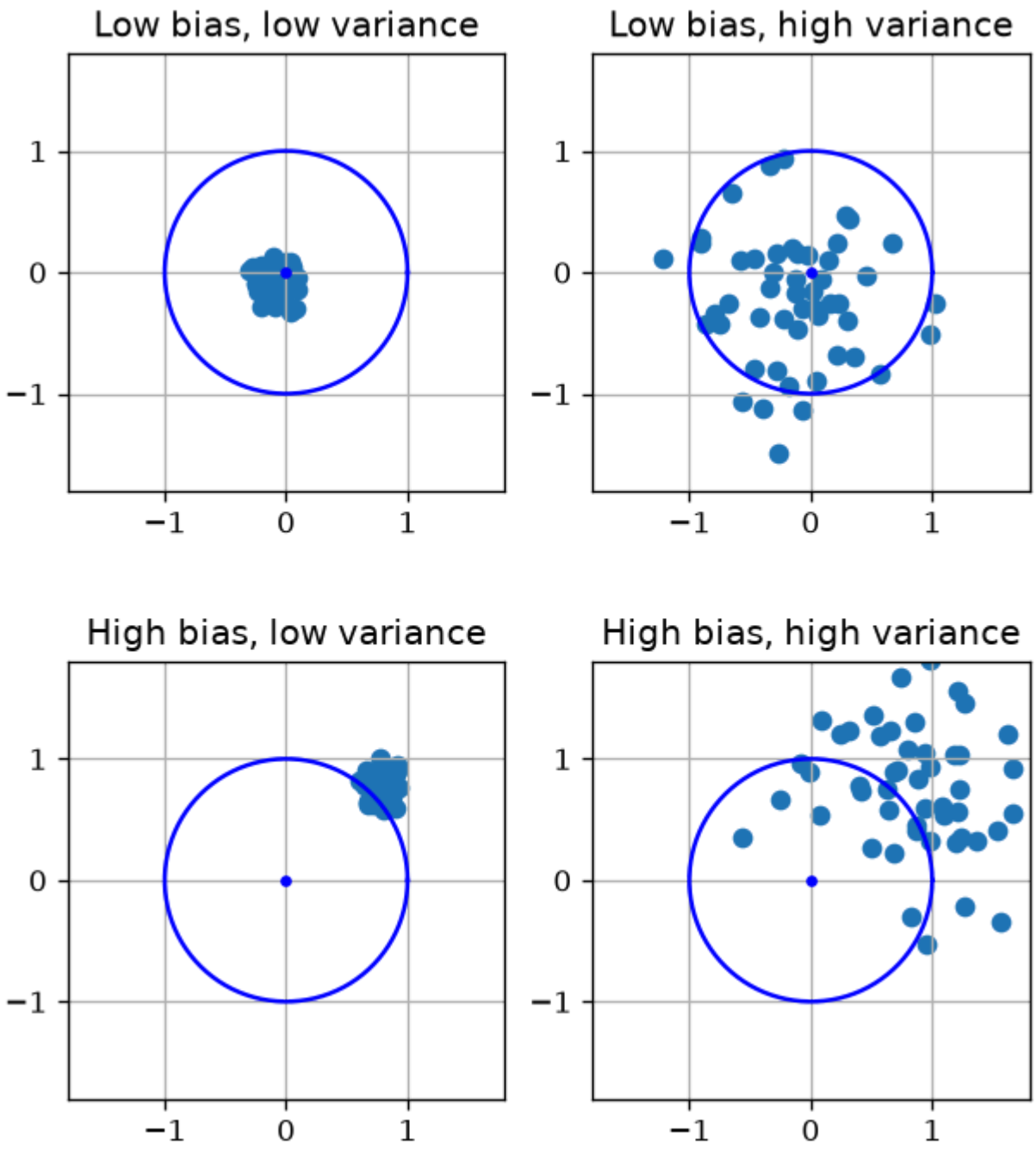
Ideally, we want an estimator with **low bias** and **low variance**. This means that:

- The estimates are close to each other (low variance),
- And they are close to the true value (low bias).

In practice, we can visualize four scenarios:

- **Low bias, low variance**: estimates are tightly clustered around the true value.
- **Low bias, high variance**: estimates are scattered but centered correctly.
- **High bias, low variance**: estimates are consistent but systematically wrong.
- **High bias, high variance**: estimates are scattered and off-target.

This is often illustrated as a target with darts: the true value is the bullseye, and each dart is an estimate. The goal is to hit close to the center, consistently.



The four cases are:

- Low bias, low variance: all estimates will be close to the true value;
- Low bias, high variance: in average, estimates will be close to the true value, but different estimates may greatly differ;
- High bias, low variance: while different estimates will be similar, they all are very far away from the true value;
- High bias, high variance: we don't have many guarantees - estimates will all be different, but also far from the real value, even in average.

It's clear that having low bias and low variance is desirable, but, as we will see, this is not always easy to achieve. In practice, we'll see that in many cases it there is a trade-off between bias and variance, meaning that **we can tweak our estimator to find a balance between these two properties**.

Hypothesis Testing

Confidence intervals provide a range of plausible values for a population parameter based on a sample. A **hypothesis test**, instead, is used to **challenge a specific claim** about that parameter. Examples of hypotheses we might want to test include:

- The average weight of cookie packages is exactly 1000g.
- Two different ovens produce cookies with the same average weight.
- The proportion of underweight packages is below a regulatory threshold.

If the hypothesis is rejected, we conclude that the claim is likely false. Otherwise, we do not have enough evidence to reject it, and we act as if it were true.

Hypothesis Testing for Cookie Weights

Let's return to our bakery. The packaging machine is set to produce cookie packages weighing **exactly 1000g**. We take a sample of $n = 1000$ packages and compute:

$$\overline{x} = 1000.0005\text{g}, \quad s_{n-1} = 0.01\text{g}$$

We assume the small deviation is due to measurement noise or natural variation, and we are inclined to believe that the **true mean is still** $\mu = 1000$**g**.

However, our quality control manager raises a concern: what if the population mean is **not** 1000g? She proposes a formal test to challenge the assumption. We define:

- **Null hypothesis** (H_0): the population mean is $\mu_0 = 1000$ g
- **Alternative hypothesis** (H_a): the population mean is different from 1000g

The null hypothesis is the hypothesis we are trying to reject (what we are trying to prove). If we do so, then we embrace the alternative hypothesis.

Before proceeding, we ask what margin of error is acceptable. The manager says she can tolerate a **5% chance of wrongly rejecting** H_0. This defines our **significance level**:

$$\alpha = 0.05$$

We now ask: **how much does our sample mean** $\overline{x} = 1000.0005g$ **deviate from the assumed mean** $\mu_0 = 1000g$? More precisely, **what is the probability of observing a difference this large (or larger) just by chance?**

To answer this, we compute the **test statistic** using the t-distribution:

$$t = \frac{\overline{x} - \mu_0}{s_{n-1}/\sqrt{n}} = \frac{1000.0005 - 1000}{0.01/\sqrt{1000}} = 1.58$$

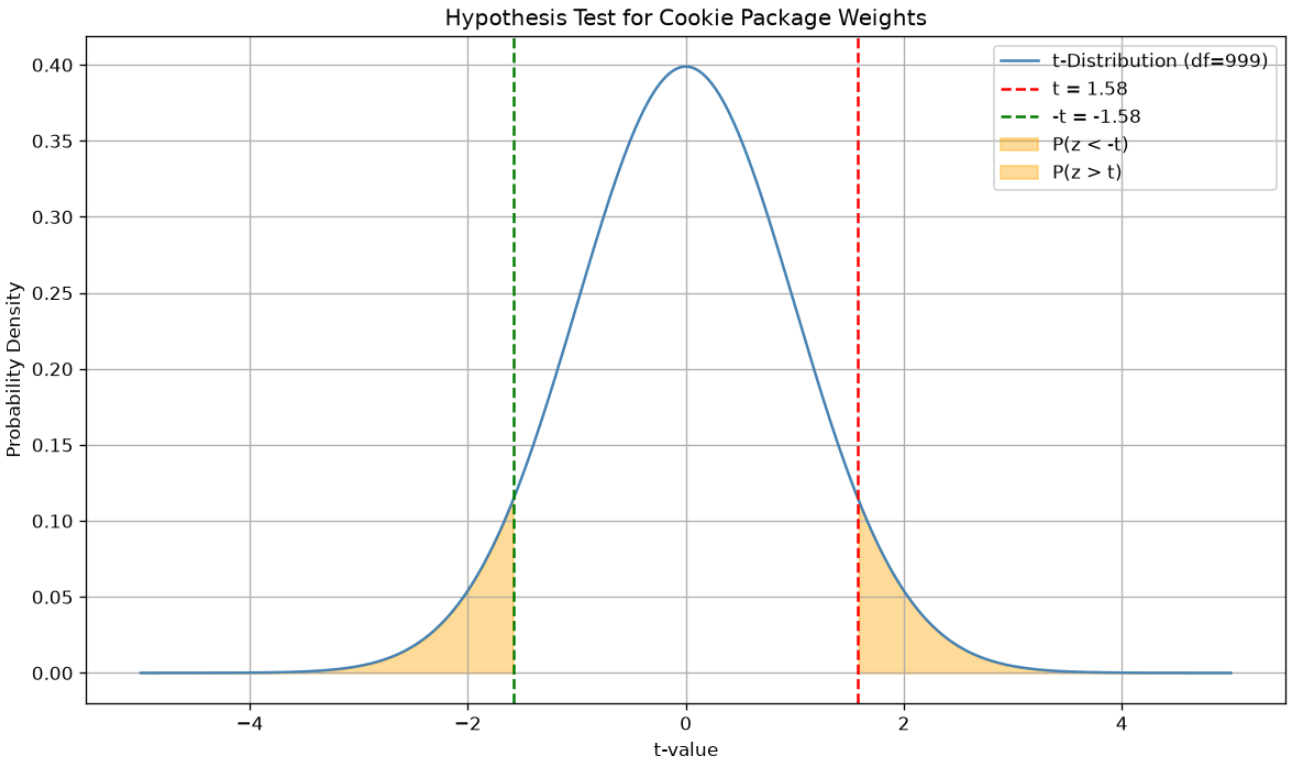
This tells us how many standard errors our estimate is away from the hypothesized mean.

We now ask: **what is the probability of observing a value this extreme or more extreme, assuming H_0 is true?** This is the **p-value**, defined as:

$$P(|z| > |t|)$$

This is the area under the tails of the t-distribution beyond t and $-t$:

```
Test statistic (t): 1.5811
```



Since the t-Student distribution is symmetrical, we can easily compute the p-value as:

$$\text{p-value} = 2(1 - CDF_t(t))$$

In our case, we obtain:

```
t_stat = (x_bar - mu_0) / (s / np.sqrt(n))
print(f"{2*(1-t.cdf(t_stat, df, loc=0, scale=1)):0.2f}")
```

```
0.11
```

This is a large number! How to interpret it?

If the true mean is $\mu = 1000$ and we repeat sampling many times ($n = 1000$), then 11% of

the times we obtain a deviation more extreme than the observed one.

We now compare this number to the significance level of 5%. If we reject the null hypothesis, we risk to make a mistake 11% of times (i.e., if we reject with this deviation, we should also reject for the other 11% of the times that the deviation is larger and we know that H_0 is true), which is above the threshold of 5%. Hence, **we cannot reject the null hypothesis** under these circumstances.

Does this mean that the two means are the same? **We don't know, the test does not tell us what to do in this case!** But we may try to collect more measurements hoping to reduce uncertainty.

We suspect that our sample is too small, so we collect a total of $n = 5000$ examples and obtains:

$$\overline{x} = 1000.0001$$

$$s_{n-1} = 0.0031$$

The mean has decreased a little and the standard deviation has decreased. We recompute the statistic and obtain:

```
2.28
```

We have a larger (more extreme) statistic. We hence compute our p-value:

```
0.02
```

This p-value is now below the threshold of 5%. We can now reject the null hypothesis and conclude that the population mean is different than $1000g$.

Types of Errors in Statistical Tests

Our statistical test provides evidence to either **reject** the null hypothesis in favor of the alternative or **fail to reject** it. However, since our decision is based on a sample, not the entire population, we can never be 100% certain. This uncertainty leads to the possibility of making two types of errors.

Type I Error (False Positive)

A **Type I Error** occurs when we **incorrectly reject a true null hypothesis**. In other words, we conclude there is an effect or a difference when, in reality, there isn't one.

The probability of committing a Type I error the **significance level alpha (α)**. When we set a significance level of $\alpha = 0.05$, we are accepting a 5% risk of making a Type I error.

Type II Error (False Negative)

A **Type II Error** occurs when we **fail to reject a false null hypothesis**. This means we miss a real effect or difference that actually exists.

Summary of Decisions

We can summarize the relationship between our decision and reality in a table:

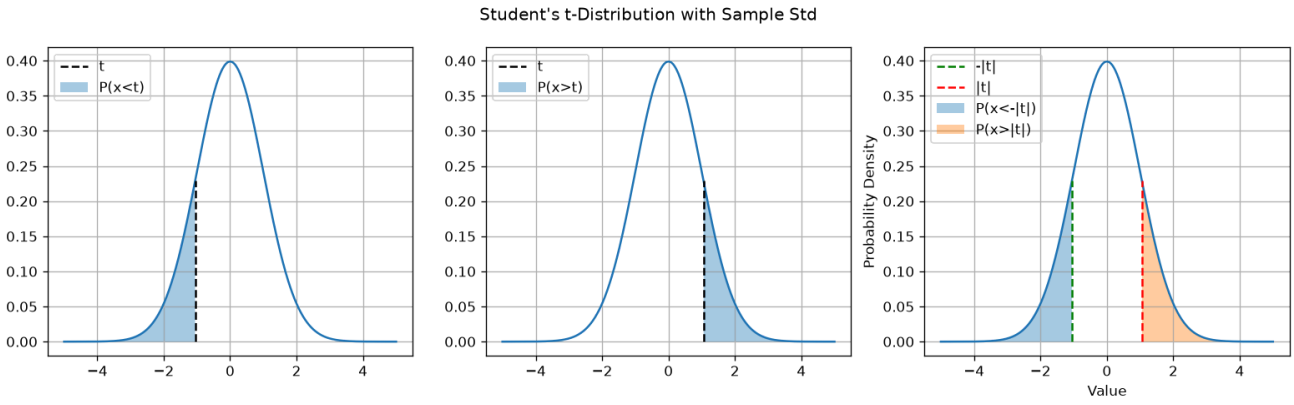
	Null Hypothesis (H_0) is True	**Null Hypothesis (H_0) is False**
Fail to Reject H_0	✅ Correct Decision (True Negative)	❌ **Type II Error** (False Negative)
Reject H_0	❌ **Type I Error** (False Positive)	✅ Correct Decision (True Positive)

One-tailed vs Two-tailed Tests

The tests seen above is a "two-tailed test" in which we summed the areas in the two tails of the distribution. Depending on the form of the alternative hypothesis, in particular:

- If the alternative hypothesis has the form $\mu \neq \mu_0$, then we want to check when the deviation from the assumed mean is larger than the observed one: $P(|x| > |z|)$;
- If the alternative hypothesis has the form $\mu > \mu_0$, then we want to check when the deviation from the assumed mean is positive and larger than the observed one: $P(x > z)$;
- If the alternative hypothesis has the form $\mu < \mu_0$, then we want to check when the deviation from the assumed mean is negative and smaller than the observed one: $P(x < z)$.

This will affect the computation of the p-value as shown in the following figure:



Hypotheses Tests in General

A hypothesis test generally includes:

- H_0: the **null hypothesis**, e.g., the means of two populations are equal;
- H_a: the **alternative hypothesis**, e.g., the means of two populations are not equal (this determines if the test is one- or two-tailed);

- a **test statistics** which quantifies how likely it is to reject the null hypothesis. The test statistics follows a specific distribution which depends on the type of statistical tests we are performing. E.g., it can follow a t-Student distribution;
- a **significance level** α which defines the sensitivity of the test. A common value is $\alpha = 0.05$, which means that we can wrongly reject the null hypothesis 5% of the times when it is in fact true. It represents the degree of error that we are willing to accept when performing hypothesis testing. Common values are $0.1, 0.05, 0.01$;
- the **p-value**: this quantifies the probability of sampling a test statistics at least as extreme as the one observed under the null hypothesis. In practice, the p-value measures the probability that the null hypothesis is true but we are observing test statistic leading to rejection nevertheless.

The null hypothesis is rejected if the **p-value is smaller than the chosen significance level** α. We will not see in details all the possible hypothesis tests, but all of them follow a similar scheme.

Other Important Tests

In this section, we briefly see the main statistical tests which can be used in practice, besides the one for means. We will not see how they are formulated, but we will see how to interpret them. We will see a few other tests when we'll talk about linear regression.

One Sample T-Test

This is the test for sample means we have previously seen. It is used to **determine whether the mean of a single sample is significantly different from a known or hypothesized value.**

This test allows to assess whether a sample has a given mean. Let us check if the average height in our dataset is equal to $170cm$:

```python
import numpy as np
from scipy import stats

# Define the null hypothesis mean (population mean you want to test against)
null_hypothesis_mean = 170

# Perform a one-sample t-test
t_stat, p_value = stats.ttest_1samp(data['height'], null_hypothesis_mean)

# Set the desired significance level (alpha)
alpha = 0.05

# Check the p-value against the significance level
if p_value < alpha:
    print(f"Reject the null hypothesis. The data provides enough evidence to conclude
        that the population mean is different from {null_hypothesis_mean}.")
else:
    print(f"Fail to reject the null hypothesis. The data does not provide enough
        evidence to conclude that the population mean is different from
        {null_hypothesis_mean}.")

print(f"t-statistic: {t_stat:.2f}")
print(f"P-value: {p_value:.4f}")
```

```
Fail to reject the null hypothesis. The data does not provide enough evidence to
conclude that the population mean is different from 170.
t-statistic: -0.69
P-value: 0.4872
```

Let us now check if the average height is 170 among males only:

```python
import numpy as np
from scipy import stats

# Define the null hypothesis mean (population mean you want to test against)
null_hypothesis_mean = 170

data2 = data[data['sex']=='M']

# Perform a one-sample t-test
t_stat, p_value = stats.ttest_1samp(data2['height'], null_hypothesis_mean)

# Set the desired significance level (alpha)
alpha = 0.05

# Check the p-value against the significance level
if p_value < alpha:
    print(f"Reject the null hypothesis. The data provides enough evidence to conclude
        that the population mean is different from {null_hypothesis_mean}.")
else:
    print(f"Fail to reject the null hypothesis. The data does not provide enough
        evidence to conclude that the population mean is different from
        {null_hypothesis_mean}.")

print(f"t-statistic: {t_stat:.2f}")
print(f"P-value: {p_value:.4f}")
```

```
Reject the null hypothesis. The data provides enough evidence to conclude that the
population mean is different from 170.
t-statistic: 45.52
P-value: 0.0000
```

Two Sample T-Test

A two-sample t-test is used to determine if there is a significant difference between the means of two independent samples. It's often used when you want to compare the means of two different populations or treatments. The test assesses whether the difference between the sample means is statistically significant or if it could have occurred due to random chance. Also in this case, the test statistic will follow a t-Student distribution.

Let us now compare the average heights in two samples we previously obtained via random sampling. We expect no differences:

```python
import scipy.stats as stats

h1 = sample1['height']
h2 = sample2['height']

# Perform independent two-sample t-test
t_stat, p_value = stats.ttest_ind(h1, h2)

# Define significance level (alpha)
alpha = 0.05

print(f"Test statistic: {t_stat:0.2f}")
print(f"Significance level: {alpha:0.2f}")
print(f"P-value: {p_value:0.2f}")

# Comment on result
if p_value < alpha:
    print("Conclusion: there is a significant difference between the mean exam scores
        of the two classes.")
else:
    print("Conclusion: there is no significant difference between the mean exam
        scores of the two classes.")
```

```
Test statistic: -0.47
Significance level: 0.05
P-value: 0.64
Conclusion: there is no significant difference between the mean exam scores of the
two classes.
```

```python
import pandas as pd
hw=pd.read_csv(DATA + "height_weight.csv")
hw['height'] = (hw['height']/2.54).astype(int)
hw['weight'] = hw['weight']/2.205
hw
```

	sex	BMI	height	weight
0	M	33.36	74	53.484771
1	M	26.54	70	38.056472
2	F	32.13	61	34.970812
3	M	26.62	68	35.999365
4	F	27.13	66	34.559390
...	...	...	...	...
4226	F	17.12	69	23.862436
4227	M	27.47	69	38.262182
4228	F	29.16	64	34.970812
4229	F	23.68	64	28.388071
4230	F	20.12	61	22.628172

4231 rows × 4 columns

If we compute the average `BMI` for males and females we obtain:

```
hw.groupby('sex')['BMI'].mean()
```

```
sex
F    26.929287
M    27.684959
Name: BMI, dtype: float64
```

We see a small difference. Is this due to chance or is it significant? If we run a two-sample t-test, we obtain the following results:

```
import scipy.stats as stats

male_bmi = hw[hw['sex']=='M']['BMI']
female_bmi = hw[hw['sex']=='F']['BMI']

# Perform independent two-sample t-test
t_stat, p_value = stats.ttest_ind(male_bmi, female_bmi)

# Define significance level (alpha)
alpha = 0.05

print(f"Test statistic: {t_stat:0.2f}")
print(f"Significance level: {alpha:0.2f}")
print(f"P-value: {p_value:0.2f}")

# Comment on result
if p_value < alpha:
    print("Conclusion: there is a significant difference between the two means.")
else:
    print("Conclusion: there is no significant difference between the two means.")
```

```
Test statistic: 4.64
Significance level: 0.05
P-value: 0.00
Conclusion: there is a significant difference between the two means.
```

Chi-Square Test for Independence

The Chi-Square Test for Independence is a statistical test used to determine whether there is an association or independence between two or more categorical variables. This test is particularly useful when we want to assess whether changes in one categorical variable are related to changes in another categorical variable. The typical scenario is to set up a contingency table to compare the observed frequencies (counts) of the joint categories of the two variables to the expected frequencies that would occur under the assumption of independence.

The null hypothesis for the Chi-Square Test for Independence is that **there is no association between the two categorical variables** (they are independent), while the alternative hypothesis suggests that there is an association (they are dependent).

The test statistics follows a Chi-square distribution (we won't see the details) in this case.

Let's consider the Titanic dataset:

```
import pandas as pd
titanic = pd.read_csv(DATA + "titanic.csv",
                      index_col='PassengerId')
titanic
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...
887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 11 columns

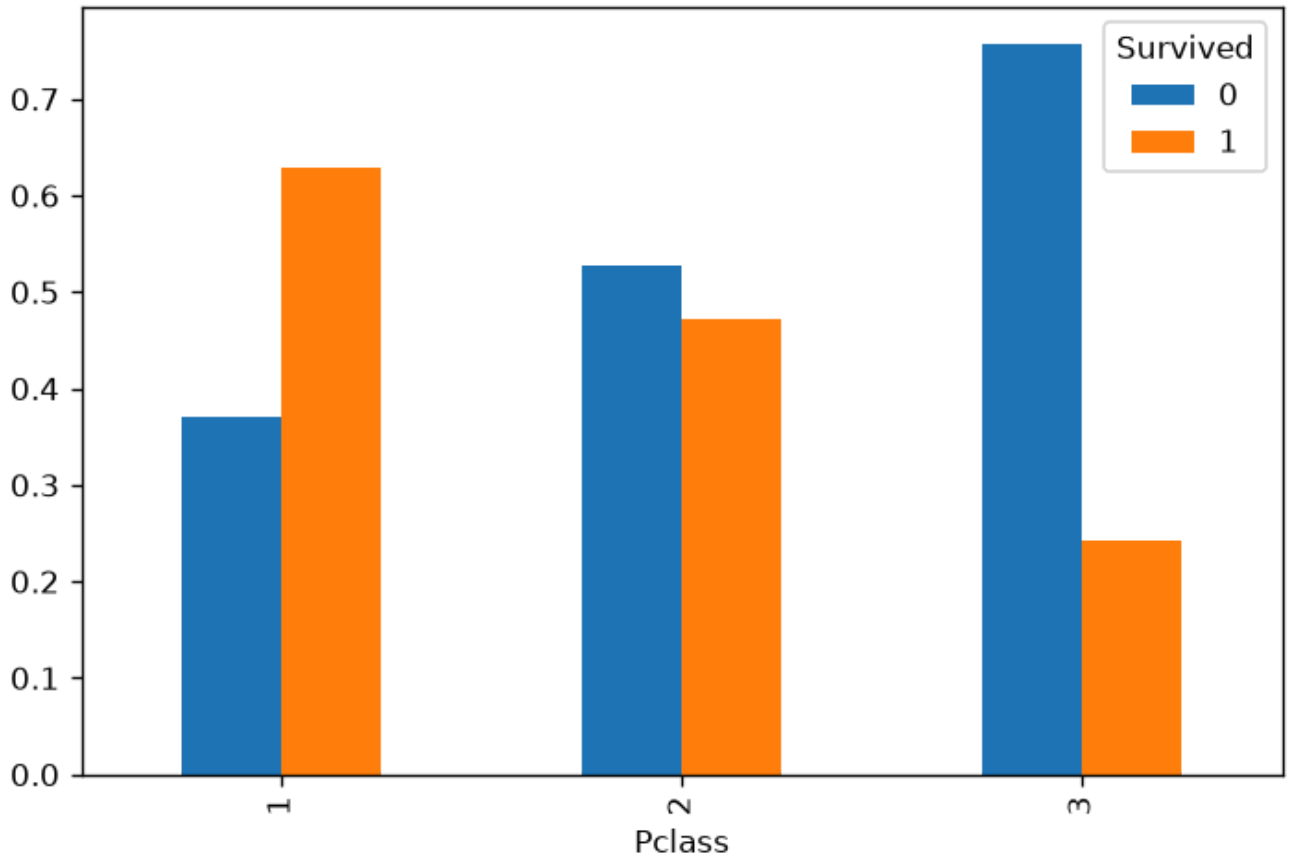
Let's consider the following contingency table:

```
contingency_table = pd.crosstab(titanic['Pclass'], titanic['Survived'])
contingency_table
```

Survived	0	1
Pclass		
1	80	136
2	97	87
3	372	119

We can visualize the distributions of Survived in the three classes for more clarity:

```
from matplotlib import pyplot as plt
pd.crosstab(titanic['Pclass'], titanic['Survived'], normalize=0).plot.bar()
plt.show()
```



We expect some form of correlation between the two variables. Indeed, the chi-square statistics and Cramer V statistics are:

```
from scipy.stats import chi2_contingency
from scipy.stats.contingency import association

print(f"Chi-square statistic: {chi2_contingency(contingency_table).statistic:0.2f}")
print(f"Cramer V statistic: {association(contingency_table):0.2f}")
```

```
Chi-square statistic: 102.89
Cramer V statistic: 0.34
```

We have numbers different from zero, but **is this due to chance or is it statistically significant**? If we run a chi-square contingency test:

```
import seaborn as sns
import pandas as pd
from scipy.stats import chi2_contingency

# Perform the Chi-Square Test for Independence
chi2, p, _, _ = chi2_contingency(contingency_table)

# Set the significance level (alpha)
alpha = 0.05

# Print the results
print("Chi-Square Statistic:", chi2)
print("p-value:", p)

# Interpret the results
if p < alpha:
    print("\nThere is a significant association between 'Pclass' and 'survived'.")
else:
    print("\nThere is no significant association between 'Pclass' and 'survived'.")
```

```
Chi-Square Statistic: 102.88898875696056
p-value: 4.549251711298793e-23

There is a significant association between 'Pclass' and 'survived'.
```

Chi-Square Goodness-of-Fit Test

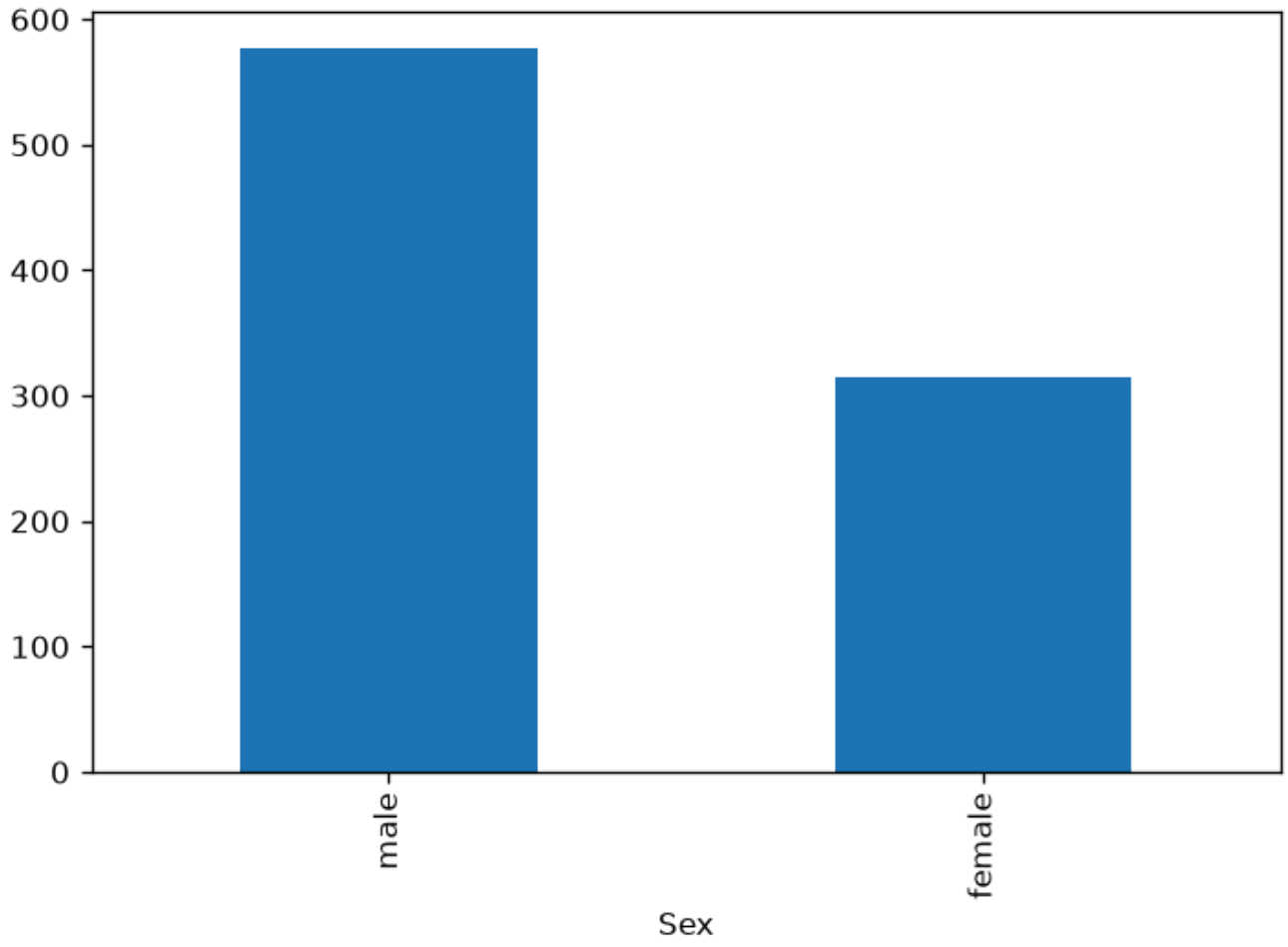
The Chi-Square Goodness of Fit test is a statistical test used to determine whether observed categorical data (frequencies) fit a specified distribution or expected frequencies. This test is often used to assess whether the observed data deviates significantly from a hypothesized distribution. The typical scenario is to compare observed frequencies with expected frequencies based on a theoretical model or prior knowledge.

The null hypothesis for the Chi-Square Goodness of Fit test is that **there is no significant difference between the observed and expected frequencies**, meaning the observed data fits the specified distribution. The alternative hypothesis suggests that there is a significant difference.

The test statistics follows a Chi-square distribution in this case.

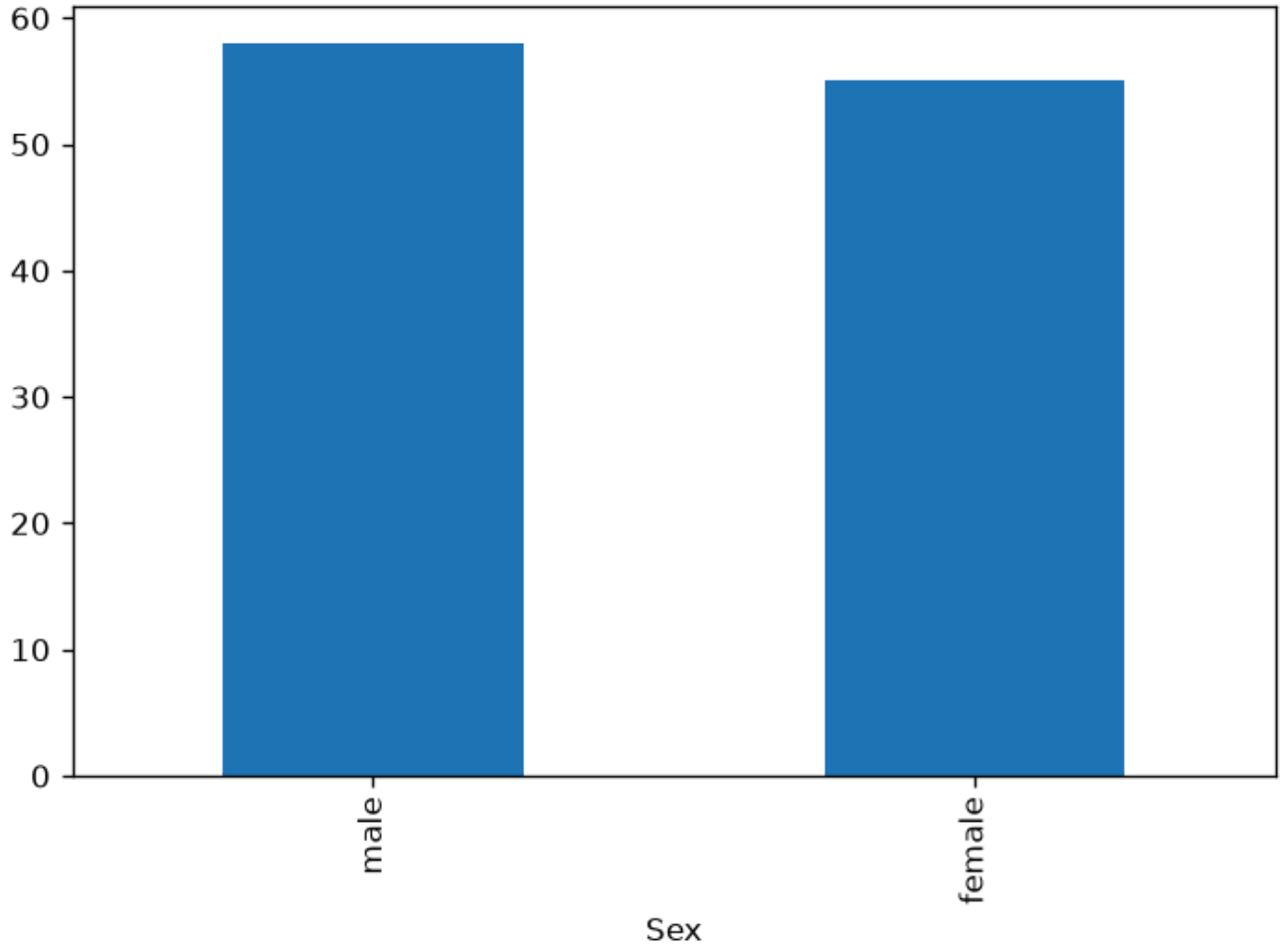
Let us consider the Titanic dataset again. We know that the distribution of `Sex` among passengers is biased:

```
titanic['Sex'].value_counts().plot.bar()
plt.show()
```



We now consider the distribution of `Sex` among passengers less than 18 years old:

```
minor=titanic[titanic['Age']<18]
observed_frequencies = minor['Sex'].value_counts()
observed_frequencies.plot.bar()
plt.show()
observed_frequencies
```




```
Sex
male      58
female    55
Name: count, dtype: int64
```

This looks less biased, but there are still minor differences between the counts. Are these due to chance? If `Sex` was distributed uniformly (as we hypothesize), we would have the following frequencies:

```
expected_frequencies = [113/2, 113/2]
expected_frequencies
```

```
[56.5, 56.5]
```

We can run a Goodness-of-fit test to check if the observed frequencies match the expected ones:

```python
import seaborn as sns
import pandas as pd
from scipy.stats import chisquare

# Load the Titanic dataset
titanic_data = sns.load_dataset("titanic")

observed_frequencies = minor['Sex'].value_counts().values

# Define expected frequencies that closely match the observed data
expected_frequencies = [113/2, 113/2]

chi2, p_value = chisquare(f_obs=observed_frequencies, f_exp=expected_frequencies)

# Set the significance level (alpha)
alpha = 0.05

# Print the results
print("Observed Frequencies:")
print(observed_frequencies)
print("\nExpected Frequencies:")
print(expected_frequencies)
print("\nChi-Square Statistic:", chi2)
print("p-value:", p_value)

# Interpret the results
if p_value < alpha:
    print("\nThe observed data significantly deviates from the expected
        distribution.")
else:
    print("\nThe observed data fits the expected distribution.")
```

```
Observed Frequencies:
[58 55]

Expected Frequencies:
[56.5, 56.5]

Chi-Square Statistic: 0.07964601769911504
p-value: 0.7777776907897473

The observed data fits the expected distribution.
```

Given the large p-value, we could not reject the null hypothesis that there are significant differences between expected and observed frequencies.

Pearson/Spearman Correlation Test

We have seen how to compute Pearson/Spearman correlation coefficient. However, what can we say when we get small values? Are those supposed to be zero, but we got something different from zero due to sampling, or are they significantly different from zero?

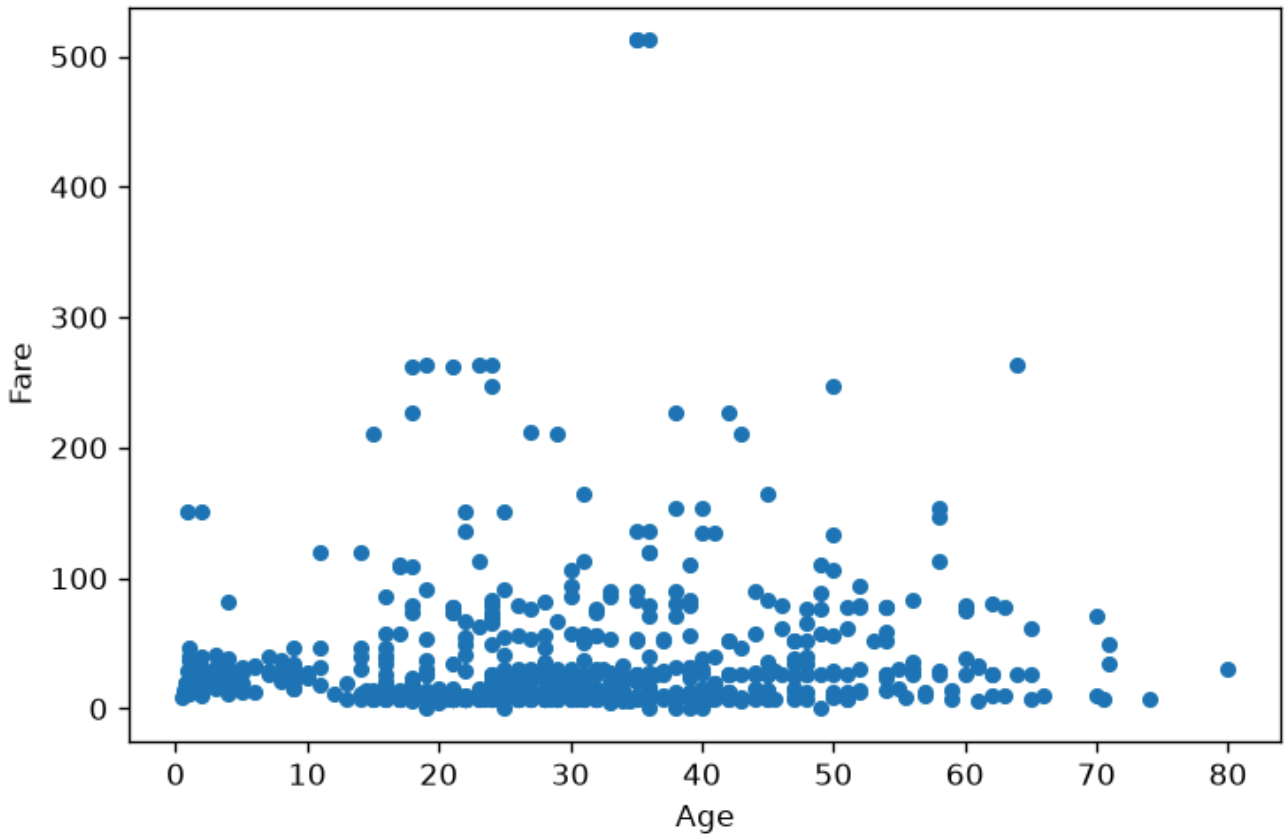
The statistical tests associated with the correlation coefficients are used to determine whether the observed correlation between two variables is statistically significant or if it might have occurred due to random chance. This test assesses whether the correlation in the sample data is likely to reflect a true correlation in the population.

The null hypothesis is that **there is no statistically significant correlation between the two variables in the population**. In other words, the true correlation coefficient in the population is zero.

Let us consider the Titanic dataset. We find the following correlation between the `Age` and `Fare` variables:

```
titanic.plot.scatter(x='Age',y='Fare')
tt=titanic.dropna()
print(f"Correlation between age and fare:
        {titanic[['Age','Fare']].dropna().corr().values[0][1]:.2f}")
```

```
Correlation between age and fare: 0.10
```



Is this small positive correlation “true” or due to chance? Let us run a statistical test:

```
import seaborn as sns
import pandas as pd
import scipy.stats as stats

tt = titanic[['Age','Fare']].dropna()

# Extract the 'age' and 'fare' columns
age = tt['Age']
fare = tt['Fare']

# Perform a Pearson correlation test
r, p_value = stats.pearsonr(age, fare)

# Define the significance level
alpha = 0.05

# Display the results
print(f"Pearson correlation coefficient (r): {r}")
print(f"P-value: {p_value}")

# Compare the p-value to alpha
if p_value ⩽ alpha:
    print("Reject the null hypothesis. There is a significant correlation between age
        and fare.")
else:
    print("Fail to reject the null hypothesis. There is no significant correlation
        between age and fare.")
```

```
Pearson correlation coefficient (r): 0.0960666917690389
P-value: 0.010216277504447095
Reject the null hypothesis. There is a significant correlation between age and
fare.
```

The p-value is small enough to reject the null hypothesis: the correlation is small but statistically significant.

Similar tests exist for Spearman coefficient:

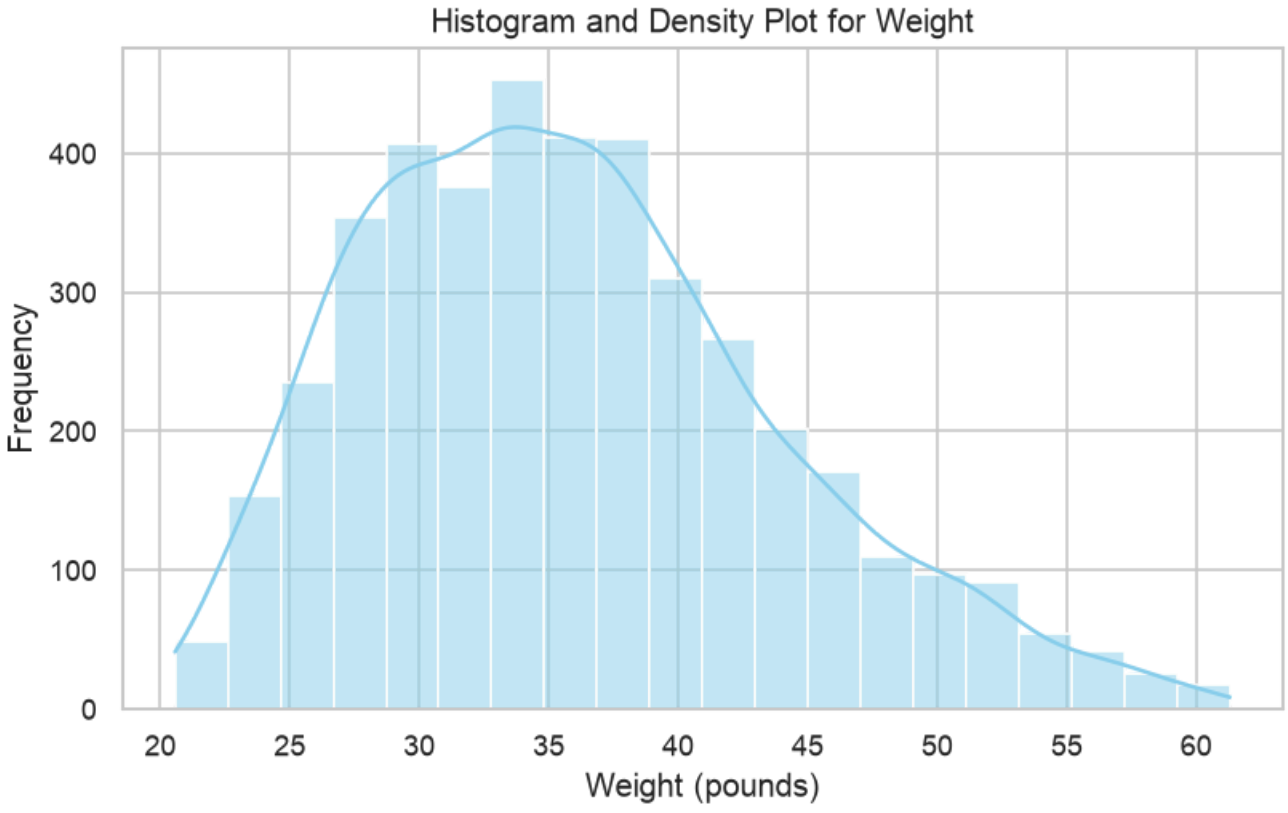
```
import seaborn as sns
import pandas as pd
import scipy.stats as stats

tt = titanic[['Age','Fare']].dropna()

# Extract the 'age' and 'fare' columns
age = tt['Age']
fare = tt['Fare']

# Perform a Pearson correlation test
r, p_value = stats.spearmanr(age, fare)

# Define the significance level
alpha = 0.05

# Display the results
print(f"Spearman correlation coefficient (r): {r}")
print(f"P-value: {p_value}")

# Compare the p-value to alpha
if p_value ⩽ alpha:
    print("Reject the null hypothesis. There is a significant correlation between age
        and fare.")
else:
    print("Fail to reject the null hypothesis. There is no significant correlation
        between age and fare.")
```

```
Spearman correlation coefficient (r): 0.1350512177342878
P-value: 0.00029580903243060916
Reject the null hypothesis. There is a significant correlation between age and
fare.
```

Assessing whether a Sample is Normally Distributed

While the Normal distribution is pervasive, in some cases, it is useful to assess whether a given sample follows a normal distribution before assuming this is true. Let us consider the dataset of heights and weights and plot the distribution of weights:

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset
hw = pd.read_csv(DATA + "height_weight.csv")
hw['height'] = (hw['height'] / 2.54).astype(int)  # Convert height to inches
hw['weight'] = hw['weight'] / 2.205               # Convert weight to pounds

# Set up the plotting style
# `sns.set` rewrites the rcParams, including the transparent background the site
# uses (see site/notes/matplotlibrc), so it is asked to leave that alone. The
# whitegrid grid lines are grey, not white, so they survive the change.
sns.set(style="whitegrid",
        rc={"figure.facecolor": "none", "axes.facecolor": "none",
            "savefig.facecolor": "none"})

# Plotting for weight
sns.histplot(hw['weight'], kde=True, color="skyblue", bins=20)
plt.title('Histogram and Density Plot for Weight')
plt.xlabel('Weight (pounds)')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```



Does the distribution look Gaussian? Let us compute Skewness and Kurtosis:

```python
import pandas as pd
from scipy.stats import skew, kurtosis

# Load and preprocess the data
hw = pd.read_csv(DATA + "height_weight_pounds.csv")

# Calculate skewness and kurtosis for weight
weight_skewness = skew(hw['weight'])
weight_kurtosis = kurtosis(hw['weight'])

print(f"Skewness of weight: {weight_skewness:.2f}")
print(f"Kurtosis of weight: {weight_kurtosis:.2f}")
```

```
Skewness of weight: 0.57
Kurtosis of weight: -0.06
```

We not that:

- We have a positive Skewness: this indicates that the distribution is skewed towards the right side (the right tail is longer) as compared to a Normal distribution;
- We have a Kurtosis slightly lower than zero: the distribution is slightly “flatter” than a Normal distribution.

While Skewness and Kurtosis can help characterize deviations from normality, there are tests which can be used.

Quantile-Quantile Plots (Q-Q Plots)

Quantile-Quantile plots, or Q-Q plots, are a powerful tool to **assess how well a sample matches a theoretical distribution**. The idea is simple: we compare the **quantiles of the empirical data** with the **quantiles of a reference distribution**—typically the normal distribution.

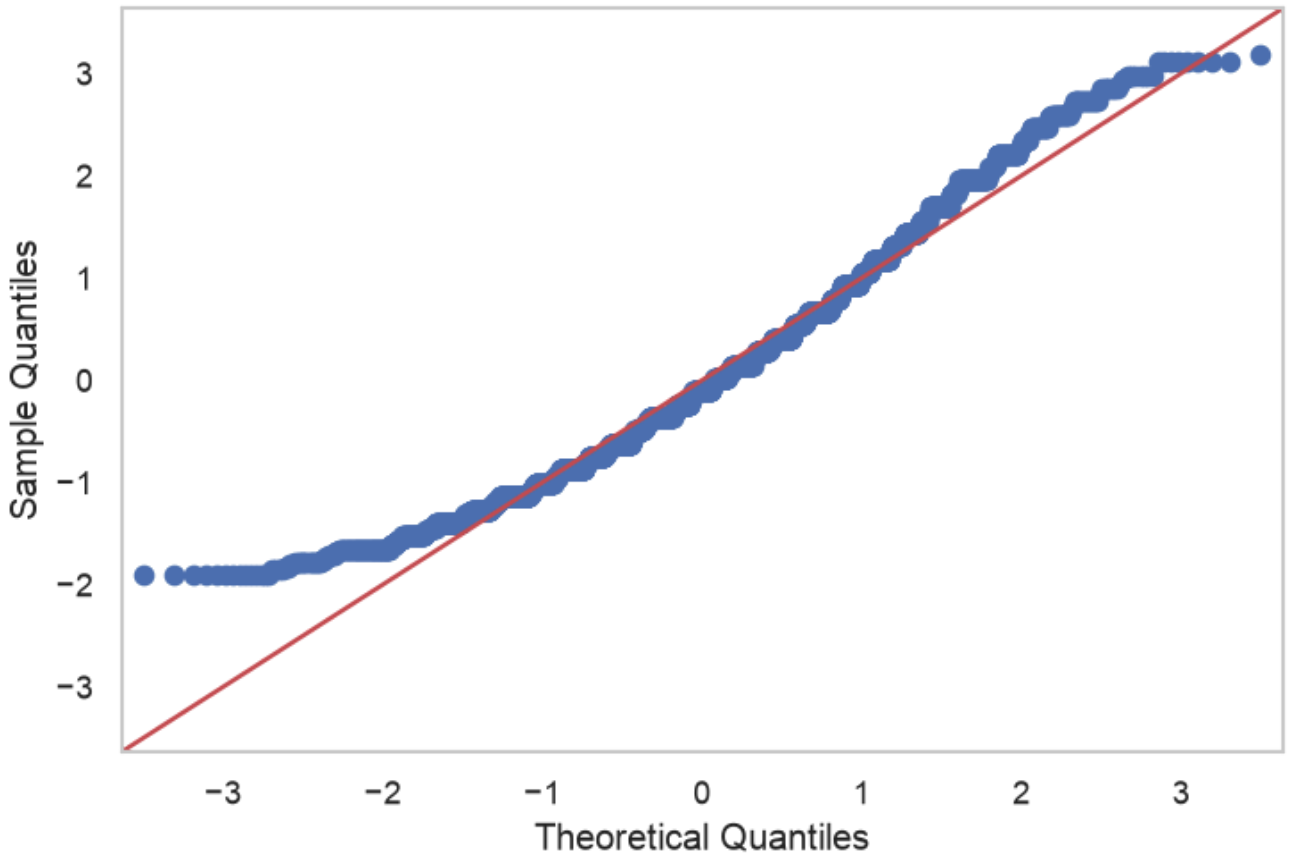
To make the comparison meaningful and remove scale effects, the empirical data is often **standardized into z-scores**. If the sample truly follows the theoretical distribution, the quantiles will align, and the points will fall along the diagonal line $y = x$.

Deviations from this line reveal **systematic differences** between the empirical and theoretical distributions. For example:

- A curve that bends away from the diagonal suggests skewness.
- S-shaped patterns may indicate heavier or lighter tails than expected.

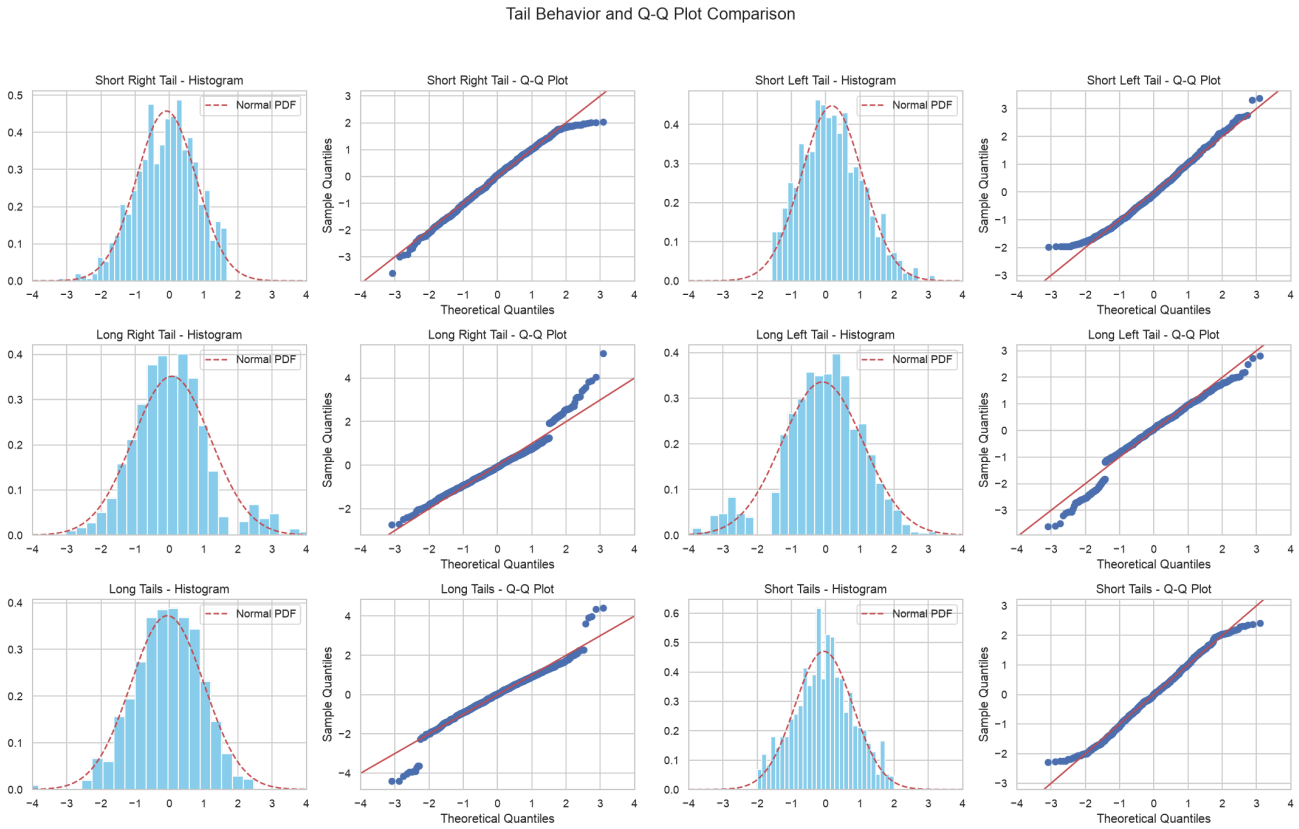
In the following graph, we show the Q-Q plot for the sample of cookie package weights. It helps us visually evaluate whether the sample distribution is approximately normal—a key assumption in many statistical methods.

```
from statsmodels.graphics.gofplots import qqplot
import pandas as pd
from matplotlib import pyplot as plt

# fit=True means the distribution should be fitted to the data
# and the data should be standardized
# line='45' means a 45-degree reference line will be plotted
qqplot(hw['weight'], fit=True, line='45')
plt.grid()
plt.show()
```



The plot relates the “theoretical” quantiles with those of the sample. The fact that the points on the plot do not lie on the diagonal indicates that there is a discrepancy between the empirical data distribution and the Gaussian distribution.

Analyzing a Q-Q Plot can be complex. In practice, **there are some guidelines to understand how a sample deviates from a theoretical distribution**. The following figure compares the q-q plots of different distributions:



Every time we observe a Q-Q Plot, we can relate these features to characteristics of the distribution. Clearly, these characteristics can also combine to create more complex Q-Q Plots, as seen in the case of weights.

Shapiro-Wilk Normality Test

The Shapiro-Wilk test is a statistical test used to assess whether a sample follows a Gaussian (normal) distribution. It is used with small samples ($n \leq 2000$) It works by comparing the observed data to what you would expect if the data were drawn from a truly Gaussian distribution.

The null hypothesis for this test is that the population is normally distributed.

We will not see the formal details of this test, but we can use it in our analyses. Here is the result on the `weight` sample in our height-weight dataset:

```python
from scipy.stats import shapiro
import pandas as pd

# Perform the Shapiro-Wilk test
statistic, p_value = shapiro(hw['weight'])

# Set the significance level (alpha)
alpha = 0.05

print(f"Test statistic: {statistic:0.2f}")
print(f"P-value: {p_value:0.2f}")

# Check the p-value against the significance level
if p_value > alpha:
    print("Sample looks Gaussian (fail to reject H0)")
else:
    print("Sample does not look Gaussian (reject H0)")
```

```
Test statistic: 0.97
P-value: 0.00
Sample does not look Gaussian (reject H0)
```

D'Agostino's K-squared test

When samples are large ($n \geq 50$), the D'Agostino's K-squared test is more used. It is based on Skewness and Kurtosis.

The null hypothesis for this test is that the population is normally distributed.

Here is the result for our example:

```python
import pandas as pd
from scipy.stats import normaltest

# Perform D'Agostino's K-squared test
statistic, p_value = normaltest(hw['weight'])

# Set the significance level (alpha)
alpha = 0.05

print(f"Test statistic: {statistic:.2f}")
print(f"P-value: {p_value:.2f}")

# Check the p-value against the significance level
if p_value > alpha:
    print("Sample looks Gaussian (fail to reject H0)")
else:
    print("Sample does not look Gaussian (reject H0)")
```

```
Test statistic: 201.64
P-value: 0.00
Sample does not look Gaussian (reject H0)
```

Exercises

Exercise 1

Consider the sample of all passengers in the titanic dataset. Compute the mean age of all passengers. Then, compute the confidence bounds for this mean setting the confidence level to 0.95. Is the estimated mean a reliable estimate of the population mean?

Exercise 2

Consider the samples of all passengers in the titanic dataset. Compute the variance of the ages of all passengers. Then, compute the confidence bounds for this variance setting the confidence level to 0.95. Is the estimated variance a reliable estimate of the population variance?

Exercise 3

Extract the sample of values of Sex of passengers in second class in the titanic dataset. Compute the contingency table of absolute counts of the Sex column. Compute the confidence bounds for the proportions of Sex in the sample. Are the estimated bounds reliable? Repeat the analysis for class 1 and 3.

Exercise 4

The average age of women in the US population is 40. Run a statistical test to assess whether the sample of ages in the `infert` dataset has the same mean as the US population. What is the result of the test? Are women in `infert` younger or older than the average?

You can load the dataset installing the `pydataset` library with `pip install pydataset` and then:

```
from pydataset import data; infert = data("infert")
```

Exercise 5

Consider the `titanic` dataset. Follow these steps to assess that males and females **are not equally distributed**:

- Obtain a table of absolute frequencies of the values of `Sex` ;
- Show a barplot comparing the proportions of male and female passengers. Do these look equally distributed?
- Run a statistical test to assess whether the proportion of males and females are equal ($p = 0.5$). What is the result of the test?
- Run the statistical test setting $p = 1/3$. Is the result different? Can we reject the null hypothesis? Why?

References

- Chapters 6-8 of [1];
- Parts of chapter 9 of [2].

[1] Gonick, L., & Smith, W. (1993). The cartoon guide to statistics. HarperCollins Publishers, Inc.

[2] Heumann, Christian, and Michael Schomaker Shalabh. Introduction to statistics and data analysis. Springer International Publishing Switzerland, 2016.

© 2026 Antonino Furnari · Department of Mathematics and Computer Science, University of Catania Built with Quarto

Credits and data sources

Linear regression

Chapter resources

Slides (PDF) Open in Colab Practice quiz

Linear regression is a fundamental and widely used statistical technique in data analysis and machine learning. It is a powerful tool for **modeling and understanding the relationships between variables** and **make predictions on unseen data**. At its core, linear regression aims to model a linear relationship between a **dependent variable** (the one you want to predict) and **one or more independent variables** (the ones used for prediction). This technique allows us to make predictions, infer associations, and gain insights into how changes in independent variables influence the target variable. Linear regression is both intuitive and versatile, making it a valuable tool for tasks ranging from simple trend analysis to more complex predictive modeling and hypothesis testing.

In this context, we will explore the concepts and applications of linear regression, its different types, and how to implement it using Python.

The Auto MPG Dataset

We will consider the Auto MPG dataset, which contains 398 measurements of 8 different properties of different cars:

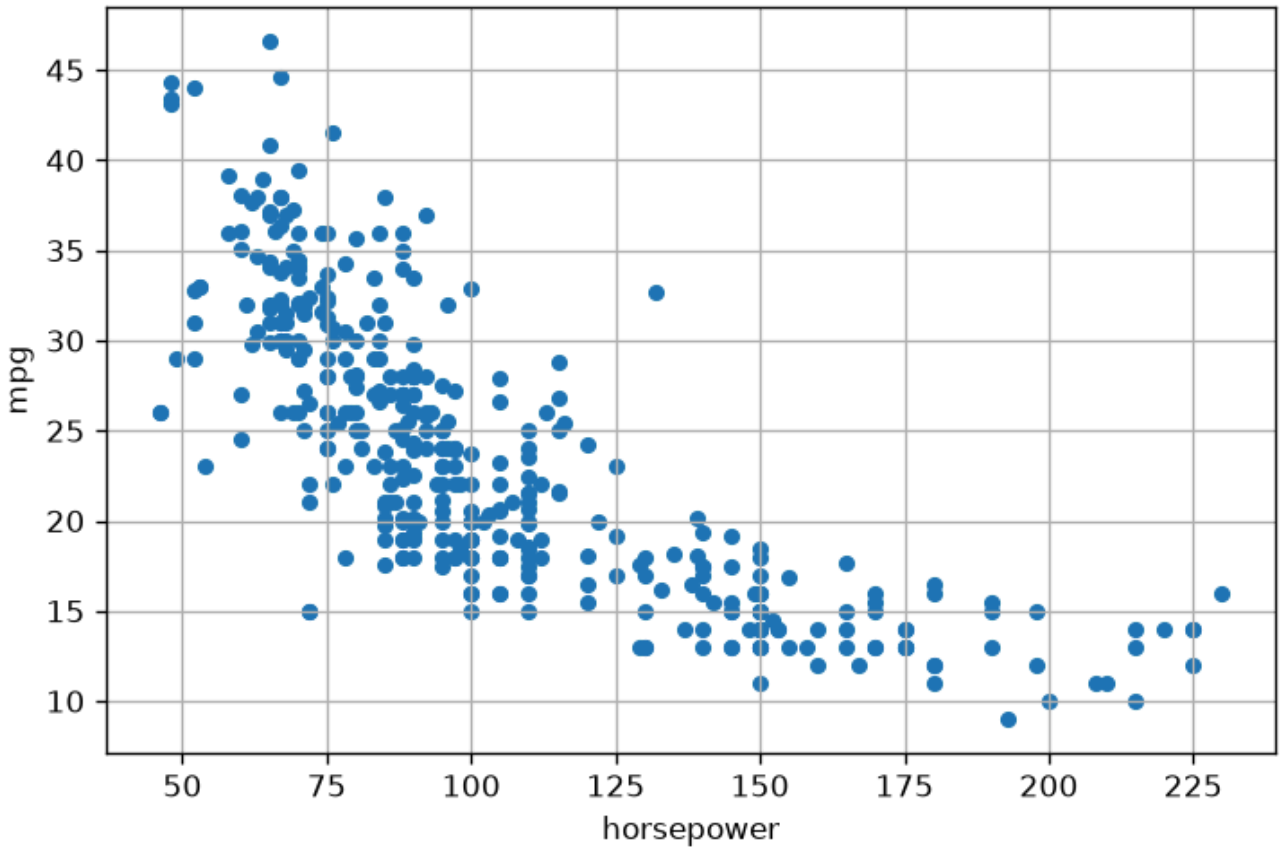
	displacement	cylinders	horsepower	weight	acceleration	model_year	origin	mpg
0	307.0	8	130.0	3504	12.0	70	1	18.0
1	350.0	8	165.0	3693	11.5	70	1	15.0
2	318.0	8	150.0	3436	11.0	70	1	18.0
3	304.0	8	150.0	3433	12.0	70	1	16.0
4	302.0	8	140.0	3449	10.5	70	1	17.0
...	...	...	...	...	...	...	...	...
393	140.0	4	86.0	2790	15.6	82	1	27.0
394	97.0	4	52.0	2130	24.6	82	2	44.0
395	135.0	4	84.0	2295	11.6	82	1	32.0
396	120.0	4	79.0	2625	18.6	82	1	28.0
397	119.0	4	82.0	2720	19.4	82	1	31.0

398 rows × 8 columns

Here is a description of the different variables:

- **Displacement**: The engine's displacement (in cubic inches), which indicates the engine's size and power.
- **Cylinders**: The number of cylinders in the engine of the car. This is a categorical variable.
- **Horsepower**: The engine's horsepower, a measure of the engine's performance.
- **Weight**: The weight of the car in pounds.
- **Acceleration**: The car's acceleration (in seconds) from 0 to 60 miles per hour.
- **Model Year**: The year the car was manufactured. This is often converted into a categorical variable representing the car's age.
- **Origin**: The car's country of origin or manufacturing. Car Name: The name or identifier of the car model.
- **MPG (Miles per Gallon)**: The fuel efficiency of the car in miles per gallon. It is the variable to be predicted in regression analysis.

We will start by exploring the relationship between the variables `horsepower` and `MPG`. Let's visualize the related scatterplot:



Regression Models

Regression models, in general, aim to **study the relationship between two variables**, X and Y, by defining a mathematical model f such that:

$$Y = f(X) + \epsilon$$

Here:

- f is a deterministic function which can be used to **predict the values of Y from the values of X**;
- ϵ is an **error term**, i.e., a variable capturing everything that is not captured by the deterministic function f. It can be due to different reasons, the main of which are:
  - f is not an accurate deterministic function of the process. Since we don't know the "true" function f and we are only estimating it, we may obtain a suboptimal f for which $Y \neq f(X)$. The error term captures the differences between our predictions and the true values.
  - Y cannot only be predicted from X, but some other variable is needed to correctly predict Y from X. For instance, X could be "years of education" and Y can be "income". While may expect that "income" is not completely predicted from "years of education". This can happen also because we don't always have observations for all relevant variables.
  - the problem has inherent stochasticity which cannot be entirely modeled within the deterministic function f. For instance, consider the problem of predicting the rate of wins in poker based on the expertise of the player. The expertise surely allows to predict the rate of wins, but wins partially depend also on random factors, such as how the deck was shuffled.

Note that, often, we model f in a way that we have its **analytical form**. This is very powerful. If we have the analytical form of the function f which **explains** how Y is influenced from X (**can be predicted from** X), then we can really understand deeply the connection between the two variables!

The function f can take different forms. The most common one is the **linear form** that we will see in the next section. While the linear form is very simple (and hence we can anticipate it will be a limited model in many cases), it has the great advantage to be **easy to interpret**.

Simple Linear Regression

Simple linear regression aims to model the **linear relationship** between two variables X and Y. In our example dataset, we will consider $X = \text{horsepower}$ and $Y = \text{mpg}$.

Since we are trying to model a linear relationship, we can imagine **a line passing through the data**. The simple linear regression model is defined as:

$$Y \approx \beta_0 + \beta_1 X$$

In our example:

$$\text{mpg} \approx \beta_0 + \beta_1 \text{horsepower}$$

It is often common to introduce a **"noise" variable** which captures the randomness due to which the expression above is approximated and write:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

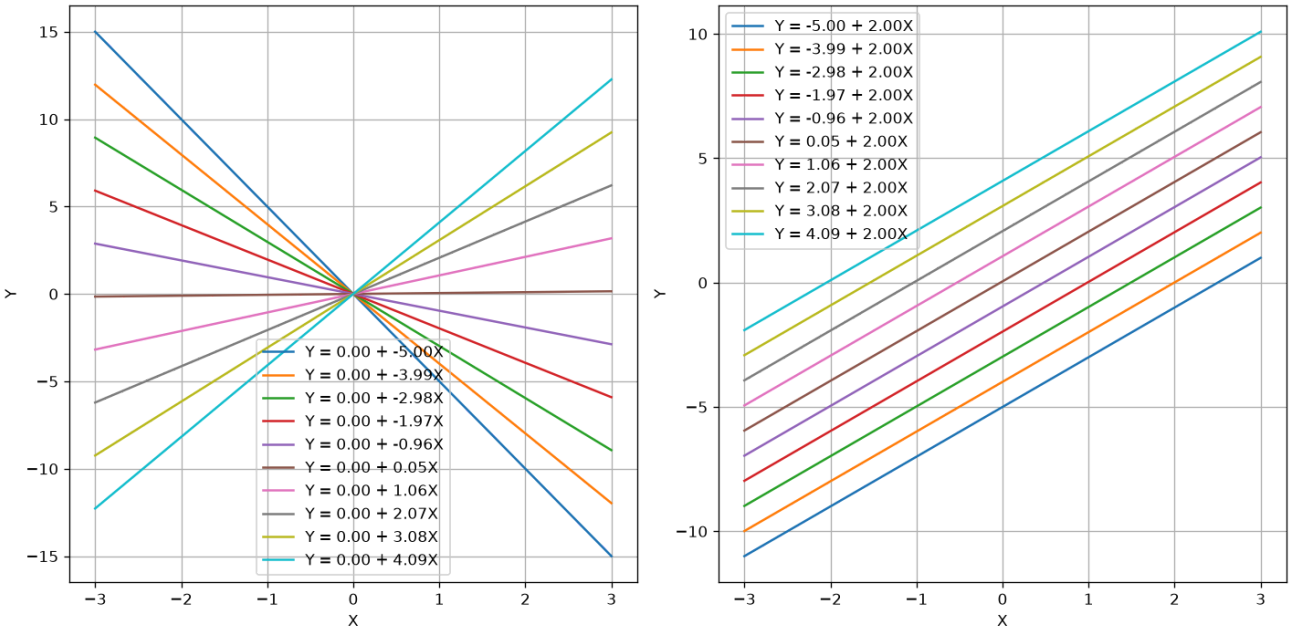
As we will see later, we expect ϵ to be **small and randomly distributed**.

Given the model above, we will call:

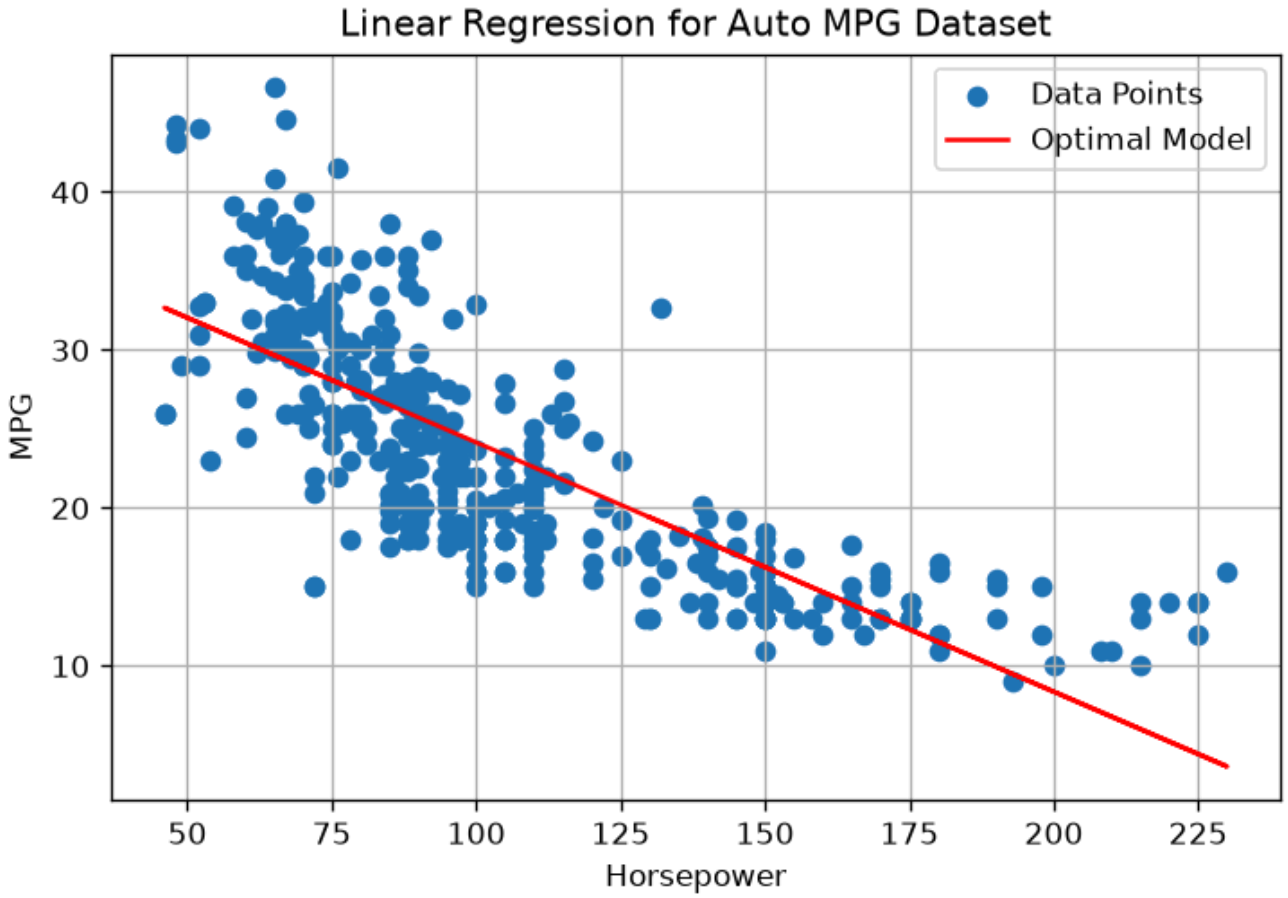
- X, the **independent variable** or **regressor**;
- Y, the **dependent variable** or **regressed variable**.

The values β_0 and β_1 are called **coefficients** or **parameters** of the model.

The mathematical model above has a geometrical interpretation. Indeed, specific values of β_0 and β_1 identify a given line in the 2D plane, as shown in the plot below:



We hence aim to estimate two appropriate values $\hat{\beta}_0$ and $\hat{\beta}_1$ from data in a way that they provide a model which represents well our data. In the case of our example, we expect the geometrical model to have this aspect:



This line will also be called the **regression line**.

Estimating the Coefficients - Ordinary Least Squares (OLS)

To estimate the coefficients of our optimal model, we should first define what is a good model. We will say that **a good model is one that predicts well the Y variable from the X one**. We already know from the example above that, since the relationship is not perfectly linear, the model will make some mistakes.

Let $\{(x_i, y_i)\}$ be our set of observations. Let

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$$

be the prediction of the model for the observation x_i. For each data point (x_i, y_i), we will define the deviation of the prediction from the y_i value as follows:

$$e_i = y_i - \hat{y}_i$$

We use the symbol e for error and also call these numbers **the residuals**, as they are the **differences (or residuals) between our prediction and the ground truth value**.

These numbers will be positive or negative based on whether we underestimate or overestimate the y_i values, while **they will be zero when the prediction is perfect**.

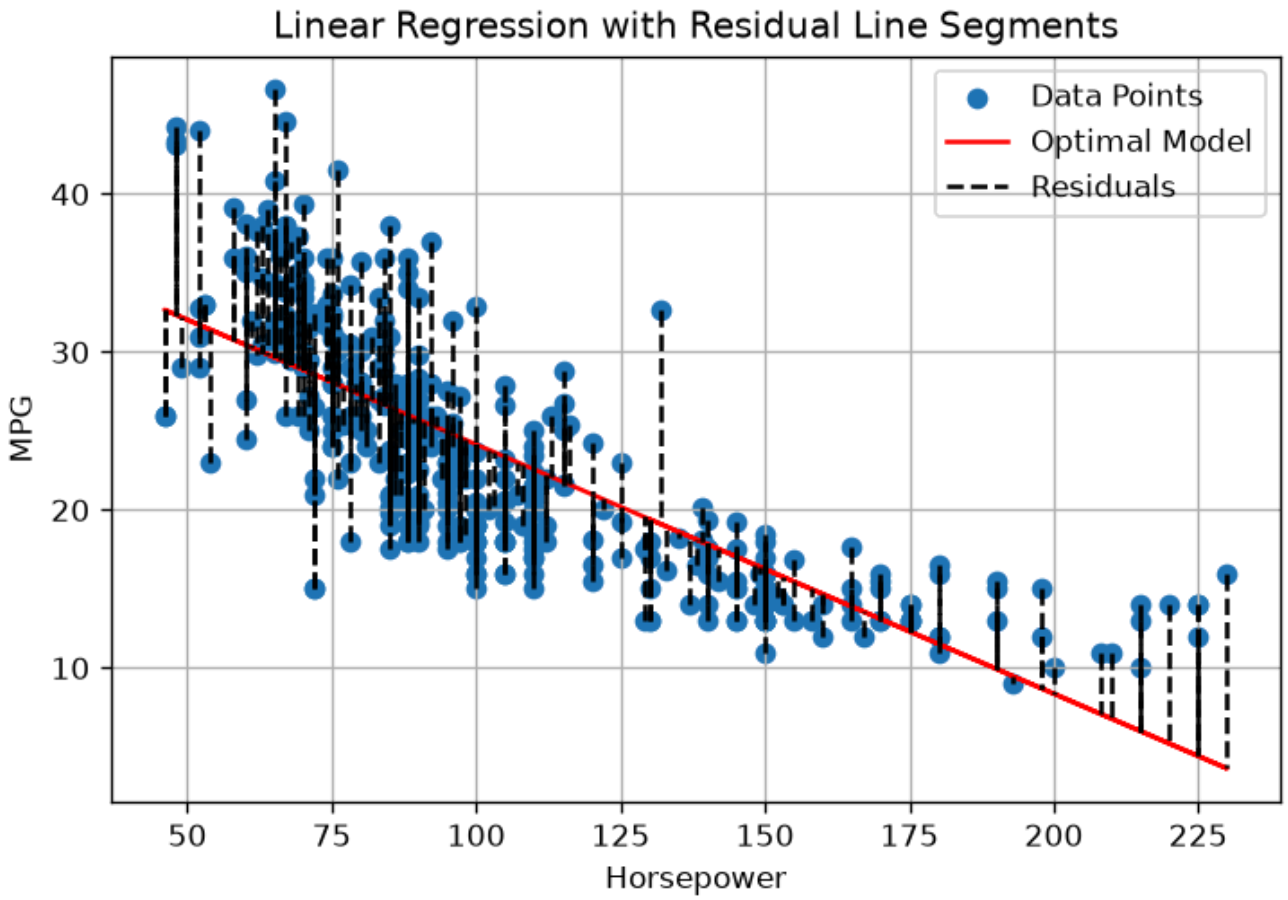
As a global error indicator for the model, given the data, we will define the **residual sum of squares (RSS)** as:

$$RSS = e_1^2 + e_2^2 + \ldots + e_n^2$$

or equivalently:

$$RSS = (y_1 - \hat{\beta}_0 - \hat{\beta}_1 x_1)^2 + (y_2 - \hat{\beta}_0 - \hat{\beta}_1 x_2)^2 + \ldots + (y_n - \hat{\beta}_0 - \hat{\beta}_1 x_n)^2$$

This number will be the sum of the square values of the dashed segments in the plot below:



Intuitively, if we minimize these numbers, we will find the line which **best fits the data**.

We can obtain estimates for $\hat{\beta}_0$ and $\hat{\beta}_1$ by minimizing the RSS using an approach called **ordinary least squares**.

We can write the RSS as a function of the parameters to estimate:

$$RSS(\beta_0, \beta_1) = \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2$$

This is also called a **cost function** or **loss function**.

We aim to find:

$$(\hat{\beta}_0, \hat{\beta}_1) = \arg\min_{\beta_0, \beta_1} RSS(\beta_0, \beta_1)$$

The minimum can be found setting:

$$\frac{\partial RSS(\beta_0, \beta_1)}{\partial \beta_0} = 0$$

$$\frac{\partial RSS(\beta_0, \beta_1)}{\partial \beta_1} = 0$$

Doing the math, it can be shown that:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y})}{\sum_{i=1}^{n}(x_i - \overline{x})}$$

$$\hat{\beta}_0 = \overline{y} - \hat{\beta}_1 \overline{x}$$

ⓘ **The same fit, seen as learning**

Minimising the residual sum of squares is a special case of a much more general recipe: choose a loss, average it over the data, and pick the hypothesis that minimises the average. That recipe is **empirical risk minimisation**, and it is the frame the whole of Module 2 is built on — see chapter 13. For now it is enough to know that ordinary least squares is what it reduces to when the loss is the squared error.

Interpretation of the Coefficients of Linear Regression

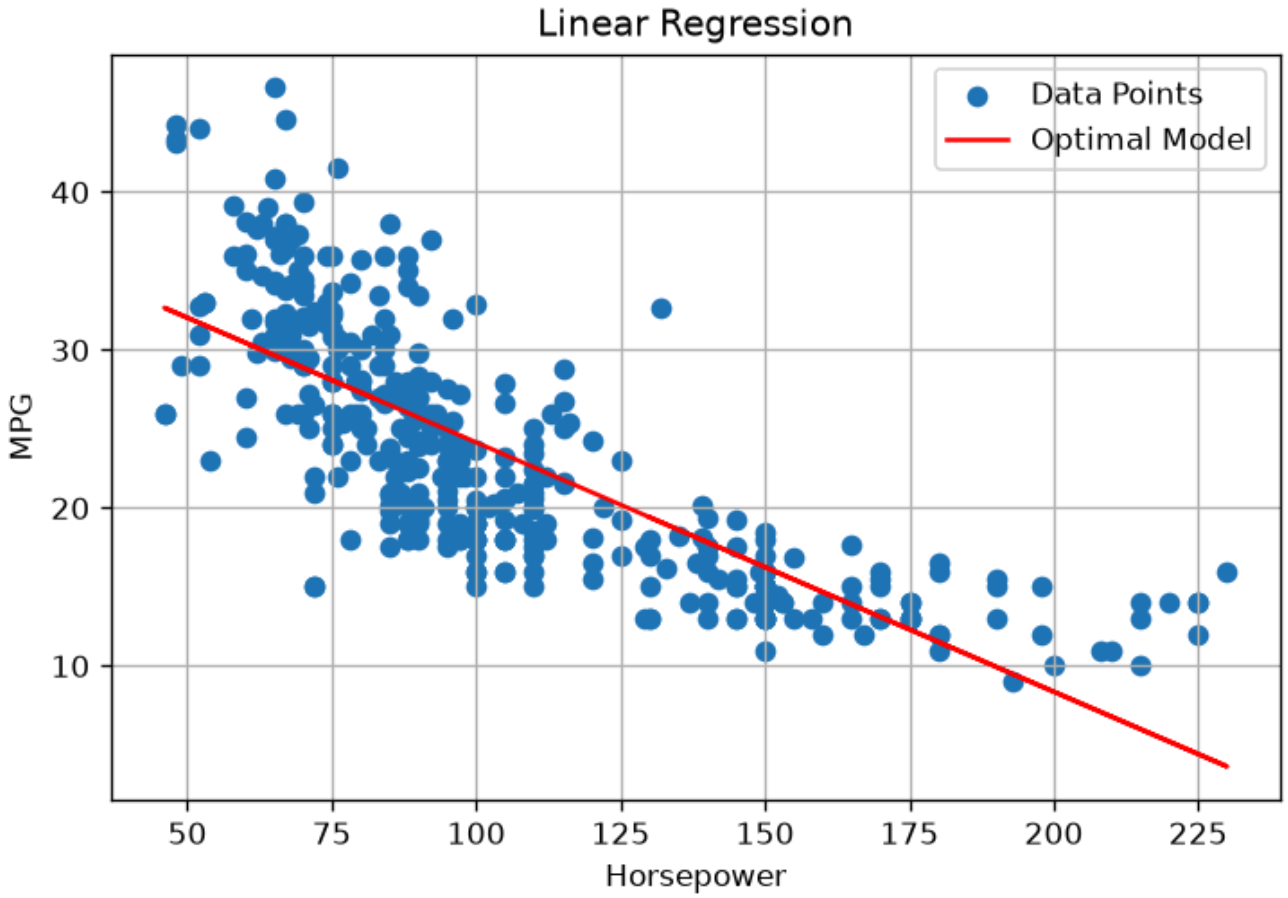
Using the formulas above, we find the following values for the example above:

```
beta_0: 39.94
beta_1: -0.16
```

These parameters identify the following line:

$$y = 39.94 - 0.16x$$

The plot below shows the line on the data:



Apart from the geometric interpretation, the coefficients of a linear regressor have an important **statistical interpretation**. In particular:

The intercept β_0 is the value of y that we get when the input value x is equal to zero $x = 0$ (i.e., $f(0)$). This value **may not always make sense**. For instance, in the example above, we have: $\beta_0 = 39.94$, which means that, **when the horsepower is 0, then the consumption in mpg is equal to 39.94**.

The coefficient β_1 indicates the steepness of the curve. If β_1 is large, then the curve is steep. This indicates that a small change in x is associated to a large change in y. In general, we can see that:

$$f(x+1) - f(x) = \beta_0 + \beta_1(x+1) - \beta_0 - \beta_1 x = \beta_1(x+1-x) = \beta_1$$

which reveals that **when we observe an increment of one unit of x, we observe an increment of β_1 units in y**. In our example, $\beta_1 = -0.16$, hence we can say that, for cars with one additional unit of horsepower, we observe a drop in mpg of -0.16 units.

Accuracy of the Coefficient Estimates

Recall that we are trying to model the relationship between two random variables X as Y with a simple linear model:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

This means that, once we find appropriate values of β_0 and β_1, we expect these to summarize the **linear relationship in the population** or the **population regression line**. Also, recall that these values are obtained using two formulas which are based on realizations of X of Y and can be hence seen as **estimators**:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y})}{\sum_{i=1}^{n}(x_i - \overline{x})^2}$$

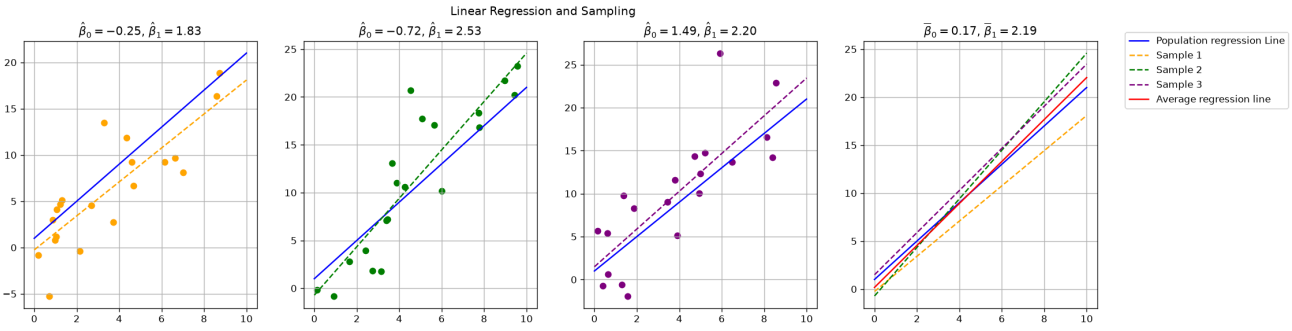
$$\hat{\beta}_0 = \overline{y} - \hat{\beta}_1 \overline{x}$$

We now recall that, being estimates, they provide values related to a given **realization of the random variables**.

Let us consider an ideal population for which:

$$Y = 2x + 1$$

Ideally, given a sample from the population, we expect to obtain $\hat{\beta}_0 \approx 1$ and $\hat{\beta}_1 \approx 2$. In practice, different samples may lead to different estimates and hence different regression lines, as shown in the plot below:



Each of the first three subplots shows a different sample drawn from the population, with its corresponding estimated regression line, along with the true population regression line. The last subplot compares the different estimated lines with the population regression line and the average regression line (in red). Coefficient estimates are shown in the subplot titles.

As can be noted, each estimate can be inaccurate, while the average regression line is very close to the population regression line. This is due to the fact that our estimators for the parameters of the regression coefficients **have non-zero variance**. In practice, it can be shown that these estimators are **unbiased** (hence the average regression line is close to the population line).

Confidence Intervals of the Regression Coefficients

Since the formulas to compute regression parameters can be seen as estimators, we can compute **standard errors** and **confidence intervals**.

For instance, for our model

$$\text{mpg} \approx \beta_0 + \text{horsepower} \cdot \beta_1$$

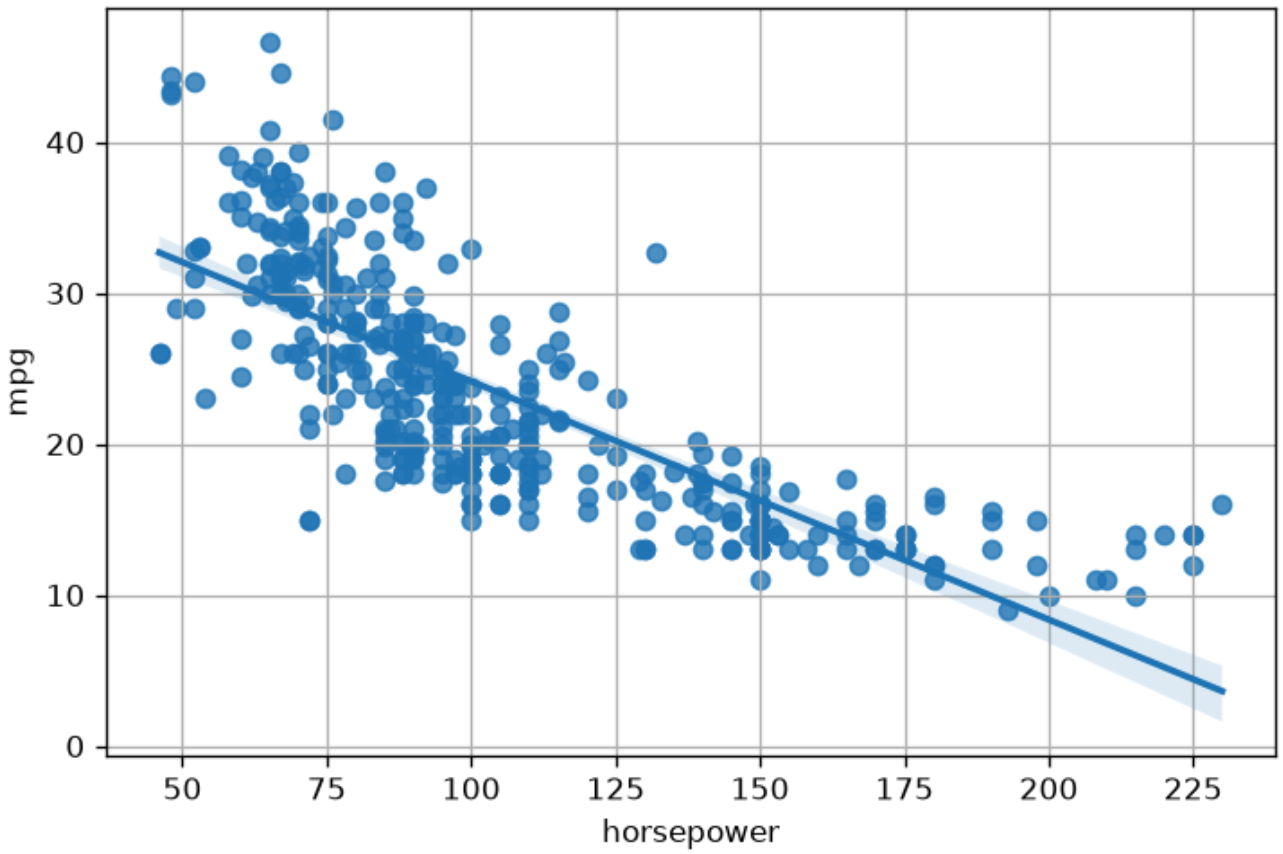
We will have the following confidence interval at a 95% confidence level (which is a common CL):

	COEFFICIENT	STD ERROR	CONFIDENCE INTERVAL
/	39.94	0.717	$[38.53, 41.35]$
/	-0.1578	0.006	$[-0.17, -0.15]$

From the table above, we can say that:

- The value of `mpg` for `horsepower=0` lies somewhere between 38.53 and 41.35;
- An increase of `horsepower` by one unit is associated to a decrease of `mpg` between -0.17 and -0.15.

It is also common to see plots like the following one:



In the plot above, the shading around the line illustrates the variability induced by the confidence intervals estimated for the coefficients.

Statistical Tests for the Significance of Coefficients

Seeing the aforementioned formulas as estimators also allows to perform **hypothesis tests** on the coefficients. In practice, it is common to perform a statistical test to assess whether the coefficient β_1 is significantly different from zero. It is interesting to check this because, if β_1 was equal to zero, then there would not be any correlation between the variables (and hence the linear regressor would not be useful). Indeed, if $\beta_1 = 0$:

$$Y = \beta_0 + \epsilon$$

Hence Y cannot be predicted from X and the two variables are not associated.

The **null hypothesis** of the test is as follows:

$$H_0 : \text{There is no association between } X \text{ and } Y \Leftrightarrow \beta_1 = 0$$

While the **alternative hypothesis** is formulated as follows:

$$H_1 : \text{There is some association between } X \text{ and } Y \Leftrightarrow \beta_1 \neq 0$$

We won't see the mathematical details of this test, but it works as usual: we compute some statistic t following a known distribution (a t-Student distribution in this case), then compute a **p-value** and reject the null hypothesis if this is under the significance level $p < \alpha$ (often set to $\alpha = 0.05$).

A similar test is conducted to check whether β_0 is significantly different from zero. But in this case it is not a big deal if the test fails as $\beta_0 = 0$ is a perfectly reasonable result (the regression line passes through the origin).

Let's see the updated table from the same example:

	COEFFICIENT	STD ERROR	t	P>|t|	CONFIDENCE INTERVAL
β_0	39.94	0.717	55.66	0	$[38.53, 41.35]$
β_1	-0.1578	0.006	-24.49	0	$[-0.17, -0.15]$

From the table above, we can conclude that both β_0 and β_1 are significantly different than 0 (the p-value is equal to zero). This can also be noted by the fact that the confidence intervals do not contain the zero number.

Assessing Model Accuracy

The hypothesis tests (like the t-test for β_1) tell us *if* a statistically significant relationship exists. They do not, however, tell us **how well the model fits the data** or **how accurate its predictions are**.

To measure performance, we use different metrics that generally fall into two categories, aligning with our "two goals":

1. **Metrics for Understanding (Goodness-of-Fit):** These are used in the "Statistical" approach to measure how well the model explains the data it was trained on. (e.g., RSE, R^2)
2. **Metrics for Prediction (Predictive Accuracy):** These are used in the "Machine Learning" approach to measure how well the model performs on new, unseen data. (e.g., MSE, RMSE, MAE)

The Building Block: Residuals and RSS

All regression metrics are built upon the **residuals**—the errors from the model. A residual (e_i) is the difference between the observed value (y_i) and the predicted value ($\hat{y}_i$):

$$e_i = y_i - \hat{y}_i$$

The **Residual Sum of Squares (RSS)** which we already saw is the foundational quantity that OLS (Ordinary Least Squares) minimizes. It is the sum of all squared residuals:

$$RSS = \sum_{i=1}^{n} e_i^2 = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Metrics for Understanding (Goodness-of-Fit)

When our goal is **understanding** (inference), we typically evaluate the model on the entire dataset it was trained on.

1. Residual Standard Error (RSE)

The RSE is the *statistical* estimate of the standard deviation of the irreducible error, ϵ. It measures the "typical" error of the model.

$$RSE = \sqrt{\frac{RSS}{n-2}} = \sqrt{\frac{1}{n-2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- **Why $n-2$?** We divide by the "degrees of freedom" ($n-2$) because we estimated *two* parameters (β_0 and β_1). This makes RSE an unbiased estimate of σ (the standard deviation of ϵ).
- **Interpretation:** RSE is an **absolute measure** of the model's "lack of fit," expressed in the **same units as Y**. A smaller RSE means the model fits the data better.

To know if an RSE value is "good," you must compare it to the scale of Y.

- **Example:** For our `mpg` data, $RSE = 4.91$. The mean value of `mpg` is $\overline{y} = 23.52$. This means our model's typical error (4.91) is about **20%** of the average value, which may be acceptable or not, depending on the context.

2. The R^2 Statistic

The RSE is an absolute measure. It's often more useful to have a *relative* measure that tells us what proportion of the data's variance our model can explain. This is the R^2 **(Coefficient of Determination)**.

To calculate R^2, we first define a "baseline" model that just predicts the average Y for every X:

$$\hat{y}_{baseline} = \overline{y}$$

The error of this "dumb" model is the **Total Sum of Squares (TSS)**:

$$TSS = \sum_{i=1}^{n} (y_i - \overline{y})^2$$

- **TSS** measures the **total variance** in Y (before any regression).
- **RSS** measures the **unexplained variance** in Y (what's left over *after* regression).

The R^2 statistic is the proportion of total variance that is "explained" by the model:

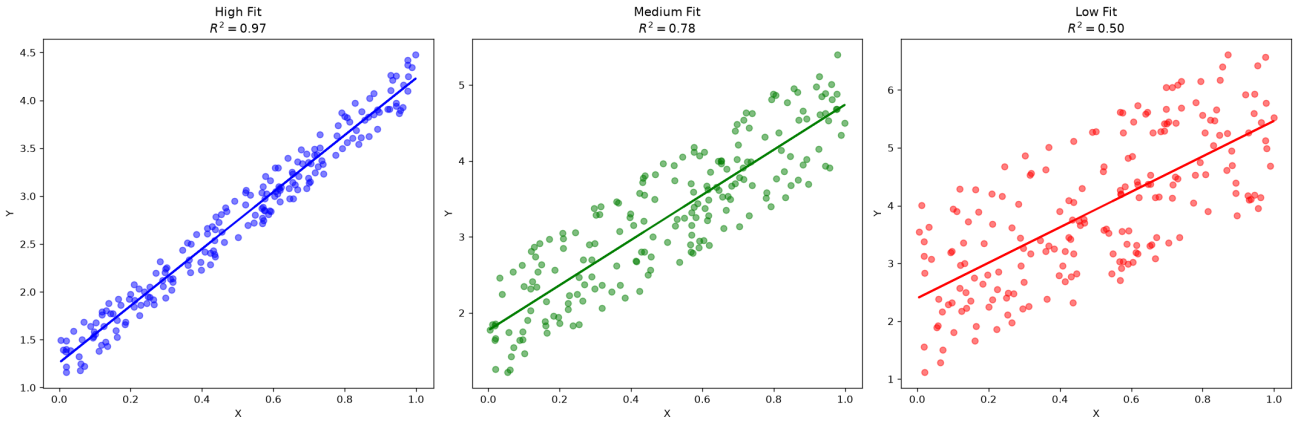
$$R^2 = \frac{TSS - RSS}{TSS} = 1 - \frac{RSS}{TSS}$$

- **Interpretation:** R^2 is a value between 0 and 1 (or 0% to 100%).
  - $R^2 = 0$: Our model is no better than just guessing the mean ($RSS = TSS$).
  - $R^2 = 1$: Our model perfectly explains all the variance ($RSS = 0$).
  - **Example:** For our `mpg` data, $R^2 = 0.61$. This means **"61% of the variance in `mpg` can be explained by `horsepower`."**

For simple linear regression (one X), R^2 is also equal to the square of the Pearson correlation coefficient: $R^2 = \rho^2$.

Visual Examples

The plot below shows examples of linear regression fits with the different evaluation measures that we saw:



Diagnostic Plots: Checking Model Validity

The metrics above (R^2, $RMSE$) are "scoring" metrics. They give you a single number to summarize *how much* error you have.

However, they don't tell you *why* you have error, or if your model is fundamentally **invalid**. A model can have a "good" R^2 but still be flawed. We use **residual plots** as *diagnostic* tools to check this.

1. The Residuals vs. Fitted Plot

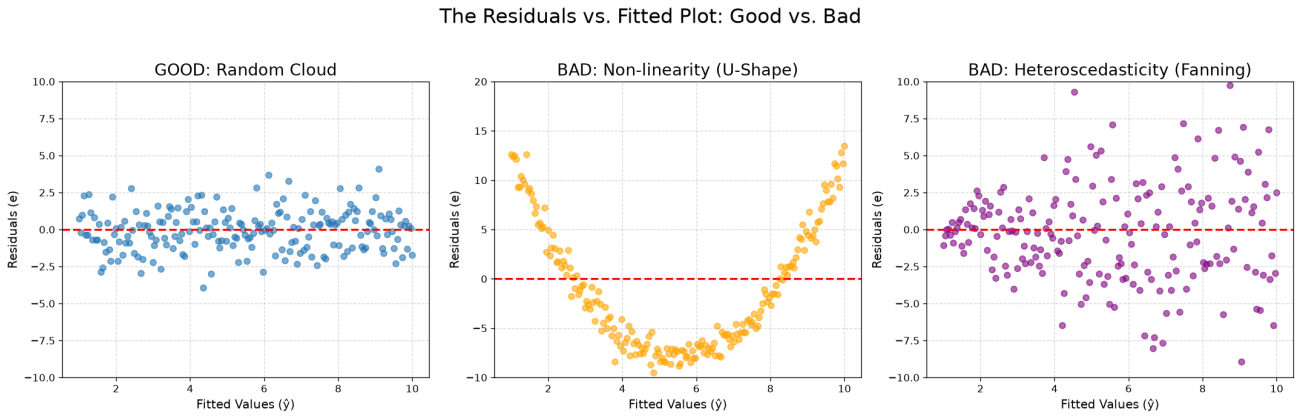
This is the most important diagnostic plot. We plot the **Residuals (e_i)** on the y-axis against the **Fitted (Predicted) Values ($\hat{y}_i$)** on the x-axis.

What we *want* to see: A boring, random cloud of points with no pattern, centered around the $y = 0$ line. This indicates our model has captured the pattern and the remaining errors are just random noise.

What we *don't* want to see (Red Flags):

- **A "U-Shape" or any Clear Pattern:** This means our model is **wrong**. The relationship between X and Y was **not linear**, and our simple linear model has failed to capture the true non-linear pattern.
  - **Diagnosis:** Violates the **Linearity** assumption.
  - **Fix:** You need a more complex model (e.g., polynomial regression).
- **A "Fanning" or "Cone" Shape:** The residuals get bigger (more spread out) as the predicted value $\hat{y}$ gets bigger.
  - **Diagnosis:** This is called **Heteroscedasticity** (non-constant variance).
  - **Problem:** This violates a core assumption of linear regression. It means our p-values and confidence intervals are unreliable because the model is less accurate for high-value predictions.

The figure below shows some examples of what to expect:

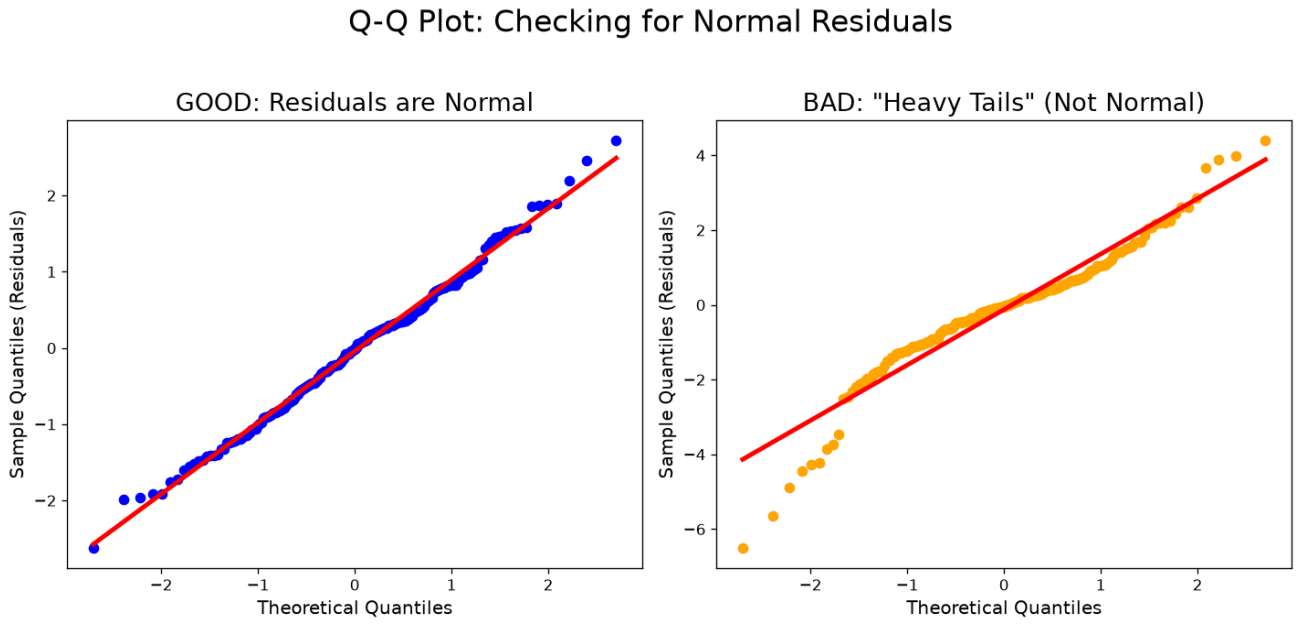


2. The Q-Q Plot (Quantile-Quantile Plot)

This is a plot used to check if the residuals are **normally distributed**. This is another key assumption for all our statistical tests (p-values, t-statistics).

- **Purpose:** To check the **Normality** assumption.
- **How to read it:** If the residuals are truly normally distributed, the points will fall perfectly along the diagonal line.
- **Problem:** If the points "S-curve" or peel away from the line, it means the residuals are not normal, and our p-values and confidence intervals may be inaccurate.

The figure below shows examples of "good" and "bad" residuals:



Multiple Linear Regression

In the **MPG example**, we saw that about 61% of the variance in Y was explained by the model. One may wonder why about 39% of the variance could not be explained. Some common reasons may be:

- There is stochasticity in the data which prevents us to learn an accurate function to predict Y from X;
- The relationship between X and Y is far from linear, so we cannot predict Y accurately;
- The prediction of Y also depends on other variables.

While in general the unexplained variance is due to a combination of the aforementioned factors, the third one is often very common and relatively easy to fix. In our case, we are trying to predict `mpg` from `horsepower`. However, we can easily imagine how other variables may contribute to the estimation of `mpg`. For instance, two cars with the same `horsepower` but different `weight` may have different values of `mpg`. We can hence try to find a model with **also uses `weight` to predict `mpg`**. This is simply done by adding an additional coefficient for the new variable:

$$mpg = \beta_0 + \beta_1 horsepower + \beta_2 weight$$

The obtained model is called **multiple linear regression**. If we fit this model (we will see how to estimate coefficients in this case), we obtain the following R^2 value:

$$R^2 = 0.71$$

An increment of $+0.1$!

In general, we can include as many variables as we think is relevant to add and define the following model:

$$Y = \beta_0 + \beta_1 X_1 + \ldots + \beta_n X_n$$

For instance, the following model:

$$mpg = \beta_0 + \beta_1 horsepower + \beta_2 weight + \beta_3 model_year$$

Has an R^2 value of:

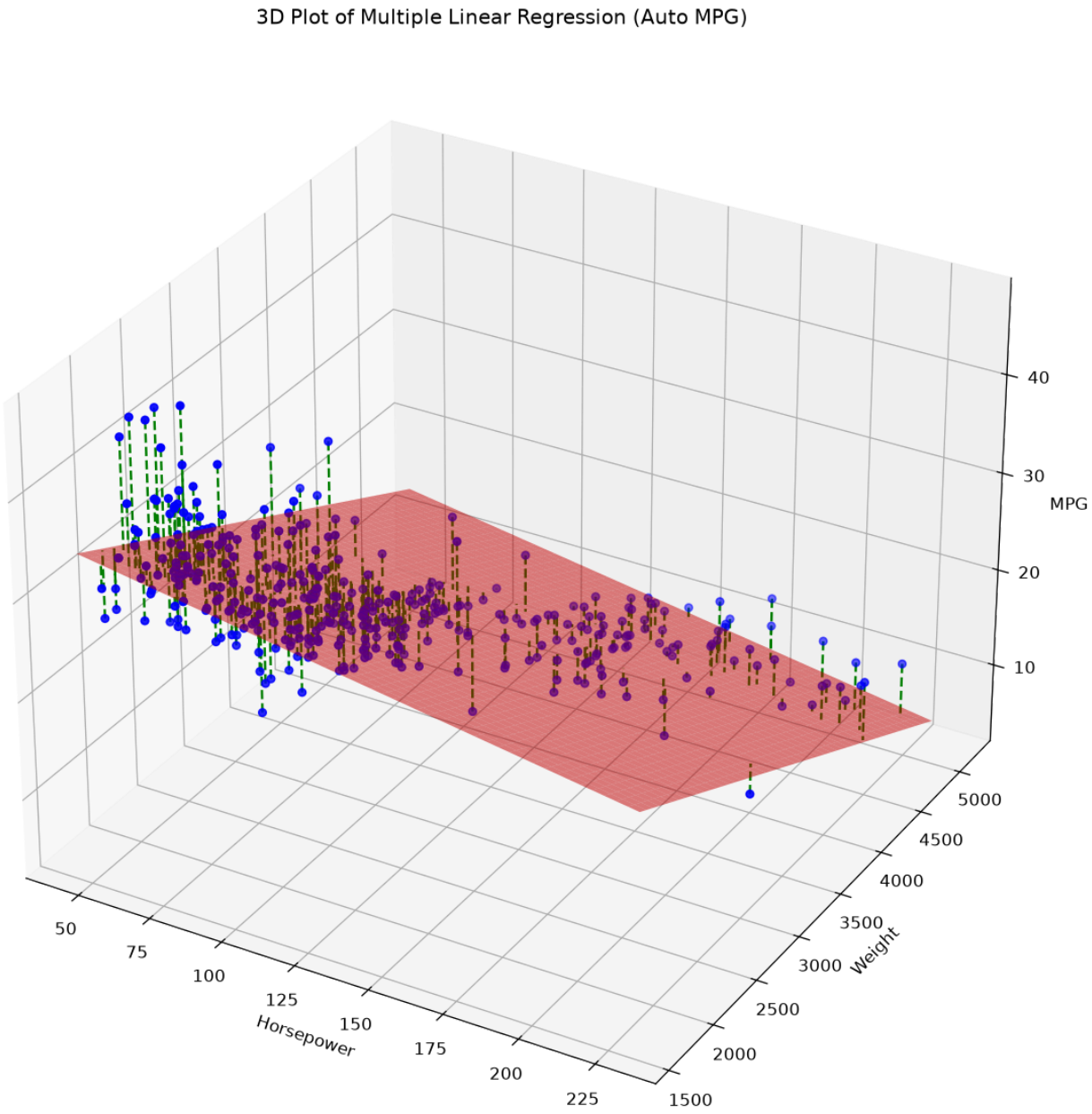
$$R^2 = 0.808$$

Geometrical Interpretation

The multiple regression model based on two variable has a geometrical interpretation. Indeed, the equation:

$$Z = \beta_0 + \beta_1 X + \beta_2 Y$$

identifies a plane in the 3D space. We can visualize the plane identified by the $mpg = \beta_0 + \beta_1 horsepower + \beta_2 weight$ model as follows:



The dashed lines indicate the residuals. The best fit of the model minimizes again the sum of squared residuals. This model makes predictions selecting the Z value which intersect the plane for given values of X and Y.

In general, when we consider n variables, the linear regressor will be **a (n-1)-dimensional hyperplane in the n-dimensional space**.

Statistical Interpretation

The statistical interpretation of a multiple linear regression model is very similar to the interpretation of a simple linear regression model. Given the general model:

$$y = \beta_0 + \beta_1 x_1 + \ldots + \beta_i x_i + \ldots + \beta_n x_n$$

we can interpret the coefficients as follows:

- The value of β_0 indicates the value of y when all independent variables are set to zero;
- The value of β_i indicates the increment of y that we expect to see when x_i increments by one unit, **provided that all other values $x_j | j \neq i$ are constant**.

In the considered example:

$$mpg = \beta_0 + \beta_1 horsepower + \beta_2 weight$$

we obtain the following estimates for the coefficients:

$\hat{\beta}_0$	$\hat{\beta}_1$	$\hat{\beta}_2$
45.64	-0.05	-0.01

We can interpret these estimates as follows:

- Cars with zero `horsepower` and zero `weight` will have an `mpg` of 45.64 ($\approx 19.4 Km/l$).
- An increment of one unit of `horsepower` is associated to a decrement of `mpg` of -0.05 units, provided that `weight` is constant. This makes sense: cars with more `horsepower` will probably consume more fuel.
- An increment of one unit of `weight` is associated to a decrement of `mpg` of `-0.01` units, provided that `horsepower` is constant. This makes sense: heavier cars will consume more fuel.

Let's compare the estimates above with the estimates of our previous model:

$$mpg = \beta_0 + \beta_1 horsepower$$

In that case, we obtained:

$\hat{\beta}_0$	$\hat{\beta}_1$
39.94	-0.16

We can note that the coefficients are different. This happens because, when we add more variables, **the model explains variance in a different way**. If we think more about it, this is coherent with the interpretation of the coefficients. Indeed:

- 39.94 is the expected value of `mpg` when `horsepower=0`, but all other variables have unknown values. 45.64 is the expected value of `mpg` when `horsepower=0` and `weight=0`. This is different, as in the second case we are (virtually) looking at a subset of data for which both horsepower and weight are zero, while in the first case, we are only looking at data for which `horsepower=0`, but `weight` can be any value. In some sense, we can see 39.94 as an average value for different values of `weight` (and all other unobserved variables).

- -0.16 is the expected increment of `mpg` when we observe an increment of one unit of `horsepower` and we don't know anything about the values of the other variables. -0.05 is the expected increment of `mpg` when `horsepower` and `weight` are held constant, so, again, we are (virtually) looking at a different subset of the data in which the relationship between `mpg` and `horsepower` may be a bit different.

Note that, also in the case of multiple regression, we can estimate confidence intervals and perform statistical tests. In our example, we will get this table:

	COEFFICIENT	STD ERROR	t	P>|t|	CONFIDENCE INTERVAL
/	45.64	0.793	57.54	0	$[44.08, 47.20]$
/	-0.05	0.011	-4.26	0	$[-0.07, -0.03]$
/	-0.01	0.001	-11.53	0	$[-0.007, -0.005]$

Estimating the Regression Coefficients

Given the general model:

$$y = \beta_0 + \beta_1 x_1 + \ldots + \beta_i x_i + \ldots + \beta_n x_n$$

We can define our **cost function** again as the residual sum of squares:

$$RSS(\beta_0, \ldots, \beta_n) = \sum_{i=1}^{m} (y_i - \beta_0 + \beta_1 x_1^{(i)} + \ldots + \beta_n x_n^{(i)})^2$$

Where m is the total number of observations and $x_j^{(i)}$ is the j^{th} variable (x_j) of the i^{th} observations.

The $\hat{\beta}_0, \ldots, \hat{\beta}_n$ values that minimize the loss function above are the **multiple least square coefficient estimates**.

To find these optimal values, it is convenient to use matrix notation. Given m observations, we will have m equations:

$$y^{(1)} = \beta_0 + \beta_1 x_1^{(1)} + \ldots + \beta_n x_n^{(1)} + e^{(1)}$$

$$y^{(2)} = \beta_0 + \beta_1 x_1^{(2)} + \ldots + \beta_n x_n^{(2)} + e^{(2)}$$

$$\dots$$

$$y^{(m)} = \beta_0 + \beta_1 x_1^{(m)} + \dots + \beta_n x_n^{(m)} + e^{(m)}$$

We can write the m equations in matrix form as follows:

$$\mathbf{y} = \mathbf{X}\beta + \mathbf{e}$$

where:

$$\mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}, \mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \vdots & & \\ 1 & x_1^{(m)} & x_2^{(m)} & \dots & x_n^{(m)} \end{bmatrix}, \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_n \end{bmatrix}, \mathbf{e} = \begin{bmatrix} e^{(1)} \\ e^{(2)} \\ \vdots \\ e^{(m)} \end{bmatrix}$$

The matrix $\mathbf{X}$ is s **design matrix**.

In the notation above, we want to minimize:

$$RSS(\beta) = \sum_{i=1}^{m} (e^{(i)})^2 = \mathbf{e}^T\mathbf{e}$$

It can be shown that, by the **least squares method**, the RSS is minimized by the estimate:

$$\hat{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

The F-Test

When fitting a multiple linear regressor, it is common to perform a statistical test to check whether at least one of the regression coefficients is significantly different from zero (in the population). This test is called an $F-test$. We define the **null and alternative hypotheses** as follows:

$$H_0 : \beta_1 = \beta_2 = \dots = \beta_n = 0$$

$$H_a : \exists j \; s.t. \; \beta_j \neq 0$$

The null hypothesis (the one we want to reject with this test) is that all coefficients are zero in the population. If this is true, than the multiple regressor is not reliable and we should discard it. Note that we exclude β_0 from these coefficients because it represents the position of the regression line (intercept) and does not denote correlation. The alternative hypothesis is that at least one of the coefficients is different from zero.

We will not see the mathematical details, but in this case, we compute an **F-statistic** which follows a **F-distribution**.

The test is hence carried out as usual, finding a p-value which indicates the probability to observe a statistic larger than the observed one if all regression coefficients are zero in the population.

In our example of regressing `mpg` from `horsepower` and `weight`, we will find:

R^2	F-statistic	Prob(F-statistic)
0.706	467.9	3.06e-104

This indicates that the regressor is statistically relevant as the p-value (Prob(F-statistic)) is very small (under the significance level $\alpha = 0.05$).

Variable Selection by Backward Elimination

Let's now try to fit a multiple linear regressor on our dataset by including all variables. Our dependent variable will be `mpg`, while the set of dependent variables will be:

```
['displacement' 'cylinders' 'horsepower' 'weight' 'acceleration'
 'model_year' 'origin']
```

We obtain the following measures of fit:

R^2	F-statistic	Prob(F-statistic)
0.821	252.4	2.04e-139

The regressor has a good R^2 and the p-value of the F-test is very small. We can conclude that there is some relationship between the independent variables and the dependent one.

The estimates of the regression coefficients will be:

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-17.2184	4.644	-3.707	0.000	-26.350	-8.087
horsepower	-0.0170	0.014	-1.230	0.220	-0.044	0.010
weight	-0.0065	0.001	-9.929	0.000	-0.008	-0.005
displacement	0.0199	0.008	2.647	0.008	0.005	0.035
cylinders	-0.4934	0.323	-1.526	0.128	-1.129	0.142
acceleration	0.0806	0.099	0.815	0.415	-0.114	0.275
model_year	0.7508	0.051	14.729	0.000	0.651	0.851
origin	1.4261	0.278	5.127	0.000	0.879	1.973

From the table, we can see that not all predictors have a p-value below the significance level $\alpha = 0.05$. In particular:

- `horsepower` has a large p-value of 0.22;

- `cylinders` has a large p-value of 0.128;
- `acceleration` has a large p-value of 0.415.

This means that, within the current regressor, there is no meaningful relationship between these variables and `mpg`. A legitimate question is

> How is it possible that `horsepower` is not associated to `mpg` in this regressor if it was associated to it before?!

However, we should recall that, when we consider a different set of variables, the interpretation of the coefficients changes. So, even if in the previous models, `horsepower` was correlated to `mpg`, now it is not correlated anymore. We can imagine that the relationship between these variables is now explained by the other variables which we have introduced.

Even if the model is statistically significant, it does make sense to get rid of the variables with poor relationships with `mpg`. After all, if we remove a variable, the estimates of the other coefficients may change.

A common way to remove these variables is by **backward selection** or **backward elimination**. This consists in iteratively removing the variable with the largest p-value. We remove one variable at a time and re-compute the results, iterating until all variables have a small p-value.

let's start by removing `acceleration`. This is the result:

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-15.5635	4.175	-3.728	0.000	-23.773	-7.354
horsepower	-0.0239	0.011	-2.205	0.028	-0.045	-0.003
weight	-0.0062	0.001	-10.883	0.000	-0.007	-0.005
displacement	0.0193	0.007	2.579	0.010	0.005	0.034
cylinders	-0.5067	0.323	-1.570	0.117	-1.141	0.128
model_year	0.7475	0.051	14.717	0.000	0.648	0.847
origin	1.4282	0.278	5.138	0.000	0.882	1.975

Note that the coefficients have changed. We now remove `cylinders`, which has the largest p-value of 0.117:

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-16.6939	4.120	-4.051	0.000	-24.795	-8.592
horsepower	-0.0219	0.011	-2.033	0.043	-0.043	-0.001
weight	-0.0063	0.001	-11.124	0.000	-0.007	-0.005
displacement	0.0114	0.006	2.054	0.041	0.000	0.022
model_year	0.7484	0.051	14.707	0.000	0.648	0.848
origin	1.3853	0.277	4.998	0.000	0.840	1.930

All variables now have an acceptable p-value ($\alpha = 0.05$). We are done. Note that, by removing the two variables, `horsepower` now has an acceptable p-value. This indicates that one of the removed variables was redundant with respect to `horsepower`.

Collinearity and the Instability of Least Squares

Let's reflect again on the issue we saw in our model: the `horsepower` p-value was significant in the simple model but became non-significant ($p > 0.05$) in the multiple regression. This isn't a contradiction; it's a classic symptom of **collinearity**.

Collinearity is a high linear correlation between two predictor variables. The more general (and common) problem is **multicollinearity**, which occurs when one predictor X_j can be accurately predicted by a linear combination of *other* predictors in the model (e.g., $X_j \approx \beta_0 + \beta_1 X_1 + \dots$).

The primary problem with multicollinearity is that it makes our coefficient estimates **unstable and unreliable**. The standard errors of the coefficients for the correlated predictors become massively inflated. This is what we saw: `horsepower` became non-significant because `weight` (which is highly correlated with it) "stole" all of its explanatory power.

This instability can be explained by **looking directly at the mathematical solution for the OLS coefficients** $\hat{\beta}$:

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

The key to this entire equation is the $(X^T X)^{-1}$ term, which is the inverse of the "design matrix." The $X^T X$ matrix is, in essence, a matrix of the covariances between all of our predictor variables.

1. **Perfect Collinearity (Singularity):** If one variable is a perfect linear combination of another (e.g., $X_1 = 2 \cdot X_2$), the columns of X are not linearly independent. This causes the $X^T X$ matrix to be **singular**, meaning its determinant is zero and its inverse **does not exist**. There is no unique solution for $\hat{\beta}$.

2. **Multicollinearity (Ill-Conditioning):** In our `mpg` case, the variables aren't *perfectly* correlated, but *highly* correlated. This means $X^T X$ is not perfectly singular, but it is **"ill-conditioned"** or **"near-**

singular". Its determinant is *almost* zero.

When we compute the inverse, $(X^T X)^{-1}$, we are "dividing by almost zero." This causes the numbers *inside* the inverse matrix to explode, becoming massive.

The formula for the **standard error** of each coefficient $\hat{\beta}_j$ is directly proportional to the values in this inverse matrix. Therefore, if the values in $(X^T X)^{-1}$ are huge, the standard errors for our coefficients become **inflated**. This leads to a tiny t-statistic ($t = \hat{\beta}_j / SE(\hat{\beta}_j)$) and a massive, non-significant p-value. This is exactly what happened to `horsepower`.

When `statsmodels` (or `numpy`) encounters a *perfectly* singular $X^T X$ matrix (e.g., you forget to drop one dummy variable, creating perfect multicollinearity), it cannot compute the standard inverse.

Instead of crashing, it computes the **Moore-Penrose Pseudoinverse**, denoted as $(X^T X)^+$. This is a generalization of the inverse that finds the *one* solution $\hat{\beta}$ that has the smallest ℓ_2-norm. In practice, `statsmodels` will identify this redundancy, report that the matrix is singular, and often arbitrarily set one of the redundant variable's coefficients to zero to provide a stable (but not unique) set of coefficients.

Adjusted R^2

While in the case of simple regression we saw that $R^2 = \rho(x, y)^2$ (where ρ is the correlation coefficient), in the case of multiple regression, it turns out that:

$$R^2 = \rho(Y, \hat{Y})^2$$

In general, having more variables in the linear regressor will reduce the error term and increase the covariance between Y and $\hat{Y}$, hence increasing the R^2. However, in general having a small increase in R^2 when we add a new variable may not be good. Indeed, we could prefer a simpler model with a slightly smaller R^2 value.

This is again a problem of bias-variance tradeoff.

To express this, we can compute the adjusted R^2 as follows:

$$\overline{R}^2 = 1 - \frac{m-1}{m-n-1}(1 - R^2)$$

Where m is the number of data points and n is the number of independent variables. The $\overline{R}^2$ re-balances the R^2 accounting for the introduction of additional variables.

For instance the last model we fit has an $R^2 = 0.820$ and $\overline{R}^2 = 0.818$. In general, when using multiple regressor and especially for variable selection, it is good practice to use the adjusted R^2 rather than standard R^2.

Qualitative Predictors

So far, we have studied relationships between continuous variables. In practice, linear regression allows to also study relationship between **continuous dependent variables** and **qualitative independent variables**. We will consider another dataset similar to the **Auto MPG** dataset:

	mpg	horsepower	fuelsystem	fueltype	length	cylinders
0	21	111.0	mpfi	gas	168.8	4
1	21	111.0	mpfi	gas	168.8	4
2	19	154.0	mpfi	gas	171.2	6
3	24	102.0	mpfi	gas	176.6	4
4	18	115.0	mpfi	gas	176.6	5
...	...	...	...	...	...	...
200	23	114.0	mpfi	gas	188.8	4
201	19	160.0	mpfi	gas	188.8	4
202	18	134.0	mpfi	gas	188.8	6
203	26	106.0	idi	diesel	188.8	6
204	19	114.0	mpfi	gas	188.8	4

205 rows × 6 columns

In this case, besides having numerical variables, we also have qualitative ones such as `fuelsystem` and `fueltype`. Let's see what are their unique values:

```
fuelsystem: ['mpfi' '2bbl' 'mfi' '1bbl' 'spfi' '4bbl' 'idi' 'spdi']
fueltype: ['gas' 'diesel']
```

We will not see the meaning of all the values of `fuelsystem`, while the values of `fueltype` are self-explanatory.

Predictors with Only Two Levels

We will first see the case in which qualitative predictors only have two levels. To handle these as independent variables, we can define a new **dummy variable** which will encode 1 as one of the two levels and 0 as the other one. For instance, we can introduce a `fueltype[T.gas]` variable defined as follows:

$$fueltype[T.gas] = \begin{cases} 1 & \text{if } fueltype = gas \\ 0 & \text{otherwise} \end{cases}$$

If we fit the model:

$$mpg = \beta_0 + \beta_1 horsepower + \beta_2 fueltype[T.gas]$$

We obtain an $R^2 = 0.661$ with $Prob(F-statistic) \approx 0$ and the following estimates for the regression parameters:

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	41.2379	1.039	39.705	0.000	39.190	43.286
fueltype[T.gas]	-2.7658	0.918	-3.013	0.003	-4.576	-0.956
horsepower	-0.1295	0.007	-18.758	0.000	-0.143	-0.116

How do we interpret this result?

- The value of `mpg` when `horsepower=0` `fueltype=diesel` (i.e., `fueltype[T.gas]=0`) is 41.2379 ;
- An increase of one unit of `horsepower` is associated to a decrease of 0.1295 units of `mpg` provided that `fueltype=diesel`;
- For gas vehicles we expect to see a decrease of `mpg` equal to 2.7658 with respect to diesel vehicles.

Predictors with More than Two Levels

When predictors have n levels, we need to introduce **multiple dummy variables**. Specifically, we need to introduce $n-1$ binary variables. For instance, if the levels of the variable `income` are `low`, `medium` and `high`, we could introduce two variables `income[T.low]` and `income[T.medium]`. These are sufficient to express all possible values of `income` as shown in the table below:

`income`	`income[T.low]`	`income[T.medium]`
`low`	1	0
`medium`	0	1
`high`	0	0

Note that we could have introduced a new variable `income[T.high]` but this would have been redundant and so **correlated to the other two variables**, which is something we know we have to avoid in linear regression.

If we fit the model which predicts `mpg` from `horsepower` and `fuelsystem` we obtain $R^2 = 0.734$, $Prob(F-statistic) \approx 0$ and the following estimates for the regression coefficients:

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	38.8638	1.234	31.504	0.000	36.431	41.297
fuelsystem[T.2bbl]	-1.6374	1.127	-1.453	0.148	-3.860	0.585
fuelsystem[T.4bbl]	-12.0875	2.263	-5.341	0.000	-16.551	-7.624
fuelsystem[T.idi]	-0.3894	1.300	-0.299	0.765	-2.954	2.175
fuelsystem[T.mfi]	-5.8285	3.661	-1.592	0.113	-13.049	1.392
fuelsystem[T.mpfi]	-5.4942	1.202	-4.570	0.000	-7.865	-3.123
fuelsystem[T.spdi]	-5.2446	1.612	-3.254	0.001	-8.423	-2.066
fuelsystem[T.spfi]	-6.1522	3.615	-1.702	0.090	-13.282	0.978
horsepower	-0.0968	0.009	-11.248	0.000	-0.114	-0.080

As we can see, we have added different correlation coefficients in order to deal with the different levels. Not all predictors have a low p-value, so we can remove those with backward elimination. We will see some more examples in the laboratory.

Linear Regression in Python

In this notebook, we will perform a complete statistical linear regression analysis. Our goal is **understanding** and **inference**. We want to find a model that is not only accurate but also **parsimonious** (simple) and **statistically valid**.

We will use the “Auto MPG” dataset and the `statsmodels` library.

Our process will be:

1. **Simple Linear Regression (SLR):** Fit a simple model and diagnose its problems.
2. **Multiple Linear Regression (MLR):** Fit a “full” model with all potential predictors.
3. **Variable Selection:** Use **Backward Elimination** based on p-values to simplify the model.
4. **Final Model Interpretation:** Interpret the coefficients of our final, parsimonious model.
5. **Diagnostic Plots:** Check the final model’s assumptions to ensure our conclusions are reliable.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Import statsmodels
# smf allows us to use the "formula API" (e.g., 'y ~ x')
# sm gives us access to more functions, like diagnostic plots
import statsmodels.api as sm
import statsmodels.formula.api as smf


# Load and Clean the Data
mpg = sns.load_dataset('mpg')

# Clean the data
mpg = mpg.drop('name', axis=1)
mpg['horsepower'] = pd.to_numeric(mpg['horsepower'], errors='coerce')

# Drop rows with missing values for this exercise
mpg_clean = mpg.dropna()

print(f"Shape of cleaned dataset: {mpg_clean.shape}")
mpg_clean.head()
```

```
Shape of cleaned dataset: (392, 8)
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin
0	18.0	8	307.0	130.0	3504	12.0	70	usa
1	15.0	8	350.0	165.0	3693	11.5	70	usa
2	18.0	8	318.0	150.0	3436	11.0	70	usa
3	16.0	8	304.0	150.0	3433	12.0	70	usa
4	17.0	8	302.0	140.0	3449	10.5	70	usa

Simple Linear Regression

Let's start with the simplest model: **"Can `horsepower` predict `mpg`?"**

We use the formula `mpg ~ horsepower`. `statsmodels` automatically adds the intercept (β_0).

```
# Define and fit the SLR model
# OLS stands for "Ordinary Least Squares"
model_slr = smf.ols('mpg ~ horsepower', data=mpg_clean).fit()

# Print the full summary table
print(model_slr.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.606
Model:                            OLS   Adj. R-squared:                  0.605
Method:                 Least Squares   F-statistic:                     599.7
Date:                Sat, 26 Sep 2026   Prob (F-statistic):           7.03e-81
Time:                        16:16:04   Log-Likelihood:                -1178.7
No. Observations:                 392   AIC:                             2361.
Df Residuals:                     390   BIC:                             2369.
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     39.9359      0.717     55.660      0.000      38.525      41.347
horsepower    -0.1578      0.006    -24.489      0.000      -0.171      -0.145
==============================================================================
Omnibus:                       16.432   Durbin-Watson:                   0.920
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               17.305
Skew:                           0.492   Prob(JB):                     0.000175
Kurtosis:                       3.299   Cond. No.                         322.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

The `.summary()` output is dense, but it's the heart of statistical analysis. Let's break it into three parts:

Part 1: The Top Table (Model Metadata)

Field	Meaning
`R-squared`	**0.606** (or 60.6%) - As we saw, this is the "proportion of variance explained." It tells us `horsepower` alone explains 60.6% of the variability in `mpg`.
`Adj. R-squared`	**0.605** - A "corrected" version of R^2 that penalizes for adding useless variables. In SLR, it's almost identical to R^2.
`Method:`	`Least Squares` - The method we used (OLS).
`No. Observations:`	**392** - The number of rows (n) used to build the model.
`F-statistic:`	**599.7** - A score that tests the *overall* significance of the model.
`Prob (F-statistic):`	**7.03e-81** - A p-value associated with the F-statistic. This is a tiny number ($0.000\ldots$ with 80 zeros). Since it is < 0.05, it tells us our model as a whole is statistically significant (i.e., it's far better than a null model that just guesses the mean).

Part 2: The Coefficients Table (The Core)

Field	`Intercept` (β_0)	`horsepower` (β_1)
`coef`	**39.9359**	**-0.1578**
`std err`	0.717	0.006
`t`	55.660	-24.489
`P>|t|`	**< 0.001**	**< 0.001**
`[0.025 ... 0.975]`	[38.525, 41.347]	[-0.170, -0.146]

Interpreting the Coefficients

- `Intercept (coef)`: 39.93. This is our β_0. Theoretically, it's the predicted value of `mpg` when `horsepower` is 0. (In this context, it has no real practical interpretation, but it's mathematically necessary).
- `horsepower (coef)`: -0.1578. This is our β_1, the slope. It's the most important part.
  - **Interpretation:** "For every **1-unit increase** in `horsepower`, we expect a **0.1578 decrease** in `mpg`, holding all else constant."

Interpreting the Significance (p-value)

- `horsepower (P>|t|)`: This is `< 0.001`. This is the p-value for the null hypothesis $H_0 : \beta_1 = 0$.
  - **Conclusion:** Because the p-value is much smaller than $\alpha = 0.05$, we **reject the null hypothesis**. We can conclude that **there is a statistically significant relationship** between `horsepower` and `mpg`.

Interpreting the Confidence Interval (CI):

- `horsepower ([0.025 ... 0.975])`: [-0.170, -0.146]
  - **Interpretation:** "We are 95% confident that the *true* value of the slope (β_1) in the population lies between -0.170 and -0.146."
  - **Note:** The interval **does not include 0**, which confirms our conclusion from the p-value.

Part 3: The Bottom Table (Residual Diagnostics)

This table provides tests to check if the residuals (ϵ) meet the regression assumptions (like normality).

Residual Diagnostics

The coefficients table (p-values, CIs, etc.) is **only valid if** our model meets the **assumptions of linear regression**. The two main checks we must perform are:

1. **Linearity and Homoscedasticity:** Residuals should be randomly scattered around 0 with no clear pattern (e.g., no "U-shape" or "funnel").
2. **Normality:** The residuals should follow a normal distribution.

We use diagnostic plots to check these.

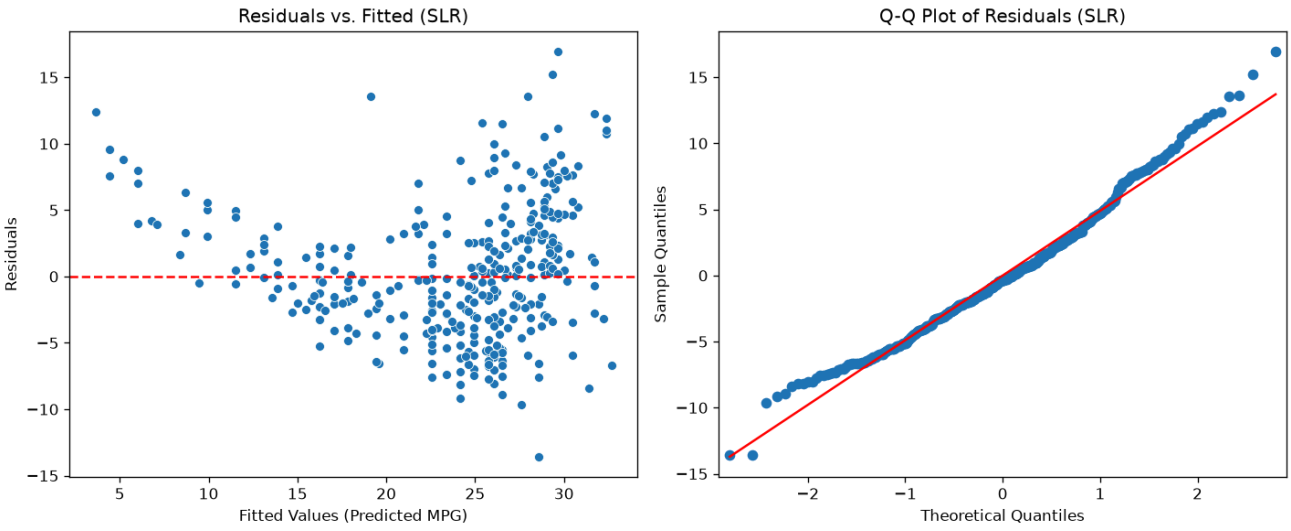
Interpreting the SLR Summary

- `Adj. R-squared`: 0.605. `horsepower` alone explains about 60.5% of the variance in `mpg`.
- `Prob (F-statistic)`: 7.03e-81. This is nearly zero, so the model is statistically significant (better than just guessing the mean).
- `horsepower coef` (β_1): -0.1578.

- `horsepower P>|t|`: < 0.001. The p-value is tiny, so we are confident there is a real, statistically significant relationship between `horsepower` and `mpg`.

This looks good, but is the model *valid*? Let's check the diagnostic plots.

```
# --- SLR Diagnostic Plots ---
fitted_values_slr = model_slr.fittedvalues
residuals_slr = model_slr.resid

plt.figure(figsize=(12, 5))

# 1. Residuals vs. Fitted Plot
ax1 = plt.subplot(1, 2, 1)
sns.scatterplot(x=fitted_values_slr, y=residuals_slr, ax=ax1)
ax1.axhline(0, color='red', linestyle='--')
ax1.set_title('Residuals vs. Fitted (SLR)')
ax1.set_xlabel('Fitted Values (Predicted MPG)')
ax1.set_ylabel('Residuals')

# 2. Q-Q Plot
ax2 = plt.subplot(1, 2, 2)
sm.qqplot(residuals_slr, line='s', ax=ax2) # 's' = standardized line
ax2.set_title('Q-Q Plot of Residuals (SLR)')

plt.tight_layout()
plt.show()
```



SLR Diagnostic Analysis

1. **Residuals vs. Fitted (Left):** This is a **problem**. The plot shows a clear **"U-shape"** (a non-linear pattern). This means our assumption of a *linear* relationship is wrong. Our model is **underfitting**.
2. **Q-Q Plot (Right):** The residuals also peel away from the line at the tails, suggesting the errors are not perfectly normally distributed.

Conclusion: The SLR model is simple and significant, but it's **fundamentally flawed** because it fails the linearity assumption. We need a more complex model.

Multiple Linear Regression - The "Full" Model

Let's build a "full" model that includes all potential predictors. This will be our starting point for backward elimination.

Our formula will be: `mpg ~ cylinders + displacement + horsepower + weight + acceleration + model_year + C(origin)`

Handling Categorical Variables: `C(origin)`

The `origin` column is categorical (USA, Europe, Japan). If we just add `origin` to the model, `statsmodels` will treat it as a number, which is wrong.

By wrapping it in `C()`, we tell `statsmodels` to treat it as a **C**ategorical variable. It will automatically:

1. Choose a "reference" level (by default, the lowest one, which in our case is Europe).
2. Create "dummy variables" for the other levels.

The output will show coefficients for `C(origin)[T.japan]` and `C(origin)[T.usa]`. These coefficients represent the *difference in mpg* compared to the Europe, holding all other variables constant.

```
# Define and fit the "Full" MLR model
all_features = 'mpg ~ cylinders + displacement + horsepower + weight + acceleration +
        model_year + C(origin)'
model_full = smf.ols(all_features, data=mpg_clean).fit()

# Print the full summary table
print(model_full.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.824
Model:                            OLS   Adj. R-squared:                  0.821
Method:                 Least Squares   F-statistic:                     224.5
Date:                Sat, 26 Sep 2026   Prob (F-statistic):          1.79e-139
Time:                        16:16:04   Log-Likelihood:                -1020.5
No. Observations:                 392   AIC:                             2059.
Df Residuals:                     383   BIC:                             2095.
Df Model:                           8
Covariance Type:            nonrobust
================================================================================
                   coef    std err          t      P>|t|      [0.025      0.975]
--------------------------------------------------------------------------------
Intercept      -15.3246      4.602     -3.330      0.001     -24.374      -6.276
C(origin)[T.japan]     0.2232      0.566      0.394      0.694      -0.890      1.336
C(origin)[T.usa]      -2.6300      0.566     -4.643      0.000      -3.744      -1.516
cylinders      -0.4897      0.321     -1.524      0.128      -1.121      0.142
displacement      0.0240      0.008      3.133      0.002       0.009      0.039
horsepower     -0.0182      0.014     -1.326      0.185      -0.045      0.009
weight         -0.0067      0.001    -10.243      0.000      -0.008      -0.005
acceleration      0.0791      0.098      0.805      0.421      -0.114      0.272
model_year     0.7770      0.052     15.005      0.000       0.675      0.879
==============================================================================
Omnibus:                       23.395   Durbin-Watson:                   1.291
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               34.452
Skew:                           0.444   Prob(JB):                     3.30e-08
Kurtosis:                       4.150   Cond. No.                     8.56e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 8.56e+04. This might indicate that there are strong multicollinearity or other numerical problems.
```

Interpreting the Full Model

- `Adj. R-squared`: 0.818. A massive improvement! Our model now explains 81.8% of the variance.
- **Multicollinearity Warning:** `statsmodels` warns us: `The condition number is large... This might indicate that there is strong multicollinearity`.
- **P-Values:** Look at the `P>|t|` column. We see several variables with high p-values (>0.05):
  - `C(origin)[T.japan]` (p=0.694)
  - `cylinders` (p=0.128)
  - `horsepower` (p=0.185)
  - `acceleration` (p=0.421)

These variables are not statistically significant *in this specific model*. This is due to **multicollinearity**. `cylinders`, `horsepower`, and `acceleration` are all measuring the same thing (engine power) and are redundant.

Backward Elimination

Our full model is bloated and unreliable. We will use **Backward Elimination** to simplify it, creating a "parsimonious" model that is both accurate and interpretable.

The Process:

1. Fit the full model.
2. Find the predictor with the **highest p-value** > 0.05.
3. Remove that predictor.
4. Refit the model and go back to step 2.
5. Stop when all remaining predictors have $p < 0.05$.

As a first step, we should remove `C(origin)[T.japan]` (p=0.694). However, this is one of the levels of `origin`, so we cannot just remove it. Instead, we can specify a different reference level than the default one to see if this improves things:

```
# Define and fit the "Full" MLR model
all_features = 'mpg ~ cylinders + displacement + horsepower + weight + acceleration +
        model_year + C(origin, Treatment(reference="usa"))'
model_full = smf.ols(all_features, data=mpg_clean).fit()

# Print the full summary table
print(model_full.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.824
Model:                            OLS   Adj. R-squared:                  0.821
Method:                 Least Squares   F-statistic:                     224.5
Date:                Sat, 26 Sep 2026   Prob (F-statistic):          1.79e-139
Time:                        16:16:04   Log-Likelihood:                -1020.5
No. Observations:                 392   AIC:                             2059.
Df Residuals:                     383   BIC:                             2095.
Df Model:                           8
Covariance Type:            nonrobust
================================================================================
================================
                                                   coef    std err          t
P>|t|      [0.025      0.975]
--------------------------------------------------------------------------------
--------------------------------
Intercept                                       -17.9546      4.677     -3.839
0.000     -27.150      -8.759
C(origin, Treatment(reference="usa"))[T.europe]     2.6300      0.566      4.643
0.000       1.516       3.744
C(origin, Treatment(reference="usa"))[T.japan]      2.8532      0.553      5.162
0.000       1.766       3.940
cylinders                                       -0.4897      0.321     -1.524
0.128      -1.121       0.142
displacement                                     0.0240      0.008      3.133
0.002       0.009       0.039
horsepower                                      -0.0182      0.014     -1.326
0.185      -0.045       0.009
weight                                          -0.0067      0.001    -10.243
0.000      -0.008      -0.005
acceleration                                     0.0791      0.098      0.805
0.421      -0.114       0.272
model_year                                       0.7770      0.052     15.005
0.000       0.675       0.879
==============================================================================
Omnibus:                       23.395   Durbin-Watson:                   1.291
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               34.452
Skew:                           0.444   Prob(JB):                     3.30e-08
Kurtosis:                       4.150   Cond. No.                     8.70e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
[2] The condition number is large, 8.7e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Great, this solved the problem for `origin`! We note that we have the same adjusted R-squared value, so we did not harm or improve the regressor.

We now have the following high p-values:

- `cylinders` (p=0.128)
- `horsepower` (p=0.185)
- `acceleration` (p=0.421)

Let's remove `acceleration` next:

```
# Define and fit the "Full" MLR model
all_features = 'mpg ~ cylinders + displacement + horsepower + weight + model_year +
        C(origin, Treatment(reference="usa"))'
model_full = smf.ols(all_features, data=mpg_clean).fit()

# Print the full summary table
print(model_full.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.824
Model:                            OLS   Adj. R-squared:                  0.821
Method:                 Least Squares   F-statistic:                     256.7
Date:                Sat, 26 Sep 2026   Prob (F-statistic):          1.49e-140
Time:                        16:16:04   Log-Likelihood:                -1020.8
No. Observations:                 392   AIC:                             2058.
Df Residuals:                     384   BIC:                             2089.
Df Model:                           7
Covariance Type:            nonrobust
================================================================================
================================
                                                   coef    std err          t
P>|t|      [0.025      0.975]
--------------------------------------------------------------------------------
--------------------------------
Intercept                                       -16.3323      4.219     -3.871
0.000     -24.628      -8.037
C(origin, Treatment(reference="usa"))[T.europe]     2.6345      0.566      4.654
0.000       1.521       3.748
C(origin, Treatment(reference="usa"))[T.japan]      2.8574      0.552      5.172
0.000       1.771       3.944
cylinders                                       -0.5028      0.321     -1.568
0.118      -1.133       0.128
displacement                                     0.0234      0.008      3.070
0.002       0.008       0.038
horsepower                                      -0.0250      0.011     -2.320
0.021      -0.046      -0.004
weight                                          -0.0065      0.001    -11.209
0.000      -0.008      -0.005
model_year                                       0.7739      0.052     14.994
0.000       0.672       0.875
==============================================================================
Omnibus:                       25.943   Durbin-Watson:                   1.290
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               39.879
Skew:                           0.468   Prob(JB):                     2.19e-09
Kurtosis:                       4.251   Cond. No.                     7.86e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
[2] The condition number is large, 7.86e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

The adjusted R-squared is the same, but we now have the following high p-values:

- `cylinders` (p=0.118)

Let's remove `cylinders`:

```
# Define and fit the "Full" MLR model
all_features = 'mpg ~  displacement + horsepower + weight + model_year + C(origin,
        Treatment(reference="usa"))'
model_full = smf.ols(all_features, data=mpg_clean).fit()

# Print the full summary table
print(model_full.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.823
Model:                            OLS   Adj. R-squared:                  0.820
Method:                 Least Squares   F-statistic:                     297.9
Date:                Sat, 26 Sep 2026   Prob (F-statistic):          2.80e-141
Time:                        16:16:04   Log-Likelihood:                -1022.0
No. Observations:                 392   AIC:                             2058.
Df Residuals:                     385   BIC:                             2086.
Df Model:                           6
Covariance Type:            nonrobust
================================================================================
================================
                                                 coef    std err          t
P>|t|      [0.025      0.975]
--------------------------------------------------------------------------------
--------------------------------
Intercept                                    -17.5036      4.160     -4.207
0.000     -25.683      -9.324
C(origin, Treatment(reference="usa"))[T.europe]     2.5958      0.567      4.581
0.000       1.482       3.710
C(origin, Treatment(reference="usa"))[T.japan]      2.7722      0.551      5.033
0.000       1.689       3.855
displacement                                 0.0155      0.006      2.699
0.007       0.004       0.027
horsepower                                  -0.0230      0.011     -2.149
0.032      -0.044      -0.002
weight                                      -0.0066      0.001    -11.449
0.000      -0.008      -0.005
model_year                                   0.7749      0.052     14.986
0.000       0.673       0.877
==============================================================================
Omnibus:                       24.729   Durbin-Watson:                   1.275
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               37.452
Skew:                           0.455   Prob(JB):                     7.37e-09
Kurtosis:                       4.211   Cond. No.                     7.73e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
[2] The condition number is large, 7.73e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

The adjusted R-squared is a bit lower, but all p-values are below the threshold now!

Final Model Interpretation

Now we have a **final, parsimonious model**.

- `Adj. R-squared`: 0.820. We removed two variables and our model's explanatory power is *still* just as high.
- **All p-values are < 0.05**. We can confidently interpret every coefficient.

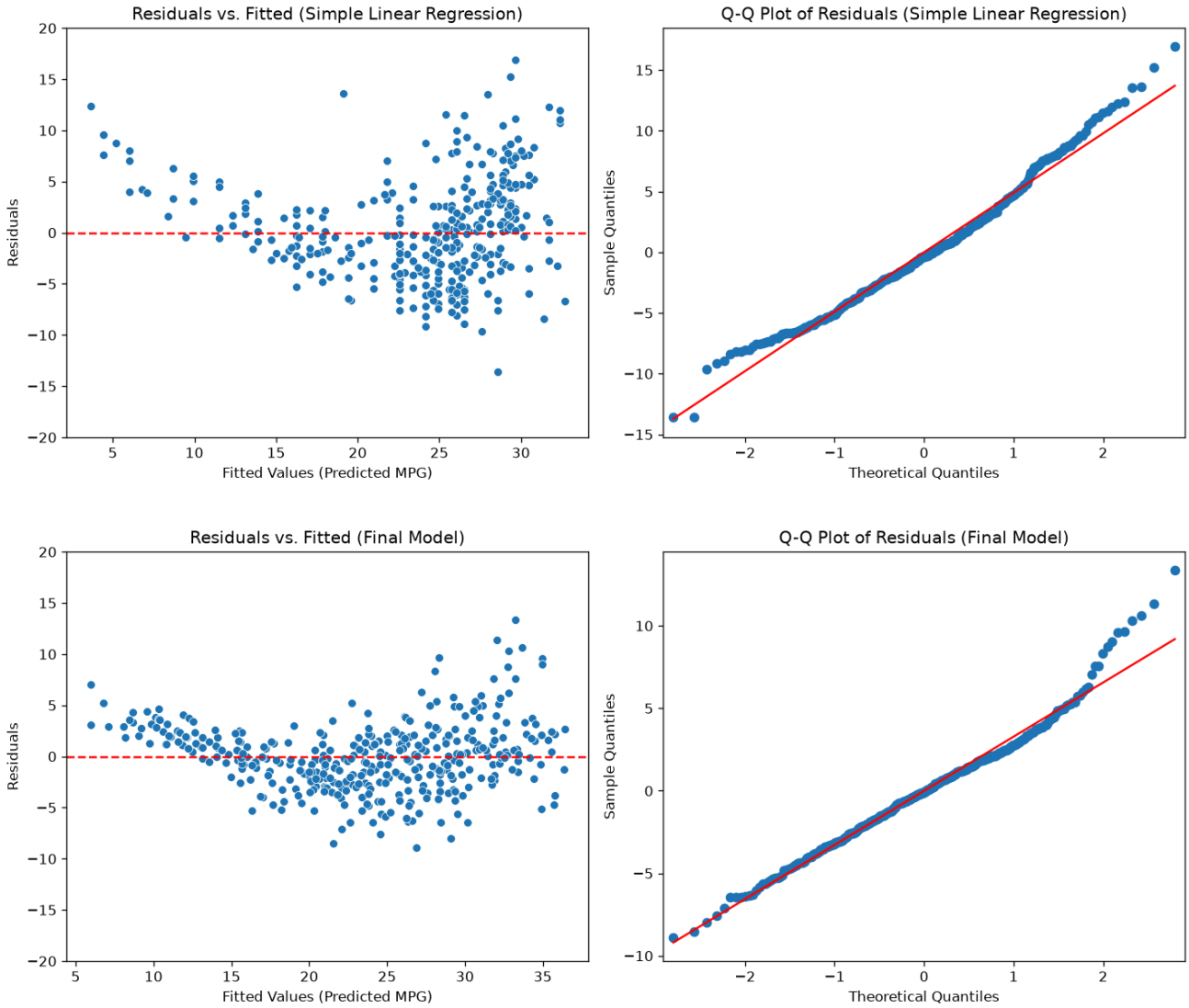
Final Coefficient Analysis:

- `weight`: -0.0066. "For every 1lb increase in `weight`, `mpg` decreases by 0.0066, holding all other variables constant. (This is the net effect of increasing weight).
- `model_year`: 0.7749. "For each newer `model_year`, `mpg` increases by 0.78, holding all other variables constant. (Cars get more efficient over time).
- `C(origin)[T.europe]`: 2.5958. "A car from `Europe` gets **2.6 more mpg** than a car from the `USA`, holding all other variables constant".
- `C(origin)[T.japan]`: 2.7722. "A car from `Japan` gets **2.77 more mpg** than a car from the `USA`, holding all other variables constant".

Final Diagnostic Check

Our p-values and coefficients are only reliable if this *final* model is valid. Let's check the residuals.

```
# --- SLR Diagnostic Plots ---
fitted_values_slr = model_slr.fittedvalues
residuals_slr = model_slr.resid

plt.figure(figsize=(12, 5))

# 1. Residuals vs. Fitted Plot
ax1 = plt.subplot(1, 2, 1)
sns.scatterplot(x=fitted_values_slr, y=residuals_slr, ax=ax1)
ax1.axhline(0, color='red', linestyle='--')
ax1.set_title('Residuals vs. Fitted (Simple Linear Regression)')
ax1.set_xlabel('Fitted Values (Predicted MPG)')
ax1.set_ylabel('Residuals')
ax1.set_ylim(-20, 20)

# 2. Q-Q Plot
ax2 = plt.subplot(1, 2, 2)
sm.qqplot(residuals_slr, line='s', ax=ax2) # 's' = standardized line
ax2.set_title('Q-Q Plot of Residuals (Simple Linear Regression)')

plt.tight_layout()
plt.show()

# --- Final Model Diagnostic Plots ---
fitted_values_final = model_full.fittedvalues
residuals_final = model_full.resid

plt.figure(figsize=(12, 5))

# 1. Residuals vs. Fitted Plot
ax1 = plt.subplot(1, 2, 1)
sns.scatterplot(x=fitted_values_final, y=residuals_final, ax=ax1)
ax1.axhline(0, color='red', linestyle='--')
ax1.set_title('Residuals vs. Fitted (Final Model)')
ax1.set_xlabel('Fitted Values (Predicted MPG)')
ax1.set_ylabel('Residuals')
ax1.set_ylim(-20, 20)

# 2. Q-Q Plot
ax2 = plt.subplot(1, 2, 2)
sm.qqplot(residuals_final, line='s', ax=ax2)
ax2.set_title('Q-Q Plot of Residuals (Final Model)')


plt.tight_layout()
plt.show()
```



1. **Residuals vs. Fitted (Left):** This is a bit better. The problematic "U-shape" is still there but less pronounced.
2. **Q-Q Plot (Right):** This also looks a bit improved, with the points falling closer to the normal-distribution line in the left part of the plot.

Exercises

Exercise 1

Consider the Titanic dataset. Calculate a linear regressor that predicts `Fare` values from `Survived`, `Pclass`, `Sex`, and `Age` values. Insert dummy variables where appropriate. Is the obtained regressor a good regressor? Which variables contribute significantly to the regression? Are there irrelevant variables? Eliminate these variables using the backward elimination technique. Discuss the meaning of the identified coefficients.

Exercise 2

Consider the Titanic dataset. Calculate a linear regressor that predicts `Age` values from `Survived`, `Pclass`, `Sex`, and `Fare` values. Insert dummy variables where appropriate. Is the obtained regressor a good regressor? Is it a better or worse regressor than the one calculated in the previous exercise? Which variables contribute significantly to the regression? Are there irrelevant variables? Eliminate such variables using the backward elimination technique. Discuss the meaning of the identified coefficients.

References

- Chapter 3 of [1]
- Parts of chapter 11 of [2]

[1] Heumann, Christian, and Michael Schomaker Shalabh. Introduction to statistics and data analysis. Springer International Publishing Switzerland, 2016.

[2] James, Gareth Gareth Michael. An introduction to statistical learning: with applications in Python, 2023.https://www.statlearning.com

© 2026 Antonino Furnari · Department of Mathematics and Computer Science, University of Catania Built with Quarto

Credits and data sources

Logistic regression

Chapter resources

Slides (PDF) Open in Colab Practice quiz

In the previous chapters we used the linear regression to model a **continuous dependent variable** as a function of the independent variables:

$$f(\mathbf{x}) = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$$

This is a **parametric** model: once we have estimated the β_i coefficients, we can discard the training data and use the model to make predictions on new examples in a very natural way. Linear regression allows to model relationships between **continuous independent and dependent variables** and **between qualitative independent variables and continuous variables**. However, it does not allow to model relationships between continuous or qualitative independent variables and **qualitative dependent variables**.

In this chapter we will see how to extend the linear model to the case of a qualitative (binary) dependent variable.

Example Data

Establishing such relationships is useful in different contexts. For instance, let us consider the Breast Cancer Wisconsin dataset:

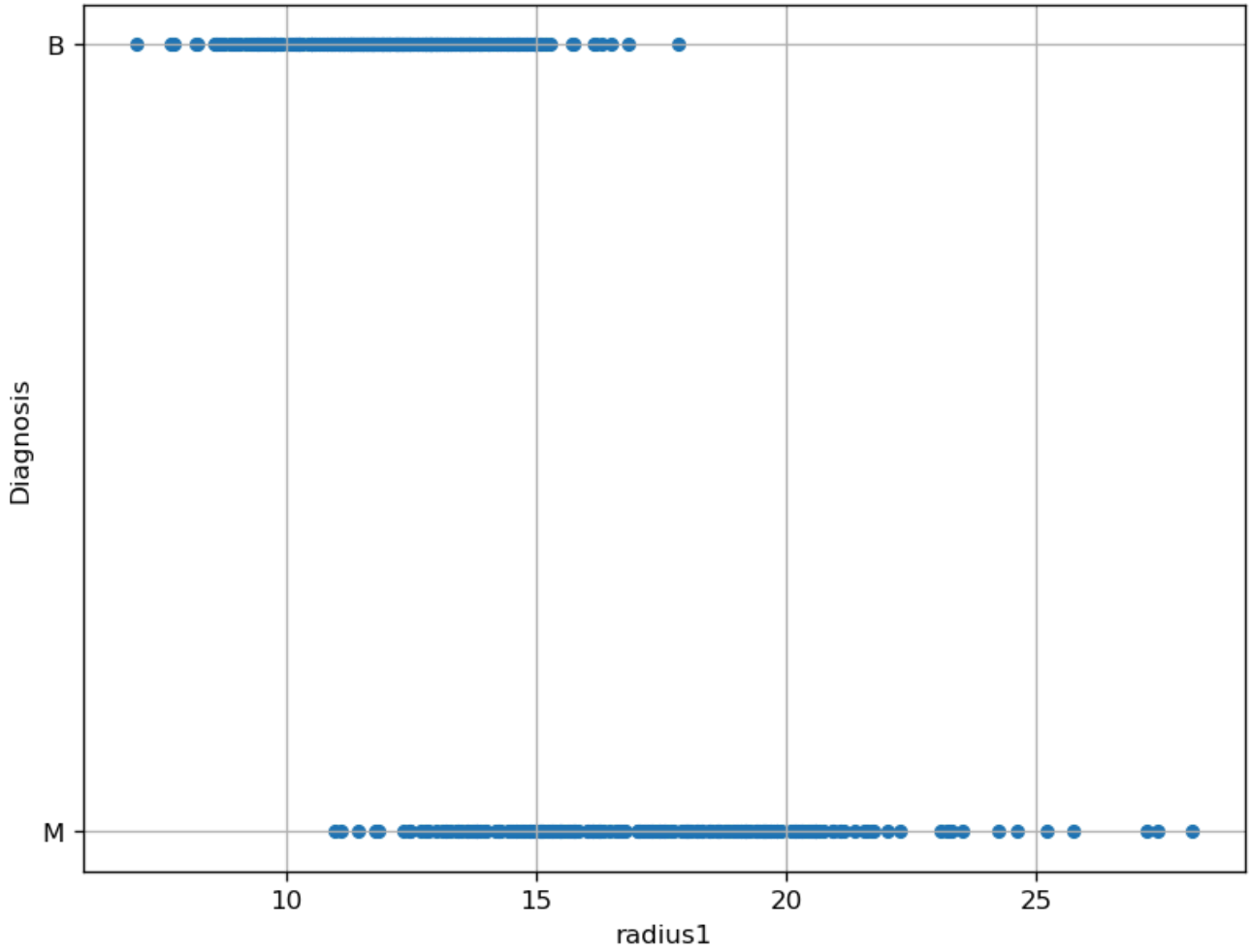
	radius1	texture1	perimeter1	area1	smoothness1	compactness1	concavity1	concave_points1	symmetr
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	0.2419
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	0.1812
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	0.2069
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	0.2597
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	0.1809
...	...	...	...	...	...	...	...	...	...
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	0.1726
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	0.1752
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	0.1590
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	0.2397
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	0.1587

569 rows × 31 columns

The dataset contains several measurements of given quantities measured from digitized image of a fine needle aspirate (FNA) of a breast mass, together with a categorical variable `Diagnosis` with two levels: `M` (malignant) and `B` (benign).

In this case, it would be good to be able to study whether a relationship exists between some of the considered independent variables and the dependent variable or simply to classify an example in one of the two classes from new observation.

We will consider the `radius1` variable for the moment. Let us plot this variable with respect to `Diagnosis`:



From the plot above, we can note that there is some form of relationship between the two variables. Indeed:

- For low values of `radius1`, we tend to have more benign cases;
- For large values of `radius1`, we tend to have more malignant cases.

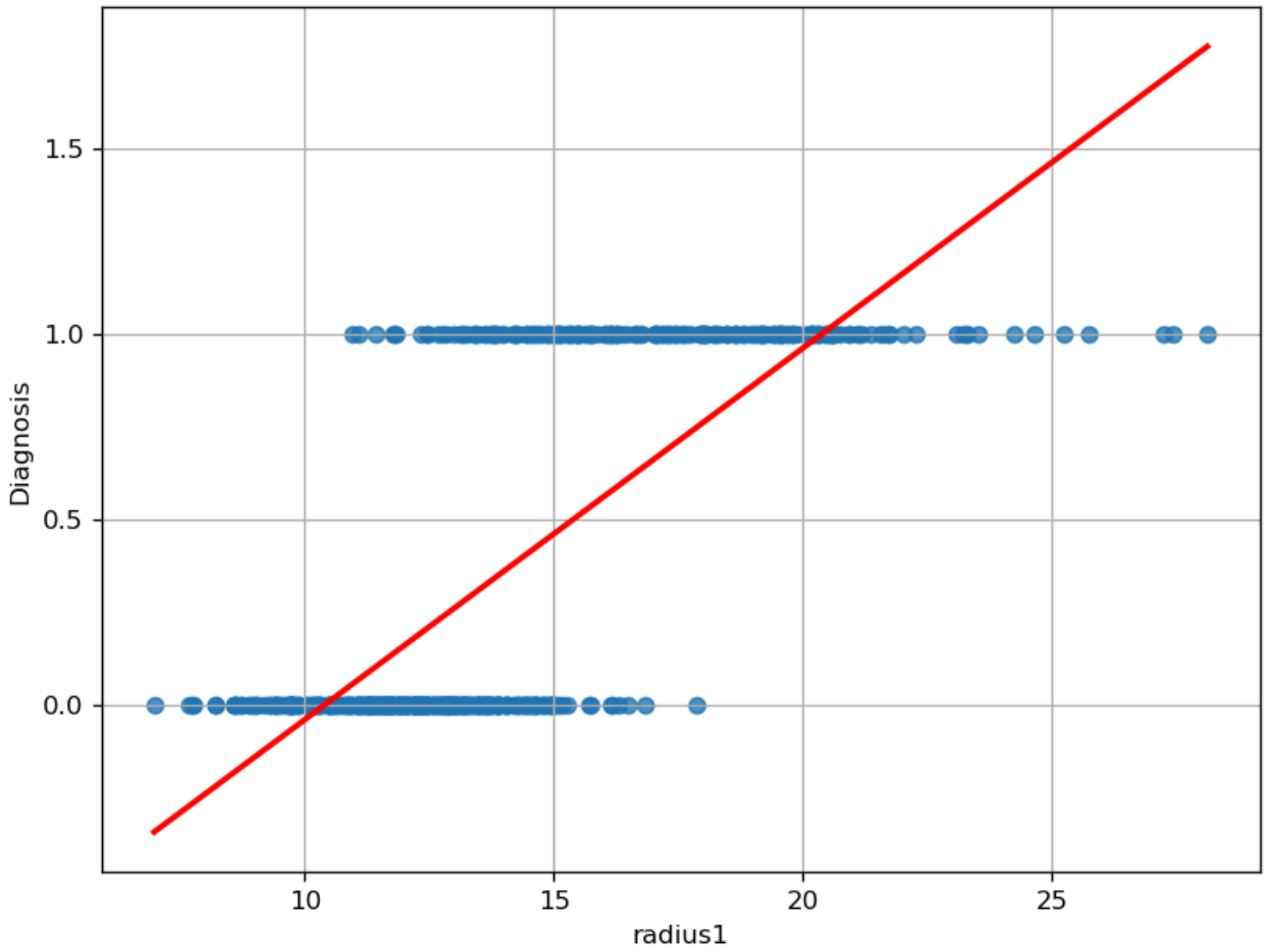
Limits of Linear Regression

Of course, we would like to quantify this relationship in a more formal way. **As in the case of a linear regressor, we want to define a model which can predict the independent variable y from the dependent variables x_i. If such model gives good predictions, than we can trust its interpretation as a means of studying the relationship between the variables.**

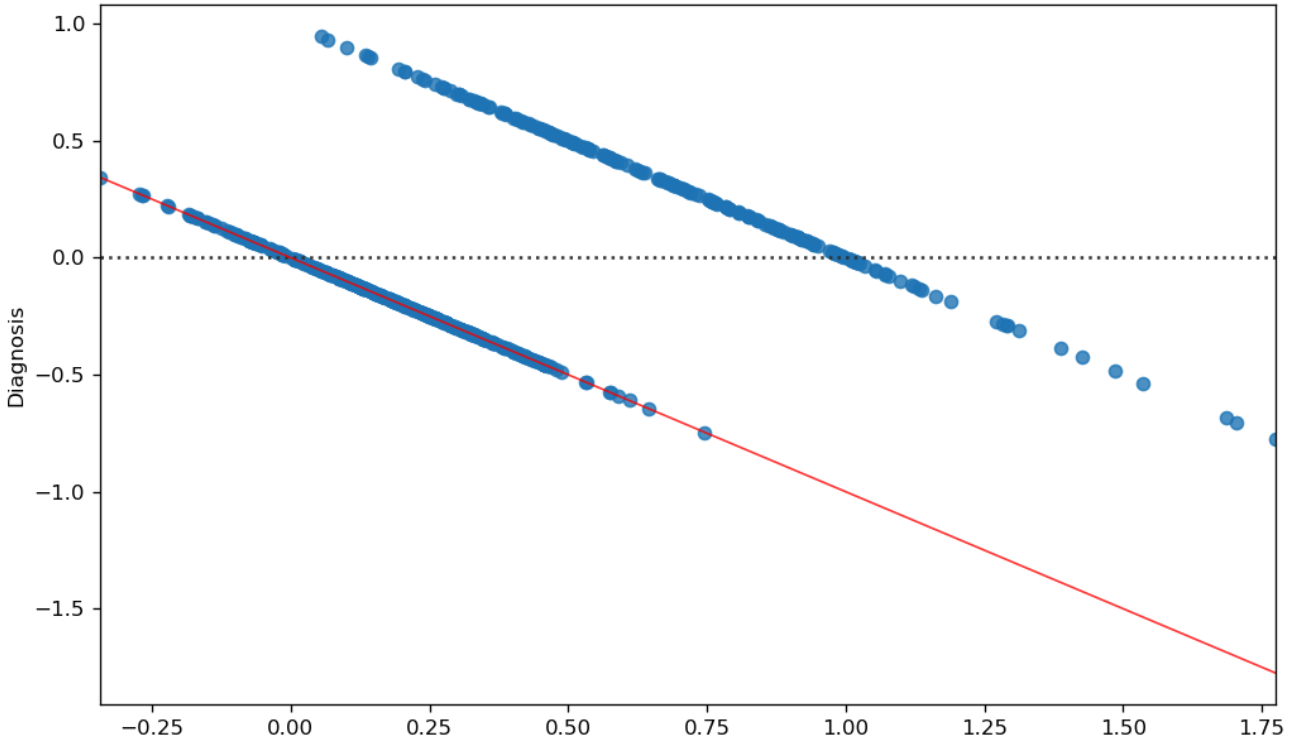
We can think of converting `B ⇒ 1` and `M ⇒ 0`, and then compute a linear regressor:

$$Diagnosis = \beta_0 + \beta_1 radius1$$

This would be the result:



We can immediately see that this function does not model the relationship between the two variables very well. While we obtain a statistically relevant regressor with $R^2 = 0.533$ and statistically relevant coefficients, the residual plot will look like this:



The correlation between the residuals and the independent variable is a strong indication that the true relationship between the two variables is not correctly modeled. After all, from a purely predictive point of view, we are using a linear regressor which takes the form:

$$f : \mathbb{R} \rightarrow \mathbb{R}$$

while the values `Diagnosis` variable belong to the set $\{0, 1\}$ and we would need instead a function with the following form:

$$f : \mathbb{R} \rightarrow \{0, 1\}$$

However, the linear regressor cannot directly predict **discrete values**.

In practice, while with a linear regressor we wanted to predict continuous values, now we want to assign observations $\mathbf{x}$ to discrete bins (in this case only two possible ones). As we will better study later in the course, this problem is known as classification.

From Binary Values to Probabilities

If we want to model some form of continuous value, we could think to transition from $\{0, 1\}$ to $[0, 1]$ using probabilities, which is a way to turn discretized values to "soft" values indicating our belief in the fact that `Diagnosis` will take either a 0 or 1 value. We could hence think to model the following probability, rather than modeling `Diagnosis` directly:

$$P(Diagnosis = 1 | radius1)$$

Note that modeling that probability directly is what **discriminative models do**.

However, even in this case, a model of the form:

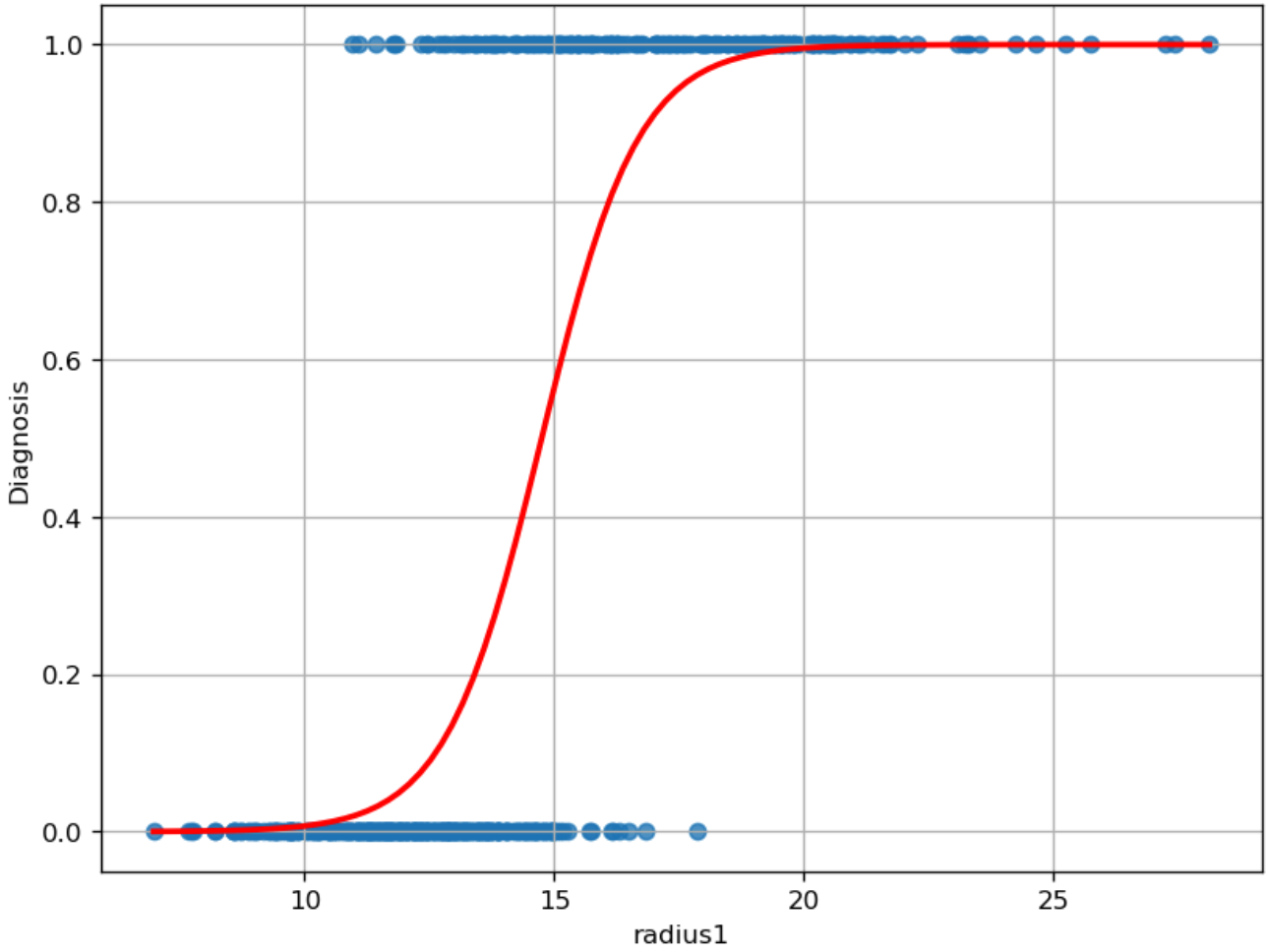
$$P(Diagnosis = 1 | radius1) = \beta_0 + \beta_1 radius1$$

Would not be appropriate. Indeed, while $P(Diagnosis = 1 | radius1)$ needs to be in the $[0, 1]$ range, the linear combination $\beta_0 + \beta_1 radius1$ will naturally output values **smaller than** 0 and **larger than** 1. How should we interpret such values?

Intuitively, we would expect to $P(Diagnosis = 1 | radius1)$ to assume values in the $[0, 1]$ range for intermediate values (say `radius` $\in [10, 20]$), while for extremely low values of (say `radius` < 10) the probability **should saturate to** 0 and for extremely large values (say `radius` > 20) the probability should saturate to 1.

When `radius1` takes large values (say larger than 20), we expect **probability to saturate to** 1.

In practice, we would expect a result similar to the following:



As can be noted, the function above is not linear, and hence it cannot be fit with a linear regressor. However, we have seen that a linear regressor can be tweaked to also represent nonlinear functions.

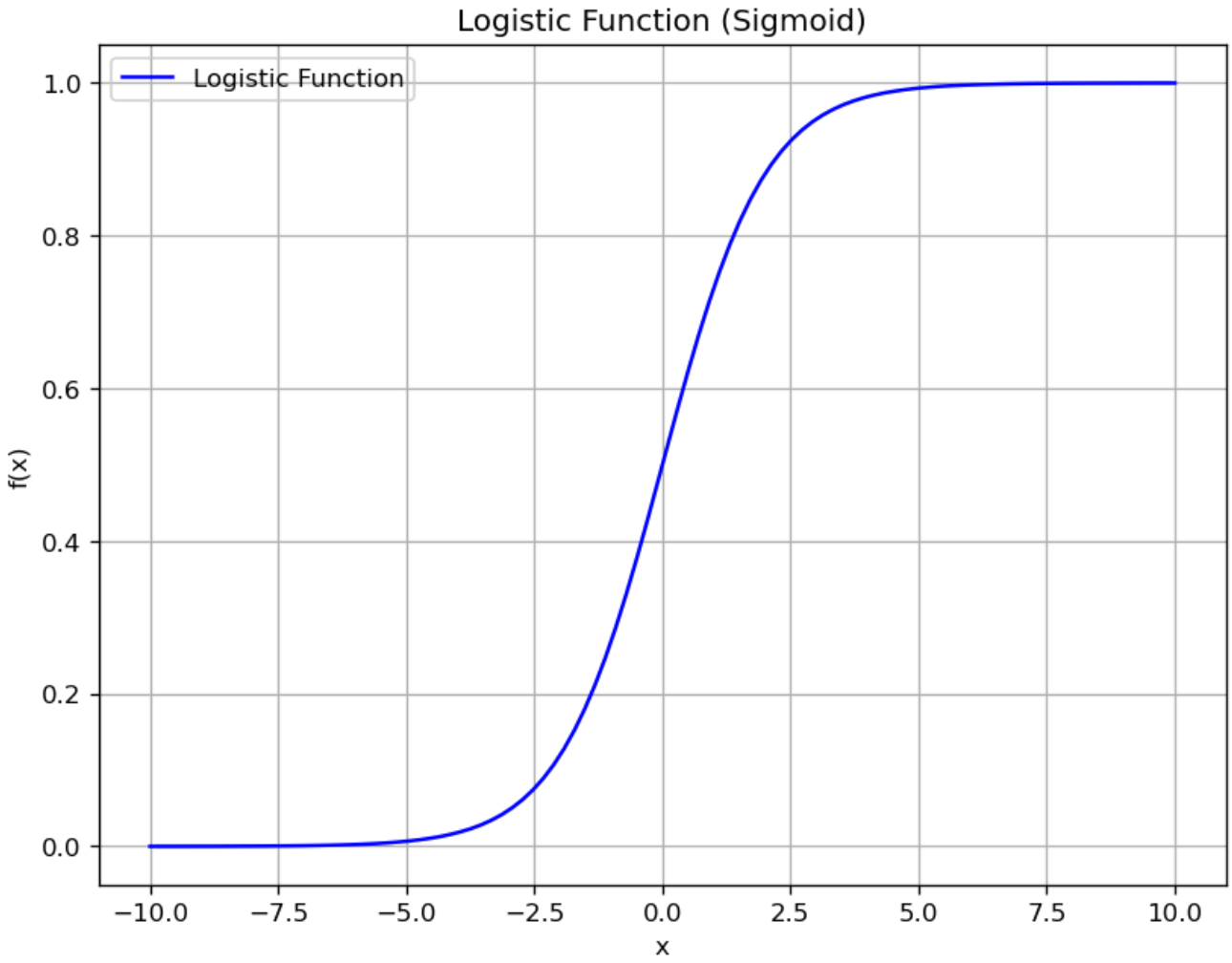
The Logistic Function

We need to find a **transformation of the formulation of the linear regressor to transform its output into a nonlinear function of the independent variables**. Of course, we do not want *any* transformation, but one that has the previously highlighted properties.

In practice the **logistic function has some nice properties that, as we will se in a moment, allow to easily interpret the resulting model in a probabilistic way**. The logistic function is defined as:

$$f(x) = \frac{1}{1 + e^{-x}}$$

and has the following shape:



As we can see, the function has the properties we need:

- Its values are comprised between 0 and 1;
- It saturates to 0 and 1 for extreme values of x.

Additionally, it is differentiable, which will be useful for optimization later.

The Logistic Regression Model

In practice, we define our model, **the logistic regressor model** as follows (**simple logistic regression**):

$$P(Diagnosis = 1|X) = f(\beta_0 + \beta_1 X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

Or, more in general (**multiple logistic regression**):

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n)}}$$

It is easy to see that:

$$p = \frac{1}{1+e^{-x}} \Rightarrow p + pe^{-x} = 1 \Rightarrow pe^{-x} = 1 - p \Rightarrow e^{-x} = \frac{1-p}{p} \Rightarrow e^{x} = \frac{p}{1-p}$$

Hence:

$$e^{\beta_0+\beta_1 x_1+\ldots+\beta_n x_n} = \frac{P(y=1|\mathbf{x})}{1-P(y=1|\mathbf{x})}$$

The term on the right is called the odd of $P(y=1|\mathbf{x})$.

The odd of $P(y=1|\mathbf{x})$ is the number of times we believe the example will be positive (observed $\mathbf{x}$) over the number of times we believe the example will be negative.

For instance, if we believe that the example will be positive 3 times out of 10, then the odd will be $\frac{3}{7}$.

By taking the logarithm of both terms, we obtain:

$$\log\left(\frac{P(y=1|\mathbf{x})}{1-P(y=1|\mathbf{x})}\right) = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$$

The expression:

$$\log\left(\frac{P(y=1|\mathbf{x})}{1-P(y=1|\mathbf{x})}\right)$$

Is the logarithm of the odd (log odd), and it is called **logit**, hence the **logistic regression** is sometimes called **logit regression**.

The expression above shows how a logistic regressor can be seen as **a linear regressor (the expression on the right side of the equation) on the logit (the log odd)**. This paves the way to useful interpretations of the model, as shown in the next section.

Estimation of the Parameters of a Logistic Regressor

To fit the model and find suitable values for the $\beta_\mathbf{i}$ parameters, we will define a **cost function**, similarly to what we have done in the case of linear regression.

Even if we can see the logistic regression problem as the linear regression problem of fitting the $logit(p) = \beta^T\mathbf{x}$ function, differently from linear regression, **we should note that we do not have the ground truth probabilities p**. Indeed, our observations only provide input examples $\mathbf{x}^{(i)}$ and the corresponding labels $y^{(i)}$.

Starting from the definition:

$$P\left(y=1|\mathbf{x};\beta\right)=f_{\beta}\left(\mathbf{x}\right)=\frac{1}{1+e^{-\beta^{T}\mathbf{x}}}=\sigma(\beta^{T}\mathbf{x})$$

We can write:

$$P\left(y=1|\mathbf{x};\beta\right)=f_{\beta}(\mathbf{x})$$

$$P\left(y=0|\mathbf{x};\beta\right)=1-f_{\beta}(\mathbf{x})$$

Since y can only take values 0 and 1, this can also be written as follows in a more compact form:

$$P\left(y|\mathbf{x};\beta\right)=\left(f_{\beta}\left(\mathbf{x}\right)\right)^{y}\left(1-f_{\beta}\left(\mathbf{x}\right)\right)^{1-y}$$

Indeed, when $y=1$, the second factor is equal to 1 and the expression reduces to $P\left(y=1|\mathbf{x};\beta\right)=f_{\beta}(\mathbf{x})$. Similarly, if $y=0$, the first factor is equal to 1 and the expression reduces to $1-f_{\beta}(x)$.

We can estimate the parameters by maximum likelihood, i.e., choosing the values of the parameters which maximize the probability of the data under the model identified by the parameters β:

$$L\left(\beta\right)=P(Y|X;\beta)$$

If we assume that the training examples are all independent, the likelihood can be expressed as:

$$L\left(\beta\right)=\prod_{i=1}^{N}P(y^{(i)}|\mathbf{x}^{(i)};\beta)=\prod_{i=1}^{N}f_{\beta}\left(\mathbf{x}^{(i)}\right)^{y^{(i)}}\left(1-f_{\beta}\left(\mathbf{x}^{(i)}\right)\right)^{1-y^{(i)}}$$

Maximizing this expression is equivalent to minimizing the negative logarithm of $L(\beta)$ (negative log-likelihood - nll):

$$nll\left(\beta\right)=-\log L\left(\beta\right)=-\sum_{i=1}^{N}\log\left[f_{\beta}\left(\mathbf{x}^{(i)}\right)^{y^{(i)}}\left(1-f_{\beta}\left(\mathbf{x}^{(i)}\right)\right)^{1-y^{(i)}}\right]=$$

$$=-\sum_{i=1}^{N}\left[y^{(i)}\log f_{\beta}\left(\mathbf{x}^{(i)}\right)+\left(1-y^{(i)}\right)\log\left(1-f_{\beta}\left(\mathbf{x}^{(i)}\right)\right)\right]$$

Hence, we will define our cost function as:

$$J\left(\beta\right)=-\sum_{i=1}^{N}\left[y^{(i)}\log f_{\beta}\left(\mathbf{x}^{(i)}\right)+\left(1-y^{(i)}\right)\log\left(1-f_{\beta}\left(\mathbf{x}^{(i)}\right)\right)\right]$$

This can be rewritten more explicitly in terms of the β parameters as follows:

$$J(\beta) = -\sum_{i=1}^{N} [y^{(i)} \log \sigma\left(\beta^T \mathbf{x}^{(i)}\right) + \left(1 - y^{(i)}\right) \log\left(1 - \sigma\left(\beta^T \mathbf{x}^{(i)}\right)\right)]$$

Similarly to linear regression, we now have a cost function to minimize in order to find the values of the β_i parameters. Unfortunately, in this case, $J(\beta)$ assumes a nonlinear form **which prevents us to use the least square principles** and **there is no closed form solution for the parameter estimation**. In these cases, parameters can be estimated using some form of **iterative solver**, which begins with an initial guess for the parameters and iteratively refine them to find the final solution. Luckily, the logistic regression cost function **is convex, and hence only a single solution is admitted, independently from the initial guess**.

Different iterative solvers can be used in practice. The most commonly used is the **gradient descent algorithm**, which requires the cost function to be differentiable.

Visualizing the Cost Function (The “Loss Landscape”)

We have just derived the cost function $J(\beta)$, also known as **Log-Loss** or **Binary Cross-Entropy**.

$$J(\beta) = -\sum_{i=1}^{N} [y^{(i)} \log \sigma\left(\beta^T \mathbf{x}^{(i)}\right) + \left(1 - y^{(i)}\right) \log\left(1 - \sigma\left(\beta^T \mathbf{x}^{(i)}\right)\right)]$$

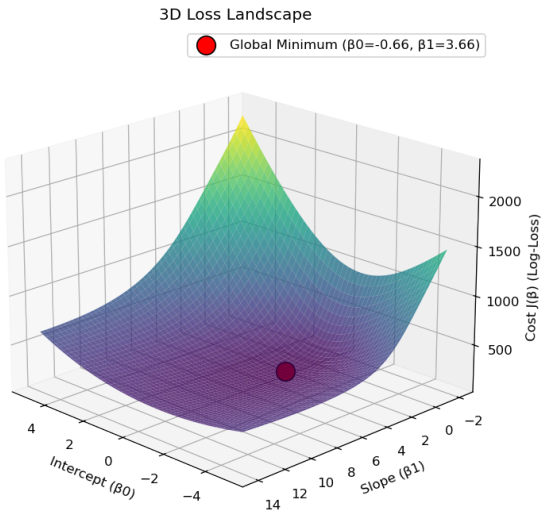
You were told that this function is **convex**, which guarantees a single global minimum. But what does that actually *look like*?

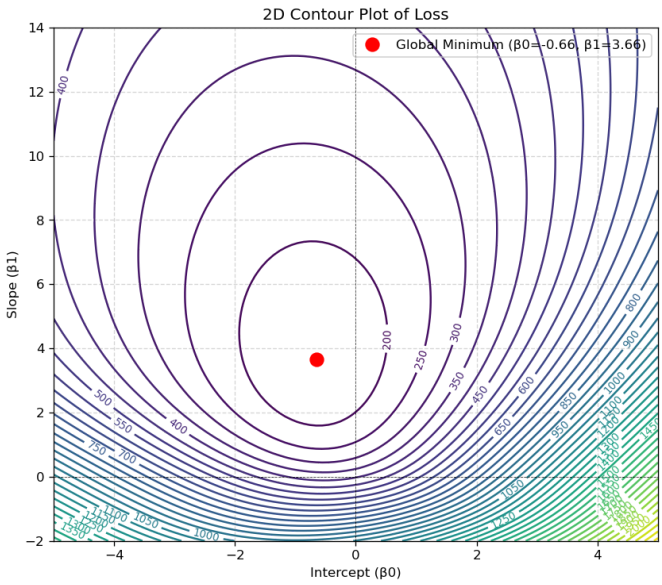
Let’s visualize it. We will build a simple model `Diagnosis ~ radius1`, which has two parameters: β_0 (intercept) and β_1 (slope). We will then plot the Cost J for every possible combination of β_0 and β_1. We are looking for a “bowl” shape.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from mpl_toolkits.mplot3d import Axes3D # Moved import to top

# --- 1. Load and Prepare Data ---
data = pd.read_csv(DATA + "breast_cancer_wisconsin.csv")

# Map Diagnosis to 0 (Benign) and 1 (Malignant)
data['Diagnosis'] = data['Diagnosis'].replace({'B':0,'M':1})

# Extract our 1D feature and target
X = data[['radius1']].values
y = data['Diagnosis'].values

# --- 2. CRITICAL STEP: Scale the Feature ---
# This is vital for visualization. Without it, the "bowl"
# will be extremely stretched and impossible to see.
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

def compute_cost(X, y, beta_0, beta_1):
    """
    Calculates the total log-loss for a given beta_0 and beta_1.
    """
    # 1. Calculate the linear part (z)
    # --- THIS IS THE FIX ---
    # We .flatten() X to make it (n,) to match y's shape (n,)
    z = beta_0 + (X.flatten() * beta_1)
    # -----------------------

    # 2. Apply the sigmoid function to get probabilities (p_hat)
    p_hat = 1 / (1 + np.exp(-z)) # p_hat is now shape (n,)

    # --- Numerical Stability ---
    # We must "clip" values to prevent log(0) or log(1-1),
    # which would result in -inf.
    epsilon = 1e-10
    p_hat = np.clip(p_hat, epsilon, 1 - epsilon)
    # -------------------------

    # 3. Calculate the log-loss for each sample
    # y (n,) and p_hat (n,) will now multiply element-wise
    cost_per_sample = -(y * np.log(p_hat) + (1 - y) * np.log(1 - p_hat))

    # 4. Return the *total* cost for the whole dataset
```

```
    return np.sum(cost_per_sample)

# 1. Create ranges for Beta 0 (intercept) and Beta 1 (slope)
beta_0_range = np.linspace(-5, 5, 100)
# --- UPDATED as requested ---
beta_1_range = np.linspace(-2, 14, 100)
# ----------------------------

# 2. Create the 2D grid
B0, B1 = np.meshgrid(beta_0_range, beta_1_range)
costs = np.zeros_like(B0)

# 3. Calculate the cost for every point on the grid
for i in range(B0.shape[0]):
    for j in range(B0.shape[1]):
        costs[i, j] = compute_cost(X_scaled, y, B0[i, j], B1[i, j])

# Find the minimum for plotting
min_cost_idx = np.unravel_index(np.argmin(costs), costs.shape)
best_b0 = B0[min_cost_idx]
best_b1 = B1[min_cost_idx]
min_cost = costs[min_cost_idx]

# --- Create the Plot ---
fig = plt.figure(figsize=(18, 7))

# --- Subplot 1: 3D Surface Plot ---
ax1 = fig.add_subplot(1, 2, 1, projection='3d')
ax1.plot_surface(B0, B1, costs, cmap='viridis', alpha=0.8)
# --- UPDATED to show both optimal parameters ---
ax1.scatter(best_b0, best_b1, min_cost, color='red', s=200,
            edgecolor='black', label=f'Global Minimum (β0={best_b0:.2f}, β1=
        {best_b1:.2f})')
# -----------------------------------------------
ax1.set_xlabel('Intercept (β0)')
ax1.set_ylabel('Slope (β1)')
ax1.set_zlabel('Cost J(β) (Log-Loss)')
ax1.set_title('3D Loss Landscape')
ax1.view_init(elev=20, azim=135) # Adjust angle
ax1.legend()

# --- Subplot 2: 2D Contour Plot ---
ax2 = fig.add_subplot(1, 2, 2)
contours = ax2.contour(B0, B1, costs, levels=50, cmap='viridis')
plt.clabel(contours, inline=True, fontsize=8)

# --- UPDATED to show both optimal parameters ---
ax2.plot(best_b0, best_b1, 'ro', markersize=10,
         label=f'Global Minimum (β0={best_b0:.2f}, β1={best_b1:.2f})')
```

```
# ------------------------------------------
ax2.set_xlabel('Intercept (β0)')
ax2.set_ylabel('Slope (β1)')
ax2.set_title('2D Contour Plot of Loss')
ax2.axhline(0, color='k', ls='--', lw=0.5)
ax2.axvline(0, color='k', ls='--', lw=0.5)
ax2.legend()
ax2.grid(True, linestyle='--', alpha=0.5)

plt.show()

print(f"Optimal parameters:")
print(f"  β0 (Intercept): {best_b0:.4f}")
print(f"  β1 (Slope):       {best_b1:.4f}")
print(f"  Minimum Cost (Log-Loss): {min_cost:.4f}")
```






```
Optimal parameters:
  β0 (Intercept): -0.6566
  β1 (Slope):       3.6566
  Minimum Cost (Log-Loss): 165.0107
```

The plots clearly show a single, smooth, **convex "bowl"**.

- There are no "local minima" or "bumpy" areas for an optimizer to get stuck in.
- This is why we were told the logistic regression cost function **is convex**.
- The **red dot** at the very bottom of this bowl represents the *one* optimal set of parameters (β_0, β_1) that minimizes the cost.

An iterative solver like **Gradient Descent** is guaranteed to find this minimum, just like a marble released from anywhere on the bowl's rim is guaranteed to roll to the bottom.

Statistical Interpretation of the Coefficients of a Logistic Regressor

Let's now see how to interpret the coefficients of a logistic regressor.

Interpretation of the intercept β_0

Remember that the regression model (in the case of simple logistic regression) is as follows:

$$\log(\frac{p}{1-p}) = \beta_0 + \beta_1 x$$

Applying what we know about logistic regressors, we can write:

$$x = 0 \Rightarrow \log \frac{p}{1-p} = \beta_0$$

To have a clearer picture, we can exponentiate both sides of the equation and write:

$$x = 0 \Rightarrow \frac{p}{1-p} = e^{\beta_0}$$

Remember that $\frac{p}{1-p}$ is the odd that the dependent variable is equal to 1 when observing x and, as such, it has a clear interpretation. For example, if the odds of an event are $\frac{3}{1}$, then it is 3 times more likely to occur than not to occur. So, **for $x = 0$, it is e^{β_0} times more likely that the dependent variable is equal to 1, rather than being equal to 0**.

Interpretation of variable coefficients β_i

We know that:

$$odds(p|x) = \frac{P(y=1|x)}{1-P(y=1|x)}$$

We can write:

$$\log odds(p|x) = \beta_0 + \beta_1 x$$

Hence:

$$\log odds(p|x+1) - \log odds(p|x) = \beta_0 + \beta_1(x+1) - \beta_0 - \beta_1 x = \beta_1(x+1) - \beta_1 x = \beta_1$$

Exponentiating both sides, we get:

$$e^{\log odds(p|x+1)-\log odds(p|x)} = e^{\beta_1} \Rightarrow \frac{e^{\log odds(p|x+1)}}{e^{\log odds(p|x)}} = e^{\beta_1} \Rightarrow \frac{odds(p|x+1)}{odds(p|x)} = e^{\beta_1} \Rightarrow odds(p|x +$$

We can thus say that **increasing the variable x by one unit corresponds to a multiplicative increase in odds by e^{β_1}**.

This analysis can be easily extended to the case of a multiple logistic regressor. Hence in general, given the model:

$$P(y=1|\mathbf{x}) = \sigma(\beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n)$$

We can say that:

- e^{β_0} is the odd of y being equal to 1 rather than 0 when $x_i = 0,\ \forall i$;
- An increment of one unit in the independent variable x_i corresponds to a multiplicative increment of e^{β_i} in the odds of $y=1$. So if $e^{\beta_i} = 0.05$, then $y=1$ is 5% more likely for a one-unit increment of x.

Example of Logistic Regression

Let us now apply logistic regression to a larger set of variables in our regression problem. We will consider the following independent variables:

- `radius1`
- `texture1`
- `perimeter1`
- `area1`
- `smoothness1`
- `compactness1`
- `concavity1`
- `symmetry1`

The dependent variable is again `Diagnosis`.

Once fit to the data, we will obtain the following parameters:

R^2	Adj. R^2	F-statistic	Prob(F-statistic)	Log-Likelihood
0.670	0.666	142.4	1.16e-129	-78.055

All values have interpretations similar to the ones obtained in the case of linear regression. The Log-Likelihood reports the value of the logarithm of the likelihood which was used to train the data.

The estimates for the coefficients are as follows:

```
ols("Diagnosis ~ radius1 + texture1 + perimeter1 + area1 + smoothness1 + compactness1
        + concavity1 + symmetry1",data).fit().summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-2.6591	0.224	-11.896	0.000	-3.098	-2.220
radius1	0.4688	0.133	3.532	0.000	0.208	0.730
texture1	0.0219	0.003	7.376	0.000	0.016	0.028
perimeter1	-0.0473	0.021	-2.272	0.023	-0.088	-0.006
area1	-0.0009	0.000	-3.985	0.000	-0.001	-0.000
smoothness1	5.1389	1.221	4.208	0.000	2.740	7.538
compactness1	0.3080	0.854	0.360	0.719	-1.370	1.986
concavity1	2.0973	0.414	5.065	0.000	1.284	2.911
symmetry1	1.2739	0.568	2.244	0.025	0.159	2.389

We notice that not all variables have a statistically relevant relationship with the dependent variable. Applying backward elimination, we remove `compactness1` and obtain the following estimates:

```
ols("Diagnosis ~ radius1 + texture1 + perimeter1 + area1 + smoothness1 + concavity1
        symmetry1",data).fit().summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-2.6708	0.221	-12.086	0.000	-3.105	-2.237
radius1	0.4360	0.097	4.517	0.000	0.246	0.626
texture1	0.0219	0.003	7.405	0.000	0.016	0.028
perimeter1	-0.0419	0.014	-2.915	0.004	-0.070	-0.014
area1	-0.0010	0.000	-4.477	0.000	-0.001	-0.001
smoothness1	5.3093	1.125	4.719	0.000	3.099	7.519
concavity1	2.1479	0.389	5.517	0.000	1.383	2.913
symmetry1	1.3132	0.557	2.359	0.019	0.220	2.407

These are now all statistically relevant. For instance, we can see that:

- When all variables are set to zero, the odds of the benign tumor are $e^{-2.6708} \approx 0.07$, or $\frac{7}{100}$. This is a base value.
- An increment in one unit of `texture1` increments the odds of a benign tumor multiplicatively by a factor of $e^{0.0219} \approx 1.02$ (a $+2\%$), when all other variables are constant.
- An increment of one unit of `perimeter1` decrements the odds of benign tumor multiplicatively by a factor of $e^{-0.0419} \approx 0.96$ (a -4%), when all other variables are constant.

Evaluating the Logistic Regression Model

The `statsmodels` summary table gives us a lot of information, including a value for `R-squared`. It's critical to understand that this is **NOT** the same R^2 we used in linear regression.

The Problem: We Can't Use R^2

In OLS (Linear Regression), $R^2 = 1 - \frac{RSS}{TSS}$. This metric is based on minimizing the *sum of squared residuals* (RSS).

In Logistic Regression, our model does not predict a continuous value. It predicts a *probability*. We don't minimize squared residuals; we minimize the **negative log-likelihood** (also called **Cross-Entropy Loss**).

Because the underlying cost function is different, the R^2 metric is not mathematically valid. A model that predicts a probability of `0.9` for an event that happens (`y=1`) is a *fantastic* prediction, but it would have a "residual" of $(1 - 0.9) = 0.1$, which OLS would want to square and "punish." This doesn't make sense.

The Solution: "Pseudo R^2"

To get a similar "0-to-1" metric for *goodness-of-fit*, statisticians have developed several **"Pseudo R^2"** measures. The one `statsmodels` reports by default is **McFadden's R^2**.

The logic is the same as OLS R^2: "How much better is our model than a 'dumb' baseline?"

- **In OLS:** We compared our model's error (RSS) to the baseline's error (TSS).
- **In Logistic Regression:** We compare our model's **Log-Likelihood** to a baseline model's Log-Likelihood.

1. **Baseline Model (Log-Likelihood Null, LL_{Null}):** A "dumb" model that only knows the *prior* (e.g., $P(y=1) = 0.35$). This is an "intercept-only" model. `statsmodels` calculates this for you.
2. **Our "Full" Model (Log-Likelihood, LL_{Model}):** This is the log-likelihood of our *fitted* model. It will be a "less negative" (better) number than LL_{Null}.

McFadden's R^2 is then defined as:

$$R^2_{McFadden} = 1 - \frac{LL_{Model}}{LL_{Null}}$$

Interpretation:

- LL_{Model} is the value reported as "Log-Likelihood" in your summary table.

- LL_{Null} is reported as "LL-Null".
- `Pseudo R-sq.`: This is the $R^2_{McFadden}$ score.

Consider the following example:

- `Log-Likelihood: -105.19` (LL_{Model})
- `LL-Null: -452.39` (LL_{Null})
- `Pseudo R-sq.: 0.77`

This `0.77` was calculated as $1 - (\frac{-105.19}{-452.39}) \approx 0.77$.

It is interpreted similarly to R^2: "Our model's features explain about **77%** of the 'deviance' (uncertainty) in the outcome, compared to a model that just guesses the average."

Important: You cannot compare a `Pseudo R-sq.` from logistic regression to an R^2 from OLS. They are not equivalent. It is only useful for comparing *nested* logistic regression models (like we do in backward elimination).

Predictive Evaluation

When we use the model in a purely predictive way, we can also use all evaluation measures we have seen for predictive analysis, such as accuracy, precision, recall, F1, confusion matrix etc.

Logistic Regression in Python

Let's look at an example of logistic regression in Python. We will use the R dataset `biopsy`. We can load it using `statsmodels`:

```
from statsmodels.datasets import get_rdataset
biopsy = get_rdataset('biopsy',package='MASS')
print(biopsy.__doc__)
```

```
====== ===============
biopsy R Documentation
====== ===============

Biopsy Data on Breast Cancer Patients
-------------------------------------

Description
~~~~~~~~~~~

This breast cancer database was obtained from the University of
Wisconsin Hospitals, Madison from Dr. William H. Wolberg. He assessed
biopsies of breast tumours for 699 patients up to 15 July 1992; each of
nine attributes has been scored on a scale of 1 to 10, and the outcome
is also known. There are 699 rows and 11 columns.

Usage
~~~~~

.. code:: R

   biopsy

Format
~~~~~~

This data frame contains the following columns:

``ID``
   sample code number (not unique).

``V1``
   clump thickness.

``V2``
   uniformity of cell size.

``V3``
   uniformity of cell shape.

``V4``
   marginal adhesion.

``V5``
   single epithelial cell size.

``V6``
   bare nuclei (16 values are missing).
```

```
``V7``
   bland chromatin.

``V8``
   normal nucleoli.

``V9``
   mitoses.

``class``
   ``"benign"`` or ``"malignant"``.

Source
~~~~~~

P. M. Murphy and D. W. Aha (1992). UCI Repository of machine learning
databases. [Machine-readable data repository]. Irvine, CA: University of
California, Department of Information and Computer Science.

O. L. Mangasarian and W. H. Wolberg (1990) Cancer diagnosis via linear
programming. *SIAM News* **23**, pp 1 & 18.

William H. Wolberg and O.L. Mangasarian (1990) Multisurface method of
pattern separation for medical diagnosis applied to breast cytology.
*Proceedings of the National Academy of Sciences, U.S.A.* **87**, pp.
9193-9196.

O. L. Mangasarian, R. Setiono and W.H. Wolberg (1990) Pattern
recognition via linear programming: Theory and application to medical
diagnosis. In *Large-scale Numerical Optimization* eds Thomas F. Coleman
and Yuying Li, SIAM Publications, Philadelphia, pp 22-30.

K. P. Bennett and O. L. Mangasarian (1992) Robust linear programming
discrimination of two linearly inseparable sets. *Optimization Methods
and Software* **1**, pp. 23-34 (Gordon & Breach Science Publishers).

References
~~~~~~~~~~

Venables, W. N. and Ripley, B. D. (2002) *Modern Applied Statistics with
S-PLUS.* Fourth Edition. Springer.
```

The dataset contains 699 observations and 11 columns. Each observation contains measurements of 9 quantities relating to tissue samples that can be “benign” or “malignant” tumors. Let’s start by manipulating the data a bit. We visualize the values of the `class` variable:

```
biopsy.data['class'].unique()
```

```
array(['benign', 'malignant'], dtype=object)
```

To calculate the logistic regression model using statsmodels, it is necessary to convert these values into integers (0 or 1). Furthermore, it is advisable to avoid calling the column `class` as this is a reserved word for statsmodels. We build a new column `cl` that contains the modified `class` values:

```
biopsy.data['cl'] = biopsy.data['class'].replace({'benign':0, 'malignant':1})
```

We will use the `logit` object form statsmodels:

```
from statsmodels.formula.api import logit
model = logit('cl ~ V1 + V2 + V3 + V4 + V5 + V6 + V7 + V8 + V9',biopsy.data).fit()
model.summary()
```

```
Optimization terminated successfully.
         Current function value: 0.075321
         Iterations 10
```

Logit Regression Results

Dep. Variable:	cl	No. Observations:	683
Model:	Logit	Df Residuals:	673
Method:	MLE	Df Model:	9
Date:	Tue, 03 Nov 2026	Pseudo R-squ.:	0.8837
Time:	13:48:36	Log-Likelihood:	-51.444
converged:	True	LL-Null:	-442.18
Covariance Type:	nonrobust	LLR p-value:	2.077e-162

	coef	std err	z	P>|z|	[0.025	0.975]
Intercept	-10.1039	1.175	-8.600	0.000	-12.407	-7.801
V1	0.5350	0.142	3.767	0.000	0.257	0.813
V2	-0.0063	0.209	-0.030	0.976	-0.416	0.404
V3	0.3227	0.231	1.399	0.162	-0.129	0.775
V4	0.3306	0.123	2.678	0.007	0.089	0.573
V5	0.0966	0.157	0.617	0.537	-0.210	0.404
V6	0.3830	0.094	4.082	0.000	0.199	0.567
V7	0.4472	0.171	2.609	0.009	0.111	0.783
V8	0.2130	0.113	1.887	0.059	-0.008	0.434
V9	0.5348	0.329	1.627	0.104	-0.110	1.179

The logistic regressor explains the relationship between variables well (R^2 high) and is significant (p-value almost zero). Some coefficients have a high p-value. Let's start by eliminating the variable `V2`, which has the highest p-value:

```
model = logit('cl ~ V1 + V3 + V4 + V5 + V6 + V7 + V8 + V9',biopsy.data).fit()
model.summary()
```

```
Optimization terminated successfully.
         Current function value: 0.075321
         Iterations 10
```

Logit Regression Results

Dep. Variable:	cl	No. Observations:	683
Model:	Logit	Df Residuals:	674
Method:	MLE	Df Model:	8
Date:	Tue, 03 Nov 2026	Pseudo R-squ.:	0.8837
Time:	13:48:36	Log-Likelihood:	-51.445
converged:	True	LL-Null:	-442.18
Covariance Type:	nonrobust	LLR p-value:	2.036e-163

	coef	std err	z	P>|z|	[0.025	0.975]
Intercept	-10.0976	1.155	-8.739	0.000	-12.362	-7.833
V1	0.5346	0.141	3.784	0.000	0.258	0.811
V3	0.3182	0.174	1.826	0.068	-0.023	0.660
V4	0.3299	0.121	2.723	0.006	0.092	0.567
V5	0.0961	0.156	0.618	0.537	-0.209	0.401
V6	0.3831	0.094	4.082	0.000	0.199	0.567
V7	0.4465	0.170	2.628	0.009	0.114	0.779
V8	0.2125	0.112	1.902	0.057	-0.006	0.432
V9	0.5341	0.328	1.630	0.103	-0.108	1.176

We proceed by removing `V5`, which has a p-value of 0.537:

```
model = logit('cl ~ V1 + V3 + V4 + V6 + V7 + V8 + V9',biopsy.data).fit()
model.summary()
```

```
Optimization terminated successfully.
         Current function value: 0.075598
         Iterations 10
```

Logit Regression Results

Dep. Variable:	cl	No. Observations:	683
Model:	Logit	Df Residuals:	675
Method:	MLE	Df Model:	7
Date:	Tue, 03 Nov 2026	Pseudo R-squ.:	0.8832
Time:	13:48:36	Log-Likelihood:	-51.633
converged:	True	LL-Null:	-442.18
Covariance Type:	nonrobust	LLR p-value:	2.240e-164

	coef	std err	z	P>|z|	[0.025	0.975]
Intercept	-9.9828	1.126	-8.865	0.000	-12.190	-7.776
V1	0.5340	0.141	3.793	0.000	0.258	0.810
V3	0.3453	0.172	2.012	0.044	0.009	0.682
V4	0.3425	0.119	2.873	0.004	0.109	0.576
V6	0.3883	0.094	4.150	0.000	0.205	0.572
V7	0.4619	0.168	2.746	0.006	0.132	0.792
V8	0.2261	0.111	2.037	0.042	0.009	0.444
V9	0.5312	0.324	1.637	0.102	-0.105	1.167

We remove `V9`, which has a p-value greater than 0.05:

```
model = logit('cl ~ V1 + V3 + V4 + V6 + V7 + V8',biopsy.data).fit()
model.summary()
```

```
Optimization terminated successfully.
         Current function value: 0.078436
         Iterations 9
```

Logit Regression Results

Dep. Variable:	cl	No. Observations:	683
Model:	Logit	Df Residuals:	676
Method:	MLE	Df Model:	6
Date:	Tue, 03 Nov 2026	Pseudo R-squ.:	0.8788
Time:	13:48:36	Log-Likelihood:	-53.572
converged:	True	LL-Null:	-442.18
Covariance Type:	nonrobust	LLR p-value:	1.294e-164

	coef	std err	z	P>|z|	[0.025	0.975]
Intercept	-9.7671	1.085	-9.001	0.000	-11.894	-7.640
V1	0.6225	0.137	4.540	0.000	0.354	0.891
V3	0.3495	0.165	2.118	0.034	0.026	0.673
V4	0.3375	0.116	2.920	0.004	0.111	0.564
V6	0.3786	0.094	4.035	0.000	0.195	0.562
V7	0.4713	0.166	2.837	0.005	0.146	0.797
V8	0.2432	0.109	2.240	0.025	0.030	0.456

All coefficients now have an acceptable p-value. Let's proceed to the analysis of the coefficients. We calculate the exponentials:

```
np.exp(model.params)
```

```
Intercept    0.000057
V1           1.863641
V3           1.418374
V4           1.401487
V6           1.460166
V7           1.602133
V8           1.275287
dtype: float64
```

- The near-zero value of the exponential of the intercept indicates that, when all variables take zero values, the odds are very low. This suggests that p is low, while $1 - p$ is very high. The probability of having a malignant tumor is therefore very low if all variables take zero values;
- An increase of one unit in the value of `V1` corresponds to an increase of approximately 86% in the odds, making the possibility of a malignant tumor higher;
- An increase of one unit in the value of `V3` corresponds to an increase of approximately 41% in the odds;
- An increase of one unit in the value of `V4` corresponds to an increase of approximately 40% in the odds;

- An increase of one unit in the value of `V6` corresponds to an increase of approximately 46% in the odds;
- An increase of one unit in the value of `V7` corresponds to an increase of approximately 60% in the odds;
- An increase of one unit in the value of `V8` corresponds to an increase of approximately 27% in the odds;

The increase of variables generally causes an increase in the odds. Therefore, we expect the values of the variables to be small in the presence of benign tumors.

Exercises

1. **Read the coefficients.** Fit a logistic regression of `Survived` on `Sex` and `Pclass`. Convert each coefficient into an odds ratio, and write one sentence per coefficient in the language of the problem — no statistical vocabulary.
2. **Odds, log-odds, probability.** For a female passenger in second class, compute the predicted log-odds, the odds and the probability by hand from the coefficients, then check against `predict`. Which of the three is linear in the predictors?
3. **A continuous predictor.** Add `Age` to the model. Is its coefficient significant? Compute how much the predicted probability changes between a 20-year-old and a 40-year-old, all else equal — and note that the answer depends on what "all else" is.
4. **Confounding, again.** Fit `Survived ~ Age` alone, then `Survived ~ Age + Pclass`. Does the age coefficient change? Relate what you see to chapter 11.
5. **Where the linear model shows.** Plot the predicted probability of survival against age for each class. Why are the three curves parallel on the log-odds scale but not on the probability scale?

References

- Chapter 4 of [1]

[1] James, Gareth Gareth Michael. An introduction to statistical learning: with applications in Python, 2023.https://www.statlearning.com

© 2026 Antonino Furnari · Department of Mathematics and Computer Science, University of Catania Built with Quarto

Credits and data sources

Extending regression models: interactions, polynomial and multinomial terms

Chapter resources

Slides (PDF) Open in Colab Practice quiz

Chapter 8 and chapter 9 introduced the two workhorse regression models in their simplest form: one straight line, one logistic curve, each with its coefficients read as a statement about the phenomenon. This chapter keeps that statistical reading and relaxes three of its restrictions.

Interaction terms let the effect of one variable depend on another — the honest way to say "it depends". **Polynomial terms** let a relationship bend without abandoning least squares. **Multinomial logistic regression** extends the logistic model from two outcomes to many.

All three are still about *explaining*: we fit with `statsmodels`, read coefficients, and check whether they say something defensible. Chapter 14 takes the same models and asks a different question — how well do they predict data they have never seen?

Extending the Linear Model with Interaction Terms

A simple linear model assumes the effect of each predictor on Y is **additive**. The formula

$$mpg = \beta_0 + \beta_1 \text{horsepower} + \beta_2 \text{weight}$$

assumes that the effect of 1 unit of `horsepower` (which is β_1) is **constant**, regardless of the car's `weight`.

Is that true? We can test this by adding an **interaction term**:

$$mpg = \beta_0 + \beta_1 \text{horsepower} + \beta_2 \text{weight} + \beta_3 (\text{horsepower} \times \text{weight})$$

```python
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf

# Load and clean the data (same as last lecture)
mpg = sns.load_dataset('mpg')
mpg = mpg.drop('name', axis=1)
mpg['horsepower'] = pd.to_numeric(mpg['horsepower'], errors='coerce')
mpg = mpg.dropna()

# Fit the simple additive model
model_additive = smf.ols('mpg ~ horsepower + weight', data=mpg).fit()

# Fit the new interaction model
model_interaction = smf.ols('mpg ~ horsepower + weight + horsepower * weight',
        data=mpg).fit()

print("--- Additive Model (R-squared) ---")
print(f"Adj. R-squared: {model_additive.rsquared_adj:.4f}")

print("\n--- Interaction Model (R-squared) ---")
print(f"Adj. R-squared: {model_interaction.rsquared_adj:.4f}")

print("\n--- Interaction Model Coefficients ---")
print(model_interaction.summary().tables[1])
```

```
--- Additive Model (R-squared) ---
Adj. R-squared: 0.7049

--- Interaction Model (R-squared) ---
Adj. R-squared: 0.7465

--- Interaction Model Coefficients ---
================================================================================
==
                         coef    std err          t      P>|t|      [0.025
0.975]
--------------------------------------------------------------------------------
--
Intercept            63.5579      2.343     27.127      0.000      58.951
68.164
horsepower           -0.2508      0.027     -9.195      0.000      -0.304
-0.197
weight               -0.0108      0.001    -13.921      0.000      -0.012
-0.009
horsepower:weight  5.355e-05   6.65e-06      8.054      0.000    4.05e-05    6.66e-
05
================================================================================
==
```

Interpreting the Interaction

Notice two things:

1. The **Adj. R-squared** increased! This indicates the interaction model is a better fit.
2. The p-value for the `horsepower:weight` term is tiny (< 0.001). This confirms the interaction is statistically significant.

But how do we interpret the coefficient for `horsepower`? We can rewrite the formula:

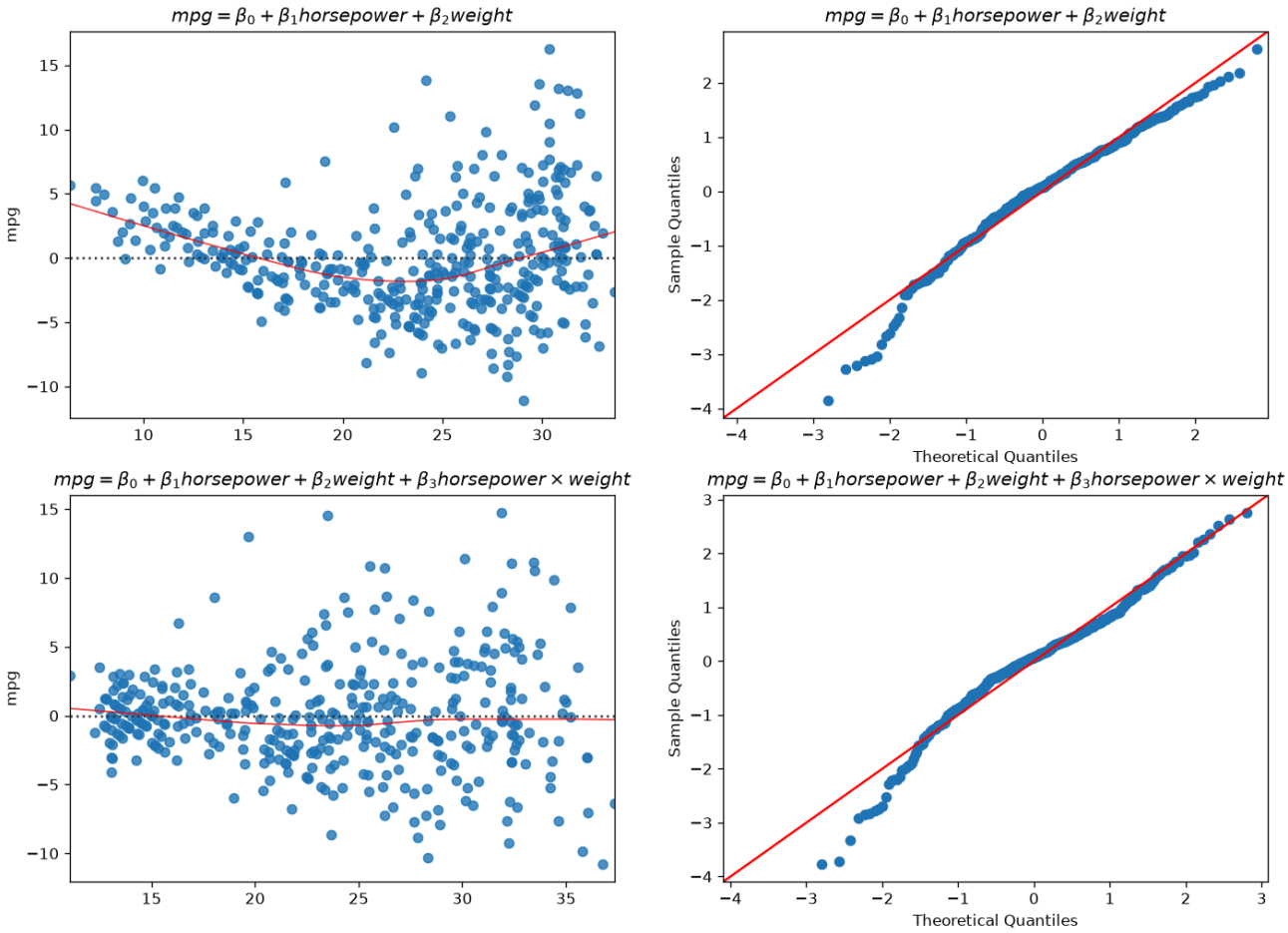
$$mpg = \beta_0 + (\beta_1 + \beta_3 \times \text{weight}) \times \text{horsepower} + \beta_2 \text{weight}$$

The "effect" of `horsepower` is no longer a single number, β_1. It is now $(\beta_1 + \beta_3 \times \text{weight})$. This means the effect of `horsepower` *depends on* the `weight` of the car.

In practice, **when we increase `weight` by one unit, we increase the effect of `horsepower` on `mpg` by β_3 units**. This means that for heavier cars, `horsepower` has a less negative effect on `mpg`.

This makes sense mechanically: in a very heavy car, horsepower is needed just to move it, so the efficiency penalty is smaller compared to a light car where extra horsepower is "overkill.

Let's diagnose our residuals to see if we gained something:

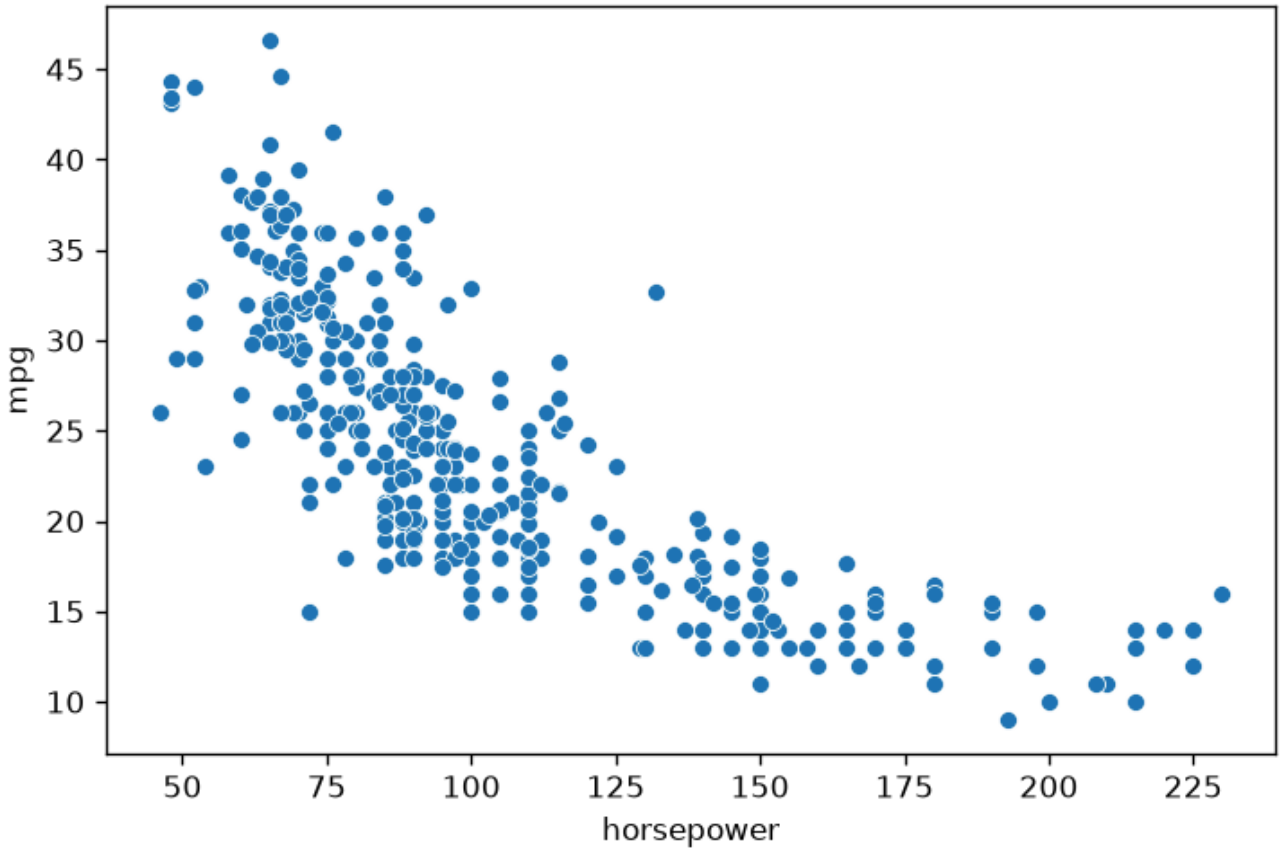


The situation improved! We now have a better model, even if not perfect.

Quadratic and Polynomial Regression

We will now take another route to address our nonlinear problem. Let's have a look at the relationship between `mpg` and `horsepower`:

```
sns.scatterplot(x='horsepower',y='mpg',data=data)
plt.show()
```



This relationship does not look linear. It looks instead as a quadratic, which would be fit by the following model:

$$mpg = \beta_0 + \beta_1 horsepower + \beta_2 horsepower^2$$

Again, the model above is nonlinear in `horsepower`, but we can still fit it with a linear regressor if we add a new variable $z = horsepower^2$.

The fit model will obtain $R^2 = 0.688$, larger than the $R^2 = 0.606$ obtained by the base model ($mpg = \beta_0 + \beta_1 horsepower$). Both have a large F-statistic.

The estimated coefficients are:

```
ols("mpg ~ horsepower + I(horsepower**2)", data).fit().summary().tables[1]
```

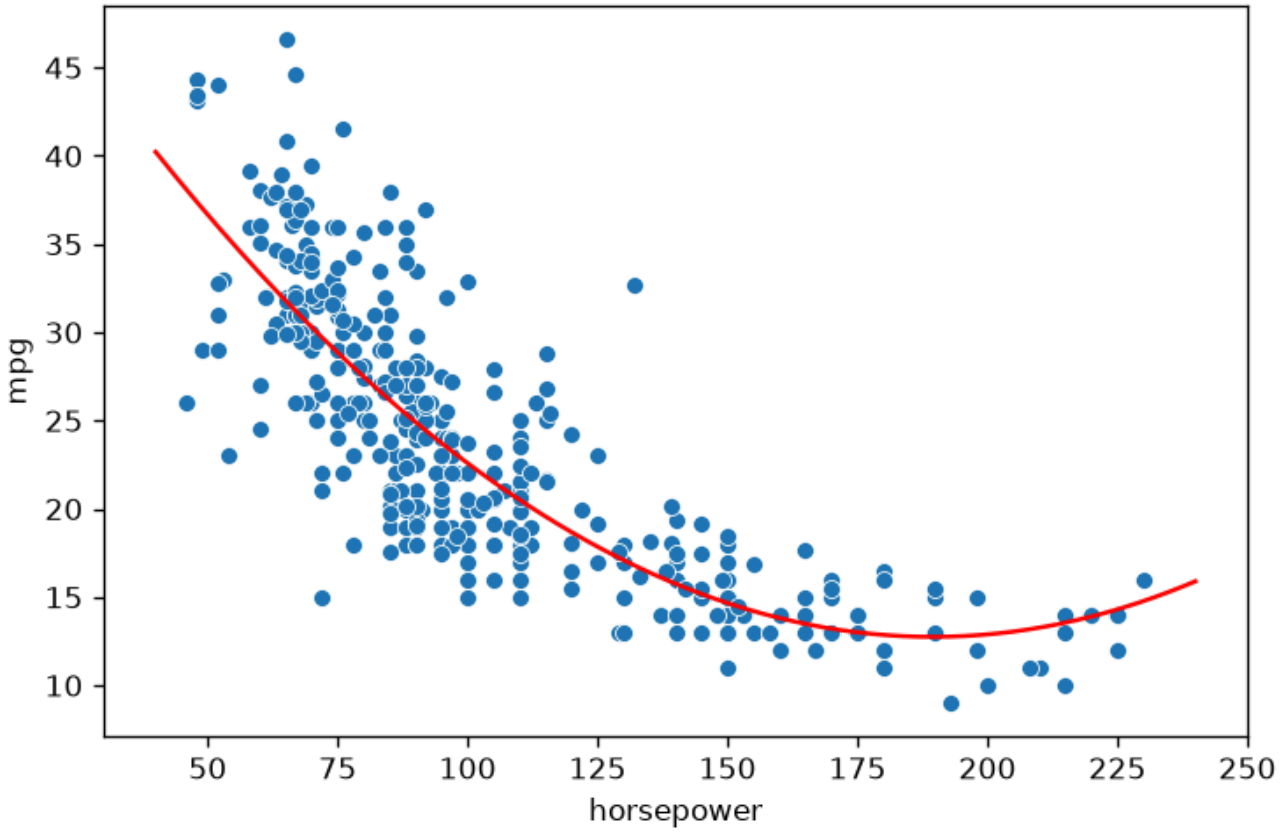
	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	56.9001	1.800	31.604	0.000	53.360	60.440
horsepower	-0.4662	0.031	-14.978	0.000	-0.527	-0.405
I(horsepower ** 2)	0.0012	0.000	10.080	0.000	0.001	0.001

The coefficients now describe the quadratic:

$$y = 56.9001 - 0.4662x + 0.0012x^2$$

If we plot it on the data, we obtain the following:

```
model=ols("mpg ~ horsepower + I(horsepower**2)", data).fit()
b0, b1, b2 = model.params.values
sns.scatterplot(x='horsepower',y='mpg',data=data)
x = np.linspace(40,240,200)
y = b0+b1*x+b2*x**2
plt.plot(x,y,'r')
plt.show()
```



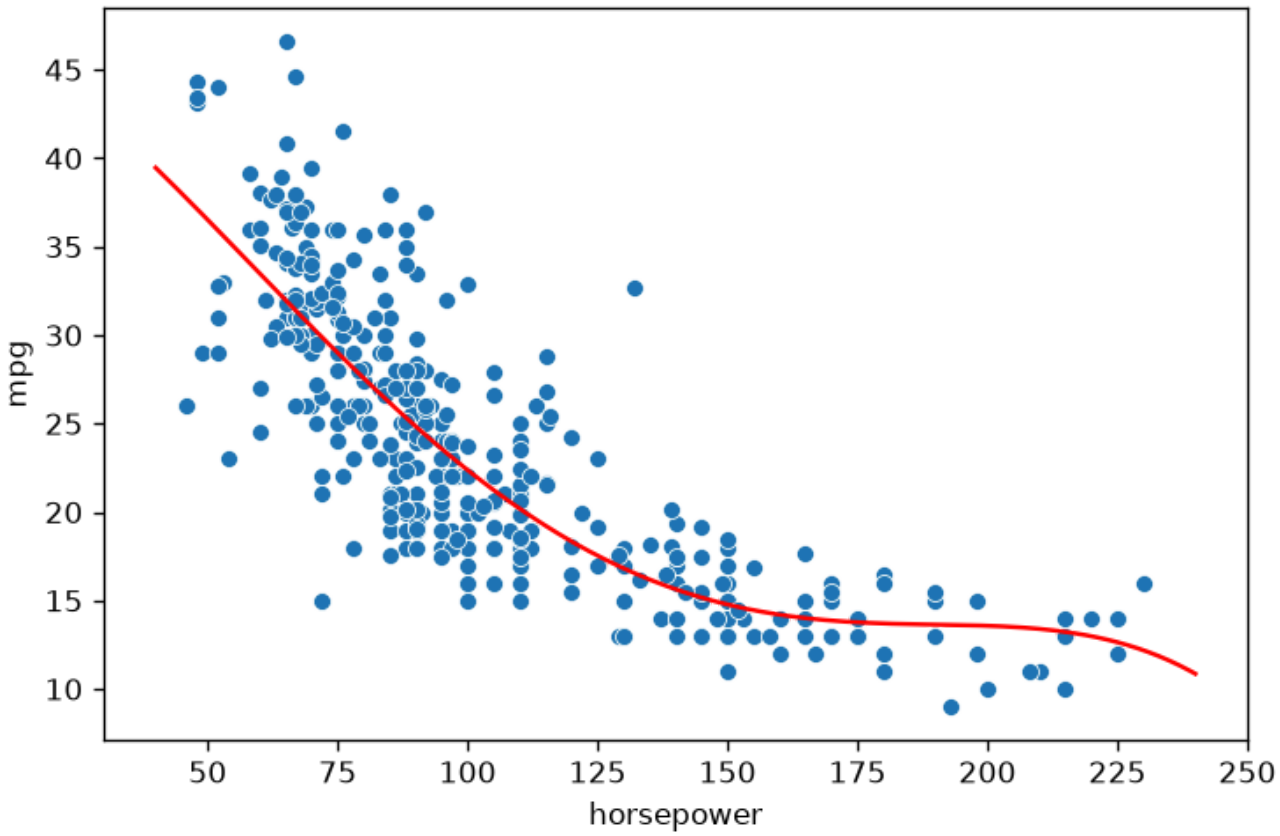
Polynomial Regression

In general, we can fit a polynomial model to the data, choosing a suitable degree d. For instance, for $d = 4$ we have:

$$mpg = \beta_0 + \beta_1 horsepower + \beta_2 horsepower^2 + \beta_3 horsepower^3 + \beta_4 horsepower^4$$

which identifies the following fit:

```
model=ols("mpg ~ horsepower + I(horsepower**2) + I(horsepower**3) +
        I(horsepower**4)", data).fit()
b0, b1, b2, b3,b4 = model.params.values
sns.scatterplot(x='horsepower',y='mpg',data=data)
x = np.linspace(40,240,200)
y = b0+b1*x+b2*x**2+b3*x**3+b4*x**4
plt.plot(x,y,'r')
plt.show()
```



This approach is known as **polynomial regression** and allows to turn the linear regression into a nonlinear model. Note that, when we have more variables, polynomial regression also includes interaction terms. For instance, the linear model:

$$y = \beta_0 + \beta_1 x + \beta_2 y$$

becomes the following polynomial model of degree 2:

$$y = \beta_0 + \beta_1 x + \beta_2 y + \beta_3 x^2 + \beta_4 y^2 + \beta_5 xy$$

As usual, we only have to add new variables for the squared and interaction term and solve the problem as a linear regression one. This is easily handled by libraries. Note that, as the number of variables increases, the number of terms to add for a given degree also increases.

The key observation here is that, while the polynomial regression function is **nonlinear in the variable**, it is still **linear in its coefficients**, which makes it still solvable with ordinary

least squares.

Multinomial Logistic Regression

In many cases, we want to study the relationship between a set of **continuous or categorical independent variable and a non-binary categorical dependent variable**.

Let us consider the `diabetes` dataset:

```
import statsmodels.api as sm

# Try loading the dataset
diabetes_data = sm.datasets.get_rdataset("Diabetes", "heplots").data
diabetes_data.head()
```

	relwt	glufast	glutest	instest	sspg	group
0	0.81	80	356	124	55	Normal
1	0.95	97	289	117	76	Normal
2	0.94	105	319	143	105	Normal
3	1.04	90	356	199	108	Normal
4	1.00	90	323	240	143	Normal

This dataset contains clinical measurements from 145 subjects, used to study glucose metabolism and insulin resistance.

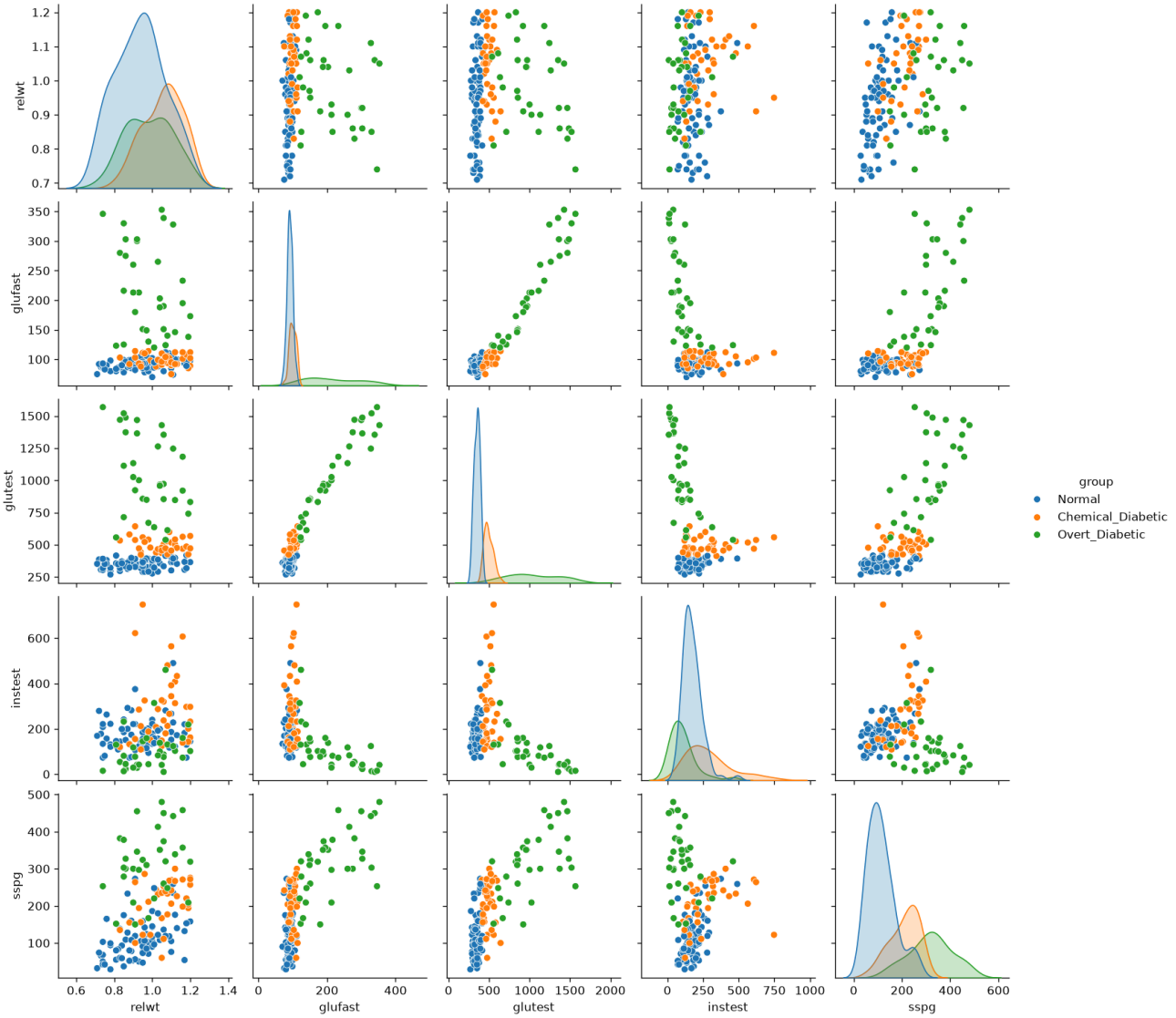
It was originally presented in *Data: A Collection of Problems from Many Fields* (Andrews & Herzberg, 1985) and is included in the R package **heplots**.

Variables

- **relwt**: Relative weight (ratio of actual to expected weight)
- **glufast**: Fasting plasma glucose level
- **glutest**: Glucose intolerance (oral glucose tolerance test area)
- **instest**: Insulin response (insulin area)
- **sspg**: Steady state plasma glucose (measure of insulin resistance)
- **group**: Diagnostic group
  - *Normal*
  - *Chemical_Diabetic*
  - *Overt_Diabetic*

Let's visualize the data:

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.pairplot(diabetes_data, hue='group')
plt.show()
```



Differently from previous examples, our categorical variable contains three levels. Also, we can easily identify an interesting **base class**, i.e., the "Normal" group.

We will define an extension of the logistic regression model, the multinomial logistic regression model, based on this concept of "base" class.

The Multinomial Logistic Regression Model

When the dependent variable can assume more than two values, **we can define the multinomial logistic regression model**. In this case, we select one of the values of the dependent variable Y as **a baseline class** (e.g., the "Normal" group). Without loss of generality, let K be the number of classes and let $Y = 1$ be the baseline class. Recall that in the case of the logistic regressor, we modeled the logarithm of the odd as our linear function:

$$\log\left(\frac{P(y=1|\mathbf{x})}{1-P(y=1|\mathbf{x})}\right) = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$$

Since we have more than one possible outcomes for the dependent variable, rather than modeling the odds, a multinomial logistic regressor models the logarithm of the ratio between a given class k and the baseline class 1 as follows:

$$\log\left(\frac{P(Y=k|X=\mathbf{x})}{P(Y=1|X=\mathbf{x})}\right) = \beta_{k0} + \beta_{k1} x_1 + \ldots + \beta_{kn} x_n$$

Note that, in practice, we need to define a different linear combination for each class $k = 1 \ldots K$, hence we need $(n+1) \times (k-1)$ parameters.

Doing the math, it can be shown that:

$$P(Y=k|X=\mathbf{x}) = \frac{e^{\beta_\mathbf{k}{}^T \mathbf{X}}}{1 + \sum_{l=2}^{K} \mathbf{e}^{\beta_\mathbf{l}^\mathbf{T} \mathbf{X}}}$$

$$P(Y=1|X=\mathbf{x}) = \frac{1}{1 + \sum_{l=2}^{K} \mathbf{e}^{\beta_\mathbf{l}^\mathbf{T} \mathbf{X}}}$$

Where $\beta_\mathbf{k} = (\beta_0, \beta_1, \ldots, \beta_k)$ and $\mathbf{X} = (1, x_1, \ldots, x_n)$.

These two expressions can be used to compute the probabilities of the classes once that the parameters have been estimated and x is observed.

ⓘ A note on the softmax parametrisation

Machine learning usually writes this same model without a baseline class, giving every class its own coefficient vector:

$$P(Y=k|X=\mathbf{x}) = \frac{e^{\beta_k^T \mathbf{x}}}{\sum_{l=1}^{K} e^{\beta_l^T \mathbf{x}}}$$

This is the **softmax** form. It is the same model — a *reparametrisation*, and an **overparametrised** one: adding a constant vector to every β_k leaves all the probabilities unchanged, so the coefficients are identified only up to that shift. Choosing a baseline, as above, is exactly what removes the ambiguity, which is why statistics does it and why single coefficients can be read as log-odds ratios here and not there.

The distinction matters when you read a library's output — `scikit-learn` fits the softmax form, `statsmodels` the baseline form — but nothing in this course depends on it, and **the exam does not ask about softmax**.

Let's fit the following model:

$$\text{group} = \beta_0 + \beta_1 \text{relwt} + \beta_1 \text{glutest}$$

We obtain the following results:

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
import pandas as pd

# Assume you already loaded the Diabetes dataset into diabetes_data
# Columns: relwt, glufast, glutest, instest, sspg, group

# Recode group into integers (baseline = Normal)
diabetes_data2 = diabetes_data.copy()
diabetes_data2['group'] = diabetes_data2['group'].map({
    'Normal': 0,
    'Chemical_Diabetic': 1,
    'Overt_Diabetic': 2
}).astype(int)

# Fit multinomial logistic regression
model = smf.mnlogit("group ~ glutest", data=diabetes_data2)
result = model.fit()

# Summary with coefficients, standard errors, z-scores, p-values
print(result.summary())

# Likelihood ratio test for overall model fit
print("\nModel Likelihood Ratio Test:")
print(result.llr, result.llr_pvalue)
```

```
Optimization terminated successfully.
         Current function value: 0.100843
         Iterations 14
                          MNLogit Regression Results
==============================================================================
Dep. Variable:                  group   No. Observations:                  145
Model:                        MNLogit   Df Residuals:                      141
Method:                           MLE   Df Model:                            2
Date:                Sat, 26 Sep 2026   Pseudo R-squ.:                  0.9013
Time:                        16:16:25   Log-Likelihood:                -14.622
converged:                       True   LL-Null:                       -148.10
Covariance Type:            nonrobust   LLR p-value:                 1.076e-58
==============================================================================
   group=1       coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept    -90.3738     42.527     -2.125      0.034    -173.726      -7.022
glutest        0.2153      0.101      2.128      0.033       0.017       0.413
------------------------------------------------------------------------------
   group=2       coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept   -109.8266     43.021     -2.553      0.011    -194.147     -25.506
glutest        0.2471      0.102      2.428      0.015       0.048       0.447
==============================================================================

Model Likelihood Ratio Test:
266.9538631863749 1.0757346252010753e-58
```

We can see that the model estimates two sets of coefficients: one for `group=1` (Chemical Diabetic) and one for `group=2` (Overt Diabetic). No coefficients are estimated for `group=0` (Normal), which serves as the baseline category. Let us interpret the results:

Model Overview

- **Dependent variable**: `group`, with three diagnostic categories
- **Predictors included**: `glutest` (glucose intolerance)
- **Estimation method**: Maximum Likelihood Estimation (MLE)
- **Convergence**: Successful after 14 iterations
- **Log-likelihood**: −14.622 (final value of our cost function)
- **Pseudo R-squared**: 0.9013 (good explanatory power)
- **Likelihood Ratio Test**:
  - Statistic: 266.95
  - p-value: almost zero → highly significant improvement over the null model

Coefficient Interpretation

- The intercept for `group=1` is -90.3738. This means that the odds of being *Chemical Diabetic* versus *Normal* are $e^{-90.3738} \approx 0$ when `glutest` is zero. This is an extremely low value, indicating

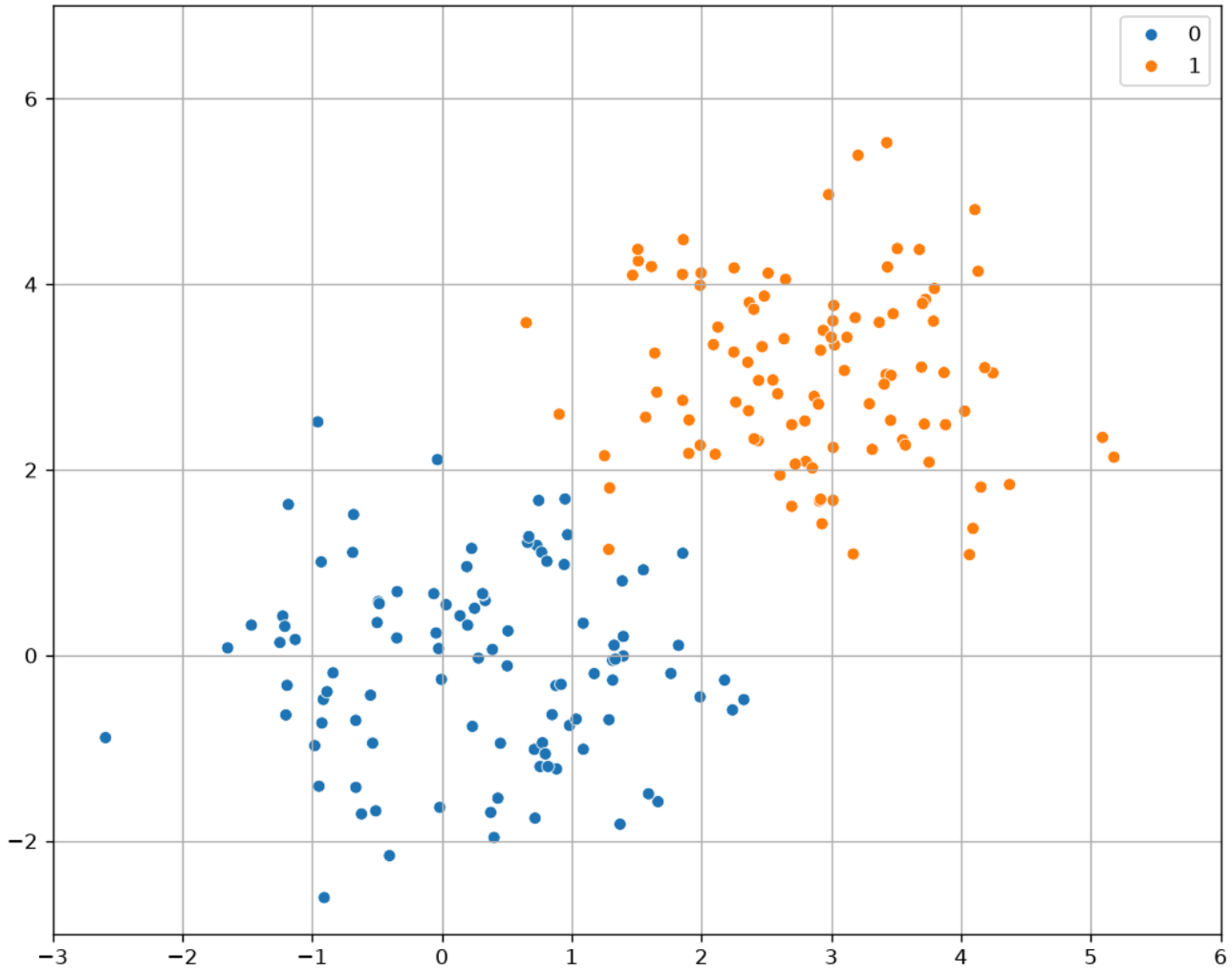
that when `glutest` is zero, it is much more probable that the subject is *Normal* rather than *Chemical Diabetic*.

- The intercept for `group=2` is -109.8266. This means that the odds of being *Overt Diabetic* versus *Normal* are $e^{-109.8266} \approx 0$ when `glutest` is zero. This indicates that when everything is zero, it is much more probable that the subject is *Normal* rather than *Overt Diabetic*. Hence, when `glutest` is zero, the subject is almost certainly *Normal*.
- The coefficient 0.2153 for `glutest` in `group=1` suggests that a one-unit increase in `glutest` increases the odds of being *Chemical Diabetic* versus *Normal* by $e^{0.2153} \approx 1.24$, an increase of about **24%**.
- The coefficient 0.2471 for `glutest` in `group=2` implies a similar increase in odds for *Overt Diabetic* versus *Normal*: $e^{0.2471} \approx 1.28$, an increase of about **28%**.

Geometrical Interpretation of the Coefficients of a Logistic Regressor

Similar to linear regression, also the coefficients of logistic regression have a geometrical interpretation, which is particularly interesting when treating the classifier from a purely predictive point of view. We will see that, while linear regression finds a «curve» that fits the data, logistic regression finds a hyperplane that separates the data.

Let us consider a simple example with bi-dimensional data $\mathbf{x} \in \mathfrak{R}^2$ as the one shown in the following:



Let us assume that we fit a logistic regressor model to this data

$$P\left(y=1|\mathbf{x}\right)=\frac{1}{1+e^{-(\beta_0+\beta_1 x_1+\beta_2 x_2)}}$$

and find the following values for the parameters:

$$\begin{cases}\beta_0=-3.47\\ \beta_1=1.17\\ \beta_2=1.43\end{cases}$$

We know that these parameters allow to find a probability value according to the formula above. We can use these values to **classify the observations $\mathbf{x}$**. In practice, a reasonable criterion to classify observations would be:

$$\hat{y}=\begin{cases}1 & \text{if } P(y=1|\mathbf{x})\geq 0.5\\ 0 & \text{otherwise}\end{cases}$$

This makes sense as we are assigning the observations to the group for which the posterior probability $P(y|\mathbf{x})$ is higher.

To understand how the data is classified, we can look at those points in which the classifier is uncertain, which is often called **the decision boundary**, i.e., those points in which $P\left(y=1|\mathbf{x}\right)=0.5$.

We note that:

$$P\left(y=1|\mathbf{x}\right)=0.5 \Leftrightarrow e^{-(\beta_0+\beta_1 x_1+\beta_2 x_2)}=1 \Leftrightarrow 0=\beta_0+\beta_1 x_1+\beta_2 x_2$$

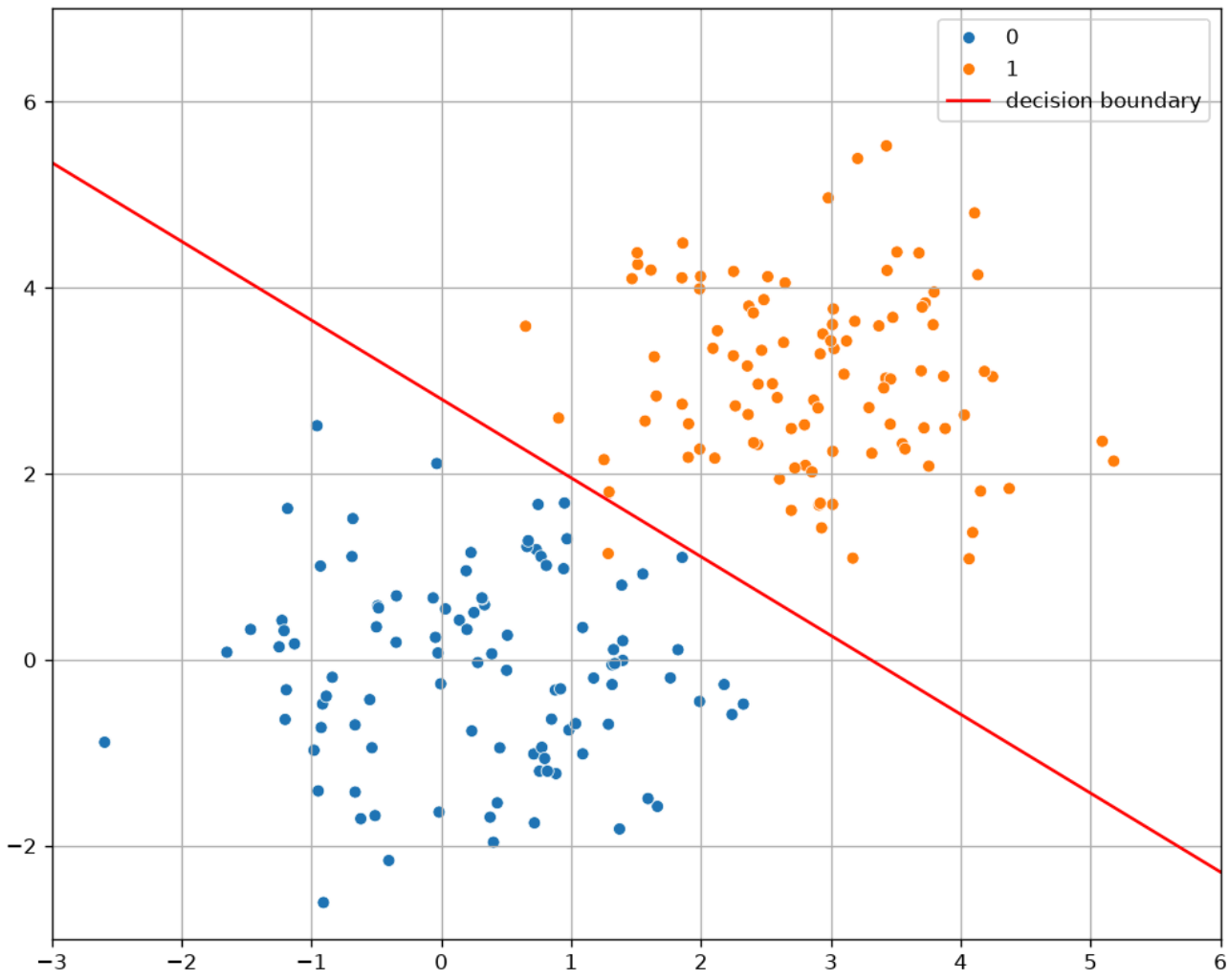
This last equation is the equation of a line (in the form $ax+by+c=0$). We can see it in explicit form:

$$x_2=-\frac{\beta_1}{\beta_2}x_1-\frac{\beta_0}{\beta_2}$$

So, we have found a line which has a

- Angular coefficient equal to $-\frac{\beta_1}{\beta_2}$;
- Intercept equal to $-\frac{\beta_0}{\beta_2}$;

If we plot this line, we obtain the **decision boundary** which separates the elements from the two classes:



As can be seen, the decision boundary found by a logistic regressor is a line. This is because **a logistic regressor is a linear classifier**, despite the logistic function is not linear!

Exercises

1. **Interaction, read aloud.** Fit `Fare ~ Age * C(Pclass)` on the Titanic data. Write one sentence saying what the interaction term means, without using the word “interaction”.
2. **When an interaction is not needed.** Fit the same model without the interaction and compare. Does the interaction earn its parameters? Justify the answer with something other than the p-value.
3. **Polynomial degree.** Fit polynomials of degree 1 to 6 to a relationship of your choice. Plot each one. Which degree would you defend to a colleague, and on what grounds — given that this chapter has no test set?
4. **Multinomial coefficients.** Fit a multinomial logistic regression of `Pclass` on `Fare` and `Age`. Interpret one coefficient with respect to the baseline category, then refit with a different baseline and confirm that the conclusions do not change.

References

- James, Witten, Hastie, Tibshirani, Taylor. *An Introduction to Statistical Learning with Applications in Python*, sections 3.3 (interactions and polynomial terms) and 4.3 (multinomial logistic regression).
- Heumann, Schomaker, Shalabh. *Introduction to Statistics and Data Analysis*, chapter 11.
- `statsmodels` formula API documentation, for the `*` and `:` operators.

© 2026 Antonino Furnari · Department of Mathematics and Computer Science, University of Catania Built with Quarto

Credits and data sources

Causal analysis

Chapter resources

Slides (PDF) Open in Colab Practice quiz

We have seen how to characterize the relationship between two or more variables with correlation and linear regression. We also said that **correlation is not causation**, meaning that, if we observe a correlation between two variables, that does not necessarily imply a cause-effect relationship between them. However, in many cases, it is still useful (or required!) to establish a cause-effect relationship (or lack thereof) between two variables. Consider the following examples:

- We assess that **a given disease is more frequent among smokers**. Is this just a correlation or is there a cause-effect relationship? If a smoker stops smoking, will their risk to get the disease be reduced?
- **We develop a new drug** and assess that people taking this drug are less likely to get disease X. Is this just a correlation or is there a cause-effect relationship? This is important, as people should take the drug only if benefits are clear.
- **A large company founds that product sites located in the geographical areas close to the sea are more productive than the others**. Is this just a correlation or is it a cause-effect relationship? Shall we move all sites closer to the sea?

Luckily, the field of **causal analysis** has, over the years, developed a set of tools which allow to answer those questions.

The goal of this lecture is to provide a primer of causal analysis, introducing the main concepts and discussing some techniques which can be applied in some simple cases.

Tutoring vs Performance

Let us consider an example:

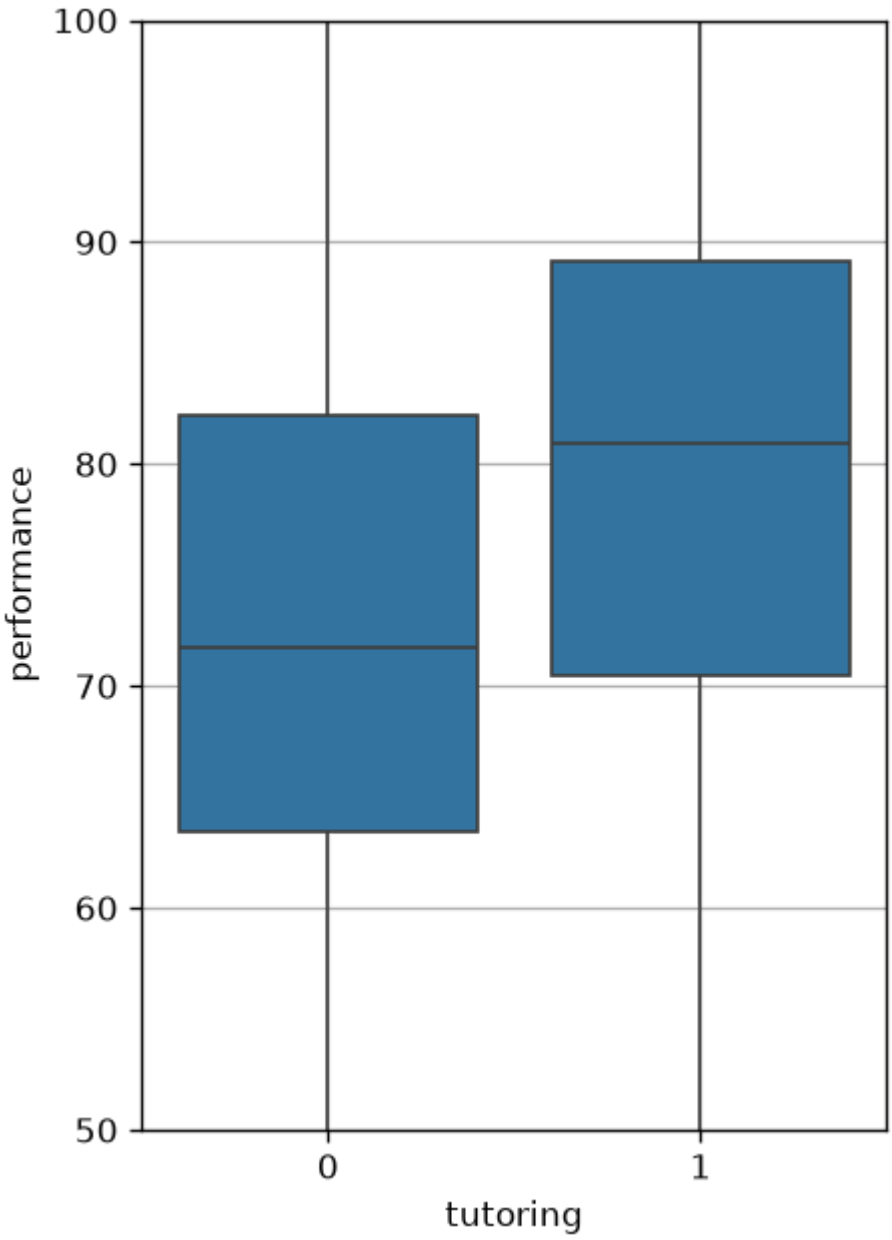
> A school wants to see if a new teaching method is effective. To do so, they ask 50 students if they want to **voluntarily enroll a tutoring program**. After one year, they measure the average performance of students who participated in the programme and compare it to those of students who did not participate in the programme.

We obtain the following data:

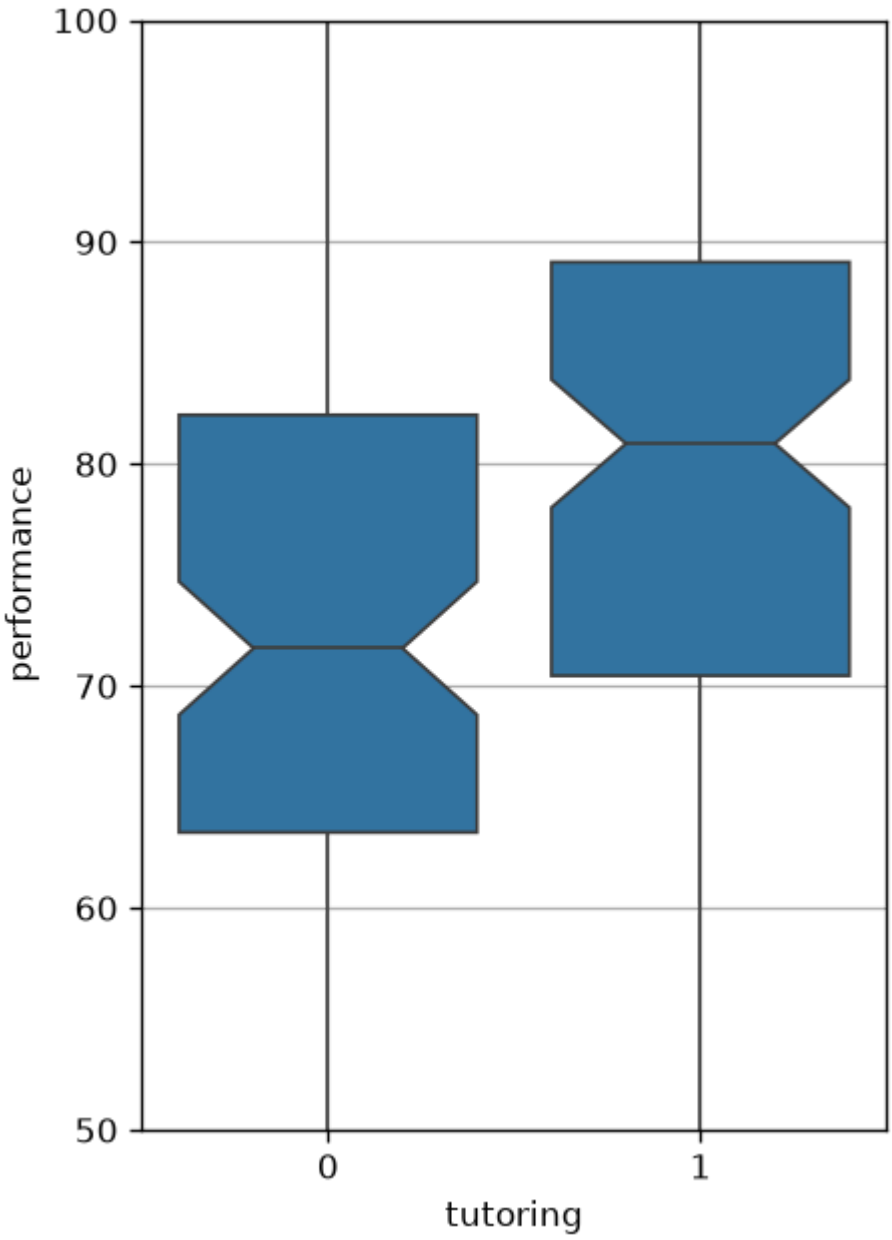
	tutoring	performance
0	0	76.432447
1	1	91.550461
2	1	85.107378
3	0	69.981030
4	1	70.111151
...	...	...
195	0	68.477160
196	0	87.783594
197	1	88.220111
198	1	93.399423
199	1	84.082421

200 rows × 2 columns

We want to understand if `tutoring` has an effect on `performance`. Let us show some boxplots:



The comparison tell us `tutoring` seems to significantly improve performance! However, we took a statistics class, so we want to check if everything is statistically significant. We add a **notch** to each plot. The notch will highlight the confidence intervals for the estimation of the median value:



Since the notches do not intersect, there is no overlap between the confidence intervals for a C.L. of 95%, hence it is very unlikely that the two median values are very close to each other. The difference seem to be statistically relevant. If we want to be very sure, we can **compute a two-sample t-Test** to ensure that the mean values of the two samples actually are distinct.

The null hypothesis will be that the two samples have the same mean. If we reject this hypothesis, then the two means are distinct. We obtain the following test statistic and p-value:

```
T-test results - T-statistic: 4.086568160195115, p-value: 6.358186591194334e-05
```

It looks like the tutoring program is a great success!

Measuring Causation - Potential

Outcomes (Counterfactuals)

The comparison above seems to be clear: we should enroll everybody in the tutoring program! But we also know that **correlation is not (necessarily) causation!**. We are seeing a correlation here - is it also causation? Of course, we want to know whether it is worth to extend the tutoring program or not!

To be able to truly assess a causal relationship between the two variables, ideally, we would like to **take the same student and let them attend and not attend the tutoring program at the same time, so that then we can compare, for the same student, the effect of tutoring alone.** This is of course impossible to do and even if we could it would be unethical. Can we ask a student not to follow a program which we suspect will harm their performance? Even if we ask the same student to first follow the tutoring, then stop, the student "before" and "after" is not the same student after all, so the two values will never be fully comparable.

We will now **enter the realm of potential outcomes**. In practice, we ask ourselves questions such as **"what if this student did not take the tutoring?"**. Let's see some notation first. We will define two variables:

- T_i: a "treatment" value indicating whether the observation i (student i) took the treatment (i.e., attended the tutoring) or not. Here we use the standard terminology of causal inference which uses the word "treatment" as in health-related sciences. We will assume this variable to be binary: $T_i = 1$ means "took the treatment", while $T_i = 0$ means "didn't take the treatment". We will call the group of observations for which $T_i(1)$ the **treatment group** and the group of observations for which $T_i(0)$ the **control group**.
- Y_i: the "outcome" of observation i. In our case, this is the average performance of student i.

If for a unit i, the treatment value is $T_i = t$, we will denote the related observation Y_i as $Y_i(t)$. We note that, for a given unit i, we will **either observe $Y_i(0)$ or $Y_i(1)$, but never both!**.

Nevertheless, we will introduce the concept of **potential outcome** as the **value we would observe if we were able to intervene on the treatment variable T_i and change it**. If for a unit i we have $T_i = 0$, we will directly observe the outcome $Y_i(0)$, but we will also **theoretically define** the potential outcome $Y_i(1)$. This **is not real and cannot be observed** but it will serve as a useful theoretical concept.

Potential outcomes are also known as **counterfactuals** because they answer to the question "what if...".

If we had potential outcomes, for each unit i we could compute the **individual treatment effect** (the effect of taking the treatment) as:

$$Y_i(1) - Y_i(0)$$

This is the effect of attending the tutoring. If we average this over the distribution of observations, we obtain the **average treatment effect**:

$$ATE = E[Y(1) - Y(0)]$$

This value will tell us **how the exam performance varies in average if the same subjects takes the tutoring or not**. As such, it would indicate **the causal effect of tutoring on examination performance**.

Alternatively, we can compute the **average treatment effect on the treated**:

$$ATT = E[Y(1) - Y(0)|T = 1]$$

This simply restricts the computation to the treated subjects, and **also establishes a causal effect**.

The Bias Arising from Correlation

Without potential outcomes, we cannot compute the ATE or ATT scores. Indeed, we have **two distinct groups**, the **treated** (i.e., those observations i such that $T_i = 1$) and the **untreated**, also known as **control group** (i.e., those observations i such that $T_i = 0$).

We could think to replace that calculation with the only data we can observe:

$$E[Y(1)|T = 1] - E[Y(0)|T = 0]$$

This is what **a correlation can measure**: the change in average value of Y in two distinct groups with different characteristics.

Note that there are no potential outcomes in the expression above as we directly observe $Y_i(1)$ when $T_i = 1$ and we directly observe $Y_i(0)$ when $T_i = 0$. However, the expression above is now **comparing different groups**. Indeed, the subjects who took the treatment are different from the ones who did not take the treatment, and we do not know if their exam performance scores are really comparable. **Imagine that, for some reason, students in group 1 are, by chance, better students, then the difference we measure is going to be biased and will not reveal a cause-effect relationship**.

We can show this in formulas. Let's add and subtract the counterfactual term $E[Y(0)|T = 1]$ to the expression above:

$$E[Y(1)|T = 1] - E[Y(0)|T = 0] = E[Y(1)|T = 1] - E[Y(0)|T = 0] - E[Y(0)|T = 1] + E$$

We note that:

$$ATT = E[Y(1) - Y(0)|T = 1] = E[Y(1)|T = 1] - E[Y(0)|T = 1]$$

Hence, we can see the expression above as follows:

$$E[Y(1)|T=1] - E[Y(0)|T=0] = E[Y(1)|T=1] - E[Y(0)|T=1] + E[Y(0)|T=1] - E$$

$$= ATT + E[Y(0)|T=1] - E[Y(0)|T=0]$$

We will call the last term the bias:

$$BIAS = E[Y(0)|T=1] - E[Y(0)|T=0]$$

So the whole expression can be seen as:

$$E[Y(1)|T=1] - E[Y(0)|T=0] = ATT + BIAS$$

which shows that the effect given by the correlation (left term) is **biased**. Note that the bias cannot be computed (it contains a counterfactual term), but we can interpret it as follows:

> The bias measures how the treated ($T=1$) and control ($T=0$) groups would differ if no one had taken the treatment.

In practice:

- If the groups are similar to each other (e.g., treated and control group are mixed students with similar abilities), then the bias will be small or zero. Indeed, in this case, the difference in exam performance among the two groups if everybody sleeps less than 8 hours should be same, as students have similar abilities.
- If the groups are dissimilar (e.g., control students are all better students), then the bias will be different from zero. Indeed, if students in the control group are "better students", if all sleep less than 8 hours, students in the control group will perform better in average, but this has nothing to do with sleeping less or more or with cause-effect relationships among the observed variables.

Note that, given the observations above, we now know **when association allows to establish a cause-effect mechanism**. This happens when the bias is zero:

$$BIAS = E[Y(0)|T=1] - E[Y(0)|T=0] = 0$$

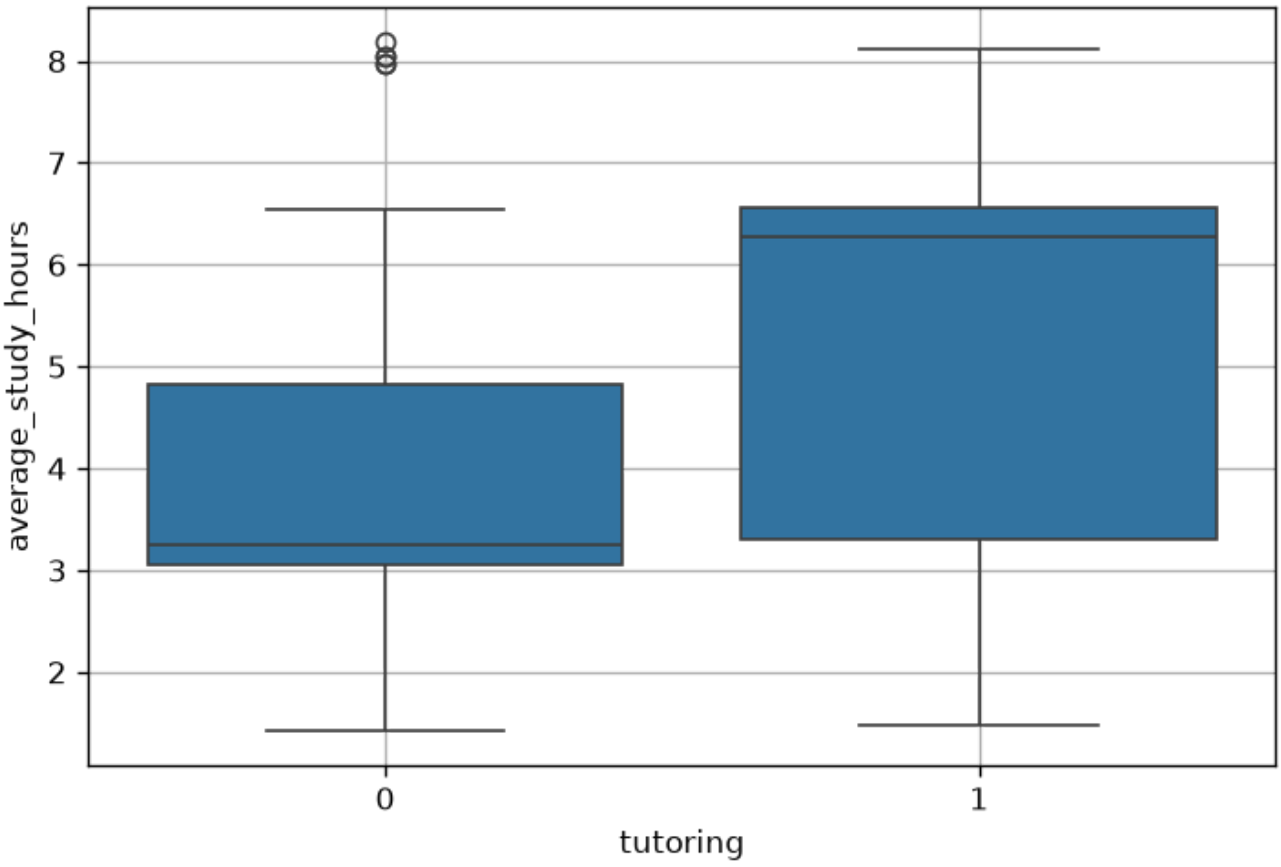
That is to say, when the difference between treated and untreated is zero (or at least very small).

Given this intuition, we now try to estimate the students' abilities by asking how many hours they study per day. We get the following data:

	tutoring	performance	average_study_hours
0	0	76.432447	3.316778
1	1	91.550461	8.025442
2	1	85.107378	6.433760
3	0	69.981030	4.758812
4	1	70.111151	3.151239
...	...	...	...
195	0	68.477160	3.124462
196	0	87.783594	6.338848
197	1	88.220111	7.859334
198	1	93.399423	7.907677
199	1	84.082421	6.264832

200 rows × 3 columns

We cannot measure bias from this data because we do not have access to counterfactual observations, but, now that we have access to a proxy for students' abilities, we can try to see if, in the two groups, abilities are equally distributed. Let's see this with a boxplot:

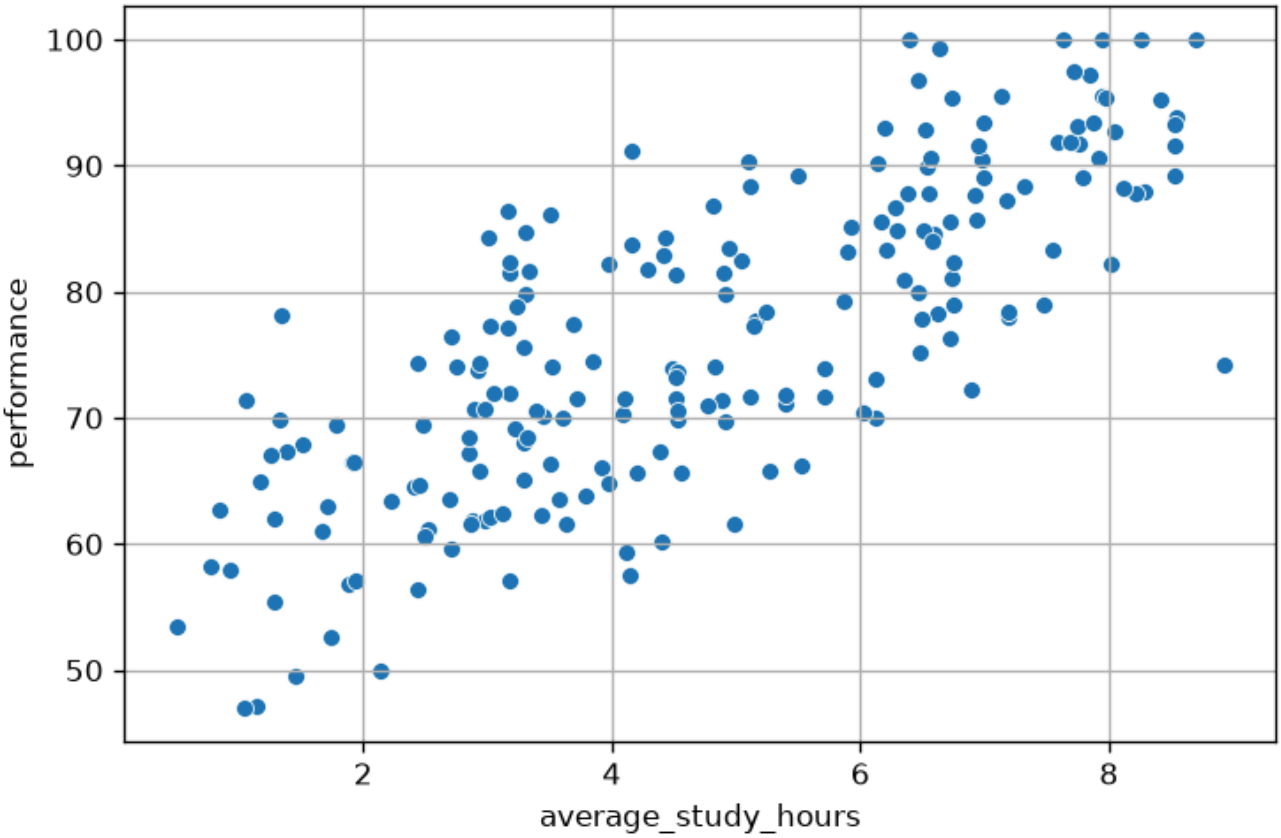


From the picture above, we see that students' skills are not equally distributed in the two groups. **This is not enough to say that the bias is different from zero, as we are not observing counterfactuals,**

but we now see that the difference in outcomes could be due to two different causes:

- Tutoring improves exam performance.
- Some of the difference may be due to the fact that students following the tutoring program are also students who study more in average (**maybe they are more motivated, so they study more and also choose the tutoring program**) and this may affect the final performance.

We can check the hypothesis that average study hours affects performance with a scatterplot:



From the picture above there seem to be a correlation between the two!

Now that we have an intuition into what may make correlation be not indicative of causation, let's see how to reduce bias to establish cause-effect relationships.

Randomized Controlled Trials (RCT)

We have seen that correlation is causation when the treated and control groups are comparable for everything except the treatment. The most robust method to remove this bias is via **randomized experiments**, or **Randomized Controlled Trials** (RCT).

A randomized experiment **randomly assigns individuals in a population to a treatment or a control group**. By performing a random assignment, we wish to make the two groups indistinguishable. Of course **one of the two groups will take the treatment and this will likely make the two groups distinguishable**, but we wish to make sure that, **apart from the treatment, the two groups are indeed indistinguishable**.

It can be shown that random assignment makes sure that the potential outcomes are conditionally independent given the treatment:

$$Y(0) \perp Y(1)|T$$

So, if the treatment T is fixed, the potential outcomes are independent. For instance, if we fix $T = 1$ (so we are taking subjects who attended the tutoring), then knowing the observed value $Y_i(1)$ (i.e., the observed exam performance of student i), does not tell me anything about the potential outcome $Y_i(0)$ (the exam performance that the same student would obtain if they did not attend the tutoring).

> Recall our example on conditional independence: if height and vocabulary are conditionally independent given age, this means that height and vocabulary are independent in the same age groups, while they are not across age groups, so **age is the only thing that make height and vocabulary dependent**. Similarly, if $Y(0) \perp Y(1)|T$, then **the treatment is the only thing generating a difference between the outcome in the treated and in the control group**.

If this conditional independence is valid, then:

$$E[Y(0)|T = 0] = E[Y(0)|T = 1] = E[Y(0)]$$

and:

$$E[Y(1)|T = 0] = E[Y(1)|T = 1] = E[Y(1)]$$

Which leads to:

$$E[Y(1)|T = 1] - E[Y(0)|T = 0] = E[Y(1)] - E[Y(0)] = E[Y(1) - Y(0)] = ATE$$

Hence, in this case, **correlation would be causation**.

Randomizing tutoring

We now have a tool to remove the bias probably arising from our prior experiment. Let's say we could repeat the experiment, now with random assignment.

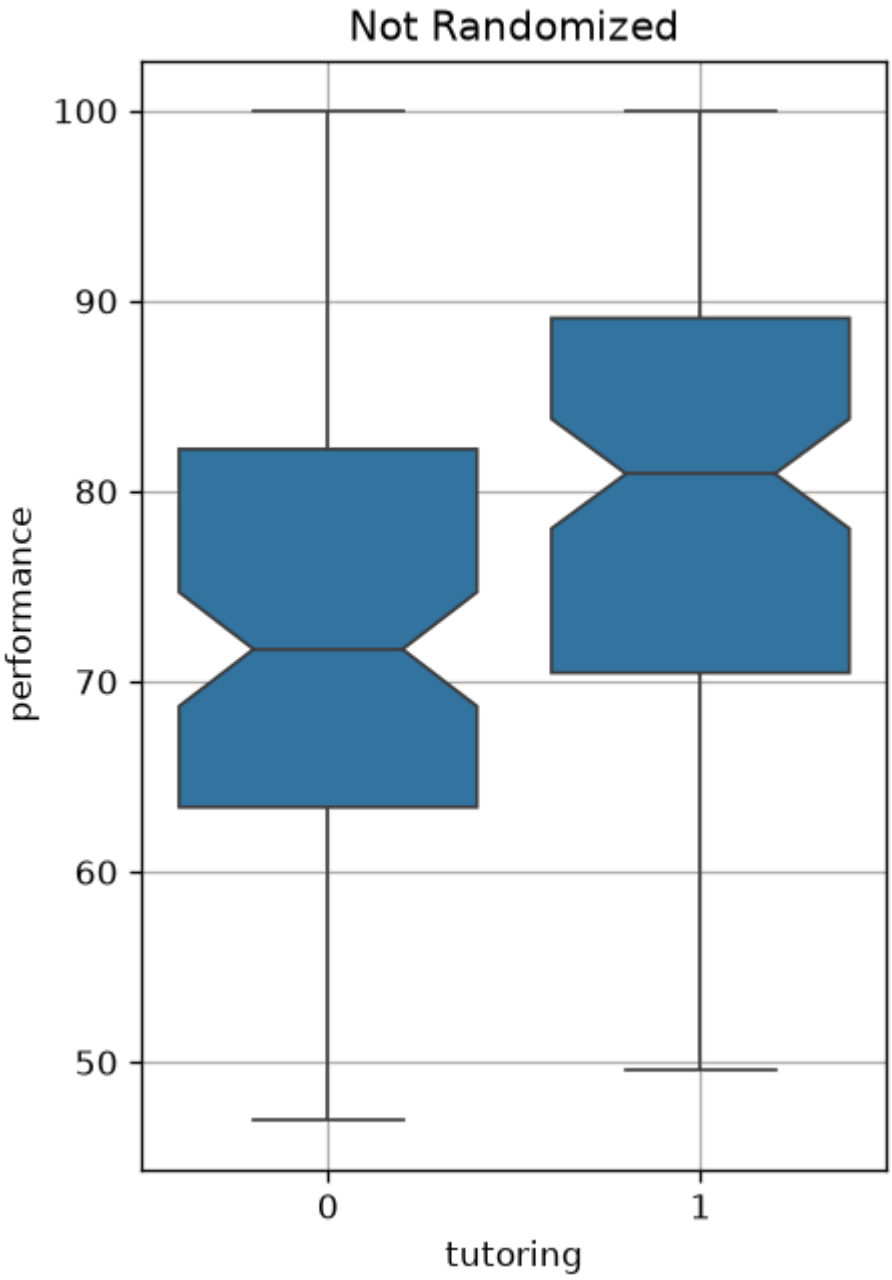
Word of caution: this is not always possible. For instance, in this case, it would not be very polite to "force" students to enroll programs, so we may not be able to carry out this experiment, but let us assume for a moment that we can.

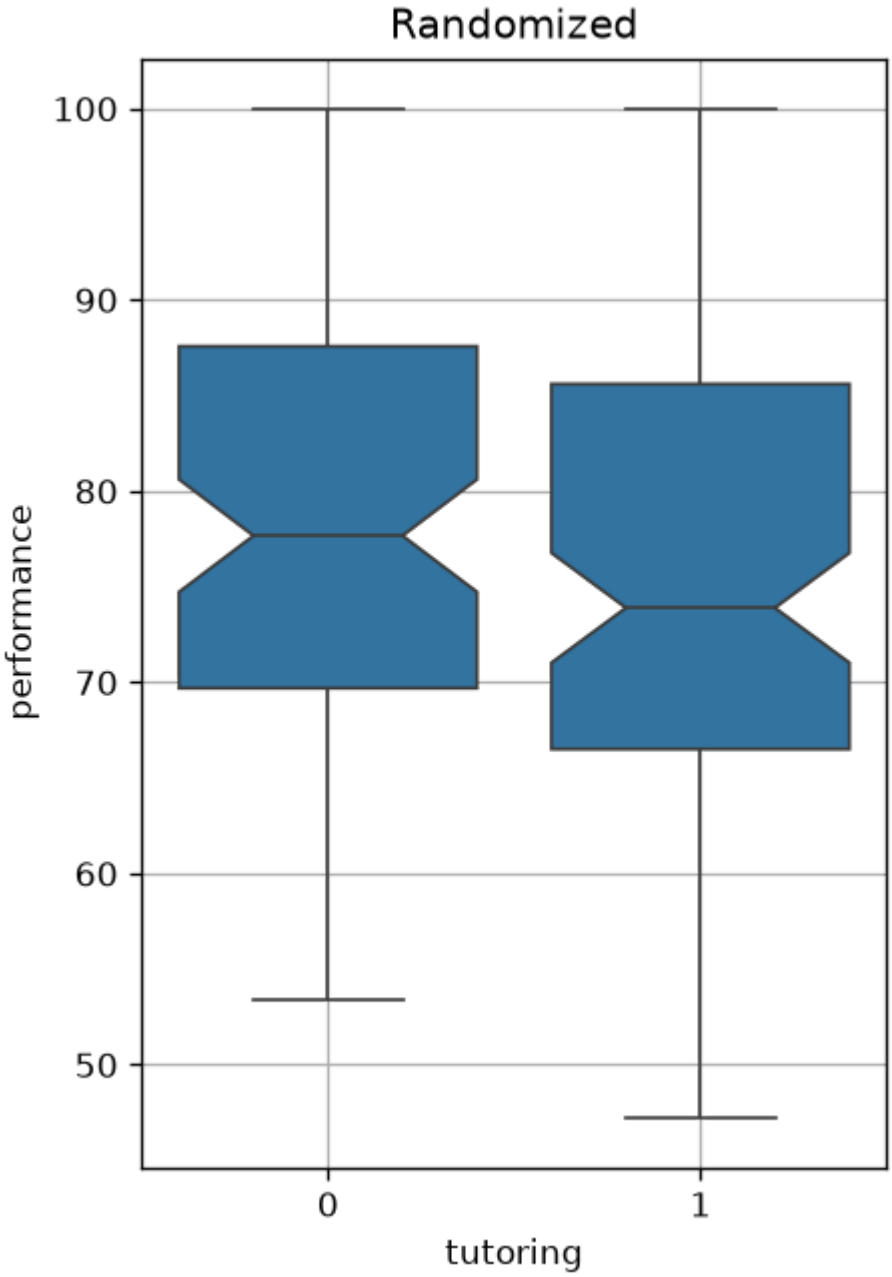
We repeat the experiment and obtain these results:

	tutoring	performance
0	0	76.432447
1	1	91.550461
2	1	85.107378
3	0	69.981030
4	1	70.111151
...	...	...
195	0	68.477160
196	0	87.783594
197	0	87.970111
198	1	93.399423
199	1	84.082421

200 rows × 2 columns

Let's visualize the boxplots:


Not Randomized
100
90
80
70
60
50
performance
0
1
tutoring



We now see a less dramatic (and actually negative!) improvement. What happened?

By performing a random assignment, we got rid of selection bias. Students voluntarily choosing to enroll in the programme were probably more motivated and keen to study. We now see that the impact of tutoring is very small. We can perform a t-test to assess if this is statistically significant:

```
T-test results - T-statistic: -1.1327518742165315, p-value: 0.2586882978200425
```

We got a large p-value, the difference is not statistically significant.

Causal Effect and Observational Studies

We have seen how to interpret an observed association as a cause-effect relationship we need to reduce bias. The gold standard is to use randomized controlled trials, however **this is not always**

feasible or ethical. Consider for instance **the problem of estimating the effect of smoking on the development of a given disease**. To perform a randomized experiment, we should select subjects randomly in two groups and ask people in one of the two groups to smoke. **This is of course unethical, considering that we suspect that smoking has a bad effect on health**.

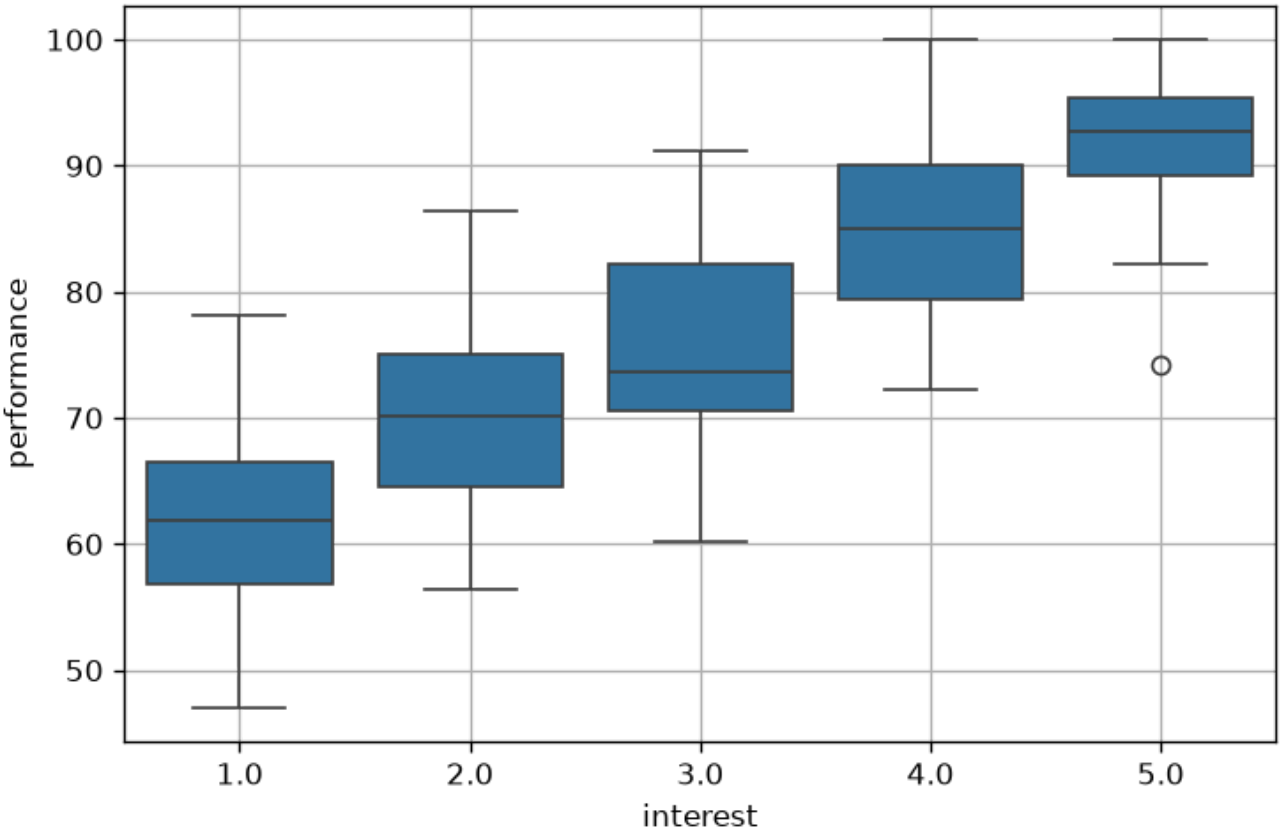
In these cases, we can only resort to **observational data**: we collect data of smokers and non-smokers and observe association in the data. However, we know that this process is **subject to bias**. How do we deal with it? In this part of the lecture, we will see that **graphical causal models** give us a framework to **set our believes on where bias come from**. If we can accurately model the source of bias, we can remove it by **controlling** on the variable "causing" the bias.

Let us consider our past example: students and tutoring. Let us assume that we cannot perform the random assignment (again, it's unethical!). Since we imagine that **the interest of students towards study can be a source of bias**, we now ask students to also tell us how interested they are in studying in general in a scale from 1 to 5. We obtain the following observations:

	tutoring	performance	interest
0	0	76.432447	2.0
1	1	91.550461	5.0
2	1	85.107378	4.0
3	0	69.981030	3.0
4	1	70.111151	2.0
...	...	...	...
195	0	68.477160	2.0
196	0	87.783594	4.0
197	1	88.220111	5.0
198	1	93.399423	5.0
199	1	84.082421	4.0

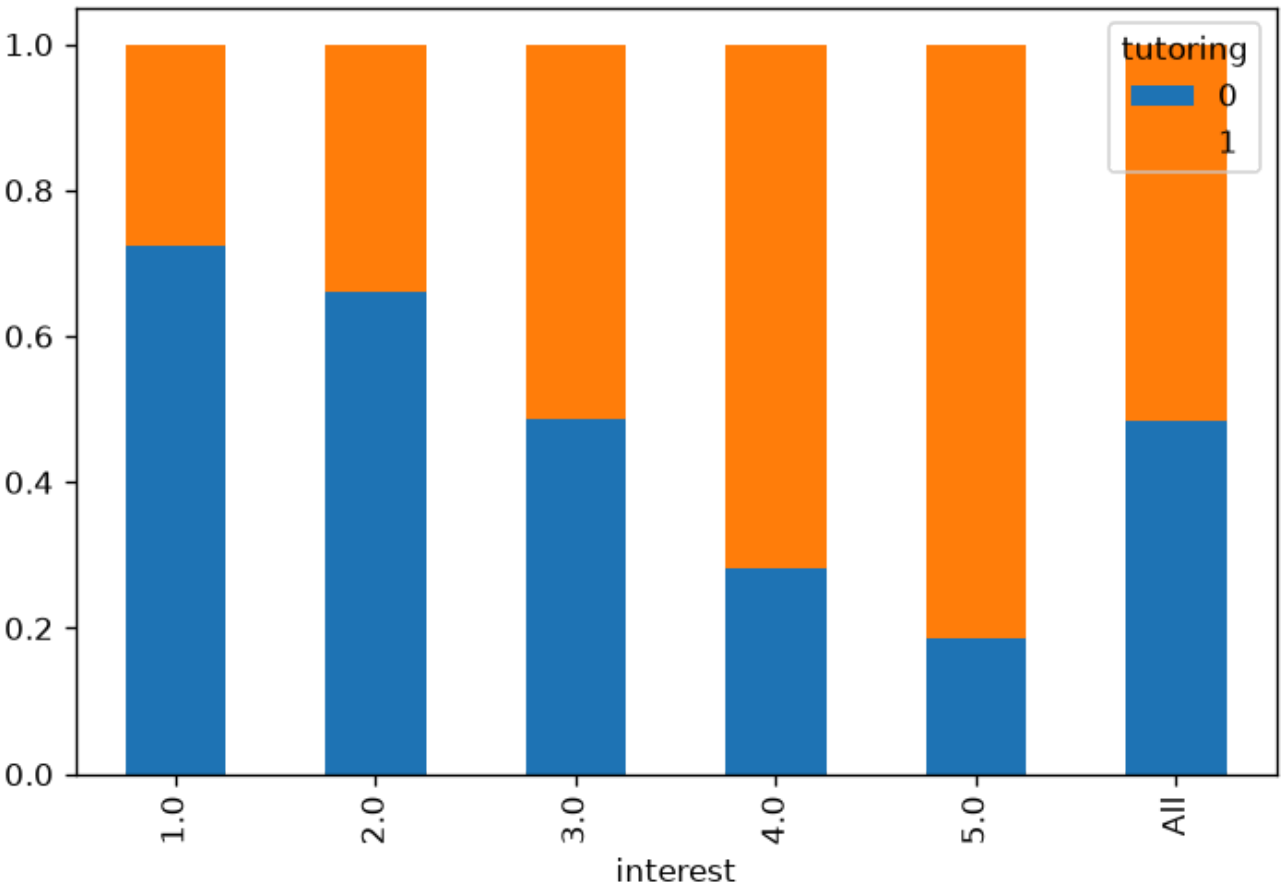
200 rows × 3 columns

These are the same observations as the first case (before random assignment), but with an additional variable. We suspected that **interested students have an average higher performance and that interested students are more likely to enroll in the tutoring program**. Let us see graphically if this is true. If we split the data by interest and compute the boxplots of performance, we get the following graph:

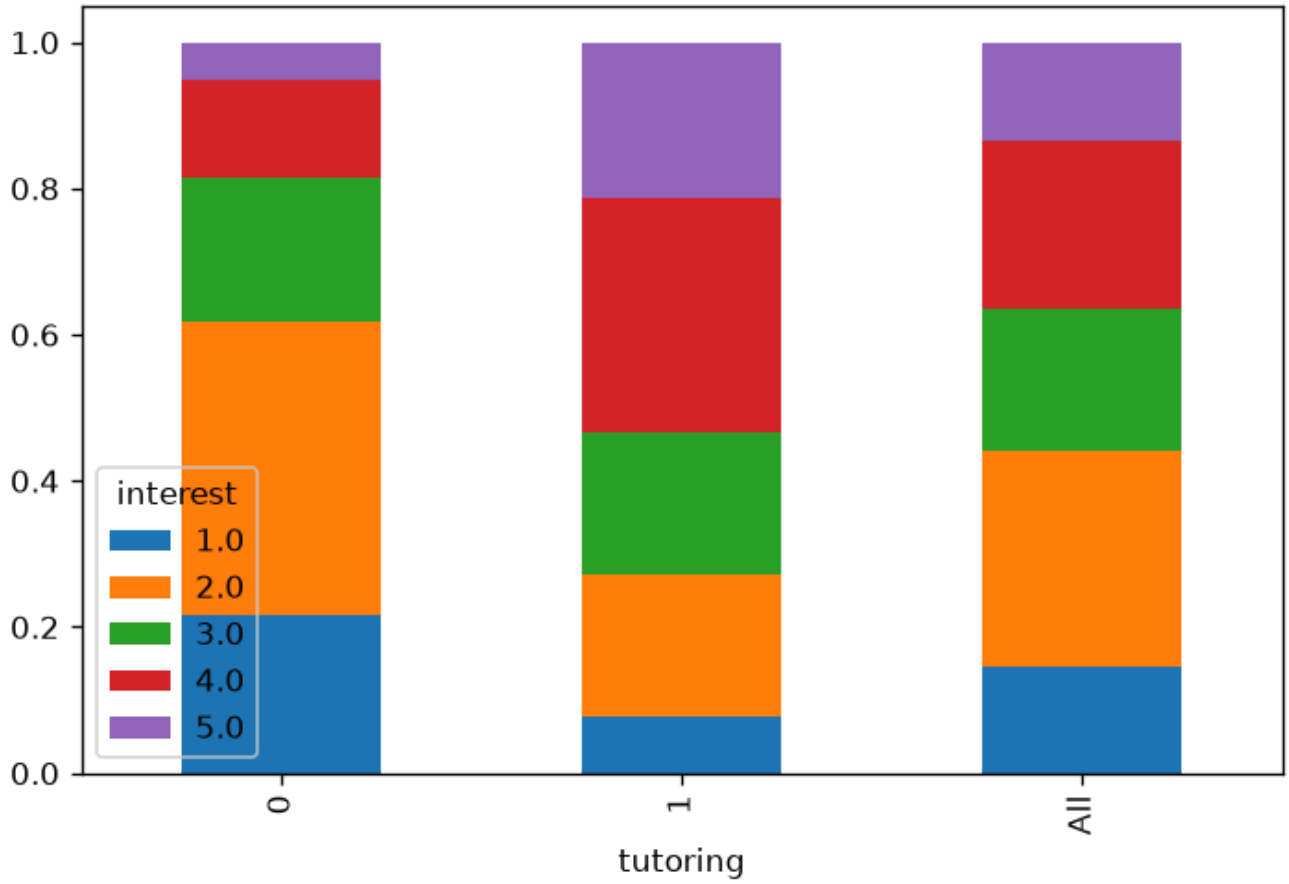


We were right! (ok, this is an exaggerated picture, but this example is made up...)

Let us check how correlated are `interest` and `tutoring` with a barplot:



We can also check the distribution of interest in the two groups:



We can go ahead and compute a χ^2 test of independence to check if the association between the two variables is statistically significant:

```
Chi-square statistic: 31.22
Cramer V statistic: 0.40
Chi-square p-value: 2.761663e-06
```

We were right also in this case! Interest affects tutoring, hence interest is our source of bias!

The correlation between the `interest` variable and `tutoring` is making the "treated" (`tutoring=1`) and the "control" (`tutoring=0`) groups not comparable. Indeed, if people in the treated group have more chances of getting a higher response value (performance), then no wonder that we observe such dramatic differences among the two groups! In other words, **the effect we observe may be due to the bias introduced by `interest`, the effect of `tutoring`, or a combination of both**.

Controlling for `interest`

How do we make the groups more comparable? Recall that we want:

$$Y(0) \perp Y(1)|T$$

We have seen that this is not true because of `interest`. Indeed, if we pick a person in the group $T = 0$ with a high $Y(0)$, we can imagine this is due to a large value of `interest`, so we can easily

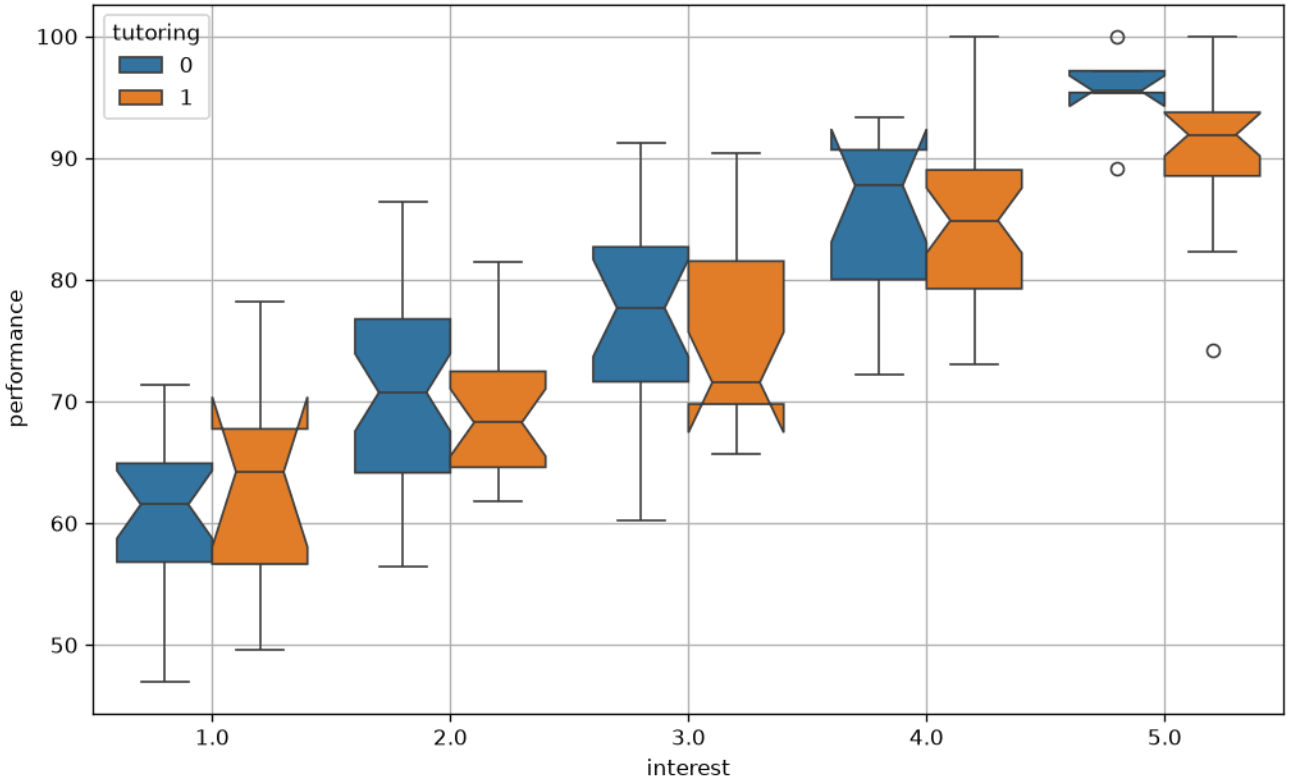
predict that $Y(1)$ will also be large. **If we can predict $Y(1)$ from $Y(0)$, then they are not independent (conditioned on $T = 1$ in this case)**.

To obtain the statement above, we should **perform a random assignment**, which we cannot do. However, we know that `interest` is the source of bias. Let us assume that it is the **only source of bias**. Then, if X represents `interest`, we can say that:

$$Y(0) \perp Y(1)|T, X$$

Indeed, if we pick a person from a group $T = 0$ and with a given interest value $X = 4$, then, given a large $Y(0)$ we cannot really say anything about $Y(1)$. Indeed, if $X = 4$ and $Y(0) = 80$, we have a student with great interest and high performance. What can we say about $Y(1)$? We could say that it will be high (the student has high interest), but any other student in this group will probably have a high score, so we cannot predict anything which will be significantly different from random (i.e., from the score of another random student within this group).

But, how do we condition on X? Well, the easiest way to do it, is to **divide the data into different groups based on the values of X and assess any association within the groups**. Let us do it:



In the plot above, we compare the effect of tutoring in the different `interest` groups. As we can see, the association between `tutoring` and `performance` tends to vanish within the groups. This is shown by the fact that the notches (confidence intervals) tend to intersect. We can get a clearer picture by performing 5 different two-sample statistical tests assessing whether the means of `performance` in the two groups differ in a statistically significant way:

	t_statistic	p_value
interest		
1.0	0.934780	0.358187
2.0	-0.781177	0.437931
3.0	-0.867406	0.391310
4.0	0.164684	0.869947
5.0	-1.383074	0.178871

As we can see, there is no significant association between `performance` and `tutoring`, which suggests the lack of a causal effect.

The act of conditioning on X (e.g., by dividing into groups) is called "controlling for X".

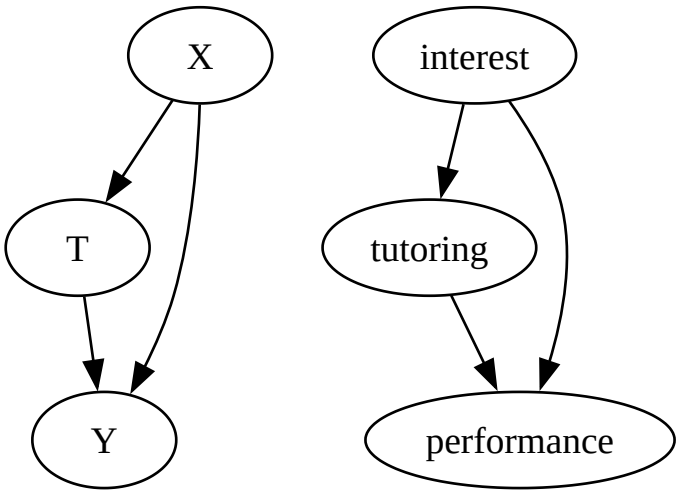
Graphical Causal Models

We have seen a simple case in which we could identify the main source of bias which was leading to an observed association not implying a cause-effect mechanism. However, not all cases are so simple and **more than one variable may be responsible for bias**. The question now is

> How do we identify the variables we should control for?

To help us in this quest, we can use graphical causal models. A graphical causal model is a **directed acyclic graph** formalizing cause-effect relationships between variables. **Each node is a random variable, while a directed edge $A \rightarrow B$ indicates that A causes B.**

The causal graph describing the hypotheses we made in this example will look like this:



Indeed, we expect `interest` (X) to influence both `performance` (Y) and whether a subject belong to the treated or control group, i.e., the `tutoring` variable (T). From the model above, we can identify

`interest` as a source of bias, but **as we will see shortly, we can use a set of rules to identify which variables we should control for or not control for from a graphical model**.

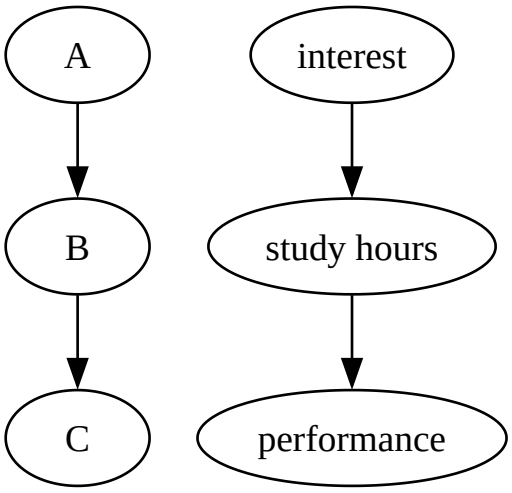
Before proceeding, a short note on **how graphical models are created**: these are generally designed by the data analyst who encodes their own **beliefs** on how the world works. These come from a "Bayesian" view of the world in which we first establish our beliefs and then use a theoretical framework to draw conclusions from them.

Main Structures of Causal Graphical Models

Once we form a graphical model, we can identify a set of standard structures which will allow us to choose which variables to control for. We will see the main structures, but others beside the ones discussed here may exist.

Chains

The most straightforward structure is a chain of this kind:



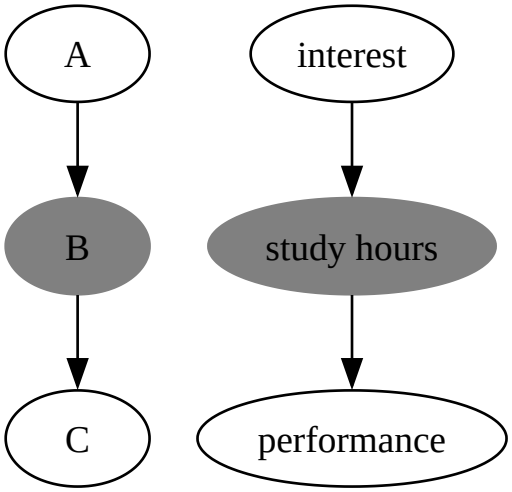
The example above, `interest` in studying causes the number of `study hours`, which in turn causes `performance`. Let us imagine no other variable is involved (i.e., `interest` is the only variable causing `study hours` and `performance` is only caused by `study hours`). Then we will easily find that:

- `interest` and `study hours` will be associated;
- `study hours` and `performance` will be associated.
- `interest` and `performance` will be associated.

Indeed, for instance:

- If a person has a great interest, then they will study a lot and have good performance;
- If a person has good performance, we can infer that it studies a lot and hence has great interest;
- If a person studies a lot, we infer that it has great interest and good performance;

Let say we **control for** B, `study hours`. We will denote the graph as follows (variable we are controlling for is in gray):



If we **control for B, `study hours`, we will find that `interest` and `performance` are not associated anymore**. Indeed:

> If we consider a group of people who study the same amount of hours and I take a person with good performance, then I don't know if they have great interest or not. Or better, I cannot distinguish a person with great interest from a person with little interest as they all study the same amount of hours.

In mathematical terms, we know that in general:

$$A \not\perp C$$

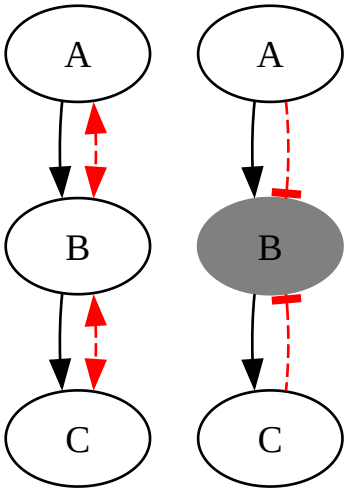
but, by controlling for B, we obtain:

$$A \perp C | B$$

Hence, in groups with the same value of B, we will see an independence between A and C.

In general, we say that **conditioning on B blocks the dependency between A and C**, hence **while there is a causal effect between A and C, we cannot see it anymore when we condition on B**.

The graph below shows two identical graphs, except that we are conditioning on B in the version on the right. Besides solid black arrows denoting cause-effect relationships, we also show **in red how dependency flows**.



- In the graph on the left, dependency flows in a bidirectional way **through variable** B. This means that A depends on B and C and similarly C depends on B and A;
- In the graph on the right, conditioning on B "stops" the dependency. Now:
  - C and A are independent of B. This makes sense because conditioning on B means fixing a value for B, so A and C will be independent of the constant;
  - A and C are independent (we are conditioning on B, so we have $A \perp C|B$).

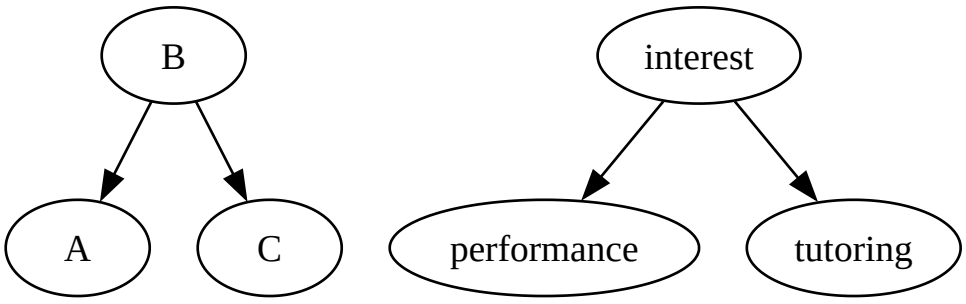
As a result, if we want to study the relation between A and C, we **should not condition on B**, as this conditioning will make the association between A and C vanish, even if there is a causal effect between these variables.

We are encountering an important concept:

> Conditioning on **any** variable is not a good way to remove bias. If we condition on the "wrong" variable, we may introduce some bias which was not there in the first place and estimate the lack of a causal effect even in cases in which the causal effect is there.

Forks

We have a fork structure when a variable causes two other variables. Let us see an example:



In this case B (`interest`) is causing both A (`performance`) and C (`tutoring`). This is the example we have seen before. In this case, as we have seen, we may see a correlation between `tutoring` and

`performance`. Indeed, a large value of `performance` will probably mean that `interest` is high, hence `tutoring` will probably be equal to 1.

> We will call B (or `interest`) the **common cause**.

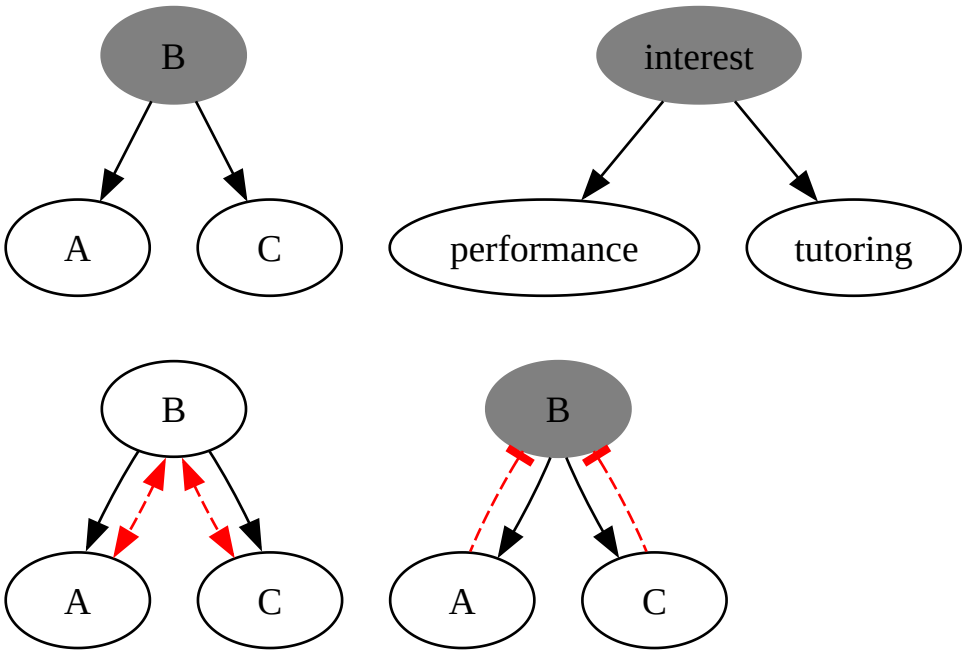
Although we will likely observe correlations between any pair of the A, B, and C variables, we know from the graph that there is no cause-effect relationship between A and C.

We say that the fork structure **creates a backdoor path such that dependence (or association) flows through B (the backdoor) to go from A to C**.

As we noted in the previous example, conditioning on B (`interest`) makes the association between A and C vanish:

$$A \perp C | B$$

Hence in the following graph, we will observe no association between `performance` and `tutoring`, hence confirming the lack of a causal effect:

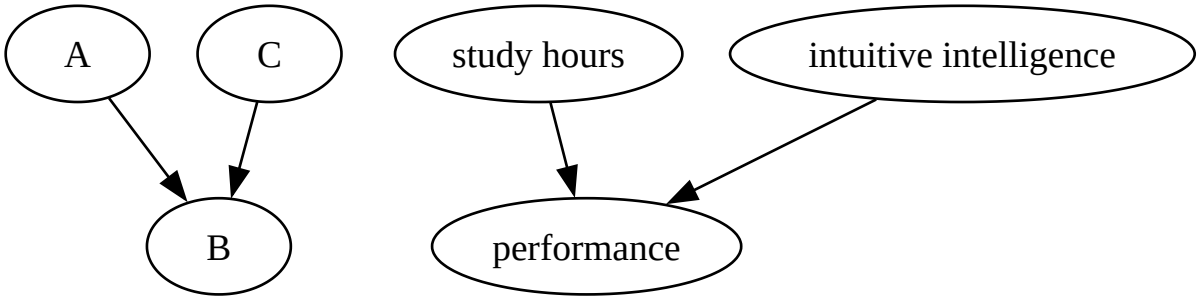


When we do not condition on B, dependency (hence association) flows from A to C through B. Hence, **we observe an association which is not due to a causal effect**.

When we condition on B, dependency (hence association) does not flow through B, so A and C will be independent and we will measure no association, as it should be.

Colliders

The last, important structure, is a collider. This is in some sense the opposite of a fork: a variable is caused by two different variables. Let us see this with an example:



The graph above tells us that performance is caused by `study hours` and `intuitive intelligence`. We are making the assumption here that `intuitive intelligence` does not affect `study hours` in any way. This might be inaccurate, but we will anyway assume so for the sake of a simple example.

This structure is called a **collider**, because two arrows (from A and C) collide on a single node (B).

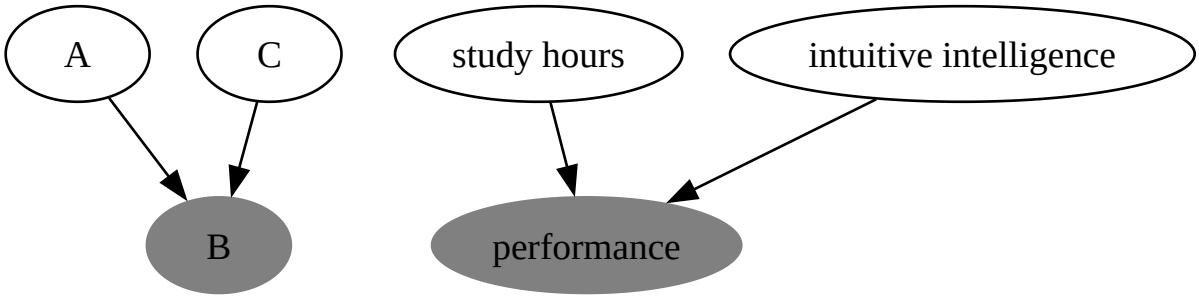
A collider **blocks dependency**. This means that by default, if we do not condition on `performance`, we will observe no association between `study hours` and `intuitive intelligence`. Indeed:

- If we know that a person studies a lot, we cannot really say anything about their intuitive intelligence;
- If we know that a person has intuitive intelligence, we cannot say how much they study.

So, in the graph above:

$$A \perp C$$

Let us not consider a case in which we condition on B (`performance`), as shown in the following graph:



Conditioning on `performance` changes things. Indeed:

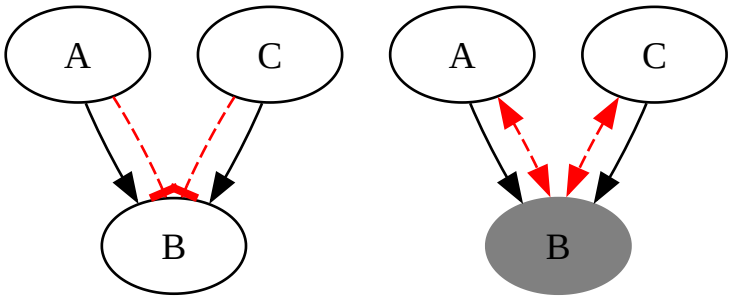
- If we know that a student has a **good performance** (i.e., we consider the group of good performing students when controlling), then **knowing that they do not study a lot, will tell us that they probably have good intuitive intelligence**. Indeed, if the fixed performance cannot be explained by the large number of study hours, it must be due to good intuitive intelligence;
- Conversely, if we know that a good performing student has bad intuitive intelligence, then their performance is probably due to a large number of study hours.

In practice, conditioning on `performance` makes `study hours` and `intuitive intelligence` dependent:

$$A \not\perp C|B$$

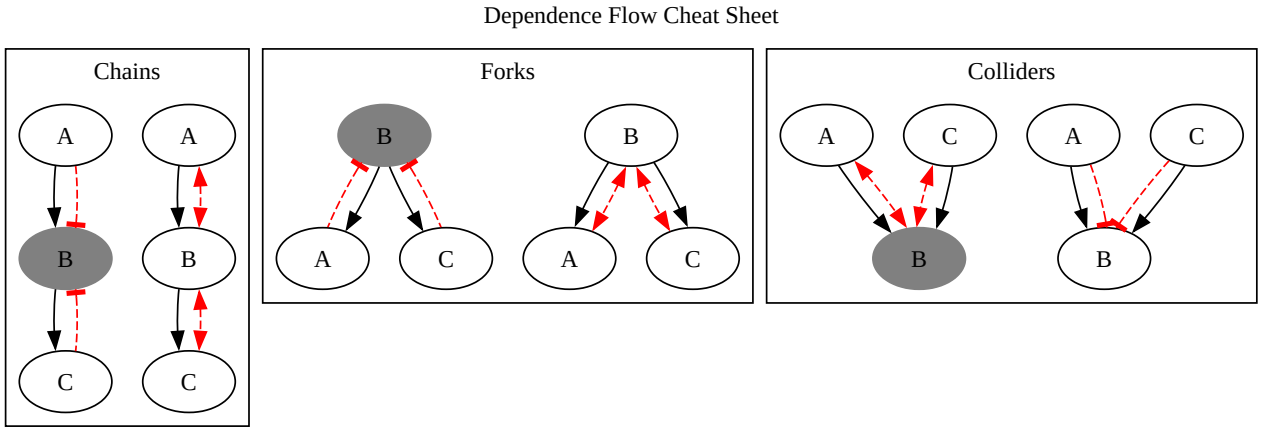
This phenomenon is called **explaining away**, because, one we fix B (`performance`), knowing A (`study hours`) already explains B, hence making C less likely.

In general, **a collider blocks dependency, while conditioning on it unblocks a dependency path**. We can use the same conventions as before to illustrate this effect:



Dependence Flow Rules

Using these structures, we can determine if two nodes A and B are dependent by checking **if an unblocked path exists between A and B**. This can be done by combining the rules we have seen above, which are also summarized in the following figure:



The cheat sheet above is replicated from here and here. You can find more information at those URLs.

We will say that a path between A and B is blocked if:

- It contains a non-collider that **has been conditioned on**;
- It contains a collider that **has not been conditioned on** and has no descendants that have been conditioned on.

Let us see an example from here and let us try to answer some questions:

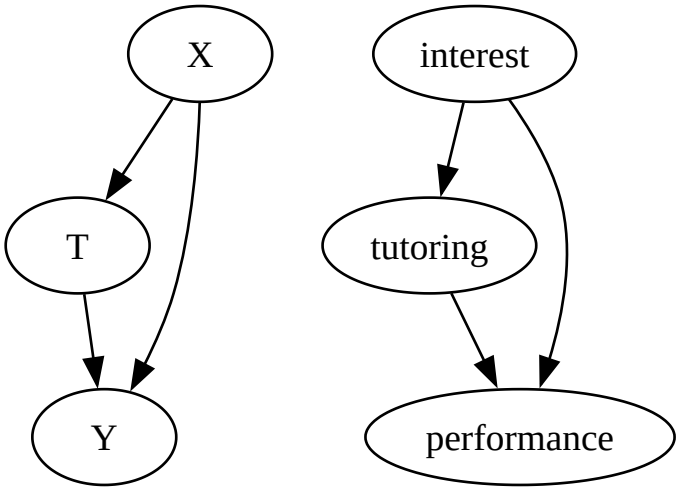
> ⓘ **Working through d-separation by hand**
>
> Deciding whether two nodes are independent given a set of others is a mechanical procedure — trace every path, block it or not according to the three rules above — and it is worth doing on paper a few times. The exercises at the end of the chapter ask you to. In practice you will rarely do it by hand: what matters for an analysis is recognising confounding and selection bias, which is what the next section is about.

Using Graphical Causal Models to Diagnose Bias

We will now see how using graphical causal models allows to diagnose bias and understand which variables we should condition on and which we should not when trying to measure a causal effect through correlation. We will see that we have two main sources of bias: **confounding bias** and **selection bias**.

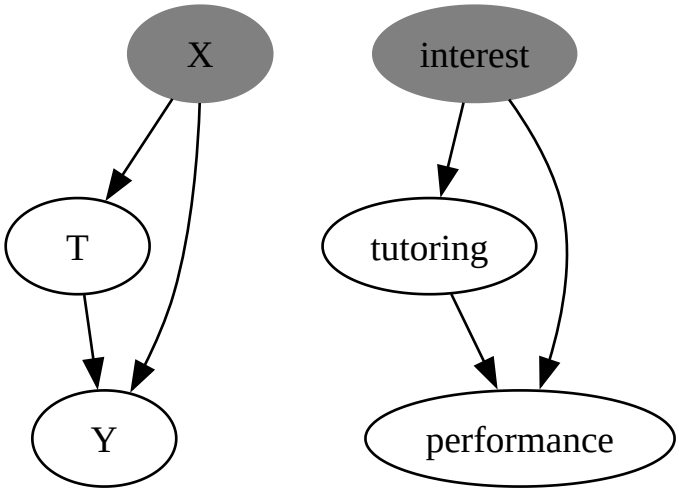
Confounding Bias

Confounding bias happens when **the treatment and the effect have a common cause**. Let us consider the following example, which we already saw previously:



Here, the outcome Y is `performance`, which is caused by both the treatment T (`tutoring`) and another common cause X (`interest`). The common cause X is called **a confounder** because it mixes ups the effects of `tutoring` on `performance` and `interest` on `performance`. Indeed, if we measure the correlation between `tutoring` and `performance`, we obtain a large number, but we do not know if this is due to the common cause `interest`. Maybe, people choose tutoring because they are interested and obtain good performance because they are interested as well.

If we want to be able to measure the **direct effect between the treatment and the outcome**, we need to **close the backdoor path between Y and T**. This is done by **conditioning on the common cause**:



Note that this corresponds to comparing subjects with the same level of interest. In this case, we will observe the direct effect of `tutoring` on `performance`, since all students in the considered group have the same level of `interest` (so any association between `tutoring` and `performance` is given by the causal effect). By conditioning on X, we obtain:

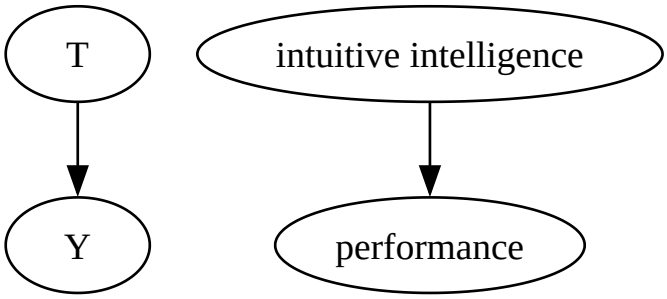
$$Y(0) \perp Y(1)|T, X$$

Selection Bias

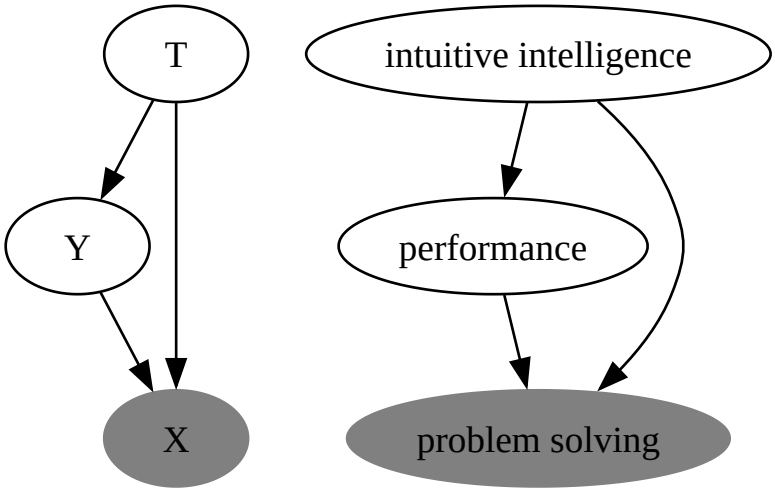
While confounding bias happens when we do not condition on a common cause, selection bias happens when **we control for too many variables**. While one may think that it is a good idea to control on any variable "just in case", this is not true in general and can lead to **selection bias**.

Controlling For a Common Effect

Let us consider the following example in which we want to measure the causal effect of `intuitive intelligence` on `performance`:

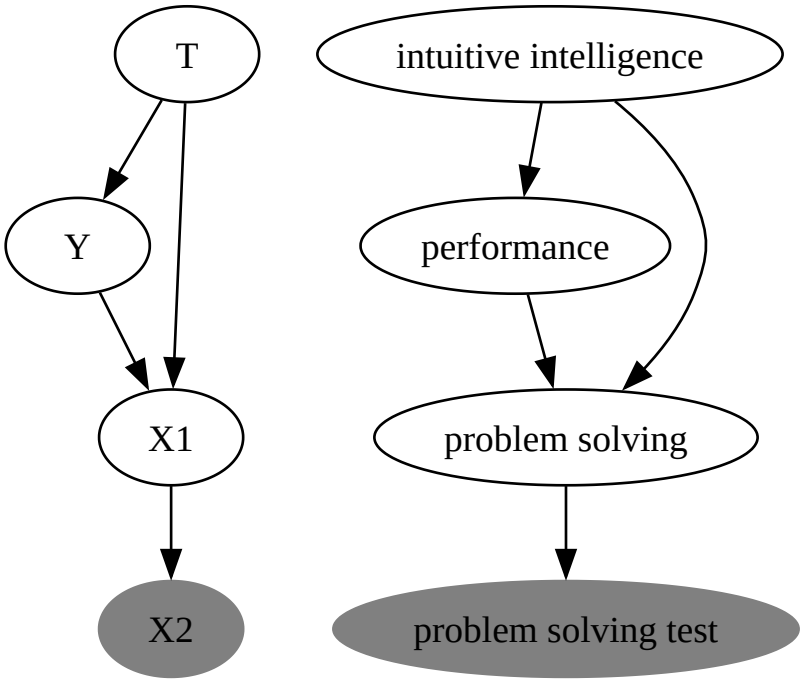


In our data, we also have a score of a test on problem solving. **"Just to be sure", we condition on** `problem solving`:



The problem is that `problem solving` is a common effect of `performance` and `intuitive intelligence`. By conditioning on the common effect, we are introducing a form of bias. Indeed, **if we look at people with good problem solving abilities, we are probably looking at people with either good performance or good intuitive intelligence**. Indeed, knowing that the person has good performance **explains away whether they have intuitive intelligence**, hence showing a correlation which does not represent a causal effect. **By conditioning on the common cause, we opened a backdoor path between Y and T which affects the perceived association between treatment and outcome**.

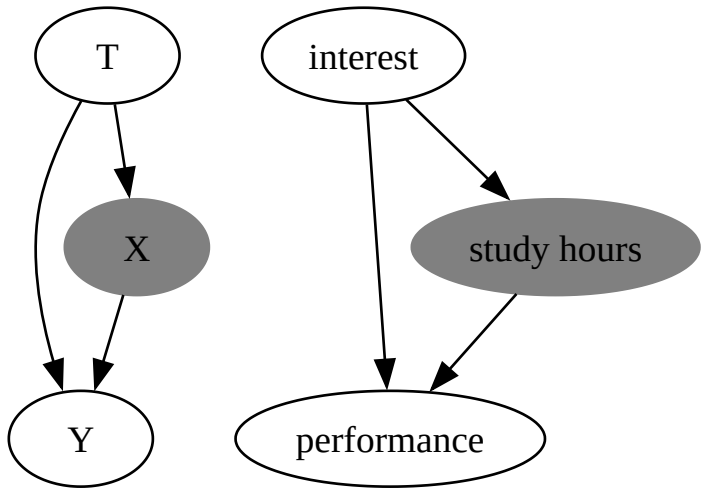
Note that the same is true if we condition on a descendant of the common cause. For instance:



Controlling on a Mediator

Another case of selection bias, arising from cases in which we control for variables we should not control on, is given by the case in which we control on a mediator of the treatment and the outcome.

Let us consider this example:



We want to assess the effect of `interest` on performance. Let's say we could randomize interest (for instance, we selected students with different levels of interest randomly across different schools). **"Just to be sure", we condition on** `study hours`. Conditioning on the mediator closes a path between `interest` and `performance`, hence reducing the measured effect between `interest` and `performance`.

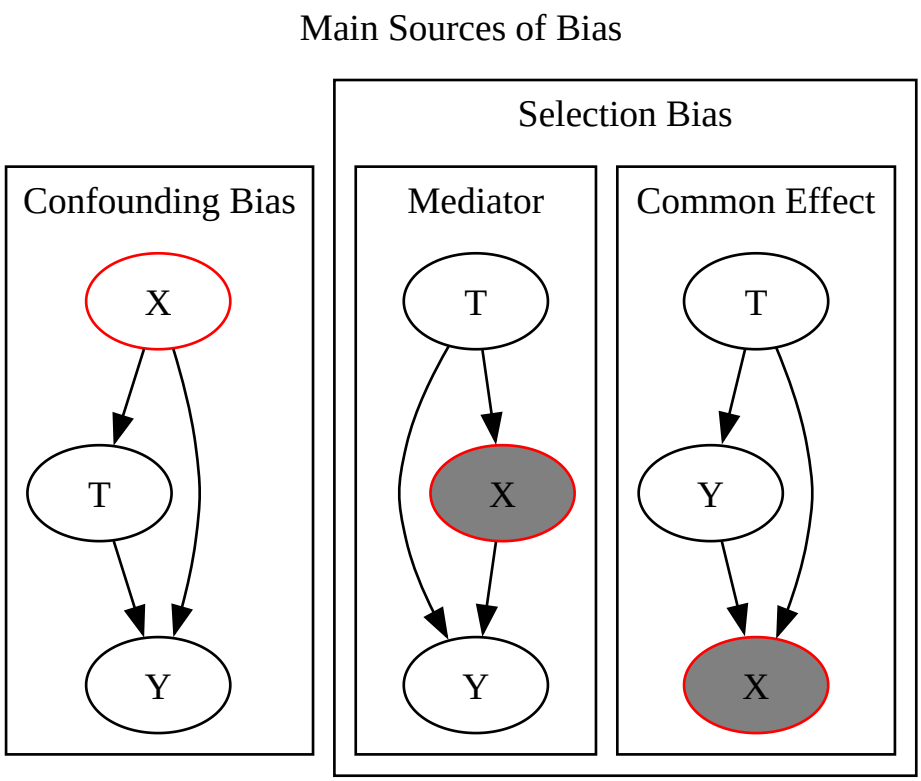
Indeed, while we expect that interested students will study more and obtain better performance, by observing only people who study a lot, `interest` **and** `performance` **may seem to be little correlated. Indeed, even people with little interest will obtain good performance if they study for long hours.**

Main Sources of Bias

To summarize, the main sources of bias are three:

- **Confounders**: we should control on the common cause;
- **Selection Bias - Common Effect**: we should not control on the common effect;
- **Selection Bias - Mediator**: we should not control on the mediator.

These are shown in the graph below:

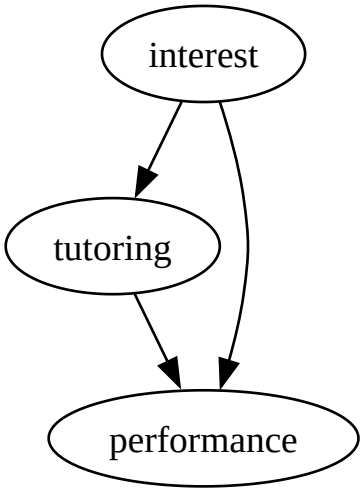


In the figure above, we mark in red the nodes that are not conditioned on, but should be conditioned on, and the nodes that are conditioned on, but should not be conditioned on.

Linear Regression for Causal Analysis

We have seen how to choose a suitable set of variables to control for when we aim to establish a cause-effect relationship. When we said "we control for X", we implied that we can divide the data in groups and measure association within each group, to then aggregate results and obtain an average effect.

In this section, we will see how we can automate this process with **linear regression**. Let us consider our previous example in which we wanted to measure the causal effect of `tutoring` on `performance`, when we know that `interest` can be a confounder:



This is the data:

	tutoring	performance	interest
0	0	76.432447	2.0
1	1	91.550461	5.0
2	1	85.107378	4.0
3	0	69.981030	3.0
4	1	70.111151	2.0
...	...	...	...
195	0	68.477160	2.0
196	0	87.783594	4.0
197	1	88.220111	5.0
198	1	93.399423	5.0
199	1	84.082421	4.0

200 rows × 3 columns

Recall that in this case the assignment is not random, so we expect `interest` to influence both `tutoring` and `performance`. If we ignore this bias, we can compute the average difference in the effect of the treated and untreated:

$$E[performance|tutoring = 1] - E[performance|tutoring = 0]$$

We can do this in computational terms as follows:

```
students[tutoring==1]['performance'].mean()-students[tutoring==0]
        ['performance'].mean()
```

```
np.float64(-2.043673682064565)
```

Another relevant value to compute is the average performance of the untreated:

```
students[students['tutoring']==0]['performance'].mean()
```

```
np.float64(72.92912990981884)
```

We can see how these values are **automatically computed within a linear regressor** of the kind:

$$performance = \beta_0 + \beta_1 tutoring$$

These are the coefficients after fitting:

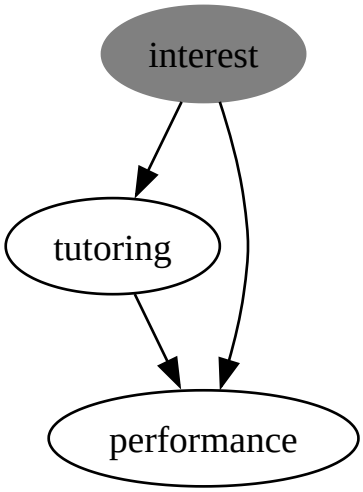
	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	72.9291	1.199	60.832	0.000	70.565	75.293
tutoring	6.8269	1.671	4.087	0.000	3.533	10.121

Coherently with the interpretation we previously gave of linear regression, we obtain estimates for those two values:

- `intercept`: is the average performance of the untreated, i.e., the value of `performance` we expect to see in average when `tutoring=0`;
- `tutoring`: is the increment of `performance` we expect to see in average when we observe an increment of one unit in `tutoring` (i.e., when passing from `tutoring=0` to `tutoring=1`). This is the average difference of `performance` when comparing the treated `tutoring=1` and the untreated `tutoring=0`.

Also, when we use a linear regressor, we get confidence intervals and the results of statistical tests "for free".

If we look at the result above, **there is non-zero correlation between `performance` and `tutoring`**. Indeed, the coefficient of `tutoring` allows us to observe that **students who enroll in the `tutoring` programme, increase their average `performance` by 6.8269 points** (remember that performance lies in the $[0, 100]$ range). However, we also know that this result is biased due to the confounder `interest`. To remove this bias, we should condition on `interest`:



We note that one way to **control for a variable is to include it among the independent variables in the linear regressor**, i.e., fitting a model like this:

$$performance = \beta_0 + \beta_1 tutoring + \beta_2 interest$$

The result is as follows:

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	54.1697	1.296	41.804	0.000	51.614	56.725
tutoring	-0.9516	1.129	-0.843	0.400	-3.178	1.275
interest	7.8097	0.441	17.711	0.000	6.940	8.679

We note that, after adding this variable, the values of the coefficients changed substantially. **We are in particular interested in the coefficient of `tutoring`, which is measuring the correlation between `tutoring` and `performance`**. We note that we can now interpret this value as follows:

> β_1 (coefficient of `tutoring`) is the increment we expect to observe in `performance` when `tutoring` passes from 0 to 1 **and** `interest` **is held constant**.

This last bit is the fundamental one: **by including the `interest` variable, we are effectively controlling for `interest`**. Indeed, the coefficient of `tutoring` now virtually looks at a subset of the data in which `interest` is constant. This is in practice the average effect of `tutoring` when comparing groups of data with constant `interest`.

Hence, **even if we cannot physically perform a random assignment, linear regression allows us to control for variables, hence enabling the analysis of observational data**. This is fundamental, as it is not in always feasible to perform random tests.

Getting back to our result above, we note that the coefficient of `tutoring` has a p-value of 0.4. This means that, while a value different from zero has been computed for it, the hypothesis test is telling us

that **we do not have enough elements to say that this value actually is different from zero**. Hence, we can conclude that `tutoring` does not have a statistically significant impact on `performance`.

Note that, **if we compute a linear regressor on the randomized set (i.e., the one in which we pretended we could perform random assignment)**

$$performance = \beta_0 + \beta_1 tutoring$$

we obtain these results:

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	72.9291	1.199	60.832	0.000	70.565	75.293
tutoring	6.8269	1.671	4.087	0.000	3.533	10.121

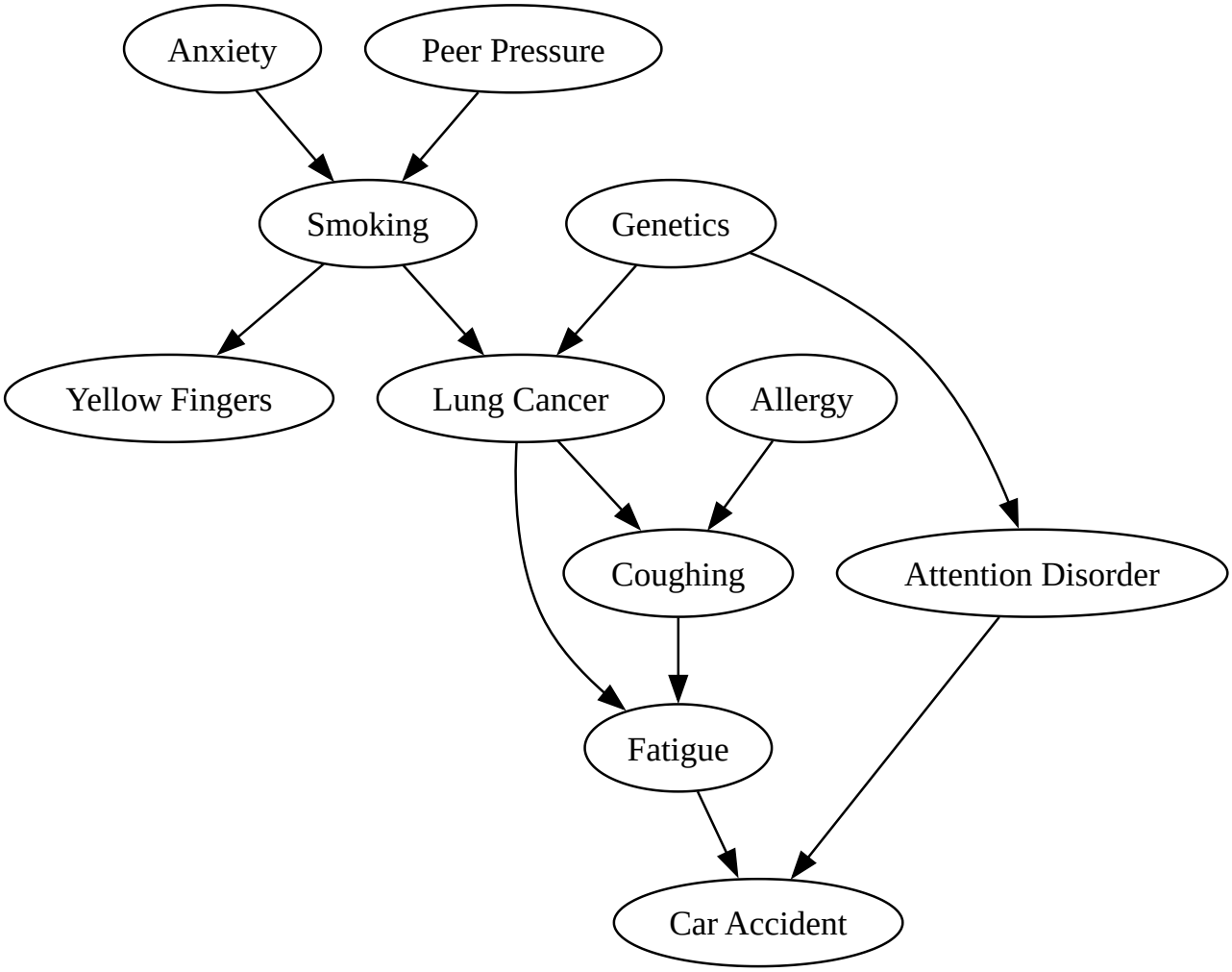
Coherently with the conclusions made with observational data, there is not a significant relationship between `tutoring` and `performance`.

Example on the LUCAS Dataset

We will see some examples on the LUCAS dataset, which can be found here:

https://www.causality.inf.ethz.ch/data/LUCAS.html

LUCAS is an **artificial dataset** in which data has been generated drawing values from known distribution to emulate a causal phenomenon. Besides the data, the authors propose the **following true causal graph** (which reflects the way data has been generated):



Let us have a look at the data:

	Smoking	Yellow_Fingers	Anxiety	Peer_Pressure	Genetics	Attention_Disorder	Born_an_Even_Day	Car_
0	0	0	1	0	0	1	0	1
1	0	1	0	0	0	0	1	0
2	1	1	1	0	1	1	1	1
3	0	0	0	1	0	0	1	0
4	1	1	1	0	0	1	1	1
...	...	...	...	...	...	...	...	...
1995	0	0	1	0	0	0	0	1
1996	1	1	1	0	0	0	0	0
1997	1	1	0	1	0	0	1	0
1998	1	1	1	0	1	1	0	1
1999	1	1	0	1	0	0	0	0

2000 rows × 12 columns

Does `Smoking` causes `Car Accident`?

We will try to answer the question as to whether there is a causal effect between `Smoking` and `Car Accident`. We can start by fitting a logistic regressor:

$$\log \frac{P(Car\,Accident|Smoking)}{P(\neg Car\,Accident|Smoking)} = \sigma(\beta_0 + \beta_1 Smoking)$$

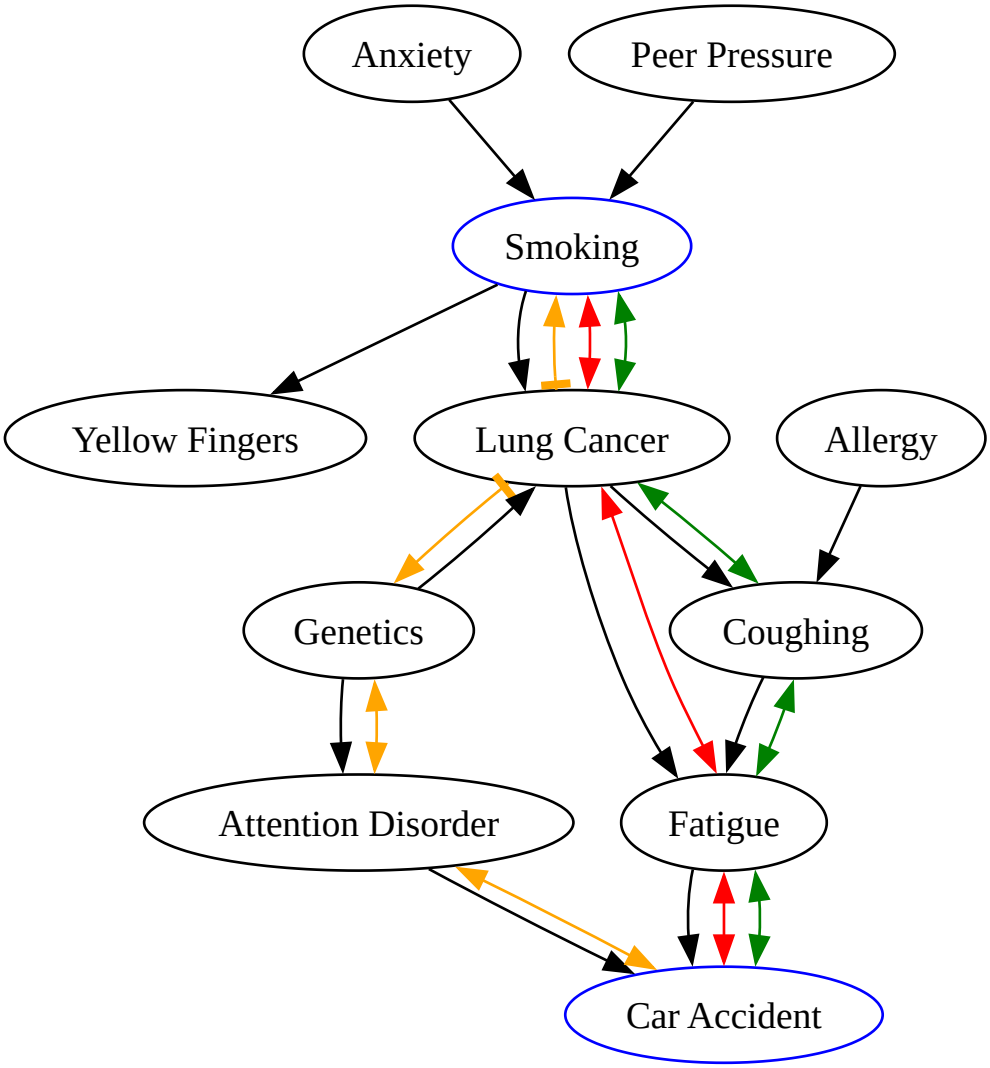
We need to fit a logistic regressor because the dependent variable is binary (all variables are binary). This is the result:

```
Optimization terminated successfully.
         Current function value: 0.588796
         Iterations 5
```

	coef	std err	z	P>|z|	[0.025	0.975]
Intercept	0.7668	0.097	7.939	0.000	0.578	0.956
Smoking	0.2596	0.113	2.299	0.022	0.038	0.481

From the observation above, it seems that `Smoking` is positively associated with `Car Accident`. Since $e^{0.2596} \approx 1.3$, we can say that smoking is associated by an increase of $+30$ in the odds of having a car accident. But, can we say that this is a causal effect? I.e., can we say that **smoking causes an increment in the odd of car accident**?

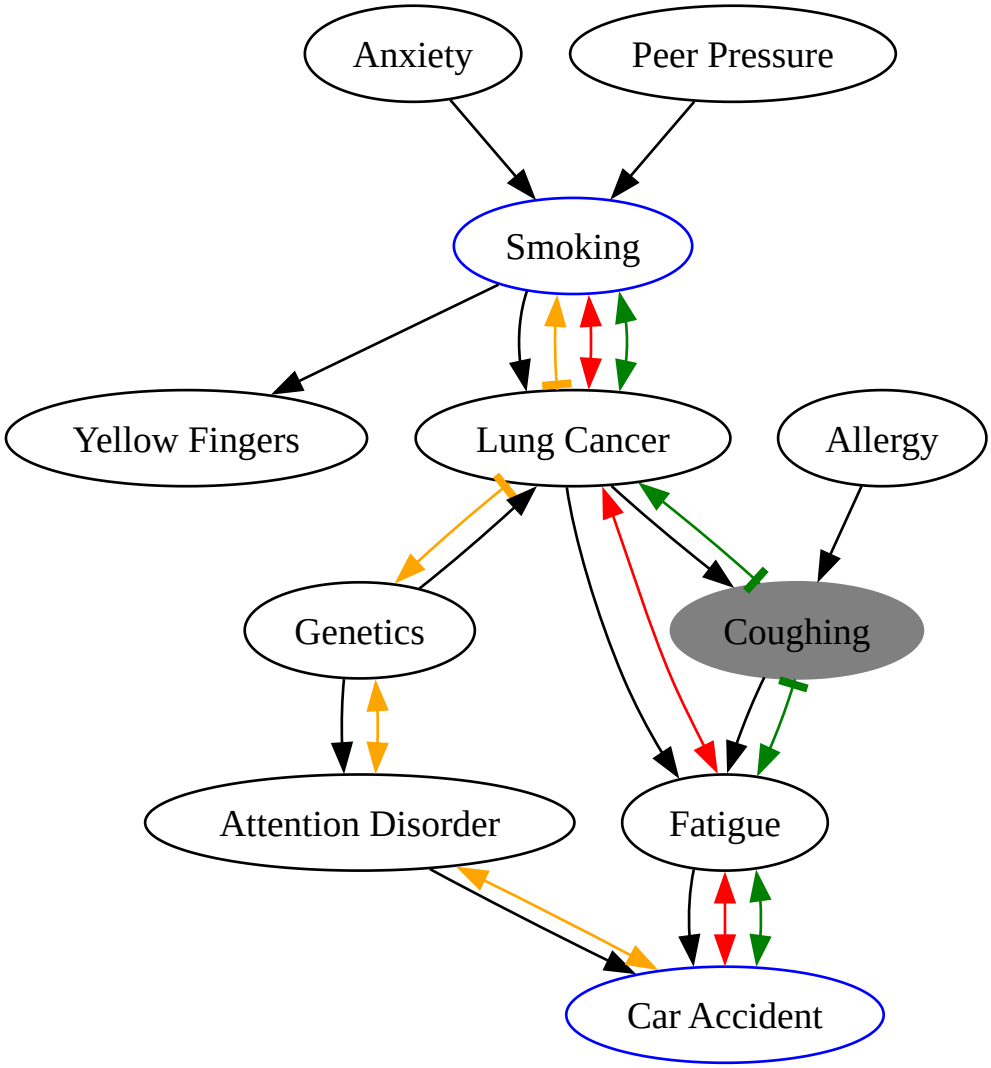
We know that there might be some bias. To check for that, let us consider the causal graph again, and let us highlight all dependence paths from the `Smoking` node to the `Car Accident` node. These represent all the different ways in which the two variables can be correlated.



We have three possible paths, highlighted in yellow, red, and green. Arrows indicate whether dependency flows or not. We can see that:

- The yellow path is blocked because there is a collider (`Lung Cancer`) that has not been conditioned on. The collider prevents `Smoking` and `Genetics` from being associated. This blocked path is ok for us. We are trying to see if smoking causes car accident, and, since, `Lung Cancer` does not cause `Genetics`, that path would not be a viable one to check for a causal effect;
- The red and green paths are not blocked. This is not ideal, because now there are two ways `Smoking` could influence `Car Accident`, i.e., directly through `Fatigue` or via `Coughing`. This may lead to a combined effect which is stronger than it is supposed to be.

If we want to estimate a causal effect, we need to block one of the two paths. The most sensible thing to do is to **condition on** `Coughing`, so that now smoking can affect Car Accident only through Lung Cancer and then Fatigue. This is illustrated as follows:



To condition on `Coughing`, we can fit the following logistic regressor:

$$\log \frac{P(Car\,Accident|Smoking, Coughing)}{P(\neg Car\,Accident|Smoking, Coughing)} = \sigma(\beta_0 + \beta_1 Smoking + \beta_2 Coughing)$$

We obtain the following coefficients:

```
Optimization terminated successfully.
         Current function value: 0.569895
         Iterations 5
```

	coef	std err	z	P>|z|	[0.025	0.975]
Intercept	0.3411	0.108	3.153	0.002	0.129	0.553
Smoking	-0.0130	0.120	-0.108	0.914	-0.248	0.222
Coughing	0.9529	0.109	8.717	0.000	0.739	1.167

We now get a different picture: the p-value of `Smoking` is very large. This means that, once we removed the influence of `Coughing`, the effect of `Smoking` on `Car Accident` is negligible (its

coefficient is not significantly different from zero). This suggests that there is no significant causal effect between the two variables.

Exercises

1. **Draw the graph.** A study finds that students who use the library more get better marks. Draw a causal graph including at least two plausible confounders. Which arrow is the causal effect of interest, and which paths are backdoor paths?
2. **d-separation by hand.** In the graph $A \to C \leftarrow B$, $C \to D$: are A and B independent? Are they independent given C? Given D? Justify each answer with the rules for chains, forks and colliders.
3. **Adjust, and over-adjust.** On the LUCAS data, estimate the effect of smoking on lung cancer (a) unadjusted, (b) adjusting for the confounders you identify from the graph,
   c. adjusting additionally for a *descendant* of the outcome. Explain why (c) is worse than (a).
4. **Selection bias in the wild.** Among hospital patients, smoking appears protective against a certain disease. Draw the graph that produces this and name the variable being conditioned on.

References

[1] Most of the content of this lecture is based on part of the awesome lecture series "Causal Inference for The Brave and True" by Matheus Facure Alves.

[2] Judea Pearl and Dana Mackenzie. "The Book of Why: The New Science of Cause and Effect". Penguin.

© 2026 Antonino Furnari · Department of Mathematics and Computer Science, University of Catania Built with Quarto

Credits and data sources